LEARN AI CODING

程序员 AI Coding 进阶实战

真实代码库里的 AI 编程工作流



好好用 AI　著

中文电子版 · 第 1 版
2026 年 10 月

LEARN AI CODING

程序员 AI Coding 进阶实战

真实代码库里的 AI 编程工作流

好好用 AI　著

第 1 版 · 2026-10-24

版本与许可

程序员 AI Coding 进阶实战
真实代码库里的 AI 编程工作流

作者：好好用 AI
公众号：好好用 AI
GitHub：DavidDingXu

电子版版本：1.0.0
修订日期：2026-10-24

原创正文与图示采用 CC BY-NC-SA 4.0，要求署名、非商业使用、相同方式共享。代码采用 PolyForm Noncommercial 1.0.0。第三方材料遵循各自许可。完整范围见仓库许可说明。

在线阅读：https://daviddingxu.github.io/learn-ai-coding/
配套仓库：https://github.com/DavidDingXu/learn-ai-coding

本版配套源码：476e41f7d56f。书中源码链接指向该提交。工具与框架可能变化，复现时请结合对应版本说明。

本书为作者自行编排的电子版。字体使用 Noto CJK，依 SIL Open Font License 1.1 提供；图示、截图与示例的来源和适用范围随正文保留。

前言

AI 可以很快生成代码，开发者却仍要回答几个老问题：需求有没有被理解，改动有没有越界，测试有没有覆盖真正的风险，其他人能不能审查这次交付。

本书围绕这些问题组织内容。先理解 Agent 的工作方式和工具边界，再进入代码库、上下文、规格、实现与验证，最后把可复用的方法留下来。工具名帮助定位具体例子，工程方法才是贯穿各章的主线。

配套项目包含一个个人效率助手和一组工作流工具。它们用于展示方法怎样落实到文件、代码与测试中。读者可以先完整阅读，再选择熟悉的需求动手；不必为了理解书中的每个判断都先配置一套工具环境。

如何阅读本书

本书适合有编程基础、已经尝试使用 AI 编程的开发者。正文会使用 Git、测试、接口和需求规格等概念；遇到不熟悉的术语，可以先查看书末索引，再回到对应章节。

前两篇帮助建立工作模型和工具工作台。接着依次阅读上下文工程、规格驱动开发、实现与验证，能够形成一条完整工作流。Skill、MCP 和多 Agent 放在这条流程之后，便于判断什么时候需要扩展能力。

本书保留课程原编号：导读、序章 00、正文 01—39 和结语。文中的示例代码、命令、提示文本与输出分别保留原貌；示例输出只说明对应案例，不能代替自己环境中的运行结果。

按主题阅读时，可从篇目扉页进入，也可以使用章内目录和书末术语索引。配套项目用于进一步验证，阅读本身无需先安装 Codex、Claude Code、OpenSpec 或其他工具。工具版本发生变化时，应以项目说明和对应官方文档核对入口。

PDF 使用 A4 版式，较密的宽图独立横排，适合固定版式阅读与打印，EPUB 可随设备调整字号。两种版本均包含完整正文和配图，不依赖联网加载图片；外部文档、源码与问题反馈需要联网访问。

目录

前言 4
如何阅读本书 5

第1篇 导读与学习地图 8
导读 程序员 AI Coding 进阶实战 9
序章 程序员如何系统学习 AI Coding 14

第2篇 理解 AI 编程的工作方式 25
第1章 程序员用 AI Coding，真正的门槛不是 Prompt 26
第2章 从聊天式开发到工程式开发 40
第3章 AI Coding 的底层工作模型 55
第4章 AI 最擅长和最不擅长的开发任务 71

第3篇 组织自己的工具工作台 88
第5章 Codex、Claude Code、Cursor、Copilot 怎么分工 89
第6章 一个程序员的 AI Coding 工作台怎么搭 104
第7章 App、CLI、IDE 怎么切换完成一次真实改动 122

第4篇 上下文工程与代码库理解 141
第8章 上下文工程：为什么不是塞越多代码越好 142
第9章 代码地图不是手工文档：让 Agent 生成可检查的仓库导航 162
第10章 让 Agent 做代码搜索：怎样约束搜索目标、证据和停止条件 181
第11章 AGENTS.md / CLAUDE.md / Copilot Instructions 202
第12章 任务上下文包：一次需求到底该给 AI 什么 220
第13章 上下文污染与上下文清理 244
第14章 模块实战：让 AI 接手一个陌生代码模块 264

第5篇 用规格约束需求变更 274
第15章 SDD 是什么：先写规格，再写代码 275
第16章 从一句需求到 requirements.md 294
第17章 design.md 怎么写，AI 才不会补丁式开发 315
第18章 tasks.md 怎么拆：把复杂需求变成小 diff 337
第19章 OpenSpec 实战：把需求变更放进 repo 360
第20章 OpenSpec、Spec Kit 与轻量 SDD 怎么选 376
第21章 模块实战：用 OpenSpec CLI 推进一次真实产品需求 393

第6篇 小步实现、测试与调试 406
第22章 计划先行：让 AI 先解释它准备怎么改 407
第23章 小步开发：为什么要限制 AI 一次改动范围 426
第24章 TDD + AI Coding：先让失败测试约束模型 445
第25章 系统化调试：让 AI 基于证据排障 465
第26章 前后端验证闭环：页面、接口、数据库和副作用怎么一起验 490
第27章 AI 说“测试失败无关”时，程序员该怎么判断 512
第28章 模块实战：用 AI 完成一次 CLI 交互改造 534

第7篇 把方法变成可复用能力 544
第29章 Skill 是什么：从提示词到可复用工作流 545
第30章 Skill 原理解析：它如何约束 Agent 行为 563
第31章 Superpowers 工作流拆解：把工程纪律变成 Skill 链 582
第32章 自定义 Skill：把你的经验固化成个人能力 599
第33章 外部工具接入：MCP 在 AI Coding 里到底有什么用 621
第34章 多 Agent 与自动化：什么时候拆，什么时候不要拆 644
第35章 模块实战：用 Superpowers 固化一次业务需求的调试和评审 667
第36章 模块实战：给 AI 接一个只读项目工具 676

第8篇 串起一次完整交付 685
第37章 综合实战：一个真实需求的 AI Coding 最佳实践 686
第38章 综合实战：从一次交付沉淀个人规则库 697

第9篇 建立个人工程系统 705
第39章 从个人提效到个人工程系统 706
结语 把 AI Coding 留在日常工程里 727

术语索引 740
章节与配套资源 741

PART 01

第1篇　导读与学习地图

先明确学习目标：把 AI 编程放进真实开发过程，理解每个阶段需要什么输入、产物和证据。

阅读重点

能根据当前经验选择阅读顺序。

导读　程序员 AI Coding 进阶实战

副标题：真实代码库里的 AI Coding 工作流

很多程序员已经开始用 AI 写代码，但真实项目很快会把问题暴露出来。

AI 能补一个函数，却不一定理解业务边界；能生成一组测试，却不一定测到关键路径；能解释一段报错，却可能忽略改前基线；能一次改很多文件，但谁来证明这些改动可审查、可回滚、可上线?

这里不讲 Prompt 技巧，也不做工具测评。问题收在一个地方：

> 如何把 AI 放进真实开发工作流，而不是停留在聊天窗口里。

导.1　为什么写这个专栏

AI Coding 的门槛看起来降得很低。装一个工具，打开仓库，描述需求，几分钟就能看到代码变化。

真正麻烦的是代码出来以后。

AI 读了哪些文件？有没有漏掉测试？它改的那一段是不是绕过了原来的业务规则？失败测试到底是历史问题，还是这次改动引入的回归？一个看起来“已经完成”的 diff，能不能交给同事 review，能不能回滚，能不能进 CI，能不能解释给未来的自己?

这些问题不是多背几个 Prompt 就能解决的。

程序员缺的不是更多工具名词，而是一套能放进日常开发的工作方法。它要能回答很细的问题：什么时候可以直接让 AI 改，什么时候必须先写规格；上下文给到什么程度刚好；怎么让一次改动小到可审查；测试失败时怎么分类；什么时候需要 Skill、OpenSpec、Superpowers、MCP 或多 Agent；哪些经验应该写回规则文件，哪些只该留在任务记录里。

这也是这个专栏要解决的问题：让 AI 参与真实开发时，所有关键判断都能落到需求、文件、diff、测试和验证证据上。

导.2　专栏讲什么

这是一套面向程序员的 AI Coding 进阶内容。

它不会从“什么是终端”“怎么安装 Node”开始讲。这里默认你已经会写代码，理解 Git、测试、接口、工程目录、命令行和代码审查。我们要处理的是更真实的问题：

- 一个陌生项目，AI 应该先读哪些文件?
- 一个复杂需求，能不能直接让 AI 开改?

- AI 生成了大 diff，怎么判断有没有越界？
- 测试失败时，怎么区分历史问题和本次回归？
- `AGENTS.md`、`CLAUDE.md`、Skill、OpenSpec、外部工具接入这些东西到底解决什么问题？
- 多 Agent、插件和自动化什么时候值得上，什么时候只会增加复杂度？

主线不会按工具清单展开，而是按一次可靠的 AI Coding 改动展开：

```
任务判断 -> 工具选择 -> 上下文侦察 -> 规格或计划 -> 小步实现 -> 测试和验证 -> diff 审查 -> 经验沉淀
```

围绕这条主线，专栏会讲工具工作台、上下文工程、SDD 规格驱动开发、OpenSpec、测试与调试、代码审查、Skill/插件机制、多 Agent、外部工具接入和个人工程资产沉淀。结构上不是先讲一堆理论、最后再集中实战，而是每个核心方法后面接一个模块实战，最后再用一个端到端需求串起来。

业务主线会用个人效率助手这类产品需求展开。配套工程放在 DavidDingXu/learn-ai-coding〔导-1〕，里面拆成业务 module 和工具 module；过程材料会保留任务卡、上下文侦察、规格、计划、实现范围、验证记录、审查风险和复盘结论。命令输出会作为验证证据出现，但读者看到的是一次需求从进入仓库到交付复盘的完整过程。

整体路线如下。

表 导-1 专栏讲什么

学习阶段	阅读重点
导读与学习地图	能根据当前经验选择阅读顺序。
理解 AI 编程的工作方式	能判断任务是否适合交给 AI，以及需要保留哪些人工判断。
组织自己的工具工作台	能为一次真实改动选择合适的工具入口。
上下文工程与代码库理解	能让 Agent 有依据地理解代码，并控制上下文污染。
用规格约束需求变更	能把一句需求推进为可实现、可验证的变更。
小步实现、测试与调试	能根据证据判断一次改动是否完成，以及失败是否来自本次变更。
把方法变成可复用能力	能识别值得固化的流程，并给工具接入设置清晰边界。
串起一次完整交付	能复盘一条完整需求，并提取有用的工程资产。
建立个人工程系统	能区分临时记录与长期规则，并安排持续维护。

这份学习地图不是要求你把所有工具一次学完。它只是把 AI Coding 放回真实开发链路里：先判断任务风险，再决定上下文、规格、实现、验证和沉淀方式。每个模块都要落到一个真实动作，工具可以换，控制点不能丢。

导.3 适合谁看

适合已经有工程经验的程序员。

如果你经常维护老项目、改复杂需求、查线上问题、补测试、做代码审查，或者已经在用 AI 写代码但觉得结果不稳定，这类内容会更适合你。

更具体一点，适合这些情况：

- 你已经会写代码，但不满足于让 AI 只补全函数或生成样板代码。
- 你想把 AI 用到真实仓库里，而不是只做一次性 demo。
- 你关心 diff 是否可审查、测试是否可信、失败是否可复现、交付是否能解释。
- 你已经用过 Codex、Claude Code、Cursor、GitHub Copilot 等工具，但还没有形成稳定工作流。
- 你想理解 OpenSpec、Skill、MCP、多 Agent 这些东西什么时候有用，什么时候只是增加复杂度。

如果你刚开始学开发，也可以跟着看，但需要愿意一边补基础一边动手：至少准备一个真实仓库，熟悉 Git 的基本操作，能跑测试或本地服务，能看懂接口、数据库、工程目录和命令行输出。专栏不会从这些基础概念讲起，但会告诉你它们在 AI Coding 流程里为什么重要。

导.4 如何学习这个专栏

不要从工具名词开始学。先从任务判断开始。

第一步，先理解 AI Coding 的边界。什么任务可以让 AI 直接做，什么任务只能让它辅助分析，什么任务必须先写清规格和验证方式。这个判断不清楚，工具越多越容易乱。

第二步，搭好自己的工作面。CLI、IDE、App、浏览器、Git 都能参与 AI Coding，但它们不是按“谁更高级”来分工，而是按权限、反馈和证据来分工。能访问仓库、改文件、跑命令并留下证据的工作面可以承接主流程；IDE、浏览器、终端和 App 按审查、debug、用户路径和材料整理的缺口补位。

第三步，重点练上下文工程。真实项目里，AI 出错常常不是因为不会写代码，而是因为读错文件、漏掉规则、带着旧假设，或者把无关上下文混进来。代码地图、项目规则、任务上下文包和上下文清理，都是为了让 Agent 的判断可检查。

第四步，再进入规格驱动和复杂需求拆解。简单文案和小 bug 不需要上重流程；但跨模块、跨接口、有兼容规则的需求，需要把“要做什么、为什么这样做、分几步做、怎么验证”写成更硬的合同。

第五步，用测试、调试、审查和复盘收口。不要接受“已经修复”这种结论，要看 baseline、命令、输出、截图、接口响应、diff 和剩余风险。一次任务结束后，再决定哪些经验写进规则文件、Skill、脚本或检查清单。

如果你已经在真实项目里用 AI，可以边看边改自己的工作流。先从三个动作开始：让 AI 列出它实际读过的文件；让每次改动都有明确验证命令；让完成报告包含改了什么、怎么验证、还有什么风险。

导.5　常见问题

这里只讲 Codex 吗?

不是。Codex 适合说明贴近仓库、命令、diff 的 Agent 工作流，但它不是唯一答案。Claude Code、Cursor、GitHub Copilot、OpenSpec、Spec Kit、Superpowers、Skill、MCP 都会回到同一个判断：它解决什么工程问题，权限边界在哪里，验证证据怎么留下来。

个人程序员需要学 OpenSpec、Superpowers、MCP 吗?

不需要一上来都学。先掌握任务判断、上下文、计划、小 diff、测试和验证。OpenSpec 适合复杂需求，Superpowers 适合把流程固化成 Skill，MCP 适合受控接入外部资源。它们是工具箱，不是入门门槛。

和团队 AI Coding 工程化有什么区别?

重点不同。团队工程化会关注平台、权限、成本、指标和组织协同；这里关注个人开发者如何在自己的真实仓库里稳定交付。个人工作流会借鉴团队工程里的控制点，但不会把组织流程当成主线。

如果项目没有测试，还能用这套方法吗?

能，但不能假装已经验证。没有自动测试时，要写清手动验证路径、接口调用、日志检查或浏览器截图。后续再逐步把高风险路径补成自动化测试。

这个专栏是否付费?

免费。专栏正文、配套代码和可公开的过程材料都会开放出来，你可以直接按文章和仓库里的 module 跟做。

会有配套工程和代码吗?

会有，但代码不是为了凑 demo。配套代码仓库是 DavidDingXu/learn-ai-coding〔导-1〕。业务 module 会围绕个人效率助手里的今日计划、逾期顺延、容量限制这类产品需求展开；工具 module 会服务代码地图、上下文审计、任务上下文包、验证记录、只读工具边界、CLI 合同改造等能力。配套资料里会保留需求过程记录，方便你看到一个需求从进入仓库到验证、审查、沉淀规则的完整路径。你不一定要照搬这些 module，但应该能把背后的控制点迁移到自己的仓库里。

为什么一直强调证据，会不会太重?

证据不是长文档。一次小改动的证据可以只是 `git diff`、一个 focused test、一个接口响应或一张截图。真正重的是 AI 误改后清理大 diff、解释线上问题、回滚混乱改动。

AI Coding 的价值，不是让程序员少看代码，而是让程序员把更多时间放在判断、边界、验证和交付质量上。

如果 AI 参与的每一次改动都能回到需求、上下文、diff 和验证证据上，它才真正进入了你的工程工作流。

本章资源

〔导-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

序章　程序员如何系统学习 AI Coding

如果你已经会写代码，第一次认真把 AI 放进真实项目里，大概率会遇到一个反差。

它确实很快。

补一个函数、解释一段报错、生成一组测试、照着已有代码仿一个实现，很多时候几分钟就能看到结果。可一旦任务稍微真实一点，问题也会马上出现：它读了哪些文件？有没有漏掉关键入口？为什么改到不相关模块？测试失败到底是旧问题还是新回归？一段看起来能跑的代码，能不能进 review、进 CI、进生产？

这些问题不是多背几个 Prompt 就能解决的。

程序员学习 AI Coding，真正要学的不是“怎么把一句话说得更像魔法咒语”，而是怎么把 AI 放进自己的工程流程里。让它能帮你读代码、拆需求、写实现、跑验证、整理证据，同时又不能越过你设定的边界。

先从一个普通程序员每天都会遇到的问题开始：

```
我手上有一个真实代码库，有一个真实需求。
我想让 AI 帮忙，但不能让它乱改。
我该怎么开始？
```

序.1　先把目标说清楚

系统学习 AI Coding，不是把所有工具都学一遍。

Codex、Claude Code、Cursor、GitHub Copilot、OpenSpec、Spec Kit、Superpowers、MCP、Skill、Plugin，这些名字会不断变化。工具值得学，但不能按工具名组织自己的能力。否则每出一个新入口，你都要重新焦虑一次。

更稳定的学习目标应该是这几个：

- 能判断一个任务适不适合交给 AI。
- 能让 AI 读对上下文，而不是凭感觉猜。
- 能把复杂需求拆成可审查的小 diff。
- 能用测试、页面、接口、日志和 diff 证明结果是真的。
- 能把反复有效的流程沉淀成规则、脚本或 Skill。

AI Coding 的核心不是“AI 会不会写代码”，而是“你能不能控制一次 AI 参与的工程改动”。

这套能力一旦建立起来，工具可以换，流程不会塌。

序.2　为什么不能只靠 Prompt

很多失败看起来像 Prompt 写得不好，实际不是。

比如你让 AI 改一个“今日计划自动纳入逾期任务”的需求。它可能很快写出函数，也可能顺手加页面、接数据库、做提醒、改排序规则。代码都像那么回事，但需求边界已经被它扩大了。

再比如你让 AI 修一个 CLI 交互问题。它修了输出文案，却没有注意 stdout、stderr、exit code 和 README 示例命令也是用户可见合同。测试可能绿了，真实用户路径仍然坏着。

还有更常见的：你让 AI 接手一个陌生模块，它说“我理解了项目结构”，但没有列出自己实际读过哪些文件；它总结得很顺，可入口、测试、生成目录、历史兼容逻辑都没核对。这个时候继续让它实现，风险已经埋下了。

所以 Prompt 只是入口。

真实项目里更重要的是这些东西：

示例 序-1　为什么不能只靠 Prompt

```
任务边界
上下文证据
实现计划
小步 diff
测试基线
验证结果
代码审查
规则沉淀
```

你可以把它们理解成程序员给 AI 装上的护栏。护栏越清楚，AI 越像一个可用的工程助手；护栏越模糊，AI 越容易变成一个很快的随机改代码机器。

序.3　这和工具教程有什么不同

很多 AI Coding 内容会从工具入口开始讲：

```
这个按钮怎么用。
这个命令怎么跑。
这个模型怎么选。
这个 Prompt 怎么写。
```

这些东西当然有用，但它们不是程序员最容易卡住的地方。

真正卡住你的，往往是工具已经会用了，代码也生成了，但你不知道该不该接受它。

比如 Agent 给你改了 6 个文件。它说“我已经完成并通过测试”。你打开 diff，发现里面有一段你没要求的重构。它解释说这是为了提高可读性。这个时候你要判断的不是 Prompt 够不够好，而是：

```
这段重构是否属于当前需求?
有没有改变外部行为?
测试能不能覆盖这个变化?
如果线上出问题，能不能单独回滚?
reviewer 能不能在一次审查里看懂?
```

这些判断靠工具教程解决不了。

再比如 AI 搜索项目后告诉你：“核心逻辑在 service 层，测试覆盖比较完整。”这句话听起来很专业，但如果它没有列出实际读取的路径，没有说明哪些测试覆盖哪些行为，没有说它排除了哪些 legacy、demo、generated 目录，你仍然只能相信它。

当工具已经能生成结果以后，真正要补的是判断结果是否可靠。很多关键动作看起来并不炫：

示例 序-2　这和工具教程有什么不同

```
先看 git status。
先让 AI 列出读过的文件。
先跑改前基线。
先限制 diff 范围。
先写清不做什么。
完成前重新跑验证。
review 先看行为风险。
```

这些动作不新，但它们决定 AI Coding 能不能进入真实项目。

序.4　可靠改动要经过哪些控制点

先把一次可靠改动拆开看。



图 序-1　程序员 AI Coding 学习路线

这几个控制点不要求一次学完，但每次真实改动都绕不开。

第一步是任务判断。

不是所有需求都应该直接交给 AI 写代码。改文案、补测试、生成脚本、解释报错，和跨模块业务需求、数据修复、线上故障，风险完全不同。任务类型不同，授权边界就不同。

第二步是选择工作面。

IDE 补全、CLI Agent、App、浏览器、终端、Git 都能参与 AI Coding，但它们不是按“哪个更高级”分工。能读仓库、改文件、跑命令、留下验证证据的工作面，更适合承接主流程；浏览器适合验页面，Git 适合审 diff，App 更适合预览、长任务管理、异步审查和材料整理。

第三步是上下文工程。

AI 会读文件，但你不能只听它说“我看过了”。你要让它列出实际读取过的路径、判断哪些入口相关、哪些目录不能碰、哪些测试要跑、哪些结论还没有证据。

第四步是规格和计划。

简单任务可以轻量计划，复杂需求要写清需求、设计、任务和验证方式。重点不是文件名，而是把“意图”从聊天窗口搬进仓库，让人和 AI 都能复查。

第五步是小步实现。

AI 很容易一次改太多。真实项目里，更好的做法是让它按小任务推进，每一步都能解释改了哪些文件、为什么改、怎么验证。

第六步是测试和调试。

不要接受“已经修复”这种口头结论。要看改前能不能复现，改后跑了什么命令，失败怎么分类，用户可见行为有没有验证。

第七步是工作流固化。

如果你反复提醒 AI 同一件事，就应该把它写进 `AGENTS.md`、`CLAUDE.md`、Skill、脚本或检查清单。规则不是为了显得专业，而是为了少犯重复错误。

最后是综合实战。

所有方法都要回到真实需求里。否则学完上下文、spec、TDD、review、Skill，还是不知道一个需求从进入仓库到交付复盘应该怎么串起来。

序.5　先从一个真实仓库开始

如果你想跟着学，不需要先准备一个大项目。

一个你熟悉的小仓库就够了。它最好满足几个条件：

```
能运行。
有基本测试或手动验证路径。
目录结构不是完全空白。
最近确实有一个小需求或小 bug。
你能判断 AI 的结果对不对。
```

不要一开始就拿线上高风险任务练。也不要只拿一次性 demo 练。demo 太顺，学不到边界；线上任务太重，容错空间太小。

比较合适的第一批任务是：

- 给 CLI 补一个用户可见错误提示。
- 给一个函数补边界测试。
- 让 AI 接手一个陌生模块但先不改代码。
- 修一个能稳定复现的小 bug。
- 给一个页面改轻量交互，然后用浏览器验证。

这些任务不大，但足够暴露真实问题：AI 是否读对文件，是否改到无关位置，是否知道怎么跑测试，是否会把完成说成一句空话。

序.6　第一次动手不要急着改代码

刚开始练 AI Coding，先做一个动作：让 AI 只读，不改。

给任务时直接这样写：

示例 序-3　第一次动手不要急着改代码

```
先不要改代码。

请围绕这个需求做一次仓库侦察：
1．列出你实际读取过的文件路径。
2．判断这个需求可能影响哪些模块。
3．标出哪些目录或文件不应该修改。
4．找到可以验证结果的测试、命令或手动路径。
5．给出一个小步实现计划。

如果证据不足，先列出缺口，不要猜。
```

这个动作看起来慢，但它会立刻暴露一个 Agent 是否可靠。

可靠的输出会具体到文件、命令和风险。不可靠的输出通常很顺滑：项目结构总结得很好，结论也像那么回事，但没有路径、没有测试、没有排除项，也没有停止条件。

第一次练习时，你只需要检查四件事：

```
它读的文件是不是真相关?
它有没有漏掉明显入口?
它给的验证命令能不能跑?
它有没有把不该做的事列出来?
```

如果这四件事都说不清，先不要让它实现。

这一小步不会制造复杂 diff，却能训练一个关键习惯：AI 的结论必须能回到证据。

第一次练习的结果可以写得很短：

示例 序-4　第一次动手不要急着改代码

```
任务：修复 CLI 未知命令时的错误提示。
AI 读取过：src/cli.js、test/cli.test.js、README.md、package.json。
相关行为：stdout、stderr、exit code、README 示例命令。
不改范围：不改命令名称，不改配置格式，不引入依赖。
验证方式：npm test；手动运行 node src/cli.js unknown。
下一步：先补失败测试，再改 CLI 分支。
```

这类记录不需要漂亮。它的价值是让你在改代码前就知道：当前任务的边界在哪里，验证在哪里，AI 有没有漏掉关键文件。

等这个动作熟了，再进入复杂需求就顺了。上下文工程不是抽象概念，它就是让这份记录更完整；规格驱动不是写文档，它就是把边界和验证写得更硬；测试和调试不是流程表演，它就是在结果出来后确认它没有骗你。

序.7　起手任务要小，但要有真实合同

第一次练习，不要选“帮我重构这个系统”。

选一个边界清楚、能验证、出错成本低的小任务。比如一个 CLI 工具现在遇到未知命令时只输出 `Unknown command`，用户不知道有哪些可用命令。你想把它改成：

示例 序-5　起手任务要小，但要有真实合同

```
unknown command: publish

available commands:
- codemap
- context-pack
- verify-ledger
```

这个需求看起来很小，但它已经有真实工程合同：

```
stdout 还是 stderr?
exit code 应该是 0 还是 1?
README 里的示例命令要不要同步?
现有测试有没有覆盖未知命令?
命令列表从哪里来，是否会和真实实现漂移?
```

直接让 AI “优化 CLI 提示”很容易跑偏。它可能顺手改命令注册方式，可能把错误输出写到 stdout，可能只改代码不改 README，也可能新增一个项目里没有用过的依赖。

更好的起手是这样：

```
先不要改代码。

请先读取 CLI 入口、现有 CLI 测试、README 命令示例和 package.json。
列出当前 unknown command 的行为、已有测试覆盖、用户可见合同和最小修改范围。
如果需要修改，请先给出一个不超过两个文件的小步计划，并说明验证命令。
```

这段话没有复杂技巧，但顺序是对的：先找入口，再确认合同，再限制范围，最后才允许它计划修改。这样你拿到的不是一段“看起来更友好”的代码，而是一条可以审查的交付路径。

这种小任务练熟后，再去做更复杂的需求，心态会完全不同。你不会急着问“这个 Prompt 怎么写得更强”，而会先问：

```
AI 需要读哪些文件?
哪些行为不能变?
改完怎么证明?
如果它要扩大范围，应该在哪里停下来?
```

这才是程序员学习 AI Coding 的起点。

序.8　看过程证据，不只看最终代码

很多人看 AI Coding 结果，只看最后生成了哪些代码。这样很容易错过真正有价值的部分。

更应该看这些材料：

示例 序-6　看过程证据，不只看最终代码

```
需求怎么被缩小。
哪些范围被拒绝。
AI 读了哪些上下文。
spec 或 plan 怎么写。
测试为什么这样设计。
最终跑了哪些验证。
review 看了哪些风险。
哪些经验写回了规则。
```

这些才是可迁移的部分。最终代码只是载体。

比如“今日计划”需求，表面上只是生成一个任务列表，真正要看的却是状态、日期、容量和排序怎么被限定住。

再比如 CLI 合同改造，表面上只是改错误提示，真正要看的却是 stdout、stderr、exit code 和 README 示例命令为什么都算用户可见行为。

只读项目工具也是一样。先证明权限边界，再考虑要不要包装成 MCP；先证明输出可复查，再考虑让 Agent 长期调用。

这些判断方式可以迁移到很多项目。

订单系统里，它可能变成“超时订单怎么关闭，但不能误关已支付订单”。工单系统里，它可能变成“SLA 快超时怎么升级，但不能把所有低优工单都推给同一个人”。内容系统里，它可能变成“审核任务怎么排优先级，但不能跳过人工复核”。

业务不同，控制点相似。

序.9　先改三个习惯

如果你已经在工作中用 AI 写代码，不需要先等工具体系全部学完。

第一，改起手式。

从现在开始，少用“帮我实现一下”。换成更可控的起手式：

```
先不要改代码。
请先读相关文件，列出证据、影响面、不可修改范围和验证方式。
```

这一句话会立刻改变 Agent 的工作顺序。

第二，给自己的项目补最小规则。

不用一开始写很完整。先写 5 件事：

```
项目怎么安装。
测试怎么跑。
哪些目录不能改。
什么算完成。
如果检查没跑，要怎么说明原因。
```

这些内容放进 `AGENTS.md`、`CLAUDE.md` 或项目已有的开发说明里。重点不是文件名，而是让 Agent 每次进项目时能读到稳定规则。

第三，选一个低风险需求练完整闭环。

完整闭环不是大项目。它只需要包含：

```
改前边界。
小步实现。
验证命令。
diff 审查。
剩余风险。
```

第一次跑通这个闭环，比追十个新工具更有价值。

每次学一个新工具或新流程，都先问一句：它能不能改进下一次真实任务？如果不能，就先放一放。AI Coding 的学习顺序不应该由工具热度决定，而应该由你手上的任务风险决定。

序.10　常见问题

已经会写代码，还需要系统学吗？

需要学的不是编程基础，而是协作方式。你已经知道什么是好代码、好测试、好设计，现在要把这些判断转成 AI 能执行、你能检查的上下文、规则和验证动作。

要不要一开始就学 OpenSpec、Superpowers、MCP？

不用。先把任务判断、上下文侦察、小步实现、测试验证跑通。等你遇到复杂需求、反复调试、流程复用或外部资源接入，再引入这些工具。工具应该解决痛点，不应该制造负担。

是不是每个任务都要走完整流程？

不是。改一个文案、补一个简单测试，不需要上完整规格流程。真正要学的是判断：什么时候直接做，什么时候先计划，什么时候必须写 spec，什么时候只能让 AI 辅助分析。

没有自动化测试，还能用 AI Coding 吗？

可以，但不能假装已经验证。没有自动化测试时，要写清手动验证路径、页面截图、接口返回、日志检查或数据检查。后面再逐步把高风险路径补成自动化测试。

只用一个工具能不能开始？

可以。真正要掌握的是控制点：读上下文、定边界、拆小步、跑验证、看 diff。工具换了，控制点仍然成立。

怎么判断 AI 的结果能不能接受？

先看证据，不看口头总结。最少看四件事：diff 是否在计划范围内，相关测试或手动验证是否跑过，用户可见行为是否确认，剩余风险是否说清楚。

序.11　先做一个只读任务

系统学习 AI Coding 的第一步，不是换工具，也不是背模板。

先拿一个真实小任务，要求 AI 只读仓库、列证据、给计划，不要改代码。你会马上看到自己的项目缺什么：规则文件、测试命令、上下文边界、验证路径，还是任务本身没有说清楚。

这些缺口，就是后面要逐个补上的能力。

当 AI 参与的每一次改动都能回到需求、上下文、diff 和验证证据上，它才真正进入了你的工程工作流。

PART 02

第2篇　理解 AI 编程的工作方式

从聊天式开发的局限出发，理解 Agent 如何读取上下文、调用工具和修改代码，以及不同任务的适用边界。

阅读重点

能判断任务是否适合交给 AI，以及需要保留哪些人工判断。

第1章　程序员用 AI Coding，真正的门槛不是 Prompt

很多程序员第一次认真用 AI 写代码，都会经历一个很短的兴奋期。

它能在几十秒里补完一个函数，能把一段报错解释得像模像样，能根据现有代码风格生成一组测试。你会觉得这东西确实能省时间。再过几天，另一个感受会冒出来：它也很容易把事情搞乱。

它会漏掉关键文件，会把测试失败归因成历史问题，会顺手重构无关代码，会把一个局部需求扩成一片大 diff。最麻烦的是，它经常讲得很有把握，让你以为它真的理解了项目。

所以程序员用 AI Coding，门槛不在于会不会写一句漂亮 prompt。

门槛在工程控制：任务能不能分级，上下文能不能给准，diff 能不能收住，验证证据能不能留下来。

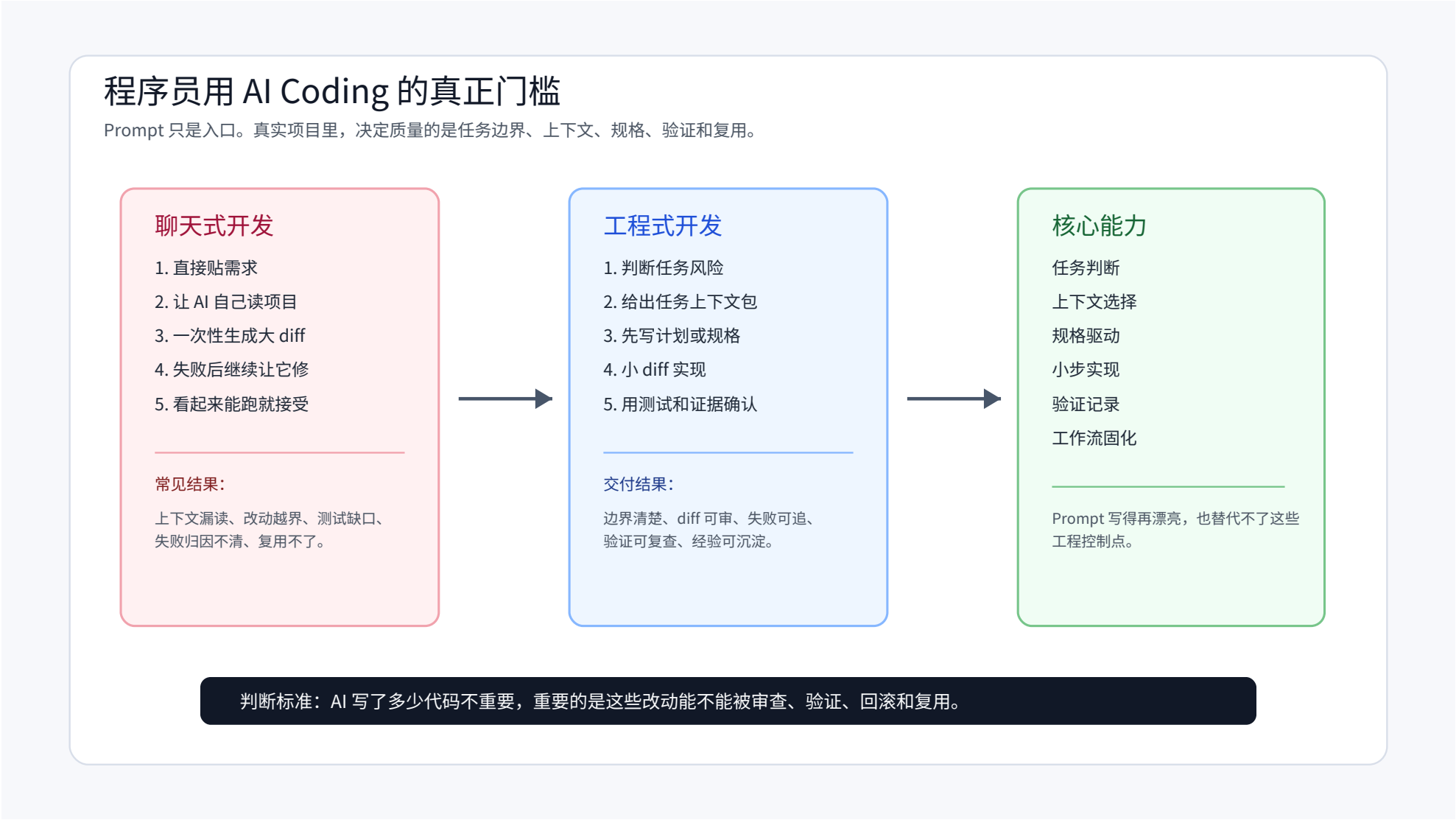


图 1-1　程序员用 AI Coding 的真正门槛

这张图的左边，是很多人默认走的聊天式开发；中间，是更适合真实项目的工程式开发；右边，是需要逐步补齐的核心能力。

1.1 Prompt 为什么不够

Prompt 有用，但它只是入口。

比如你可以这样写：

```
请你作为资深后端工程师，帮我修复订单状态回退的问题。要求代码简洁、考虑边界、补充测试。
```

这句话看起来很完整。它有角色，有任务，有质量要求。问题是，对真实代码库来说，它缺了最关键的东西：

- 订单状态有哪些历史兼容逻辑?
- 哪些状态流转不能改变?
- 哪些文件是入口，哪些文件只是同名但无关?
- 当前失败能不能稳定复现?
- 修完以后跑哪个测试能证明问题解决?
- 如果测试失败，怎么判断是不是本次改动导致?

这些信息不在 prompt 里，AI 就只能猜。

程序员会觉得“这些我心里有数”。但 AI 不知道。它只看到你给它的上下文、它自己读到的文件、工具返回的结果，以及对代码模式的推断。

先记住一个事实：**你脑子里的项目经验，如果没有变成上下文、规则、测试或验证记录，就不能稳定约束 AI。**

1.2 会写代码的人，常常会低估流程

不懂代码的人用 AI，经常会被卡在基础问题上：环境起不来、报错看不懂、依赖装不上。

程序员的问题不一样。你知道怎么修，也知道 AI 生成的大概方向对不对，于是很容易跳过工程流程，直接让它开改。

这反而危险。

因为你会在脑子里自动补全很多缺失信息：这个接口不能改、这个 helper 要复用、这个测试环境要先起 Redis、这个字段历史上有兼容逻辑。你知道这些，但 AI 不知道。你越熟悉项目，越容易以为它也“应该知道”。

真实项目里，AI 写错代码通常不是最严重的问题。最严重的是它在错误上下文里写出了看起来很合理的代码。

比如：

示例 1-1 会写代码的人，常常会低估流程

```
你让 AI 修复优惠券过期判断。
AI 找到 CouponService，补了一个日期判断。
测试也过了。

但真实入口在 CouponAvailableQueryService。
CouponService 只是老接口兼容层。
它改对了一个局部文件，却没有改到真实调用路径。
```

这类错误靠更长的 prompt 不一定能解决。它需要代码地图、调用链、测试入口和任务上下文包。

1.3 先区分两种 AI Coding

我建议先把 AI Coding 分成两类。

第一类是聊天式开发。

它的流程大概是：

```
描述需求
让 AI 实现
看结果
继续让 AI 修
```

这种方式适合低风险任务。比如写一个脚本、补一个小函数、解释一段报错、生成一份临时 SQL。它快，成本低，不需要很重的流程。

第二类是工程式开发。

它的流程是：

示例 1-2 先区分两种 AI Coding

```
判断任务风险
选择工具工作面
准备上下文
写计划或规格
小步实现
运行验证
审查 diff
沉淀规则
```

这种方式适合真实项目里的需求开发、bug 修复、前后端联调、老项目改造和代码审查。

两者不是谁替代谁。关键是别拿聊天式开发去处理工程式任务。

如果一个任务满足下面任意两条，就应该按工程式开发处理：

- 需要改多个文件。
- 涉及业务状态、权限、金额、数据一致性。
- 需要兼容旧行为。
- 需要补测试或改测试。
- 失败后定位成本高。
- 改错后回滚成本高。
- 需要别人审查。

1.4 门槛一：任务判断

让 AI 开始写代码之前，先问一个问题：

> 这个任务到底能交给 AI 到什么程度？

不是所有任务都能直接授权。

可以按下面这张表判断：

表 1-1 门槛一：任务判断

任务	AI 可以做	人必须控制
局部函数	生成实现、补边界测试	输入输出、异常语义
陌生模块理解	建代码地图、列调用链	判断它是否读到关键路径
Bug 排查	分析日志、列假设、写复现脚本	确认根因和修复策略
跨模块需求	写计划、拆任务、生成小 diff	架构边界和兼容策略
权限/金额/状态	辅助列风险和测试点	业务规则、上线决策
重构	拆小步、补回归测试	外部行为不变和回滚路径

这一步的价值很直接：它决定你应该给 AI 多少权限。

有些任务可以让 AI 直接改代码。有些任务只能让 AI 先读文件、列影响面。有些任务可以让 AI 写测试，但修复策略要人先定。有些线上风险任务，AI 只能做分析助手。

一个可靠的起手式是：

示例 1-3 门槛一：任务判断

```
先不要改代码。

请你先判断这个任务适不适合交给 AI 实现：
1．这个任务属于局部改动、跨模块需求、bug 排查、重构，还是高风险业务改动？
2．你需要先读取哪些文件？请列出路径。
3．哪些行为不能改变？
4．如果要实现，建议拆成几个小步骤？
5．每一步用什么命令或测试证明完成？
```

这段话不会让 AI 变聪明，但会让它先暴露理解过程。

1.5 把任务判断落成授权级别

任务分级如果只停留在脑子里，对 Agent 没有约束力。真正要写给 Agent 的，是授权级别。

可以把一次任务分成四档：

表 1-2 把任务判断落成授权级别

授权级别	适用场景	给 AI 的权限	人要检查什么
只读分析	线上事故、需求不清、业务规则未确认	读文件、读日志、列事实和假设	是否把猜测写成事实
计划优先	跨模块需求、状态机、权限、金额	读仓库、列文件、写计划，不改代码	文件是否读对，计划是否越界
小步实现	局部 bug、已有测试、影响范围可控	按一个 task 修改，完成后停止	diff 是否只服务这个 task
自动执行	文档、样例、局部机械同步	直接改并验证	命令输出和残余风险

这个表的作用，是把“AI 能不能做”改成“AI 现在能做到哪一步”。

例如订单状态回退问题，第一轮通常不该让 AI 直接实现。更合适的授权是：

示例 1-4 把任务判断落成授权级别

```
你现在只做计划优先模式。

允许：
- 读取订单状态流转相关文件。
- 列出实际读取过的路径。
- 整理现有状态迁移和测试覆盖。
- 给出最小实现计划。

不允许：
- 修改代码。
```

```
- 修改测试。
- 推断业务规则。
- 新增依赖或重构状态机。

停止条件：
- 发现状态规则需要业务确认。
- 发现真实入口不止一个。
- 发现现有测试和代码语义冲突。
```

这段约束看起来比一句 prompt 硬，但它符合真实开发。程序员不会把一个没确认业务规则的状态机任务直接交给新人改；AI 也一样。

如果你不写授权级别，Agent 往往会默认自己可以继续往下做。它读到文件以后会改代码，测试失败后会继续修，修不动时会扩大范围。很多大 diff 就是这样长出来的。

授权失败通常不是一句话就能看出来。它会表现成几种信号：Agent 没有读到真实入口却开始实现；计划里出现你没有要求的重构；测试失败后它直接修改断言；为了让命令通过新增依赖或调整配置。看到这些信号，不要继续追问“再修一下”，要把任务退回到只读分析或计划优先模式。

1.6 门槛二：上下文选择

AI Coding 的质量，很大程度取决于上下文。

上下文不是把仓库全塞进去，也不是只贴一个需求描述。上下文要刚好覆盖当前任务。

一个最小任务上下文包可以包括：

清单 1-5 门槛二：上下文选择

```
# Task Context

## Goal
- 要改变什么行为?
- 哪些行为不能改变?

## Files
- 入口:
- 核心实现:
- 测试:
- 配置:
- 历史兼容逻辑:

## Constraints
- 不新增依赖:
- 不改公开 API:
- 不改 generated 文件:
- 不做顺手重构:

## Done When
```

```
- 哪些测试通过：
- 哪个页面或接口验证：
- 哪个 bug 不再复现：
- diff 限制在哪些文件：
```

这个模板不复杂，但它能解决两个问题。

第一，它让 AI 知道边界。

第二，它逼你自己先想清楚任务。

如果你写不出 Done When，就说明你还没定义完成。如果你列不出核心文件，就说明现在让 AI 改代码还太早。

配套代码仓库 DavidDingXu/learn-ai-coding〔1-1〕 的工具 module 里有一个最小工具，可以直接生成任务上下文包：

清单 1-6 门槛二：上下文选择

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js task-context \
  --title "修复订单状态回退问题" \
  --files "src/order.ts,test/order.test.ts" \
  --constraints "不改公开 API,不新增依赖" \
  --done "npm test,人工验证支付回调"
```

它不会替你理解业务，但能把一次 AI Coding 任务最容易漏掉的信息固定下来。

1.7 门槛三：小步实现

AI 很容易一次改很多。

它会觉得顺手整理命名、抽函数、改测试结构、删掉“重复逻辑”都是好事。对它来说，这些都是合理的代码改进。对真实项目来说，这些可能都是风险。

所以要限制 diff。

可以这样要求：

示例 1-7 门槛三：小步实现

```
这次只完成第一个小步骤。

限制：
- 只改计划中列出的文件。
- 不做顺手重构。
- 不新增依赖。
- 不改公开 API。
```

```
- 如果必须扩大范围，先停下来说明原因。

完成后输出：
- 改了哪些文件。
- 每个文件为什么改。
- 跑了什么验证。
- 还有什么风险没覆盖。
```

小步不是慢。小步是为了让审查、测试和回滚变便宜。

如果 AI 一次生成 800 行代码，你可能要花半小时才能判断它有没有越界。如果它一次只改 60 行，并且每个文件都有理由，审查就容易很多。

程序员要控制的不是一次生成多少代码，而是每次改动是否能被检查。

1.8 门槛四：验证证据

AI 说“已经修复”没有意义。

你要看证据。

一次改动至少要留下这些信息：

清单 1-8 门槛四：验证证据

```
## Baseline
- 改前怎么复现问题?

## Change Summary
- 改了什么?

## Verification Commands
```bash
npm test
```

1.9 Evidence

- PASS 多少测试?
- 页面或接口怎么验证?

1.10 Residual Risk

- 哪些路径还没覆盖?
```

示例 1-9 Residual Risk

````
配套工程的工具 module 里也放了一个验证记录检查器：

```bash
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
````

它会检查验证记录里有没有关键段落、命令块和通过信号。

这不是为了形式化写文档，而是为了防止一个常见问题：AI 把“我认为修好了”包装成“已经完成”。

如果没有 baseline，就不知道问题原来是否存在。 如果没有命令，就不知道验证怎么做。 如果没有通过信号，就不知道结果是否真实。 如果没有残留风险，就不知道还有哪些路径没覆盖。

1.11 门槛五：复用经验

如果你第二次提醒 AI 同一个规则，就应该考虑把它写下来。

比如：

- 这个项目用 pnpm，不要用 npm。
- 前端改动必须打开页面验证。
- 不要编辑 generated 文件。
- 不要新增生产依赖，除非先确认。
- 测试失败不能直接归因历史问题，必须先跑改前基线。

这些规则可以写进 `AGENTS.md`、`CLAUDE.md`、Copilot instructions、Skill、脚本或 checklist。

规则文件解决的是项目上下文。 Skill 解决的是重复流程。 脚本解决的是确定性动作。 MCP 解决的是必要外部资源接入。 多 Agent 解决的是可并行、可隔离的任务拆分。

它们不是互相替代的概念，而是一条递进线。

你先让 Agent 会做事，再约束它做对事；再把有效流程固化下来；再决定是否给它接外部资源；最后才考虑多 Agent。

1.12 能力自测：你现在卡在哪一层

可以用这张表给自己打分。

表 1-3 能力自测：你现在卡在哪一层

能力	低阶表现	专业表现
任务判断	看到需求就让 AI 改	先判断风险和授权范围
````

能力	低阶表现	专业表现
上下文选择	贴需求或塞大量文件	给代码地图和任务上下文包
计划拆解	接受 AI 一次性实现	要求先计划，再小步改
代码审查	看结果能不能跑	看 diff 是否越界
测试验证	接受“已修复”	要求 baseline、命令、证据
调试	让 AI 连续猜	复现、假设、验证逐步推进
经验复用	每次重新解释	写进规则、Skill 或脚本

如果你只能优先补三项，我建议补：

- 任务判断。
- 上下文选择。
- 验证证据。

这三项决定下限。工具技巧、框架和多 Agent 都是后面的事。

1.13 这件事怎么落到日常开发

日常开发里，不需要每次都搞一套很重的流程。你可以先给自己定三个默认动作。

第一个默认动作：让 AI 先说明它读了什么。

很多 AI Coding 事故都和“以为它读了，其实没读”有关。让它输出实际读取过的文件路径，不是为了形式感，而是为了把理解范围暴露出来。它如果没读测试文件，就不要相信它对验证的判断；它如果没读调用入口，就不要相信它对影响面的判断。

第二个默认动作：让 AI 先给计划，不要直接改。

计划不是长篇方案。一个小需求可以只要 3 行计划：改哪个文件、为什么改、怎么验证。只要 AI 写不出这个计划，就说明它还没准备好进入实现。

第三个默认动作：把“完成”改成证据。

不要问“你修好了吗”。要问“你跑了什么命令，输出是什么，diff 是否只在计划范围内，还有哪些风险没覆盖”。这样问，AI 的回答才会从主观承诺变成可检查记录。

这三个动作不会让流程变复杂。它们只是把程序员本来就在脑子里做的判断，提前放到台面上。

如果你在团队里做 code review，也可以用同样的标准看 AI 生成的 PR：

- PR 描述里有没有任务边界?
- diff 是否出现无关重构?

- 测试是否覆盖真实风险?
- 失败路径有没有被解释清楚?
- 是否有“AI 觉得无关”但没有证据的测试失败?

一旦这样审查，你会发现 AI Coding 和普通开发并没有两套质量标准。AI 只是让代码生成速度变快了，但工程质量仍然要靠同一套东西：边界、测试、审查、验证和复盘。

个人使用 AI Coding 时，也可以按 PR 的标准要求自己。哪怕没有真正提交 PR，也要能回答：这次改动为什么必要，哪些文件被改，哪些验证跑过，哪些风险没有覆盖。只要这些问题答不上来，说明任务还没有进入可交付状态。

1.14 跟着做一次小练习

找一个你最近真的要改的小需求。不要让 AI 直接实现。

先让它做这件事：

示例 1-10 跟着做一次小练习

```
先不要改代码。

请你根据当前仓库，帮我完成一次实现前检查：
1．列出你需要读取的文件路径。
2．判断这个任务属于低风险、中风险还是高风险。
3．写一个最多 4 步的小步计划。
4．每一步说明如何验证。
5．如果上下文不足，只提问题，不要猜。
```

然后你自己检查它的输出：

- 它列的文件是否真的是关键路径?
- 它有没有漏测试?
- 它有没有把高风险任务说成低风险?
- 它的验证方式是不是可执行命令?
- 它有没有准备一次改太多?

这一步练熟以后，再让 AI 写代码。

这一轮做完再进入实现，AI 的输出审查成本会低很多。原因不在某句 prompt，而在于任务已经被放进了明确的工程流程。

1.15 常见问题

Prompt 还重要吗？

重要，但不要把它当全部。

Prompt 负责表达当前任务。项目规则、上下文包、测试、验证记录和 Skill，负责承载长期稳定的工程约束。真实项目里，这几类东西要一起用。

我是不是每个小需求都要写规格？

不用。

局部、低风险、可快速验证的任务，可以直接走轻量流程：任务说明、相关文件、小 diff、跑测试。跨模块、影响业务状态、回滚成本高的任务，再进入规格驱动。

AI 写的大 diff 如果看起来很好，要不要接受？

先拆。

让 AI 解释每个文件为什么改，再把无关改动剥离出去。真正好的 AI Coding 结果，不只是代码能跑，还应该能被正常审查。

什么时候该写 Skill？

当一个流程重复出现，而且每次都要解释很多规则时。

比如调试流程、代码审查流程、完成前验证流程、前端截图验证流程。这类流程适合沉淀成 Skill。一次性的任务背景不适合写 Skill，放在任务上下文包里就够了。

MCP 是不是也要早点学？

不需要。

先学会任务判断、上下文、验证和规则文件。等你遇到 AI 需要查文档、看 Issue、读只读数据、操作浏览器、访问代码索引时，再学 MCP 接入边界。MCP 的价值是安全接外部资源，不是自动让代码变好。

工程流程会不会限制 AI 的发挥？

会限制，而且应该限制。

真实项目需要的是可审查的输出，不是最大化生成内容。AI 一次改很多文件，看起来像效率高，但 reviewer 看不懂、测试说不清、回滚切不开，最后还是返工。

工程流程不是让 AI 少做事，而是让它在正确边界里做事。低风险任务可以放宽，高风险任务必须收紧。

1.16 先把底座搭起来

程序员用 AI Coding，起点不是把 prompt 写得更漂亮。

先把底座搭起来：判断任务风险，给对上下文，限制 diff，留下验证证据，把反复有效的经验沉淀下来。

这些动作做稳以后，AI 才会从会写代码的聊天窗口，变成可以被约束、检查和复用的开发工具。

1.17 可复查资料

- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔1-2〕
- OpenAI Codex `AGENTS.md` 指南：https://developers.openai.com/codex/guides/agents-md〔1-3〕
- Anthropic Claude Code 最佳实践：https://www.anthropic.com/engineering/claude-code-best-practices〔1-4〕
- GitHub Copilot coding agent 任务说明：https://docs.github.com/en/copilot/tutorials/coding-agent/get-the-best-results〔1-5〕

本章资源

〔1-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔1-2〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔1-3〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

〔1-4〕 https://www.anthropic.com/engineering/claude-code-best-practices
https://www.anthropic.com/engineering/claude-code-best-practices

〔1-5〕 https://docs.github.com/en/copilot/tutorials/coding-agent/get-the-best-results
https://docs.github.com/en/copilot/tutorials/coding-agent/get-the-best-results

第2章　从聊天式开发到工程式开发

很多 AI Coding 失败，不是因为 AI 不会写代码，而是因为我们把真实项目当成聊天窗口来处理。

聊天窗口里，事情看起来很简单：描述需求，让 AI 改，测试失败再让它修。几轮下来，代码也许能跑，但 diff 变大了，修改范围说不清了，测试失败到底是不是本次引入也说不清了。

聊天式开发的典型问题就在这里：它把开发过程压成一段对话，却丢掉了真实工程里的控制点。

工程式 AI Coding 要做的事正好相反：不追求一次聊完，而是把任务拆成可检查的阶段。

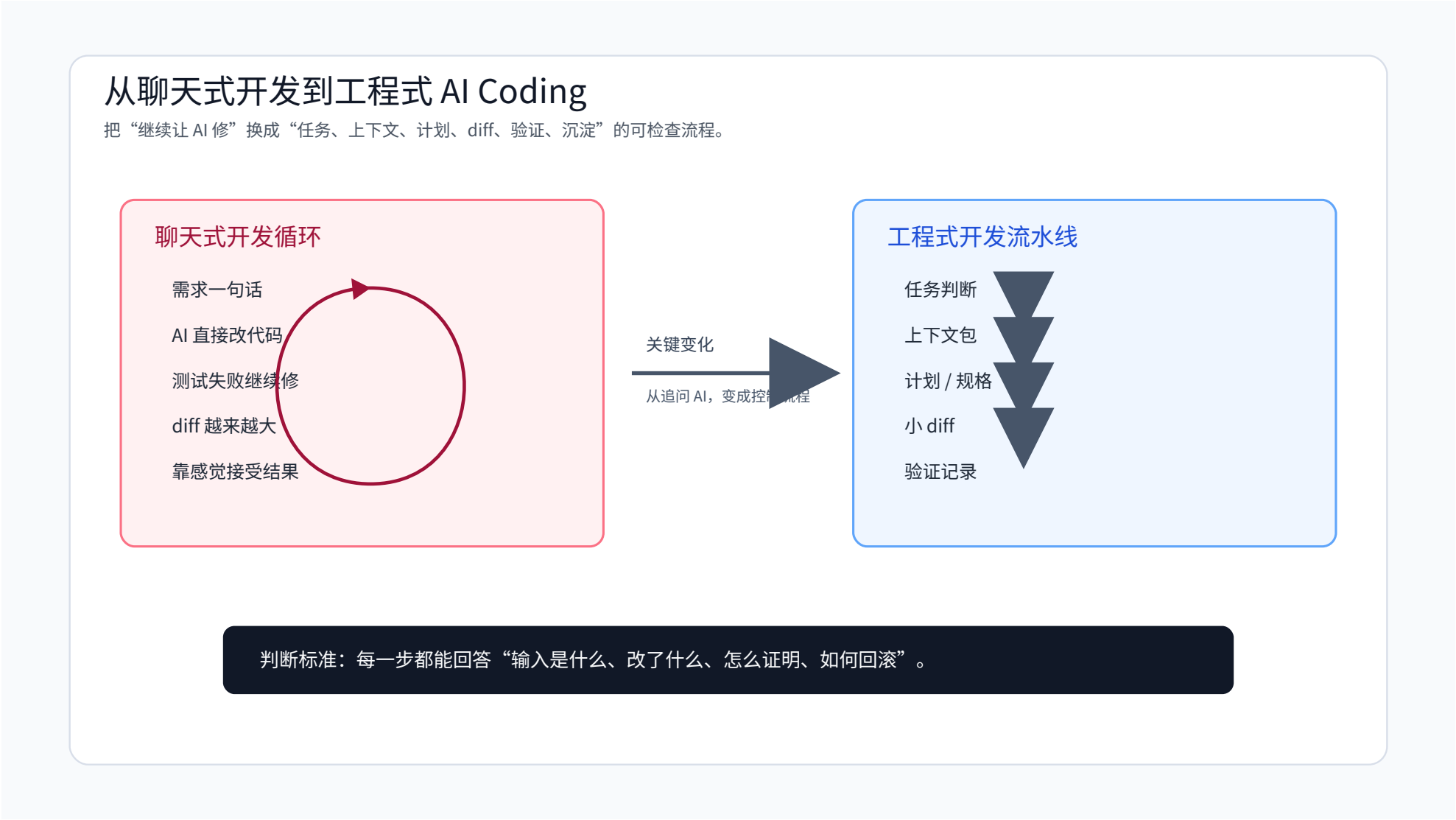


图 2-1　从聊天式开发到工程式 AI Coding

2.1 聊天式开发为什么容易失控

先看一个常见流程：

示例 2-1 聊天式开发为什么容易失控

```
我：帮我修一下用户无法提交订单的问题。
AI：我已经修复了。
我：测试失败了。
AI：我会继续修复。
我：页面还是不对。
AI：我会调整前端逻辑。
```

这段对话的问题不是“AI 态度不好”。它的问题是从一开始就没有工程边界。

它没有问清楚订单提交失败的复现路径，没有确认前后端入口，没有区分接口失败、状态错误、权限拦截还是前端表单校验，也没有定义修完之后怎么证明完成。

于是 AI 会沿着它最容易看到的代码路径修改。它读到一个 submit 方法，就改 submit；读到一个错误提示，就改错误提示；读到一个测试失败，就让测试通过。每一步都像在解决问题，但整体可能越来越偏。

真实项目里，这种失控常见在五个地方：

- 上下文失控：AI 读了相似文件，却没读到真实入口。
- 范围失控：局部 bug 被改成跨模块重构。
- 验证失控：测试失败后没有改前基线，无法判断责任。
- 语义失控：AI 把业务规则理解成普通 if/else。
- 复用失控：这次踩过的坑，下次还要重新解释。

聊天式开发不是不能用。它适合低风险小事。问题是，很多人拿它处理真实需求。

2.2 工程式开发的核心变化

工程式 AI Coding 不是把流程变复杂，而是把隐含判断显式化。

程序员平时做需求，本来就会问这些问题：

- 这个需求影响哪些模块?
- 哪些行为不能改?
- 有没有历史兼容逻辑?
- 先改哪一小步最安全?
- 改完跑什么测试?
- 如果失败，怎么回滚?

只是以前这些判断在你脑子里。AI 加入以后，这些判断必须写出来。

一个工程式流程可以很轻：

示例 2-2 工程式开发的核心变化

```
1. 任务判断：这个任务风险多高？
2. 上下文包：AI 应该先读哪些文件？
3. 计划：它准备改哪些文件，为什么？
4. 小 diff：一次只完成一个明确步骤。
5. 验证：跑命令、看输出、记录证据。
6. 沉淀：把反复出现的规则写回项目。
```

这六步不一定每次都写成长文。低风险任务可以很短，高风险任务要更完整。关键是顺序不能反。

2.3 第一步：先定义任务类型

不要一上来就让 AI 写代码。

先让它判断任务类型：

示例 2-3 第一步：先定义任务类型

```
先不要改代码。

请判断这个任务属于哪一类：
- 局部实现
- 陌生模块理解
- bug 排查
- 跨模块需求
- 重构
- 高风险业务改动

然后说明：
1. AI 可以负责哪些部分。
2. 哪些部分必须由人确认。
3. 实现前还缺哪些上下文。
```

这一步能过滤掉很多不该直接授权的任务。

比如“按钮样式不对”可能是低风险局部改动。“支付成功后订单状态偶尔回退”就不是。后者涉及状态流转、异步回调、幂等、历史数据和测试复现，必须先做上下文和验证设计。

工程式开发的第一条规则是：任务风险决定流程重量。

2.4 切换到工程式流程的几个硬信号

很多人不是不知道工程式流程有用，而是不知道什么时候该切。

可以记几个硬信号。只要出现其中两个，就不要继续聊天式开发。

第一个信号：AI 需要自己找入口。

如果你不能告诉它入口文件、测试文件、配置文件，只能说“你读一下项目”，那就说明任务还在侦察阶段。侦察阶段应该让 Agent 列事实和路径，不应该授权实现。

第二个信号：任务里有不能改变的行为。

比如旧接口响应、历史数据兼容、权限拦截、状态流转、字段语义。这些东西需要写进约束。只写“修一下”不够。

第三个信号：验证不是一条命令能覆盖。

前后端联动、异步任务、数据库状态、副作用、浏览器交互，都说明验证要分层。此时“跑了测试”只能覆盖一部分风险。

第四个信号：当前 diff 已经解释不清。

如果 AI 改了 8 个文件，但你只能说出其中 3 个为什么要改，就先停。让它分类 diff，再决定保留、拆分还是回滚。

第五个信号：测试失败开始被归因。

AI 说“这个失败和本次改动无关”时，不要直接接受。先让它给改前基线、失败命令、失败日志和影响面。没有这些证据，就不能继续在失败基础上叠补丁。

这些信号的共同点是：任务已经离开简单代码生成，进入工程责任区。

还有一个判断很实用：如果这次改动未来要被别人 review、要进 CI、要回滚、要写发布说明，就不要只按聊天式开发处理。真实交付天然需要证据链。

2.5 已经失控时怎么恢复

很多时候，你不是一开始就走工程式流程，而是聊了几轮之后才发现不对。

这时不要急着重开一个更长的 prompt。先把当前状态收住。

第一步，冻结修改。

```
不要继续编辑文件。
请只分析当前状态。
```

第二步，整理 diff。

```
列出当前已修改文件。
把每个文件分成三类：
- 直接服务原任务。
- 顺手重构或无关修改。
- 可能改变外部行为但未被授权。
```

第三步，回到事实。

```
列出已确认事实、未验证假设、已排除假设。
标出当前失败命令和输出。
```

第四步，重新定义下一步。

```
如果只保留最小修复，下一步应该改哪个文件？
需要先补哪条测试或复现证据？
哪些改动建议拆到独立任务？
```

这个恢复流程的目的不是让 AI 自己回滚。真实工作区可能有你的改动，也可能有已经确认过的改动。Agent 先分类，人再决定怎么处理。

一旦恢复流程跑完，就不要继续用原来的大对话推进。把清理结果整理成任务卡，再让 Agent 执行一个最小步骤。

2.6 第二步：让上下文可复查

聊天式开发里，最常见的一句话是：

```
你先读一下这个项目。
```

这句话没什么约束力。

AI 读哪些文件、有没有漏关键路径、是否把同名但无关的文件当成入口，你都不知道。先让它列出实际读取路径，并解释为什么读。

可以这样问：

```
请先建立任务上下文，不要改代码：
1．列出你实际读取过的文件路径。
2．标出入口文件、核心实现、测试文件、配置文件。
3．说明哪些文件看起来相关但暂时不该改。
4．如果缺少关键上下文，先提问。
```

代码地图可以单独沉淀。这里先记住一个原则：上下文必须可复查。

你不能只听 AI 说“我已经理解项目”。你要看它到底读了什么。

配套代码仓库 DavidDingXu/learn-ai-coding〔2-1〕 的工具 module 里有代码地图命令，可以帮你先做一层仓库扫描：

清单 2-4 第二步：让上下文可复查

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js codemap --root .
```

它不会替代人工判断，但能快速列出文档、配置和测试入口，避免 AI 一开始就漏掉基础文件。

2.7 第三步：把实现变成计划

聊天式开发里，AI 很容易直接进入实现。工程式开发里，实现之前要有计划。

计划不需要复杂。一个小需求可以只有这几行：

清单 2-5 第三步：把实现变成计划

```
## Plan

1. 修改 `src/order.ts` 的状态回退判断。
   - 原因：当前入口在这里。
   - 验证：新增失败测试。

2. 补充 `test/order.test.ts` 的已支付订单回退用例。
   - 原因：锁定业务边界。
   - 验证：`npm test`。
```

计划的作用不是让 AI 写方案文档，而是让你在代码生成前有机会发现问题。

如果计划里出现这些信号，就要暂停：

- 它准备改无关文件。
- 它没有提测试。
- 它想新增依赖。
- 它把重构混进 bug 修复。
- 它说“顺便优化”。
- 它跳过了你明确要求保留的旧行为。

计划阶段发现问题，成本很低。代码写完再发现，成本就高多了。

2.8 第四步：限制 diff

AI 一次改太多，是工程式开发最大的敌人之一。

可以给它一个固定约束：

示例 2-6 第四步：限制 diff

```
只执行计划的第 1 步。

要求：
- 不改计划外文件。
- 不做重构。
- 不新增依赖。
- 如果必须扩大范围，先停止并说明原因。
```

这条规则看起来简单，但非常有效。

它会逼 AI 把复杂任务拆小，也会降低审查成本。一个 50 行 diff 和一个 700 行 diff，审查难度不是线性差别。后者经常会让人放弃认真看。

工程式开发里，diff 必须服务任务，不服务 AI 的发挥。

2.9 第五步：验证不是一句话

“已修复”不是验证。

验证至少要包含三类信息：

清单 2-7 第五步：验证不是一句话

```
## 改前
- 问题如何复现？

## 改后
- 跑了什么命令？
- 输出是什么？

## 剩余风险
- 哪些路径没覆盖？
```

如果是前端改动，还要看页面。 如果是接口改动，还要看请求和响应。 如果是状态流转，还要看正向路径和禁止路径。 如果是权限逻辑，还要看允许和拒绝两类用例。

验证证据还要和风险匹配。改一行文案，截图或页面预览可能够了；改状态机，只看单元测试就不够，还要补禁止流转、重复回调和历史数据场景。AI 最容易漏掉的不是 happy path，而是那些过去靠程序员经验兜住的边界。

不要让 AI 用一句“测试通过”带过。让它贴命令、输出和影响范围。

配套工程的工具 module 里的验证记录检查器可以作为最小约束：

清单 2-8 第五步：验证不是一句话

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
```

如果记录缺少关键段落、命令块或通过信号，它会直接指出缺口。

2.10 第六步：把经验写回规则

工程式开发最后一步是沉淀。

如果这次任务里你反复提醒 AI：

- 不要改 generated 文件。
- 不要新增依赖。
- 测试失败先跑 baseline。
- 前端改动必须截图。
- 只改计划内文件。

那这些内容就应该进入规则文件、检查清单或 Skill。

聊天式开发的问题是，每次都重新聊一遍。工程式开发的价值是，下一次不用从零开始。

规则沉淀可以很轻：

清单 2-9 第六步：把经验写回规则

```
## AI Coding Rules

- Keep diffs scoped to the requested task.
- Do not edit generated files.
- Run baseline before blaming existing failures.
- Ask before adding dependencies.
- For UI changes, verify in browser and keep screenshot evidence.
```

不用一次写很多。只写高频、稳定、会影响结果质量的规则。

2.11 失败路径：越修越乱是怎么发生的

很多人以为 AI Coding 失控是突然发生的。实际不是。它通常是一点点滑下去的。

第一步，需求没有边界。

你只说“修复订单状态回退”，没有说哪些状态允许回退、哪些状态禁止回退。AI 会从代码里猜业务规则。它看到 `status !== CANCELLED`，可能就会推断“非取消都可以改”。这类推断看起来合理，但业务上可能完全错。

第二步，上下文没有入口。

AI 读到了 `OrderService`，但真实入口在异步回调。它改了同步下单路径，测试也能补上，但线上问题依旧存在。这个时候如果继续说“还不对，再修”，AI 往往会沿着已经读过的错误路径继续挖。

第三步，测试被当成目标，而不是证据。

测试失败后，AI 可能会调整测试断言，让它匹配当前实现。它不是故意偷懒，而是它的优化目标经常会变成“让测试通过”。如果没有明确告诉它业务边界，测试本身也会被它改成错误现实的证明。

第四步，diff 扩散。

为了让新逻辑跑通，它开始改工具函数、类型定义、前端状态、测试 helper。每个修改都有解释，但合在一起已经超出原任务。人再去审这个 diff，会很难判断哪一行是真正必要的。

第五步，验证缺失。

AI 最后告诉你“问题已修复”。但没有改前复现，没有命令输出，没有页面截图，没有接口响应，没有残留风险说明。你只能靠感觉接受。

聊天式开发最危险的地方，不是每一步都错，而是每一步都缺少边界。

2.12 什么时候可以保持轻量

工程式开发不是要把所有小事都变复杂。

下面这些任务可以轻量处理：

- 改一段不会影响逻辑的文案。
- 补一个局部函数的边界判断。
- 解释一段日志或异常栈。
- 生成一个临时脚本。
- 给已有测试补一个简单 case。

轻量处理也不是完全无规则。至少要保留三个动作：

```
1. 指定相关文件。
2. 限制改动范围。
3. 跑一个能证明结果的命令。
```

比如：

```
只修改 `src/format-price.ts`。
补充人民币分转元的格式化边界。
不要改其它文件。
完成后运行 `npm test -- format-price`。
```

这已经足够，不需要写完整 spec。

真正需要工程式流程的是这些任务：

- 改状态流转。
- 改权限判断。
- 改金额、库存、优惠、账务逻辑。
- 改跨前后端接口。
- 改历史兼容逻辑。
- 做重构。
- 修复无法稳定复现的问题。
- 处理测试大量失败。

任务越接近业务不变量，越不能只靠聊天。

2.13 一套可以直接复用的转换清单

当你发现自己已经进入“继续让 AI 修”的循环，可以立刻停下来，用下面这套清单重启任务。

清单 2-10 一套可以直接复用的转换清单

```
# AI Coding Reset Checklist

## 1. 当前问题
- 现在失败的现象是什么?
- 是否有稳定复现路径?

## 2. 当前 diff
- AI 已经改了哪些文件?
- 哪些文件不在原计划内?
- 是否需要先回滚部分改动?

## 3. 真实入口
- 用户动作或系统事件从哪里进入?
- 后端入口、前端入口、异步任务入口分别是什么?

## 4. 业务边界
- 哪些行为必须保持不变?
- 哪些旧数据或旧接口需要兼容?
```

```
## 5. 下一步
- 只保留哪一个最小改动?
- 用哪个命令证明它有效?
```

这张清单可以把已经发散的对话重新拉回工程事实。

如果当前 diff 已经很乱，最好的动作通常不是继续修，而是让 AI 帮你分类：

示例 2-11 一套可以直接复用的转换清单

```
请不要继续修改。

请分析当前 diff:
1. 哪些改动直接服务原任务?
2. 哪些改动是顺手重构?
3. 哪些改动可能改变外部行为?
4. 哪些改动建议回滚?
5. 如果只保留最小修复，需要留下哪些文件?
```

让 AI 审自己的 diff，比让它在混乱 diff 上继续写代码更有价值。

2.14 怎么把工程式流程放进工具

工程式 AI Coding 不是只能靠人工记忆。你可以把关键动作放进工具和规则里。

第一层，写进项目规则。

比如：

清单 2-12 怎么把工程式流程放进工具

```
## AI Coding Rules

- Start with a short plan for any change touching more than one file.
- Keep the first diff focused on one behavior change.
- Do not change tests just to match the current implementation.
- Report baseline, command, output, and residual risk before saying done.
```

第二层，写成任务模板。

比如任务上下文包、debug worksheet、verification ledger。这些模板不需要绑定某个工具，Codex、Claude Code、Cursor 都能用。

第三层，写成 Skill 或脚本。

当流程开始重复出现，比如每次调试都要 baseline、假设、验证、ledger，就可以写成 Skill。确定性的检查动作，比如验证记录是否缺段落、代码地图是否能生成，可以写成脚本。

`ai-coding-workflow-kit` 就放在第三层。它不是业务系统，也不模拟一个假业务，而是服务 AI Coding 本身的真实工作流：代码地图、任务上下文、只读上下文包和验证记录。

这三层的边界要守住。

规则文件写长期约束，不能写一次需求。任务模板写本次目标，不能写项目百科。Skill 或脚本写可复用流程，不能包办业务判断。

比如“不要修改 generated 文件”可以放规则文件。

“这次只修 session 过期后首次提交失败”应该放任务上下文。

“调试时先记录 baseline、再列假设、再验证”可以放 Skill。

“用户登录失败时应该重试几次”不是工具规则，而是产品和系统设计决策，需要人确认。

把这些边界分清，工程式流程才不会变成文档堆叠。

2.15 读者迁移时要注意什么

把这套流程迁移到你的项目，不要照抄文件名。

前端项目重点看组件入口、状态管理、路由、接口封装、浏览器验证。 后端项目重点看 service 入口、持久化层、异步任务、事务边界、测试环境。 老项目重点看历史兼容逻辑、隐式配置、脚本命令、无法轻易跑通的测试。 多仓项目重点看 API 合同、版本依赖、发布顺序和回滚路径。

迁移时先做一件小事：找一个真实低风险需求，按工程式流程跑一遍。

不要一开始就挑战最复杂需求。你需要先验证流程能不能在你的项目里跑通：AI 能不能读到正确入口，测试命令能不能执行，规则文件有没有效果，验证记录能不能留下来。

流程跑通以后，再逐步提高任务复杂度。

2.16 一个完整对照

同样是“修复订单状态回退问题”，两种做法差别很明显。

聊天式写法：

```
帮我修一下订单状态回退的问题，补一下测试。
```

工程式写法：

示例 2-13 一个完整对照

```
先不要改代码。

目标：修复已支付订单被回退到 pending 的问题。

请先做实现前检查：
1. 读取订单状态流转入口、异步回调、相关测试。
2. 列出实际读取过的文件路径。
3. 判断哪些状态不能回退。
4. 给出最多 3 步计划。
5. 每一步说明验证命令。

限制：
- 不改公开 API。
- 不新增依赖。
- 不做顺手重构。
- 如果需要扩大范围，先停下来说明原因。
```

区别不在于第二段更长。区别在于第二段把控制点写清楚了。

AI 读什么、能改什么、不能改什么、怎么验证、什么时候停下来，这些都变成了明确约束。

2.17 常见问题

工程式开发会不会太慢?

对低风险小事，确实没必要重流程。改一个文案、补一个简单判断，可以轻量处理。

但对真实需求，工程式开发省的是返工时间。它让问题更早暴露在计划和上下文阶段，而不是等大 diff 出来以后再补救。

是不是所有需求都要写 spec?

不是。

规格驱动适合复杂需求、跨模块改动和高风险业务逻辑。普通小改动只需要任务上下文、计划和验证。流程重量要跟风险匹配。

AI 已经能自动读仓库，还需要上下文包吗?

需要。

自动读仓库不等于读对文件。上下文包的价值是告诉 AI 优先级和边界，也让你能复查它有没有漏关键路径。

如果 AI 不按计划改怎么办?

不要继续让它补救大 diff。先要求它解释每个文件为什么改，然后把计划外改动剥离出来。必要时直接回滚到上一步，重新按小任务执行。

工程式 AI Coding 最先该练什么？

先练三件事：实现前检查、小步计划、验证记录。

这三件事不依赖特定工具，也不需要引入框架。练熟以后，再上 SDD、OpenSpec、Skill、MCP、多 Agent，才不会变成概念堆叠。

2.18 切换的标志

判断是否该切到工程式流程，不看需求描述有多长，而看它是否进入真实代码库、真实业务边界和真实交付责任。

低风险任务可以继续聊天式处理。改一段文案、补一个明显判断、解释一段错误日志，没有必要把流程拉满。

一旦改动需要被审查、回滚、发布或复盘，就要切到工程式流程：任务判断、上下文、计划、小 diff、验证、沉淀。

这一步切过去，AI Coding 才会回到开发本身，而不是停在连续追问里。

2.19 可复查资料

- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔2-2〕
- OpenAI Codex `AGENTS.md` 指南：https://developers.openai.com/codex/guides/agents-md〔2-3〕
- Anthropic Claude Code 最佳实践：https://www.anthropic.com/engineering/claude-code-best-practices〔2-4〕
- GitHub Copilot coding agent 任务说明：https://docs.github.com/en/copilot/tutorials/coding-agent/get-the-best-results〔2-5〕

本章资源

〔2-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔2-2〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔2-3〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

〔2-4〕 https://www.anthropic.com/engineering/claude-code-best-practices
https://www.anthropic.com/engineering/claude-code-best-practices

〔2-5〕 https://docs.github.com/en/copilot/tutorials/coding-agent/get-the-best-results
https://docs.github.com/en/copilot/tutorials/coding-agent/get-the-best-results

第3章 AI Coding 的底层工作模型

很多程序员第一次认真用 AI Coding，都会高估一件事：以为 Agent 真的“理解了整个项目”。

它没有。

更准确的说法是：Agent 在一组指令、有限上下文和可用工具之间循环工作。它能读文件、改文件、执行命令、观察输出，再根据新的输出继续调整。它看起来像在理解项目，本质上是在不断把“可见证据”转成下一步行动。

这个差别会影响你的用法。

如果你把 AI 当成一个已经熟悉项目的同事，就会自然地给它一句模糊任务：“帮我修一下订单提交失败”。它会开始搜索、猜测、修改、运行测试，最后给你一句“已完成”。如果你把它看成一个基于上下文和工具反馈工作的开发 Agent，你就会先关心另外几个问题：

- 它实际读了哪些文件?
- 它有没有看到真实入口?
- 它的计划是否扩大了改动范围?
- 它跑的命令能证明什么，不能证明什么?
- 它遇到失败后，是基于证据收敛，还是在继续试错?

AI Coding 的稳定性，不来自更会写 prompt，而来自你能不能看懂这个工作循环，并把关键证据钉住。

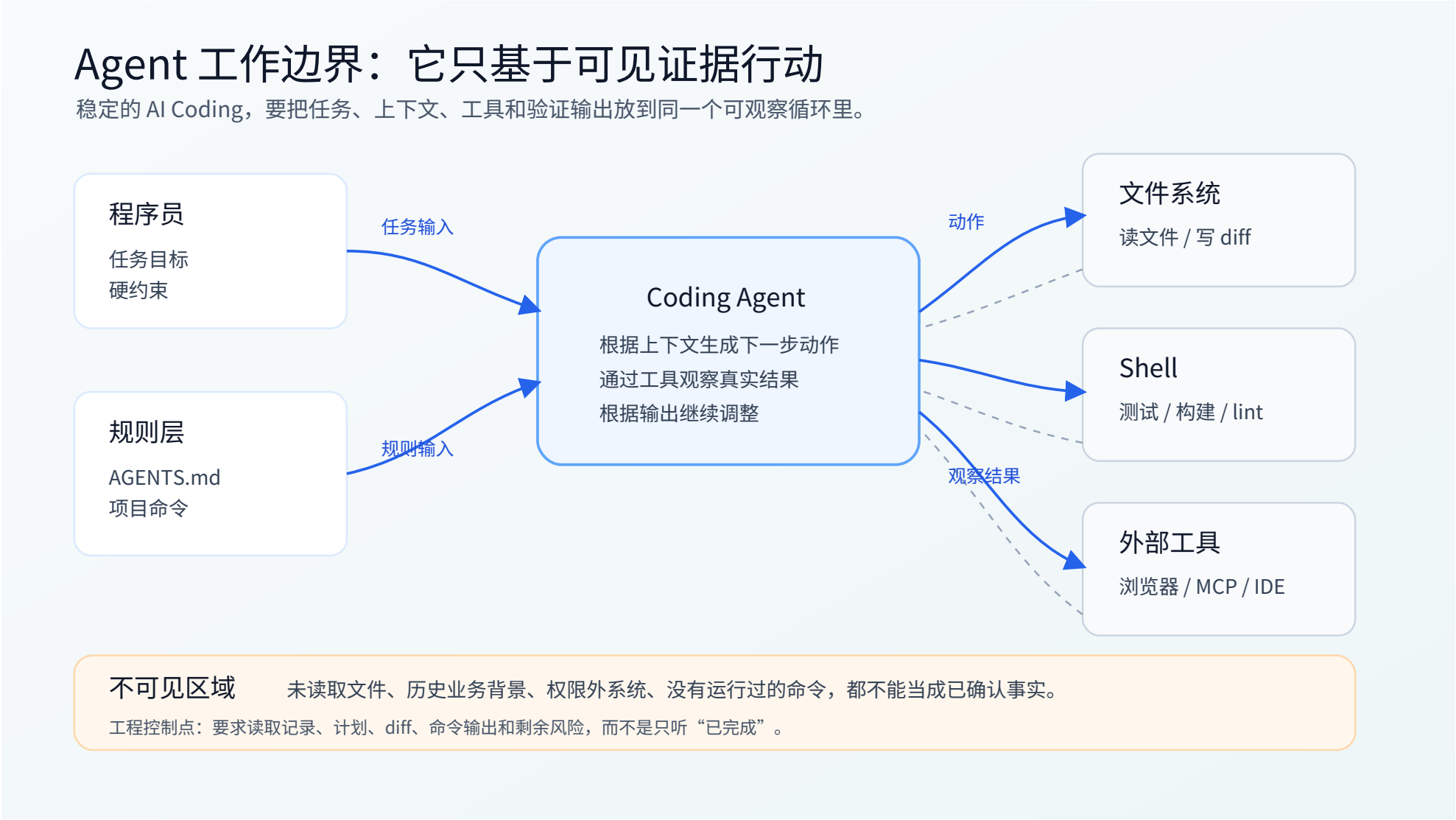


图 3-1　Agent 工作边界：模型、工具、仓库和证据

3.1 Agent 不是一个“会写代码的模型”

单独看模型，它只是根据输入生成输出。放到 AI Coding 场景里，事情变成了一个更复杂的系统：

- 有一层指令：系统规则、个人规则、项目规则、当前任务要求。
- 有一块上下文：用户粘贴的需求、Agent 读取的文件、搜索结果、历史对话。
- 有一组工具：文件读取、文件编辑、Shell、测试命令、浏览器、MCP 服务、IDE 能力。
- 有一个反馈通道：命令输出、测试失败、lint 报错、浏览器截图、Git diff。

真正参与开发的不是“模型”一个东西，而是模型加工具形成的工作循环。

OpenAI Codex 官方文档对 Codex CLI 的描述也很直接：它是在本机选定目录里读代码、改代码、运行代码的 coding agent。Codex 的最佳实践也强调几个方向：给对任务上下文、复杂任务先计划、把可复用规则放进 `AGENTS.md`、用配置和外部系统约束 Agent 行为。

这些建议背后的共同点不是“提示词更精致”，而是把 Agent 工作时依赖的输入、工具和反馈变得更可控。

程序员需要先接受一个事实：Agent 的代码能力很强，但它没有天然的项目记忆。它没有读过的文件，对它来说就是不存在；它没有跑过的命令，对它来说就不是证据；它没有被明确约束的边界，很容易被当前上下文里最显眼的线索带偏。

3.2 一个真实任务在 Agent 眼里是什么样

拿一个常见需求举例：

```
用户在订单页点击提交后，偶尔会提示“状态异常”，但订单其实已经创建成功。
请修复这个问题。
```

人看到这句话，会自然联想到订单状态、接口幂等、前端重复提交、支付回调、后端事务、历史兼容逻辑。

Agent 不会自动拥有这些背景。它要先把任务转成一组可搜索线索：

清单 3-1 一个真实任务在 Agent 眼里是什么样

```
rg -n "状态异常|submitOrder|createOrder|checkout|order status" src test
rg --files | rg "order|checkout|payment|test|spec"
git status --short
```

这些命令不是为了展示命令行技巧，而是为了回答三个基础问题：

1. 相关入口在哪里。
2. 有没有现成测试或规格文件。

3. 工作区是否已经有未提交改动。

如果这一步跳过，Agent 很可能只改到最先搜索出来的前端提示文案。它看到了“状态异常”，就修改错误提示；看到了 `submitOrder`，就加一个重试；看到了测试失败，就让测试通过。每一步都像在工作，但问题可能根本不在这里。

Agent 的行动路径高度依赖它先看到什么。

所以，程序员不能只问“它会不会写代码”，还要问“它是从哪里开始看的”。

3.3 Agent 的工作循环

一个可控的 AI Coding 过程，通常会经过六个阶段。

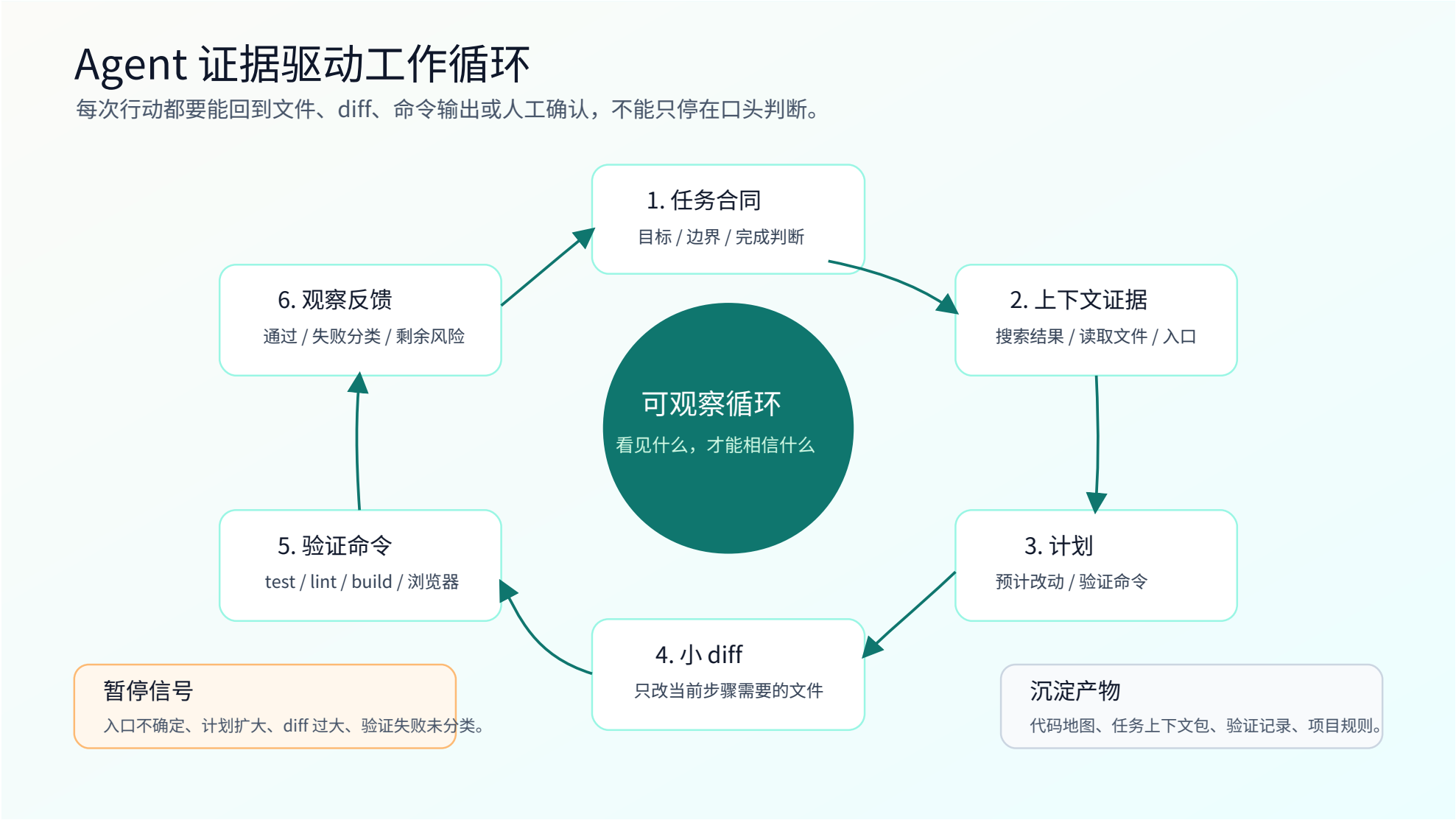


图 3-2　Agent 证据驱动工作循环

第一步是任务合同。你给它的不只是需求描述，还包括不能改什么、完成后怎么证明、是否允许新增依赖、是否允许跨模块修改。

第二步是上下文获取。Agent 通过搜索和读文件建立当前任务视野。这里最重要的不是“读了很多文件”，而是读到了入口、调用链、测试、配置和约束。

第三步是计划。计划不是文档仪式，而是在写代码前暴露它的判断。它准备改哪些文件，为什么改这些文件，预计跑什么命令，都应该先说出来。

第四步是小范围编辑。真正的工作边界不是它口头说“我只改一点”，而是 Git diff。一次改动越大，审查成本越高，问题归因越难。

第五步是验证命令。测试、lint、类型检查、构建、浏览器验证、接口调用，都是把代码行为变成证据的手段。命令通过不等于业务完全正确，但命令没跑就更谈不上完成。

第六步是观察和复盘。Agent 要解释命令输出、失败原因、剩余风险，并决定是否继续。很多失控发生在这里：测试失败后没有分类，直接继续改，最后把原本的小问题改成大 diff。

这六步可以很轻，也可以很重。按钮文案这种低风险任务，可以两三步完成。涉及状态机、权限、金额、数据迁移的任务，就必须把每一步都写清楚。

3.4 指令层：Agent 首先会被什么约束

Agent 开始工作前，会先读到一组指令。不同工具的文件名和加载规则不完全一样，例如 Codex 使用 `AGENTS.md` 提供项目指导，Claude Code 常见的是 `CLAUDE.md`，GitHub Copilot 也有 repository instructions 和自定义指令能力。

这里不要把重点放在文件名上。重点是指令分层。

一份有效的项目规则，应该回答这些问题：

清单 3-2 指令层：Agent 首先会被什么约束

```
# Project Instructions

## Commands

- Install: `npm install`
- Test: `npm test`
- Lint: `npm run lint`

## Work Rules

- Do not edit generated files under `dist/`.
- Do not add runtime dependencies without approval.
- Keep bug fixes scoped to the failing behavior.
```

```
- Before changing backend behavior, read the related tests first.

## Verification

- For behavior changes, run the focused test first.
- If broader shared code changed, run the full test suite.
- Report command output and remaining risk.
```

这类规则的价值不是“提醒 AI 要认真”。它会改变 Agent 的默认行动路径。

如果规则里写清楚 `dist/` 不能改，Agent 在发现构建产物报错时就不应该直接编辑构建产物。如果规则里写清楚新增依赖要先确认，它遇到一个解析问题时就不能顺手安装包。如果规则里写清楚共享代码改动要跑全量测试，它就不能只跑一个局部用例后宣布完成。

项目规则不需要长。长而空的规则反而容易污染上下文。好规则通常来自真实事故：AI 改错过哪里，就把能预防复发的规则写下来。

3.5 上下文层：搜索不是理解，读到关键路径才算开始

Agent 最容易犯的错之一，是把相似文件当成关键文件。

比如一个仓库里同时有：

```
src/order/submit.ts
src/order/submit.legacy.ts
src/demo/order-submit.ts
test/order-submit.test.ts
docs/order-submit.md
```

如果任务是修线上订单提交，`demo` 和 `docs` 可能只是参考；`legacy` 可能只服务旧接口；真正入口可能在路由、service 或 controller 里。Agent 只读到一个文件并不能说明它找到了入口。

让 Agent 建立上下文时，可以直接要求它输出一份读取记录：

示例 3-3 上下文层：搜索不是理解，读到关键路径才算开始

```
先不要改代码。

请完成上下文侦察：
1．列出你实际读取过的文件路径。
2．标出入口文件、核心实现、测试文件和配置文件。
3．说明哪些文件看起来相关但暂时不应该改。
4．如果无法确认真实入口，停止并提问。
```

这个要求有两个作用。

第一，它迫使 Agent 暴露自己的证据。你能看到它是否只读了表层文件。

第二，它让你在实现前纠偏。实现前纠偏很便宜，写完代码再纠偏就会变成清理 diff。

配套代码仓库 DavidDingXu/learn-ai-coding〔3-1〕 的工具 module 里有一个轻量代码地图命令，可以在接手陌生仓库时先扫一遍结构：

清单 3-4 上下文层：搜索不是理解，读到关键路径才算开始

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js codemap --root .
```

它不会替代 rg 和人工判断，也不会解析完整 AST。它的价值是把文档、配置、测试入口先列出来，给 Agent 和人一个共同起点。

在真实项目里，我更建议把它当作“第一层索引”：先知道仓库有哪些明显入口，再让 Agent 针对具体任务搜索调用链。

3.6 计划层：先暴露判断，再允许实现

很多人不让 AI 写计划，是担心它浪费时间。这个担心有一半是对的：如果计划只是泛泛写“分析、实现、测试”，确实没价值。

有价值的计划应该短，但必须具体。

清单 3-5 计划层：先暴露判断，再允许实现

```
## Plan

1. Read `src/order/submit.ts` and `test/order-submit.test.ts`.
   Reason: confirm current submit flow and existing behavior coverage.

2. Add or adjust one failing test for paid order status rollback.
   Reason: lock the bug before changing implementation.

3. Change only the status transition guard in `src/order/submit.ts`.
   Reason: keep the fix scoped to the observed failure.

4. Run `npm test -- order-submit`.
   Reason: verify the targeted behavior before wider checks.
```

看计划时，重点看五个信号：

- 它有没有把读文件和写代码分开。
- 它有没有说明为什么改这些文件。
- 它有没有提到测试或可观察验证。
- 它有没有混入“顺手重构”。
- 它有没有扩大到不相关模块。

如果计划里出现“优化整体结构”“统一抽象”“顺便清理旧逻辑”，就要停一下。不是这些事情永远不能做，而是它们不应该混在一个 bug fix 里。

AI Coding 最容易产生的问题，不是代码写不出来，而是它在一个任务里做了太多事。计划阶段就是防止这个问题的第一道门。

3.7 编辑层：真正的边界是 diff

Agent 说“我只做了小改动”没有意义。真正的边界看 diff。

每次实现后，至少要看两类信息：

清单 3-6 编辑层：真正的边界是 diff

```
git diff --stat
git diff
```

`git diff --stat` 看改动范围，`git diff` 看实际语义。很多问题只看测试是看不出来的。

例如，测试通过但 diff 里出现这些内容，就要谨慎：

- bug fix 顺手改了公共工具函数。
- 前端交互问题顺手改了后端接口。
- 为了让测试通过删除了断言。
- 修改了锁文件却没有说明新增依赖。
- 改动了格式化大文件，遮住真实逻辑变化。
- 把错误吞掉了，页面不报错但业务状态也没处理。

这里有一个很实用的交互方式：

示例 3-7 编辑层：真正的边界是 diff

```
只执行计划第 2 步。

约束：
- 不改计划外文件。
- 不新增依赖。
- 不做重构。
- 如果必须扩大范围，先停止并说明原因。

完成后输出：
- 修改了哪些文件。
- 每个文件为什么必须改。
- 还没有跑的验证命令。
```

这段话的重点不是措辞，而是把 Agent 的自由度压到当前步骤里。

如果它真的发现必须扩大范围，例如测试入口依赖一个公共状态枚举，而枚举定义有缺陷，那也可以扩。但必须先停下来说明。这个停顿很关键，它让人重新判断风险。

3.8 验证层：命令输出是证据，不是结论

Agent 很喜欢说“测试通过”。程序员不能只看这句话。

你要看它跑了什么命令，在什么目录跑的，输出是什么，命令和本次改动的关系是什么。

验证报告要写到这个粒度：

清单 3-8 验证层：命令输出是证据，不是结论

```
## Verification

- Command: `npm test -- order-submit`
- Directory: `learn-ai-coding/checkout-service`
- Result: passed, 8 tests
- Why this command matters: covers submit flow and paid-order rollback guard

## Not verified

- Real payment provider callback was not exercised locally.
- Existing integration test suite was not run because it requires staging credentials.
```

这里的“不确定”不是缺点，反而是专业表现。真实项目里很多东西无法在本地完全验证，例如第三方支付回调、生产数据分布、异步任务延迟、浏览器兼容性。Agent 如果把这些都说成“已完成”，风险更大。

配套工程的工具 module 里的验证记录校验命令，就是为了让完成报告不只停留在口头上：

清单 3-9 验证层：命令输出是证据，不是结论

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
```

这份记录至少要求有 baseline、change summary、verification commands、evidence 和 residual risk。它不能证明业务一定正确，但能防止最糟糕的情况：没有命令、没有证据、没有风险说明，却说完成了。

3.9 反馈层：测试失败以后，先分类再修改

测试失败是 AI Coding 里最容易失控的节点。

很多 Agent 会看到失败就继续改。改一次不行再改一次，最后测试可能绿了，但真实问题已经被绕过去。

失败后先分类。

表 3-1 反馈层：测试失败以后，先分类再修改

失败类型	典型信号	应该怎么处理
改前就失败	baseline 没跑，或同一命令在改前已失败	先记录基线，不把旧失败算成本次修复目标
本次引入失败	失败测试和 diff 直接相关	回到最小改动，优先修实现或测试预期
命令环境失败	缺依赖、缺服务、缺密钥、端口占用	不改业务代码，先处理环境或说明无法本地验证
规格冲突	测试期望和需求描述冲突	停下来让人确认业务规则
覆盖不足	命令通过，但关键路径没被覆盖	补验证路径，不要把通过当成完成

这张表在真实工作里很值钱。它能阻止 Agent 把环境问题改成业务代码，把历史失败算成自己修复，把规格冲突硬编码进实现。

你可以直接把它转成提示：

示例 3-10 反馈层：测试失败以后，先分类再修改

```
测试失败后不要直接修改代码。

请先分类：
- 这是改前已存在失败，还是本次引入失败？
- 是实现问题、测试预期问题、环境问题，还是需求不清？
- 你需要查看哪些文件或输出才能确认？

分类完成前，不要继续编辑文件。
```

这种提示比“仔细一点”有效得多，因为它改变了 Agent 的下一步动作。

3.10 四种最常见的偏航原因

第一种是上下文缺口。

Agent 只读到 controller，没有读 service；只读到前端组件，没有读 API 契约；只读到新测试，没有读历史兼容用例。它会基于残缺上下文做出看似合理的修改。

解决办法不是一次性塞更多文件，而是让它解释每个文件和任务的关系。上下文要精，不是要多。

第二种是指令冲突。

用户说“尽快修好”，项目规则说“不能改公共接口”，历史对话里又出现“可以重构一下”。Agent 可能会选择当前最显眼的指令。处理这种情况，要把本次任务的硬约束写在任务开头，例如“这次只修 bug，不重构，不改接口”。

第三种是验证太弱。

只跑类型检查，不能证明业务路径正确；只跑单元测试，不能证明页面交互正确；只看页面不报错，不能证明接口状态正确。验证必须和风险匹配。

第四种是长任务积累错误结论。

一次对话太长后，Agent 可能会把早期猜测当成事实。尤其是排障任务，前几轮假设如果没被及时淘汰，会一直影响后续判断。遇到这种情况，最好让它重新整理证据：

```
请停止修改，重新整理当前事实：
1．已确认的事实有哪些？
2．已排除的假设有哪些？
3．仍未验证的假设有哪些？
4．下一步最小验证动作是什么？
```

这个动作不是浪费时间。它是在清理错误上下文。

3.11 一套可以直接复用的交互协议

下面这套协议适合中等风险以上的任务。它不是固定格式，你可以按项目改短。

第一步，建立上下文：

示例 3-11 一套可以直接复用的交互协议

```
先不要改代码。

请围绕这个任务做上下文侦察：
- 用搜索找到可能相关的入口、实现、测试和配置。
- 列出实际读取过的文件。
- 标记你认为必须改和不应该改的文件。
- 如果真实入口不确定，先提问。
```

第二步，给计划：

示例 3-12 一套可以直接复用的交互协议

```
请给出最小实现计划。

要求：
- 每一步只对应一个可审查 diff。
- 说明预计修改文件和验证命令。
- 不允许新增依赖、重构或改接口，除非先说明必要性。
```

第三步，执行一小步：

示例 3-13 一套可以直接复用的交互协议

```
只执行计划的第一步。

完成后不要继续下一步，先输出：
- git diff --stat 的结果摘要。
- 修改文件和修改原因。
- 需要我确认的风险。
```

第四步，验证：

示例 3-14 一套可以直接复用的交互协议

```
运行和本次改动最相关的验证命令。

输出：
- 命令、目录、结果。
- 这个命令覆盖了什么。
- 没覆盖什么。
- 如果失败，先分类，不要直接继续改。
```

第五步，收口：

示例 3-15 一套可以直接复用的交互协议

```
请给出完成报告：
- 改了什么。
- 为什么这样改。
- 跑了哪些验证。
- 仍然存在什么风险。
- 哪些规则应该写回项目说明或工作流。
```

这套协议的核心是把 Agent 的每次行动都绑定到证据。它不要求每次都写长文，但要求每次都有可复查结果。

3.12 用配套工具把证据留在仓库里

很多 AI Coding 经验只停留在聊天记录里。下一次换任务、换工具、换对话，就要重新解释。

高频产物应该留在仓库里，下一次任务可以直接读取和复用。

配套工程的工具 module 目前提供四个小工具：

清单 3-16 用配套工具把证据留在仓库里

```
cd learn-ai-coding/ai-coding-workflow-kit

# 生成轻量代码地图
node src/cli.js codemap --root .
```

```
# 生成任务上下文包
node src/cli.js task-context \
  --title "修复订单状态回退问题" \
  --files "src/order.ts,test/order.test.ts"

# 生成只读上下文包
node src/cli.js context-pack \
  --title "检查 CLI 合同" \
  --files "src/cli.js,test/cli.test.js" \
  --done "npm test"

# 校验验证记录
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
```

这四个工具对应 Agent 工作循环里的四个证据点：

- `codemap` 让上下文起点可见。
- `task-context` 让任务目标、文件和约束可见。
- `context-pack` 把任务上下文、代码地图和验证交接组合成只读上下文包。
- `verify-ledger` 让完成证据和剩余风险可见。

它们不是平台，不是框架，也不需要进生产依赖。它们的价值是给真实项目提供可复制的工作形态：任务开始前先留下上下文，任务结束后留下验证证据。

3.13 什么时候可以放轻流程

不是所有任务都需要完整协议。

这些任务可以放轻：

- 改一处明显文案。
- 调整独立样式，不影响状态逻辑。
- 补一条文档说明。
- 改测试夹具里的拼写错误。
- 在已有模式下添加同类字段。

但只要出现下面任意一种信号，就不要直接让 Agent 实现：

- 涉及金额、权限、状态机、数据迁移。
- 涉及多个模块或多个仓库。
- 没有现成测试。
- 线上 bug 原因还不清楚。
- 需要判断历史兼容逻辑。

- Agent 提议新增依赖或重构。
- 测试失败需要解释。

流程轻重由风险决定。低风险任务追求速度，高风险任务追求可控。

3.14 常见问题

Agent 已经会自己搜索文件，还需要我要求读取记录吗？

需要。自动搜索只能说明它会找文件，不能说明它找对了。读取记录让你看到它的判断路径。很多错误在“它读了什么”这个层面就能发现。

我能不能直接把整个仓库塞给 AI？

一般不值得。上下文越多，噪音越多，旧约束和无关实现也会进入判断。先用搜索和代码地图找到候选路径，再按任务选择关键文件。

命令都跑通过了，还要看 diff 吗？

要。测试覆盖不了所有语义。AI 可能通过删除断言、吞异常、改无关逻辑让命令变绿。diff 是你判断工程边界的核心材料。

Agent 说环境缺依赖，能让它直接安装吗？

先看项目规则。生产项目里，安装依赖可能改变锁文件、运行时体积和安全面。让它说明缺什么、为什么缺、是否已有替代方案，再决定。

什么时候需要开新对话？

当对话里已经积累了很多被推翻的假设，或者任务从排障切到实现、从实现切到重构时，开新对话通常更干净。新对话要带上已确认事实、已排除假设和当前 diff，不要只带最终结论。

3.15 把这个模型迁移到自己的项目

你不需要一次搭完整体系。先从四件事做起。

第一，写一份短的项目规则文件。只放每次都要遵守的命令、目录、限制和完成判断，不放口号。

第二，给陌生模块生成一份代码地图。至少知道入口、测试、配置、生成文件和禁止修改目录。

第三，每个中等风险任务开始前，写一个任务上下文包。目标、相关文件、不能改的边界、预期验证命令，都放进去。

第四，任务结束后留下验证记录。哪怕只有三行，也要写清命令、结果和没覆盖的风险。

做完这四件事，Agent 就不再只是“会聊天写代码”的工具，而会进入一个可观察、可审查、可复用的工程流程。

接下来要处理任务授权问题：哪些任务可以让 Agent 主导，哪些任务只能让它辅助，哪些任务必须人先判断？这需要一张任务风险分级表。

3.16 可复查资料

- OpenAI Codex CLI 官方文档：https://developers.openai.com/codex/cli〔3-2〕
- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔3-3〕
- OpenAI Codex `AGENTS.md` 指南：https://developers.openai.com/codex/guides/agents-md〔3-4〕

本章资源

〔3-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔3-2〕 https://developers.openai.com/codex/cli
https://developers.openai.com/codex/cli

〔3-3〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔3-4〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

第4章 AI 最擅长和最不擅长的开发任务

判断一个任务能不能交给 AI，不要先问“AI 会不会写这段代码”。

先问四个问题：这个任务的目标是否清楚，影响范围是否可控，验证信号是否明确，关键业务判断是否已经有人做过。

这四个问题，比模型名字、工具名字、prompt 技巧都更重要。

AI Coding 最容易被误用的地方，是把所有任务都按“简单/复杂”来分。很多任务代码量不大，但风险很高；很多任务看起来复杂，但只要边界清楚、测试充分，反而适合交给 Agent 执行。

比如：

- 给已有接口补一条参数校验，可能很适合 AI 主导。
- 修改“订单已支付后是否允许回退状态”，代码也许只有两行，但不该让 AI 自己拍板。
- 把一批老 API 从 `callback` 改成 `async/await`，代码量很大，但如果有测试、有规则、有范围，也可以让 AI 分批做。
- 线上鉴权异常正在影响用户登录，这时 AI 可以帮你读日志和列假设，但关键决策必须人先控住。

所以，程序员用 AI Coding，先要建立任务分级能力。



图 4-1　AI Coding 任务风险分级矩阵

4.1 任务不是按“难不难”分，而是按“可控不可控”分

官方文档里其实有一致的方向。

OpenAI Codex 的最佳实践建议在任务里给清楚 goal、context、constraints、done when，复杂、模糊或难描述的任务先让 Codex plan 再实现。Anthropic 的 Claude Code 最佳实践强调给 Agent 一个可运行的检查信号，并把 explore、plan、code 分开。GitHub Copilot cloud agent 的文档也明确说，分配给 agent 的 issue 最好有清晰描述、完整完成条件和相关文件方向；复杂宽泛、敏感关键、需求模糊的任务更适合人自己处理。

这不是三家工具的偶然相似。它们都在提醒同一件事：Agent 更适合处理有边界、有上下文、有反馈的任务。

程序员可以把任务拆成四个维度来看：

表 4-1 任务不是按“难不难”分，而是按“可控不可控”分

维度	低风险信号	高风险信号
目标清晰度	一句话能说明改什么、完成后行为是什么	需求里有“优化一下”“体验更好”“兼容历史情况”
影响范围	单文件、单模块、已有相同模式	跨模块、跨仓库、公共接口、数据结构变化
验证信号	有单测、类型检查、截图、接口用例或可复现步骤	只能靠肉眼判断，或者依赖线上数据和第三方系统
业务风险	文案、样式、局部工具、非核心路径	金额、权限、状态机、数据迁移、隐私、安全、线上事故

这四个维度比“前端/后端”“简单/复杂”更有用。

一个前端按钮改色，如果影响设计系统组件，也可能不小。一个后端逻辑修复，如果只改一个已有 guard，并且有失败用例锁住边界，反而可以让 AI 主导实现。

4.2 四种任务处理模式

我通常把 AI Coding 任务分成四类。

第一类：AI 主导，人做快速审查。

适合目标明确、影响范围小、验证信号强的任务。例如：

- 文档补充和结构调整。
- 明确的测试补充。

- 已有模式下新增字段。
- 局部 bug，有复现用例。
- UI 小改，有截图对比。
- 简单脚本修复，有输入输出样例。

这类任务可以直接给 Agent 执行，但仍然要看 diff 和命令输出。

第二类：AI 实现，人控制计划。

适合中等复杂度任务。需求已经清楚，但涉及多个文件或少量架构边界。例如：

- 给现有模块加一个新能力。
- 补充接口字段，并同步类型、测试和页面展示。
- 修复一个涉及前后端状态一致性的 bug。
- 小范围重构，但范围已经被锁定。

这类任务不要让 Agent 一上来写代码。先让它读文件、出计划、拆步骤。人确认计划后，再让它一步一步改。

第三类：人主导判断，AI 辅助执行。

适合业务判断重、上下文复杂、验证困难的任务。例如：

- 支付、结算、退款、优惠券叠加规则。
- 权限模型和数据隔离。
- 历史兼容逻辑。
- 性能瓶颈定位。
- 复杂重构方案。
- 数据迁移策略。

这类任务里，AI 的价值很大，但不是拍板。它可以帮你画调用链、读历史提交、整理测试缺口、生成候选方案、补机械改动。关键业务规则和方案取舍要人先定。

第四类：人控制现场，AI 只做辅助分析。

适合高风险、不可逆、时间敏感的任务。例如：

- 线上事故处置。
- 安全漏洞修复。
- 用户隐私和鉴权异常。
- 生产数据修复。
- 大规模依赖升级。

· 删除数据、改迁移、改权限策略。

这里不要把 Agent 放到驾驶位。你可以让它读日志、总结证据、列排查路径、写回滚脚本草稿，但执行和决策必须有人确认。

4.3 一个任务路由流程

把上面的分类落到日常工作里，可以用下面这张路由图。

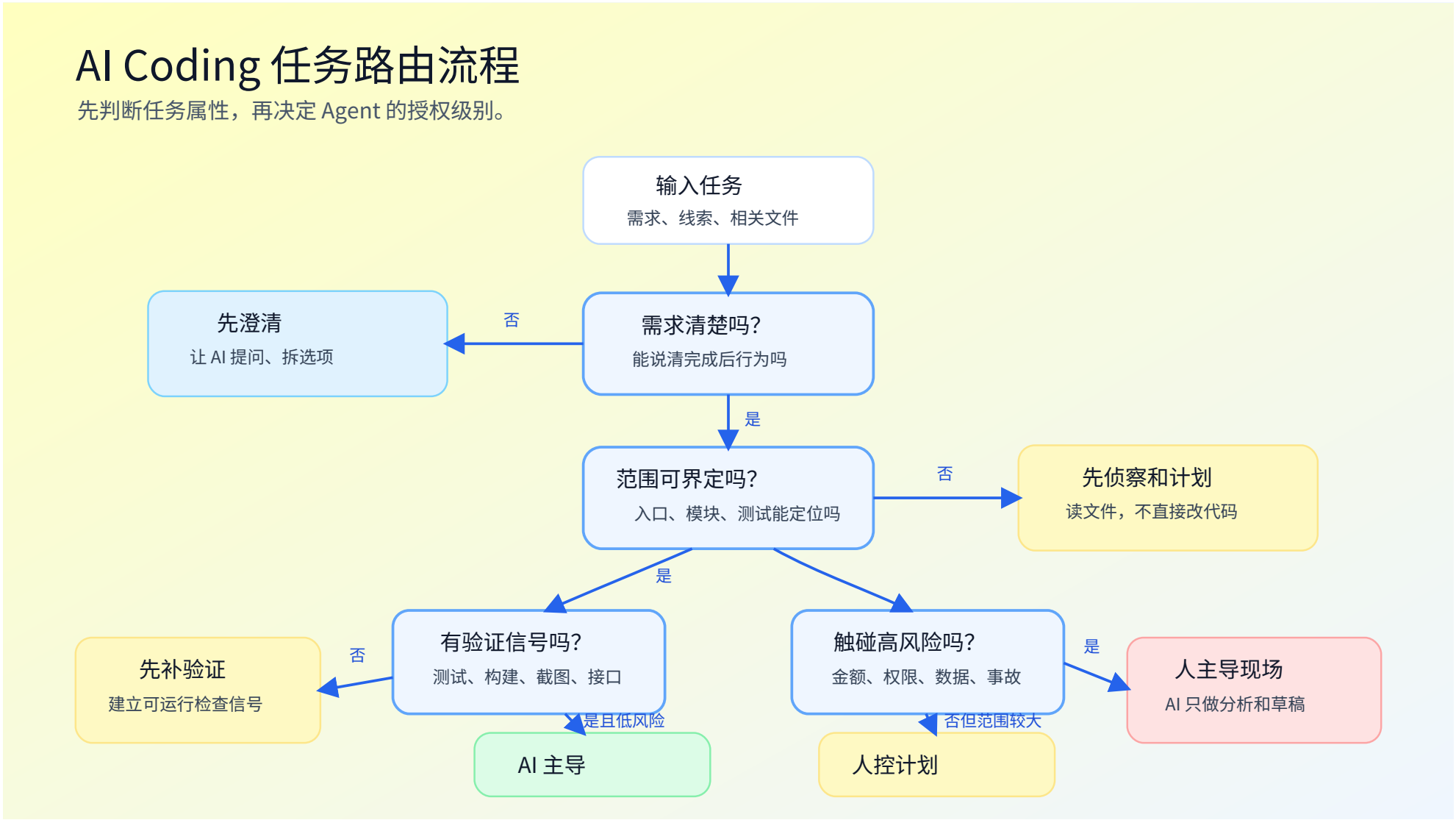


图 4-2　AI Coding 任务路由流程

这个流程看起来简单，但能挡掉很多事故。

第一问：需求是否清楚?

如果需求连“完成后什么行为应该改变”都说不清，不要直接实现。先让 AI 采访你，或者让它把需求改写成几个可确认的选项。模糊需求直接进实现，后面很容易变成“它改得很勤快，但不是我要的”。

第二问：影响范围是否能界定?

如果只涉及一个模块，而且你能指出入口、测试和约束，可以继续。如果 Agent 需要自己判断跨模块影响，就先让它做代码侦察和计划，不要授权改代码。

第三问：是否有可运行验证?

没有验证信号的任务，不代表不能做，但不能让 AI 自己宣布完成。至少要补一个命令、截图、接口调用、人工检查点或对比样例。验证信号越弱，人介入越多。

第四问：是否触碰高风险业务?

金额、权限、隐私、状态机、数据迁移、线上事故，不应该因为“代码看起来简单”就放给 AI 主导。这里的风险不在代码行数，而在错误后果。

4.4 AI 真正擅长的任务

AI 擅长的不是“简单任务”，而是可描述、可搜索、可验证的任务。

第一类是模式迁移。

比如你已经有三个类似组件，现在要按同样模式加第四个；已经有两个接口用了同样鉴权方式，现在要给第三个接口补上；已有测试文件里有正常路径，现在要补一个边界路径。AI 在这种任务上表现通常不错，因为它可以从已有代码里提取模式。

提示可以这样写：

示例 4-1 AI 真正擅长的任务

```
请参考 `src/features/profile/EditProfileForm.tsx` 的实现方式，
在 `src/features/account/AccountSettingsForm.tsx` 增加同类校验。

要求：
- 只复用现有校验工具，不新增依赖。
- 补充对应测试。
- 跑相关测试命令。
```

这里的关键是“参考已有模式”，不是让它自由设计。

第二类是测试补充。

AI 很适合根据已有测试风格补测试，尤其是边界条件、异常路径、回归用例。前提是你要告诉它目标行为，而不是只说“提高覆盖率”。

示例 4-2 AI 真正擅长的任务

```
请补一个回归测试：
- 场景：订单已经 paid 时，不允许回退到 pending。
- 期望：返回业务错误，不修改订单状态。
- 参考测试文件：`test/order-status.test.ts`。

先只写失败测试，不改实现。
```

“先只写失败测试”很重要。它把 AI 从“写一堆能过的测试”拉回到“先锁住真实 bug”。

第三类是局部 bug 修复。

如果 bug 有复现步骤、有错误输出、有相关文件路径，AI 可以很快定位并修复。好的输入应该包含：

示例 4-3 AI 真正擅长的任务

```
现象：
- 用户在 session 过期后重新登录，第一次提交表单失败。

复现：
- 登录后等待 token 过期。
- 刷新页面。
- 提交表单。

已知线索：
- 错误日志：`401 refresh token missing`
- 相关目录：`src/auth/`
- 测试：`test/auth-refresh.test.ts`

要求：
- 先读相关文件并给出计划。
- 不改登录接口协议。
- 用失败测试锁定问题后再改实现。
```

这种任务很适合 AI，因为它有明确的症状和验证路径。

第四类是机械性同步。

例如类型定义、文档、测试夹具、配置样例需要跟随一个小变更同步。AI 做这种事比人更不容易漏文件，但前提是改动范围可枚举。

示例 4-4 AI 真正擅长的任务

```
字段 `displayName` 已加入 `UserProfile`。
请同步：
- TypeScript 类型
```

```
- API fixture
- 对应展示测试
- README 中的字段表

不要修改接口语义。
完成后输出 diff 摘要。
```

这类任务要避免一个坑：同步不是重构。如果 Agent 借机整理结构，要停。

4.5 AI 不擅长独立负责的任务

第一类是目标模糊的任务。

“优化体验”“整理架构”“让代码更优雅”“修一下偶发问题”，这些话对人也不够，对 AI 更不够。Agent 会把模糊目标转成它能执行的局部动作，例如改命名、抽函数、加重试、吞异常。结果可能看起来更整洁，但不一定解决问题。

如果目标模糊，先让 AI 做需求澄清：

示例 4-5 AI 不擅长独立负责的任务

```
先不要改代码。

请把这个需求拆成需要我确认的问题：
- 用户可观察行为是什么？
- 哪些行为不能改变？
- 完成后怎么证明？
- 哪些信息缺失会影响方案？
```

第二类是领域规则重的任务。

AI 可以读代码，但它不知道你们公司为什么当年留下一个奇怪的分支。很多业务规则不是从代码表面能推出来的。

例如优惠券叠加、风控拦截、支付状态回退、商家结算周期，代码里可能有历史兼容、临时政策、灰度逻辑、人工运营规则。AI 如果只按“更合理”的工程结构来改，可能会删掉业务上必须保留的脏逻辑。

这类任务要先把业务合同写清楚：

清单 4-6 AI 不擅长独立负责的任务

```
## 业务规则

- paid 订单不能回退到 pending。
- refunding 状态可以进入 refunded，但不能进入 canceled。
- 历史订单缺少 `paymentProvider` 时，按 legacy provider 处理。
- 不改变旧接口返回码。
```

```
## 人工确认

- 退款状态流转由业务负责人确认。
- 历史订单兼容规则不能由 AI 推断。
```

第三类是大范围重构。

AI 很容易把重构做成“能跑但难审”的大 diff。大范围重构不是不能用 AI，而是不能让它一口气做完。要把任务拆成可验证的机械步骤：

```
这次只做第一步：
- 把 `src/payment/legacy.ts` 中的类型定义移动到 `src/payment/types.ts`。
- 不改变运行逻辑。
- 跑现有测试。
- 输出 diff 摘要后停止。
```

重构任务里，AI 最适合做可机械验证的步骤；抽象边界和迁移顺序要人控制。

第四类是线上事故。

事故现场最怕“看起来合理的改动”。AI 可以帮你整理日志、列假设、写查询、生成回滚检查项，但不要让它直接改生产路径。事故里，优先级是止血、定位、回滚、记录，不是让代码更优雅。

可以这样用：

示例 4-7 AI 不擅长独立负责的任务

```
请只做分析，不要写修复代码。

输入：
- 事故时间线
- 错误日志
- 最近 3 个相关提交

输出：
- 已确认事实
- 可能原因，按证据强弱排序
- 下一步最小验证动作
- 需要人工确认的风险
```

这种用法能提高排障效率，但不把控制权交出去。

4.6 用一张任务卡决定授权级别

给 Agent 开始任务前，可以先写一张很短的任务卡。

清单 4-8 用一张任务卡决定授权级别

```
# Task Risk Card

## Objective

修复用户 session 过期后第一次提交表单失败的问题。

## Scope

- 可能相关：`src/auth/`、`test/auth-refresh.test.ts`
- 不允许修改：登录接口协议、token 存储格式

## Risk

- 涉及登录状态，但不涉及权限模型变更。
- 不允许吞掉 401 错误。

## Evidence

- 复现日志：`401 refresh token missing`
- 预期验证：`npm test -- auth-refresh`

## Agent Mode

- 先读文件和出计划。
- 人确认计划后再改代码。
- 如果需要扩大范围，先停止。
```

配套代码仓库 DavidDingXu/learn-ai-coding〔4-1〕 的工具 module 里有一个命令可以生成任务上下文包。它不会替你做风险判断，但能把任务信息固定下来：

清单 4-9 用一张任务卡决定授权级别

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js task-context \
  --title "修复 session 过期后首次提交失败" \
  --files "src/auth/,test/auth-refresh.test.ts"
```

生成出来的上下文包，可以再手动补上 scope、risk、evidence 和 Agent mode。这里不要追求模板漂亮。真正有用的是把“能改什么、不能改什么、怎么证明”写清楚。

4.7 五个真实工作里的判断例子

例子一：补一个空状态页面。

需求是“列表为空时显示空状态文案和按钮”，已有同类页面，设计稿明确，有前端测试或截图验证。这个任务可以 AI 主导。你要给它参考组件、目标文件和截图验证要求。

示例 4-10 五个真实工作里的判断例子

```
参考 `src/pages/order/EmptyState.tsx` 的模式，
给 `src/pages/invoice/ListPage.tsx` 增加空状态。

要求：
- 不改列表查询逻辑。
- 不新增组件库依赖。
- 跑相关前端测试，必要时提供截图对比。
```

例子二：修一个接口字段映射错误。

后端返回 `display_name`，前端类型写成 `displayName`，页面显示为空。相关文件明确，有 fixture 和测试。这个任务可以 AI 主导，或者 AI 实现、人快速审查。重点是别让它改接口协议。

```
修复字段映射，不改变后端接口协议。
请先定位 mapping 层和测试，再给最小 diff。
```

例子三：订单状态偶发回退。

这类任务代码可能很少，但属于状态机和业务规则。不要让 AI 自己判断。先让它整理状态流转、历史测试、日志证据，再由人确认规则。

示例 4-11 五个真实工作里的判断例子

```
先不要改代码。
请整理订单状态流转：
- 当前代码允许哪些状态迁移？
- 哪些测试覆盖了 paid/refunding/refunded？
- 日志里回退发生在哪个入口？
- 你认为需要业务确认的问题有哪些？
```

例子四：把一个模块迁移到新框架。

这是大范围任务。AI 可以参与，但必须拆成阶段。先迁移测试夹具，再迁移只读路径，再迁移写路径，最后清理旧代码。每一步都要有验证命令和回滚边界。

这类任务如果直接说“帮我迁移”，很容易得到一个难审的大 diff。

例子五：线上登录大面积失败。

这里 AI 只能辅助。让它读日志、比较最近提交、列假设、写临时查询都可以。但是否回滚、是否开关降级、是否改鉴权逻辑，必须由人控制。

```
请基于下面日志只做排障分析，不改代码。
按证据强弱列出假设，并给每个假设的最小验证命令。
```

这五个例子背后的判断很一致：AI 的授权级别跟代码量无关，跟风险、上下文和验证有关。

4.8 一个简单评分法

如果任务很多，可以用 8 分法快速分级。每个维度 0-2 分：

表 4-2 一个简单评分法

维度	0 分	1 分	2 分
目标	模糊	大致清楚	行为明确
范围	跨系统	多文件但可界定	单模块或同类模式
验证	无法本地验证	部分可验证	有明确命令或截图
风险	关键业务/安全/数据	普通业务逻辑	文档、测试、局部展示

得分不是越高越好，而是用来决定授权：

表 4-3 一个简单评分法

分数	建议模式
7-8	AI 主导，人审 diff 和证据
5-6	AI 实现，人先确认计划
3-4	人主导方案，AI 做侦察和局部执行
0-2	暂不授权实现，先澄清需求或人工处理

这个评分法不需要严格。它的价值是让你在任务开始前停 30 秒，避免把高风险任务当成普通代码生成。

4.9 任务分级后的提示写法

不同授权级别，对 Agent 的提示也不一样。

AI 主导型：

```
请直接完成这个小改动：
- 目标：补充 invoice 列表空状态。
- 参考：`src/pages/order/EmptyState.tsx`
- 约束：不改查询逻辑，不新增依赖。
- 完成判断：相关测试通过，并输出 diff 摘要。
```

AI 实现、人控计划型：

示例 4-12 任务分级后的提示写法

```
先不要改代码。
请读取相关文件并给出最小实现计划：
- 预计改哪些文件。
- 每个文件为什么要改。
- 需要跑哪些验证命令。
- 哪些风险需要我确认。
```

人主导、AI 辅助型：

示例 4-13 任务分级后的提示写法

```
请只做分析和方案候选，不要编辑文件。
输出：
- 已确认事实。
- 缺失上下文。
- 方案 A/B 的影响范围。
- 每个方案的验证方式和风险。
```

高风险辅助型：

```
请只基于提供的日志和提交记录做排障分析。
不要给出生产执行命令。
不要修改代码。
按证据强弱排序假设，并列出需要人工确认的点。
```

把提示写成这样，Agent 就不容易越权。不是因为它“更听话”，而是因为你明确了它当前的角色。

4.10 改完怎么确认不是误用 AI

任务完成后，至少看四个结果。

第一，看 diff 范围。

清单 4-14 改完怎么确认不是误用 AI

```
git diff --stat
git diff
```

如果任务是局部 bug，但 diff 涉及公共基础库、锁文件、多个无关模块，要回到计划重新判断。

第二，看验证命令。

清单 4-15 改完怎么确认不是误用 AI

```
npm test -- auth-refresh
npm run lint
```

命令要和任务风险匹配。文档改动不一定要跑全量测试，状态机改动只跑格式化肯定不够。

第三，看失败分类。

如果测试失败，先确认是改前已有、环境问题、本次引入，还是需求冲突。不要让 Agent 在没分类前继续改。

第四，看剩余风险。

完成报告里应该有“未覆盖什么”。比如：

清单 4-16 改完怎么确认不是误用 AI

```
未覆盖：
- 没有连接真实支付网关。
- 没有跑浏览器兼容性测试。
- 没有验证历史订单数据。
```

这几行不是为了承认失败，而是让人知道下一步该看哪里。

4.11 常见误区

误区一：AI 适合做简单活，不适合做难活。

这个说法太粗。AI 适合做边界清楚、反馈明确的活。难任务如果能拆成小步，也可以让 AI 做很多；小任务如果涉及核心业务，也不能随便放手。

误区二：测试多就可以放心交给 AI。

测试多只是降低风险，不会自动消除风险。你还要看测试覆盖的是不是本次改动路径，Agent 有没有为了通过测试修改测试本身，是否存在未覆盖的外部系统。

误区三：让 AI 自己判断风险就行。

AI 可以帮你列风险，但不能替你承担责任。业务优先级、线上影响、数据后果、用户承诺，这些判断必须由人来定。

误区四：复杂任务只要 prompt 写得长就能解决。

长 prompt 可能只是把噪音变多。复杂任务需要的是规格、计划、小 diff 和验证，不是把所有背景塞进一段话。

误区五：AI 不适合重构。

AI 可以做重构，但重构必须被拆成机械步骤。先移动类型、再替换调用、再删除旧路径，每一步都能验证。让 AI 一次性“重构得更优雅”，风险很高。

4.12 可以先改的一件事

下次你准备把任务交给 AI 前，先写四行：

清单 4-17 可以先改的一件事

```
目标：
范围：
验证：
风险：
```

如果这四行写不出来，就不要直接实现。先让 AI 帮你澄清。

如果这四行写得很清楚，就继续判断授权级别：AI 主导、AI 实现人控计划、人主导 AI 辅助，还是只让 AI 做分析。

AI Coding 的效率，不来自把更多任务扔给 AI，而来自把合适的任务以合适的授权级别交出去。

接下来进入工具工作台。工具很多，先解决一个问题：在不同任务分级下，应该用 App、CLI、IDE、补全、浏览器和 Git 形成怎样的工作面。

4.13 常见问题

任务评分一定要写成文档吗？

不用。

低风险任务可以在脑子里快速过一遍。只要涉及跨模块、状态、权限、金额、数据迁移、线上问题，就建议写成短任务卡。任务卡不需要漂亮，能说明目标、范围、验证和授权级别就够。

AI 判断任务风险靠谱吗？

可以让 AI 辅助列风险，但不能让它最终拍板。

AI 能根据代码、测试和文档发现技术风险，却不一定知道业务承诺、上线窗口、历史兼容、用户影响和组织决策。风险判断可以让它先列证据，再由人确认。

小任务是不是都可以交给 AI 主导？

不一定。

任务小不等于风险低。两行金额计算、一次权限判断、一个状态迁移条件，都可能影响生产。判断授权级别时，看错误后果和验证信号，不看代码行数。

没有测试的任务是不是不能用 AI？

可以用，但要降低授权级别。

没有测试时，可以让 AI 先整理复现步骤、生成手工验证清单、写最小测试候选、查调用链。等验证路径建立起来，再让它实现。不能让它在没有证据的情况下自己宣布完成。

重构类任务怎么用 AI 才合适?

先把重构拆成机械步骤。

例如先移动类型，再替换引用，再删除旧入口，每一步都跑测试。不要让 AI 一次性“整理架构”。重构最怕行为改动混进结构调整里，拆小步是为了让 reviewer 能看懂每一轮变化。

4.14 可复查资料

- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔4-2〕
- Anthropic Claude Code 最佳实践：https://www.anthropic.com/engineering/claude-code-best-practices〔4-3〕
- GitHub Copilot cloud agent 任务最佳实践：https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results〔4-4〕

本章资源

〔4-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔4-2〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔4-3〕 https://www.anthropic.com/engineering/claude-code-best-practices
https://www.anthropic.com/engineering/claude-code-best-practices

〔4-4〕 https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results
https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results

PART 03

第3篇　组织自己的工具工作台

工具界面服务于开发任务。比较 App、CLI、IDE 的分工，把代码、终端、测试和审查放进同一个工作过程。

阅读重点

能为一次真实改动选择合适的工具入口。

第5章 Codex、Claude Code、Cursor、Copilot 怎么分工

程序员不需要每天换一个 AI Coding 工具。

真正要解决的问题是：一个真实需求先在哪个工作面推进，什么时候才有必要切到另一个工具。

工具选错，AI 能力再强也会难用。

很多人的 AI Coding 工作流卡在一个误区：把 Codex、Claude Code、Cursor、Copilot 当成同一类工具比较。于是问题变成“哪个更强”。这个问题太粗。

先按真实开发动作看边界，不要先按工具名排座次：

- App 类工作面：适合聊需求、拆任务、看结果、管长任务、做文件预览、异步审查和材料整理。
- CLI 类工作面：适合把 Agent 放进真实仓库，读文件、改文件、跑命令、留下可复查输出。
- IDE / AI 编辑器工作面：适合贴近代码上下文做阅读、跳转、diff 审查、断点 debug、局部编辑和交互式修改。
- 补全：适合当前文件内的低风险连续编码。
- 后台 Agent：适合清楚、范围小、可验证、可以异步处理的任务。

App 和 CLI 不是两套能力。只要 CLI Agent 能读仓库、调用工具、运行命令，规划、解释、审查、总结这些动作它也能做。差别更多在交互形态：App 更适合看结果、管理长任务、预览文件或图片、处理异步协作；CLI 更贴近 repo、终端、Git、测试和本地脚本。

一个工具可能覆盖多个工作面。比如 Codex 既有 App，也有 CLI；Claude Code 是终端里的 agentic coding 工具；Cursor 是带 AI 能力的编辑器；GitHub Copilot 同时覆盖 IDE 补全、Chat、review、coding agent 等能力。真正要学的不是工具名，而是把任务放到能产生证据、也符合自己编码习惯的工作面。



图 5-1　AI Coding 工具工作面分工

5.1 不要用一个工具承包所有动作

先看一个常见但低效的流程：

```
1. 在 IDE 里问 AI：帮我实现这个需求。
2. AI 改了几个文件。
3. 测试失败，继续让它修。
4. 发现需求理解错了，回到聊天窗口解释一遍。
5. diff 越来越大，最后自己手动收拾。
```

问题不一定是 IDE 工具不好，而是工作面混在一起了。

需求澄清、代码侦察、实现、测试、调试、审查，本来就是不同性质的动作。把它们全部塞到一个聊天窗口或一个编辑器侧边栏里，信息很容易乱。

更接近真实日常的做法是：先选一个主工作面，把需求、实现和验证尽量在一条线上跑完；只有出现证据、审查或调试缺口时才切换。主工作面不固定，取决于它能不能看到真实仓库、执行必要动作、留下证据。

```
一种可替换组合：App / CLI 类工作面聊需求、拆任务、跑工作流、开发验证
另一种可替换组合：IDE / AI 编辑器作为主工作面，终端补验证和 Git 证据
审查调试：IDE 读调用链、看 diff、断点 debug、做局部修正
界面行为：浏览器或 Playwright 复现用户路径
最终收口：回到 Git diff、测试输出和残余风险
```

这不是标准答案。有人习惯从 CLI 开始，有人习惯从 IDE 开始，有人会先在 App 里把需求聊清楚。关键不是入口，而是这条链路最后能不能回答：读了哪些文件、改了哪些 diff、跑了哪些验证、还有哪些风险。

5.2 App 什么时候值得用

App 不是代码改动的必经入口。CLI 能做的计划、解释、审查，App 通常也能做；App 值得打开，通常是因为它的界面更适合看和管理结果。

例如：

- 管理多个异步任务。
- 预览文件、图片、报告和审查结果。
- 对比几种方案的取舍。
- 把一段调试记录整理成复盘。
- 整理文章、规则、任务卡或上下文包。
- 做独立审查，不直接碰当前 worktree。

App 的优势不是它比 CLI 更懂工程，而是展示和协作体验更好。它的缺点也很直接：如果拿不到真实工作区、命令输出和 Git diff，它就只能基于你给的材料判断。你如果只在 App 里粘几段代码，让它判断整个项目，风险很高。

适合 App 的提示通常是这样的：

示例 5-1 App 什么时候值得用

```
我准备在真实项目里做这个改动。
先不要写代码，请帮我判断任务风险：

需求：
用户 session 过期后，重新登录第一次提交表单失败。

已知上下文：
- 前端 React
- auth 逻辑在 src/auth/
- 已有 auth-refresh 测试

请输出：
- 需要确认的问题
- 可能涉及的模块
- 不适合直接交给 Agent 改的部分
- 建议的第一步验证动作
```

这类工作不需要立刻编辑文件。App 可以帮你把风险和边界整理清楚，再把结果带回仓库。需求已经清楚时，完全可以直接在 CLI 里完成这一步。

但不要在 App 里做这种事：

```
我把几个文件贴给你，你直接给我完整 diff。
```

除非任务很小，否则这会让 diff 脱离真实仓库。真实项目里的 import、类型、测试、格式化、锁文件、生成文件、路径别名，App 可能都看不见。

5.3 CLI 适合做什么

CLI 是 AI Coding 里最接近真实工程执行面的工作面。

它能在当前仓库里读文件、搜索、编辑、跑命令、看 Git 状态。Codex CLI 和 Claude Code 这类工具的核心价值在于：Agent 可以进入你的项目目录，执行一组可观察动作。

CLI 适合：

- 接手陌生模块。
- 做代码搜索和上下文侦察。
- 执行小步实现。

- 跑测试、lint、build。
- 根据命令输出排障。
- 整理 diff 和完成报告。
- 把规则、模板、验证记录写回仓库。

例如：

清单 5-2 CLI 适合做什么

```
rg -n "refresh token|auth-refresh|session expired" src test
git status --short
npm test -- auth-refresh
```

这些命令的价值不是“有命令行感觉”，而是让 Agent 的判断有证据。CLI 输出能告诉你它读到了什么、改了什么、跑了什么。

CLI 的弱点也很明显：它不一定适合精细 UI 调整，不一定适合设计稿对齐，不一定适合你边跳转边看多个文件的编辑习惯。此时可以把 CLI 用作仓库侦察、工作流执行和验证证据面，把 IDE 用作代码审查、debug 和局部修正面。反过来也成立：如果 IDE 能可靠读仓库、改多文件、跑命令并返回输出，它也可以成为主工作面。

CLI 提示可以这样写：

示例 5-3 CLI 适合做什么

```
先不要改代码。

请在当前仓库中完成代码侦察：
- 用搜索找到相关入口、实现、测试。
- 列出实际读取过的文件。
- 给出最小实现计划。
- 说明建议运行的验证命令。

未经确认不要编辑文件。
```

这能让 CLI Agent 先建立证据，再进入实现。

5.4 IDE 适合做什么

Cursor、VS Code 里的 Copilot Chat、JetBrains 里的 AI 能力，本质上都把 AI 放到了编辑器工作面。

IDE 的优势是贴近代码。你可以选中一段函数，让 AI 改；可以在当前文件里补测试；可以一边看类型报错一边调整；可以利用编辑器的跳转、诊断、Git 面板、调试器。

IDE 适合：

- 读调用链和定位真实实现入口。

- 审查 Agent 改出来的 diff。
- 断点 debug 或观察运行时状态。
- 当前文件或相邻文件的小改。
- 根据已有模式补代码。
- 修类型报错和 lint。
- 补单元测试。
- 改 UI 细节。
- 看 diff 时做小修。
- 在人工主导方案下执行局部代码修改。

例如：

```
只修改当前选中的函数。
目标：当 response.status 为 401 且 refresh token 存在时，先刷新 token 再重试一次。
约束：不改变外部接口，不新增依赖，不吞掉非 401 错误。
```

IDE 的危险在于太顺手。你选中一段代码让 AI 改，很容易忘记它没看完整调用链；你在侧边栏聊了很久，也很容易忘记当前 diff 已经扩大。

所以 IDE 工作面要配两条规则：

```
1. 选区改动只允许影响选区和必要相邻测试。
2. 任何跨文件、跨模块、改接口的建议，先回到计划阶段。
```

如果一个 IDE Agent 提议“顺便重构一下 auth 管理”，这已经不是当前文件编辑了。应该停下来，让它把方案写出来，再回到主工作面的任务计划或规格流程里判断。

5.5 补全适合做什么

补全是最容易被低估，也最容易被误用的 AI Coding 能力。

它不适合替你做业务判断，但非常适合在你已经知道要写什么的时候加速输入。

适合补全的场景：

- 重复字段映射。
- 枚举分支补齐。
- 测试断言补全。
- 类型声明。
- 小工具函数。
- 注释到代码的直接转换。

- 按已有模式写相邻代码。

补全的前提是你在驾驶位。你知道下一行应该是什么，AI 只是帮你少敲。

不适合补全的场景：

- 你还没想清业务规则。
- 你不知道该改哪个文件。
- 你不确定这个分支是否应该存在。
- 你在写安全、鉴权、金额、迁移逻辑。

很多线上问题不是因为 AI Agent 大规模改错，而是因为补全悄悄补了一段“看起来合理”的逻辑，人顺手接受了。补全越丝滑，越要保持怀疑。

一个实用做法是：高风险代码先关掉自动接受习惯，只保留手动触发。你可以用补全写重复代码，但不要让它替你决定业务语义。

5.6 后台 Agent 适合做什么

后台 Agent 的吸引力很明显：你开一个任务，它自己拉分支、改代码、跑检查、给结果。

但它适合的任务其实更窄。

适合后台 Agent 的任务通常有这些特征：

- Issue 描述清楚。
- 完成条件明确。
- 影响范围可预计。
- 有测试或检查命令。
- 不需要频繁人机来回确认。
- 失败后可以安全回滚或重新生成。

例如：

清单 5-4 后台 Agent 适合做什么

```
## Task

Add empty state to invoice list page.

## Scope

- `src/pages/invoice/ListPage.tsx`
- related tests only
```

```
## Done when

- empty invoice response shows existing `EmptyState` component
- no query behavior changes
- related tests pass
```

这种任务适合让后台 Agent 处理。

不适合后台 Agent 的任务：

- 线上事故。
- 需求还在变。
- 需要业务现场确认。
- 涉及权限、金额、隐私、数据修复。
- 需要频繁看 UI 和调细节。
- 测试缺失，完成条件只能靠人判断。

后台 Agent 不是“更强的自动驾驶”，而是异步执行器。你给它的任务越像清晰 issue，它越有用；你给它的任务越像“帮我想办法”，它越容易跑偏。

5.7 一张主工作面判断表

下面这张表不是标准答案，只是判断入口的辅助。你可以按自己的习惯替换工具，只要最后能留下证据。

表 5-1 一张主工作面判断表

任务状态	首选工作面	原因	注意点
需求模糊	App / CLI	先澄清问题、拆目标、列风险	不要直接给完整 diff
陌生模块	CLI	能搜索、读文件、跑命令、留证据	先侦察，后实现
当前文件小改	IDE	贴近代码上下文，编辑效率高	限制选区和相邻文件
重复代码/断言	补全	人在驾驶位，AI 加速输入	不接受业务判断型补全
中等风险实现	能访问仓库并留下证据的主工作面 + IDE/终端补位	主工作面做侦察、计划、实现和验证；IDE 做审查、debug 或局部修正	每步看 diff
UI 交互验证	IDE + 浏览器	编辑和视觉反馈近	截图或录屏留证据
清晰异步任务	后台 Agent	可以独立执行、提交结果	issue 要写清完成条件
线上事故/高风险	App / CLI 辅助	分析证据和排查路径	人控制执行

这张表不是要求你每次切很多工具，而是帮你决定主工作面。

低风险任务可以一直在 IDE 里做。中等风险任务要先选一个能读仓库、改文件、跑命令、留下 diff 和验证输出的主工作面；这个主工作面可以是 CLI，也可以是具备同等能力的 IDE 或 App。高风险任务里，AI 更适合先做证据整理、方案对比和风险枚举，关键业务决策仍然由人确认。

5.8 一个真实需求怎么组合工具

假设需求是：

```
用户 session 过期后重新登录，第一次提交表单失败。
```

第一步，在 App 或 CLI 当前对话里做风险判断：

示例 5-5 一个真实需求怎么组合工具

```
请判断这个任务适合哪种 AI 授权级别：
- AI 主导
- AI 实现，人控计划
- 人主导，AI 辅助
- 只做分析

请说明理由，并列出缺失上下文。
```

这个任务涉及登录状态，但不一定涉及权限模型变化。大概率属于“AI 实现，人控计划”。

第二步，切到 CLI 做上下文侦察：

```
先不要改代码。
请搜索 session 过期、refresh token、表单提交相关入口。
列出实际读取文件，并给最小实现计划。
```

CLI 可以跑：

清单 5-6 一个真实需求怎么组合工具

```
rg -n "refresh token|session expired|401|submit" src test
git status --short
```

第三步，人确认计划后，让 CLI 或 IDE 做第一小步。

如果只是补一个失败测试，CLI 很适合。如果需要在一个组件里调局部状态，IDE 更顺手。

第四步，验证回到 CLI：

清单 5-7 一个真实需求怎么组合工具

```
npm test -- auth-refresh
npm run lint
git diff --stat
```

第五步，必要时用浏览器验证真实交互。

如果这个 bug 和页面状态有关，只看测试不够。需要打开本地页面，复现过期登录后的提交路径，记录截图或步骤。

第六步，把完成证据整理成验证记录。下面用配套代码仓库 DavidDingXu/learn-ai-coding〔5-1〕 里的 verification ledger 样例说明格式：

清单 5-8 一个真实需求怎么组合工具

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
```

实际项目里可以照着这个格式写自己的 verification ledger。

这条路径不要求工具越多越好。它要求每一步有明确目的：规划归规划，编辑归编辑，验证归验证。

5.9 Codex、Claude Code、Cursor、Copilot 的实际分工

这里不做“谁更强”的排名。工具变化很快，排名很容易过期。先看任务需要哪个工作面。

Codex 适合承担仓库级 Agent 工作：理解任务、读文件、改文件、跑命令、报告验证。Codex CLI 适合本地仓库内的小步实现和验证；Codex 云端/网页工作面适合异步任务、PR/review、材料整理、文件预览和与 GitHub 工作流结合的场景。具体能力要以官方文档和当前产品为准。

Claude Code 适合终端里的 agentic coding，尤其是你希望 Agent 在仓库中探索、计划、编码、测试，并且能结合 `CLAUDE.md`、hooks、skills、MCP、subagents 等机制时。它的工程化能力很强，但也更需要你给清楚规则和边界。

Cursor 适合以编辑器为中心的工作。它的价值不是替代 CLI，而是在你读代码、跳转、选区编辑、局部重构、UI 调整时，把 AI 放到最贴近代码的位置。对程序员来说，Cursor 更像一个 AI-native IDE 工作面。

GitHub Copilot 适合 IDE 内补全、Chat、PR review、GitHub 任务上下文和 cloud agent 等场景。它和 GitHub issue、pull request、repository instructions 的结合很自然，适合把清晰任务放进 GitHub 工作流。

这些分工可以组合。

例如：

```
Codex/Claude Code：仓库侦察、计划、小步实现、命令验证
Cursor/Copilot Chat：局部编辑、跳转、选区修改、UI 调整
Copilot completion：当前文件补全
GitHub/Codex cloud agent：清晰异步 issue
App：结果预览、异步任务管理、方案审查、复盘和材料整理
```

不要因为一个工具能做很多事，就让它做所有事。

5.10 工具组合的反例

反例一：在 IDE 里直接做复杂需求。

```
帮我把支付流程重构一下，顺便修复偶发支付失败。
```

这个任务应该先变成规格、计划和验证路径。IDE 只适合执行其中的局部步骤。

反例二：把 UI 细节全交给 CLI。

```
帮我把这个页面调得更舒服一点。
```

CLI 可以改代码，但它不一定看得到真实视觉效果。UI 任务需要浏览器截图、设计稿、交互状态。否则 Agent 很容易按代码想象页面。

反例三：让后台 Agent 处理模糊 issue。

```
用户说登录体验不好，优化一下。
```

这种 issue 不适合异步 Agent。它应该先拆成可观察行为，例如“session 过期后不丢表单输入”“刷新 token 失败时显示重新登录提示”。

反例四：用补全写业务规则。

当你写到：

清单 5-9 工具组合的反例

```
if (order.status === 'paid') {
```

补全可能会给出一个看起来合理的分支。但 paid 订单能不能取消、能不能退款、能不能回退，不是补全该决定的事。

5.11 给自己的最小工具工作台

不需要一次装满所有工具。一个程序员的最小工作台，可以先是这样：

```
1. 一个仓库级 Agent：Codex CLI 或 Claude Code
2. 一个 AI 编辑器工作面：Cursor 或带 Copilot 的 IDE
3. 一个补全工具：Copilot 或编辑器内置补全
4. 一个浏览器验证方式：本地浏览器、Playwright 或浏览器插件
5. 一个 Git 和测试命令面：终端
```

再配三类仓库资产：

```
AGENTS.md / CLAUDE.md / repository instructions
任务上下文包
验证记录
```

这就够了。

工具多不是能力。先把一条路径跑顺：任务分级、上下文侦察、计划、小 diff、验证、审查。其他工具只在主工作面不够用时介入。

5.12 什么时候该切换工具

切换工具不是为了仪式感，而是为了解决当前工作面的短板。

遇到这些情况，应该从 App 切到 CLI：

- 需要读真实仓库文件。
- 需要跑命令验证。
- 需要查看 Git diff。
- 需要生成可落盘产物。

遇到这些情况，应该从 CLI 切到 IDE：

- 需要频繁跳转和人工阅读。
- 需要精细改当前文件。
- 需要借助类型提示、调试器或 UI 插件。
- 需要你亲自掌控局部代码。

遇到这些情况，应该从 IDE 切回 CLI：

- 改动跨多个文件。
- 需要跑测试和整理输出。
- 需要检查 diff 范围。
- 需要把结果写入任务记录。

遇到这些情况，可以考虑后台 Agent：

- 任务已经写成清晰 issue。
- 完成条件明确。
- 失败不会影响现场。
- 你可以稍后 review PR 或 diff。

遇到这些情况，不要切给后台 Agent：

- 需求还没定。
- 线上正在出问题。
- 需要人工连续判断。
- 涉及核心权限、数据和金额。

5.13 常见问题

只用一个工具行不行？

可以。低风险任务完全可以只用一个工具。但随着任务进入真实项目，规划、编辑、验证和审查的需求会分化。工具组合不是为了复杂，而是为了让每个动作在合适的位置发生。

Codex 和 Claude Code 都是 CLI Agent，怎么选？

看你现有环境、模型偏好、团队约束和工作流资产。对个人来说，更重要的是把项目规则、上下文包和验证命令跑顺。工具可以替换，工程证据不能省。

Cursor 和 Copilot 会不会和 CLI Agent 冲突？

不会。IDE 工具适合局部编辑和当前文件上下文，CLI Agent 适合仓库级计划和验证。冲突通常来自人没有划边界，让多个工具同时改同一批文件。

后台 Agent 能不能替我写完整需求？

可以处理清晰、范围小、可验证的需求。复杂需求要先拆。后台 Agent 最怕模糊任务和频繁变化的上下文。

工具越多，成本会不会越高？

会。工具太多会带来上下文切换、规则重复和结果合并成本。最小工作台先跑通，再按真实痛点补工具。

5.14 可以先建立的个人路由规则

把下面这段放进自己的工作笔记就够用：

清单 5-10 可以先建立的个人路由规则

```
# AI Coding Tool Routing

## App

Use for previews, async task management, review reading, recap, and material organization.

## CLI Agent

Use for repository exploration, small diffs, tests, debugging, and verification records.

## IDE Agent

Use for selected-code edits, local refactors, UI adjustments, and type-driven fixes.

## Completion

Use only when I already know the intended logic.

## Background Agent

Use only for clear, scoped, verifiable async tasks.
```

后面再把它细化到自己的项目，例如哪些命令必须跑，哪些目录不能改，哪些任务必须先写 spec。

工具工作台搭好以后，下一个问题会自然出现：你的本地环境、终端、浏览器、Git、测试命令和项目规则，怎么组织成一条能反复使用的开发路径。否则工具再多，也只是分散的入口。

5.15 可复查资料

- OpenAI Codex 官方文档：https://developers.openai.com/codex/〔5-2〕
- OpenAI Codex CLI：https://developers.openai.com/codex/cli〔5-3〕
- OpenAI Codex cloud：https://developers.openai.com/codex/cloud〔5-4〕
- Anthropic Claude Code overview：https://code.claude.com/docs/en/overview〔5-5〕
- GitHub Copilot 文档：https://docs.github.com/en/copilot〔5-6〕
- Cursor 官方文档入口：https://cursor.com/docs〔5-7〕

本章资源

〔5-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔5-2〕 https://developers.openai.com/codex/
https://developers.openai.com/codex/

〔5-3〕 https://developers.openai.com/codex/cli
https://developers.openai.com/codex/cli

〔5-4〕 https://developers.openai.com/codex/cloud
https://developers.openai.com/codex/cloud

〔5-5〕 https://code.claude.com/docs/en/overview
https://code.claude.com/docs/en/overview

〔5-6〕 https://docs.github.com/en/copilot
https://docs.github.com/en/copilot

〔5-7〕 https://cursor.com/docs
https://cursor.com/docs

第6章　一个程序员的 AI Coding 工作台怎么搭

AI Coding 工作台不是把所有 AI 工具都装一遍，也不是给 App、CLI、IDE 分配一套固定职责。

更真实的情况是：不同程序员会有不同主工作面。有人先在 App 或 CLI 里聊需求、拆任务、跑工作流、开发验证；有人把 IDE 或 AI 编辑器当主工作面，只在验证和 Git 收口时回到终端。这些都成立。

工作台要解决的不是“哪个工具更正统”，而是让一次真实需求不要断在半路。

有用的工作台，要让你在真实仓库里拿到一个任务后，稳定完成这几件事：

- 找到正确上下文。
- 让 Agent 知道项目规则。
- 把需求拆成可审查的小步骤。
- 在本地跑验证命令。
- 用浏览器或接口确认用户可见行为。
- 看清 Git diff。
- 把完成证据和剩余风险留下来。

如果这些动作接不上，工具再多也只是入口很多。你会在 App、IDE、终端、浏览器之间来回复制信息，最后不知道 AI 到底读了什么、改了什么、验证了什么。

程序员的工作台应该反过来设计：先定义一条自己真的会用的开发路径，再把工具放到路径上。

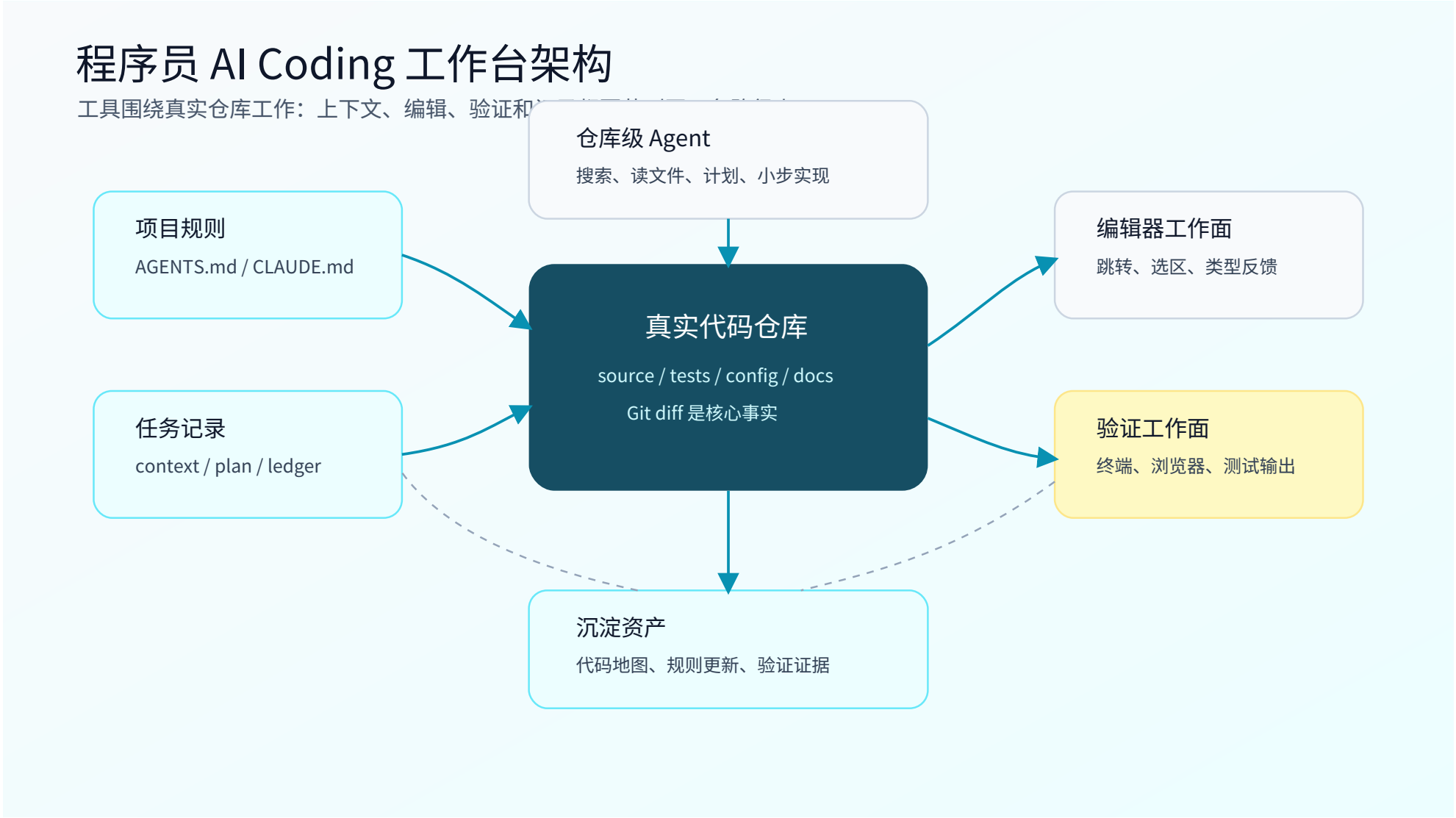


图 6-1　程序员 AI Coding 工作台架构

6.1 工作台先解决三个问题

第一个问题：上下文从哪里来。

AI 不能凭空理解项目。它需要项目规则、代码地图、相关文件、历史问题、测试入口、运行命令。工作台要让这些信息容易被找到，而不是每次靠人临时回忆。

第二个问题：改动在哪里发生。

有些改动适合让仓库级 Agent 沿着任务工作流推进，有些适合在 IDE 里读调用链、打断点、审 diff 后局部改。工作台要让你随时看见 diff，不能让多个工具同时在同一批文件上乱改。

第三个问题：完成怎么证明。

AI 说完成没有意义。工作台要让测试、lint、build、浏览器、接口调用、日志和人工检查点变成可复查证据。

这三个问题一旦清楚，工具选择就简单了。

6.2 最小工作台长什么样

一套够用的个人工作台，可以先从你已经在用的链路开始。

示例 6-1 最小工作台长什么样

```
1. 主工作面：App、CLI、IDE 或 AI 编辑器，选一个真实需求里最常用的入口
2. 代码审查和调试面：IDE / AI 编辑器
3. 终端证据面：Git、测试、构建、脚本、日志
4. 浏览器或接口验证面：页面路径、Network、截图、接口响应
5. 项目规则：AGENTS.md、CLAUDE.md、repository instructions
6. 任务记录：context package、plan、verification ledger
```

这些部分不要求绑定某个厂商，也不要求每次同时使用。你可以把 App / CLI 类 Agent 当主工作面，也可以把 IDE、AI 编辑器或编辑器内 AI 能力当主工作面；其他工具只补它不擅长的部分。

关键是每一部分都要有明确职责。

表 6-1 最小工作台长什么样

组成	职责	不该承担
主工作面	需求讨论、仓库侦察、计划、小步实现、验证收口	在证据不足时直接拍板
IDE / 编辑器	阅读调用链、审查 diff、断点 debug、局部修改、类型反馈	大范围自主重构
补全	当前文件内加速输入	业务规则决策

组成	职责	不该承担
终端	Git、测试、构建、脚本、环境检查	只看 AI 口头结论
浏览器	用户可见行为验证	代替接口和状态检查
项目规则	稳定约束 Agent 行为	塞满一次性背景
任务记录	留下上下文和验证证据	写成形式化报告

工作台不是为了让流程变重，而是为了让每个动作有去处。

6.3 目录结构先建起来

建议先在自己的项目里放三类轻量文件。不是每个仓库都要完全一样，但思路可以复用。

示例 6-2 目录结构先建起来

```
.
├── AGENTS.md
├── docs/
│   ├── ai-coding/
│   │   ├── code-map.md
│   │   ├── task-context-template.md
│   │   └── verification-ledger-template.md
├── scripts/
│   └── ai/
│       ├── test-focused.sh
│       └── check-worktree.sh
└── package.json
```

`AGENTS.md` 放长期规则。比如安装命令、测试命令、禁止修改目录、新增依赖规则、完成报告要求。

`docs/ai-coding/code-map.md` 放项目结构索引。它不用很完整，但要让新对话知道入口、模块、测试、配置、生成文件在哪里。第一版可以由 Agent 或脚本生成，人只复核关键边界。

`task-context-template.md` 用来记录单次任务的目标、范围、相关文件、风险、验证命令。

`verification-ledger-template.md` 用来记录改完之后的命令、输出、证据和未覆盖风险。

`scripts/ai/` 放可以重复运行的小脚本。不要为了 AI 写一堆复杂自动化，先放最常用的检查命令就够了。

如果你不想改现有项目目录，也可以把这些文件放在 `docs/dev/`、`.github/` 或团队已有文档目录里。目录位置不是重点，重点是稳定可找。

6.4 AGENTS.md 只写长期有效规则

很多人写项目规则文件时，会把一次任务的背景也塞进去。这样很快就乱。

`AGENTS.md` 适合写长期有效的工程约束：

清单 6-3 AGENTS.md 只写长期有效规则

```
# Project Instructions

## Commands

- Install: `npm install`
- Focused test: `npm test -- <pattern>`
- Full test: `npm test`
- Lint: `npm run lint`

## Worktree

- Run `git status --short` before editing.
- Do not overwrite user changes.
- Do not edit generated files under `dist/`.

## Change Rules

- Bug fixes must stay scoped to the failing behavior.
- Do not add dependencies without approval.
- Do not mix refactor with behavior changes.
- If implementation requires a wider change, stop and explain.

## Verification

- Report exact commands and results.
- Explain what each command covers.
- List unverified risks before completion.
```

不要写这种规则：

清单 6-4 AGENTS.md 只写长期有效规则

```
- Please be careful.
- Improve code quality as much as possible.
- Use best practices.
- Make the project better.
```

这些话不会给 Agent 提供可执行约束。

长期规则应该来自真实项目经验。比如 Agent 曾经改过生成文件，就写“不要改生成文件”。它曾经为了修测试删除断言，就写“不要删除断言来让测试通过”。它曾经顺手加依赖，就写“新增依赖必须先确认”。

6.5 代码地图不要做成大而全文档

代码地图的目标不是介绍整个系统，而是帮 Agent 和人快速找到入口。

一份有效的代码地图可以很短：

清单 6-5 代码地图不要做成大而全文档

```
# Code Map

## Runtime Entry

- `src/server.ts`: HTTP server bootstrap.
- `src/routes/`: route definitions.
- `src/modules/order/`: order domain logic.

## Tests

- `test/order/`: order behavior tests.
- `test/auth/`: auth refresh and session tests.

## Config

- `package.json`: scripts.
- `vite.config.ts`: frontend build.
- `.env.example`: local env keys.

## Do Not Edit Directly

- `dist/`
- `generated/`
- lock files unless dependency changes are approved.

## Common Commands

- `npm test -- order`
- `npm run lint`
```

可以用配套代码仓库 DavidDingXu/learn-ai-coding〔6-1〕 的工具 module 先生成一版轻量地图：

清单 6-6 代码地图不要做成大而全文档

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js codemap --root .
```

如果要在任务开始前检查风险边界，可以再生成上下文审计报告：

清单 6-7 代码地图不要做成大而全文档

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js context-audit --root .
```

它会列出 Agent 应优先读取的文件、候选入口、测试和验证提示，以及 generated、dist、legacy 这类需要谨慎处理的路径。生成后不要直接当事实，要复核真实运行入口、业务模块、禁止修改目录和 focused verification。

代码地图最重要的是更新频率。它不需要覆盖所有文件，但关键入口变了要改。否则 Agent 会被过期地图带偏。

6.6 任务上下文包把一次需求固定下来

每次中等风险任务开始前，不要只给 Agent 一句话。写一个短上下文包。

清单 6-8 任务上下文包把一次需求固定下来

```
# Task Context

## Objective

修复 session 过期后重新登录，第一次提交表单失败的问题。

## Scope

- 可能相关：`src/auth/`、`src/forms/submit.ts`
- 测试：`test/auth-refresh.test.ts`
- 不允许修改：登录接口协议、token 存储格式

## Evidence

- 错误日志：`401 refresh token missing`
- 复现路径：等待 token 过期，刷新页面，提交表单

## Verification

- `npm test -- auth-refresh`
- 浏览器复现路径手动验证

## Agent Mode

先读文件和出计划。人确认后再实现。
```

这个文件不需要正式。你也可以直接放在 issue、PR 描述或任务评论里。关键是让目标、范围、证据和验证命令同时出现。

配套工程的工具 module 里有生成基础上下文包的命令：

清单 6-9 任务上下文包把一次需求固定下来

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js task-context \
  --title "修复 session 过期后首次提交失败" \
  --files "src/auth/,test/auth-refresh.test.ts"
```

生成出来的内容只是起点。真正的工程判断需要你确认风险和边界。

6.7 终端要成为证据面

AI Coding 里，终端不是附属工具。它是证据面。

至少要把这些命令固定下来：

清单 6-10 终端要成为证据面

```
git status --short
git diff --stat
git diff
npm test
npm run lint
npm run build
```

具体命令按项目替换。Java 项目可能是：

清单 6-11 终端要成为证据面

```
./mvnw test
./mvnw -DskipTests package
```

Go 项目可能是：

清单 6-12 终端要成为证据面

```
go test ./...
go vet ./...
```

前端项目可能是：

清单 6-13 终端要成为证据面

```
npm test -- auth-refresh
npm run typecheck
npm run lint
```

不要只记命令名字，要知道每个命令覆盖什么风险。

`git status --short` 确认工作区有没有已有改动。

`git diff --stat` 看改动范围。

`git diff` 看语义。

focused test 看本次路径。

full test 看共享影响。

typecheck 看类型合同。

lint 看格式和静态规则。

build 看打包路径。

如果 Agent 只说“测试通过”，你要追问：哪个命令，在哪个目录，输出是什么，覆盖什么，没覆盖什么。

6.8 浏览器验证要接到工作流里

前端和交互任务不能只看测试。

浏览器验证至少要覆盖：

- 页面能否打开。
- 关键状态是否正确。
- 用户操作是否触发预期请求。
- 错误状态是否可见。
- 移动端或小屏是否变形。
- 截图中是否有文字重叠、遮挡、错位。

如果你用 Playwright，可以把 UI 验证写成可重复运行的命令：

清单 6-14 浏览器验证要接到工作流里

```
npx playwright test auth-refresh.spec.ts
```

如果只是本地手动验证，也要留下步骤：

清单 6-15 浏览器验证要接到工作流里

```
## Browser Verification

- URL: `http://localhost:3000/settings`
- Steps:
  1. Login.
  2. Wait for token expiration.
  3. Refresh page.
  4. Submit profile form.
- Expected:
  - token refresh happens once
  - form submission succeeds
  - no duplicate error toast
```

不要让浏览器验证变成“我看了一下没问题”。用户可见行为应该能复现。

6.9 Git 是最后的事实来源之一

AI 写完后，最可靠的材料之一是 Git diff。

每次完成前，至少看：

清单 6-16 Git 是最后的事实来源之一

```
git status --short
git diff --stat
git diff
```

如果是多步任务，建议每一步结束后就看一次 diff。不要等整个任务结束再看。

看 diff 时关注五件事：

1. 是否改了计划外文件。
2. 是否混入重构、格式化或无关清理。
3. 是否删除测试或降低断言强度。
4. 是否新增依赖、脚本、配置。
5. 是否改变公共接口或数据结构。

Agent 的完成报告可以参考这个格式：

清单 6-17 Git 是最后的事实来源之一

```
## Change Summary

- Added refresh-token retry before form submit.
- Kept login API contract unchanged.

## Files Changed

- `src/auth/refresh.ts`: add retry guard.
- `test/auth-refresh.test.ts`: add regression case.

## Verification

- `npm test -- auth-refresh`: passed, 6 tests.
- Browser manual path: passed.

## Residual Risk

- Did not test real third-party SSO provider.
```

这比“已修复”有用得多。

6.10 一次改动的反馈路径

工作台搭好后，一次任务应该能沿着下面这条路径走。

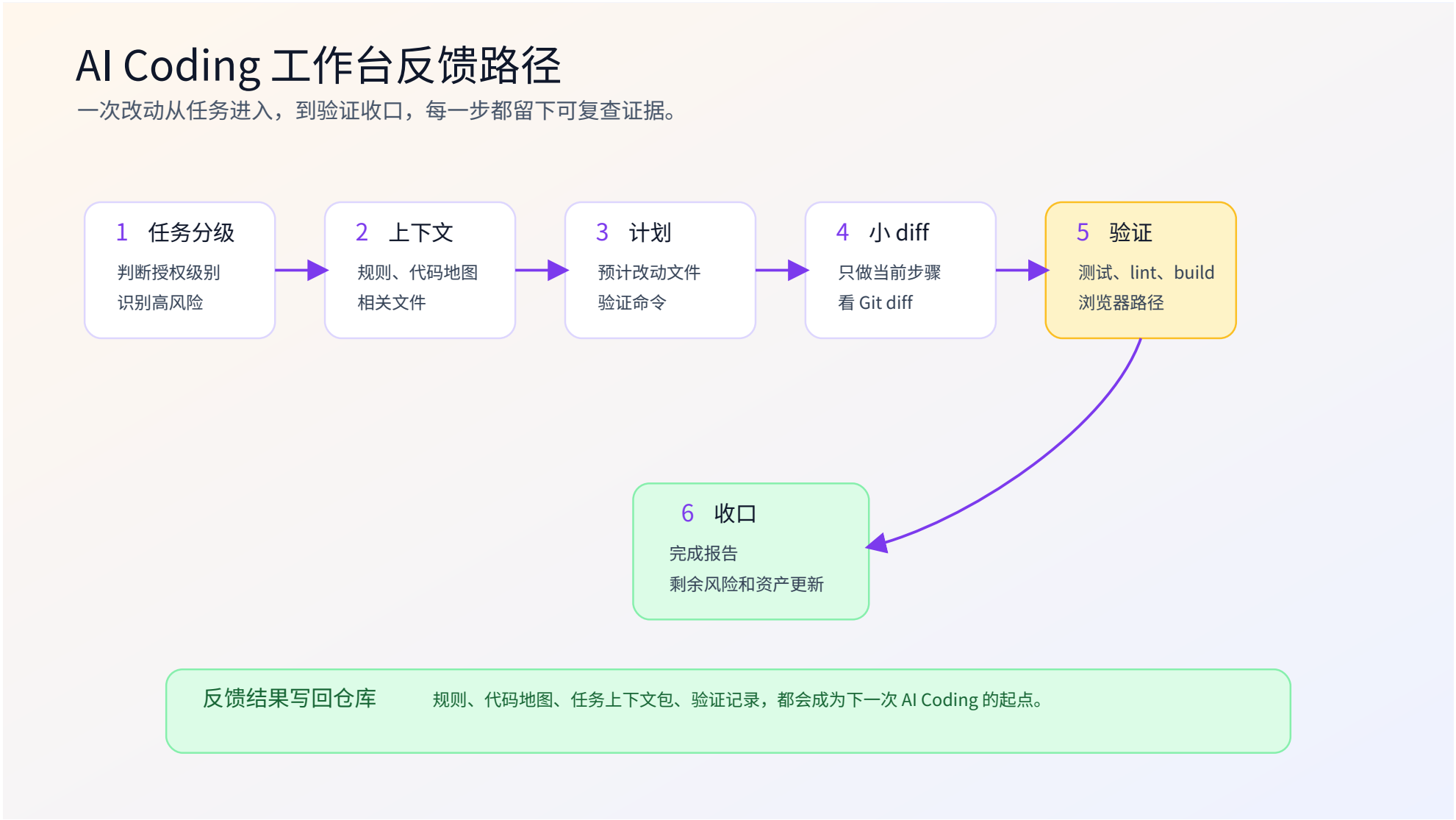


图 6-2　AI Coding 工作台反馈路径

第一步，任务分级。

先判断是 AI 主导、AI 实现人控计划、人主导 AI 辅助，还是只做分析。这个判断决定后面的流程重量。

第二步，生成上下文。

让 Agent 读取项目规则、代码地图和任务上下文包。陌生模块先搜索和列读取文件，不直接改。

第三步，计划。

要求它给出预计修改文件、原因和验证命令。计划没有通过，不进入编辑。

第四步，小步实现。

CLI 或 IDE 只做当前步骤。实现后停下来，看 diff。

第五步，验证。

跑 focused test、lint、build 或浏览器路径。失败先分类，不直接继续改。

第六步，收口。

输出 change summary、commands、evidence、residual risk。必要时把规则或代码地图更新回仓库。

这条路径的好处是：无论用 Codex、Claude Code、Cursor 还是 Copilot，工作台的证据结构都不变。

6.11 不要把工作台搭成工具负担

工作台也会过度设计。

这些做法要避免：

- 为每个任务新建一套复杂项目。
- 为了 AI 写一堆没人维护的脚本。
- 把规则文件写成几十页。
- 每个小文案改动都要求完整上下文包。
- 为了“流程完整”强制跑无关全量命令。
- 让多个 Agent 同时改同一批文件。

工作台应该按风险伸缩。

低风险任务：

```
IDE 修改 -> git diff -> focused test 或人工检查
```

中等风险任务：

```
任务上下文 -> CLI 侦察 -> 计划 -> 小diff -> focused test -> diff review
```

高风险任务：

```
需求澄清 -> 业务规则确认 -> 规格或设计 -> 分步实现 -> 多层验证 -> 人工审查
```

越高风险，越需要证据；越低风险，越要避免形式主义。

6.12 常见坑

坑一：规则文件越写越长。

规则文件应该是长期有效约束，不是任务记录。一次性背景放到任务上下文包里。过期规则要删，否则 Agent 会被旧信息污染。

坑二：代码地图生成后没人更新。

代码地图过期比没有地图更危险。入口、测试命令、生成目录变了，要及时改。可以把更新代码地图作为模块改造后的收尾动作。

坑三：终端命令只为给 AI 看。

命令是给人和机器共同看的证据。不要只跑一个看起来会绿的命令。验证命令要覆盖本次风险。

坑四：浏览器验证没有记录。

“页面看起来没问题”无法复查。至少留下 URL、操作步骤、期望结果。重要页面最好有截图或 Playwright 测试。

坑五：多个工具抢同一个 diff。

CLI Agent、IDE Agent、补全同时改同一批文件，很容易造成上下文冲突。一次只让一个工作面承担主要编辑，另一个用于审查或验证。

6.13 你可以今天就搭的版本

先不要做复杂系统。建三个文件就够了。

第一个：`AGENTS.md`

清单 6-18 你可以今天就搭的版本

```
# Project Instructions

## Commands

- Test: `npm test`
- Lint: `npm run lint`
- Build: `npm run build`

## Rules

- Run `git status --short` before editing.
- Do not add dependencies without approval.
- Do not edit generated files.
- Keep bug fixes scoped.

## Completion Report

- Changed files.
- Commands run.
- Result.
- Residual risk.
```

第二个：docs/ai-coding/code-map.md

清单 6-19 你可以今天就搭的版本

```
# Code Map

## Entry

- `src/main.ts`: main runtime entry

## Tests

- Focused test: `npm test -- <module>`

## Do Not Edit

- `dist/` and generated client directories
```

第三个：docs/ai-coding/task-context-template.md

清单 6-20 你可以今天就搭的版本

```
# Task Context

## Objective

## Scope

## Constraints
```

```
## Verification

## Agent Mode
```

然后用一次真实小需求试跑。不要挑最复杂的需求，也不要挑纯文案。选一个有代码、有测试、有一点风险但能本地验证的任务。

跑完后看四件事：

- Agent 是否读对文件。
- diff 是否小。
- 命令是否能证明结果。
- 有哪些规则值得写回。

这比一次性搭完整体系更有效。

6.14 常见问题

没有测试的老项目怎么搭工作台？

先不要追求自动化完整。把可复查动作写出来：手动复现步骤、接口调用、日志观察、页面截图、数据库只读查询。后续再逐步补测试。

项目很大，代码地图会不会维护不动？

不要做全量地图。只维护运行入口、核心模块、测试命令、配置、生成文件和禁止修改目录。按模块逐步补。

规则文件应该全局一份还是每个目录一份？

先全局一份。只有当目录差异明显，例如前端、后端、移动端命令完全不同，再考虑分目录规则。规则越多，维护成本越高。

我用 Cursor 或 Copilot，还需要 CLI Agent 吗？

不一定。当前文件小改可以留在 IDE。涉及仓库搜索、计划、命令验证和多文件修改时，再切到 CLI 工作面。

工作台会不会让 AI Coding 变慢？

低风险任务不应该变慢。中高风险任务看起来多了几个步骤，但减少的是返工、误改和不可审查 diff。真实项目里，这通常更快。

6.15 把工作台放进个人习惯

一套工作台要能长期用，关键不是文件齐不齐，而是你能不能形成固定动作：

```
开始前：任务分级 + 上下文
实现前：计划 + 边界
实现中：小 diff + 单一编辑面
完成前：命令 + 浏览器/接口验证 + diff
完成后：验证记录 + 规则更新
```

这些动作一旦固定，后续再讲 SDD、OpenSpec、Skill、Superpowers、MCP、多 Agent，才有落点。否则新工具只会让入口变多，证据更散。

工作台搭起来以后，还要解决一个具体问题：App、CLI、IDE 什么时候切换、怎么交接上下文、怎么避免多个 Agent 互相打架。

6.16 工作台不是工具清单

很多人搭工作台，会从“我要装哪些工具”开始。这个顺序容易跑偏。

真正要先定的是四个出口：

```
1. 任务从哪里进入。
2. 代码由哪个工作面主要编辑。
3. 验证结果在哪里留下证据。
4. 规则和经验最后沉淀到哪里。
```

如果这四个出口不清楚，工具越多越乱。

例如一个前端 bug：

```
入口：任务卡写清楚页面、复现步骤、限制范围。
主工作面：能访问仓库、改文件、跑测试并留下证据的 Agent 负责推进。
审查调试面：IDE 读调用链、审 diff、必要时断点 debug。
验证证据：浏览器截图、接口响应、测试命令输出。
沉淀位置：如果发现规则缺口，写回 AGENTS.md 或验证模板。
```

另一个局部类型修复：

```
入口：IDE 选中报错位置。
主编辑面：IDE 补全或 Agent 修改当前文件。
验证证据：typecheck 或 focused test。
沉淀位置：通常不需要沉淀，除非重复出现同类错误。
```

工作台的标准不是“工具全”，而是每次任务都能回答：谁在改、改到哪里、怎么验、验完留下些什么。

6.17 最小工作台的退出标准

一个最小工作台搭好以后，可以用一次真实任务验证。

不是看文档是否齐全，而是看这几个结果：

示例 6-21 最小工作台的退出标准

```
Agent 能读到项目规则。
Agent 能说出实际读取文件。
Agent 能给出小步计划。
主工作面没有同时和另一个 Agent 抢同一批文件。
验证命令能运行，失败时能分类。
完成报告里有 diff、命令、结果、未覆盖路径。
```

如果这些做不到，继续装插件没有意义。

反过来，如果这些已经跑通，就算工具只有 CLI、IDE 和浏览器，也已经能支撑大多数个人 AI Coding 任务。

6.18 工作台出问题时先查哪里

工作台跑不顺，不一定是工具不够。

先查三个地方。

第一，入口是否混乱。任务是从聊天记录、issue、PR 评论、临时口头需求里来的，如果没有整理成目标、范围和验证，Agent 很难稳定执行。

第二，编辑面是否冲突。CLI Agent、IDE 补全、另一个聊天窗口同时改同一批文件，最终 diff 会混在一起。一次任务只保留一个主编辑面，其它工作面只做审查、预览或验证。

第三，验证出口是否缺失。如果不知道最终要跑什么命令、看哪个页面、查哪条日志，Agent 就会选择它最容易完成的验证。

很多“AI 不稳定”的问题，根因在这里。不是模型不会写代码，而是工作台没有把入口、编辑和验证收成一条线。

6.19 先用一次任务校准工作台

工作台搭完以后，不要马上拿最大需求试。

选一个真实但可控的任务，例如修一个 CLI 参数、补一个接口边界测试、改一个表单校验。这个任务要满足两个条件：涉及真实代码，并且能本地验证。

试运行时只看三件事。

第一，Agent 是否能从规则文件和任务卡里读到同一组约束。如果规则写“不新增依赖”，任务执行时仍然准备安装包，说明规则没有进入计划。

第二，主编辑面是否稳定。如果 CLI 已经在改文件，IDE 补全就不要同时改同一段逻辑。工具之间可以协作，但不能抢同一个 diff。

第三，完成证据是否足够复查。只有“已完成”不算。至少要有改动文件、命令输出、未覆盖路径和必要的截图或接口结果。

这次试运行结束后，只沉淀一个最有价值的改进。可能是一条规则，也可能是一个验证命令，或者是删掉一段误导 Agent 的旧说明。一次只改一个点，工作台才会稳定变好。

6.20 可复查资料

- OpenAI Codex `AGENTS.md` 指南：https://developers.openai.com/codex/guides/agents-md〔6-2〕
- OpenAI Codex CLI：https://developers.openai.com/codex/cli〔6-3〕
- Anthropic Claude Code memory：https://code.claude.com/docs/en/memory〔6-4〕
- GitHub Copilot repository instructions：https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions〔6-5〕
- Playwright 官方文档：https://playwright.dev/docs/intro〔6-6〕

本章资源

〔6-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔6-2〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

〔6-3〕 https://developers.openai.com/codex/cli
https://developers.openai.com/codex/cli

〔6-4〕 https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/memory

〔6-5〕 https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions
https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions

〔6-6〕 https://playwright.dev/docs/intro
https://playwright.dev/docs/intro

第7章　App、CLI、IDE 怎么切换完成一次真实改动

AI Coding 进入日常开发后，很快会遇到一个具体问题：什么时候留在当前工具，什么时候才值得切换。

App、CLI、IDE 的能力有重叠，不需要每次都用一遍。多数真实需求应该先选一个主工作面跑下去，只有在需求讨论、代码审查、debug、用户路径验证或结果整理出现缺口时，才切到另一个工具。

这篇不讲工具安装，也不比较工具优劣。我们直接走一遍中等风险改动：

```
用户 session 过期后重新登录，第一次提交资料表单失败。
```

这个需求不算特别大，但足够真实。它涉及前端状态、鉴权刷新、表单提交、错误提示和浏览器验证。它不适合直接丢给后台 Agent，也不需要上来就写完整规格。

一种贴近日常的走法是：先在 App、CLI 或 IDE 里把需求、风险和验证路径聊清楚，再让能访问仓库的主工作面做仓库侦察、实现和验证；如果问题定位到少数文件，切到 IDE 读调用链、审 diff、断点 debug 或做局部修正；最后回到测试、Git diff 和浏览器路径收口。这里没有“必须切工具”的标准动作，只有证据链哪里断了，就在哪里补工具。

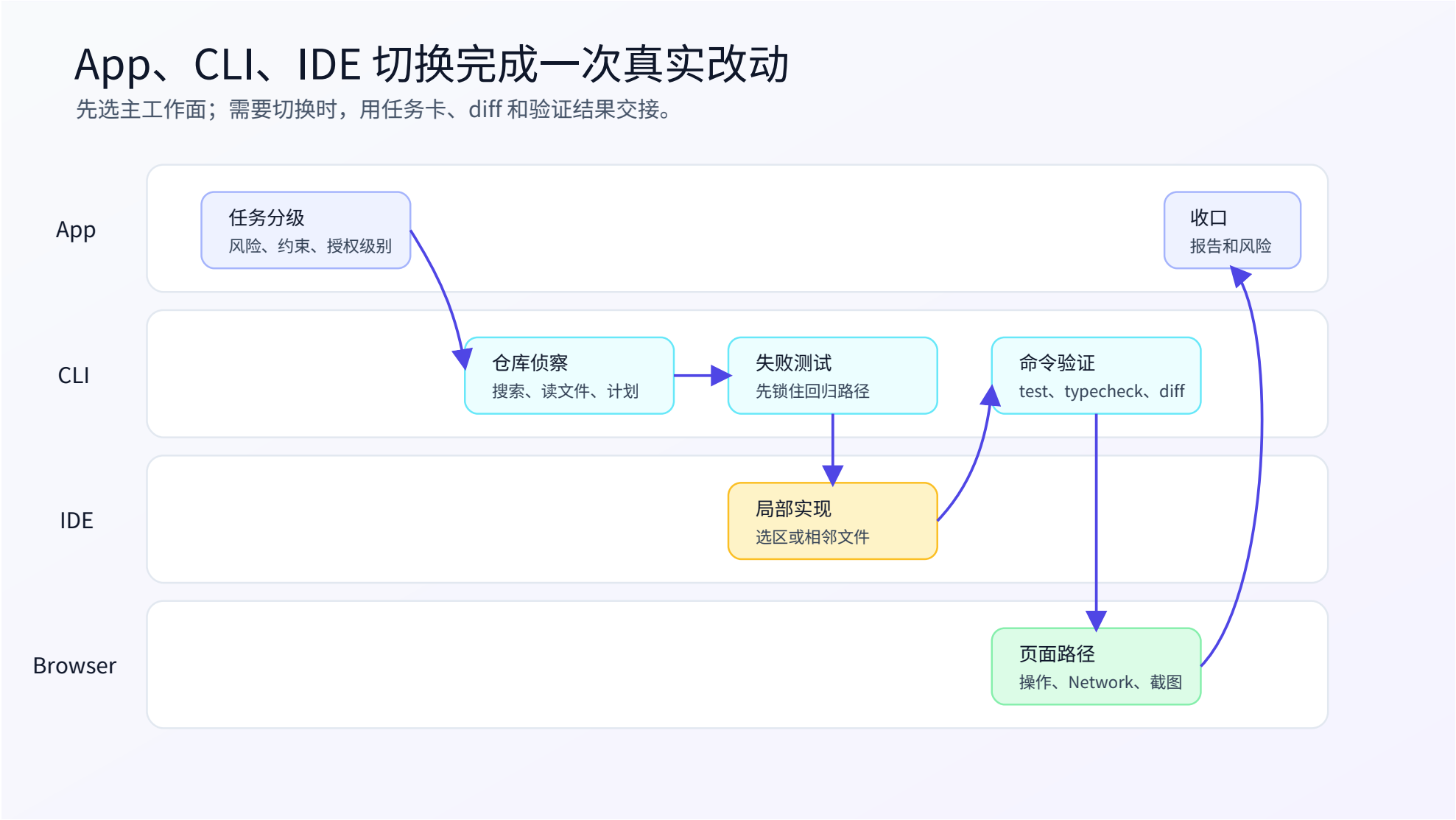


图 7-1　App、CLI、IDE 切换完成一次真实改动

7.1 第一步：先判断主工作面够不够

如果需求已经清楚，可以直接从你常用的主工作面开始。App 的价值是讨论、预览、长任务管理和审查，不是每次改代码的必经入口。需求含糊、风险高、你需要先拆任务时，可以先用 App 把任务风险、缺失上下文和第一步动作说清楚；如果你习惯在 CLI 或 IDE 里聊需求，也完全可以留在当前工作面完成这一步。

可以这样问：

示例 7-1 第一步：先判断主工作面够不够

```
我准备在一个真实项目里修这个问题：

用户 session 过期后重新登录，第一次提交资料表单失败。

先不要写代码。请帮我判断：
1. 这个任务适合哪种 AI 授权级别？
2. 可能涉及哪些模块？
3. 哪些风险需要人确认？
4. 第一轮应该让 CLI Agent 做哪些侦察？
```

理想输出不是一段泛泛分析，而是类似这样：

清单 7-2 第一步：先判断主工作面够不够

```
## 授权级别

AI 实现，人控计划。

原因：
- 涉及登录状态和表单提交，不能直接改。
- 但范围可能集中在 auth refresh 和 submit flow。
- 可以通过测试和浏览器路径验证。

## 需要侦察

- session 过期和 refresh token 的实现位置。
- 表单 submit 入口。
- 401 错误处理逻辑。
- 现有 auth/form 测试。

## 人工确认

- 不改变登录接口协议。
- 不改变 token 存储格式。
- 不吞掉非 401 错误。
```

这一步的产物是任务上下文，而不是代码。

你可以把它压缩成一张任务卡：

清单 7-3 第一步：先判断主工作面够不够

```
# Task Context

## Objective

修复 session 过期后重新登录，第一次提交资料表单失败的问题。

## Constraints

- 不改变登录接口协议。
- 不改变 token 存储格式。
- 不吞掉非 401 错误。
- 不新增依赖。

## Agent Mode

CLI 先侦察并出计划；人确认后再实现。
```

这张任务卡后面可以交给 CLI Agent，也可以继续留在当前 Codex 工作面执行。工具切换时，靠的不是“你记得我们刚才聊过”，而是明确产物。

7.2 第二步：主工作面做仓库侦察，不改代码

切到仓库根目录后，先确认工作区状态。

清单 7-4 第二步：主工作面做仓库侦察，不改代码

```
git status --short
```

如果这里已经有用户改动，不要让 Agent 直接动手。先弄清楚哪些文件属于当前任务，哪些是别人或自己之前的改动。

然后让当前主工作面里的 Agent 做侦察。这里可以是 CLI，也可以是能访问仓库和命令输出的 App 或 IDE Agent：

示例 7-5 第二步：主工作面做仓库侦察，不改代码

```
先不要改代码。

任务：
修复 session 过期后重新登录，第一次提交资料表单失败。

约束：
- 不改变登录接口协议。
- 不改变 token 存储格式。
- 不吞掉非 401 错误。
- 不新增依赖。

请做仓库侦察：
```

```
1. 用搜索找到 auth refresh、401 处理、form submit 相关入口。
2. 列出实际读取过的文件。
3. 标出可能需要修改和不应该修改的文件。
4. 给出最小实现计划和验证命令。
5. 未经确认不要编辑文件。
```

Agent 可能会跑：

清单 7-6 第二步：主工作面做仓库侦察，不改代码

```
rg -n "refresh token|refreshToken|401|session expired|submit" src test
rg --files | rg "auth|session|form|submit|test|spec"
```

你要看的不是它搜了多少，而是它是否找到了四类文件：

- auth refresh 逻辑。
- form submit 入口。
- API 客户端或请求拦截器。
- 相关测试。

如果它只读了表单组件，没有读 auth；或者只读了 auth，没有读 submit；或者完全没找到测试，就不要进入实现。

侦察阶段可以用配套代码仓库 DavidDingXu/learn-ai-coding〔7-1〕 的工具 module 先生成一份基础上下文包：

清单 7-7 第二步：主工作面做仓库侦察，不改代码

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js task-context \
  --title "修复 session 过期后首次提交失败" \
  --files "src/auth/,src/forms/,test/auth-refresh.test.ts"
```

生成文件不是最终答案，只是把目标和候选文件固定下来。接下来要由真实仓库搜索结果补全。

7.3 第三步：确认计划，不让它一步到位

Agent 给出的计划应该具体到文件和命令。

可接受的计划大概是：

清单 7-8 第三步：确认计划，不让它一步到位

```
## Plan

1. Add a regression test in `test/auth-refresh.test.ts`.
```

```
   - Case: expired session, refresh succeeds, first submit retries once.
   - Verification: focused auth-refresh test should fail before implementation.

2. Update `src/auth/request.ts`.
   - Add retry guard for 401 when refresh token exists.
   - Do not swallow non-401 errors.

3. Run focused tests.
   - `npm test -- auth-refresh`

4. Run browser verification.
   - login, expire session, refresh page, submit form.
```

不可接受的计划通常有这些味道：

清单 7-9 第三步：确认计划，不让它一步到位

```
- 重构 auth 模块。
- 统一所有请求错误处理。
- 顺便优化表单提交体验。
- 增加一个新的状态管理库。
- 修改登录接口返回结构。
```

这些不是完全不能做，而是不应该混在这次 bug 修复里。

确认计划时，你可以给 CLI Agent 一个明确授权：

示例 7-10 第三步：确认计划，不让它一步到位

```
只执行计划第 1 步：补回归测试。

要求：
- 不改实现。
- 不改计划外文件。
- 测试应当在当前实现下失败。
- 完成后停止，输出 diff 摘要和测试结果。
```

这一步非常关键。先写失败测试，能防止 AI 直接写一个“看起来修了”的实现。

7.4 第四步：IDE 做局部编辑，但不接管全局判断

有些改动在 IDE 里做更舒服。

比如测试已经写好，问题定位到 `src/auth/request.ts` 的一个请求封装函数。你可以在 IDE 里打开这个文件，选中相关函数，让 AI 做局部修改：

示例 7-11 第四步：IDE 做局部编辑，但不接管全局判断

```
只修改当前选中的请求函数。

目标：
- 当请求返回 401 且 refresh token 存在时，刷新 token 后重试原请求一次。

约束：
- 不改变函数签名。
- 不改变登录接口协议。
- 不吞掉非 401 错误。
- 不新增依赖。
- 不修改其他文件。
```

这里 IDE 的作用是提高局部编辑效率。你仍然要控制边界。

如果 IDE Agent 开始建议改全局状态管理、封装新客户端、修改多个调用方，就停。把建议拿回 CLI 或 App 做计划审查。

IDE 编辑完成后，马上看 diff：

清单 7-12 第四步：IDE 做局部编辑，但不接管全局判断

```
git diff --stat
git diff src/auth/request.ts
```

不要等整轮任务结束再看。局部编辑最容易“顺手多改一点”。

7.5 第五步：验证回到能留下证据的地方

实现完成后，验证要回到能留下证据的地方。很多时候这是 CLI，因为命令输出和 diff 最容易复查；如果你的 App 或 IDE 能稳定运行命令、展示 diff、保存输出，也可以继续留在当前工作面。

先跑 focused test：

清单 7-13 第五步：验证回到能留下证据的地方

```
npm test -- auth-refresh
```

如果通过，再看是否需要更广一点的命令：

清单 7-14 第五步：验证回到能留下证据的地方

```
npm run lint
npm run typecheck
```

如果改到了共享请求封装，可能还要跑更广的测试：

清单 7-15 第五步：验证回到能留下证据的地方

```
npm test
```

每个命令都要回答一个问题：

表 7-1 第五步：验证回到能留下证据的地方

命令	覆盖什么
`npm test -- auth-refresh`	session refresh 和首次提交回归路径
`npm run typecheck`	请求函数签名和调用方类型
`npm run lint`	静态规则和格式问题
`npm test`	共享请求层是否影响其他测试

如果测试失败，不要立刻让 Agent 继续改。先分类：

示例 7-16 第五步：验证回到能留下证据的地方

```
测试失败后先不要改代码。

请判断：
- 是本次引入失败，还是改前已有失败？
- 是测试预期问题、实现问题、环境问题，还是需求冲突？
- 需要查看哪些输出或文件才能确认？
```

这个动作能挡住很多无意义试错。

7.6 第六步：浏览器验证用户路径

这个 bug 涉及用户提交表单，所以测试通过还不够。

需要打开本地页面，走一次真实路径：

清单 7-17 第六步：浏览器验证用户路径

```
## Browser Verification

URL: `http://localhost:3000/profile`

Steps:
1. 登录。
2. 等待 session 过期，或用测试环境手动置为过期状态。
3. 刷新页面。
4. 修改资料表单。
5. 点击提交。
```

```
Expected:
- 请求遇到 401 后刷新 token。
- 原提交请求重试一次。
- 表单提交成功。
- 不出现重复错误 toast。
- 非 401 错误仍然显示原错误。
```

如果项目有 Playwright，可以把这条路径写成测试。没有也没关系，先把手动验证步骤写清楚。

浏览器验证要特别注意两个细节：

第一，观察 Network。

确认是否真的发生了 refresh，再重试原请求。页面不报错不代表逻辑正确，可能只是错误被吞掉了。

第二，观察错误路径。

不要只验证 refresh 成功。还要确认 refresh 失败、非 401 错误、重复提交这些路径没有被破坏。

这就是为什么前面不能让 AI 吞掉非 401 错误。

7.7 第七步：完成报告要能复查

最后不要只写：

```
已修复。
```

完成报告至少要包含四块：

清单 7-18 第七步：完成报告要能复查

```
## Change Summary

- Added regression coverage for expired-session first submit.
- Added one retry after token refresh for 401 responses.
- Kept login API contract and token storage format unchanged.

## Files Changed

- `test/auth-refresh.test.ts`
- `src/auth/request.ts`

## Verification

- `npm test -- auth-refresh`: passed, 6 tests.
- `npm run typecheck`: passed.
- Browser path: login -> expire session -> refresh -> submit, passed.

## Residual Risk
```

```
- Did not verify third-party SSO provider in staging.
- Did not run full end-to-end suite.
```

验证记录可以照配套工程的工具 module 里的 verification ledger 格式写。

清单 7-19 第七步：完成报告要能复查

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
```

这个命令不是要替代你的判断。它只是提醒完成记录必须有 baseline、change summary、verification commands、evidence 和 residual risk。

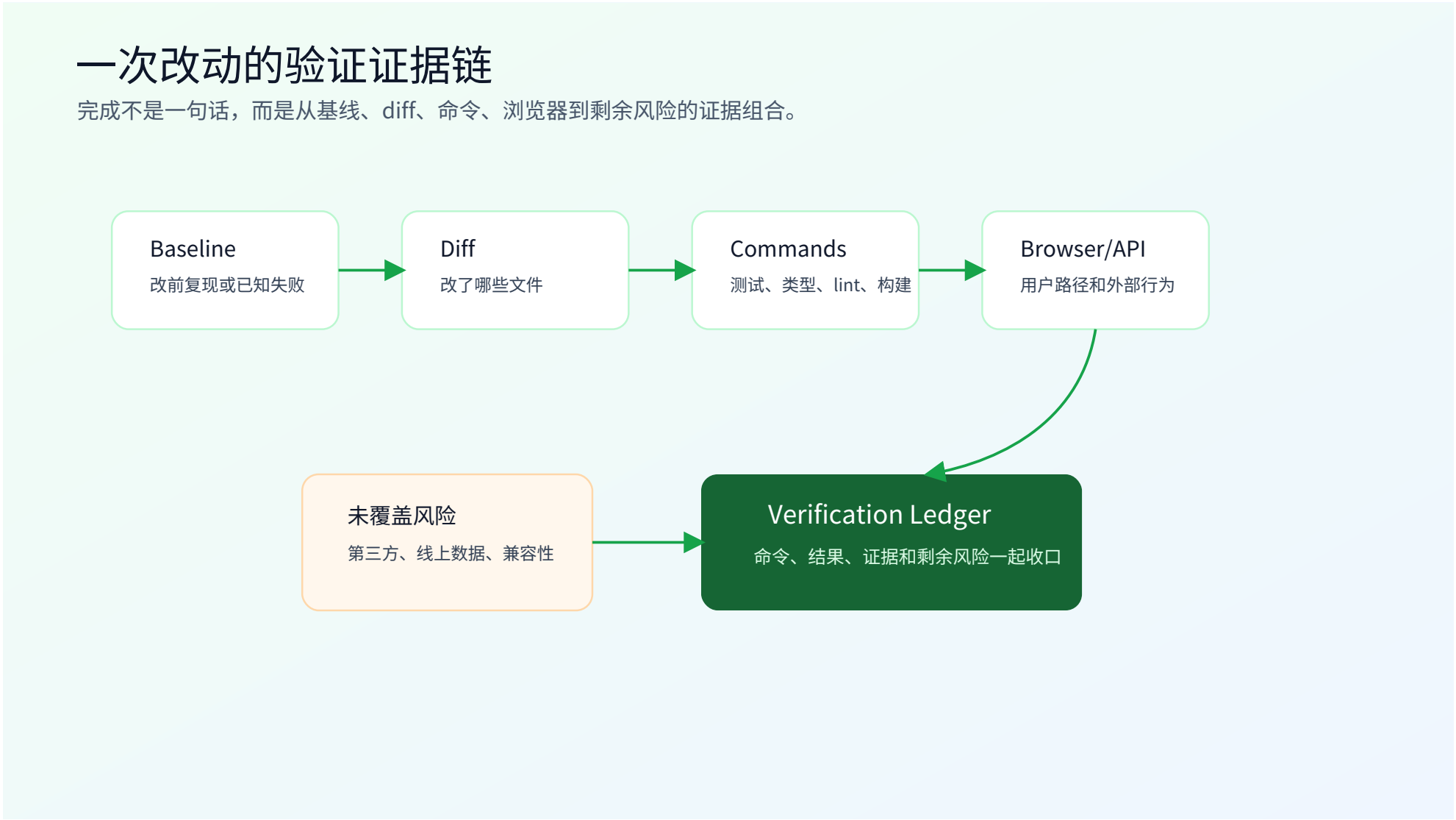


图 7-2　一次改动的验证证据链

7.8 工具交接时最容易丢什么

第一，丢约束。

App 里讨论过“不改登录协议”，切到 CLI 后没写进任务卡，Agent 就可能忘。工具切换时，约束要变成文本产物。

第二，丢文件边界。

CLI 侦察出只需要改 `src/auth/request.ts`，IDE 里选区修改时又让 AI 扩到别的文件。每次切工具，都要重申当前授权范围。

第三，丢验证命令。

IDE 里改完后很容易只看类型提示，不跑真实测试。实现可以在 IDE，验证要回 CLI。

第四，丢失败分类。

测试失败后，如果换到另一个工具继续修，很容易把环境问题、旧失败、本次失败混在一起。失败分类要在同一个记录里沉淀。

第五，丢最终 diff。

多个工具轮流改后，你可能不知道哪些改动是谁引入的。每个阶段结束都看 `git diff --stat`，能及时发现偏航。

7.9 一个常见错误：每个工具都用一遍

很多人学习 AI Coding 后，会下意识把所有工具串起来。

先在 App 里讨论需求，再让 CLI 搜索，再在 IDE 里改，再让另一个 Agent review，再打开浏览器验证。流程看起来完整，但如果没有明确主工作面，最后很容易变成工具之间互相传话。

真实项目里，工具切换有成本。

第一，状态会丢。App 里的结论如果没有整理成任务卡，CLI 只能看到一段自然语言讨论。它不一定知道哪一句是硬约束，哪一句只是猜测。

第二，权限会变。CLI 可以读仓库和跑命令，IDE 可能只看到当前文件，App 可能只能根据你提供的材料推理。你不能假设不同工具看到的是同一个世界。

第三，证据会断。CLI 跑过测试，IDE 里继续改了代码，浏览器又验证了一次。如果最后没有统一的 verification record，别人 review 时不知道哪个验证对应哪个 diff。

所以，工具组合的原则不是“都用上”，而是“一次任务一个主工作面”。

可以按下面方式判断：

表 7-2 一个常见错误：每个工具都用一遍

任务状态	主工作面	其他工具
需求还不清楚	App 或对话型工作面	只产出任务卡，不改代码
仓库内多文件改动	能访问仓库、改文件、跑命令的主工作面	IDE 辅助阅读、debug 和审 diff，浏览器验证用户路径
当前文件局部修改	IDE / 编辑器 AI	终端或主工作面跑测试和看 diff
用户路径验证	浏览器/Playwright	主工作面提供启动命令和日志
结果审查和复盘	App 或独立 review Agent	输入 diff、命令、风险记录

如果你选 App 或 CLI 做主工作面，就让它保留完整任务上下文、计划、实现和验证。IDE 主要用来读代码、审查 diff、断点 debug 和局部修正，不要让它重新定义任务。 如果你选 IDE 做主工作面，就让它承担主要实现上下文；CLI 或 App 只补它缺的能力，例如运行命令、查看 Git diff、复现失败或整理长任务结果，不要另起一套实现计划。 如果你需要 App 参与，就把它的输出整理成任务卡或 review 结论，不要让一段长讨论直接进入实现。

7.10 真实切换记录应该长什么样

下面是一份可复查记录。它不需要很正式，但要能回答“这次为什么切工具、每一步留下了什么证据”。

清单 7-20 真实切换记录应该长什么样

```
# Tool Handoff Record

## Task

session 过期后重新登录，第一次提交资料表单失败。

## Main Surface

CLI Agent.

Reason:
- 需要搜索仓库、读多个文件、跑测试、看 diff。
- 改动可能触及 request wrapper 和 form submit flow。

## App Output

Used only for task framing.

Confirmed:
- Agent mode: plan first.
- Constraints: do not change login API contract; do not change token storage; do not swallow
non-401 errors.
```

```
## CLI Recon

Files read:
- `src/auth/request.ts`
- `src/forms/ProfileForm.tsx`
- `test/auth-refresh.test.ts`

Excluded:
- `src/generated/api-client.ts`: generated file.
- `src/demo/ProfileForm.tsx`: demo path.

Plan:
1. Add regression test for expired-session first submit.
2. Implement one retry after successful refresh.
3. Run focused auth tests.

## IDE Edit

Used only for local edit in `src/auth/request.ts`.

Scope:
- current request wrapper function.
- no signature change.
- no dependency change.

## Verification

- `npm test -- auth-refresh`: passed.
- `npm run typecheck`: passed.
- Browser path: login -> expire session -> refresh -> submit, passed.

## Residual Risk

- Did not run third-party SSO staging path.
- Did not run full E2E suite.
```

这份记录看起来长，但实际写起来很快。它把工具切换变成证据链，而不是聊天记忆。

程序员在这里要做的不是当“工具调度员”，而是控制信息边界：每次切换都要带着目标、约束、文件、验证和残余风险。

7.11 什么时候应该停下来

工具切换过程中，有几类信号说明任务已经不该继续往前推。

第一，CLI 找不到真实入口。

如果 Agent 只能列出几个“可能相关”的文件，但无法解释用户动作从哪里进入，就不要让它实现。先让它扩大侦察，或者回到需求确认。

第二，IDE 修改需要扩大范围。

你本来只让 IDE 改请求函数，它却发现要改状态管理、路由和多个调用方。这时不要在 IDE 里继续扩。把发现带回 CLI 或任务计划，重新拆步骤。

第三，浏览器验证和测试冲突。

测试通过但浏览器仍失败，说明验证路径没有覆盖真实行为。此时不要只调页面，也不要只改测试。应该记录冲突，回到请求、状态、接口和浏览器 Network 证据上。

第四，App 审查提出的新方向没有文件证据。

App 很适合审查方案，但如果它建议“可能是并发刷新问题”，必须回到仓库搜索、日志或测试复现。没有证据的新方向，只能进入假设列表，不能直接进入实现。

第五，diff 已经超过计划。

只要 `git diff --stat` 显示计划外文件，就先停。让 Agent 解释每个计划外文件为什么出现，再决定保留、拆分还是回滚。

这里的停，不是降低效率。它是在防止后面出现一个没人敢 review 的 diff。

7.12 一个可复制的工具切换模板

你可以把下面这段放到自己的工作笔记里。

清单 7-21 一个可复制的工具切换模板

```
# AI Coding Change Workflow

## 1. Optional App: Task Framing

- Clarify objective.
- Classify risk.
- List constraints.
- Decide agent mode.

## 2. CLI: Recon

- Run `git status --short`.
- Search relevant files.
- Read entry, implementation, tests, config.
- Produce plan.

## 3. Human: Plan Review

- Check changed-file list.
- Check verification command.
- Reject refactor/dependency/interface changes unless intended.

## 4. CLI or IDE: Small Change
```

```
- Execute one step only.
- Do not edit out-of-scope files.
- Stop after diff.

## 5. CLI: Verification

- Run focused test.
- Run wider checks if shared code changed.
- Classify failures before editing again.

## 6. Browser/API: Behavior Check

- Verify user-visible path or API behavior.
- Record steps and expected result.

## 7. Close

- Summarize changed files.
- List commands and results.
- Record residual risk.
- Update reusable rule if needed.
```

这个模板的核心不是七个步骤，也不是按工具名分配职责，而是每次切换都要说明能力和证据：

- 这个工作面能看到哪些上下文。
- 它能不能改文件、跑命令、打开页面或调用接口。
- 它留下了什么证据：任务卡、diff、命令输出、截图、接口响应或风险记录。
- 下一个工作面接手时，哪些结论是硬约束，哪些只是待验证假设。

如果一个 App 已经能访问仓库、运行命令并展示 diff，它也可以作为主工作面；如果一个 IDE 能稳定搜索全仓、修改多文件并返回测试输出，它也可以承担主工作面。工具名字不重要，证据能力才重要。

7.13 什么时候不用这么完整

低风险任务不需要完整流程。

比如：

```
把按钮文案从“保存”改成“提交”。
```

可以直接在 IDE 里改，然后看 diff。

清单 7-22 什么时候不用这么完整

```
git diff
```

如果有快照测试或页面截图，再跑一下。没必要写上下文包、计划和完整验证记录。

但下面这些信号一出现，就要升级流程：

- 涉及登录、权限、金额、状态机。
- 改动超过一个模块。
- 测试失败需要解释。
- AI 提议改接口或新增依赖。
- 需要浏览器验证复杂交互。
- 需要多人 review。

流程重量由风险决定，不由工具决定。

7.14 常见问题

App 分析出来的结论要不要直接粘给 CLI?

要粘，但要整理成任务卡。不要把整段闲聊复制过去。CLI 需要的是目标、约束、相关线索、验证命令和授权级别。

App、CLI 和 IDE 都能改代码，应该选哪个?

按任务风险和证据链选。当前文件、选区、类型驱动的小改留在 IDE 更自然；跨文件、需要命令验证和完成报告的任务，用能访问仓库、跑命令、留下输出的 App / CLI / IDE Agent 更自然。无论谁改，最后都要回到 Git diff 和验证命令。

浏览器验证能不能让 AI 自动做?

可以，前提是工具能真的打开页面、点击、截图，并把结果返回。否则就手动验证并记录步骤。不要把“页面应该没问题”当成验证。

如果 CLI Agent 的计划和 IDE 里的直觉冲突怎么办?

先看证据。谁读了真实文件、谁能解释调用链、谁给出可验证命令，就优先相信谁。如果仍冲突，停下来让 Agent 整理事实，不要直接改。

一次任务里能不能用多个 Agent?

可以，但不要让它们同时编辑同一批文件。一个负责侦察，一个负责 review，一个负责实现，都要有明确边界。多数个人任务不需要多 Agent。

什么时候 App 比 CLI 更合适?

当任务更像讨论、审查、预览和整理，而不是马上在 repo 里改文件。比如需求要先拆、风险要先分级、你需要对两个实现方案做取舍，或者你想让一个独立上下文审查计划和 diff。只要进入真实文件修改、断点 debug 和命令验证，就要回到能访问仓库、IDE 或终端证据的工作面。

CLI 能做计划和总结，那 App 还有必要吗？

有，但不是必经。App 的优势通常在长任务管理、结果预览、材料整理、异步审查和跨任务复盘。需求明确、本地仓库可访问、验证命令可运行时，CLI 直接开始更自然；如果你的 App 工作面已经接入仓库和命令，也可以继续留在 App 里推进。

IDE 的 AI 已经能搜索全项目，还需要 CLI 吗？

看验证和 debug 需求。如果 IDE 能可靠读仓库、改多文件、跑命令、返回输出，也可以作为主工作面。也可以把 IDE 留给代码审查和断点 debug，把 App/CLI 留给任务工作流和验证收口。关键不是名字叫 IDE 还是 CLI，而是它能不能留下可复查证据。

工具之间结论冲突，以谁为准？

以证据为准。读了真实文件、能解释调用链、能跑出命令输出、能展示 diff 的结论优先。没有证据的判断只能当假设。

7.15 把组合流程迁移到自己的项目

迁移时不要先买工具，先固定三个动作。

第一，任务入口统一。

每个中等风险任务先写一张任务卡：目标、约束、范围、验证、授权级别。

第二，编辑入口单一。

一次只让一个工作面承担主要编辑。App/CLI 做主工作面时，IDE 主要做阅读、审查、debug 和微调；IDE 做主工作面时，App/CLI 主要补需求拆解、命令验证、diff 总结和复盘材料。

第三，验证出口统一。

不管中间用了哪个工具，最后都输出同一种完成报告：改了什么、跑了什么、证据是什么、还有什么风险。

做到这三点，工具就不会互相打架。

工具工作台先讲到这里。后面进入上下文工程：为什么给 AI 更多代码，不一定会让它做得更好；过多、过期、无关的上下文反而会把它带偏。

还有一个细节：切换工作面时，不要复制整段聊天记录。

把上一个工作面的结论整理成任务卡再交给下一个工作面：

示例 7-23 把组合流程迁移到自己的项目

```
目标：
已确认事实：
当前 diff：
限制：
验证命令：
未解决问题：
```

这样 CLI、IDE、App 之间传递的是工程事实，不是聊天噪音。

7.16 可复查资料

- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔7-2〕
- OpenAI Codex CLI：https://developers.openai.com/codex/cli〔7-3〕
- Anthropic Claude Code 最佳实践：https://www.anthropic.com/engineering/claude-code-best-practices〔7-4〕
- GitHub Copilot cloud agent 任务最佳实践：https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results〔7-5〕
- Playwright 官方文档：https://playwright.dev/docs/intro〔7-6〕

本章资源

〔7-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔7-2〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔7-3〕 https://developers.openai.com/codex/cli
https://developers.openai.com/codex/cli

〔7-4〕 https://www.anthropic.com/engineering/claude-code-best-practices
https://www.anthropic.com/engineering/claude-code-best-practices

〔7-5〕 https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results
https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results

〔7-6〕 https://playwright.dev/docs/intro
https://playwright.dev/docs/intro

PART 04

第4篇　上下文工程与代码库理解

接手陌生仓库时，先找到入口、依赖和验证方式，再构建代码地图与任务上下文。关注检索依据、规则边界和停止条件。

阅读重点

能让 Agent 有依据地理解代码，并控制上下文污染。

第8章　上下文工程：为什么不是塞越多代码越好

很多程序员第一次用 AI 改真实项目，会自然想到一个办法：多给它代码。

把相关目录贴进去，把文档贴进去，把报错贴进去，把历史聊天也贴进去。看起来越完整，AI 越应该懂项目。

实际经常相反。

上下文越多，模型能看到的信息越多，但噪音、旧结论、冲突规则、相似但无关的代码也会一起进来。Agent 不是把所有材料变成可靠事实，它会根据当前上下文里的线索生成下一步判断。你给它一堆材料，如果没有优先级和边界，它就可能被最显眼、最新、最相似但错误的线索带偏。

上下文工程不是“喂更多”，而是“给对、给少、给得有结构”。

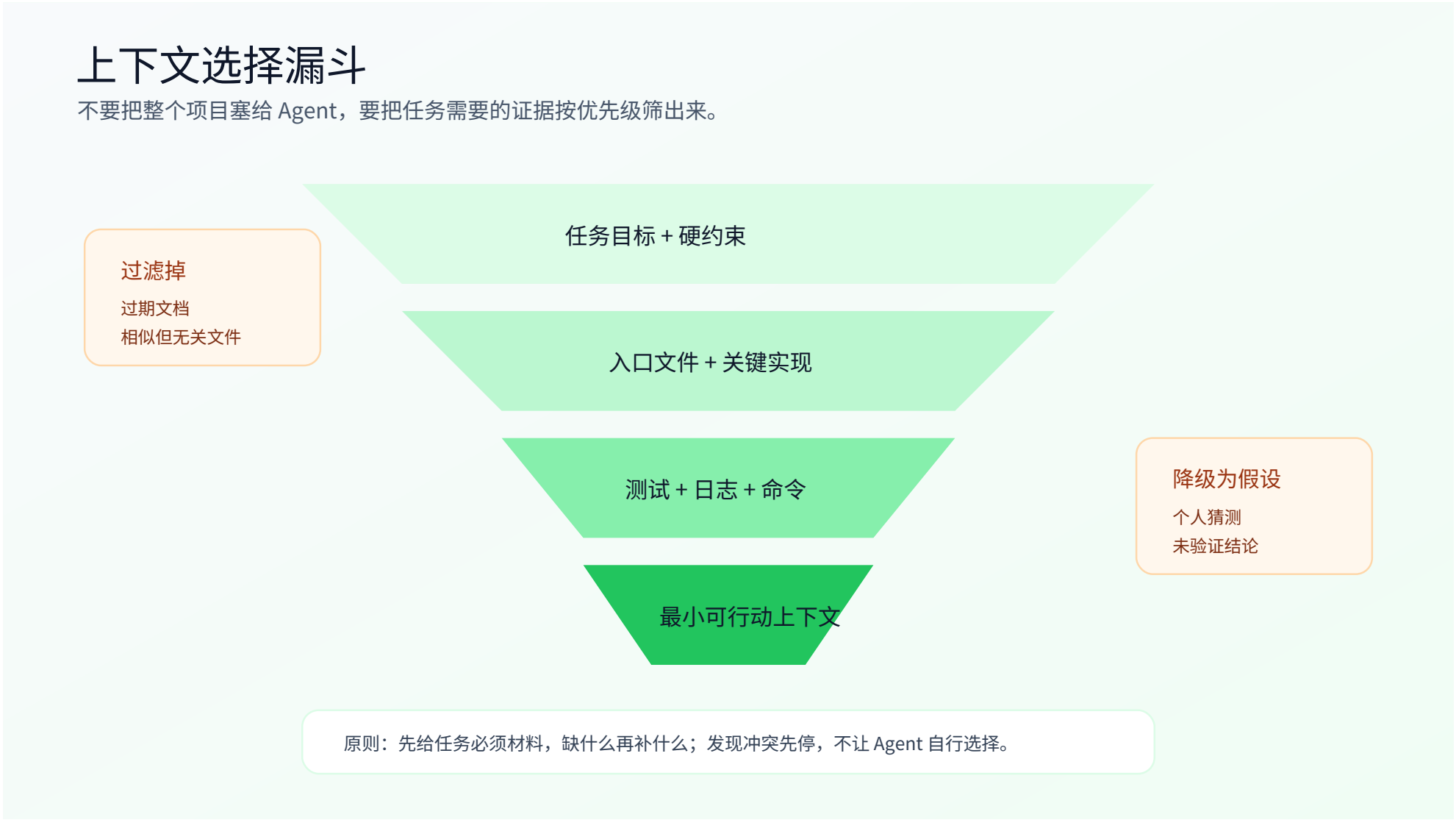

上下文选择漏斗
不要把整个项目塞给 Agent，要把任务需要的证据按优先级筛出来。
过滤掉
过期文档
相似但无关文件
任务目标 + 硬约束
入口文件 + 关键实现
测试 + 日志 + 命令
最小可行动上下文
降级为假设
个人猜测
未验证结论
原则：先给任务必须材料，缺什么再补什么；发现冲突先停，不让 Agent 自行选择。

图 8-1　上下文选择漏斗

8.1 为什么更多上下文会变差

真实项目里，错误上下文主要有五种。

第一种，相似但无关的文件。

比如你要修线上订单提交，仓库里同时有：

```
src/order/submit.ts
src/order/submit.legacy.ts
src/demo/order-submit.ts
docs/order-submit.md
test/order-submit.test.ts
```

如果 Agent 先读到 `demo`，它可能按 demo 逻辑改真实代码；如果先读到 `legacy`，它可能误以为旧接口仍是主路径；如果只读 docs，可能忽略当前实现已经偏离文档。

第二种，过期文档。

文档很有用，但文档会过期。尤其是接口、状态机、部署脚本、环境变量说明。Agent 如果把过期文档当作当前事实，修改方向会很危险。

第三种，冲突规则。

用户说“快速修好”，项目规则说“不要改公共接口”，历史对话里又有人说“可以重构”。这些指令混在一起，Agent 可能抓住最显眼的那条执行。

第四种，失败假设。

排障过程中，前几轮猜测如果没有被清理，会一直污染后续判断。比如一开始怀疑是缓存，后来日志证明不是，但对话里保留了大量缓存讨论，Agent 之后仍可能继续沿着缓存方向改。

第五种，无验证结论。

“应该是前端问题”“看起来是后端返回慢”“可能是 token 过期”，这些话如果没证据，就不能进入实现上下文。它们最多是待验证假设。

上下文工程的第一条原则是：不要把未验证判断和已确认事实混在一起。

8.2 Agent 需要的不是完整项目，而是任务视野

一次具体任务需要的上下文通常分成六层。

表 8-1 Agent 需要的不是完整项目，而是任务视野

层级	内容	作用
任务目标	要改变的用户可见行为或代码行为	防止实现跑偏
硬约束	不能改的接口、目录、依赖、兼容规则	防止越界
入口文件	请求入口、页面入口、命令入口、模块入口	确定从哪里开始看
关键实现	业务逻辑、状态流转、核心函数	决定怎么改
验证材料	测试、复现步骤、日志、截图、命令	证明改动有效
项目规则	命令、目录、代码风格、完成报告	保持工程一致性

这六层比“把相关目录都给它”更有用。

例如修 session 过期后的首次提交失败，给 Agent 这些材料就足够开始：

清单 8-1 Agent 需要的不是完整项目，而是任务视野

```
## Objective

session 过期后重新登录，第一次提交资料表单应该成功。

## Constraints

- 不改变登录接口协议。
- 不改变 token 存储格式。
- 不吞掉非 401 错误。

## Candidate Files

- `src/auth/request.ts`
- `src/forms/ProfileForm.tsx`
- `test/auth-refresh.test.ts`

## Evidence

- 错误日志：`401 refresh token missing`
- 复现路径：登录 -> 等待 session 过期 -> 刷新 -> 提交表单

## Verification

- `npm test -- auth-refresh`
- 浏览器复现路径
```

如果 Agent 读完这些后发现缺上下文，再让它搜索和补充。不要一开始就把整个 `src/` 目录塞进去。

8.3 上下文要有优先级

不是所有材料地位相同。

一个实用的优先级可以这样排：

示例 8-2 上下文要有优先级

```
1. 当前任务目标和硬约束
2. 当前仓库真实源码
3. 当前测试和命令输出
4. 项目规则文件
5. 官方文档和接口契约
6. 历史 issue、PR、聊天记录
7. 个人猜测和经验判断
```

如果源码和文档冲突，不能直接相信文档；要确认当前实现和测试。 如果用户猜测和日志冲突，不能按猜测改；要回到日志。 如果历史 issue 和当前规则冲突，不能直接照历史 issue；要看它是否仍适用。

你可以把优先级写进提示：

示例 8-3 上下文要有优先级

```
如果资料之间冲突，请按下面顺序判断：
1. 当前任务硬约束
2. 当前源码和测试
3. 实际命令输出
4. 项目规则
5. 历史文档和讨论

发现冲突时先停止并说明，不要自行选择。
```

这段话能减少 Agent 自己“合理化”的空间。

8.4 一个上下文选择流程

每次任务开始前，可以按下面流程选文件。



图 8-2　上下文污染和清理机制

第一步，写目标。

目标必须是行为，不是愿望。

差：

```
优化登录体验。
```

好：

```
session 过期后重新登录，第一次提交资料表单应该成功，不应出现重复错误 toast。
```

第二步，写硬约束。

硬约束比背景更重要。

```
不改变登录接口协议。
不改变 token 存储格式。
不新增依赖。
不吞掉非 401 错误。
```

第三步，找入口。

用 rg 先找候选入口：

清单 8-4　一个上下文选择流程

```
rg -n "refresh token|refreshToken|401|session expired|submit" src test
rg --files | rg "auth|session|form|submit|test|spec"
```

不要让 Agent 只凭文件名判断。候选入口要读过才能确认。

第四步，读测试。

很多人先读实现，后读测试。AI Coding 里更建议尽早读测试。测试会告诉你当前行为边界、已有命名、断言风格和可运行命令。

第五步，标记不该改的文件。

上下文包里不仅要写“可能相关”，还要写“看起来相关但暂时不该改”。例如：

清单 8-5 一个上下文选择流程

```
## Do Not Edit Initially

- `src/auth/login.ts`：登录接口协议不在本次范围。
- `src/auth/storage.ts`：token 存储格式不改。
- `src/demo/`：示例页面，不是生产路径。
```

这能避免 Agent 把相似文件都纳入改动。

第六步，缺什么再补什么。

如果计划阶段发现缺调用链，再让 Agent 搜索；如果测试失败需要环境变量，再补 `.env.example`；如果浏览器路径不清楚，再补路由文件。上下文是按证据逐步扩展，不是一次性塞满。

8.5 让 Agent 做上下文侦察，而不是自己整理材料

这里容易产生误解。

上下文工程不是让程序员手工复制一堆文件、手工筛选几十条搜索结果、手工写一份漂亮文档。现在的 Agent 本来就能搜索仓库、读文件、运行命令。程序员要做的是定义侦察目标和验收格式。

一个可用的上下文侦察提示可以这样写：

示例 8-6 让 Agent 做上下文侦察，而不是自己整理材料

```
先不要改代码。

任务：
session 过期后重新登录，第一次提交资料表单失败。

请做上下文侦察，并输出 Context Recon Report。

要求：
1. 先列出搜索关键词，不要直接实现。
2. 搜索入口、实现、测试、配置和项目规则。
3. 每个候选文件都说明为什么相关。
4. 区分 production、test、demo/example、legacy、generated。
5. 标出看起来相关但暂时不应该修改的文件。
6. 给出缺失上下文和下一步最小验证动作。
7. 不要编辑任何文件。
```

Agent 的输出应该长这样：

清单 8-7 让 Agent 做上下文侦察，而不是自己整理材料

```
# Context Recon Report

## Objective
```

```
session 过期后重新登录，第一次提交资料表单应该成功。

## Search Terms

- session, refreshToken, 401, unauthorized, submit, ProfileForm

## Files Read

- `src/auth/request.ts`
  - reason: HTTP 401 handling and retry behavior.
- `src/forms/ProfileForm.tsx`
  - reason: user-visible submit entry.
- `test/auth-refresh.test.ts`
  - reason: existing refresh behavior tests.
- `AGENTS.md`
  - reason: project rules and verification commands.

## Excluded For Now

- `src/generated/api-client.ts`
  - generated file, do not edit directly.
- `src/demo/ProfileForm.tsx`
  - demo route, not production path.
- `docs/auth-old.md`
  - appears outdated; conflicts with current request wrapper.

## Missing Context

- Need to confirm whether original request should retry exactly once.
- Need browser path for expired session reproduction.

## Next Action

- Add or locate a failing regression test for expired-session first submit.
```

这个报告比“我读完了项目”有用得多。你能直接检查它有没有漏测试、有没有把 demo 当生产、有没有把 generated 放进修改范围。

配套代码仓库 DavidDingXu/learn-ai-coding〔8-1〕 的工具 module 里的 `context-audit` 可以作为第一层自动检查：

清单 8-8 让 Agent 做上下文侦察，而不是自己整理材料

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js context-audit --root .
```

它只做启发式扫描，不证明真实调用路径。它的价值是提醒 Agent 和人：先看哪些文件，哪些路径要谨慎，哪些结论必须复核。

8.6　生产项目里最常见的上下文过载

上下文过载不一定是 token 超限。更常见的是材料太多，但优先级不清楚。

第一类，旧 PR 和当前代码混在一起。

很多项目会把历史 PR、issue、设计文档贴给 Agent。它们可能解释了当时为什么这么做，但不一定代表当前系统事实。尤其是经历过多次重构的模块，旧 PR 里的入口、命名、命令可能都变了。

处理方式很简单：历史材料只能作为线索，不能直接作为实现依据。让 Agent 回到当前源码、测试和命令输出确认。

```
下面的 issue 只能作为历史线索。
请以当前源码、测试和命令输出为准。
如果 issue 与当前实现冲突，列出冲突点，不要直接按 issue 修改。
```

第二类，多个相似模块一起进入上下文。

比如 `order`、`order-v2`、`legacy-order`、`demo-order` 同时存在。Agent 很容易选择最短、最整洁、最像标准答案的那条路径，但生产路径往往不是最漂亮的那条。

这时要让它做路径分类：

清单 8-9　生产项目里最常见的上下文过载

```
## Path Classification

- production path:
- compatibility path:
- demo/example path:
- generated path:
- test fixture:
```

分类不清楚，就不能实现。

第三类，失败尝试污染当前判断。

调试时，前两轮假设会留下很多文本。比如先怀疑缓存，再怀疑后端，再怀疑 token。即使后面日志排除了缓存，缓存讨论仍然在上下文里，Agent 可能继续沿着旧假设修改。

处理方式是把已排除假设单独列出来：

清单 8-10　生产项目里最常见的上下文过载

```
## Excluded Hypotheses

- Not cache: reproduced after clearing local storage and disabling cache.
- Not backend 500: submit API returns 401 before backend handler.
```

这比简单说“继续”更能保住当前证据链。

第四类，工具输出未经整理。

一大段 rg 输出、测试日志、浏览器 console，很容易把关键行淹没。Agent 需要原始输出时可以读，但进入实现前应该先整理成摘要。

摘要必须保留证据，不要只留结论：

清单 8-11 生产项目里最常见的上下文过载

```
## Evidence Summary

- `test/auth-refresh.test.ts:42` covers refresh success but not original submit retry.
- `src/auth/request.ts:88` handles 401 and refresh.
- No test found for duplicate submit after refresh.
```

第五类，官方文档被当成项目事实。

官方文档能说明工具或框架当前支持什么，但你的项目可能锁在旧版本，也可能有历史约定。写法应该是：

```
先确认当前依赖版本和项目配置。
只有在版本和配置匹配时，才按官方文档建议修改。
```

这些场景都不是理论问题。真实生产里，AI 很少因为“完全没材料”犯错，更多是因为材料太多、层级不清、旧结论没被排除。

8.7 给 Agent 的上下文包模板

可以直接复用这个模板：

清单 8-12 给 Agent 的上下文包模板

```
# Task Context Package

## Objective

用一句话说明完成后的行为。

## Constraints

- 不允许改变什么。
- 不允许新增什么。
- 必须保留什么兼容行为。

## Known Evidence
```

```
- 复现步骤：
- 错误日志：
- 截图或接口响应：

## Files To Inspect First

- `path/to/entry`
- `path/to/implementation`
- `path/to/test`

## Files Not To Edit Initially

- `path/to/generated`
- `path/to/legacy`

## Verification

- focused test:
- broader check:
- browser/API path:

## Open Questions

- 需要人确认的问题。
```

这个模板不是为了好看，而是为了把上下文分层。

配套工程的工具 module 里可以生成基础任务上下文：

清单 8-13 给 Agent 的上下文包模板

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js task-context \
  --title "修复 session 过期后首次提交失败" \
  --files "src/auth/request.ts,src/forms/ProfileForm.tsx,test/auth-refresh.test.ts"
```

生成后要复核 constraints、evidence、not-to-edit 和 verification。不要把生成结果当最终判断。

8.8 代码地图是上下文工程的入口

陌生仓库里，先别让 Agent 直接改。

先让它读代码地图，或者生成一份轻量代码地图：

清单 8-14 代码地图是上下文工程的入口

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js codemap --root .
```

代码地图应该帮它回答：

· 运行入口在哪里。

- 测试目录在哪里。
- 配置文件在哪里。
- 生成文件和构建产物在哪里。
- 哪些目录不应该直接改。
- 常用验证命令是什么。

代码地图不是全量架构文档。它是一张导航图。

一个老项目里，Agent 最容易误改的地方往往不是业务核心，而是旁边那些看起来相关的文件：旧入口、示例目录、生成代码、测试夹具、历史迁移脚本。代码地图要把这些地方标出来。

8.9 上下文污染的典型症状

你怎么知道上下文已经污染?

看这些信号。

第一，Agent 引用已经被否定的假设。

比如日志已经证明不是缓存问题，它仍然说“我们可以清理缓存”。这说明旧上下文还在影响判断。

第二，Agent 修改了相似但无关文件。

比如任务是生产订单提交，它改了 `demo/order-submit.ts`。这说明文件优先级没讲清。

第三，Agent 反复扩大范围。

一开始修 bug，后来变成重构 auth，再后来变成统一错误处理。上下文里可能混入了“顺便优化”的目标。

第四，Agent 无法解释为什么改某个文件。

如果它只能说“这个文件相关”，而不能说明入口、调用关系和验证方式，就说明上下文选择不够干净。

第五，完成报告没有“未验证风险”。

这通常意味着它把可见材料当成完整事实。

遇到这些情况，不要继续补更多材料。先清理上下文。

8.10 如何清理上下文

清理上下文不是删除对话，而是重新整理事实。

可以让 Agent 停下来输出：

示例 8-15 如何清理上下文

```
请停止修改，重新整理当前上下文。

只按证据分类：
1. 已确认事实。
2. 已排除假设。
3. 未验证假设。
4. 当前任务硬约束。
5. 下一步最小验证动作。

不要继续写代码。
```

一个好的整理结果应该像这样：

清单 8-16 如何清理上下文

```
## 已确认事实

- 表单提交时第一跳请求返回 401。
- refresh token 存在。
- refresh 接口成功返回新 token。
- 原提交请求没有重试。

## 已排除假设

- 不是表单字段校验失败。
- 不是后端提交接口 500。

## 未验证假设

- 请求封装层是否禁止重复提交。
- 是否存在并发 refresh 问题。

## 硬约束

- 不改变登录接口协议。
- 不改变 token 存储格式。

## 下一步

- 补一个 refresh 成功后重试原请求的回归测试。
```

这份整理可以作为新对话或下一步实现的干净上下文。

如果对话已经很长，直接开新对话也可以。但新对话要带整理后的事实，而不是带原始长聊天。

8.11 什么时候应该补更多上下文

少给上下文不是目的。给够才是目的。

需要补上下文的信号：

- Agent 无法确认真实入口。
- 计划里出现“可能”“猜测”但没有文件证据。
- 测试失败和当前 diff 对不上。
- 实现涉及公共接口或共享模块。
- 日志指向另一个服务或异步任务。
- 浏览器行为和单元测试结果冲突。

补上下文时，要补具体材料：

```
不是：再看看整个项目。
而是：请读取 `src/auth/request.ts` 的调用方，并列出哪些路径会触发表单提交。
```

或者：

```
不是：看看测试为什么失败。
而是：请读取失败测试输出、对应测试文件和本次 diff，判断失败类型。
```

越具体，越能避免新噪音。

8.12 如何处理官方文档和项目源码冲突

AI Coding 经常会用到官方文档。官方文档优先级很高，但不是项目当前事实。

例如，官方文档说某个配置默认开启，但你的项目里显式关闭；官方建议某个 API 用法，但项目里因为兼容旧版本保留了不同写法；框架文档已经更新，但项目依赖还是旧版本。

这时应该让 Agent 这样处理：

示例 8-17 如何处理官方文档和项目源码冲突

```
请区分官方文档事实和当前项目事实。

如果二者冲突：
- 列出冲突点。
- 确认项目依赖版本和当前配置。
- 不要直接按官方最新写法修改。
- 先说明迁移成本和兼容风险。
```

工具文档、框架文档、社区经验，都不能绕过当前仓库。

8.13 常见错误写法

错误写法一：

```
我把整个 src 目录都给你，你帮我看看怎么改。
```

问题是没有目标、没有优先级、没有禁止修改范围。

更好的写法：

```
先围绕 session 过期后首次提交失败做上下文侦察。
只读 auth refresh、form submit 和相关测试。
列出读取文件和缺失上下文，不要改代码。
```

错误写法二：

```
你已经看过这个项目了，继续修。
```

问题是“看过”不等于当前任务上下文仍然有效。

更好的写法：

```
继续前先重新列出：
- 当前已确认事实
- 当前 diff
- 还没验证的假设
- 下一步最小动作
```

错误写法三：

```
这是之前的讨论记录，你全部参考。
```

问题是讨论记录里通常混有猜测、被推翻的假设、旧方案和无关信息。

更好的写法：

```
下面是整理后的事实和约束。
只把“已确认事实”和“硬约束”作为实现依据。
“未验证假设”只能用于设计验证动作。
```

8.14 让上下文进入仓库，而不是停在聊天里

好的上下文不要只停在对话窗口。

至少有三类内容值得落到仓库：

```
docs/ai-coding/code-map.md
docs/ai-coding/task-context-template.md
docs/ai-coding/verification-ledger-template.md
```

项目规则放 `AGENTS.md` 或对应工具支持的规则文件里。 任务上下文可以放 issue、PR、临时 markdown 或任务系统里。 验证记录可以放 PR 描述、变更记录或本地复盘文档里。

落到仓库的好处是：换工具、换对话、换 Agent，关键上下文仍然存在。

这会影响后面讲到的 Skill、OpenSpec 和 Superpowers。它们不是“多一套框架”，而是把上下文、规则和验证从聊天里搬到可复用资产里。

8.15 在不同项目里怎么落地

不同项目的上下文重点不一样。

前端项目里，最容易漏的是用户路径。

Agent 读了组件，不代表它理解页面从哪里进来。上下文里要有路由、状态管理、接口封装、关键组件和浏览器验证路径。

清单 8-18 在不同项目里怎么落地

```
## Frontend Context

- Route: `src/routes/profile.tsx`
- Page: `src/pages/ProfilePage.tsx`
- Form: `src/forms/ProfileForm.tsx`
- API wrapper: `src/api/request.ts`
- State: `src/stores/session.ts`
- Browser path: login -> expire session -> refresh -> submit profile form
```

后端项目里，最容易漏的是事务和异步路径。

Agent 读到 controller 不代表它理解 job、consumer、transaction、retry 和幂等。上下文要覆盖入口、service、repository、消息队列、事务边界和测试命令。

清单 8-19 在不同项目里怎么落地

```
## Backend Context

- HTTP entry: `OrderController.cancel`
- Domain service: `OrderStatusService`
- Async callback: `PaymentCallbackConsumer`
- Transaction boundary: `OrderRepository.updateStatus`
- Focused test: `./mvnw test -Dtest=OrderStatusServiceTest`
```

老项目里，最容易漏的是隐式规则。

README 可能过期，测试可能只覆盖一部分，脚本可能只有老同事知道。上下文里要显式写“不知道什么”，不要假装完整。

清单 8-20 在不同项目里怎么落地

```
## Legacy Context

- README test command is outdated.
- `legacy/` still handles old clients; inspect only unless task says otherwise.
- `db/migrations/` is production history; do not edit existing migration files.
- Payment callback has no automated test; manual verification path required.
```

多仓项目里，最容易漏的是合同边界。

前端仓库能看到请求，后端仓库才有业务规则；SDK 仓库有生成客户端，schema 仓库有接口定义。不要让 Agent 在当前仓库里硬猜另一个仓库的行为。

清单 8-21 在不同项目里怎么落地

```
## Cross-Repo Boundary

- Current repo contains frontend submit path.
- Backend validation lives in `order-service`.
- API contract is generated from `api-schema`.
- Do not change frontend behavior until contract is confirmed.
```

这几类写法背后的原则一样：上下文不是材料堆积，而是把当前任务真正会用到的入口、约束和验证路径摆出来。

8.16 上下文质量怎么自检

任务开始前，可以用下面这张表检查上下文是否能进入实现。

表 8-2 上下文质量怎么自检

检查项	合格	不合格
目标	写成可验证行为	只有“优化”“修一下”
入口	有真实入口和证据	只有相似文件名
测试	有测试或明确缺口	没提验证
禁止修改	写出 generated/legacy/API/依赖边界	只写“不要影响其他”
事实/假设	分开列	混在一段话里
冲突处理	说明冲突时停止	让 Agent 自己选
下一步	是最小验证或最小 diff	“继续看看”

如果表里有两项不合格，就不要授权实现。先让 Agent 补上下文。

可以直接用这段：

示例 8-22 上下文质量怎么自检

```
请检查当前上下文是否满足实现条件。
按表格输出：
- Objective
- Entry evidence
- Test or verification
- Do-not-edit boundary
- Facts vs hypotheses
- Conflicts
- Next smallest action

不满足时只补上下文，不要写代码。
```

这段话适合放进 `AGENTS.md` 或个人 Skill 里，作为中高风险任务的默认门槛。

8.17 可以先实践的一件事

下次使用 AI 改代码前，不要先问它“帮我改”。先给它这段：

示例 8-23 可以先实践的一件事

```
先不要改代码。

请基于当前任务建立上下文：
1. 任务目标是什么。
2. 硬约束是什么。
3. 需要先读哪些文件。
4. 哪些文件看起来相关但暂时不该改。
5. 需要哪些验证命令。
6. 还有哪些缺失上下文。
```

然后看它的回答。如果它说不清入口、测试和验证，就不要让它实现。

AI Coding 的上下文不是越大越好。有效上下文要能把 Agent 的下一步行动限定在正确文件、正确假设和正确验证上。

后面会讲代码地图：在陌生项目里，怎样让 AI 快速知道入口、模块、测试和禁止修改目录，而不是一上来就靠搜索结果乱猜。

8.18 常见问题

上下文给少了，AI 不会漏信息吗？

会。所以目标不是少给，而是按证据逐步给。第一轮给目标、约束、入口、测试和验证；如果 Agent 证明缺调用链、配置或外部合同，再补。不要一开始把所有材料混在一起。

Agent 自己能读仓库，还需要上下文工程吗？

需要。能读仓库不等于读对文件。上下文工程的价值是让它的搜索、读取、排除和下一步动作可检查。

官方文档和当前代码冲突，听谁的？

先听当前项目事实。官方文档说明工具或框架能力，但项目可能锁定旧版本、保留兼容写法或有特殊配置。冲突出现时，先列冲突点，再确认依赖版本和项目规则。

历史聊天记录要不要给 AI？

不要原样给。先整理成 Confirmed Facts、Excluded Hypotheses、Open Questions 和 Constraints。原始聊天里通常混有猜测和被推翻的方案。

上下文包要不要长期保存？

普通小任务不一定。高风险任务、跨天任务、多人 review 或以后可能复盘的任务，建议保存。保存时不要保留所有聊天，只保留目标、证据、约束、验证和剩余风险。

8.19 可复查资料

- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔8-2〕
- OpenAI Codex `AGENTS.md` 指南：https://developers.openai.com/codex/guides/agents-md〔8-3〕
- Anthropic Claude Code memory：https://code.claude.com/docs/en/memory〔8-4〕
- GitHub Copilot repository instructions：https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions〔8-5〕

本章资源

〔8-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔8-2〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔8-3〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

〔8-4〕 https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/memory

〔8-5〕 https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions
https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions

第9章　代码地图不是手工文档：让 Agent 生成可检查的仓库导航

让 AI 接手陌生项目，不要第一句就说“你先读一下这个项目”。

这句话太虚。Agent 会自己搜索，但你不知道它先读了什么；它会总结项目，但你不知道总结基于哪些文件；它会说“我理解了”，但你不知道它有没有漏掉入口、测试、生成目录和项目规则。

陌生项目的第一步，不是让人手写一份大文档，而是让 Agent 或工具先生成一份可检查的仓库导航。

代码地图不是完整架构文档，也不是静态大图。它是一份给人和 Agent 共用的导航索引：项目入口在哪里，核心模块在哪里，测试在哪里，配置在哪里，哪些目录不能改，常用验证命令是什么。

有了这份导航，AI Coding 的起点就从“随便搜”变成了“按证据侦察”：Agent 负责读取和生成初稿，程序员负责确认真实入口、禁止修改路径和验证命令。

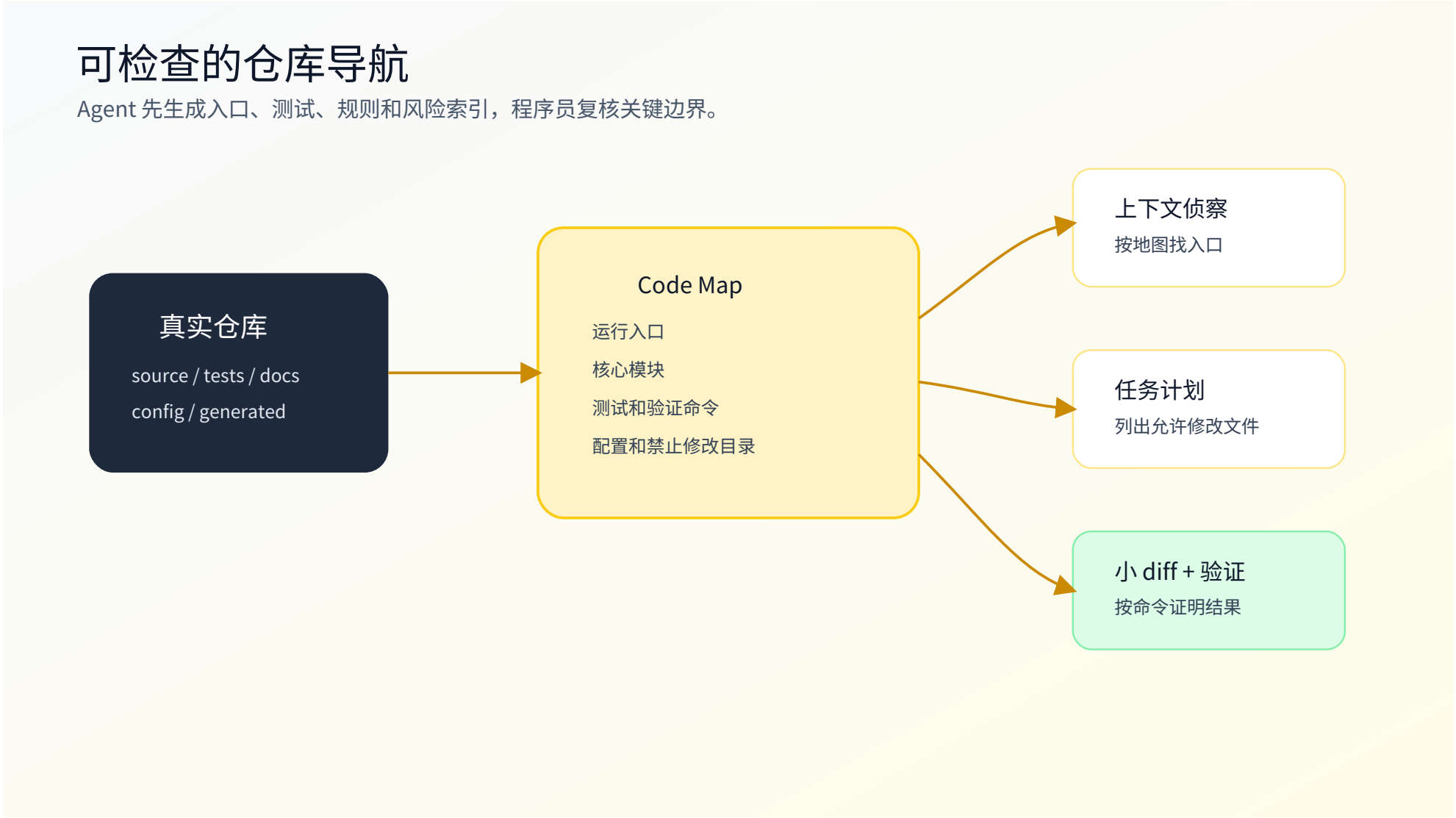


图 9-1　代码地图在 AI Coding 中的位置

9.1　没有代码地图，Agent 会怎样接手项目

一个陌生仓库里，Agent 常见起手动作是：

清单 9-1　没有代码地图，Agent 会怎样接手项目

```
ls
find . -maxdepth 2 -type f
rg -n "keyword" .
cat README.md
```

这些动作没错，但不够。

真实项目里，文件名和目录名经常会误导它：

示例 9-2　没有代码地图，Agent 会怎样接手项目

```
src/order/
src/order-v2/
src/demo/order/
src/legacy/order/
docs/order-flow.md
generated/order-client.ts
test/order.test.ts
```

哪个是生产路径？哪个是旧逻辑？哪个只是示例？哪个是生成文件？哪个测试覆盖当前行为？

如果这些信息没有提前标出来，Agent 可能会：

- 改到 demo 目录。
- 按过期 docs 理解当前逻辑。
- 修改 generated 文件。
- 忽略现有测试。
- 在错误入口上补实现。
- 把 legacy 兼容逻辑删掉。

代码地图的价值，就是在 Agent 动手前，把这些导航信息显式化。

9.2　代码地图解决的不是“看全项目”

代码地图不是为了让 AI 一次看完整个项目。

它解决四个更具体的问题。

第一，让 Agent 知道先看哪里。

README、项目规则、package/pom/build 配置、测试目录、模块入口，是接手项目的基础材料。

第二，让 Agent 知道哪些地方危险。

生成代码、迁移脚本、历史兼容目录、构建产物、外部 API 适配层，都不应该随便改。

第三，让 Agent 知道怎么验证。

如果项目常用命令藏在 README、Makefile、package scripts 里，Agent 很可能跑错命令或只跑一个无关命令。

第四，让上下文能复用。

同一个项目里，每次新对话都让 Agent 从零搜索，会浪费时间，也容易重复踩坑。代码地图把稳定知识沉淀下来。

所以代码地图不是一次性图片，而是仓库内可维护文档。

9.3　一份合格代码地图长什么样

先看一个简化版：

清单 9-3　一份合格代码地图长什么样

```
# Code Map

## Runtime Entry

- `src/server.ts`: HTTP server bootstrap.
- `src/routes/`: route definitions.
- `src/modules/order/`: order domain logic.

## Frontend Entry

- `src/App.tsx`: app root.
- `src/pages/`: route-level pages.
- `src/features/`: feature modules.

## Tests

- `test/order/`: order behavior tests.
- `test/auth/`: auth refresh/session tests.
- Focused command: `npm test -- order`

## Config

- `package.json`: scripts and dependencies.
- `tsconfig.json`: TypeScript config.
- `.env.example`: local env keys.

## Do Not Edit Directly

- `dist/`: build output.
```

```
- `generated/`: generated API client.
- lock files unless dependency changes are approved.

## AI Coding Notes

- Bug fixes should start from tests when possible.
- Do not change API contracts without explicit approval.
- If a task touches `src/modules/payment/`, run payment tests and manual callback check.
```

这份地图没有列出所有文件，但它给出了 Agent 接手项目时最容易漏的东西。

代码地图要短。太长会变成又一份噪音文档。它应该像机场导览图，不像城市规划档案。

9.4　先生成审计报告，再复核关键边界

完全靠人从零写代码地图，常常会拖延，也会只写自己记得住的部分。可执行的方式是先让 Agent 或脚本生成一份导航和审计报告，再由程序员复核关键判断。

配套代码仓库 DavidDingXu/learn-ai-coding〔9-1〕 的工具 module 里有一个轻量命令：

清单 9-4　先生成审计报告，再复核关键边界

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js codemap --root .
```

当前输出类似：

清单 9-5　先生成审计报告，再复核关键边界

```
# Code Map

- Total files scanned: 12
- Source files: 8
- Test files: 3
- Docs: 1
- Configs: 1

## Top Directories

- `src`: 5
- `test`: 3
- `.`: 2
- `examples`: 2

## Files AI Should Inspect First

- `README.md`
- `package.json`
```

```
- `test/codemap.test.js`
- `test/task-context.test.js`
- `test/verification-ledger.test.js`
```

这个命令做了几件很朴素的事：

- 统计源码文件。
- 识别测试文件。
- 识别 README、docs、`AGENTS.md`、`CLAUDE.md`。
- 识别常见配置文件。
- 统计顶层目录。
- 列出 AI 应该优先读取的文件。

它不解析 AST，不生成调用图，不判断业务边界。这是刻意的。

陌生项目接手的第一层索引，不需要一上来就做复杂代码智能。先把入口、测试、配置和文档显出来，再由人和 Agent 针对具体任务继续搜索调用链。

如果要在任务开始前做一次更贴近生产使用的检查，可以生成上下文审计报告：

清单 9-6　先生成审计报告，再复核关键边界

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js context-audit --root .
```

它会把“优先读取文件、候选入口、测试和验证提示、需要谨慎处理的路径”放在一起，并明确提醒哪些结论需要人确认。

注意它仍然只是启发式报告。它能发现 `generated/`、`dist/`、`legacy/` 这类高风险路径，但不能证明真实运行入口，也不能替你判断业务主路径。

9.5　自动地图必须复核三类信息

自动生成后，至少要复核三类内容。

第一，真实入口。

工具能看到 `src/`、`test/`、`README.md`，但不知道哪个文件是运行入口，也不知道哪个模块是当前业务主路径。你要让 Agent 给出依据，并由人确认：

清单 9-7　自动地图必须复核三类信息

```
## Runtime Entry

- `src/main.ts`: app bootstrap.
- `src/routes/order.ts`: order route entry.
- `src/modules/order/service.ts`: order business logic.
```

第二，禁止修改范围。

工具可以识别一些常见目录，但无法知道每个项目的实际维护约定。你要确认哪些文件是生成的、哪些是 vendor、哪些是历史兼容：

清单 9-8　自动地图必须复核三类信息

```
## Do Not Edit Directly

- `src/generated/`: generated from OpenAPI.
- `migrations/`: production migration history.
- `legacy/`: compatibility layer, modify only with explicit task approval.
```

第三，验证命令。

工具能找到 `package.json`，但不知道这次任务最小可验证路径。你要确认 focused test、full test 和必要的用户路径：

清单 9-9　自动地图必须复核三类信息

```
## Verification Commands

- Focused backend test: `npm test -- order`
- Type check: `npm run typecheck`
- Full test: `npm test`
- Frontend manual path: login -> order list -> submit order
```

这三类信息能直接减少误改。它们不是为了让文档好看，而是为了让 Agent 动手前先暴露依据。

9.6　代码地图不是调用链

很多人会把代码地图做成复杂调用图。调用图有用，但不是第一步。

代码地图回答的是“从哪里开始看”。调用链回答的是“这个任务影响谁”。

比如你要修订单状态回退：

代码地图告诉你：

```
订单模块在 `src/modules/order/`
订单测试在 `test/order/`
订单路由在 `src/routes/order.ts`
生成客户端在 `src/generated/`，不要直接改
```

任务开始后，Agent 还要继续查调用链：

清单 9-10　代码地图不是调用链

```
rg -n "rollback|status|paid|pending" src/modules/order test/order
rg -n "updateOrderStatus|OrderStatus" src
```

这两层不要混。代码地图保持稳定，调用链跟任务走。

如果把所有调用关系都塞进代码地图，地图很快过期，维护成本会超过收益。

9.7　给 Agent 使用代码地图的提示

代码地图写好后，不要只让 Agent “参考”。要让它按地图工作。

示例 9-11　给 Agent 使用代码地图的提示

```
先读取 `docs/ai-coding/code-map.md` 和项目规则。

然后围绕当前任务做侦察：
1. 根据代码地图找到候选入口、测试和禁止修改目录。
2. 用搜索确认真实调用路径。
3. 列出实际读取文件。
4. 标出可能需要修改和不应该修改的文件。
5. 给出最小计划。

在完成侦察前不要编辑文件。
```

这个提示把地图变成了工作步骤，而不是背景材料。

如果 Agent 输出计划时没有引用地图里的入口、测试或禁止修改目录，要让它补：

```
你的计划没有说明是否遵守代码地图里的 Do Not Edit 范围。
请重新检查计划，列出每个修改文件为什么允许修改。
```

9.8　代码地图如何保持更新

代码地图的风险是过期。

过期地图会比没有地图更糟，因为 Agent 会把旧信息当成项目事实。

建议把更新时机固定在这些节点：

- 新增或迁移运行入口。

- 测试命令变化。
- 生成文件目录变化。
- 模块拆分或合并。
- 新增重要项目规则。
- 某次 AI Coding 误改暴露了新风险。

每次更新不用大改。只改和导航有关的几行。

比如一次任务里发现 `src/generated/` 不能改，就补：

清单 9-12　代码地图如何保持更新

```
## Do Not Edit Directly

- `src/generated/`: generated API client, run generation command instead of manual edits.
```

发现 auth 模块有专门测试命令，就补：

清单 9-13　代码地图如何保持更新

```
## Verification Commands

- Auth focused test: `npm test -- auth-refresh`
```

代码地图不是文档项目，它是工作流资产。谁踩坑，谁补一条。

9.9　一份代码地图的最小检查清单

写完后，用下面清单检查。

清单 9-14　一份代码地图的最小检查清单

```
## Code Map Check

- [ ] 是否列出运行入口?
- [ ] 是否列出主要业务模块?
- [ ] 是否列出测试目录和 focused test 命令?
- [ ] 是否列出配置文件和环境说明?
- [ ] 是否列出生成文件、构建产物和禁止修改目录?
- [ ] 是否说明常见验证命令?
- [ ] 是否足够短，能让 Agent 快速读取?
- [ ] 是否避免过期的历史设计说明?
```

这里要控制长度。代码地图越短，越容易保持准确。

9.10　常见错误

错误一：把代码地图写成架构说明书。

清单 9-15　常见错误

```
本系统采用分层架构，包含表现层、服务层、数据访问层……
```

这种描述可能没错，但对 Agent 接手任务帮助有限。它更需要真实路径和命令。

直接写路径、用途和验证线索：

清单 9-16　常见错误

```
- `src/routes/order.ts`: order HTTP entry.
- `src/modules/order/service.ts`: order status transition.
- `test/order/status.test.ts`: status transition tests.
```

错误二：只列目录，不写用途。

清单 9-17　常见错误

```
- `src/`
- `test/`
- `docs/`
```

这和 `ls` 没区别。至少要写一句用途。

错误三：不写禁止修改目录。

AI 最容易误改生成文件、构建产物、历史迁移脚本。代码地图里必须把这些地方标出来。

错误四：不写验证命令。

没有验证命令，Agent 完成时就容易只说“已检查”。代码地图至少要提供常用 focused test 和 full test。

错误五：一次生成后不维护。

自动生成只是第一版。关键边界复核和持续更新才是价值。

9.11　在老项目里怎么做

老项目最适合从代码地图开始。

不要一上来要求 Agent 理解全部历史。先做一份“能动手不迷路”的地图：

清单 9-18　在老项目里怎么做

```
# Legacy Project Code Map

## Current Entry
```

```
- `web/controller/`: current web entry.
- `service/`: business services.

## Legacy Compatibility

- `legacy/`: old endpoints, do not delete without migration task.
- `adapter/`: third-party integration compatibility.

## Tests

- `tests/`: partial coverage only.
- Manual verification required for payment callback.

## Do Not Edit Directly

- `db/migrations/`: production history.
- `generated/`: generated clients.

## Known Gaps

- Some modules have no automated tests.
- README command is outdated; use `make test`.
```

老项目里，要把“不知道”写出来。Agent 如果看到“partial coverage only”，就不会把测试通过当成完整证明。

9.12　代码地图和 AGENTS.md 怎么分工

这两个文件容易混。

`AGENTS.md` 写长期工作规则：

清单 9-19　代码地图和 AGENTS.md 怎么分工

```
- Run `git status --short` before editing.
- Do not add dependencies without approval.
- Report commands and residual risk.
```

代码地图写项目导航：

清单 9-20　代码地图和 AGENTS.md 怎么分工

```
- Runtime entry is `src/server.ts`.
- Order tests are under `test/order/`.
- Do not edit `src/generated/`.
```

规则回答“怎么工作”。地图回答“去哪里看”。

这两个文件可以互相引用，但不要重复太多。重复越多，越容易一处更新另一处忘。

9.13　代码地图和任务上下文包怎么分工

代码地图是长期资产，任务上下文包是单次任务材料。

代码地图：

```
订单模块在哪里，测试在哪里，哪些目录不能改。
```

任务上下文包：

```
这次要修订单 paid 状态回退，相关文件是哪些，不能改什么，怎么验证。
```

任务上下文包应该引用代码地图，而不是复制整份地图。

清单 9-21　代码地图和任务上下文包怎么分工

```
## Context Sources

- Code map: `docs/ai-coding/code-map.md`
- Project rules: `AGENTS.md`
```

这样做的好处是：地图更新后，后续任务自然受益。

9.14　可以今天就落地的版本

先让 Agent 或脚本生成一份初稿：

清单 9-22　可以今天就落地的版本

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js context-audit --root .
```

然后把输出中确认过的信息沉淀到仓库里的导航文件，例如：

```
docs/ai-coding/code-map.md
```

这份文件可以很短：

清单 9-23　可以今天就落地的版本

```
# Code Map

## Entry

- `src/main.ts`: runtime entry

## Core Modules
```

```
- `src/modules/order/`: main order flow

## Tests

- `test/order/`: order behavior tests
- Focused command: `npm test -- order`

## Config

- `package.json`: scripts and dependencies
- `.env.example`: local environment keys

## Do Not Edit Directly

- `dist/`: build output
- `src/generated/`: generated API client

## Common Verification

- Focused test: `npm test -- order`
- Full test: `npm test`
- Browser path: login -> order list -> submit order
```

不要直接把 TODO 长期留在仓库里。让 Agent 基于审计报告和实际文件给出候选内容：

示例 9-24　可以今天就落地的版本

```
请先读取项目规则，并运行或复核上下文审计报告。
然后生成 `docs/ai-coding/code-map.md` 草稿。
要求：
1. 每个入口、测试和禁止修改路径都要写出依据。
2. 无法确认真实运行入口时停下来提问。
3. 不要修改业务代码。
```

程序员要做的是验收这份草稿：入口是否是真的，禁止修改路径是否完整，验证命令是否能跑。通过验收后，它才是可复用的上下文资产。

9.15　一次真实接手应该留下什么输出

代码地图不是写完就结束。它要能进入一次真实任务。

比如你准备让 Agent 接手一个陌生模块，可以要求它先输出下面这份记录：

清单 9-25　一次真实接手应该留下什么输出

```
# Onboarding Report

## Project Shape
```

```
- Node CLI project.
- Source under `src/`.
- Tests under `test/`.
- Package scripts defined in `package.json`.

## Files Read

- `README.md`
  - purpose: commands and project boundary.
- `package.json`
  - purpose: test command and module type.
- `src/cli.js`
  - purpose: command dispatch entry.
- `src/lib/codemap.js`
  - purpose: code map generation.
- `test/cli.test.js`
  - purpose: CLI behavior contract.

## Candidate Runtime Entries

- `src/cli.js`
  - evidence: package bin points here; process argv is parsed here.

## Test Entries

- `test/cli.test.js`
  - evidence: process-level CLI behavior tests.
- `test/codemap.test.js`
  - evidence: codemap output contract.

## Treat Carefully

- `examples/`
  - fixture data; do not rewrite without checking tests.
- lock files
  - dependency changes require explicit approval.

## Verification Commands

- `npm test`
- focused: `node --test test/cli.test.js`

## Open Questions

- Does this task require CLI process behavior or only library output?
- Should generated context be persisted or printed only?
```

这份记录有几个好处。

第一，它把 Agent 读过的文件列出来。你可以直接发现它是否漏了 `package.json`、测试或项目规则。

第二，它把“候选入口”和“证据”放在一起。没有证据的入口只是猜测。

第三，它把谨慎路径提前标出来。很多误改发生在 examples、fixtures、generated、legacy、migrations 这些地方。

第四，它把验证命令前置。等实现结束后再找命令，Agent 往往会挑最容易通过的一条。

这个记录可以由 Agent 生成，不需要你手写。你的责任是验收：证据是否成立，入口是否真实，验证命令是否能跑，禁止修改范围是否符合项目约定。

9.16　代码地图里的“证据”怎么写

很多代码地图的问题是只写结论。

清单 9-26　代码地图里的“证据”怎么写

```
- `src/order/service.ts`: order logic.
```

这句话有用，但不够。更好的写法是加一小段证据：

清单 9-27　代码地图里的“证据”怎么写

```
- `src/order/service.ts`: order status transition.
  - evidence: imported by `src/routes/order.ts`; covered by `test/order/status.test.ts`.
```

证据不需要长。只要能让读者知道这个判断从哪里来。

常见证据来源有几类：

表 9-1　代码地图里的“证据”怎么写

结论	可用证据
运行入口	package bin、server bootstrap、route registration、framework config
测试入口	test file、script command、CI config、Makefile target
生成文件	generated header、OpenAPI config、build output path、docs rule
legacy 路径	import graph、README note、route prefix、历史兼容测试
禁止修改	project rule、generator command、migration policy、owner note

如果 Agent 不能给证据，就让它继续侦察，不要让它把结论写进长期地图。

9.17　什么时候不要维护代码地图

不是所有项目都需要独立代码地图。

下面几种情况可以不写，或者只写在 `AGENTS.md` 里：

- 项目很小，入口、测试、命令在 README 和 package scripts 里已经清楚。
- 你只做一次性短任务，不会长期维护这个仓库。
- 仓库已经有准确的开发者文档，而且 Agent 能稳定读取。
- 代码结构变化太快，地图维护成本明显高于收益。

但只要出现下面任意信号，就值得维护：

- 多个同名模块容易混淆。
- 有 generated、legacy、migrations、fixtures 等高风险目录。
- 常用验证命令不明显。
- README 过期，但暂时无法彻底修。
- 多个 Agent 或多次对话都要接手同一仓库。
- 曾经发生过 AI 改错入口、删错兼容逻辑、跑错命令。

代码地图不是为了“文档完整”。它是为了降低接手成本和误改概率。没有这些问题，就不要为了形式感维护它。

9.18　代码地图放在哪里

位置没有唯一标准，但要满足两个条件：Agent 能找到，人能维护。

常见放法有三种。

第一，放在 `docs/ai-coding/code-map.md`。

适合已经有开发文档目录的项目。优点是不会污染根目录，能和任务上下文、验证记录放在一起。

第二，放在 `AGENTS.md` 的一个短节里。

适合小项目。比如：

清单 9-28　代码地图放在哪里

```
## Project Map

- CLI entry: `src/cli.js`
- Library code: `src/lib/`
- Tests: `test/`
- Do not edit fixtures under `examples/` unless a test requires it.
```

第三，放在模块目录下。

适合大仓库中的复杂模块。例如：

```
src/modules/payment/AI-CODING.md
```

这种方式要小心，不要到处散落。只有模块规则、命令、风险和全局明显不同，才值得单独放。

不管放哪里，都要在项目规则里告诉 Agent：

清单 9-29　代码地图放在哪里

```
Before editing an unfamiliar module, read the code map if present.
If the code map conflicts with current source or tests, report the conflict before editing.
```

9.19　代码地图怎么进 review

代码地图不应该变成“写了但没人看”的文档。

当一次 AI Coding 任务踩出新规则时，review 里应该顺手检查是否需要更新地图。

比如 PR 改了订单状态入口：

清单 9-30　代码地图怎么进 review

```
Review note:
- Runtime entry moved from `src/routes/order.ts` to `src/routes/order/index.ts`.
- Update `docs/ai-coding/code-map.md`.
```

或者发现测试命令变化：

清单 9-31　代码地图怎么进 review

```
Review note:
- Old focused command `npm test -- order` no longer matches current test runner.
- Update Common Verification section.
```

这不是给 review 增加负担。相反，它能减少下一次 Agent 接手时的重复解释。

代码地图更新也要小 diff。不要趁机重写整份文档。只改与本次任务有关的入口、风险和验证命令。

9.20　常见问题

代码地图要不要自动更新？

可以，但不要完全依赖自动更新。自动扫描能发现文件变化，发现不了业务主路径和实际维护约定。自动生成后还要复核关键判断。

要不要生成调用关系图?

可以作为具体任务的补充，不建议塞进长期代码地图。调用关系变化快，长期地图只保留入口、模块、测试、配置和禁止修改范围。

每个模块都要一份地图吗?

先全局一份。只有模块很大、命令和规则差异明显时，再做模块级地图。目录层级太多会增加维护成本。

代码地图会不会泄露业务信息?

它在仓库内使用，按项目权限管理即可。不要在公开文章、公开 issue 或外部工具里暴露敏感路径、内部系统名和生产配置。

代码地图和架构图有什么区别?

架构图解释系统结构，代码地图服务开发定位。AI Coding 里，代码地图更偏“下一步去哪看、哪里不能改、跑什么命令”。

9.21　迁移建议

先别追求完整。找一个你最近最常让 AI 修改的项目，让 Agent 或脚本生成一份短地图草稿。

重点验收：

- 运行入口是否来自真实启动路径。
- 当前最常改的模块。
- 测试命令是否能直接运行。
- 生成文件和禁止修改目录是否完整。
- 最近踩过的 AI 误改规则是否沉淀进去。

然后在下一次任务里要求 Agent 先读地图，再侦察，再计划。

如果它因此少改了一个错误文件，代码地图就已经值回成本。

接下来要讲的是代码搜索和上下文选择。代码地图能告诉 Agent 从哪里开始，但具体任务仍然需要用 `rg`、文件树、调用点和测试文件一步步确认真实上下文。

9.22　可复查资料

- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔9-2〕
- OpenAI Codex `AGENTS.md` 指南：https://developers.openai.com/codex/guides/agents-md〔9-3〕
- Anthropic Claude Code memory：https://code.claude.com/docs/en/memory〔9-4〕

- GitHub Copilot repository instructions：https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions〔9-5〕

本章资源

〔9-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔9-2〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔9-3〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

〔9-4〕 https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/memory

〔9-5〕 https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions
https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions

第10章　让 Agent 做代码搜索：怎样约束搜索目标、证据和停止条件

AI Coding 里，代码搜索不应该变成程序员手工跑一堆命令，再把输出贴给 AI。

现代 Agent 本来就能读文件、搜索代码、运行命令。程序员真正要做的是约束它怎么搜、搜完交付什么证据、什么时候停止，避免它拿第一个命中结果就开改。

搜索不是为了找到“看起来相关”的文件，而是为了确定 Agent 应该读哪些文件、不要读哪些文件、先改哪里、怎么验证。

很多失败发生在第一轮搜索。

任务是修订单提交失败，Agent 搜到一个 `submitOrder` 就开始改；任务是修登录状态，Agent 看到一个 `401` 就加重试；任务是改页面交互，Agent 只读组件不读接口和状态管理。后面即使代码写得不错，方向也可能错了。

程序员要把代码搜索当成上下文选择协议。`rg`、文件树、调用点、测试文件、配置文件、Git diff 是 Agent 可以使用的工具；你的职责是规定搜索目标、分类方式、证据格式和停止条件。

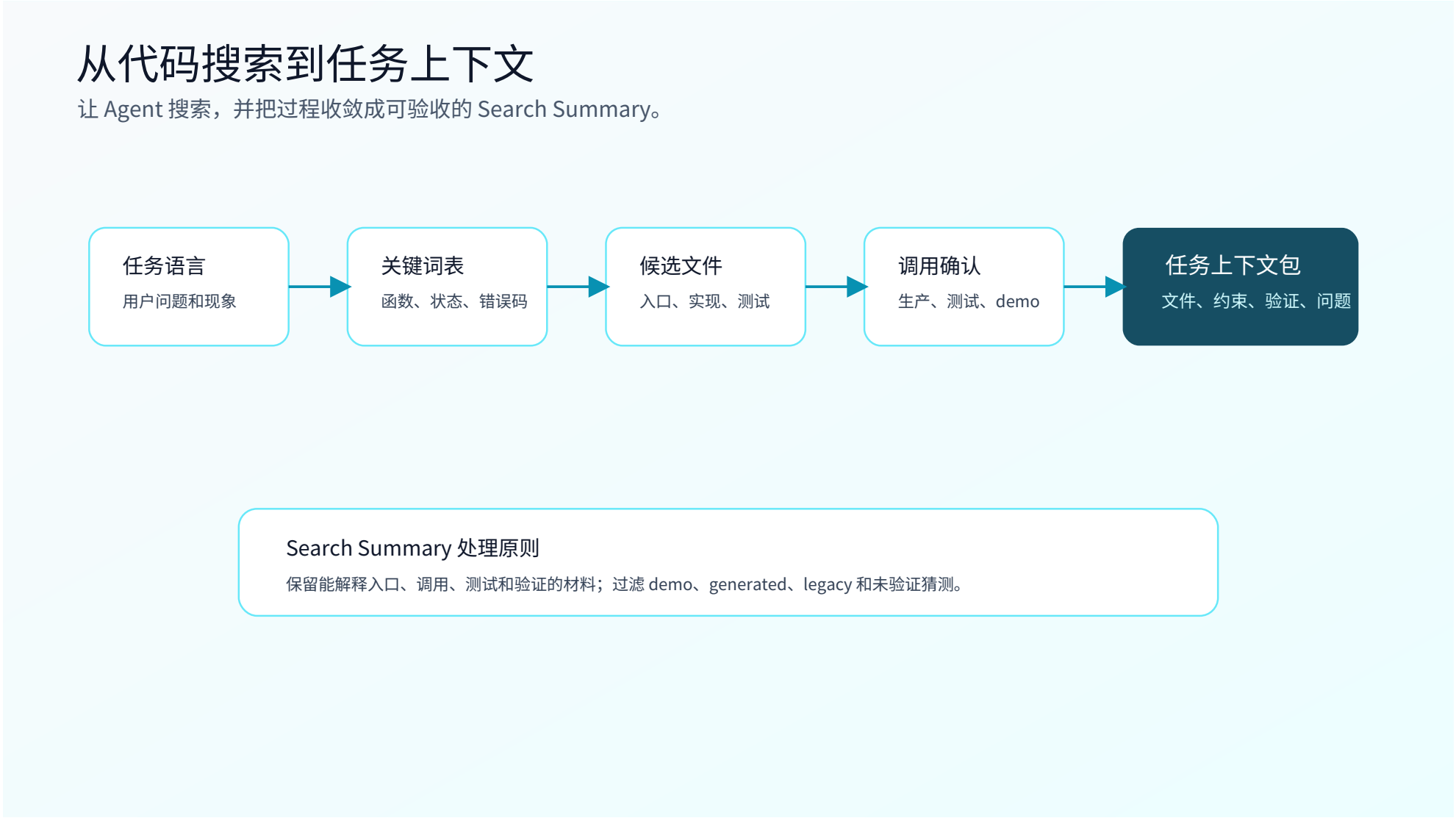


图 10-1　从代码搜索到任务上下文

10.1　先明确搜索要回答什么

不要一上来就搜关键词。

先问四个问题：

1. 用户可见问题是什么？
2. 入口可能在哪里？
3. 业务规则可能在哪里？
4. 修完怎么验证？

比如任务是：

```
session 过期后重新登录，第一次提交资料表单失败。
```

搜索目标不是“找 session”，而是确认：

- 表单提交入口在哪里。
- 401 处理在哪里。
- refresh token 逻辑在哪里。
- submit 是否会重试。
- 是否有相关测试。
- 是否有页面或 API 验证路径。

同样是搜索，目标不同，命令就不同。

10.2　第一组动作：先看文件结构

进入陌生仓库后，先让 Agent 看结构。下面这些命令不是要求你每次亲手执行，而是告诉 Agent 这一轮侦察应该覆盖哪些信息；当结果有争议时，你可以用它们复查。

清单 10-1　第一组动作：先看文件结构

```
rg --files | sed -n '1,120p'
rg --files | rg "README|AGENTS|CLAUDE|package.json|pom.xml|build.gradle|Makefile|justfile|
Taskfile"
rg --files | rg "test|spec|__tests__"
```

第一条看文件树的样子。第二条找规则、文档和配置。第三条找测试。

注意：`rg --files | sed -n '1,120p'` 只是快速观察，不是为了把文件全贴给 AI。它帮 Agent 判断项目类型、目录命名、测试位置和生成目录。

如果项目很大，可以缩小范围：

清单 10-2　第一组动作：先看文件结构

```
rg --files src | sed -n '1,120p'
rg --files test tests __tests__ 2>/dev/null
```

看结构时要标记几类目录：

示例 10-3　第一组动作：先看文件结构

```
src/              可能是源码
test/             测试
docs/             文档，可能过期
generated/        生成代码，谨慎修改
dist/ build/      构建产物，不改
legacy/           兼容逻辑，不能随便删
examples/ demo/   示例，不一定是生产路径
```

这一步的输出应该收敛成代码地图、上下文审计或任务上下文，而不是直接进入实现。

10.3　第二组动作：从用户语言到代码语言

用户说的是“提交失败”，代码里可能叫：

示例 10-4　第二组动作：从用户语言到代码语言

```
submit
save
commit
create
update
mutation
request
checkout
```

不要只搜一个词。先列同义词。

清单 10-5　第二组动作：从用户语言到代码语言

```
rg -n "submit|save|create|update|mutation" src test
```

如果任务是 session 过期，可以搜：

清单 10-6　第二组动作：从用户语言到代码语言

```
rg -n "session|token|refresh|401|unauthorized|expired" src test
```

如果任务是订单状态，可以搜：

清单 10-7　第二组动作：从用户语言到代码语言

```
rg -n "OrderStatus|status|paid|pending|cancel|refund" src test
```

这里的关键是把用户语言翻译成代码语言。很多 Agent 会漏这一步，直接拿用户原话搜索。中文业务词、页面文案、错误提示未必出现在代码里。

可以让 Agent 先做词表：

```
先不要搜索。
请把“session 过期后重新登录，第一次提交表单失败”翻译成可能出现在代码里的英文关键词、函数名、状态名和错误码。
然后再给搜索命令。
```

这个小动作能显著提高命中率。

10.4　第三组动作：找入口，而不是找所有匹配

`rg` 输出很多行时，不要让 Agent 直接进入实现。

先要求它筛入口。

前端任务常见入口：

清单 10-8　第三组动作：找入口，而不是找所有匹配

```
rg -n "ProfileForm|SettingsPage|routes|createBrowserRouter|submit" src
```

后端任务常见入口：

清单 10-9　第三组动作：找入口，而不是找所有匹配

```
rg -n "router|controller|handler|endpoint|POST|GET|/orders" src
```

CLI 或脚本任务：

清单 10-10　第三组动作：找入口，而不是找所有匹配

```
rg -n "commander|yargs|process.argv|main\\(|if __name__" .
```

测试入口：

清单 10-11　第三组动作：找入口，而不是找所有匹配

```
rg -n "auth-refresh|session|submit" test tests __tests__ 2>/dev/null
```

入口的判断标准不是文件名像不像，而是它是否连接用户动作和实现逻辑。

例如：

```
ProfileForm.tsx         可能是页面组件
request.ts              可能是请求封装
auth-refresh.test.ts    可能是验证路径
token-storage.ts        可能是约束文件，不一定要改
```

这时任务上下文可以写：

清单 10-12　第三组动作：找入口，而不是找所有匹配

```
## Candidate Entry

- `src/forms/ProfileForm.tsx`: form submit entry.
- `src/auth/request.ts`: HTTP 401 handling.
- `test/auth-refresh.test.ts`: existing auth refresh coverage.
```

这比“找到很多相关文件”更有用。

10.5　第四组动作：沿调用点确认真实路径

找到候选文件后，要确认调用关系。

如果发现函数 `refreshToken`，继续搜调用点：

清单 10-13　第四组动作：沿调用点确认真实路径

```
rg -n "refreshToken\\(" src test
```

如果发现类型 `OrderStatus`：

清单 10-14　第四组动作：沿调用点确认真实路径

```
rg -n "OrderStatus" src test
```

如果发现错误码：

清单 10-15　第四组动作：沿调用点确认真实路径

```
rg -n "AUTH_EXPIRED|TOKEN_EXPIRED|401" src test
```

调用点搜索要回答：

- 谁调用它？
- 调用前后有什么状态变化？
- 是否有多条路径？
- 哪些路径有测试？
- 是否有旧接口或兼容层？

一个常见错误是只读函数定义，不读调用点。函数本身看起来可以改，但调用方可能依赖旧行为。

给 Agent 的要求可以这样写：

示例 10-16　第四组动作：沿调用点确认真实路径

```
请不要只读函数定义。
找到候选函数后，继续搜索调用点，并区分：
- 生产调用路径
- 测试调用路径
- demo/example 调用路径
- legacy 调用路径
```

这能避免它把示例路径当生产路径。

10.6　第五组动作：找测试，不要后补

AI Coding 里，测试不是实现后的收尾，它是上下文的一部分。

先搜测试：

清单 10-17　第五组动作：找测试，不要后补

```
rg -n "session|refresh|submit|401" test tests __tests__ 2>/dev/null
rg --files | rg "(test|spec)\\.(js|ts|tsx|java|kt|go|py)$"
```

读测试时看四件事：

1. 当前行为怎么描述。
2. 旧边界是什么。
3. 测试命令是什么。
4. 是否有类似路径可复制。

如果没有测试，也要写清楚：

清单 10-18　第五组动作：找测试，不要后补

```
## Test Coverage

- No existing test found for expired-session first submit.
- Existing auth tests only cover login success and refresh failure.
- Need either a regression test or manual browser verification.
```

没有测试不是不能改，但不能假装有验证。

10.7　第六组动作：查项目命令和规则

很多 AI 失败不是代码错，而是命令跑错。

先找项目命令：

清单 10-19　第六组动作：查项目命令和规则

```
cat package.json
rg -n "\"scripts\"" package.json
rg -n "test|lint|typecheck|build" README.md Makefile justfile Taskfile.yml 2>/dev/null
```

如果是 Java：

清单 10-20　第六组动作：查项目命令和规则

```
rg --files | rg "pom.xml|build.gradle|gradlew|mvnw"
```

如果是 Go：

清单 10-21　第六组动作：查项目命令和规则

```
rg --files | rg "go.mod|Makefile"
```

再读项目规则：

清单 10-22　第六组动作：查项目命令和规则

```
rg --files | rg "AGENTS.md|CLAUDE.md|copilot-instructions|instructions"
```

项目规则和命令应该进入任务上下文：

清单 10-23　第六组动作：查项目命令和规则

```
## Project Rules And Commands

- Project rules: `AGENTS.md`
- Focused test: `npm test -- auth-refresh`
- Type check: `npm run typecheck`
- Do not edit: `src/generated/`
```

否则 Agent 很容易用默认命令猜。

10.8　搜索结果应该怎样交付

不要让 Agent 把几十行 `rg` 输出原样丢回来，也不要只接受一句“我搜过了”。

要求它整理成结构化的 Search Summary：

清单 10-24　搜索结果应该怎样交付

```
## Search Summary

### Objective

session 过期后重新登录，第一次提交资料表单应该成功。

### Candidate Files

- `src/forms/ProfileForm.tsx`
  - reason: form submit entry
- `src/auth/request.ts`
  - reason: HTTP 401 handling
- `src/auth/token-storage.ts`
  - reason: token storage, do not edit initially
- `test/auth-refresh.test.ts`
  - reason: related auth refresh tests

### Not Relevant Initially

- `src/demo/ProfileForm.tsx`
  - reason: demo page, not production route
- `src/generated/api-client.ts`
  - reason: generated file

### Verification

- `npm test -- auth-refresh`
- browser path: login -> expire session -> refresh -> submit
```

整理的过程就是上下文选择。

如果 Agent 只返回原始输出，它仍然把优先级判断留给了后续生成；如果它先整理证据，你就能检查它是否把 demo、legacy、generated、tests 和生产路径区分清楚。

10.9　Search Summary 怎么验收

Agent 给了 Search Summary，不代表它就能开始改。

你至少要检查六件事。

第一，是否覆盖入口。

对于用户可见问题，必须能找到用户动作进入系统的地方。前端是页面、路由、组件或 submit handler；后端是 route、controller、handler、job 或 consumer；CLI 是命令入口和 argv 解析。只找到工具函数，不算找到入口。

第二，是否覆盖测试。

Agent 如果没有找测试，就很容易把实现写成自己的理解。没有测试也要明说，而不是跳过。

第三，是否区分路径类型。

Search Summary 里至少要区分：

示例 10-25　Search Summary 怎么验收

```
production path
test path
demo/example path
legacy path
generated path
docs-only path
```

这些路径混在一起，后面一定会出问题。

第四，是否有排除理由。

“Not relevant initially” 不是垃圾桶。每个排除文件都应该有理由。比如 generated、demo、旧文档、测试夹具、历史兼容层、当前任务不允许修改。

第五，是否给出验证入口。

搜索结束时应该知道至少一个验证动作：focused test、接口调用、浏览器路径、日志检查、脚本命令。没有验证入口，就不该实现。

第六，是否列出开放问题。

真实项目里，搜索很少能回答所有问题。Open Questions 不是失败，而是防止 Agent 自己拍板。比如“原请求是否允许重试一次”“并发提交怎么处理”“旧 token 格式是否需要兼容”。

可以用下面这段话验收：

示例 10-26　Search Summary 怎么验收

```
请自检你的 Search Summary：
1. 是否找到真实入口？证据是什么？
2. 是否读取或确认没有相关测试？
3. 是否区分 production/test/demo/legacy/generated？
4. 每个排除文件的理由是什么？
5. 第一个验证动作是什么？
6. 哪些问题必须由人确认？

如果任一项答不上来，不要开始实现。
```

这段自检不是形式。它能把“搜过了”变成“搜对了没有”。

10.10　搜索误区对照

代码搜索误区对照

同样是搜索，差别在于是否把输出整理成可验证上下文。

错误搜法

只搜用户原话

只读第一个命中文件

只看实现，不看测试

只看定义，不看调用点

只说“我搜过了”

```
rg -n "提交失败" .
```

工程化搜法

先把用户语言翻译成代码词表

区分生产、测试、demo、legacy

测试文件作为上下文的一部分

沿调用点确认真实路径

整理成 Search Summary

```
rg -n "submit|401|refresh|token" src test
```

图 10-2　代码搜索误区对照

误区一：只搜用户原话。

用户说“提交失败”，代码里可能没有“提交”。要翻译成函数名、状态名、错误码、接口路径。

误区二：只读第一个命中文件。

第一个命中可能是 demo、legacy、docs 或测试夹具。要确认是否生产路径。

误区三：只搜实现，不搜测试。

测试是行为边界。没有测试，就要补测试或手动验证路径。

误区四：只看定义，不看调用点。

定义告诉你函数做什么，调用点告诉你谁依赖它。

误区五：只看当前文件，不看项目规则。

项目规则可能明确禁止改生成目录、禁止新增依赖、要求跑某些命令。

10.11　一个完整搜索协议的骨架

不要急着写脚本。先把 Agent 的搜索顺序固定下来。下面这些命令可以作为协议里的可复查证据，也可以在 Agent 输出异常时手动复核。

清单 10-27　一个完整搜索协议的骨架

```
# 1. worktree
git status --short

# 2. project shape
rg --files | sed -n '1,120p'
rg --files | rg "README|AGENTS|CLAUDE|package.json|Makefile|pom.xml|build.gradle"

# 3. tests
rg --files | rg "test|spec|__tests__"

# 4. task keywords
rg -n "session|token|refresh|401|unauthorized|expired|submit" src test 2>/dev/null

# 5. candidate call sites
rg -n "refreshToken\\(|submitProfile\\(|request\\(" src test 2>/dev/null

# 6. commands
rg -n "test|lint|typecheck|build" package.json README.md Makefile 2>/dev/null
```

每一组命令后都要让 Agent 回答：这组输出改变了上下文选择吗？

如果没有，就不要把它塞进任务上下文。

10.12　让 Agent 搜索时怎么约束

可以直接让 Agent 执行搜索，但要约束输出。

示例 10-28　让 Agent 搜索时怎么约束

```
请用 `rg` 做上下文侦察，暂时不要改代码。

要求：
1. 先列关键词，不要直接搜索。
2. 搜索入口、实现、测试、配置。
3. 区分生产路径、测试路径、demo/example、legacy、generated。
4. 输出 Search Summary，不要贴完整原始输出。
5. 如果真实入口不确定，停止并提问。
```

这个提示比“你先搜一下”强得多。

它让 Agent 明确产出：关键词、分类、候选文件、排除文件、缺失上下文。

10.13　什么时候扩大搜索范围

一开始不要全仓乱搜，但也不能死守小范围。

需要扩大范围的信号：

- 候选入口读完后找不到实际调用。
- 测试失败指向另一个模块。
- 日志里出现异步任务或队列。
- 配置决定了运行路径。
- 接口契约在生成客户端或 schema 里。
- 同名函数有多个实现。

扩大搜索时要说清楚原因：

```
当前 `src/auth/request.ts` 无法解释失败。
错误日志指向 `refresh queue`。
请扩大搜索到 queue 和 retry 相关文件：
`rg -n "refresh queue|retry|pending request" src test`
```

不要说：

```
再看看别的地方。
```

模糊扩大范围最容易产生大 diff。

10.14　什么时候停止搜索

搜索也会失控。

有些 Agent 搜着搜着会把整个项目都读一遍，然后产出一份很长的总结。看起来勤奋，实际会增加噪音。

停止搜索的标准不是“再也没有文件可看”，而是已经具备实现前需要的最小证据。

可以用下面五个条件判断：

```
1. 找到真实入口。
2. 找到核心实现或确认实现缺失。
3. 找到相关测试，或确认没有相关测试。
4. 找到至少一个可执行验证动作。
5. 列出本次不应该修改的相似路径。
```

满足这五项，就可以停止第一轮搜索，进入计划阶段。

如果继续搜索，必须说明新增搜索要回答什么问题。比如：

```
继续搜索 only if it answers one of these:
- original request retry policy
- concurrent refresh behavior
- existing regression test location
```

不要用“再全面看一下”这种要求。它会把 Agent 带回全仓浏览。

搜索停止后，Agent 应该输出一个最小计划，而不是直接改代码：

清单 10-29　什么时候停止搜索

```
## Minimal Plan

1. Add a regression test for expired-session first submit.
2. Implement retry-after-refresh in request wrapper only if the test proves the current failure.
3. Run focused auth test and typecheck.
```

搜索的终点是计划，不是代码。

10.15　搜索失败时的处理策略

搜索失败也要有流程。

第一种，关键词不匹配。

比如用户说“提交资料”，代码里叫 `saveProfile`、`mutateProfile`、`updateUserInfo`。这时让 Agent 先做词表扩展，不要猜实现。

```
当前关键词没有命中真实入口。
请从 UI 文案、route、API path、测试名、类型名五个方向扩展关键词。
```

第二种，入口在框架约定里。

有些项目入口不明显，比如 Next.js app router、Rails routes、Spring annotations、NestJS decorators、FastAPI route decorators。单纯搜 `router` 未必够。

要求 Agent 先判断框架，再按框架约定搜：

```
请先判断项目框架和路由约定。
然后按该框架的入口规则搜索页面、route、handler 或 controller。
```

第三种，入口在另一个仓库或服务。

如果前端只看到 API 调用，但业务逻辑在后端仓库，不要让 Agent 在当前仓库硬改。

清单 10-30　搜索失败时的处理策略

```
Search Result:
- Current repo contains frontend submit path.
- Backend validation logic is not present.
- Need backend repo, API schema, or documented contract before changing behavior.
```

第四种，现有测试覆盖缺失。

没有测试时，搜索结果应该明确写：

清单 10-31　搜索失败时的处理策略

```
No existing regression test found.
Next action: add failing test or define manual verification path before implementation.
```

第五种，搜索结果互相冲突。

比如 README 写 `npm test`，package scripts 里只有 `pnpm test`；docs 说 `generated/` 不要改，但当前代码里有人手改过 generated 文件。冲突不能靠 Agent 猜。

```
请列出冲突点、各自证据来源，并停止实现。
```

搜索失败不是坏事。坏的是失败后还继续实现。

10.16　如何处理搜索不到

搜索不到不等于不存在。

可能是：

- 关键词不对。
- 逻辑在配置里。
- 逻辑在生成代码里。
- 行为来自第三方库。
- 文案来自后端。
- 入口在另一个仓库。

这时不要让 Agent 猜。

可以这样问：

```
当前关键词没有找到入口。
请列出 5 个可能原因，并为每个原因给一个最小验证动作。
不要修改代码。
```

例如：

清单 10-32　如何处理搜索不到

```
## Possible Reasons

1. form submit uses generic `mutate` helper.
   - search: `rg -n "mutate\\(" src`

2. 401 handling is in generated API client.
   - inspect generated client, but do not edit directly.

3. error text comes from backend.
   - search API response schema and backend error code.
```

搜索不到时，最重要的是调整搜索策略，而不是直接实现。

10.17　从搜索到任务上下文包

最终要把搜索结果收敛成任务上下文包。

清单 10-33　从搜索到任务上下文包

```
# Task Context

## Objective

session 过期后重新登录，第一次提交资料表单应该成功。
```

```
## Files To Inspect First

- `src/forms/ProfileForm.tsx`: submit entry.
- `src/auth/request.ts`: 401 handling.
- `test/auth-refresh.test.ts`: related tests.

## Files Not To Edit Initially

- `src/auth/token-storage.ts`: token storage format is out of scope.
- `src/generated/api-client.ts`: generated file.
- `src/demo/ProfileForm.tsx`: demo only.

## Verification

- `npm test -- auth-refresh`
- `npm run typecheck`
- browser path: login -> expire session -> refresh -> submit

## Open Questions

- Does refresh success guarantee original request can be safely retried once?
- How should concurrent submit requests behave?
```

这份上下文包才是进入实现阶段的材料。

搜索本身不是成果。能指导下一步行动、能被人复查的上下文才是成果。

10.18　生产里怎么留下搜索证据

搜索证据不一定要永久保存，但中高风险任务应该能在 PR 或任务记录里复查。

最小格式可以是：

清单 10-34　生产里怎么留下搜索证据

```
## Search Evidence

### Goal

Find expired-session first-submit path.

### Commands / Agent Actions

- searched `session|token|refresh|401|submit` under `src` and `test`
- inspected route, request wrapper, form submit, and auth tests

### Files Read

- `src/forms/ProfileForm.tsx`
- `src/auth/request.ts`
- `test/auth-refresh.test.ts`
```

```
### Excluded

- `src/demo/ProfileForm.tsx`: demo only.
- `src/generated/api-client.ts`: generated.

### Verification Candidate

- `npm test -- auth-refresh`
- browser path: login -> expire session -> refresh -> submit
```

这份记录不需要贴完整 rg 输出。保留完整输出反而会制造噪音。真正要留下的是搜索目标、读过的文件、排除理由和验证候选。

如果团队 review 要求更严格，可以把关键命中行也写进去：

清单 10-35　生产里怎么留下搜索证据

```
## Key Hits

- `src/auth/request.ts`: contains 401 handling and refresh call.
- `test/auth-refresh.test.ts`: has refresh success test, no original submit retry test.
```

注意不要写成：

清单 10-36　生产里怎么留下搜索证据

```
I searched around and found auth files.
```

这句话没有复查价值。

10.19　搜索和代码审查怎么连接

好的搜索会降低 review 成本。

reviewer 最关心的不是你搜了多少，而是你为什么只改这些文件。

PR 里可以这样写：

清单 10-37　搜索和代码审查怎么连接

```
## Why These Files

- `test/auth-refresh.test.ts`
  - adds regression coverage for current failure.
- `src/auth/request.ts`
  - only production path that handles 401 refresh and retry.

## Why Not These Files
```

```
- `src/auth/token-storage.ts`
  - storage was verified to contain refreshed token.
- `src/generated/api-client.ts`
  - generated client, inspected only.
- `src/demo/ProfileForm.tsx`
  - demo path, not production route.
```

这比一句“修改了 auth 逻辑”有用得多。

如果 reviewer 问“为什么没改 token storage”，你可以回到证据：Network 显示 token refresh 成功，问题在原请求未重试。这样 review 会围绕事实，而不是围绕感觉。

10.20　搜索协议怎么沉淀成规则

如果你发现自己经常提醒 Agent：

- 先找入口再实现。
- 区分 demo 和 production。
- 先找测试。
- 搜索不到就列原因，不要猜。
- 不要把 generated 文件当修改目标。

这些内容就应该写进长期规则或 Skill。

`AGENTS.md` 里可以放轻量版本：

清单 10-38　搜索协议怎么沉淀成规则

```
## Search Before Editing

- For unfamiliar code, search and list actual files read before editing.
- Distinguish production, test, demo/example, legacy, and generated paths.
- Do not edit generated files unless the task explicitly requests generated output changes.
- If no test is found, state the verification gap before implementation.
- If the real entry is unclear, stop and ask.
```

Skill 里可以放完整流程：关键词生成、搜索、分类、Search Summary、自检、停止条件。

规则越具体，Agent 越不容易把“搜索”理解成随便找几个命中行。

10.21　搜索不是代码索引的替代品

这里讲 `rg` 和 Agent 搜索，不代表所有项目都只靠文本搜索。

大型代码库里，语言服务器、IDE 索引、代码搜索平台、调用图、OpenGrok、Sourcegraph、内部代码平台都很有价值。它们能更准确地找定义、引用、调用关系和跨仓依赖。

但原则不变：工具越强，越要让结果进入 Search Summary。

你可以把搜索动作替换成：

```
- IDE find references
- language server go to definition
- Sourcegraph query
- internal code search
- dependency graph
```

但输出仍然要回答：

- 真实入口是什么？
- 关键实现在哪里？
- 哪些路径不能改？
- 测试和验证在哪里？
- 哪些结论还没证据？

不要把代码索引工具的命中结果直接当答案。它只是更强的证据来源。

10.22　常见问题

为什么不用 IDE 全局搜索就够了？

IDE 搜索也可以。重点不是工具，而是搜索策略。`rg` 的好处是快、可复制、容易放进 Agent 工作流和完成记录里。

搜索输出很多，应该给 AI 全部看吗？

不建议。先分类和摘要。完整输出只有在需要复查时再给。大量无关匹配会制造噪音。

Agent 自己会搜索，我还需要关心命令吗？

需要关心搜索目标、分类要求和证据格式。命令可以由 Agent 执行，但 Agent 会搜索不代表它会按正确优先级搜索。

没有测试目录怎么办？

把“没有测试”写进上下文，并补手动验证路径、接口调用或日志检查。不要让 Agent 把没有测试理解成无需验证。

搜索到多个入口怎么办？

让 Agent 区分生产、测试、demo、legacy、generated。无法确认时先提问，不要直接选一个。

10.23　迁移建议

下次处理陌生任务时，先不要开改。让 Agent 花 5 分钟做 Search Summary。

固定写这五块：

清单 10-39　迁移建议

```
Objective:
Candidate files:
Not relevant initially:
Verification:
Open questions:
```

如果这五块写不出来，说明上下文还没选好。

AI Coding 不是让 Agent 替你乱搜。它应该让搜索结果更快变成可执行、可验证、可审查的任务上下文。

接下来会讲项目规则文件：`AGENTS.md`、`CLAUDE.md`、Copilot instructions 这类文件到底该写什么，哪些规则适合长期保留，哪些只应该放在单次任务里。

10.24　可复查资料

- ripgrep 官方仓库：https://github.com/BurntSushi/ripgrep〔10-1〕
- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔10-2〕
- OpenAI Codex `AGENTS.md` 指南：https://developers.openai.com/codex/guides/agents-md〔10-3〕
- Anthropic Claude Code memory：https://code.claude.com/docs/en/memory〔10-4〕

本章资源

〔10-1〕 https://github.com/BurntSushi/ripgrep
https://github.com/BurntSushi/ripgrep

〔10-2〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔10-3〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

〔10-4〕 https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/memory

第11章 AGENTS.md / CLAUDE.md / Copilot Instructions

AI Coding 里，项目规则文件不是给 AI 写的“温馨提示”。

它的作用更具体：把每次都要遵守的工程约束放到仓库里，让 Agent 在读代码、改文件、跑命令、写完成报告时有稳定边界。

不同工具叫法不一样。Codex 有 `AGENTS.md`，Claude Code 常用 `CLAUDE.md` 承接项目记忆和约定，GitHub Copilot 支持 repository instructions 和自定义指令。文件名和加载规则会随工具变化，写作前要看官方文档；但背后的工程问题一致：

什么规则应该长期放在项目里？什么信息只属于单次任务？规则文件怎样写，才能真正约束 Agent，而不是制造更多噪音？



图 11-1　AI Coding 项目规则分层

11.1 规则文件解决什么问题

没有规则文件时，每次任务都要重复交代：

示例 11-1 规则文件解决什么问题

```
先看测试。
不要改生成文件。
不要新增依赖。
不要做无关重构。
改完跑 npm test。
如果测试失败先分类。
```

重复几次以后，人会省略，Agent 会忘，结果又回到聊天式开发。

项目规则文件把这些稳定约束放进仓库。它解决四类问题：

第一，减少重复说明。

安装命令、测试命令、禁止修改目录、完成报告格式，不应该每次都靠人重新输入。

第二，限制 Agent 的默认行为。

Agent 遇到测试失败、生成文件、依赖缺失、格式化大文件时，如果没有规则，可能会按最容易完成的方向改。

第三，保持工具切换时的一致性。

你可能在 Codex、Claude Code、Cursor、Copilot 之间切换。规则文件能让项目约束留在仓库里，而不是留在某个聊天窗口。

第四，把事故沉淀成防线。

AI 曾经改错过生成文件，就写“不要改 generated”；曾经为了测试通过删断言，就写“不要删除断言来绕过失败”；曾经新增依赖没说明，就写“新增依赖先确认”。

规则文件不是写给未来的理想流程，而是写给真实踩坑后的下一次任务。

11.2 规则文件不应该写什么

规则文件最容易变成垃圾桶。

这些内容不应该长期放进去：

- 单次任务背景。
- 临时需求讨论。
- 某次 bug 的排查过程。

- 已经过期的方案争论。
- 大段产品 PRD。
- 个人猜测。
- “请认真”“代码要优雅”这类空话。
- 工具使用心得流水账。

例如，不要写：

清单 11-2 规则文件不应该写什么

```
- 最近我们在修登录问题，可能和 refresh token 有关。
- 这个项目最好尽量优化用户体验。
- AI 修改代码时要小心。
- 如果有机会，可以顺便重构 auth 模块。
```

这些话要么过期，要么不可执行，要么会诱导 Agent 扩大范围。

长期规则应该满足四个条件：

```
长期有效
可执行
可验证
能减少真实风险
```

如果一条规则不满足这四点，就应该放到任务上下文包、issue、PR 描述或临时计划里。

11.3 好规则和坏规则的区别

坏规则：

清单 11-3 好规则和坏规则的区别

```
- 写高质量代码。
- 使用最佳实践。
- 注意不要出错。
- 尽量提升可维护性。
```

这类规则的问题是没有动作。Agent 看完也不知道该改什么、不该改什么、跑什么命令。

好规则：

清单 11-4 好规则和坏规则的区别

```
- Run `git status --short` before editing.
- Do not edit files under `src/generated/`; update the schema and run generation instead.
- Do not add runtime dependencies without approval.
```

```
- Bug fixes must stay scoped to the failing behavior; do not mix refactor with bug fix.
- For behavior changes, run the focused test first and report the exact command output.
- If a test fails, classify it before editing again: baseline failure, introduced failure,
environment failure, or requirement conflict.
```

这些规则有共同点：

- 能执行。
- 能检查。
- 能阻止常见误改。
- 不依赖抽象价值判断。

规则文件不是价值观文档。它是 Agent 工作边界。

11.4 规则必须绑定证据

一条规则如果没有证据出口，很容易变成口号。

比如：

清单 11-5 规则必须绑定证据

```
- Run tests before completion.
```

这句话看起来没问题，但它没有说清楚跑什么、什么时候跑、怎么报告。Agent 很可能跑一个最轻的命令，然后写“测试通过”。

可以改成：

清单 11-6 规则必须绑定证据

```
- For behavior changes, run one focused test that covers the changed path.
- If shared code changed, run the broader module test.
- Completion report must include exact command, exit result, and unverified paths.
```

这样规则就绑定了证据。

再比如：

清单 11-7 规则必须绑定证据

```
- Keep changes scoped.
```

这句话也太虚。可以改成：

清单 11-8 规则必须绑定证据

```
- Before editing, list expected files.
- If a new file must be edited, stop and explain why scope changed.
- Completion report must mention every changed file and reason.
```

AI Coding 里的规则不是为了表达态度，而是为了改变 Agent 的下一步动作。

11.5 一份项目规则的基本结构

可以从这个结构开始：

清单 11-9 一份项目规则的基本结构

```
# Project Agent Instructions

## Project Shape

- Runtime entry:
- Main modules:
- Test directories:
- Generated directories:

## Commands

- Install:
- Focused test:
- Full test:
- Lint:
- Type check:
- Build:

## Worktree Rules

- Run `git status --short` before editing.
- Do not overwrite user changes.
- Keep changes scoped to the task.

## Change Rules

- Do not add dependencies without approval.
- Do not edit generated files directly.
- Do not mix refactor with behavior change.
- If scope must expand, stop and explain.

## Verification

- Report exact commands and results.
- Explain what each command covers.
- List residual risks before completion.

## Reporting
```

```
- Changed files.
- Why they changed.
- Commands run.
- Unverified paths.
```

这不是固定模板。不同项目可以删减。

文档项目可以不写 build。后端服务要写启动命令和测试命令。前端项目要写浏览器验证。老项目要写手动验证和覆盖缺口。

关键是别把规则文件写得像项目百科。它只需要覆盖 Agent 工作时最容易出错的边界。

11.6 规则分层：全局、项目、目录、任务

AI Coding 的规则不是一层。

可以按四层理解：

表 11-1 规则分层：全局、项目、目录、任务

层级	内容	示例
全局规则	个人长期偏好和工作方式	回复语言、不要覆盖用户改动、验证优先
项目规则	当前仓库长期约束	命令、目录、依赖、完成报告
目录规则	模块差异明显时的局部约束	前端目录、后端目录、移动端目录
任务上下文	单次任务目标、风险、相关文件	这次修什么、不能改什么、怎么验证

项目规则不要承担任务上下文的职责。

比如“这次修 session 过期后首次提交失败”，不应该写进长期规则；它属于任务上下文。

而“不要改变登录接口协议，除非任务明确要求”，如果这是项目长期约束，可以放进规则文件。

目录规则也不要过早拆。只有当目录之间命令、风险和开发方式明显不同，才值得分目录写。例如：

```
frontend/AGENTS.md
backend/AGENTS.md
mobile/AGENTS.md
```

如果只是为了显得细，就别拆。规则层级越多，冲突和过期风险越高。

11.7 规则筛选流程

每次想把一句话写进规则文件前，先过这张流程。

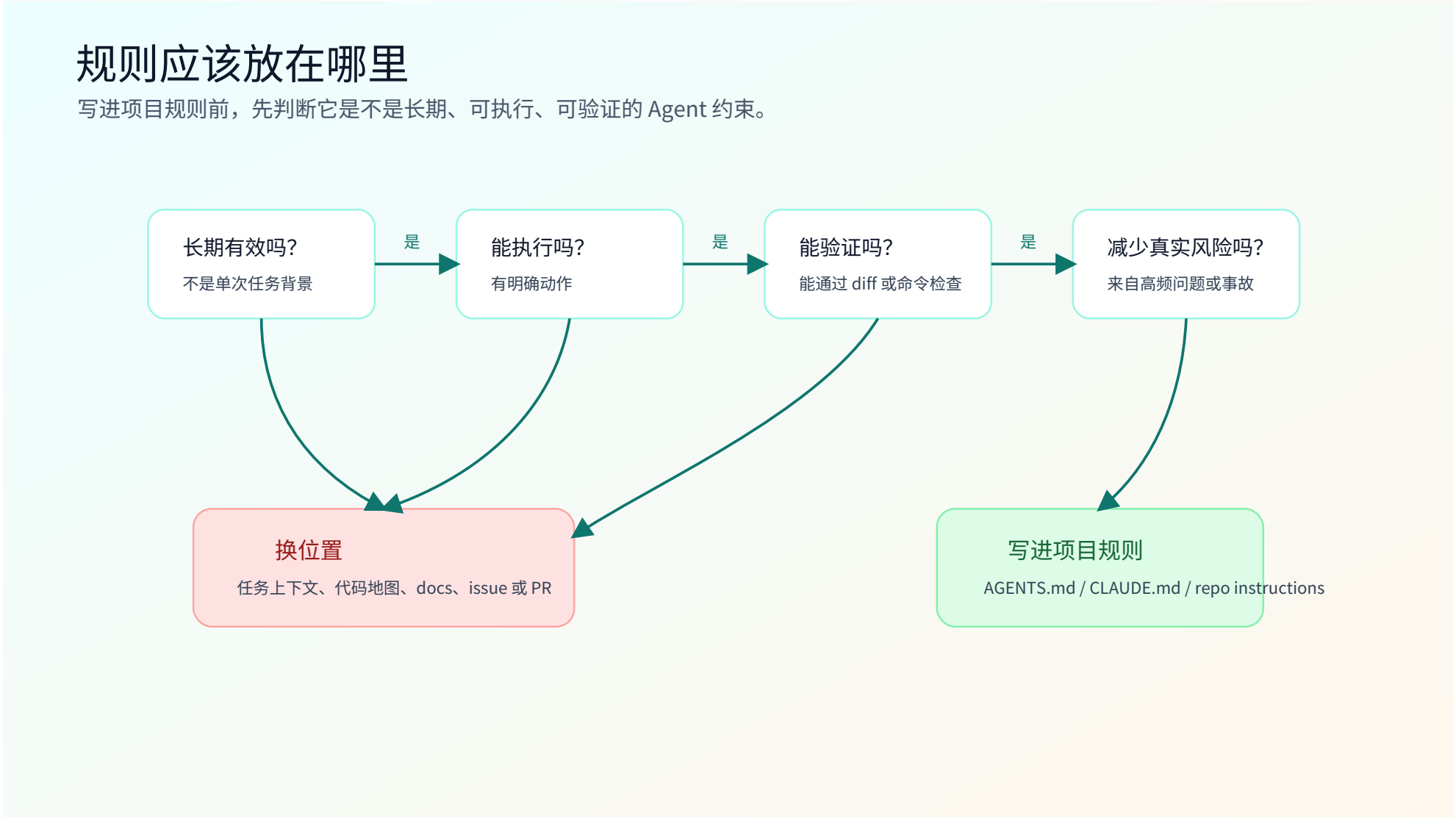


图 11-2　规则应该放在哪里

第一问：这条规则是不是长期有效?

如果只对本次任务有效，放任务上下文。

第二问：它能不能执行?

“代码要优雅”不能执行。“不要混合重构和 bug fix”可以执行。

第三问：能不能验证?

“运行 `npm test -- auth-refresh` 并报告结果”能验证。“保证没有问题”不能验证。

第四问：它是否减少真实风险?

如果只是偏好，不要写太多。规则越多，Agent 越难抓住重点。

第五问：是否已有更合适的位置?

架构说明放 docs，接口契约放 spec，任务细节放 issue，规则文件只放 Agent 工作约束。

11.8 一个可直接迁移的 AGENTS.md

下面是一份精简版本：

清单 11-10 一个可直接迁移的 AGENTS.md

```
# Project Agent Instructions

## Commands

- Install: `npm install`
- Focused test: `npm test -- <pattern>`
- Full test: `npm test`
- Lint: `npm run lint`
- Type check: `npm run typecheck`

## Worktree

- Run `git status --short` before editing.
- Do not overwrite user changes.
- Do not edit generated files under `src/generated/` or build outputs under `dist/`.

## Change Scope

- Keep bug fixes scoped to the failing behavior.
- Do not mix refactor with behavior changes.
- Do not add runtime dependencies without approval.
- If the task requires changing public API, stop and explain the impact first.

## Verification
```

```
- For behavior changes, run the focused test first.
- If shared code changed, run broader tests.
- Report exact commands and results.
- If a command cannot run, explain why and what evidence is missing.

## Failure Handling

- Do not edit code immediately after a test failure.
- First classify the failure: baseline, introduced, environment, or requirement conflict.
- Do not delete tests or weaken assertions to make tests pass.

## Completion Report

- Changed files.
- Why each file changed.
- Commands run.
- Result.
- Residual risk.
```

这份规则不长，但已经能约束很多常见问题。

你可以按项目改：

- Java 项目把命令换成 `./mvnw test` 或 `./gradlew test`。
- Go 项目换成 `go test ./...`。
- 前端项目补浏览器验证。
- 老项目补手动验证路径和覆盖缺口。

11.9 CLAUDE.md 怎么写

Claude Code 的 memory 文档把 `CLAUDE.md` 用作项目记忆和约定入口。写法上，原则和 `AGENTS.md` 很接近：放项目长期有效的开发约束、命令、结构、偏好。

可以这样组织：

清单 11-11 CLAUDE.md 怎么写

```
# Project Memory

## Useful Commands

- `npm test`
- `npm run lint`
- `npm run typecheck`

## Architecture Notes

- Auth refresh logic lives in `src/auth/request.ts`.
- Generated API client lives in `src/generated/`; do not edit directly.
```

```
## Coding Rules

- Keep bug fixes scoped.
- Do not add dependencies without approval.
- Prefer existing test style.

## Verification

- Report commands and residual risk.
```

注意，不要把 `CLAUDE.md` 写成长期聊天记录。

如果一段信息只对当前 bug 有用，放任务上下文；如果是项目长期导航，才放 `CLAUDE.md`。

11.10 Copilot Instructions 怎么写

GitHub Copilot 的 repository instructions 和 custom instructions 适合给 Copilot 提供项目上下文和编码约束。

这里同样不要写大段背景。适合写：

- 项目使用的语言和框架。
- 测试命令和代码风格。
- 常见目录和生成文件。
- 不要新增依赖或改接口的约束。
- PR 或 review 时要重点检查的内容。

例如：

清单 11-12 Copilot Instructions 怎么写

```
This repository is a React + TypeScript application.

Guidelines:
- Use existing component and test patterns.
- Do not edit generated API client files under `src/generated/`.
- Do not add runtime dependencies without approval.
- For behavior changes, include or update focused tests.
- Keep bug fixes scoped; do not include unrelated refactors.
```

Copilot instructions 往往更贴近 IDE 和 GitHub 工作流。它不一定替代 `AGENTS.md`，而是服务 Copilot 使用场景。

如果项目同时使用多种工具，建议先维护一份主规则，再按工具格式同步精简版本。不要让不同工具的规则互相冲突。

同步时还要保留差异。

比如 Codex 场景里，规则更强调仓库读取、命令验证、diff 和完成报告。Claude Code 场景里，`CLAUDE.md` 往往还会承担项目记忆作用。Copilot instructions 更贴近 IDE、PR 和 GitHub 工作流。Cursor rules 可能更贴近编辑器内的代码生成和目录规则。

所以不要把一份长规则原样复制到所有工具里。

合理做法是：

```
主规则：项目事实、禁止事项、验证要求、完成报告。
Codex：保留命令、diff、验证、权限边界。
Claude Code：保留项目记忆、架构入口、命令、长期约束。
Copilot：保留编码风格、测试模式、PR/review 重点。
Cursor：保留目录规则、生成文件限制、当前技术栈模式。
```

核心约束一致，工具细节分开。这样既不会冲突，也不会让某个工具读到大量无关信息。

11.11 多工具规则如何避免冲突

很多程序员会同时用 Codex、Claude Code、Cursor、Copilot。问题不是不能多工具，而是规则不能多套互相打架。

建议建立一个“主规则源”。

```
AGENTS.md 或 docs/ai-coding/project-rules.md
```

然后按工具需要派生：

```
CLAUDE.md
.github/copilot-instructions.md
Cursor rules
```

同步时遵守三点：

第一，核心约束一致。

比如“不要改生成文件”“新增依赖先确认”“bug fix 不混重构”，所有工具都应该一致。

第二，工具特有能力单独写。

Claude Code 的 hooks、Codex 的配置、Copilot 的 PR 场景，可以写在对应文件里，不要硬塞到所有规则。

第三，定期删旧规则。

规则文件不是只增不删。项目结构变了，命令变了，工具能力变了，旧规则要清理。

可以每月做一次 10 分钟检查：

清单 11-13 多工具规则如何避免冲突

```
git grep -n "npm test\\|pnpm test\\|mvn test\\|generated\\|dist/" AGENTS.md CLAUDE.md .github 2>/
dev/null
```

这类命令不是为了机械找词，而是快速发现规则里是否还写着过期命令、过期目录和互相冲突的工具约束。

11.12 规则太多会怎样

规则太多会产生三个问题。

第一，Agent 抓不住重点。

如果规则文件几千行，真正关键的“不要改生成文件”可能被淹没。

第二，规则冲突。

前面写“保持最小 diff”，后面又写“顺便优化代码结构”，Agent 会自己选择。

第三，过期信息污染上下文。

旧命令、旧目录、旧架构说明，会把 Agent 带到错误路径。

所以规则文件要定期做瘦身。

可以用这个检查：

清单 11-14 规则太多会怎样

```
## Rule Review

- 这条规则过去一个月是否仍然有效？
- 是否能被执行或验证？
- 是否和其他规则重复或冲突？
- 是否更适合放到代码地图、任务上下文或 docs？
- 是否来自真实事故或高频风险？
```

删规则和加规则一样重要。

11.13 规则文件如何验证有效

写了规则，不代表 Agent 会稳定遵守。

可以用一个低风险任务试跑：

示例 11-15 规则文件如何验证有效

```
请读取项目规则，然后完成一个小 bug fix。
实现前先输出：
- 你读到了哪些规则。
- 这些规则如何影响你的计划。
- 哪些文件不能改。
- 需要跑哪些命令。
```

如果 Agent 输出里完全没有提到关键规则，说明规则写法或加载方式有问题。

也可以故意观察它是否会踩线：

- 它是否尝试修改 generated。
- 它是否直接新增依赖。
- 它是否把 refactor 混进 bug fix。
- 它是否跑了错误命令。
- 它是否没有报告剩余风险。

这些不是要为难 Agent，而是验证规则是否真的进入工作流。

11.14 规则如何进入一次任务

规则文件写好了，还要让它进入当前任务。

实际操作可以很简单：

示例 11-16 规则如何进入一次任务

```
请先读取项目规则文件，不要改代码。

输出：
- 本任务必须遵守的规则。
- 这些规则会影响哪些文件和命令。
- 哪些动作需要先获得确认。
- 如果规则和任务冲突，先指出冲突，不要自行取舍。
```

这一步适合中高风险任务。低风险任务不用每次都复述规则。

如果 Agent 读完规则后只输出“我会遵守项目规范”，说明规则没有变成执行计划。你要让它具体到文件、命令、限制和停止条件。

例如前端改动：

```
规则影响：
- 不能改 generated client。
- UI 改动需要浏览器验证。
- 只允许修改 `src/pages/order/` 和对应测试。
- 如果需要改 API response shape，先停止。
```

这种输出才有用。

11.15 规则没有生效时怎么排查

规则写了但 Agent 没遵守，先不要急着把规则写得更长。

按顺序查四件事。

第一，工具是否真的加载了规则。不同工具的文件名、位置和作用范围不同，`AGENTS.md`、`CLAUDE.md`、repository instructions 不一定互相识别。先看官方文档和当前工具输出，不要凭印象判断。

第二，规则是否太抽象。比如“保持高质量”不会改变行动，“如果测试失败，先分类 baseline、introduced、environment、requirement conflict”才会。

第三，规则是否冲突。前面写“不要改 public API”，后面又写“必要时调整接口更合理”，Agent 会自己取一个看起来顺手的方向。

第四，任务上下文是否覆盖了规则。一次任务里如果写“可以自由重构”，就会和项目规则里的“小 diff”冲突。冲突时应该先停下来让人确认，而不是让 Agent 自行取舍。

很多时候，修规则不是加一段说明，而是删掉冲突、改成动作、补上验证出口。

11.16 常见错误

错误一：规则文件写成愿望清单。

“提高质量”“保持优雅”“遵循最佳实践”都太虚。换成具体动作。

错误二：把任务背景写进长期规则。

一次 bug 的上下文应该放任务上下文包。长期规则只放稳定约束。

错误三：不写验证命令。

没有命令，Agent 就会猜。至少写 focused test、full test、lint 或手动验证路径。

错误四：规则文件从不更新。

项目命令变了，目录变了，生成文件路径变了，规则也要变。

错误五：多工具规则不一致。

一个工具允许改 generated，另一个工具禁止改 generated，最后一定出问题。核心约束要统一。

错误六：把规则写成给人的项目文档。

架构背景、业务解释、接口历史可以放 docs 或 spec。规则文件只保留 Agent 工作时必须遵守的边界。

错误七：只写禁止，不写替代路径。

不要只写“不能改 generated”。还要写“应该改 schema，然后运行生成命令”。否则 Agent 知道不能做什么，却不知道该怎么做。

11.17 今天可以先加的五条规则

如果你的项目还没有规则文件，先从五条开始：

清单 11-17 今天可以先加的五条规则

```
# Project Agent Instructions

- Run `git status --short` before editing.
- Do not overwrite user changes.
- Do not add dependencies without approval.
- Do not edit generated files or build outputs.
- For behavior changes, run a focused verification command and report the result.
```

然后根据最近一次 AI Coding 事故补一条：

清单 11-18 今天可以先加的五条规则

```
- Do not delete tests or weaken assertions to make tests pass.
```

或者：

清单 11-19 今天可以先加的五条规则

```
- If a fix touches payment, order status, or auth, stop and ask for explicit plan approval first.
```

规则要从真实风险长出来。

11.18 迁移建议

把规则文件落到项目里后，下一次任务先让 Agent 读它：

示例 11-20 迁移建议

```
请先读取项目规则文件。
不要改代码。
```

```
输出：
- 你读到的关键规则。
- 本任务需要遵守的硬约束。
- 计划中哪些动作受这些规则影响。
```

如果它读完规则还能给出合理计划，说明规则文件开始发挥作用。

如果它读完只复述空话，说明规则还不够可执行。

项目规则解决的是长期约束。单次任务还需要自己的上下文包：目标、范围、相关文件、验证命令和开放问题。接下来就讲一次需求到底该给 AI 什么。

11.19 常见问题

`AGENTS.md` 和 `CLAUDE.md` 要不要同时维护？

看工具使用情况。

如果项目主要用 Codex，先维护 `AGENTS.md`。如果主要用 Claude Code，先维护 `CLAUDE.md`。如果两个工具都高频使用，可以维护一份主规则，再同步精简版本。不要为了“兼容所有工具”复制多份没人维护的长文档。

规则文件应该写中文还是英文？

看团队和工具习惯。

中文项目、中文协作可以写中文；工具关键词、命令、路径、错误分类建议保留英文。关键不是语言，而是规则是否具体、可执行、可验证。

规则越详细，Agent 表现越好吗？

不是。

过长规则会占上下文，也会让关键约束被淹没。规则应该少而硬：命令、禁止目录、依赖策略、验证要求、完成报告、失败处理。一次性背景不要放进去。

工具官方规则变化怎么办？

工具能力和加载规则变化很快，涉及 `AGENTS.md`、`CLAUDE.md`、Copilot instructions、hooks、Skill、subagents 时，写作和改项目之前都应该看当前官方文档。不要凭几个月前的印象写死。

规则文件能替代代码审查吗？

不能。

规则文件能减少重复错误，但它不会理解业务取舍。AI 生成的 diff 仍然要审查，尤其是公共接口、测试断言、依赖、数据和权限相关改动。

11.20 可复查资料

- OpenAI Codex `AGENTS.md` 指南：https://developers.openai.com/codex/guides/agents-md〔11-1〕
- Anthropic Claude Code memory：https://code.claude.com/docs/en/memory〔11-2〕
- GitHub Copilot repository instructions：https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions〔11-3〕

本章资源

〔11-1〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

〔11-2〕 https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/memory

〔11-3〕 https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions
https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions

第12章　任务上下文包：一次需求到底该给 AI 什么

一次需求开始前，给 AI 的材料不是越多越好，也不是越少越好。

真正要给的是一份能约束行动的任务上下文包：目标、范围、相关文件、禁止修改、已知证据、验证方式、开放问题和 Agent 授权级别。

很多 AI Coding 失控，不是因为 Agent 不会写代码，而是因为任务开始时缺少合同。

你只说：

```
帮我修一下 session 过期后第一次提交失败的问题。
```

Agent 只能自己猜：是前端表单问题？请求封装问题？token 存储问题？登录接口问题？还是后端返回问题？它猜得快，改得也快，但方向不一定对。

任务上下文包的目的，就是在实现前把这些判断显式化。它可以由 Agent 根据 issue、日志、代码搜索和上下文审计报告生成草稿，但硬约束、业务风险和授权级别必须由程序员确认。



图 12-1　任务上下文包的组成

12.1　任务上下文包不是 PRD

PRD 讲需求背景、用户价值、功能范围。任务上下文包讲一次代码改动怎么安全开始。

它不需要长。中等风险任务，一页就够。

一份有效的任务上下文包要回答八个问题：

表 12-1　任务上下文包不是 PRD

问题	作用
要改变什么行为	防止实现跑偏
哪些行为不能变	防止破坏兼容
先读哪些文件	防止乱搜和漏入口
哪些文件暂时不改	防止误改 demo、legacy、generated
已知证据是什么	区分事实和猜测
怎么证明完成	让验证在实现前出现
还有哪些未确认问题	防止 Agent 自行拍板
Agent 当前授权级别	控制它能否直接编辑

这八个问题比“写一个好 prompt”更重要。

12.2　一个坏输入长什么样

看一个常见输入：

```
登录过期后提交表单失败，你帮我修一下。
不要影响其他功能。
```

它的问题很多：

- “登录过期”没有复现路径。
- “提交表单”没有入口文件。
- “失败”没有错误日志。
- “不要影响其他功能”太空。
- 没有验证命令。
- 没有限制是否可以改登录接口。
- 没说明是否可以新增依赖。

Agent 看到这段话，会按自己搜到的第一条线索行动。

更好的输入是：

清单 12-1　一个坏输入长什么样

```
# Task Context

## Objective

session 过期后重新登录，第一次提交资料表单应该成功，不应出现重复错误 toast。

## Files AI Should Inspect First

- `src/forms/ProfileForm.tsx`: form submit entry.
- `src/auth/request.ts`: HTTP 401 handling.
- `test/auth-refresh.test.ts`: related auth refresh tests.

## Constraints

- Do not change login API contract.
- Do not change token storage format.
- Do not add runtime dependencies.
- Do not swallow non-401 errors.

## Evidence

- Error log: `401 refresh token missing`.
- Repro path: login -> wait for session expiration -> refresh page -> submit form.

## Done When

- `npm test -- auth-refresh` passes.
- `npm run typecheck` passes if request types changed.
- Browser path passes: login -> expire session -> refresh -> submit.

## Agent Mode

First inspect files and produce a plan. Do not edit before plan approval.
```

这份材料并不复杂，但它把 Agent 的行动空间限定住了。

12.3　任务上下文包的八个部分

第一部分：Objective。

目标要写成行为，而不是愿望。

差：

```
优化登录体验。
```

好：

```
session 过期后重新登录，第一次提交资料表单应该成功，不应出现重复错误 toast。
```

行为越具体，验证越容易。

第二部分：Scope。

范围要写候选文件和原因。

清单 12-2　任务上下文包的八个部分

```
## Files AI Should Inspect First

- `src/forms/ProfileForm.tsx`: form submit entry.
- `src/auth/request.ts`: HTTP 401 handling.
- `test/auth-refresh.test.ts`: related tests.
```

不要只列路径。路径后面要写为什么相关。

第三部分：Constraints。

约束要写硬边界。

清单 12-3　任务上下文包的八个部分

```
## Constraints

- Do not change login API contract.
- Do not change token storage format.
- Do not add runtime dependencies.
- Do not edit generated files under `src/generated/`.
```

这比“不要影响其他功能”有效得多。

第四部分：Known Evidence。

证据是已经确认的事实，不是猜测。

清单 12-4　任务上下文包的八个部分

```
## Known Evidence

- Error log: `401 refresh token missing`.
- Current behavior: first submit fails after session refresh.
- Refresh API itself returns 200 in local reproduction.
```

如果只是猜测，要标成假设：

清单 12-5　任务上下文包的八个部分

```
## Hypotheses

- The original request may not be retried after refresh.
- Concurrent refresh may drop pending submit.
```

事实和假设不能混。

第五部分：Verification。

验证要在实现前写。

清单 12-6　任务上下文包的八个部分

```
## Done When

- `npm test -- auth-refresh` passes.
- `npm run typecheck` passes.
- Browser path passes: login -> expire session -> refresh -> submit.
```

如果没有自动测试，也要写手动验证：

清单 12-7　任务上下文包的八个部分

```
- Manual check: Network shows refresh request followed by one retry of original submit.
```

第六部分：Open Questions。

有不确定就写出来。

清单 12-8　任务上下文包的八个部分

```
## Open Questions

- Should original request retry exactly once or follow global retry policy?
- How should concurrent submits behave during refresh?
```

开放问题的价值是阻止 Agent 自己拍板。

第七部分：Agent Mode。

明确它当前能做什么。

清单 12-9 任务上下文包的八个部分

```
## Agent Mode

- Inspect and plan only.
- Do not edit before plan approval.
- If scope must expand, stop and explain.
```

低风险任务可以写：

清单 12-10 任务上下文包的八个部分

```
Agent may implement directly, but must keep diff scoped and report commands.
```

第八部分：Context Sources。

告诉 Agent 先读哪些长期资产。

清单 12-11 任务上下文包的八个部分

```
## Context Sources

- Project rules: `AGENTS.md`
- Code map: `docs/ai-coding/code-map.md`
```

这能把长期规则和单次任务接起来。

12.4 用 Agent 或工具生成草稿

任务上下文包不应该靠人从空白文档开始写。先让 Agent 根据搜索结果生成草稿，或者用配套工具生成骨架。

配套代码仓库 DavidDingXu/learn-ai-coding〔12-1〕 的工具 module 里有一个最小命令：

清单 12-12 用 Agent 或工具生成草稿

```
cd learn-ai-coding/ai-coding-workflow-kit
node src/cli.js task-context \
  --title "修复 session 过期后首次提交失败" \
  --files "src/auth/request.ts,src/forms/ProfileForm.tsx,test/auth-refresh.test.ts"
```

当前输出会是：

清单 12-13 用 Agent 或工具生成草稿

```
# Task Context: 修复 session 过期后首次提交失败

## Goal
```

```
- Describe the behavior change before asking AI to edit code.

## Files AI Should Inspect First

- src/auth/request.ts
- src/forms/ProfileForm.tsx
- test/auth-refresh.test.ts

## Constraints

- List APIs, dependencies, generated files, or behaviors that must not change.

## Done When

- List tests, commands, UI checks, or manual verification evidence.

## Questions Before Coding

- Which existing behavior must remain compatible?
- Which test or command proves the change?
- What should AI do if it needs to expand the diff?
```

这个命令不会替你判断业务，也不会自动知道风险。它的作用是让 Agent 和人都别忘了最关键的几个字段。

复杂项目里，真正重要的是后续确认：

- 目标行为。
- 不能改变的接口和兼容规则。
- 错误日志和复现步骤。
- 验证命令。
- Agent 授权级别。

工具和 Agent 生成草稿，程序员确认工程判断。

12.5 草稿生成后怎么验收

Agent 写出的任务上下文包，不能直接当合同。

你至少要做一次验收。

第一，看 Objective 是否是行为。

“优化登录体验”不合格。“session 过期后重新登录，第一次提交资料表单应该成功”才合格。

行为目标应该能被测试、页面路径、接口响应或日志证明。

第二，看 Files 是否有理由。

下面这种只列路径不合格：

清单 12-14 草稿生成后怎么验收

```
- `src/auth/request.ts`
- `src/forms/ProfileForm.tsx`
```

要写成：

清单 12-15 草稿生成后怎么验收

```
- `src/auth/request.ts`
  - reason: handles HTTP 401 and token refresh.
- `src/forms/ProfileForm.tsx`
  - reason: user-visible submit entry.
```

第三，看 Constraints 是否能阻止越界。

“不要影响其他功能”不合格。“不改变登录 API contract，不改变 token storage format，不吞掉非 401 错误，不新增 runtime dependency”才有约束力。

第四，看 Evidence 是否区分事实和假设。

错误日志、复现路径、命令输出、截图是事实。“可能是 refresh 没重试”是假设。

它们必须分开写。

第五，看 Done When 是否能执行。

“功能正常”不合格。“`npm test -- auth-refresh` 通过，浏览器路径 login -> expire session -> refresh -> submit 通过，Network 显示 refresh 后原请求重试一次”才是可验证完成。

第六，看 Agent Mode 是否明确。

高风险任务不能只写任务上下文，还要写授权边界：

清单 12-16 草稿生成后怎么验收

```
## Agent Mode

Inspect and plan only.
Do not edit files before plan approval.
If scope expands beyond auth request wrapper and profile form submit path, stop.
```

验收可以用下面这段话让 Agent 自检：

示例 12-17　草稿生成后怎么验收

```
请检查这份 Task Context 是否足以授权你进入实现。

逐项回答：
1. Objective 是否能被验证？
2. Files 是否都写了用途和证据？
3. Constraints 是否足够阻止越界？
4. Evidence 和 Hypotheses 是否分开？
5. Done When 是否包含可执行命令或操作路径？
6. Agent Mode 是否明确说明你能不能编辑文件？

如果任一项不满足，请只修订 Task Context，不要写代码。
```

这一步能避免很多“任务包写了，但还是乱改”的问题。

12.6　不同材料怎么合并进任务上下文

一次真实任务的材料经常来自多个地方。

示例 12-18　不同材料怎么合并进任务上下文

```
用户描述
issue 评论
错误日志
截图
代码搜索结果
代码地图
项目规则
官方文档
历史 PR
```

不要把这些材料按原样塞进上下文包。它们的可信度不同，用法也不同。

可以按下面方式合并。

用户描述进入 Objective 和 Open Questions。

用户说“提交失败”，只能说明现象；不能直接说明根因。要把它写成目标和待确认问题：

清单 12-19　不同材料怎么合并进任务上下文

```
## Objective

session 过期后重新登录，第一次提交资料表单应该成功。

## Open Questions

- submit failure is caused by frontend retry, backend 401, or token storage?
```

错误日志进入 Known Evidence。

日志要带上下文，不要只贴一句：

清单 12-20 不同材料怎么合并进任务上下文

```
## Known Evidence

- Browser Network: first submit returns 401.
- Console log: `refresh token missing`.
- Refresh endpoint returns 200 in local reproduction.
```

代码搜索结果进入 Files To Inspect First 和 Files Not To Edit Initially。

清单 12-21 不同材料怎么合并进任务上下文

```
## Files AI Should Inspect First

- `src/auth/request.ts`
  - evidence: contains 401 handling and refresh call.

## Files Not To Edit Initially

- `src/generated/api-client.ts`
  - evidence: generated client; inspect only.
```

项目规则进入 Context Sources 和 Constraints。

如果 `AGENTS.md` 写了不能新增依赖，就不要只在 Context Sources 里引用，要把本任务相关约束拉进 Constraints。

官方文档进入 Reference，不直接覆盖项目事实。

清单 12-22 不同材料怎么合并进任务上下文

```
## Reference

- Framework docs say retry middleware can wrap request calls.
- Current project uses a custom request wrapper; follow current project contract first.
```

历史 PR 进入 Hypotheses 或 Background，不能直接变成当前事实。

清单 12-23 不同材料怎么合并进任务上下文

```
## Background

- Historical PR suggests token storage changed in an earlier refactor.
- Need current code confirmation before editing.
```

合并材料时，最重要的是保持层级。事实、假设、约束、参考资料不能混在一个段落里。

12.7　任务上下文包和权限控制

任务上下文包不是纯文档，它也是权限控制。

同一个需求，不同上下文包会给 Agent 完全不同的权限。

低风险写法：

清单 12-24　任务上下文包和权限控制

```
## Agent Mode

Direct implement allowed.
Keep diff limited to `src/pages/invoice/ListPage.tsx`.
Run focused UI test or provide screenshot.
```

中风险写法：

清单 12-25　任务上下文包和权限控制

```
## Agent Mode

Plan first.
Human approves plan before editing.
Implement one step at a time.
```

高风险写法：

清单 12-26　任务上下文包和权限控制

```
## Agent Mode

Analysis only.
Do not edit files.
Do not suggest production execution commands.
Output confirmed facts, hypotheses, and next verification action.
```

这里要明确一点：Agent Mode 不是礼貌提示，是任务边界。

如果 Agent 在 Inspect only 模式下开始改文件，这不是“它很主动”，而是违反任务合同。应该停下来回滚或让它解释原因。

12.8　什么时候需要升级到 SDD 或 OpenSpec

任务上下文包适合一次改动开始前的约束，但不是所有复杂需求都应该停留在一页上下文里。

下面这些信号说明该升级：

- 需求会持续多天。
- 需要改多个模块或多个仓库。

- 涉及公开 API、状态机、权限、金额、数据迁移。
- 需要多人 review。
- 需要把当前行为和未来行为同时记录下来。
- 任务会拆成多个可并行或可排期的小步骤。

这时要从 Task Context 升级到规格驱动：

```
requirements.md 讲清楚用户行为和验收条件。
design.md 讲清楚技术取舍、兼容性和数据流。
tasks.md 把实现拆成可验证的小 diff。
OpenSpec changes/ 把拟议变更和当前系统行为分开管理。
```

不要把超大需求硬塞进一个任务上下文包。上下文包是启动合同，不是完整需求治理工具。

12.9 不同风险任务怎么调整上下文包

低风险任务可以很短。

清单 12-27 不同风险任务怎么调整上下文包

```
# Task Context

## Objective

把发票列表空状态文案从“暂无数据”改成“暂无发票”。

## Files

- `src/pages/invoice/ListPage.tsx`

## Constraints

- 不改查询逻辑。
- 不改组件结构。

## Done When

- `git diff` only shows copy change.
- 页面空状态显示正确。
```

中等风险任务要完整。

清单 12-28 不同风险任务怎么调整上下文包

```
# Task Context

## Objective
```

```
session 过期后重新登录，第一次提交资料表单应该成功。

## Agent Mode

Inspect and plan first. Human approves plan before editing.
```

高风险任务要更严格。

清单 12-29　不同风险任务怎么调整上下文包

```
# Task Context

## Objective

分析订单 paid 状态偶发回退原因。

## Agent Mode

Analysis only. Do not edit files.

## Hard Constraints

- Do not change order status transition logic.
- Do not alter payment callback behavior.
- Do not write migration scripts.

## Required Output

- Confirmed facts.
- Excluded hypotheses.
- Remaining hypotheses.
- Next smallest verification action.
- Questions for business owner.
```

同样叫任务上下文包，风险不同，授权级别不同。

12.10　Agent Mode 是很多人漏掉的关键

任务上下文包里一定要写 Agent Mode。

常用模式有四种：

表 12-2　Agent Mode 是很多人漏掉的关键

模式	用法
Inspect only	只读文件、搜索、整理事实
Plan first	先给计划，人确认后再实现
Implement one step	只执行计划中的一小步

模式	用法
Direct implement	低风险任务可直接实现

例如：

清单 12-30 Agent Mode 是很多人漏掉的关键

```
## Agent Mode

Inspect only. Do not edit files.
```

或者：

清单 12-31 Agent Mode 是很多人漏掉的关键

```
## Agent Mode

Implement step 1 only. Stop after diff summary and verification result.
```

没有 Agent Mode，Agent 会默认自己决定行动级别。

这在低风险任务里可能没问题，在中高风险任务里很危险。

12.11 任务上下文包如何和代码地图配合

代码地图是长期导航，任务上下文包是单次任务合同。

代码地图告诉 Agent：

```
auth 模块在哪里，测试在哪里，generated 不要改。
```

任务上下文包告诉 Agent：

```
这次只修 session 过期后首次提交失败，先读这三个文件，不改 token 存储。
```

任务上下文包应该引用代码地图：

清单 12-32 任务上下文包如何和代码地图配合

```
## Context Sources

- Code map: `docs/ai-coding/code-map.md`
- Project rules: `AGENTS.md`
```

但不要复制整份地图。复制会造成过期和冲突。

12.12 任务上下文包如何和项目规则配合

项目规则写长期约束：

清单 12-33 任务上下文包如何和项目规则配合

```
- Do not add dependencies without approval.
- Do not edit generated files.
- Run focused tests for behavior changes.
```

任务上下文包写本次约束：

清单 12-34 任务上下文包如何和项目规则配合

```
- Do not change login API contract.
- Do not change token storage format.
- Verify expired-session submit path.
```

如果任务上下文和项目规则冲突，要先停下来。

可以在任务包里写：

清单 12-35 任务上下文包如何和项目规则配合

```
## Conflict Handling

If task instructions conflict with project rules, stop and explain the conflict before editing.
```

这条规则能防止 Agent 自己在冲突里选一边。

12.13 任务上下文包放在哪里

不用固定。

可以放：

- issue 描述。
- PR 描述。
- 本地 `docs/ai-coding/tasks/<date>-<slug>.md`。
- 任务系统评论。
- Agent 对话第一条消息。

关键是它要能被复查。

如果是小任务，直接放对话里就行。 如果是中高风险任务，建议落到文件或 issue 里。 如果任务会拆多天，建议放仓库内文档，方便换对话和复盘。

一个轻量目录可以是：

```
docs/ai-coding/tasks/
├── auth-refresh-submit.md
└── order-status-rollback.md
```

任务结束后，不一定长期保留所有任务包。重要任务可以归档，普通任务可以只把规则和代码地图更新回长期资产。

12.14 常见错误

错误一：只写目标，不写不能改什么。

AI 很容易为了解决一个点改动接口、依赖或状态结构。约束必须写。

错误二：只写相关文件，不写为什么相关。

路径后面写一句用途。否则 Agent 不知道优先级。

错误三：把猜测写成事实。

“可能是 refresh 没重试”要写到 Hypotheses，不要写到 Known Evidence。

错误四：实现前不写验证。

验证必须前置。否则实现完成后，Agent 会挑最容易通过的命令证明自己。

错误五：不给 Agent 授权级别。

同一份上下文，Inspect only 和 Direct implement 的风险完全不同。

12.15 一份高质量任务上下文包

下面是一份可以直接迁移的版本：

清单 12-36 一份高质量任务上下文包

```
# Task Context: expired session first submit failure

## Objective

After session expiration and re-login, the first profile form submit should succeed.
No duplicate error toast should appear.

## Context Sources

- Project rules: `AGENTS.md`
- Code map: `docs/ai-coding/code-map.md`

## Files AI Should Inspect First
```

```
- `src/forms/ProfileForm.tsx`
  - form submit entry
- `src/auth/request.ts`
  - HTTP 401 handling and retry behavior
- `test/auth-refresh.test.ts`
  - existing auth refresh tests

## Files Not To Edit Initially

- `src/auth/token-storage.ts`
  - token storage format is out of scope
- `src/generated/api-client.ts`
  - generated file

## Constraints

- Do not change login API contract.
- Do not change token storage format.
- Do not add runtime dependencies.
- Do not swallow non-401 errors.

## Known Evidence

- Error log: `401 refresh token missing`.
- Repro path: login -> wait for expiration -> refresh page -> submit form.

## Hypotheses

- Original request may not be retried after refresh.
- Pending submit may be dropped during refresh.

## Done When

- Add or update regression test for expired-session submit.
- `npm test -- auth-refresh` passes.
- `npm run typecheck` passes if request types change.
- Browser path passes: login -> expire session -> refresh -> submit.

## Open Questions

- Should original request retry exactly once or follow global retry policy?
- How should concurrent submits behave during refresh?

## Agent Mode

Inspect and plan first. Do not edit before plan approval.
If scope must expand, stop and explain why.
```

这份材料已经足够让 Agent 开始侦察，但还没有授权它直接改。

12.16　完成后怎么回收上下文

任务结束后，任务上下文包不要原封不动塞进长期规则。

要做一次回收：

清单 12-37 完成后怎么回收上下文

```
## Reuse After Completion

- 是否新增项目规则？
- 是否更新代码地图？
- 是否沉淀验证命令？
- 是否记录常见失败分类？
- 是否有可复用测试模式？
```

例如这次任务结束后，可能只需要沉淀两条：

清单 12-38 完成后怎么回收上下文

```
AGENTS.md:
- Do not swallow non-401 errors when changing auth retry logic.

code-map.md:
- Auth refresh tests: `npm test -- auth-refresh`
```

任务包本身可以归档，但长期资产只吸收稳定规则。

12.17 任务上下文包怎么进入 PR

很多团队的问题不是没有上下文，而是上下文只留在 AI 对话里。

如果这次改动需要 review，任务上下文包应该至少有一部分进入 PR 描述。

可以压缩成下面这种格式：

清单 12-39 任务上下文包怎么进入 PR

```
## Task

session 过期后重新登录，第一次提交资料表单应该成功。

## Constraints

- 不改变登录 API contract。
- 不改变 token storage format。
- 不吞掉非 401 错误。

## Files

- `test/auth-refresh.test.ts`: regression coverage.
- `src/auth/request.ts`: 401 refresh and retry path.

## Verification
```

```
- `npm test -- auth-refresh`: passed.
- `npm run typecheck`: passed.
- Browser path: login -> expire session -> refresh -> submit, passed.

## Residual Risk

- 没有验证第三方 SSO staging path。
```

这不是把任务上下文包完整复制进 PR，而是把 reviewer 必须知道的边界带过去。

reviewer 看到这几段，就能知道：

- 这次要改什么。
- 哪些行为不能改。
- 为什么只改这些文件。
- 用什么证明结果。
- 哪些风险还没覆盖。

AI 生成的 PR 最怕“代码很多，解释很少”。任务上下文包能把解释补回来。

12.18 任务上下文包和 issue 的关系

issue 通常描述问题，任务上下文包描述如何安全开始改代码。

一个 issue 可能是：

清单 12-40 任务上下文包和 issue 的关系

```
用户反馈 session 过期后重新登录，第一次提交资料表单失败。
```

它还不是任务上下文包。你还需要补：

清单 12-41 任务上下文包和 issue 的关系

```
## Engineering Context

- suspected path:
- constraints:
- known evidence:
- verification:
- agent mode:
```

如果 issue 本身写得很好，可以直接把任务上下文包放在 issue 描述里。如果 issue 很长、讨论很多，建议在开始开发前由 Agent 生成一个精简 Task Context，并由人确认。

不要让 Agent 直接读取几十条 issue 评论后开始实现。issue 评论里常常混有旧猜测、过期截图、被否定方案和无关讨论。先收敛成任务上下文包。

12.19 任务上下文包的反例

下面这份看起来完整，但不合格：

清单 12-42 任务上下文包的反例

```
# Task Context

## Goal

修复登录问题，提升用户体验。

## Files

- `src/auth/`
- `src/forms/`

## Constraints

- 不要影响其他功能。

## Done When

- 功能正常。

## Agent Mode

请谨慎修改。
```

问题很具体：

- Goal 不是可验证行为。
- Files 是目录，不是入口和证据。
- Constraints 不可执行。
- Done When 没有命令或操作路径。
- Agent Mode 没有权限边界。

把它改成合格版本：

清单 12-43 任务上下文包的反例

```
# Task Context

## Objective

session 过期后重新登录，第一次提交资料表单应该成功，不出现重复错误 toast。

## Files AI Should Inspect First

- `src/forms/ProfileForm.tsx`
```

```
  - form submit entry.
- `src/auth/request.ts`
  - HTTP 401 handling and retry behavior.
- `test/auth-refresh.test.ts`
  - existing refresh tests.

## Constraints

- Do not change login API contract.
- Do not change token storage format.
- Do not add runtime dependencies.
- Do not swallow non-401 errors.

## Done When

- Regression test covers expired-session first submit.
- `npm test -- auth-refresh` passes.
- Browser Network shows refresh followed by one retry of original submit.

## Agent Mode

Inspect and plan first. Do not edit before plan approval.
```

这才是一份可以授权 Agent 做侦察和计划的上下文。

12.20　上下文包过期了怎么办

任务拖得越久，上下文包越容易过期。

常见过期信号：

- 计划里的文件已经被别人改过。
- 任务目标发生变化。
- 测试命令变化。
- Agent 发现真实入口不是原来写的入口。
- 新证据排除了原来的主要假设。

上下文包过期时，不要直接在原文下面继续追加。先更新关键段落：

清单 12-44　上下文包过期了怎么办

```
## Context Update

Date:

Changed:
- Original hypothesis `token storage missing` is excluded.
- New confirmed fact: original submit request is not replayed.
```

```
Current Next Action:
- Add retry regression test before implementation.
```

如果变化很大，就生成新版本。不要让 Agent 同时读旧上下文和新上下文，然后自己判断哪个优先。

上下文包不是越多越好。它必须保持当前有效。

12.21 常见问题

每个任务都要写任务上下文包吗？

不用。低风险小改可以写很短，甚至只写对话里的几行。中高风险任务建议写完整。

任务上下文包是不是会降低速度？

低风险任务可能会。但中高风险任务里，它减少的是返工和误改。写 5 分钟上下文，通常比清理一个大 diff 便宜。

可以让 AI 帮我写任务上下文包吗？

应该优先让 AI 写草稿。它可以根据 issue、日志、代码搜索结果和上下文审计报告整理材料。程序员不要把草稿当最终合同，重点确认硬约束、业务风险、验证命令和授权级别。

任务上下文包和 SDD 有什么区别？

任务上下文包更轻，适合一次改动开始前的上下文和约束。SDD 会进一步把需求、设计和任务拆成规格驱动流程，适合更复杂需求。

任务包里要不要贴代码片段？

尽量贴路径和搜索结果摘要，不要大段复制代码。让 Agent 去真实仓库读文件。只有外部系统、日志或无法访问材料才需要贴片段。

12.22 可以先实践的一件事

下次中等风险任务开始前，先写这五行：

清单 12-45 可以先实践的一件事

```
Objective:
Files AI should inspect first:
Constraints:
Done when:
Agent mode:
```

如果五行写不出来，就不要让 AI 实现。先让它帮你做侦察和澄清。

任务上下文包的价值，是让 AI 在写代码前知道自己处在哪个边界里。项目规则解决长期约束，代码地图解决导航，任务上下文包解决本次任务合同。

接下来会讲上下文污染与上下文清理。一次任务推进过程中，旧假设、失败尝试和无关材料会不断进入对话；如果不清理，Agent 很容易把已经被否定的内容当成事实继续执行。

12.23　可复查资料

- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔12-2〕
- OpenAI Codex `AGENTS.md` 指南：https://developers.openai.com/codex/guides/agents-md〔12-3〕
- Anthropic Claude Code memory：https://code.claude.com/docs/en/memory〔12-4〕
- GitHub Copilot repository instructions：https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions〔12-5〕

本章资源

〔12-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔12-2〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔12-3〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

〔12-4〕 https://code.claude.com/docs/en/memory
https://code.claude.com/docs/en/memory

〔12-5〕 https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions
https://docs.github.com/en/copilot/how-tos/copilot-on-github/customize-copilot/add-custom-instructions/add-repository-instructions

第13章　上下文污染与上下文清理

AI Coding 做到一半，最危险的不是 Agent 不知道，而是它以为自己知道。

排障开始时怀疑是缓存，后来日志证明不是；第一版计划准备改请求封装，后来发现真实入口在表单状态；前几轮测试失败是环境问题，后来又出现本次引入失败。对话越长，里面混入的猜测、旧方案、被否定结论、无关文件越多。

如果不清理，Agent 会继续被这些材料影响。

上下文污染不是抽象问题。它会直接导致误改文件、扩大范围、重复排错、把旧失败当成本次责任，甚至把已经排除的假设重新写进代码。

程序员要学会在合适的节点让 Agent 停下来，重新整理事实。



图 13-1　上下文污染来源

13.1 上下文污染从哪里来

第一类污染：未验证假设。

比如：

```
可能是 refresh token 没写入。
可能是后端返回慢。
可能是表单重复提交。
```

这些话在排障早期有价值，但它们不是事实。如果后来没有被分类，Agent 很容易把“可能”当成实现依据。

第二类污染：被否定的旧结论。

日志已经证明 refresh token 写入成功，但对话里保留了大量“token 没写入”的讨论。后续 Agent 仍可能回到 token storage 方向。

第三类污染：过期计划。

第一版计划说改 `src/auth/request.ts`，后面发现真实问题在 `src/forms/ProfileForm.tsx`。如果计划没更新，Agent 可能继续围绕旧文件工作。

第四类污染：无关文件。

搜索时读了 demo、legacy、generated、docs，它们进入上下文后，Agent 后面可能继续引用。尤其是 demo 和 docs，看起来很像生产逻辑。

第五类污染：工具输出混杂。

测试输出、lint 输出、环境报错、旧失败、本次失败混在一起，没有分类。Agent 看到红色输出就开始修，结果可能修错层。

第六类污染：人类随口补充。

“应该就是前端问题”“可能顺手重构一下”“这个模块本来就有点乱”。这些话如果不加边界，会改变 Agent 的目标。

上下文污染的本质，是事实、假设、偏好、旧结论、无关材料混在一个窗口里。

13.2 污染出现时的信号

出现下面这些信号，就该清理上下文。

第一，Agent 开始引用已经被排除的原因。

比如你已经确认不是缓存，它又说“我们可以清缓存试试”。

第二，Agent 的计划和最新证据不一致。

最新日志指向表单状态，它的计划还在改请求封装。

第三，它无法解释为什么改某个文件。

它只说“这个文件相关”，但说不出入口、调用点和验证命令。

第四，测试失败后它直接继续改。

没有先判断是 baseline、introduced、environment 还是 requirement conflict。

第五，diff 越来越大。

小 bug 变成跨模块重构，说明目标和边界已经被污染。

第六，完成报告没有未验证风险。

它把当前可见材料当成完整事实，说明上下文里缺少不确定性分类。

这些信号出现时，不要继续喂更多上下文。先停。

13.3 清理上下文的核心动作

清理不是把对话删掉，也不是让 Agent 重新总结一遍。

清理要把当前材料分成五类：

```
已确认事实
已排除假设
未验证假设
当前硬约束
下一步最小动作
```

可以直接这样要求：

示例 13-1 清理上下文的核心动作

```
停止修改。

请只基于当前证据整理上下文：
1. 已确认事实。
2. 已排除假设。
3. 未验证假设。
4. 当前硬约束。
5. 当前 diff 涉及哪些文件。
6. 下一步最小验证动作。
```

```
不要继续写代码。
不要把猜测写成事实。
```

这段话很适合在排障、测试失败、计划变化、大 diff 之前使用。

整理后的输出应该像这样：

清单 13-2 清理上下文的核心动作

```
## Confirmed Facts

- First submit after session expiration returns 401.
- Refresh request succeeds and returns a new token.
- Original submit request is not retried.
- Current diff only changes `test/auth-refresh.test.ts`.

## Excluded Hypotheses

- Token storage is not empty after refresh.
- Backend submit endpoint is not returning 500.

## Unverified Hypotheses

- Request wrapper does not replay original request after refresh.
- Concurrent submits may be dropped during refresh.

## Constraints

- Do not change login API contract.
- Do not change token storage format.
- Do not swallow non-401 errors.

## Next Smallest Action

- Read `src/auth/request.ts` and add one retry guard test before implementation.
```

这就是干净上下文。

13.4 清理流程图

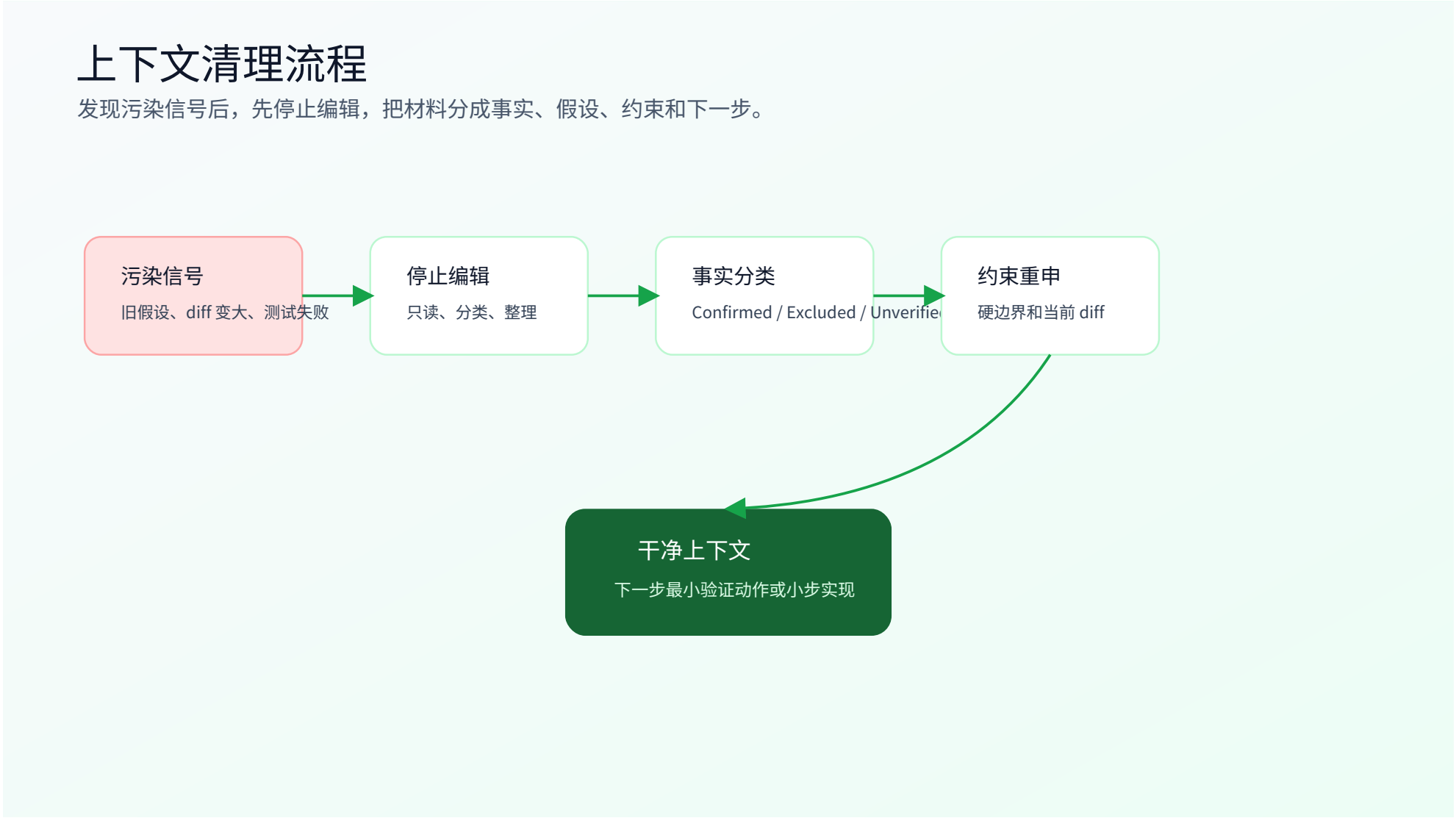


图 13-2　上下文清理流程

这个流程有三个关键点。

第一，先停止编辑。

只要还在继续改文件，污染就会扩散。清理时禁止写代码。

第二，先分类，再决策。

不要一边整理一边提出修复方案。先把事实和假设分清。

第三，下一步必须小。

清理后的下一步不应该是“大概重构一下”。它应该是一个最小验证动作或一小步实现。

13.5 什么时候应该开新对话

不是每次污染都要开新对话。

这些情况可以在当前对话里清理：

- 任务仍然是同一个。
- diff 范围还小。
- 主要问题是测试失败分类。
- 只需要剔除一两个旧假设。

这些情况建议开新对话：

- 对话已经很长。
- 早期猜测大量存在。
- 任务从排障转成实现。
- 任务从实现转成重构。
- 多个失败尝试混在一起。
- Agent 反复回到已排除假设。

开新对话时，不要把原对话全部复制过去。

带一份交接包：

清单 13-3 什么时候应该开新对话

```
# Handoff Context

## Objective
```

```
session 过期后重新登录，第一次提交资料表单应该成功。

## Confirmed Facts

- ...

## Excluded Hypotheses

- ...

## Current Diff

- `test/auth-refresh.test.ts`: added failing regression test.

## Constraints

- ...

## Next Action

- Inspect `src/auth/request.ts` and propose minimal implementation plan.

## Do Not Use

- Do not continue the earlier token-storage hypothesis.
- Do not modify login API contract.
```

新对话需要的是整理后的事实，不是原始聊天记录。

13.6 失败分类是上下文清理的一部分

测试失败后，最容易污染。

一个命令失败，可能有很多原因：

表 13-1 失败分类是上下文清理的一部分

类型	说明	处理
baseline failure	改前就失败	记录基线，不作为本次引入
introduced failure	本次 diff 引入	回到最小 diff 修复
environment failure	依赖、服务、密钥、端口问题	不改业务代码，先处理环境
requirement conflict	测试期望和需求冲突	停下来确认业务规则
coverage gap	命令通过但没覆盖关键路径	补验证，不宣布完成

可以直接给 Agent：

示例 13-4 失败分类是上下文清理的一部分

```
测试失败后不要改代码。

请先分类：
- baseline failure
- introduced failure
- environment failure
- requirement conflict
- coverage gap

说明证据和下一步最小动作。
```

如果没有这一步，Agent 很可能把环境问题改成业务代码，或者把旧失败算成自己要修的内容。

13.7 一个真实清理案例

看一个更完整的过程。

任务是：

```
session 过期后重新登录，第一次提交资料表单失败。
```

第一轮，Agent 做了侦察：

清单 13-5 一个真实清理案例

```
Files read:
- `src/auth/request.ts`
- `src/forms/ProfileForm.tsx`
- `test/auth-refresh.test.ts`
```

它的初始假设是：

清单 13-6 一个真实清理案例

```
Hypothesis:
- refresh token may not be written to storage.
```

于是它准备改 `src/auth/token-storage.ts`。

这时你让它先看日志和 Network。结果发现：

清单 13-7 一个真实清理案例

```
Evidence:
- refresh endpoint returns 200.
- new access token appears in memory state.
- original submit request is not replayed.
```

这个证据已经排除了“refresh token 没写入”的主要方向。可是如果不清理上下文，前面关于 token storage 的讨论仍然留在对话里。Agent 后续可能继续建议改 storage。

这时应该停下来：

```
停止修改。
请重新整理上下文。
把 token storage 相关判断按证据分类，不要继续沿旧假设实现。
```

整理结果应该变成：

清单 13-8 一个真实清理案例

```
## Confirmed Facts

- Refresh request succeeds.
- New access token is available after refresh.
- Original submit request is not replayed.

## Excluded Hypotheses

- Token storage is empty after refresh.
- Refresh endpoint itself fails.

## Unverified Hypotheses

- Request wrapper refreshes token but does not retry the original request.
- Submit handler may drop the pending request while refresh is running.

## Active Constraints

- Do not change token storage format.
- Do not change login API contract.
- Do not swallow non-401 errors.

## Next Smallest Action

- Add regression test for retrying original submit after successful refresh.
```

注意这个整理做了三件事。

第一，它把 token storage 从候选修改路径里移出。不是因为它一定无关，而是当前证据不支持先改它。

第二，它把下一步从“修 storage”改成“补 retry regression test”。这让任务回到可验证路径。

第三，它保留了未验证假设。比如 submit handler 是否 drop pending request，还没有证据，不能当事实。

这就是上下文清理的价值。它不是写总结，而是改变下一步动作。

13.8 清理前后对比

污染状态下，Agent 的计划可能是：

清单 13-9 清理前后对比

```
Plan:
1. Refactor token storage to ensure refresh token is saved.
2. Update request wrapper to read token from storage.
3. Adjust ProfileForm submit error handling.
4. Update tests.
```

看起来完整，但风险很高。它把三个方向混在一起：storage、request wrapper、form submit。任何一个方向错了，diff 都会变大。

清理后，计划应该变成：

清单 13-10 清理前后对比

```
Plan:
1. Add failing test: refresh succeeds, original submit retries once.
2. If test fails as expected, update request wrapper only.
3. Run focused auth-refresh test.
4. Use browser Network to confirm refresh followed by original submit retry.
```

差别在于：清理后的计划只保留当前证据支持的最小路径。

一个程序员看 AI 的计划，不要只看它有没有步骤。要看每一步是否来自已确认事实，是否避开了已排除假设，是否能用命令或操作验证。

13.9 上下文清理和 Git 的关系

清理上下文时，也要清理工作区认知。

不是让 Agent 直接回滚，而是先看清楚当前 diff。

清单 13-11 上下文清理和 Git 的关系

```
git status --short
git diff --stat
git diff
```

这三条命令分别回答：

- 工作区有哪些改动。
- 哪些文件改动最大。
- 具体改了什么。

然后让 Agent 做 diff 分类：

示例 13-12 上下文清理和 Git 的关系

```
请基于当前 git diff 分类：
1. Directly related to current task.
2. Possibly related but not yet proven.
3. Refactor or cleanup.
4. Out of scope.
5. Should not decide without human review.

不要执行 git checkout、reset 或删除文件。
```

这里必须强调：不要让 Agent 擅自 `git checkout -- .` 或 `git reset --hard`。真实工作区里可能有用户改动、上一轮已经确认的改动，或者其他任务残留。上下文清理先做分类，不做破坏性恢复。

如果需要收缩 diff，可以让 Agent 输出建议：

清单 13-13 上下文清理和 Git 的关系

```
## Diff Shrink Proposal

Keep:
- `test/auth-refresh.test.ts`
  - regression test for current bug.
- `src/auth/request.ts`
  - minimal retry-after-refresh implementation.

Move to separate task:
- `src/auth/error-message.ts`
  - copy cleanup, not required for current bug.

Discard candidate:
- `src/auth/token-storage.ts`
  - token storage hypothesis was excluded by Network evidence.
```

程序员确认后，再决定怎么处理这些文件。

13.10 上下文清理和 review 的关系

如果一段 AI 生成的 PR 很难 review，通常不是 review 能力不够，而是上下文没有清理。

PR 里应该能看到：

清单 13-14 上下文清理和 review 的关系

```
## Confirmed Facts

## Excluded Hypotheses

## Change Scope
```

```
## Verification

## Residual Risk
```

如果 PR 描述只有：

```
修复 session 过期后提交失败问题。
```

reviewer 就要从 diff 里反推上下文。这会非常低效。

你可以在 review 前要求 Agent：

示例 13-15 上下文清理和 review 的关系

```
请根据当前 diff 生成 review handoff。
必须包含：
- 本次任务目标。
- 已确认事实。
- 已排除假设。
- 为什么这些文件被修改。
- 验证命令和结果。
- 未覆盖风险。
```

这份 handoff 可以直接放进 PR 描述，也可以作为自己 review 的检查材料。

大厂代码审查里很重要的一点是：reviewer 不应该只看代码，还要看变更意图、验证证据和风险边界。AI Coding 也是同一套要求，只是更需要显式写出来。

13.11 diff 变大时如何收缩

上下文污染常常表现为 diff 变大。

你可以让 Agent 做一次 diff 审查：

示例 13-16 diff 变大时如何收缩

```
停止继续实现。

请基于当前 diff 做范围审查：
1. 哪些改动直接服务本任务？
2. 哪些改动是重构、清理或顺手修改？
3. 哪些文件不应该在本任务里改？
4. 如果要收缩到最小 diff，应该保留什么、撤回什么？

不要执行撤回动作，只输出建议。
```

注意最后一句：不要执行撤回动作。

因为工作区里可能有用户改动，Agent 不应该擅自还原。先让它分类，再由人决定怎么处理。

diff 收缩的目标不是让代码更少，而是让每一行都能追溯到本次任务。

13.12 事实整理表

可以把下面这个表放进任务记录里：

清单 13-17 事实整理表

```
# Context Cleanup

## Confirmed Facts

-

## Excluded Hypotheses

-

## Unverified Hypotheses

-

## Active Constraints

-

## Current Diff

-

## Verification State

- Commands passed:
- Commands failed:
- Not run:

## Next Smallest Action

-
```

这张表最适合三个节点使用：

- 长对话中段。
- 测试失败后。
- 准备开新对话前。

13.13 别把总结当清理

很多 Agent 会给出漂亮总结：

```
我们已经分析了问题，并确定需要优化请求重试逻辑。接下来将完善 auth 模块并增强错误处理。
```

这不叫清理。

清理必须可复查：

- 哪些事实有证据。
- 哪些假设已排除。
- 哪些还没验证。
- 当前 diff 是什么。
- 下一步为什么最小。

如果总结里没有分类，就让它重写：

```
不要写概括性总结。
请按 Confirmed / Excluded / Unverified / Constraints / Next Action 分类。
```

13.14 长任务里的节奏控制

中等以上任务建议固定三个清理点。

第一个清理点：侦察后。

```
读完入口、实现和测试后，整理已确认事实和缺失上下文。
```

第二个清理点：第一次验证失败后。

```
分类失败类型，不直接继续改。
```

第三个清理点：完成前。

```
整理命令、diff、未验证风险和需要沉淀的规则。
```

如果任务跨天，开新对话前再做一次交接包。

13.15 常见错误

错误一：继续补更多上下文。

污染时继续塞材料，常常让问题更乱。先分类，再补材料。

错误二：把猜测整理成事实。

“应该是请求没重试”要放 Unverified，不要放 Confirmed。

错误三：清理时继续改代码。

清理阶段只读、分类、计划。编辑会扩大不确定性。

错误四：开新对话带原始聊天记录。

原始聊天记录里有大量噪音。要带整理后的交接包。

错误五：完成前不写剩余风险。

没有剩余风险，通常说明验证边界没有说清。

13.16 一套可复用提示

排障中段：

示例 13-18 一套可复用提示

```
停止修改。
请把当前上下文整理成：
- Confirmed Facts
- Excluded Hypotheses
- Unverified Hypotheses
- Active Constraints
- Next Smallest Action
不要写代码。
```

测试失败后：

示例 13-19 一套可复用提示

```
测试失败后不要改代码。
先分类失败类型，并说明证据：
- baseline
- introduced
- environment
- requirement conflict
- coverage gap
```

diff 变大时：

示例 13-20 一套可复用提示

```
请审查当前 diff：
- 哪些改动服务本任务？
- 哪些是无关重构或清理？
```

```
- 哪些文件不应该改?
- 如何收缩到最小 diff?
不要执行撤回。
```

开新对话前：

示例 13-21 一套可复用提示

```
请生成新对话交接包:
- Objective
- Confirmed Facts
- Excluded Hypotheses
- Current Diff
- Constraints
- Next Action
- Do Not Use
```

13.17 如何迁移到自己的项目

先在任务记录里加一个小节：

清单 13-22 如何迁移到自己的项目

```
## Context Cleanup
```

每次遇到测试失败、计划变化或 diff 扩大时，就填一次。

如果你使用仓库内任务文档，可以放：

```
docs/ai-coding/tasks/<task>.md
```

如果只是轻量任务，直接在对话里要求 Agent 输出这张表也可以。

关键是形成习惯：长任务不是一直往前推，而是定期把事实、假设和约束重新摆正。

13.18 可以先练的一次清理

找一个最近和 AI 反复改过的任务，不管最后有没有成功。

不要继续让它写代码。只让它做一次清理：

示例 13-23 可以先练的一次清理

```
请根据当前对话和 diff，输出 Context Cleanup。

必须包含:
1. Confirmed Facts
```

```
2. Excluded Hypotheses
3. Unverified Hypotheses
4. Active Constraints
5. Current Diff
6. Verification State
7. Next Smallest Action

不要继续修改文件。
```

然后你自己检查三件事：

- 它有没有把猜测写进 Confirmed Facts。
- 它有没有漏掉当前 diff 中的计划外文件。
- 它给出的 Next Smallest Action 是否真的最小。

如果这三件事都过了，再让它继续下一步。

这个练习很适合老项目。老项目里问题不一定能马上修好，但先把事实整理干净，已经能减少一大半误改。

13.19 哪些清理结果要写回长期资产

不是每次清理都要沉淀。

只把会反复出现、能减少后续误判的内容写回去。

适合写进 `AGENTS.md` 的，是长期规则：

清单 13-24 哪些清理结果要写回长期资产

```
- Before editing after a test failure, classify the failure as baseline, introduced, environment,
requirement conflict, or coverage gap.
- Do not continue from excluded hypotheses.
- Do not run destructive git commands to shrink diff without human approval.
```

适合写进代码地图的，是项目事实：

清单 13-25 哪些清理结果要写回长期资产

```
- `src/generated/` is generated from OpenAPI; inspect only.
- Auth refresh focused test: `npm test -- auth-refresh`.
```

适合写进 Skill 的，是重复流程：

```
debug cleanup -> classify evidence -> shrink diff proposal -> verification ledger
```

适合只留在任务记录里的，是本次特有事实：

清单 13-26 哪些清理结果要写回长期资产

```
- This bug is not caused by token storage.
- Network trace showed refresh success but original submit was not replayed.
```

这个区分很重要。把一次性事实写进长期规则，会污染后续任务；把长期规则只留在聊天里，下次还会重犯。

13.20 什么时候清理也不够

上下文清理只能解决“材料混乱”的问题，不能替你解决需求本身不清楚的问题。

如果整理后发现这些情况，就不要继续让 Agent 实现：

- 业务规则互相冲突。
- 没有人能确认历史兼容行为。
- 验证路径完全缺失。
- 任务目标从 bug 修复变成架构调整。
- 当前 diff 已经包含多类不相关变更。
- 需要改生产数据、权限、金额或状态机。

这时下一步不是再清理一遍，而是回到需求确认、方案设计或规格驱动。比如订单状态回退问题，如果整理后发现 paid、refunding、refunded、canceled 的状态规则没有人能确认，就应该暂停编码，先把状态合同写清楚。

上下文清理的价值，是让你看清自己到底卡在哪里。有时它会把任务重新拉回最小实现；有时它会明确告诉你：这个任务已经不适合继续靠一段对话推进。

清理输出还要能被另一个人接手。

如果换一个 Agent 或 reviewer 读完清理记录，仍然不知道当前事实、当前 diff、验证状态和下一步动作，那这次清理只是重新总结了一遍聊天记录。真正有用的清理应该让人能立刻判断：继续、拆分、回滚，还是回到需求确认。

可以把这条当成验收：清理记录能不能支撑一次交接。

上下文工程到这里已经形成一条完整链路：代码地图让 Agent 知道从哪里开始，搜索让上下文可解释，项目规则约束长期行为，任务上下文包约束本次任务，上下文清理防止长任务跑偏。

接下来进入 SDD 和 OpenSpec。复杂需求不能只靠上下文包，还需要把需求、设计和任务拆成可审查、可归档的规格合同。

13.21 常见问题

清理上下文是不是浪费时间?

如果只是改文案，确实没必要。清理适合长对话、测试失败、diff 扩大、假设变化和准备交接的时候。它省的是后面继续沿错误方向改的时间。

清理后一定要开新对话吗?

不一定。当前对话还短、diff 还小、只需要排除一两个假设时，可以在当前对话里继续。对话很长、早期假设很多、任务阶段变化明显时，再开新对话。

Agent 总结得很好，还需要按表格清理吗?

要看总结是否可复查。如果总结没有区分 Confirmed、Excluded、Unverified、Constraints 和 Next Action，它只是概括，不是清理。

清理时能不能让 Agent 自动回滚?

不要默认允许。先让它分类 diff，再由人决定保留、拆分还是回滚。真实工作区里可能有用户改动，不能让 Agent 擅自执行破坏性命令。

清理和 verification ledger 有什么关系?

清理发生在任务中段，目的是把事实和假设分开；verification ledger 发生在完成前，目的是记录命令、证据和剩余风险。两者都服务同一个目标：让 AI 的结论能被复查。

13.22 可复查资料

- OpenAI Codex 最佳实践：https://developers.openai.com/codex/learn/best-practices〔13-1〕
- Anthropic Claude Code 最佳实践：https://www.anthropic.com/engineering/claude-code-best-practices〔13-2〕
- GitHub Copilot cloud agent 任务最佳实践：https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results〔13-3〕

本章资源

〔13-1〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔13-2〕 https://www.anthropic.com/engineering/claude-code-best-practices
https://www.anthropic.com/engineering/claude-code-best-practices

〔13-3〕 https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results
https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results

第14章　模块实战：让 AI 接手一个陌生代码模块

陌生模块接手，不是让 AI 进项目跑几条命令，然后听它说“我理解了”。

真实开发里，程序员更关心的是另一件事：在授权 AI 改代码之前，能不能先拿到一份可复查的接手记录。它要说明 AI 读了哪些文件、找到了哪些入口、测试在哪里、哪些目录不能乱碰、第一个小 diff 应该从哪里开始。

配套代码仓库 DavidDingXu/learn-ai-coding〔14-1〕 按一个大仓库组织，仓库根目录下有业务 module 和工具 module。这里用工具 module：`ai-coding-workflow-kit`。它服务上下文工程和工具工作流，不是综合业务实战主线。

这一篇用 `ai-coding-workflow-kit` 的一次接手过程做案例。过程记录在：

```
learn-ai-coding/ai-coding-workflow-kit/dev-process/01-module-onboarding.md
```

对应的真实需求是：

```
第一次接手这个 CLI 工具项目，准备让 AI 后续改进 CLI，但现在不能直接授权它修改代码。
```

这就是很多程序员每天会遇到的场景：项目不大，但你并不熟；需求看起来简单，但入口、测试、README、examples、OpenSpec 材料混在一起；如果 AI 一上来就改，很容易改到错误位置。



图 14-1　陌生模块接手流程

14.1　起点：先禁止改代码

过程记录里的起始请求不是“帮我优化这个项目”，而是：

```
我想让 AI 帮我改进这个 AI Coding 工具项目。先不要改代码，先搞清楚项目结构、CLI 入口、测试覆盖和第一个安全的小改动应该从哪里开始。
```

这句话有三个关键约束：

```
只做仓库侦察。
不改文件。
输出接手记录。
```

如果没有这三条，AI 很可能直接进入实现：重构 CLI、补功能、改 README，甚至顺手调整测试。你还没搞清楚模块，它已经给你一个 diff。

陌生模块的第一步不是“提高效率”，而是降低误改概率。

14.2　第一轮：让 AI 读真实入口

接手记录先要求 AI 读取这些文件：

示例 14-1　第一轮：让 AI 读真实入口

```
README.md
package.json
src/cli.js
src/lib/codemap.js
src/lib/task-context.js
src/lib/verification-ledger.js
test/cli.test.js
test/codemap.test.js
```

这组文件不是随便列的。它覆盖了当前项目的四个事实：

```
README.md              用户入口和可运行命令
package.json           测试命令和模块类型
src/cli.js             CLI 分发入口
src/lib/*.js           核心能力实现
test/*.test.js         当前行为合同
```

让 AI 读文件时，不要只让它总结“项目做什么”。要让它带证据回答：

```
项目提供哪些 CLI 命令?
每个命令对应哪个实现文件?
当前测试覆盖哪些行为?
examples 和 openspec 目录在这次任务里是什么角色?
第一个小改动建议是什么，为什么风险低?
```

这些问题会迫使 AI 回到文件，而不是泛泛说“这是一个提高 AI Coding 效率的工具”。

14.3　第二轮：生成代码地图，但不把它当结论

可以在配套工程的工具 module 里运行：

清单 14-2　第二轮：生成代码地图，但不把它当结论

```
cd learn-ai-coding/ai-coding-workflow-kit
npm test
node src/cli.js codemap --root .
```

这两条命令的作用不同。

`npm test` 是改前基线。当前项目应当看到：

```
tests 12
pass 12
fail 0
```

`codemap` 是导航证据。它会输出源码、测试、文档、配置和建议优先阅读的文件。

但代码地图不是架构真相。它只是第一层索引。程序员还要复核：

```
src/cli.js 是否真的是入口?
test/cli.test.js 是否覆盖进程级行为?
examples/ 是否只是验证记录样例?
openspec/changes/ 是否是历史变更材料，还是当前要改的 source of truth?
```

这一步很重要。AI 容易把“看起来相关的目录”都当成当前实现路径。代码地图帮你缩小范围，但不能替你做工程判断。

14.4　第三轮：形成接手记录

过程记录里的 AI handoff prompt 是这样写的：

示例 14-3 第三轮：形成接手记录

```
你现在只做仓库侦察，不改文件。

请阅读 README.md、package.json、src/cli.js、src/lib/*.js 和 test/*.test.js，
输出一份接手记录：
1. CLI 当前命令列表。
2. 每个命令对应的实现文件和测试文件。
3. 第一个小改动建议。
4. 不应该修改的目录或文件。
5. 最小验证命令。
```

一份合格的接手记录至少应该包含：

示例 14-4 第三轮：形成接手记录

```
CLI 入口：src/cli.js
核心能力：codemap、context-audit、task-context、context-pack、verify-ledger
测试入口：test/*.test.js
验证命令：npm test
谨慎目录：examples/、openspec/changes/
第一个小改动候选：补齐 CLI 合同或改进 codemap 输出
```

如果 AI 没列出实际读过的文件，或者只给一段项目介绍，就不能进入实现。

接手记录的价值不在文字漂亮，而在于你能逐条检查。

14.5 第四轮：把第一个小改动写成任务上下文

接手完成后，仍然不要直接说“那你改吧”。

先把第一个小需求写窄。例如：

```
改进 codemap 的优先阅读文件说明，让 AI 接手陌生项目时知道先读哪些文件，并保持现有统计字段不变。
```

可以生成任务上下文包：

清单 14-5 第四轮：把第一个小改动写成任务上下文

```
node src/cli.js task-context \
  --title "改进 codemap 的优先阅读文件说明" \
  --files "src/lib/codemap.js,test/codemap.test.js,src/cli.js,README.md" \
  --constraints "不引入 AST 解析,不改变 codemap 命令名,不删除现有统计字段" \
  --done "npm test 通过,codemap 输出仍包含 Code Map 和 Files AI Should Inspect First"
```

这里的关键不是命令本身，而是任务上下文里的四个字段：

```
目标：改什么行为。
文件：先读哪些文件。
约束：哪些事不能做。
完成定义：怎么证明完成。
```

这比“优化一下代码地图”可靠得多。

14.6　第五轮：人工复核后再授权

过程记录里的人工复核点包括：

示例 14-6　第五轮：人工复核后再授权

```
src/cli.js 是命令分发入口。
src/lib/codemap.js 是代码地图实现。
test/codemap.test.js 是行为保护点。
examples/ 是验证记录样例，不是生产输入。
openspec/changes/ 是变更材料，不是当前运行路径。
当前最小验证命令是 npm test。
```

复核通过后，才允许 AI 做第一个小 diff。

授权可以写得很具体：

```
只修改 src/lib/codemap.js 和 test/codemap.test.js。
先补测试，证明输出里应该包含优先阅读文件。
不要修改 CLI 命令名。
不要引入 AST 解析。
完成后运行 npm test，并说明 diff 为什么没有影响 verify-ledger。
```

这就是从“接手模块”进入“开发需求”的边界。

如果 AI 此时要求改 `src/cli.js`、README 或 OpenSpec，也不一定错，但必须先解释为什么原授权范围不够。不能让它在同一轮里自动扩大范围。

14.7　验证：命令只是证据

陌生模块接手后的验证不止 `npm test`。

可以看三类证据。

第一，测试基线：

清单 14-7　验证：命令只是证据

```
npm test
```

第二，目标命令输出：

清单 14-8　验证：命令只是证据

```
node src/cli.js codemap --root .
node src/cli.js context-audit --root .
```

第三，diff 范围：

清单 14-9　验证：命令只是证据

```
git diff --stat
git diff -- src/lib/codemap.js test/codemap.test.js
```

这些命令不能替代接手记录。它们只能证明两件事：

```
当前项目还能跑。
这次授权的文件范围没有明显越界。
```

如果没有前面的任务卡、上下文侦察和人工复核，单看命令输出只能说明工具能运行，不能说明 AI 接手是可靠的。



图 14-2　上下文到小 diff

14.8　这套做法怎么迁移

换到你的项目里，可以照这个顺序走。

先写真实需求：

```
我第一次接手这个模块，准备让 AI 后续协助修改，但现在只允许它做侦察。
```

再限定文件侦察：

```
请先找入口、核心实现、测试、配置、文档和禁止修改目录。
每个结论都要带文件路径。
不要改文件。
```

然后要求接手记录：

示例 14-10　这套做法怎么迁移

```
模块职责。
调用入口。
当前测试。
风险目录。
第一个安全小改动。
最小验证命令。
不确定点。
```

最后才授权小 diff：

```
只改一个行为。
只动明确文件。
先补测试。
跑 focused test。
报告剩余风险。
```

大仓库里不要让 AI 一次“理解整个项目”。先把范围缩到一个模块、一条入口、一组测试。

14.9　常见错误

错误一：把代码地图当架构文档。

代码地图只负责导航。真实调用链、业务边界和修改权限仍要回到源码和测试确认。

错误二：AI 没列读过的文件就让它改。

没有文件证据的理解，只是模型根据项目名和目录猜出来的解释。

错误三：接手记录写得很长，但没有第一个小改动。

接手不是写百科。最终要导向一个低风险、可验证的小 diff。

错误四：只跑测试，不看 diff。

测试绿了不代表范围没越界。陌生模块接手后，尤其要看 AI 是否顺手改了 README、examples、OpenSpec 或无关工具。

错误五：把不确定点藏起来。

真实项目里，不确定点比“我理解了”更有价值。比如调用方是否还有外部脚本、README 命令是否进入 CI、历史变更包是否仍有效。

14.10　可复查资料

- 接手过程记录：`learn-ai-coding/ai-coding-workflow-kit/dev-process/01-module-onboarding.md`
- 项目 README：`learn-ai-coding/ai-coding-workflow-kit/README.md`
- CLI 入口：`learn-ai-coding/ai-coding-workflow-kit/src/cli.js`
- 代码地图实现：`learn-ai-coding/ai-coding-workflow-kit/src/lib/codemap.js`
- 任务上下文实现：`learn-ai-coding/ai-coding-workflow-kit/src/lib/task-context.js`
- 测试目录：`learn-ai-coding/ai-coding-workflow-kit/test/`

本章资源

〔14-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

PART 05

第5篇　用规格约束需求变更

把需求写成可核对的规格，再连接设计和小任务。通过 OpenSpec 案例理解文档产物怎样约束实现。

阅读重点

能把一句需求推进为可实现、可验证的变更。

第15章　SDD 是什么：先写规格，再写代码

复杂需求里，AI Coding 最容易失控的地方，不是代码生成，而是需求没有变成可执行合同。

一句话需求很快：

> 帮我把订单取消逻辑改一下，已支付订单不能直接取消，要走退款流程。

Agent 也很快。它会搜索 `cancelOrder`、修改几个判断、补一点测试，然后告诉你完成了。

问题是，这句话里藏了太多没有说清的东西：

- 已支付订单能不能申请取消？
- 取消和退款在业务上是什么关系？
- 退款中、退款失败、部分退款怎么处理？
- 老接口返回码是否保持兼容？
- 管理后台和用户端行为是否一致？
- 历史订单怎么处理？
- 哪些测试证明完成？

这些问题如果不先写清楚，AI 会在实现时自己补。它补得越快，风险越高。

SDD，也就是 Spec-Driven Development，解决的不是“多写文档”，而是先把需求、设计和任务变成 AI 可以执行、程序员可以审查、结果可以验证的规格合同。



图 15-1　从规格到代码的工作流

15.1 SDD 不是瀑布文档

很多程序员听到“先写规格”，第一反应是慢。

如果 SDD 变成几十页需求文档、审批流程和模板填空，那确实慢，而且不适合个人 AI Coding。

这里讲的 SDD 要轻得多。它只要求在写代码前，把三类东西写清楚：

```
requirements.md  写用户可见行为和验收条件
design.md        写技术方案、边界和取舍
tasks.md         写可执行的小步骤和验证方式
```

复杂一点的框架，比如 GitHub Spec Kit 或 OpenSpec，会把这些材料组织得更系统：从用户意图生成 spec、clarify、plan、tasks，再进入 implementation；OpenSpec 还会把 proposal、spec delta、design、tasks、archive 等流程放进仓库。

但你先不用急着上框架。

SDD 的最小价值是：让 AI 在写代码前先接受一份规格，而不是在实现过程中自己发明需求。

15.2 为什么 AI Coding 更需要 SDD

人类程序员拿到模糊需求时，会主动追问，也会在脑子里保留很多业务边界。

Agent 不会稳定拥有这些边界。它会根据当前上下文推断最可能的实现路径。

这在低风险任务里没问题。比如文案、样式、同类字段、局部测试。

但复杂需求里，推断会变成风险：

- 它可能把业务规则理解成普通 if/else。
- 它可能把兼容逻辑当成旧代码清掉。
- 它可能把测试改成符合新实现，而不是锁住需求。
- 它可能在一个大 diff 里混入重构。
- 它可能忽略异常路径和历史数据。

SDD 的作用，是把这些隐含判断提前暴露。

在写代码前，你要先看到：

清单 15-1 为什么 AI Coding 更需要 SDD

```
## Requirement

已支付订单不能直接取消。用户发起取消时，系统应该进入退款申请流程。

## Must Keep

- 未支付订单仍可直接取消。
- 已发货订单仍按现有规则拒绝取消。
- 旧接口返回码保持兼容。

## Verification

- paid order cancel returns refund-required state.
- pending order cancel still succeeds.
- shipped order cancel still fails with existing error code.
```

这段规格比一句 prompt 更能约束 Agent。

15.3 聊天式需求和 SDD 的差别



图 15-2　聊天式需求和 SDD 对比

聊天式需求通常是：

```
用户说一个问题。
AI 搜代码。
AI 直接改。
测试失败再修。
最后看起来能跑。
```

SDD 流程是：

示例 15-2 聊天式需求和 SDD 的差别

```
用户说问题。
AI 和人把问题转成 requirements。
AI 给 design，人审边界。
AI 拆 tasks，每步可验证。
AI 按 task 小步实现。
测试和 diff 证明完成。
```

区别不是多了几个文件，而是控制点前移。

聊天式开发把风险留到实现后暴露。SDD 把风险放到实现前讨论。

15.4 SDD 最适合哪些任务

不是所有任务都需要 SDD。

适合 SDD 的任务：

- 涉及多个模块。
- 涉及状态机、权限、金额、数据迁移。
- 需求有多个角色或多个入口。
- 需要保持历史兼容。
- 测试和验证路径比较复杂。
- 需要拆多步实现。
- 人和 AI 对需求理解可能不同。

不太需要 SDD 的任务：

- 单处文案。
- 局部样式。
- 已有模式下加字段。
- 小测试补充。
- 明确的一行 bug fix。

任务越复杂，越需要先写规格。

可以用一个简单判断：

```
如果你没法在 5 行内说清：
目标行为、不能改变的行为、相关文件、验证方式、风险边界，
就不要直接让 AI 写代码。
```

这时至少写一个轻量 SDD。

15.5 三种粒度：直接改、轻量 SDD、框架化 SDD

真实项目里，SDD 不是开关。不是“用了就是规范开发，不用就是乱写代码”。

真正要判断的是：这次改动需要多强的前置约束。

表 15-1 三种粒度：直接改、轻量 SDD、框架化 SDD

改动类型	合适方式	判断标准
单点低风险改动	直接给 Agent 任务上下文	一两个文件内完成，旧行为清楚，验证命令明确
中等复杂需求	轻量 SDD	涉及业务分支、状态、权限、兼容性、异常路径
多阶段需求变更	OpenSpec / Spec Kit 这类框架	需要 proposal、spec delta、任务归档、多人审查或长期追踪

比如修一个按钮文案，写三份规格文件就是负担。 但改订单状态机、结算规则、权限策略、通知链路，只靠一句 prompt 就是在赌。

程序员要做的不是每次都上重流程，而是判断复杂度已经超过聊天式开发能控制的范围。超过了，就把需求、设计和任务拆出来，让 Agent 按合同工作。

15.6 SDD 在生产项目里放在哪里

个人项目可以把规格放在 `docs/specs/<change-name>/`。公司项目里还要考虑审查和追踪，常见位置有三种。

第一种，放在仓库内。

```
docs/specs/order-cancel-refund/
├── requirements.md
├── design.md
└── tasks.md
```

这种方式适合需求会影响代码结构，或者后续有人需要追溯为什么这么改。规格和代码同版本，PR 能一起审。

第二种，放在 issue 或需求系统里，仓库只保留链接。

清单 15-3 SDD 在生产项目里放在哪里

```
Spec:
- Requirements: <issue link>
- Design: <design doc link>
- Tasks: `docs/specs/order-cancel-refund/tasks.md`
```

这种方式适合公司已经有需求平台，不希望把业务需求全文放进代码仓库。但任务拆分和验证命令最好仍然靠近代码，否则 Agent 很难基于真实目录执行。

第三种，用 OpenSpec 这类工具管理变更。

示例 15-4 SDD 在生产项目里放在哪里

```
openspec/
├── specs/
└── changes/
    └── order-cancel-refund/
        ├── proposal.md
        ├── design.md
        └── tasks.md
```

这种方式适合规格资产开始变多，或者同一个能力会被多次变更。它的价值不是“看起来专业”，而是让变更有状态、有 delta、有归档。

不要为了形式把所有需求都塞进框架。规格文件只有在能降低误改、误解和误验收时才值得存在。

15.7 从 issue 到 SDD 的最小流程

真实需求通常不会一开始就是整齐的 spec。它可能是一条 issue、一次线上报警、一段用户反馈，或者同事在群里丢过来的一句话。

可以按下面顺序把它变成轻量 SDD。

示例 15-5 从 issue 到 SDD 的最小流程

```
原始材料
-> Agent 搜索相关入口、测试、错误码、配置
-> 生成 Current Behavior 摘要
-> 人确认目标行为和非目标
-> Agent 生成 requirements/design/tasks 草稿
```

```
-> 人审草稿
-> Agent 按 task 执行
-> 用测试、diff、PR 描述回写证据
```

注意，第一步不是让 Agent 写代码，而是让它先查证。

比如：

```
先不要改代码。
请基于 issue 描述搜索订单取消相关入口、状态枚举、错误码和测试。
输出 Current Behavior 摘要，每条带文件路径。
不确定的业务规则不要推断，写到 Open Questions。
```

这一步能把 SDD 从“凭空写文档”变成“基于仓库证据整理规格”。

如果 Current Behavior 摘要已经发现测试缺失，也不要急着实现。先在 tasks 里把“补回归测试”放到第一步。否则后面实现出来的代码，很可能只是满足 Agent 自己理解的需求。

15.8 最小 SDD 目录结构

一个最小结构可以这样放：

```
docs/specs/
└── order-cancel-refund/
    ├── requirements.md
    ├── design.md
    └── tasks.md
```

也可以放到：

```
specs/order-cancel-refund/
.ai/specs/order-cancel-refund/
openspec/changes/order-cancel-refund/
```

目录名不是重点。重点是三类文件分工清楚。

`requirements.md` 只回答“什么行为应该发生”。

清单 15-6 最小 SDD 目录结构

```
# Requirements

## User Story

用户取消已支付订单时，系统不应直接取消订单，而应进入退款申请流程。
```

```
## Acceptance Criteria

- pending 订单仍可直接取消。
- paid 订单取消返回 refund-required 状态。
- shipped 订单仍按现有规则拒绝取消。
- 旧接口错误码保持兼容。
```

`design.md` 回答“怎么改，边界在哪里”。

清单 15-7 最小 SDD 目录结构

```
# Design

## Current Flow

`cancelOrder` currently handles pending and paid orders in the same branch.

## Proposed Change

Add an explicit paid-order branch that creates refund request state.

## Non-goals

- Do not change payment provider callback logic.
- Do not change shipped order cancellation.
- Do not migrate historical order data.
```

`tasks.md` 回答“按什么顺序做，每步怎么验证”。

清单 15-8 最小 SDD 目录结构

```
# Tasks

1. Add regression tests for pending, paid, shipped cancellation.
   - Verify: focused order cancel tests fail on paid path before implementation.

2. Update cancel order status transition.
   - Verify: focused order cancel tests pass.

3. Run broader order tests.
   - Verify: `npm test -- order`.
```

这三个文件已经足够支持一次中等复杂需求。

15.9 SDD 文件之间的边界

很多人写 SDD 会把三类内容混在一起。

`requirements.md` 不应该写技术方案。

差：

清单 15-9 SDD 文件之间的边界

```
在 `cancelOrder` 里新增 if 判断 paid 状态。
```

好：

清单 15-10 SDD 文件之间的边界

```
已支付订单取消时，系统应进入退款申请流程，不应直接变为 canceled。
```

`design.md` 不应该重新定义需求。

差：

清单 15-11 SDD 文件之间的边界

```
我们希望用户体验更好，所以订单取消要更合理。
```

好：

清单 15-12 SDD 文件之间的边界

```
在订单状态流转层新增 paid -> refund_requested 分支，保留 pending -> canceled。
```

`tasks.md` 不应该写成大任务。

差：

清单 15-13 SDD 文件之间的边界

```
完成订单取消改造。
```

好：

清单 15-14 SDD 文件之间的边界

```
1. 补 paid 订单取消回归测试。
2. 修改状态流转 guard。
3. 跑 order focused tests。
```

边界清楚，AI 才不会把需求、设计、实现混在一个大 diff 里。

15.10 一个完整例子：订单取消改退款

任务原话：

```
已支付订单不能直接取消，要走退款。
```

先写 requirements：

清单 15-15 一个完整例子：订单取消改退款

```
# Requirements

## Goal

已支付订单取消时，系统进入退款申请流程，而不是直接取消订单。

## Acceptance Criteria

- pending 订单取消后状态为 canceled。
- paid 订单取消后状态为 refund_requested。
- shipped 订单取消仍返回 existing shipped-order error。
- 旧接口返回结构不变，只允许状态值变化。

## Out Of Scope

- 支付渠道回调。
- 历史订单迁移。
- 管理后台批量取消。
```

再写 design：

清单 15-16 一个完整例子：订单取消改退款

```
# Design

## Current Behavior

`cancelOrder` currently allows paid order to enter canceled branch.

## Proposed Change

Add explicit status transition:

- pending -> canceled
- paid -> refund_requested
- shipped -> unchanged rejection path

## Compatibility

Keep public response shape unchanged.

## Risks

- Existing clients may assume paid cancellation returns canceled.
- Refund workflow must not be triggered twice.
```

最后写 tasks：

清单 15-17 一个完整例子：订单取消改退款

```
# Tasks

1. Add regression tests for pending, paid, shipped cancellation.
   - Verify focused tests fail only on paid path.

2. Update status transition logic.
   - Keep response shape unchanged.
   - Do not touch payment callback logic.

3. Run verification.
   - `npm test -- order-cancel`
   - `npm test -- order`
```

现在再让 Agent 实现，风险会小很多。

15.11 让 AI 参与写规格，而不是直接写代码

SDD 不要求人手写全部规格。AI 很适合根据原始需求、代码搜索结果、历史 bug、接口契约生成规格草稿。

可以这样开始：

示例 15-18 让 AI 参与写规格，而不是直接写代码

```
先不要改代码。

请基于下面需求生成轻量 SDD 草稿：
- requirements.md
- design.md
- tasks.md

要求：
- requirements 只写用户可见行为和验收条件。
- design 写技术方案、非目标和风险。
- tasks 拆成小 diff，每步带验证。
- 不确定的问题写到 Open Questions。
```

然后人审。

重点审五个地方：

- 是否遗漏旧行为。
- 是否把技术方案写进需求。
- 是否把猜测写成事实。
- 是否缺少异常路径。
- tasks 是否太大。

AI 写规格草稿没问题，但规格不能不审。

15.12 怎么审 AI 写出来的 SDD 草稿

让 Agent 写规格草稿是合理的，但不能把草稿当结论。审查时不要只看语言是否通顺，要看它有没有做不该做的判断。

可以按下面顺序审。

第一，审需求是否来自输入，而不是模型推断。

如果原始需求只说“已支付订单不能直接取消”，AI 草稿却写出“退款请求必须异步创建”，这就是未确认结论。它应该进入 Open Questions，而不是 Acceptance Criteria。

第二，审设计是否基于真实代码。

`design.md` 里每个 Current Flow 都应该能对应到文件路径、入口函数或测试。如果它写的是“订单模块会处理取消逻辑”，但没有路径，这份 design 不能作为执行依据。

第三，审任务是否能形成小 diff。

一个 task 如果同时写“改状态机、接退款服务、改前端、补测试”，就太大。真实执行时要拆开，否则审 diff 时不知道哪个改动导致行为变化。

第四，审验证是否覆盖风险。

如果 design 写了“重复取消可能创建重复退款”，tasks 里就必须有重复取消测试或幂等检查。风险和验证断开，规格只是提醒，不会改变执行结果。

第五，审非目标是否足够具体。

“不做额外改动”没有约束力。要写成“不改支付回调”“不改后台批量取消”“不迁移历史订单”“不改变 public API response shape”。

一份可执行的 SDD 草稿，至少要经得起这几个问题：

```
这些需求是否都能追溯到原始输入或已确认事实?
这些设计判断是否基于真实源码?
这些 task 是否能单独审查?
每个风险是否有验证动作?
哪些地方必须停止并让人确认?
```

答不上来，就不要让 Agent 进入实现。

15.13 SDD 和任务上下文包的关系

任务上下文包适合一次中等风险改动。它回答：

```
这次任务要改什么，先看哪些文件，不能改什么，怎么验证。
```

SDD 适合更复杂需求。它进一步拆成：

```
需求合同 -> 技术方案 -> 可执行任务
```

两者不是替代关系。

一个任务上下文包可以升级成 SDD：

```
Objective        -> requirements.md
Constraints      -> requirements + design
Files            -> design
Done When        -> tasks
Open Questions   -> requirements/design clarify
```

当任务上下文包写到很长，说明它可能需要 SDD。

15.14 SDD 和 OpenSpec / Spec Kit 的关系

SDD 是方法。OpenSpec 和 Spec Kit 是把这个方法工具化的框架。

GitHub 的 Spec Kit 把 spec-driven development 做成一套流程工具，强调从 spec、plan、tasks 进入实现。OpenSpec 也把 proposal、spec delta、design、tasks、archive 等材料放进仓库，适合管理需求变更。

对个人程序员来说，不需要一开始就决定“必须用哪个框架”。

先理解 SDD 的核心：

```
先写需求行为。
再写技术边界。
再拆可验证任务。
最后让 AI 小步实现。
```

等你发现手写 specs 变多、变更需要归档、多个任务互相影响，再引入 OpenSpec 或 Spec Kit。

15.15 SDD 不是让 AI 变慢，而是让返工变少

很多人排斥 SDD，是因为把它理解成“写完文档再开发”。在 AI Coding 里，真正耗时的通常不是写规格，而是后面几类返工：

- Agent 改对了一个入口，漏了另一个入口。
- 测试被改到适配新实现，旧行为没有锁住。

- 需求里的非目标没写，AI 顺手重构了相邻模块。
- 兼容性问题上线前才被前端或调用方发现。
- 一个大 diff 里混了业务改动、格式化和重命名，review 无法判断。

轻量 SDD 的成本大约是十几到几十分钟。它换来的不是“文档完整”，而是改动前先拿到几个关键证据：

```
行为合同：requirements.md
技术边界：design.md
执行顺序：tasks.md
验证证据：每个 task 的命令和结果
```

如果这些证据能让后面少一次大返工，就已经值了。

15.16 SDD 的交付证据长什么样

一次合格的 SDD 驱动改动，最终不应该只留下代码 diff。PR 或变更记录里至少应该能看到这些信息：

清单 15-19 SDD 的交付证据长什么样

```
Spec:
- requirements.md: paid order cancel enters refund flow
- design.md: status transition layer owns paid cancel rule
- tasks.md: 5 tasks, executed 1-5

Verification:
- `npm test -- order-cancel` PASS
- `npm test -- order` PASS
- API response shape compatibility check PASS

Not changed:
- payment provider callback
- admin batch cancellation
- historical order migration

Residual risk:
- old mobile client needs display check for `refund_requested`
```

这段不是给领导看的报告，而是给 reviewer、未来的自己和后续 Agent 看。半年后再改退款逻辑时，它能说明当时为什么没有动支付回调，为什么新增了 `refund_requested`，验证过哪些路径。

15.17 SDD 和团队流程怎么兼容

即使专栏讲的是个人 AI Coding，很多读者也在公司项目里工作。轻量 SDD 不应该绕过团队流程，而是把个人使用 Agent 的过程变得可审。

如果团队已经要求 issue、设计评审、PR、CI，你可以这样嵌进去：

- issue 里保留需求背景和确认结论。
- 仓库里保留和代码强相关的 requirements/design/tasks。
- PR 描述里链接 spec，并列出验证命令。
- CI 仍然是最终合并前的硬检查。
- code review 重点看 diff 是否符合 spec，而不是只看实现是否能跑。

这套做法不会要求团队接受某个 AI 工具。即使 reviewer 不用 Agent，也能看懂：需求是什么、设计边界是什么、任务怎么拆、测试跑了什么。

个人 AI Coding 想进入生产项目，最需要的不是“把 AI 写得多厉害”，而是让别人能审查你的 AI 参与过程。

15.18 常见错误

错误一：把 SDD 写成文档负担。

如果文件很长，但没有验收条件、非目标和验证命令，那不是 SDD，只是长文档。

错误二：requirements 里写实现。

需求应该写行为，不写代码位置。技术方案放 design。

错误三：design 里不写非目标。

非目标很关键。它告诉 Agent 这次不改什么。

错误四：tasks 太大。

“完成订单取消改造”不是 task。task 要能形成小 diff，并有验证方式。

错误五：规格写完不更新。

实现过程中发现需求变化，要更新规格。否则规格会变成过期上下文。

15.19 常见问题

小需求要不要写 SDD?

不用。单点低风险改动可以直接用任务上下文包，写清目标、文件范围、不能改什么、验证命令就够。SDD 用在需求本身会分叉、会影响兼容性、会跨模块的场景。

SDD 会不会限制 Agent 的发挥?

会限制它在错误地方发挥。好的规格不限制实现细节，只限制业务边界、非目标和验证结果。Agent 仍然可以搜索代码、提出方案、拆任务，但不能绕过已确认的合同。

requirements、design、tasks 一定要三个文件吗?

不一定。小需求可以合成一个 `spec.md`。当内容开始变长、不同人需要审不同部分时，再拆成三个文件。拆分是为了降低混乱，不是为了满足格式。

OpenSpec 或 Spec Kit 是不是必选?

不是。它们适合规格资产变多、变更需要归档或团队需要统一流程时使用。个人程序员先掌握轻量 SDD，再判断是否引入框架。

如果实现过程中发现规格错了怎么办?

先停，不要让 Agent 继续补。更新 requirements 或 design，再重新生成后续 tasks。规格错了还继续实现，只会把错误固化进代码和测试。

15.20 可以先实践的一件事

下次遇到复杂需求，不要直接写代码。先建一个目录：

```
docs/specs/<task-name>/
├── requirements.md
├── design.md
└── tasks.md
```

然后让 AI 先写草稿：

```
先不要改代码。
请把这个需求整理成 requirements/design/tasks 三个文件的草稿。
不确定的地方写 Open Questions。
每个 task 必须能形成小 diff，并说明验证方式。
```

人审完后，再让 Agent 只执行第一个 task。

这就是 SDD 在个人 AI Coding 里的最小落地方式。

如果 `requirements.md` 写不清，后面的 design 和 tasks 都会漂。先把模糊需求变成能测试、能追问、能拒绝越界改动的需求合同。

15.21 可复查资料

- GitHub Spec Kit 官方文档：https://github.github.com/spec-kit/〔15-1〕
- GitHub Spec Kit 仓库：https://github.com/github/spec-kit〔15-2〕

- GitHub Blog: Spec-driven development with AI：https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/〔15-3〕
- OpenSpec 官方站：https://openspec.dev/〔15-4〕
- OpenSpec GitHub 仓库：https://github.com/Fission-AI/OpenSpec/〔15-5〕

本章资源

〔15-1〕 https://github.github.com/spec-kit/
https://github.github.com/spec-kit/

〔15-2〕 https://github.com/github/spec-kit
https://github.com/github/spec-kit

〔15-3〕 https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/
https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/

〔15-4〕 https://openspec.dev/
https://openspec.dev/

〔15-5〕 https://github.com/Fission-AI/OpenSpec/
https://github.com/Fission-AI/OpenSpec/

第16章　从一句需求到 requirements.md

复杂需求开始前，最不该省的是 `requirements.md`。

不是因为文档重要，而是因为需求如果没有变成可验证的行为合同，AI 会在实现过程中自己补空白。

一句话需求：

```
已支付订单不能直接取消，要走退款。
```

听起来清楚，其实不够。它没有说明 pending 订单怎么办，shipped 订单怎么办，退款失败怎么办，旧接口返回结构能不能变，管理后台是不是同一规则，历史订单是否迁移。

`requirements.md` 的目的，就是把这些隐含问题在写代码前摊开。

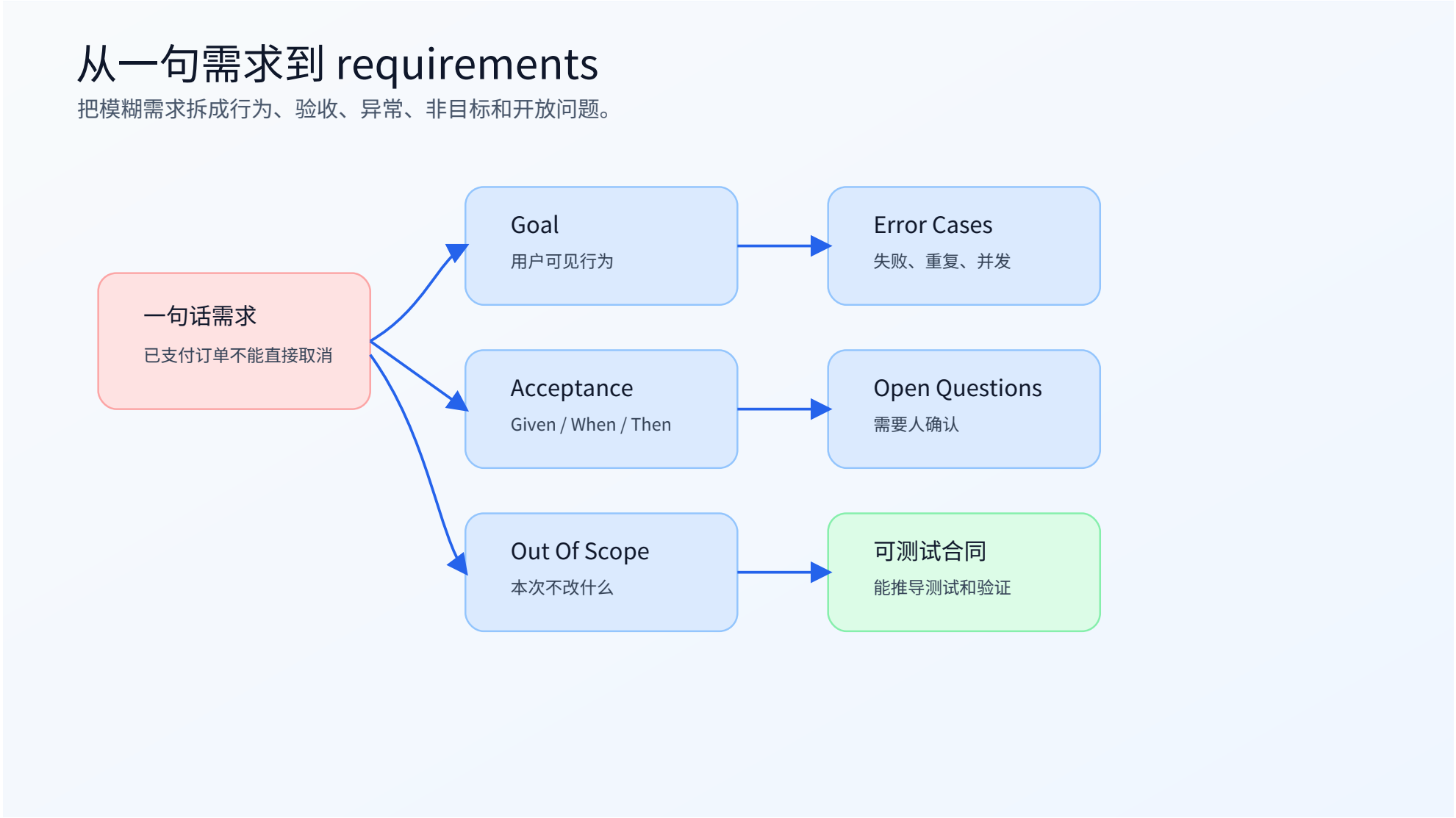


图 16-1　从一句需求到 requirements

16.1 requirements.md 写什么，不写什么

`requirements.md` 写行为，不写实现。

它要回答：

- 谁在什么场景下触发。
- 系统应该表现出什么行为。
- 哪些旧行为必须保留。
- 哪些异常路径必须处理。
- 哪些内容不在本次范围。
- 怎么判断需求完成。
- 哪些问题还需要确认。

它不应该写：

- 改哪个函数。
- 新增哪个 if。
- 抽什么类。
- 用哪个数据库字段。
- 怎么拆 task。

这些属于 `design.md` 和 `tasks.md`。

把技术方案写进 requirements，容易让需求合同变形。后面如果实现方案变了，需求也跟着乱。

16.2 一个坏的 requirements 长什么样

坏版本：

清单 16-1 一个坏的 requirements 长什么样

```
# Requirements

已支付订单不能直接取消，要在 `cancelOrder` 里面判断 paid 状态，然后调用退款服务。
如果是 pending 就照旧，如果是 shipped 就报错。
```

这段话看起来很具体，但问题很多。

第一，它直接指定了实现位置和实现方式。

第二，它没有说明返回给用户的行为。

第三，它没有定义退款服务失败怎么办。

第四，它没有说明旧接口返回结构是否保持。

第五，它没有说明验收条件。

更好的版本：

清单 16-2 一个坏的 requirements 长什么样

```
# Requirements

## Goal

用户取消已支付订单时，系统不应直接取消订单，而应进入退款申请流程。

## Acceptance Criteria

- pending 订单取消后状态为 canceled。
- paid 订单取消后状态为 refund_requested。
- shipped 订单取消仍按现有规则拒绝。
- API response shape remains compatible with existing clients.

## Error Cases

- If refund request creation fails, order status must remain paid.
- User should receive a retryable error message.

## Out Of Scope

- Payment provider callback behavior.
- Historical order data migration.
- Admin batch cancellation.

## Open Questions

- Should paid order cancellation create refund request synchronously or asynchronously?
```

这才是需求合同。

16.3 从一句话需求拆成六块

一份可用的 `requirements.md` 通常包括六块。

示例 16-3 从一句话需求拆成六块

```
Goal
Actors / Scenarios
Acceptance Criteria
Error Cases
Out Of Scope
Open Questions
```

不是每个需求都要六块都很长，但这六块能防止 Agent 漏掉关键边界。

Goal

Goal 写一句清楚的行为目标。

差：

```
优化订单取消。
```

好：

```
用户取消已支付订单时，系统进入退款申请流程，而不是直接取消订单。
```

Goal 不写技术方案。

Actors / Scenarios

写谁在什么场景下触发。

清单 16-4 Actors / Scenarios

```
## Scenarios

- Buyer cancels a pending order before payment.
- Buyer cancels a paid order before shipping.
- Buyer cancels a shipped order.
- Support agent cancels an order from admin console.
```

如果不同角色规则不同，要写出来。否则 Agent 可能把用户端规则套到管理后台。

Acceptance Criteria

验收条件要可验证。

差：

清单 16-5 Acceptance Criteria

```
- 订单取消逻辑正确。
```

好：

清单 16-6 Acceptance Criteria

```
- Given a pending order, when user cancels, then status becomes canceled.
- Given a paid order, when user cancels, then status becomes refund_requested.
- Given a shipped order, when user cancels, then existing rejection behavior remains unchanged.
```

Given/When/Then 的价值不在格式，而在强迫你写出前置状态、动作和结果。

Error Cases

异常路径不能留给实现时猜。

清单 16-7 Error Cases

```
## Error Cases

- If refund request creation fails, order status remains paid.
- If duplicate cancel request arrives, no duplicate refund request is created.
- If payment provider is unavailable, user receives retryable error.
```

很多线上 bug 都出在异常路径。AI 如果只看 happy path，会写出表面可用的实现。

Out Of Scope

非目标是 `requirements.md` 里最重要但最容易漏的部分。

清单 16-8 Out Of Scope

```
## Out Of Scope

- Do not change payment provider callback logic.
- Do not migrate historical order data.
- Do not change admin batch cancellation.
- Do not change public API response shape.
```

非目标告诉 Agent 这次不改什么。没有它，AI 很容易“顺便”做大。

Open Questions

不确定的问题要显式列出来。

清单 16-9 Open Questions

```
## Open Questions

- Should paid cancellation create refund request synchronously?
- Should support agents bypass the buyer-side restriction?
- What error copy should be shown when refund request creation fails?
```

Open Questions 的价值是暂停，而不是让 Agent 自己决定。

16.4 把现有行为和新行为分开写

生产需求里，最容易出问题的不是“新行为没实现”，而是旧行为被顺手改坏。

所以 requirements 里建议加一张行为表，尤其是状态机、权限、接口兼容类需求。

清单 16-10 把现有行为和新行为分开写

```
## Behavior Matrix

| Scenario | Current Behavior | Expected Behavior |
| --- | --- | --- |
| pending order cancel | status becomes `canceled` | unchanged |
| paid order cancel | status becomes `canceled` | status becomes `refund_requested` |
| shipped order cancel | rejected with `ORDER_ALREADY_SHIPPED` | unchanged |
| duplicate paid cancel | may create duplicate side effect | no duplicate refund request |
```

这张表对 Agent 很有用。它能明确哪些路径要改，哪些路径不能动。

它对 reviewer 也有用。review diff 时，不只看 paid 分支，还会看 pending、shipped 和 duplicate 路径有没有测试覆盖。

很多“AI 改坏旧逻辑”的问题，本质上是 requirements 没有把旧行为写成约束。Agent 看到新目标后，会自然围绕新目标重组代码。如果旧行为没有被写进合同，它就不一定知道哪些东西不能碰。

16.5 需求确认不是让产品补文档

个人 AI Coding 场景里，很多需求来自自己、同事、issue 或线上问题。你不一定有完整 PRD，也不应该为了写 requirements 去发起一轮重流程。

可执行的做法是：让 Agent 先根据现有材料生成草稿，然后你把不确定点发给相关人确认。

比如：

清单 16-11 需求确认不是让产品补文档

```
Need confirmation:

1. Paid cancellation should create refund request synchronously or asynchronously?
2. Support agent cancellation follows buyer-side rule or has override permission?
3. Existing mobile clients can display `refund_requested` or need fallback copy?
```

这三条比“请确认订单取消需求”有效得多。它们直接指向实现会分叉的地方。

如果没有产品或业务方可以确认，也要把决定写清楚：

清单 16-12 需求确认不是让产品补文档

```
Decision:
- For this change, buyer cancellation creates refund request synchronously.
- Support agent cancellation remains out of scope.
- Mobile fallback display will be handled in a separate change.
```

```
Reason:
- Current refund service already exposes synchronous create API.
- No admin batch cancellation requirement is confirmed.
```

这不是形式主义。它是在给未来的代码审查和线上问题留证据。

16.6 线上 bug 的 requirements 怎么写

很多程序员觉得只有新需求才需要 requirements，线上 bug 直接修就行。AI Coding 里，这个想法很容易出问题。

比如线上报警：

```
用户重复点击取消订单后，出现两条退款申请。
```

如果直接让 Agent 修，它可能只在按钮上加防重复点击，也可能只在接口里加锁，还可能改测试让它通过。真正需要先写清楚的是：重复请求在系统层面应该如何表现。

可以写成：

清单 16-13 线上 bug 的 requirements 怎么写

```
# Requirements: duplicate paid cancellation is idempotent

## Goal

When a buyer repeats paid order cancellation, the system must not create duplicate refund
requests.

## Current Behavior

- Duplicate cancel requests may create multiple refund requests.
- Existing pending order cancellation is not part of this bug.

## Acceptance Criteria

- Given a paid order with no refund request, when buyer cancels once, then one refund request is
created.
- Given a paid order with an existing refund request, when buyer cancels again, then no new refund
request is created.
- Given a duplicate request, response shape remains compatible with existing clients.

## Error Cases

- If refund request lookup fails, order status remains unchanged.

## Out Of Scope
```

```
- Do not change UI button debounce in this fix.
- Do not change payment provider callback behavior.
- Do not migrate historical duplicate refund records.
```

这份 requirements 把修复范围限定在后端幂等行为。UI 防重复点击可以作为后续增强，但不能代替服务端幂等。

这就是 requirements 的价值：它先定义正确行为，再让 design 决定技术手段。

16.7　需求合同检查矩阵



图 16-2　requirements 检查矩阵

写完 `requirements.md` 后，用四个维度检查。

第一，行为是否具体。

能不能从文字直接写出测试？如果不能，说明还太虚。

第二，旧行为是否保留。

新需求通常只改变一部分路径。哪些旧行为不变必须写出来。

第三，异常路径是否覆盖。

失败、重复、并发、权限、外部系统不可用，这些是否需要说明。

第四，非目标是否明确。

不在本次范围的东西，如果不写，Agent 可能当成优化空间。

16.8 让 AI 帮你写 requirements 草稿

AI 很适合把原始需求整理成 requirements 草稿，但不要让它直接写成最终合同。

可以这样提示：

示例 16-14 让 AI 帮你写 requirements 草稿

```
先不要写代码。

请把下面需求整理成 requirements.md 草稿：

需求：
已支付订单不能直接取消，要走退款。

要求：
- 只写用户可见行为和业务验收条件。
- 不写技术实现方案。
- 使用 Goal / Scenarios / Acceptance Criteria / Error Cases / Out Of Scope / Open Questions。
- 不确定的地方写 Open Questions，不要自行决定。
```

然后你要审。

重点看：

- 是否把实现写进了 requirements。
- 是否遗漏旧行为。
- 是否把未确认规则写成确定事实。
- 是否缺异常路径。
- 是否缺非目标。

AI 可以生成草稿，但需求合同必须人确认。

16.9 一个完整 requirements 示例

下面是订单取消改退款的完整版本。

清单 16-15 一个完整 requirements 示例

```
# Requirements: paid order cancellation enters refund flow

## Goal

When a buyer cancels a paid order before shipping, the system should create a refund request
instead of directly canceling the order.

## Scenarios

- Buyer cancels a pending order before payment.
- Buyer cancels a paid order before shipping.
- Buyer cancels a shipped order.
- Buyer repeats a cancel request for the same paid order.

## Acceptance Criteria

- Given a pending order, when the buyer cancels it, then the order status becomes `canceled`.
- Given a paid order, when the buyer cancels it, then the order status becomes `refund_requested`.
- Given a shipped order, when the buyer cancels it, then the existing shipped-order rejection
remains unchanged.
- Given a repeated cancel request for a paid order with an existing refund request, then no
duplicate refund request is created.
- Existing API response shape remains compatible with current clients.

## Error Cases

- If refund request creation fails, the order status remains `paid`.
- If payment provider is temporarily unavailable, the user receives a retryable error.

## Out Of Scope

- Payment provider callback handling.
- Historical order data migration.
- Admin batch cancellation.
- Changing public API response shape.

## Open Questions

- Should refund request creation be synchronous or asynchronous?
- What user-facing copy should appear when refund request creation fails?
- Should support agents follow the same rule as buyers?
```

这份 requirements 没有说改哪个函数。它只定义行为。

requirements 通过后，`design.md` 才有明确输入：根据当前代码结构决定在哪里实现、如何保持兼容、风险在哪里。

16.10　验收条件怎么写得可测试

验收条件写得好不好，看它能不能变成测试。

差：

清单 16-16　验收条件怎么写得可测试

```
- 系统应该正确处理已支付订单。
```

这句话没法直接测试。

好：

清单 16-17　验收条件怎么写得可测试

```
- Given a paid order, when the buyer cancels it, then the order status becomes `refund_requested`.
```

它能直接变成测试用例：

```
setup paid order
call cancel
assert status refund_requested
```

再比如：

清单 16-18　验收条件怎么写得可测试

```
- Given refund request creation fails, when buyer cancels a paid order, then order status remains
`paid`.
```

这能直接变成失败注入测试。

如果一个验收条件不能变成测试、截图、接口调用或人工检查步骤，它就还不够清楚。

16.11　非目标怎么写

非目标不要写成“暂不考虑性能”这种空话。

要写具体边界：

清单 16-19　非目标怎么写

```
## Out Of Scope

- Do not change payment provider callback behavior.
```

```
- Do not change admin batch cancellation.
- Do not migrate historical order data.
- Do not change public API response shape.
```

非目标有两个作用。

第一，它保护现有系统。 第二，它让 tasks 可以拆得更小。

如果非目标没写，Agent 很可能把“支付回调也顺便改一下”当成合理方案。

16.12 Open Questions 不要怕多

Open Questions 是好东西。

它说明你没有把不确定性伪装成确定结论。

比如：

清单 16-20 Open Questions 不要怕多

```
## Open Questions

- Refund request creation should be sync or async?
- Should support agents follow buyer-side cancel restriction?
- What happens when refund provider returns a duplicate request error?
```

如果这些问题影响实现，就先确认；如果不影响第一步，可以写进 design 的风险和 tasks 的后续项。

AI Coding 里，真正危险的是没有 Open Questions。那通常意味着 Agent 自己在替你做业务决定。

16.13 requirements 和测试的关系

`requirements.md` 不是测试文件，但它应该天然能推导出测试。

可以在 requirements 末尾加一段：

清单 16-21 requirements 和测试的关系

```
## Verification Ideas

- Unit test: pending order cancel remains canceled.
- Unit test: paid order cancel creates refund_requested.
- Unit test: shipped order cancel remains rejected.
- Unit test: duplicate paid cancel does not create duplicate refund request.
- API compatibility check: response shape unchanged.
```

这不是 `tasks.md`，只是提示后续测试方向。

真正的测试命令和实现顺序放到 tasks。

16.14 让 Agent 生成 requirements 时，要给它证据边界

不要只说“帮我写 requirements”。这会让 Agent 按常识补业务规则。

更合适的输入是：

示例 16-22 让 Agent 生成 requirements 时，要给它证据边界

```
请生成 requirements.md 草稿，不要写代码。

可用证据：
- 用户需求：已支付订单不能直接取消，要走退款。
- 当前代码：请先搜索订单取消入口、订单状态枚举、相关测试。
- 只能把源码、测试和需求中能确认的内容写成事实。
- 不确定的业务规则写进 Open Questions。

输出要求：
- 每条 Current Behavior 带文件路径。
- Acceptance Criteria 使用 Given/When/Then。
- Out Of Scope 写清不能改的模块和行为。
```

这里的关键不是模板，而是证据边界。

Agent 可以搜索代码，可以读取测试，可以根据需求生成候选验收条件。但它不能把“常见电商系统通常怎么做”写成你项目的事实。

审草稿时，重点看两类句子。

第一类是没有来源的确定句：

清单 16-23 让 Agent 生成 requirements 时，要给它证据边界

```
Refund request should be created asynchronously.
```

如果原始需求和源码都没支持这个结论，它应该改成 Open Question。

第二类是过度抽象句：

清单 16-24 让 Agent 生成 requirements 时，要给它证据边界

```
System should improve cancellation experience.
```

这句话不能测试，应该改成状态、响应、页面行为或错误码。

16.15 requirements 通过前，不要进入 design

有些需求看起来急，容易直接跳到设计。但 requirements 没过，design 越详细越危险。

进入 design 前，至少确认这些问题：

```
新行为是否能写成测试?
旧行为是否写进合同?
异常路径是否有期望结果?
非目标是否具体到模块、接口或行为?
Open Questions 里是否还有会影响实现路径的问题?
```

如果还有会影响实现的问题，比如“退款同步还是异步”“后台是否同规则”“新增状态老客户端能否识别”，就不要让 Agent 设计方案。

可以先做两件事：

- 让 Agent 搜索现有代码，补齐 Current Behavior。
- 把业务决策问题发给人确认。

这比让 Agent 在 design 里自作主张要便宜得多。

16.16 PR 里怎么呈现 requirements

requirements 不一定要完整贴进 PR 描述，但 PR 需要能追溯到它。

可以这样写：

清单 16-25 PR 里怎么呈现 requirements

```
Requirement summary:
- paid order cancellation now enters refund flow
- pending and shipped cancellation behavior unchanged
- response shape unchanged

Spec:
- `docs/specs/order-cancel-refund/requirements.md`

Verification mapped from requirements:
- pending cancel regression test
- paid cancel refund_requested test
- shipped cancel existing error test
- duplicate paid cancel idempotency test
```

reviewer 不需要读很长的需求文档，也能知道这次行为合同是什么。

对 AI Coding 来说，这一步还有另一个价值：后续让 Agent 做 code review 或补测试时，它可以直接拿 PR 描述和 requirements 对照，而不是重新猜需求。

16.17 状态值、错误码和字段名要引用真实系统

requirements 写行为，但并不意味着它可以随便命名状态和错误码。

如果当前系统里订单状态叫 `PAID`、`CANCELLED`，你在 requirements 里写 `paid`、`canceled` 只是示意没有问题；但进入真实任务时，状态值必须和代码、接口或数据库一致。

可以要求 Agent 先查这些材料：

示例 16-26 状态值、错误码和字段名要引用真实系统

```
请先搜索：
- order status enum
- cancel related error codes
- public API response schema
- existing order cancel tests

输出 requirements 草稿时：
- 业务描述用自然语言。
- 状态值、错误码、字段名使用源码里的真实名字。
- 找不到的名字不要发明，写到 Open Questions。
```

这能避免一个很隐蔽的问题：requirements 看起来对，但实现时 Agent 为了匹配文档新增了一套不该存在的命名。

比如系统里已有错误码：

```
ORDER_ALREADY_SHIPPED
ORDER_CANCEL_NOT_ALLOWED
```

requirements 就不要写成：

清单 16-27 状态值、错误码和字段名要引用真实系统

```
- shipped order returns `SHIPPED_ORDER_ERROR`.
```

应该写：

清单 16-28 状态值、错误码和字段名要引用真实系统

```
- shipped order still returns existing `ORDER_ALREADY_SHIPPED` error.
```

这句话同时表达了两个约束：错误码真实存在，并且本次不改。

16.18 需求质量不够时，Agent 会怎么误解

看一个更贴近真实工作的例子：

```
把会员到期提醒做一下，别让用户错过续费。
```

这句话如果直接交给 Agent，至少会出现这些分叉：

- 到期前几天提醒?
- 站内信、短信、邮件、App push 哪些渠道?
- 免费会员、付费会员、企业会员是否一样?
- 已取消自动续费的用户是否提醒?
- 多次提醒是否去重?
- 发送失败是否重试?
- 法务和隐私策略是否允许发送?

如果 requirements 只写“用户到期前收到提醒”，实现就会非常随意。

更合适的写法是先把范围压住：

清单 16-29 需求质量不够时，Agent 会怎么误解

```
## Goal

Paid individual members should receive one in-app reminder 7 days before membership expiration.

## Acceptance Criteria

- Given a paid individual member expires in 7 days, when daily reminder job runs, then one in-app
reminder is created.
- Given reminder already exists for the same expiration cycle, when job runs again, then no
duplicate reminder is created.
- Given enterprise member, when job runs, then no reminder is created in this change.

## Out Of Scope

- SMS, email, and push notifications.
- Enterprise membership reminder.
- Auto-renew payment retry.
- Marketing campaign copy.
```

这段 requirements 没有解决所有续费问题，但它把这次变更定义清楚了。Agent 可以围绕“个人付费会员、站内信、7 天、一次、去重”去搜索和实现。

程序员写 requirements，不是为了把需求说得宏大，而是把这次要改的行为压到可执行范围。

16.19 常见错误

错误一：把技术实现写进需求。

需求写行为，技术方案写 design。

错误二：只有 happy path。

复杂需求必须写 error cases。尤其是外部系统失败、重复请求、并发和权限。

错误三：没有非目标。

非目标不写，Agent 容易扩大范围。

错误四：验收条件不可测试。

“体验更好”“逻辑正确”都不可测试。改成状态、响应、页面行为或命令结果。

错误五：把不确定问题写成确定规则。

不确定就写 Open Questions，先确认。

16.20 常见问题

requirements 要不要写中文还是英文？

看团队习惯。代码标识、状态值、错误码、命令保留英文；业务描述用中文也可以。关键是可测试、可追溯，不是语言选择。

如果需求来自线上 bug，requirements 还要写吗？

要写，但可以短。线上 bug 也需要定义“修复后应该是什么行为”“哪些旧行为不能变”“怎么证明没复发”。否则很容易只修当前报错路径。

验收条件是不是越多越好？

不是。验收条件要覆盖会改变结果的路径。重复写同一类场景会增加噪音。优先写状态分支、权限分支、异常路径、兼容性分支。

Open Questions 会不会显得需求没想清楚？

不会。Open Questions 是把不确定性显式暴露。真正危险的是没确认就让 Agent 默认选择一个实现。

requirements 变了，后面的 design 和 tasks 怎么办？

需要同步更新。尤其是 Acceptance Criteria、Out Of Scope、Error Cases 变化时，design 的方案和 tasks 的验证命令都可能要重拆。

16.21 可以直接复用的模板

清单 16-30 可以直接复用的模板

```
# Requirements: <change name>

## Goal
```

```
<One sentence describing the behavior change.>

## Scenarios

- <Actor + situation>

## Acceptance Criteria

- Given <state>, when <action>, then <observable result>.

## Error Cases

- If <failure>, then <expected behavior>.

## Out Of Scope

- <What this change must not touch.>

## Open Questions

- <Question that needs human confirmation.>

## Verification Ideas

- <Test, command, screenshot, API call, or manual check idea.>
```

第一次写不要追求完美。先写出 Goal、Acceptance Criteria、Out Of Scope。复杂需求再补 Error Cases 和 Open Questions。

16.22 迁移建议

下次遇到一句模糊需求，先不要让 AI 改代码。让它生成 `requirements.md` 草稿，然后你只审三件事：

- 行为是否可测试。
- 旧行为是否保留。
- 非目标是否明确。

如果这三件事过不了，就不要进入 design。

`requirements.md` 写清后，`design.md` 才有基础。需求合同只定义要什么，设计文档要回答怎么在当前代码库里改，怎么控制兼容性、数据流、架构边界和验证风险。

还有一种情况要直接拒绝继续：需求本身要求绕过现有安全、权限、审计或合规边界，但没有明确授权。

比如：

```
临时让所有客服都能取消已支付订单，先上线再说。
```

这不能直接写成 Acceptance Criteria。至少要先确认：

- 哪些角色可以取消。
- 是否需要操作日志。
- 是否影响退款审批。
- 是否需要灰度或开关。
- 现有权限测试是否要保留。

AI Coding 的速度越快，越不能把权限和审计当成实现细节。requirements 阶段发现这类问题，就应该停在 Open Questions，而不是让 design 替业务和安全决策背书。

16.23 可复查资料

- GitHub Spec Kit 官方文档：https://github.github.com/spec-kit/〔16-1〕
- GitHub Spec Kit 仓库：https://github.com/github/spec-kit〔16-2〕
- GitHub Blog: Spec-driven development with AI：https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/〔16-3〕
- OpenSpec 官方站：https://openspec.dev/〔16-4〕

本章资源

〔16-1〕 https://github.github.com/spec-kit/
https://github.github.com/spec-kit/

〔16-2〕 https://github.com/github/spec-kit
https://github.com/github/spec-kit

〔16-3〕 https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/
https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/

〔16-4〕 https://openspec.dev/
https://openspec.dev/

第17章　design.md 怎么写，AI 才不会补丁式开发

`requirements.md` 写清楚“要什么”之后，不能直接让 AI 开始改。

复杂需求真正危险的地方，往往在“怎么改”。

同一个需求：

```
已支付订单不能直接取消，要走退款。
```

可以有很多实现方式：

- 在 controller 里加一个 paid 判断。
- 在 service 里改状态机。
- 在订单领域层新增状态迁移规则。
- 在退款模块创建 refund request。
- 在接口层保持响应结构不变。
- 在前端根据新状态展示退款处理中。

如果不先写 `design.md`，Agent 很可能选择最容易落地的方式：在它最先看到的函数里加一个判断。这个改动可能能让测试通过，但会把业务规则散落在错误层级里。

`design.md` 的价值，是把 requirements 映射到当前代码结构，提前说明方案边界、兼容性、数据流、风险和验证策略。

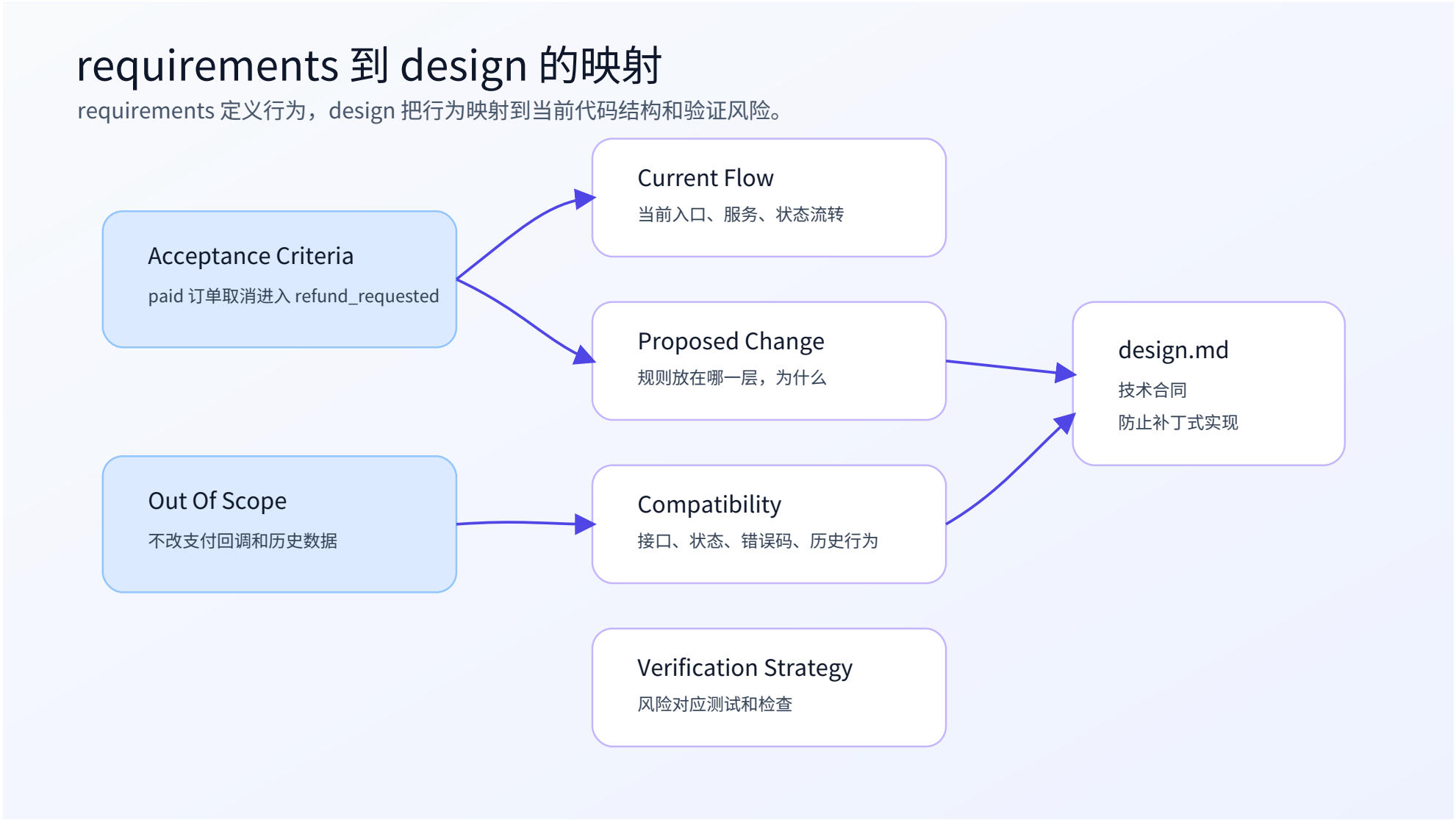


图 17-1　requirements 到 design 的映射

17.1 design 不是架构长文

很多 design 文档写失败，是因为它变成了架构作文。

好的 `design.md` 不需要长。它要回答七个问题：

1. 当前系统怎么做。
2. 这次改哪里。
3. 为什么改这里。
4. 哪些地方不改。
5. 数据和状态怎么流转。
6. 兼容性风险是什么。
7. 怎么验证设计成立。

它不需要写愿景，不需要写大而全架构，不需要复述 requirements。

`requirements.md` 是行为合同。 `design.md` 是技术合同。

它们之间的边界要清楚。

17.2 一个坏的 design 长什么样

坏版本：

清单 17-1 一个坏的 design 长什么样

```
# Design

我们需要优化订单取消逻辑，让系统更加合理。
实现时在取消订单的地方判断 paid，如果已支付就走退款。
同时顺便整理订单状态相关代码。
```

这段 design 有几个问题。

第一，目标太虚。

第二，没有说当前逻辑在哪里。

第三，没有说明为什么在那个地方改。

第四，“顺便整理”扩大了范围。

第五，没有兼容性说明。

第六，没有验证策略。

更好的版本：

清单 17-2 一个坏的 design 长什么样

```
# Design

## Current Flow

`OrderCancelService.cancel()` currently handles pending and paid orders in the same cancellation branch.

## Proposed Change

Move paid-order handling into the order status transition layer:

- pending -> canceled
- paid -> refund_requested
- shipped -> existing rejection path

`OrderCancelService` should call `RefundRequestService.create()` only after the transition layer returns `refund_requested`.

## Non-goals

- Do not change payment provider callback handling.
- Do not change public API response shape.
- Do not migrate historical order data.

## Compatibility

Existing clients should continue receiving the same response shape. Only `status` may differ for paid order cancellation.

## Verification

- focused order cancel tests
- response shape compatibility test
- duplicate paid cancel test
```

这份 design 才能约束 AI 不要补丁式开发。

17.3 design 的核心：把规则放到正确层级

AI 最常见的补丁式开发，是把规则加在最容易改的地方。

比如：

清单 17-3 design 的核心：把规则放到正确层级

```
if (order.status === 'paid') {
  return { status: 'refund_requested' };
}
```

如果这段逻辑加在 controller 里，可能看起来能跑，但真实问题是：订单状态流转规则被写进接口层了。后面后台任务、管理端、批处理入口可能绕过它。

`design.md` 必须回答：这个规则应该属于哪一层。

可以这样写：

清单 17-4 design 的核心：把规则放到正确层级

```
## Design Decision

Paid-order cancellation is a status transition rule, not a controller rule.

Therefore:
- controller keeps request/response handling only
- service orchestrates cancel flow
- status transition module owns allowed transitions
- refund module owns refund request creation
```

这段话能阻止 Agent 把业务规则随手塞进 controller。

17.4 Current Flow 要带证据，不要只写结论

设计文档里最容易被忽略的部分是 Current Flow。很多失败的 design 不是方案写错，而是它建立在错误的当前理解上。

可以让 Agent 先生成一张证据表：

清单 17-5 Current Flow 要带证据，不要只写结论

```
## Current Flow Evidence

| Claim | Evidence |
| --- | --- |
| Buyer cancel request enters order controller | `src/order/order-controller.ts` |
| Cancel service loads order and checks status | `src/order/order-cancel-service.ts` |
| pending and paid currently share cancel branch | `src/order/order-cancel-service.ts` |
| shipped cancellation returns existing error | `test/order/cancel.test.ts` |
| refund request service already exists | `src/refund/refund-request-service.ts` |
```

这张表不是为了贴路径，而是为了防止 design 凭想象写。

如果 Agent 找不到证据，就要停下来继续搜索，或者把结论写成假设：

清单 17-6 Current Flow 要带证据，不要只写结论

```
Assumption:
- No dedicated status transition module was found.

Needs verification:
- Search generated order client and admin cancel path before implementation.
```

假设和事实必须分开。AI Coding 里很多误改，都是因为 Agent 把“我没看到”当成“项目没有”。

17.5 一份 design 的推荐结构

清单 17-7 一份 design 的推荐结构

```
# Design: <change name>

## Current Flow

当前代码怎么运行，入口在哪里，关键逻辑在哪。

## Proposed Change

这次准备怎么改，改在哪一层，为什么。

## Data / State Flow

数据、状态、事件或请求如何流转。

## Compatibility

哪些接口、字段、状态、错误码、历史行为必须保持。

## Non-goals

明确不做什么。

## Risks

可能出错的地方和需要验证的边界。

## Verification Strategy

需要哪些测试、命令、页面路径或人工检查。
```

你可以按任务删减。关键是保留 Current Flow、Proposed Change、Non-goals、Verification。

17.6 Current Flow 怎么写

Current Flow 不是复述代码。它只写和需求相关的当前行为。

清单 17-8 Current Flow 怎么写

```
## Current Flow

- `OrderController.cancel()` receives cancel request and calls `OrderCancelService.cancel()`.
- `OrderCancelService.cancel()` loads order and checks current status.
- pending and paid orders currently enter the same `cancelOrder()` branch.
- shipped orders are rejected with existing `ORDER_ALREADY_SHIPPED` error.
```

注意这里没有贴代码。

它写的是调用链和行为边界。

Current Flow 要基于真实源码和搜索结果，不要凭想象。可以要求 Agent：

```
先不要写 design。
请读取相关入口、实现和测试，先输出 Current Flow。
每条都要带文件路径。
```

如果 Agent 写不出 Current Flow，就不能进入 Proposed Change。

17.7 Proposed Change 怎么写

Proposed Change 要说明改动位置和理由。

清单 17-9 Proposed Change 怎么写

```
## Proposed Change

Add an explicit paid-order transition in `OrderStatusTransition`:

- pending -> canceled
- paid -> refund_requested
- shipped -> rejected with existing behavior

`OrderCancelService` will use the transition result to decide whether to call
`RefundRequestService.create()`.

Reason:
- status transition rules should stay in one module
- controller response shape can remain stable
- refund creation can be tested independently
```

这里有三个关键信息：

- 改哪里。
- 怎么改。
- 为什么不是改别处。

“为什么不是改别处”很重要。它能减少 AI 选择近路。

17.8 Data / State Flow 怎么写

涉及状态、数据、事件的需求，必须写流转。

清单 17-10 Data / State Flow 怎么写

```
## State Flow

pending cancel:
buyer cancel -> status transition -> canceled

paid cancel:
buyer cancel -> status transition -> refund_requested -> create refund request

shipped cancel:
buyer cancel -> status transition -> reject with existing shipped error
```

如果是前后端需求，可以写请求流：

清单 17-11 Data / State Flow 怎么写

```
## Request Flow

ProfileForm submit
-> API client
-> request wrapper handles 401
-> refresh token
-> retry original submit once
-> return submit result to form
```

这类流转图会直接影响实现。Agent 如果跳过流转，容易只修一个入口。

17.9 多入口需求要先画入口表

很多线上系统不是一个按钮对应一个后端函数。订单取消可能有用户端、客服后台、定时任务、批处理、开放接口。只改一个入口，很容易留下绕过路径。

design 里可以先列入口表：

清单 17-12 多入口需求要先画入口表

```
## Entry Points

| Entry | In Scope | Expected Behavior |
| --- | --- | --- |
| buyer order detail cancel | yes | paid -> refund_requested |
| buyer order list cancel | yes | same as detail page |
| support admin cancel | no | unchanged |
| scheduled cleanup job | no | unchanged |
| public partner API cancel | needs confirmation | do not change before confirmation |
```

这张表能直接影响方案。

如果用户端有两个入口，都走同一个 service，方案可以放在 service 或状态层。 如果客服后台不在范围内，但共享同一个 service，就要确认改动会不会影响后台。 如果 partner API 也会调用同一逻辑，但业务规则未确认，就应该写到 Open Questions 或 Stop 条件。

Agent 做代码搜索时，也要围绕入口表搜索，而不是只搜第一个命中的 `cancel`。

17.10 Compatibility 怎么写

兼容性是 design 里最容易被 AI 忽略的部分。

要明确：

- public API response shape 是否变。
- error code 是否变。
- 状态值是否新增。
- 老客户端是否能识别。
- 历史数据是否需要迁移。
- 旧测试是否保留。

例如：

清单 17-13 Compatibility 怎么写

```
## Compatibility

- Public response shape remains unchanged.
- Existing shipped-order error code remains unchanged.
- New status value `refund_requested` may be returned for paid cancellation.
- No historical data migration in this change.
- Existing pending cancellation behavior remains unchanged.
```

如果新增状态值本身就是兼容性风险，要写：

清单 17-14 Compatibility 怎么写

```
## Compatibility Risk

Some clients may assume cancel success always returns `canceled`.
Need response-shape compatibility test and frontend display check for `refund_requested`.
```

这能提醒 tasks 里补测试。

17.11 方案选择要写出取舍，不是让 AI 投票

复杂需求通常不止一种实现方案。`design.md` 不需要把每种方案写成论文，但要说明为什么选当前方案。

比如订单取消改退款，至少有三种做法。

表 17-1 方案选择要写出取舍，不是让 AI 投票

方案	优点	风险	结论
controller 里判断 paid	改动少，见效快	其他入口可能绕过，业务规则散落	不选
cancel service 里判断 paid	能覆盖主要入口	状态规则仍然和编排混在一起	可用于临时修复
status transition 层新增规则	规则集中，可测试，可复用	需要确认状态值兼容性	作为默认方案

这张表不是让 AI “综合比较后得出最佳方案”。真正的决策来自代码结构和风险边界。

如果项目里没有独立状态机，强行新建一个复杂状态模块也可能过度设计。此时可以写：

清单 17-15 方案选择要写出取舍，不是让 AI 投票

```
Decision:
- Put paid cancellation guard in `OrderCancelService` for this change.
- Do not introduce a new status transition module.

Reason:
- Current project has no dedicated transition layer.
- Only buyer-side cancel path is in scope.
- Introducing a new module would mix behavior change with architecture migration.

Follow-up:
- If admin cancellation joins the same rule, extract transition module in a separate change.
```

专业的 design 不是永远选“架构上最漂亮”的方案，而是把当前约束下的取舍写清楚。

17.12 设计里也要写“为什么不这么做”

一个成熟的 design 经常会写 rejected options。不是为了凑内容，而是防止后续实现或 review 又绕回同一个坑。

比如：

清单 17-16 设计里也要写“为什么不这么做”

```
## Rejected Options

### Add paid check in controller

Rejected because:
- admin and background cancel paths may bypass controller
- controller should not own order state transition rules
- duplicate refund prevention would be hard to test at controller level
```

```
### Change refund callback behavior

Rejected because:
- callback behavior is not part of this requirement
- changing callback affects payment reconciliation
- no source evidence suggests callback is currently wrong
```

这类内容对 Agent 很重要。否则它在实现遇到阻力时，可能又退回最容易改的 controller 分支。

Rejected options 不需要很多。只写那些看起来诱人、但风险明显的方案。

17.13 Non-goals 怎么写

Non-goals 是防止补丁式开发扩大成重构的关键。

清单 17-17 Non-goals 怎么写

```
## Non-goals

- Do not refactor the whole order status module.
- Do not change payment provider callback logic.
- Do not change admin batch cancellation.
- Do not migrate historical order data.
- Do not change public API response shape.
```

不要写：

清单 17-18 Non-goals 怎么写

```
暂不考虑其他问题。
```

这种非目标没有约束力。

Non-goals 要具体到模块、行为或接口。

17.14 Risks 和 Verification 怎么联动

风险不能只列出来，要对应验证。



图 17-2　design 风险和验证边界

例如：

清单 17-19 Risks 和 Verification 怎么联动

```
## Risks

1. Duplicate paid cancel may create duplicate refund requests.
2. Existing clients may not handle `refund_requested`.
3. Refund request creation failure may leave order in wrong state.

## Verification Strategy

- Add duplicate paid cancel test.
- Add response shape compatibility test.
- Add refund request failure test.
- Run focused order cancel tests.
```

风险如果没有验证方式，就只是提醒。

设计文档要把风险变成 tasks 的输入。

17.15 让 AI 写 design 草稿

AI 很适合根据 requirements 和代码搜索结果写 design 草稿。

提示可以这样：

示例 17-20 让 AI 写 design 草稿

```
先不要改代码。

请基于 requirements.md 和当前代码搜索结果，生成 design.md 草稿。

要求：
- 先写 Current Flow，每条带文件路径。
- Proposed Change 必须说明改动层级和原因。
- 写 Non-goals，明确不改哪些模块和行为。
- 写 Compatibility，说明接口、错误码、状态值和历史数据影响。
- 写 Risks 和对应 Verification Strategy。
- 不要写具体 task 列表。
```

人审时重点看：

- Current Flow 是否基于真实文件。
- 改动层级是否正确。
- 是否把兼容性写清。
- 是否混入重构。
- 风险是否有对应验证。

AI 可以写草稿，但 design 决策不能全交给 AI。

17.16 design 和代码地图的关系

代码地图告诉你系统入口和模块在哪里。Design 告诉你这次为什么改这些地方。

代码地图：

```
订单模块在 `src/modules/order/`
退款模块在 `src/modules/refund/`
订单测试在 `test/order/`
```

design：

```
本次把 paid cancel 规则放在 `OrderStatusTransition`，
由 `OrderCancelService` 编排 refund request，
不改 controller response shape。
```

不要把代码地图复制进 design。引用即可。

清单 17-21 design 和代码地图的关系

```
## Context Sources

- Code map: `docs/ai-coding/code-map.md`
- Requirements: `requirements.md`
```

17.17 design 和 tasks 的关系

design 不能写成任务清单。

它可以说明验证策略，但不负责拆步骤。

design：

清单 17-22 design 和 tasks 的关系

```
Verification Strategy:
- duplicate paid cancel test
- refund failure test
- response shape compatibility test
```

tasks：

清单 17-23 design 和 tasks 的关系

```
1. Add failing tests for paid cancel and duplicate cancel.
2. Implement status transition change.
3. Add refund request failure handling.
4. Run focused and broader tests.
```

如果 design 里已经写满步骤，后续 tasks 很容易重复和冲突。

17.18 design review 要问的几个问题

在让 Agent 按 design 拆 tasks 前，先做一次设计审查。审查不需要开会，自己读一遍也行。

第一，Current Flow 是否可信。

每条当前流程都应该能追溯到文件路径、函数、路由、测试或配置。没有证据的 Current Flow，本质上是猜。

第二，规则是否放在正确层级。

如果业务规则写进 UI、controller、测试夹具、生成代码或脚本里，要问为什么。很多 AI 补丁就是这样埋下长期问题。

第三，兼容性是否被正面处理。

新增状态、字段、错误码、事件、权限规则，都可能影响调用方。design 不能只写“保持兼容”，要说清楚什么不变、什么会变、如何验证。

第四，是否混入无关重构。

“顺便整理”“统一封装”“抽象一下”要非常谨慎。除非它是实现当前需求的必要条件，否则放到后续独立任务。

第五，风险是否能进入 tasks。

design 里每个风险，后面都应该对应一个测试、命令、页面检查或人工验证步骤。如果没有，就删掉这个风险，或者补出验证办法。

可以把审查结果写成短评：

清单 17-24 design review 要问的几个问题

```
Design review:
- Current flow verified against `src/order/cancel-service.ts` and `test/order/cancel.test.ts`.
- Paid cancel rule belongs to status transition logic, not controller.
- Response shape remains unchanged; `status` may become `refund_requested`.
- No migration, no payment callback change.
- Tasks must include duplicate cancel and refund failure tests.
```

这段短评可以放在 PR 描述或 spec 末尾。它比“设计已确认”有用，因为它说清楚确认了什么。

17.19 跟着真实仓库跑一次 design 审查

`design.md` 不是写完就算数。它至少要经得起一次仓库级审查：文档里的 Current Flow、Proposed Change、Compatibility 和 Verification Strategy，都要能回到真实文件、测试或命令。

可以按这个顺序跑：

清单 17-25 跟着真实仓库跑一次 design 审查

```
rg -n "cancel|refund|OrderCancel|OrderStatus|refund_requested" src test
rg --files | rg "order|refund|cancel|status|test|spec"
git status --short
```

第一条命令找业务入口和调用点，第二条命令看相关文件分布，第三条命令确认当前工作区是否干净。真实项目里不要让 Agent 在未知改动上继续写 design，否则它可能把已有未提交改动也当成当前需求的一部分。

如果项目已有 focused test，再跑一遍基线：

清单 17-26 跟着真实仓库跑一次 design 审查

```
npm test -- order
```

Java 项目可以替换成：

清单 17-27 跟着真实仓库跑一次 design 审查

```
./mvnw test -Dtest=OrderCancelTest
```

Go 项目可以替换成：

清单 17-28 跟着真实仓库跑一次 design 审查

```
go test ./... -run OrderCancel
```

跑基线不是为了证明现在没问题，而是为了知道 design 里的 Verification Strategy 后续要和哪个结果对比。老项目可能一开始就有失败测试，这时要记录 baseline，不要让 Agent 把无关失败混进当前需求。

审 design 时，把每条关键结论压成“声明 + 证据”：

清单 17-29 跟着真实仓库跑一次 design 审查

```
| Design Claim | Evidence |
| --- | --- |
| paid cancel should not be handled in controller | `src/order/order-controller.ts` only delegates
```

```
request handling |
| cancel orchestration is in service layer | `src/order/order-cancel-service.ts` |
| refund request service exists | `src/refund/refund-request-service.ts` |
| current tests cover shipped rejection | `test/order/cancel.test.ts` |
```

如果某条 design claim 没有证据，就不能进入实现。要么继续搜索，要么写成假设和停止条件：

清单 17-30 跟着真实仓库跑一次 design 审查

```
Assumption:
- No partner API cancel path was found by current search.

Stop if:
- implementation finds another cancel entry that shares the same service.
```

这一步的价值很直接：它能防止 AI 把“看起来合理的设计”写成“没有源码证据的设计”。

17.20 design 什么时候应该让实现停下来

`design.md` 不只是指导实现，也要写出停止条件。

下面这些情况出现时，不应该让 Agent 继续写代码：

清单 17-31 design 什么时候应该让实现停下来

```
Stop implementation if:
- Current Flow conflicts with existing tests.
- Public API response shape must change.
- A database migration becomes necessary.
- Payment callback logic must be modified.
- A new dependency is required.
- Generated client or generated schema needs manual edit.
```

停止条件能防止任务从“订单取消规则调整”扩张成“支付系统改造”。

真实生产项目里，范围膨胀经常不是人故意的，而是实现时发现一个点牵出另一个点。Agent 会倾向继续解决它看到的问题。design 必须告诉它：哪些发现需要停下来让人重新判断。

17.21 前后端需求的 design 要多写一段展示和兼容

很多 design 示例只讲后端，但真实需求经常跨前后端。

如果后端新增 `refund_requested`，前端至少要确认：

清单 17-32 前后端需求的 design 要多写一段展示和兼容

```
Frontend Display:
- Order detail page should show "退款处理中" for `refund_requested`.
- Order list should keep existing cancel button hidden for shipped orders.
- Unknown status fallback must not crash.

Compatibility:
- Old clients may receive `refund_requested`.
- API response shape unchanged.
- Mobile fallback copy needs product confirmation if old client cannot display the new state.
```

这不代表一定要在同一个 PR 改前端。它只是让 design 正视状态值被谁消费。

如果前端不在本次范围，也要写：

清单 17-33 前后端需求的 design 要多写一段展示和兼容

```
Out of scope:
- Native mobile display update.

Residual risk:
- Old mobile clients may show generic status copy for `refund_requested`.
```

这类残余风险不能藏起来。它决定 tasks 是否需要加兼容测试、灰度开关或后续 issue。

17.22 数据变更要单独写边界

只要需求碰到数据库字段、状态值、枚举、消息队列事件、索引或迁移脚本，design 里就要单独写 Data Change。不要把它混在 Proposed Change 里。

例如：

清单 17-34 数据变更要单独写边界

```
## Data Change

- No database schema migration in this change.
- Existing order status enum will add `refund_requested`.
- Historical paid orders remain unchanged.
- Refund request table already exists and will be reused.
- No backfill job is required.
```

如果确实需要迁移，要写得更具体：

清单 17-35 数据变更要单独写边界

```
## Data Migration

- Add nullable column `refund_request_id` to orders.
```

```
- Backfill is not required because existing paid orders do not have refund requests.
- Rollback must drop only the new nullable column.
- Deployment order: migration first, application change second.
```

数据变更是 AI Coding 里不能含糊的地方。Agent 很容易为了实现方便新增字段、改枚举、调整 schema，却没有考虑部署顺序、回滚和历史数据。

如果 design 里出现下面几类内容，要强制停下来审：

- 新增非空字段。
- 修改现有枚举含义。
- 改 public API 字段语义。
- 增加外部事件。
- 引入后台 backfill。
- 需要多服务协同上线。

这些不是不能做，而是不能被 Agent 在实现过程中临时决定。

17.23 常见错误

错误一：design 只写“怎么实现”，不写 current flow。

没有 current flow，就无法判断方案是否贴合当前代码。

错误二：把规则放错层。

业务状态机写进 controller，错误处理写进 UI，接口兼容写进测试夹具，都是补丁式开发。

错误三：没有 Non-goals。

没有非目标，AI 很容易把需求做成重构。

错误四：风险没有验证。

风险列表不对应测试或检查，就不会进入 tasks。

错误五：design 过长。

太长会变成新噪音。只写和当前需求相关的技术合同。

17.24 常见问题

design 要不要画架构图?

看复杂度。跨模块、跨服务、状态流转或外部系统调用，建议画一张简图。单模块改动用文字流程就够。图的价值是暴露边界，不是装饰。

AI 写的 design 看起来很完整，还要审吗？

要审。完整不等于正确。重点看 Current Flow 是否有源码证据，Proposed Change 是否符合当前架构，Compatibility 和 Non-goals 是否具体。

design 能不能包含代码片段？

可以，但不要变成实现文件。设计里适合放状态表、接口契约、伪代码、关键类型，不适合放大段最终代码。大段代码应该进入 task 执行阶段。

如果当前代码结构很差，design 要不要趁机重构？

除非不重构就无法完成需求，否则不要混在一起。可以在 design 里写 follow-up，但当前 tasks 只做必要改动。AI 最容易在“顺便整理”里扩大 diff。

design 和 OpenSpec 的 design 有区别吗？

核心问题一样：当前怎么做、准备怎么改、边界和风险是什么。OpenSpec 提供目录、proposal、spec delta、archive 等组织方式；轻量 SDD 只保留个人任务所需的最小集合。

17.25 一个可复用模板

清单 17-36 一个可复用模板

```
# Design: <change name>

## Context Sources

- Requirements: `requirements.md`
- Code map: `docs/ai-coding/code-map.md`

## Current Flow

- `<path>`: <current behavior>

## Proposed Change

<Where to change, what to change, and why this layer owns the rule.>

## Data / State Flow

- <state/request/event flow>

## Compatibility

- <API, status, error code, data, client compatibility>

## Non-goals

- <modules or behaviors this change must not touch>
```

```
## Risks

- <risk>

## Verification Strategy

- <test, command, browser/API/manual check tied to risks>
```

第一次写可以先填 Current Flow、Proposed Change、Non-goals、Verification。复杂需求再补 Data Flow、Compatibility、Risks。

17.26 迁移建议

下次写完 `requirements.md` 后，不要直接让 Agent 实现。

先让它回答：

```
当前代码里，这个规则应该属于哪一层？
为什么不是 controller / UI / test fixture / generated client？
哪些接口和旧行为必须保持兼容？
哪些风险需要对应测试？
```

如果这些问题答不清，就先写 `design.md`。

`design.md` 的目标不是让文档更完整，而是让 AI 不再把复杂需求做成几个散落的补丁。

设计确定后，复杂需求还要拆成小 diff。否则再清楚的方案，也可能被一次性实现成难审的大改动。

17.27 可复查资料

- GitHub Spec Kit 官方文档：https://github.github.com/spec-kit/〔17-1〕
- GitHub Spec Kit 仓库：https://github.com/github/spec-kit〔17-2〕
- OpenSpec concepts：https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md〔17-3〕
- OpenSpec getting started：https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md〔17-4〕

本章资源

〔17-1〕 https://github.github.com/spec-kit/
https://github.github.com/spec-kit/

〔17-2〕 https://github.com/github/spec-kit
https://github.com/github/spec-kit

〔17-3〕 https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md
https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md

〔17-4〕 https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md
https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md

第18章　tasks.md 怎么拆：把复杂需求变成小 diff

`requirements.md` 写清行为，`design.md` 写清方案，最后还不能直接让 AI 一口气实现。

复杂需求真正落地时，风险集中在 diff 粒度。

一个任务写成：

清单 18-1　本章示例

```
- 完成订单取消改退款流程。
```

Agent 会把测试、状态机、退款请求、接口兼容、前端展示、错误处理混在一起改。最后可能测试能跑，但 diff 难审、失败难定位、回滚也麻烦。

`tasks.md` 的目标，是把设计拆成一串可审查、可验证、可停下来的小 diff。

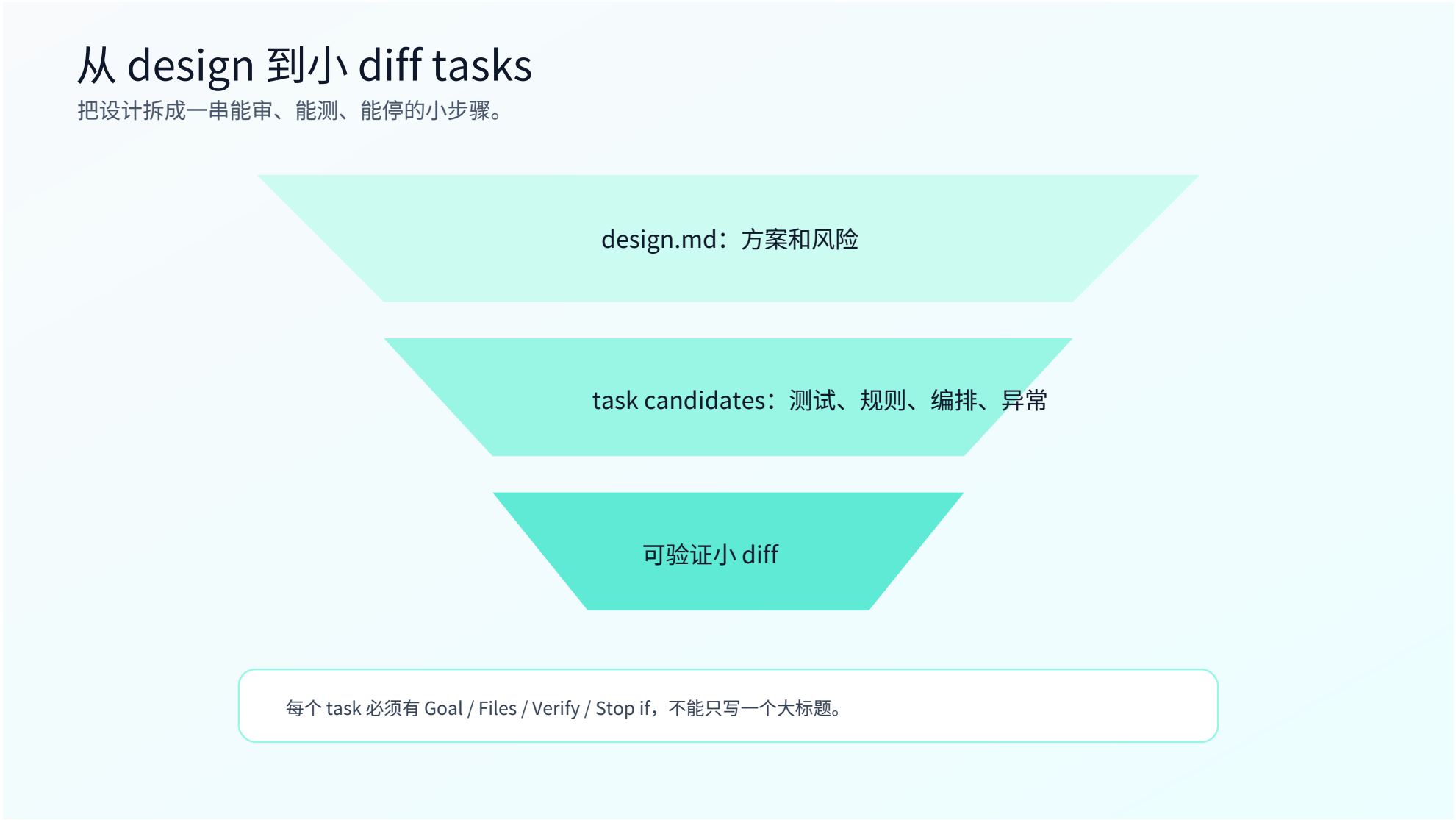


图 18-1　从 design 到小 diff tasks

18.1 好 task 的标准

一个好 task 要满足五个条件：

```
单一目的
明确文件范围
可验证
可审查
失败后能停
```

坏 task：

清单 18-2 好 task 的标准

```
- 实现订单取消退款。
```

好 task：

清单 18-3 好 task 的标准

```
1. Add failing tests for paid order cancellation.
   - Files: `test/order/cancel.test.ts`
   - Verify: focused test fails only on paid path.

2. Add paid -> refund_requested transition.
   - Files: `src/order/status-transition.ts`
   - Verify: focused status transition tests pass.

3. Wire refund request creation in cancel service.
   - Files: `src/order/cancel-service.ts`
   - Verify: order cancel tests pass.
```

这不是为了流程漂亮，而是为了让 AI 一次只改一个能审的东西。

18.2 tasks.md 不是 todo list

普通 todo list 只写要做什么。`tasks.md` 必须写：

- 做什么。
- 改哪些文件。
- 为什么现在做。
- 怎么验证。
- 依赖哪个前置任务。
- 失败时是否停止。

例如：

清单 18-4 tasks.md 不是 todo list

```
## Task 1: Add regression tests

Goal:
- Lock pending, paid, shipped cancel behavior before implementation.

Files:
- `test/order/cancel.test.ts`

Steps:
- Add pending order cancel test.
- Add paid order cancel test expecting `refund_requested`.
- Add shipped order cancel test expecting existing rejection.

Verify:
- `npm test -- order-cancel`
- Expected before implementation: paid path fails, existing paths pass.

Stop if:
- Existing pending or shipped behavior fails before implementation.
```

这才是 AI 可以执行的 task。

18.3 拆任务的基本顺序

复杂需求通常按这个顺序拆：

示例 18-5 拆任务的基本顺序

```
1. 锁定现有行为和新行为
2. 修改核心规则
3. 接入编排流程
4. 补异常路径
5. 扩大验证范围
6. 更新文档或规则
```

这个顺序不是固定，但它有一个原则：先让测试和证据出现，再让实现跟上。

订单取消改退款可以拆成：

清单 18-6 拆任务的基本顺序

```
# Tasks

## 1. Add behavior tests

- pending cancel remains canceled
- paid cancel becomes refund_requested
- shipped cancel remains rejected
- duplicate paid cancel does not create duplicate refund
```

```
Verify:
- `npm test -- order-cancel`
- Expected before implementation: paid and duplicate paid cases fail.

## 2. Add status transition rule

- Add paid -> refund_requested.
- Keep pending -> canceled.
- Keep shipped rejection.

Verify:
- status transition focused tests pass.

## 3. Wire refund request creation

- Use transition result to create refund request.
- Do not change payment callback.

Verify:
- order cancel service tests pass.

## 4. Add failure handling

- If refund request creation fails, order remains paid.

Verify:
- refund failure test passes.

## 5. Run broader verification

- `npm test -- order`
- API response shape compatibility check.
```

每个 task 都有边界和验证。

18.4 任务粒度怎么判断

一个 task 太大，会出现这些信号：

- 涉及多个业务规则。
- 同时改测试、实现、UI、接口、数据。
- 验证命令不止一个层级。
- 失败后不知道是哪一部分导致。
- review 时很难一句话说明 diff。

一个 task 太小，也会有问题：

- 每步只有机械改名。
- 验证成本高于改动价值。

· 任务之间没有独立意义。

比较合适的粒度是：一个 task 完成后，diff 能单独解释，验证能单独运行。

例如：

```
补 paid cancel 测试
```

这是合理 task。

```
新增一个 import
```

太小，除非它是一次机械迁移里的可验证步骤。

```
完成退款流程和前端展示
```

太大。

18.5 把坏 tasks 改成可执行 tasks

看一个常见坏版本：

清单 18-7 把坏 tasks 改成可执行 tasks

```
# Tasks

- 实现后端退款逻辑
- 改前端展示
- 补测试
- 自测
```

这份清单几乎不能用。它没有文件范围，没有顺序，没有停止条件，也没有说明哪些测试先失败、哪些测试后通过。

可以改成：

清单 18-8 把坏 tasks 改成可执行 tasks

```
# Tasks: paid order cancellation enters refund flow

## Task 1: Lock current and expected cancel behavior

Goal:
- Add behavior tests before implementation.

Files:
- `test/order/cancel.test.ts`
```

```
Verify:
- `npm test -- order-cancel`

Expected:
- pending and shipped cases pass.
- paid and duplicate paid cases fail before implementation.

Stop if:
- pending or shipped cases fail before implementation.

## Task 2: Add paid cancel transition

Goal:
- Make paid cancellation return `refund_requested`.

Files:
- `src/order/status-transition.ts`

Verify:
- `npm test -- order-cancel`

Stop if:
- implementation requires public response shape change.

## Task 3: Create refund request from cancel flow

Goal:
- Wire refund request creation after paid cancel transition.

Files:
- `src/order/cancel-service.ts`
- `src/refund/refund-request-service.ts`

Verify:
- `npm test -- order-cancel`

Stop if:
- payment provider callback logic must change.
```

这个改写的重点不是把格式变漂亮，而是让 Agent 每次只拿到一个可执行边界。

好的 tasks 读起来应该能回答三个问题：

```
这一步为什么现在做?
这一步最多允许改哪里?
这一步完成后用什么证据证明?
```

如果回答不了，就继续拆。

18.6 每个 task 都要写 Stop 条件

很多 AI Coding 失控，是因为任务失败后继续往下做。

`tasks.md` 里应该写 Stop if：

清单 18-9 每个 task 都要写 Stop 条件

```
Stop if:
- Existing pending cancel test fails before implementation.
- Public API response shape must change.
- New dependency appears necessary.
- Generated file needs manual edit.
- Design assumption conflicts with current code.
```

Stop 条件不是消极。它是保护边界。

如果 Task 1 发现 pending 订单旧测试本来就失败，不能继续 Task 2。要先分类：baseline failure 还是需求理解错误。

18.7 先处理 baseline failure

真实仓库里，测试不一定一开始就是绿的。Agent 执行 Task 1 时，如果发现旧测试已经失败，不能把失败混进当前需求。

`tasks.md` 里可以写：

清单 18-10 先处理 baseline failure

```
Baseline check:
- Run `npm test -- order-cancel` before adding new tests.
- If existing pending or shipped cancellation tests fail, stop and report baseline failure.
- Do not modify implementation while classifying baseline failure.
```

baseline failure 有三种常见情况。

第一种，和当前需求无关。

比如订单列表快照测试因为文案变化失败，但取消逻辑测试是绿的。可以记录为 unrelated baseline failure，并说明当前 task 只跑 focused tests。

第二种，和当前需求相关。

比如 pending cancel 旧测试已经失败，这时不能继续实现 paid cancel。要先修复或确认现有行为，否则新需求没有稳定基线。

第三种，测试环境问题。

比如本地数据库没启动、外部服务 mock 缺失、依赖版本不一致。此时要补环境说明或替代验证，不能让 Agent 把业务代码改到测试通过。

baseline failure 的处理方式要写进报告：

清单 18-11 先处理 baseline failure

```
Baseline:
- `npm test -- order-cancel` failed before code changes.
- Failure: pending cancel expected `canceled`, got `pending`.
- Classification: related baseline failure.
- Action: stop before Task 2.
```

这一步很关键。否则 AI Coding 很容易把“修旧问题”和“做新需求”搅在一个 diff 里。

18.8 任务依赖和验证关系

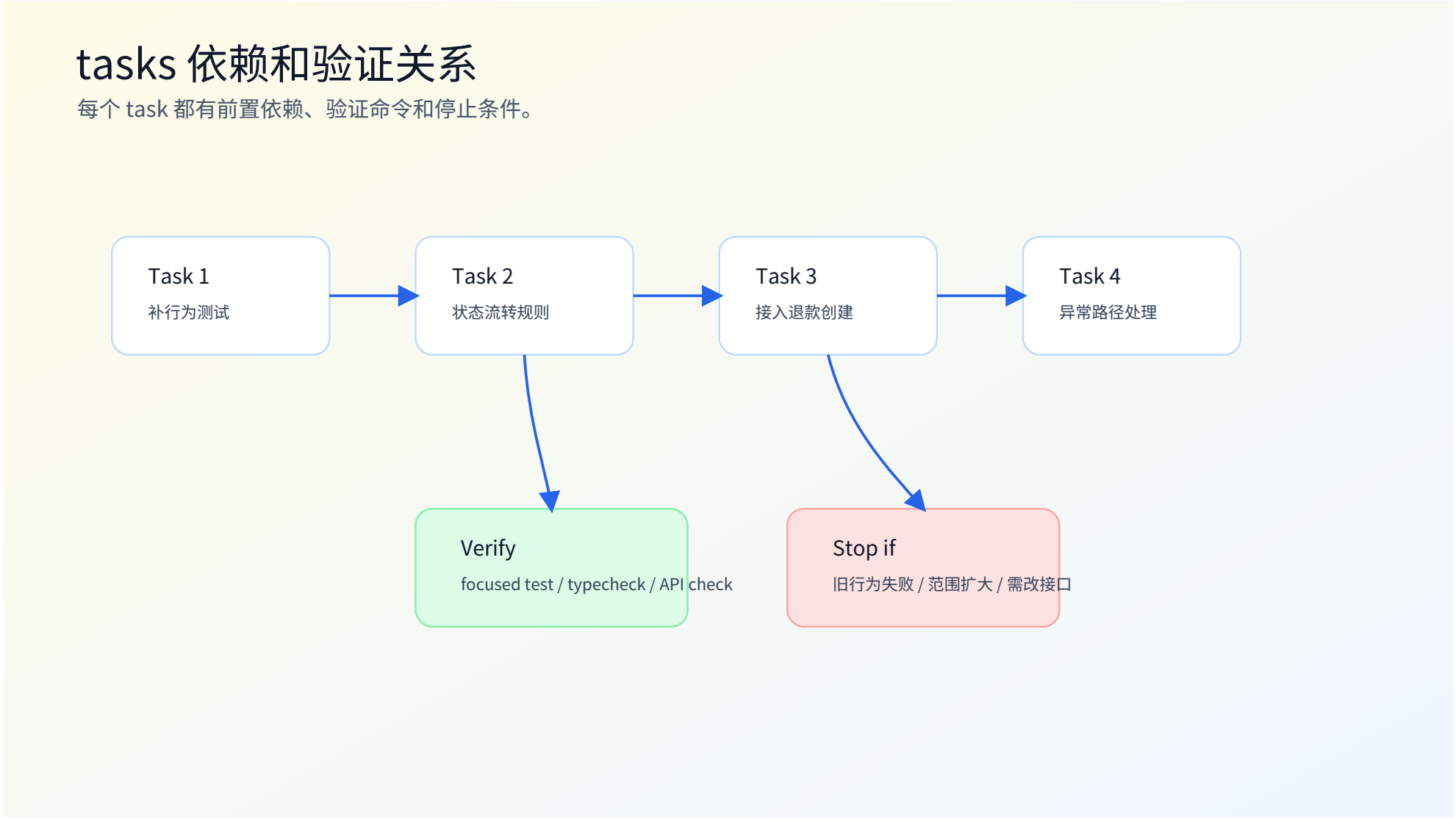


图 18-2　tasks 依赖和验证关系

tasks 不是平铺列表。它们有依赖。

比如：

```
Task 2 依赖 Task 1 的测试。
Task 3 依赖 Task 2 的状态结果。
Task 4 依赖 Task 3 的退款创建路径。
Task 5 依赖前面所有改动。
```

可以在 `tasks.md` 里写：

清单 18-12 任务依赖和验证关系

```
## Dependencies

- Task 2 must not start until Task 1 tests are added.
- Task 3 must not start until status transition tests pass.
- Task 5 must run only after all focused tests pass.
```

这能阻止 Agent 一口气做完。

18.9 让 AI 只执行一个 task

写好 `tasks.md` 后，不要说：

```
按 tasks.md 实现。
```

这会让 Agent 一次性跑完整个清单。

一次只授权一个 task：

示例 18-13 让 AI 只执行一个 task

```
只执行 `tasks.md` 的 Task 1。

要求：
- 不改实现代码。
- 不改 Task 1 以外文件。
- 跑指定验证命令。
- 输出 diff 摘要和测试结果。
- 完成后停止。
```

Task 1 完成后，人看 diff 和命令，再授权 Task 2。

对复杂需求，这种节奏比“一次全做完”更快。因为你少了大量返工。

18.10 测试先行怎么写进 tasks

不是每个任务都必须严格 TDD，但复杂需求建议先让行为测试出现。

Task 可以这样写：

清单 18-14 测试先行怎么写进 tasks

```
## Task 1: Add failing regression tests

Goal:
- Convert requirements into executable behavior checks.

Files:
- `test/order/cancel.test.ts`

Expected before implementation:
- pending and shipped cases pass.
- paid cancellation case fails because current behavior returns `canceled`.
- duplicate paid cancellation case fails because duplicate refund prevention is not implemented.

Stop if:
- pending or shipped cases fail before implementation.
```

这能让 AI 明确：测试失败不是坏事，预期失败是为了锁住需求。

18.11 tasks 里的验证命令怎么写

验证命令不能只写“run tests”。

要写具体命令和覆盖范围：

清单 18-15 tasks 里的验证命令怎么写

```
Verify:
- `npm test -- order-cancel`
  - covers pending, paid, shipped cancel behavior
- `npm test -- order`
  - covers broader order module impact
- API compatibility check
  - ensures response shape unchanged
```

如果命令不能本地跑，也要写：

清单 18-16 tasks 里的验证命令怎么写

```
Not locally verifiable:
- Real payment provider callback.

Manual check:
- staging callback path after merge candidate.
```

不要让 Agent 自己选择最容易通过的命令。

18.12 回滚边界怎么写

复杂需求里，tasks 还要考虑回滚。

清单 18-17 回滚边界怎么写

```
Rollback:
- Task 2 can be reverted independently if status transition tests fail.
- Task 3 should not be merged without Task 2.
- No database migration in this change.
- No public API response shape change.
```

回滚边界让任务更可审。

如果一个 task 无法单独回滚，说明它可能太大或依赖没写清。

18.13 task、commit 和 PR 不是一回事

很多人会把 `tasks.md` 误解成提交清单。实际生产里，task、commit、PR 的粒度可以不同。

task 是给 Agent 执行的最小授权单位。 commit 是本地历史记录单位。 PR 是给 reviewer 审查和交付的单位。

一个复杂需求可能是：

示例 18-18 task、commit 和 PR 不是一回事

```
Task 1: 补行为测试
Task 2: 改状态规则
Task 3: 接退款请求
Task 4: 补失败处理
Task 5: 跑扩大验证

Commit 1: tests for order cancel behavior
Commit 2: implement paid cancel refund flow
Commit 3: handle refund request failure

PR 1: paid order cancellation enters refund flow
```

也可能拆成两个 PR：

```
PR 1: backend status and refund request behavior
PR 2: frontend display for refund_requested
```

什么时候拆 PR?

· 前后端可以独立上线。

- 数据迁移需要先合并。
- reviewer 不同。
- 风险需要分阶段灰度。
- 一个 PR 已经大到无法在合理时间内审完。

不要机械要求一个 task 一个 commit，也不要把十几个 task 塞进一个难审 PR。真正的目标是让每个交付单位可理解、可验证、可回滚。

18.14 常见错误

错误一：tasks 只是大标题。

“实现后端”“实现前端”“补测试”都太大。要写文件范围和验证。

错误二：没有 Stop 条件。

失败后 Agent 会继续改，污染 diff。

错误三：任务顺序错。

先实现再补测试，容易让测试配合实现，而不是锁定需求。

错误四：一个 task 混多个层级。

状态机、接口、前端展示、文档更新不要混在同一个 task。

错误五：没有验收命令。

没有验证命令，task 完成只剩口头声明。

18.15 一个完整 tasks 示例

清单 18-19 一个完整 tasks 示例

```
# Tasks: paid order cancellation enters refund flow

## Task 1: Add behavior tests

Goal:
- Lock pending, paid, shipped and duplicate paid cancel behavior.

Files:
- `test/order/cancel.test.ts`

Verify:
- `npm test -- order-cancel`

Expected before implementation:
```

```
- pending and shipped cases pass.
- paid and duplicate paid cases fail.

Stop if:
- pending or shipped cases fail.

## Task 2: Add status transition rule

Goal:
- Add paid -> refund_requested transition.

Files:
- `src/order/status-transition.ts`

Verify:
- `npm test -- order-cancel`

Stop if:
- public API response shape must change.

## Task 3: Wire refund request creation

Goal:
- Create refund request when transition result is refund_requested.

Files:
- `src/order/cancel-service.ts`
- `src/refund/refund-request-service.ts`

Verify:
- `npm test -- order-cancel`

Stop if:
- refund request creation requires payment callback changes.

## Task 4: Add failure handling

Goal:
- Keep order paid if refund request creation fails.

Files:
- `src/order/cancel-service.ts`
- `test/order/cancel.test.ts`

Verify:
- `npm test -- order-cancel`

## Task 5: Broader verification

Verify:
- `npm test -- order`
- API response shape compatibility check

Report:
```

```
- changed files
- commands and results
- residual risk
```

这份 tasks 可以直接驱动 Agent 小步执行。

18.16 每个 task 完成后要输出什么

让 Agent 执行 task 时，不要只让它说“完成”。要让它输出可审查结果。

可以固定要求：

示例 18-20 每个 task 完成后要输出什么

```
完成后停止，并输出：
1. 改了哪些文件。
2. 每个文件为什么改。
3. 执行了哪些验证命令。
4. 命令结果是什么。
5. 是否触发 Stop 条件。
6. 剩余风险或需要人确认的问题。
```

对应到 Task 2，输出应该像这样：

清单 18-21 每个 task 完成后要输出什么

```
Task 2 report:

Changed files:
- `src/order/status-transition.ts`: add paid -> refund_requested transition
- `test/order/status-transition.test.ts`: update expected transition behavior

Verification:
- `npm test -- order-cancel` PASS

Stop conditions:
- public API response shape did not change
- no payment callback changes required

Residual risk:
- frontend display for `refund_requested` not covered in this task
```

这个报告不需要长，但必须有证据。否则你只能相信 Agent 的口头总结。

18.17 并行任务要非常谨慎

Agent 能并行做事，但不是所有 tasks 都能并行。

可以并行的任务通常满足：

- 文件范围不同。
- 依赖关系弱。
- 验证命令互不影响。
- 失败后不会污染同一段代码。

比如：

```
Task A：后端订单状态测试
Task B：前端状态文案展示
```

如果两者只共享接口契约，且契约已经在 requirements/design 中固定，可以并行。

不适合并行的任务：

```
Task 1：修改状态迁移规则
Task 2：基于新状态接退款创建
Task 3：基于退款创建补失败处理
```

这些任务有明确顺序。并行只会制造冲突和重复修复。

个人 AI Coding 里，我更建议默认串行。只有当任务边界特别清楚、文件不重叠、验证互不影响时，再让多个 Agent 并行。

18.18 task 审查协议

每执行完一个 task，先审四件事，再授权下一步。

第一，diff 是否只覆盖 task 范围。

如果 Task 1 只要求补测试，结果改了实现，直接退回。测试先行任务最容易被 Agent “顺手修掉”。

第二，验证命令是否真的跑了。

不要接受“应该通过”。要看命令、结果和失败输出。如果命令没法跑，要写清原因和替代验证。

第三，是否触发 Stop 条件。

Stop 条件被触发时，不要让 Agent 继续解释着往下做。先更新 requirements 或 design，再重拆 tasks。

第四，是否产生新风险。

比如实现时发现需要新增状态值，老客户端可能不识别。这不是 Task 2 自己可以决定的范围，应该回到 design 的 Compatibility。

可以直接把这段话作为执行要求：

```
执行 Task 3。完成后停止。
如果发现需要改 Task 3 以外的文件、需要新依赖、需要改 public API response shape，先报告，不要继续实现。
```

这类授权语句比“按 tasks 做”更适合真实项目。

18.19 任务失败要分类，不要马上让 Agent 猜修法

当一个 task 失败时，先分类，再决定下一步。

常见失败类型有五种。

表 18-1 任务失败要分类，不要马上让 Agent 猜修法

失败类型	例子	处理
预期失败	新增 paid cancel 测试失败	可以进入实现 task
基线失败	旧 pending 测试先失败	停止，先确认现有行为
范围冲突	实现需要改 public API	回到 design
环境失败	数据库或 mock 没启动	修验证环境或记录替代验证
设计错误	Current Flow 与源码不符	更新 design 和 tasks

不要把所有失败都交给 Agent “继续修”。它可能为了让命令通过，改掉测试、扩大范围，或者绕过真正的设计问题。

可以这样要求：

```
如果验证失败，先分类失败原因。
只有预期失败或 task 范围内的实现错误可以继续修。
如果是基线失败、范围冲突、环境失败或设计错误，停止并报告。
```

这条规则会明显减少大 diff 失控。

18.20 tasks 怎么映射到 PR 描述

任务做完后，PR 描述不要只写“完成订单取消改造”。应该把 tasks 的执行证据压缩进去。

清单 18-22 tasks 怎么映射到 PR 描述

```
What changed:
- paid order cancellation now creates refund request state
- pending and shipped cancellation behavior unchanged
```

```
- duplicate paid cancellation does not create duplicate refund request

Spec:
- `docs/specs/order-cancel-refund/requirements.md`
- `docs/specs/order-cancel-refund/design.md`
- `docs/specs/order-cancel-refund/tasks.md`

Task execution:
- Task 1 behavior tests: done
- Task 2 status transition: done
- Task 3 refund request wiring: done
- Task 4 failure handling: done
- Task 5 broader verification: done

Verification:
- `npm test -- order-cancel` PASS
- `npm test -- order` PASS

Not changed:
- payment provider callback
- admin batch cancellation
- historical data migration
```

这段 PR 描述给 reviewer 的不是过程流水账，而是审查索引。reviewer 可以沿着 requirements 看行为，沿着 design 看方案，沿着 tasks 看执行和验证。

18.21 本地验证、CI 和人工审查各管一段

`tasks.md` 里的验证命令不能替代 CI，也不能替代人工审查。它们负责的东西不同。

本地验证适合快速确认 task 是否做对：

```
focused unit tests
typecheck
lint
small integration tests
browser smoke test
```

CI 负责更完整的自动化检查：

示例 18-23 本地验证、CI 和人工审查各管一段

```
full test suite
build
package checks
e2e tests
security or dependency checks
required status checks
```

人工审查负责自动化覆盖不到的判断：

```
需求是否理解正确
diff 是否越界
设计层级是否合理
兼容性风险是否被接受
残余风险是否写清
```

Agent 可以运行本地命令，可以读取 CI 失败日志，也可以辅助做 review checklist。但它不能替你承担业务决策。

所以 tasks 的最后一步不要只写“跑测试”，可以写成：

清单 18-24 本地验证、CI 和人工审查各管一段

```
## Task 5: Delivery verification

Verify locally:
- `npm test -- order-cancel`
- `npm test -- order`
- `npm run typecheck`

Prepare PR evidence:
- summarize changed files
- map tests to requirements
- list not changed areas
- list residual risks

Wait for:
- CI required checks
- reviewer confirmation on compatibility
```

这段能把 Agent 执行、CI 和 review 连接起来。真实交付不是 Agent 说完成就结束，而是证据能通过机器和人的两层检查。

18.22 tasks 也要防止“验证污染”

AI Coding 里还有一种常见问题：实现没改对，但测试被改到通过。

所以在 tasks 里可以写清楚哪些验证资产不能被随意修改：

清单 18-25 tasks 也要防止“验证污染”

```
Protected verification:
- Do not weaken existing pending or shipped cancellation assertions.
- Do not delete response shape compatibility test.
- Do not replace behavior assertions with snapshot-only checks.
- Do not mark failing tests as skipped.
```

这类约束尤其适合放在测试先行任务之后。

如果 Agent 在 Task 2 修改实现时顺手改了 Task 1 的失败测试，要先问：这是需求变化，还是为了让测试通过？如果是需求变化，回到 requirements；如果只是为了通过测试，退回。

验证资产也是代码库的一部分。让 Agent 改测试没有问题，但它必须解释测试为什么要改，改动是否仍然锁住原始需求。

18.23 让 AI 生成 tasks 草稿

可以这样提示：

示例 18-26 让 AI 生成 tasks 草稿

```
先不要改代码。

请基于 requirements.md 和 design.md 生成 tasks.md 草稿。

要求：
- 每个 task 只产生一个可审查小 diff。
- 每个 task 写 Goal / Files / Verify / Stop if。
- 先测试或验证，再实现。
- 不要把重构和行为改动混在同一个 task。
- 标出任务依赖关系。
```

人审时重点看：

- Task 是否太大。
- 是否遗漏测试。
- 是否混入重构。
- Stop 条件是否具体。
- 验证命令是否匹配风险。

18.24 tasks 和 Agent 执行的关系

`tasks.md` 不是给 Agent 一次性消费的清单。

它更像分段授权表。

每次只授权一个 task：

```
执行 Task 2。
不要执行 Task 3。
完成后停止，输出 diff 摘要和验证结果。
```

如果 Task 2 失败，就不要继续 Task 3。

这能让人保持控制权，也能让 Agent 的上下文更干净。

这里还有一个生产里的细节：失败 task 不要直接改成“通过”。

先记录失败原因：

示例 18-27 tasks 和 Agent 执行的关系

```
failed command:
observed output:
likely category:
- implementation bug
- spec/design gap
- test expectation wrong
- environment issue
- scope expansion needed
decision:
- fix within task
- revise requirements/design
- split new task
- stop for human confirmation
```

这个记录很短，但能防止 Agent 在任务失败后把实现、测试和设计一起改掉。

如果失败原因属于 spec 或 design 缺口，不要在当前 task 里顺手补。先回到上游文档，把约束写清，再重新拆任务。这样后续 review 才能看出需求变化发生在哪一层。

18.25 迁移建议

下次复杂需求写完 design 后，先别实现。

让 AI 生成 tasks 草稿，然后你只审三件事：

- 每个 task 是否能形成小 diff。
- 每个 task 是否有验证命令。
- 每个 task 是否有停止条件。

如果这三件事过不了，就不要开工。

复杂需求不是靠一次强 prompt 做完的。它要被拆成一串能看、能测、能停的小步骤。

手写 requirements/design/tasks 能解决很多个人任务。当需求变更多、需要归档、需要 spec delta 和 proposal 时，就可以引入框架来管理这些规格资产。

18.26 常见问题

每个 task 都必须测试先行吗？

不必须。行为变更、bug 修复、状态机、接口兼容这类任务建议先写测试。纯文档、配置整理、验证补充可以先做检查命令或快照。关键是 task 开始前知道怎么判断完成。

Agent 一次执行多个 task 会不会更快？

小改动可以。复杂需求里，一次执行多个 task 经常让 diff 变大，失败也难定位。默认一个 task 一次授权，验证后再继续。

task 写得很细，会不会拖慢开发？

如果细到“新增一个 import”，会拖慢。合适粒度是一个 task 能形成独立可审 diff，并能用一个清楚的验证动作判断结果。

任务失败后要让 Agent 继续修吗？

看失败类型。如果是 task 范围内的测试失败，可以让它基于失败输出修。如果触发 Stop 条件，比如需要改公共接口、引入新依赖、推翻 design，就应该停下来重新审。

tasks.md 要不要长期保留？

中高风险需求建议保留，至少保留到 PR 合并后一段时间。它能解释改动顺序和验证证据。低风险小任务可以把任务摘要放在 PR 描述里，不一定保留文件。

18.27 可复查资料

- GitHub Spec Kit 官方文档：https://github.github.com/spec-kit/〔18-1〕
- GitHub Spec Kit 仓库：https://github.com/github/spec-kit〔18-2〕
- OpenSpec getting started：https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md〔18-3〕
- OpenSpec concepts：https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md〔18-4〕

本章资源

〔18-1〕 https://github.github.com/spec-kit/
https://github.github.com/spec-kit/

〔18-2〕 https://github.com/github/spec-kit
https://github.com/github/spec-kit

〔18-3〕 https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md
https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md

〔18-4〕 https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md
https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md

第19章 OpenSpec 实战：把需求变更放进 repo

手写 `requirements.md`、`design.md`、`tasks.md` 已经能解决很多个人任务，但它有一个明显短板：这些文件经常只服务一次开发，需求完成后就散落在分支、聊天记录或临时目录里。

问题通常出现在下一次变更。

你让 AI 改订单取消逻辑：已支付订单取消后进入退款流程。第一次做完时，`requirements.md` 写得很清楚，测试也跑过。两个月后，产品又要增加“部分发货订单允许取消未发货部分”。AI 需要知道现在系统到底怎么工作：

- pending 订单能不能取消。
- paid 订单是不是会进入 refund_requested。
- shipped 订单现在是否拒绝取消。
- duplicate cancel 有没有幂等保护。
- 退款请求失败时订单状态怎么处理。

如果这些信息只在旧 PR、旧聊天窗口和开发者脑子里，AI 很容易重新推断一遍，然后改出一组看起来合理但和现有系统冲突的代码。

OpenSpec 解决的不是“让 AI 更会写代码”，而是把需求变更纳入 repo：当前行为有稳定位置，拟议变更有独立目录，完成后再把变更合并回当前行为规格。

这对程序员很有用。因为真实项目里，AI Coding 最大的风险不是写不出代码，而是它不知道哪些行为是现在系统的合同，哪些只是你这次想改的内容。

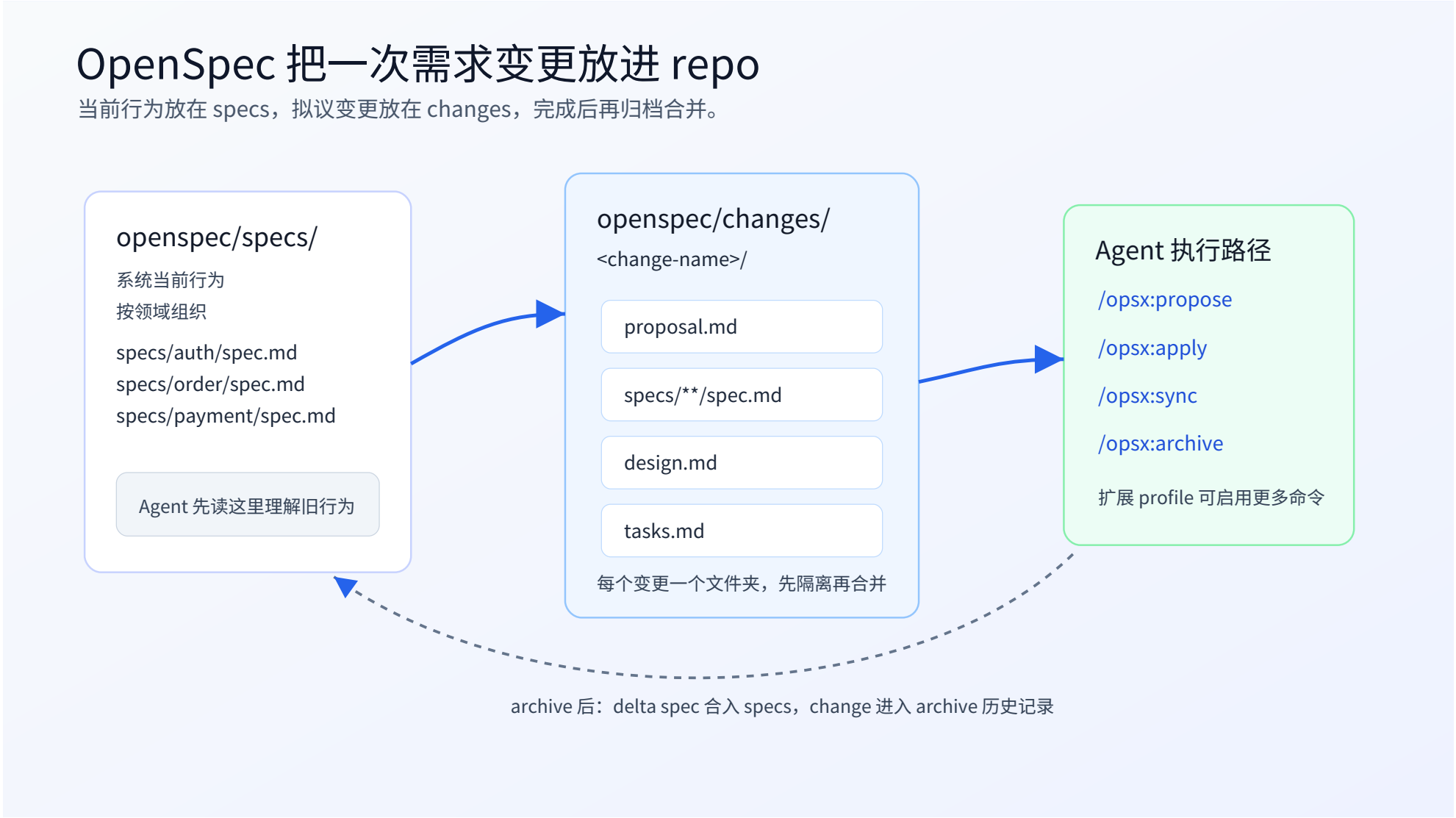


图 19-1　OpenSpec 变更工作流

19.1 OpenSpec 适合解决什么问题

OpenSpec 当前官方文档把它描述为一种和 AI coding assistant 对齐“要构建什么”的方式。它的 repo 结构围绕两个目录展开：

示例 19-1 OpenSpec 适合解决什么问题

```
openspec/
├── specs/              # 系统当前行为
│   └── <domain>/
│       └── spec.md
├── changes/            # 拟议变更
│   └── <change-name>/
│       ├── proposal.md
│       ├── design.md
│       ├── tasks.md
│       └── specs/
│           └── <domain>/
│               └── spec.md
└── config.yaml
```

这套结构背后的工程判断很直接：

`specs/` 写“系统现在怎么工作”，它不是愿望清单。

`changes/` 写“这次准备怎么改”，它还没有进入当前行为。

`archive` 以后，变更里的 delta spec 才合并回 `specs/`，变成新的系统事实。

对个人 AI Coding 来说，这比把所有需求都塞进一个 prompt 稳得多。Agent 可以先读 `openspec/specs/` 理解旧行为，再读 `openspec/changes/<change>/` 理解当前变更，最后按 `tasks.md` 小步实现。

19.2 先确认它是不是该用在这次任务上

OpenSpec 不是每个小改动都要上。

这类任务可以直接用轻量 SDD：

- 改一个字段展示。
- 补一个边界测试。
- 修一个明确 bug。
- 做一次小范围重命名。
- 调整一段内部实现，不改变外部行为。

这类任务更适合 OpenSpec：

- 需求会改变系统行为合同。

- 改动跨多个模块。
- 需要保留变更历史，方便后续 AI 继续接手。
- 同一领域会反复迭代。
- 需要同时写清当前行为和拟议行为。
- 需求容易被模型误读，例如状态机、权限、支付、库存、风控、消息投递。

一个简单判断：如果你已经准备写 `requirements.md`、`design.md`、`tasks.md`，并且希望这次变更完成后还能沉淀为长期规格，OpenSpec 就值得考虑。

如果只是改按钮文案，用它会增加不必要的目录和步骤。

19.3 安装和初始化要先看前置条件

OpenSpec 官方安装文档会列出当前 Node.js 版本要求和支持的安装方式。写入项目规则前，先以官方安装文档和本机 `openspec --version` 结果为准。最常见的是：

清单 19-2 安装和初始化要先看前置条件

```
npm install -g @fission-ai/openspec@latest
openspec --version
```

进入项目根目录后初始化：

清单 19-3 安装和初始化要先看前置条件

```
cd your-project
openspec init
```

不要在一个脏工作区里直接初始化。先看工作树：

清单 19-4 安装和初始化要先看前置条件

```
git status --short
```

建议在单独分支里做：

清单 19-5 安装和初始化要先看前置条件

```
git switch -c chore/adopt-openspec
openspec init
git status --short
```

你要看的不是命令有没有输出“成功”，而是它给 repo 增加了什么文件：

清单 19-6 安装和初始化要先看前置条件

```
find openspec -maxdepth 3 -type f | sort
```

初始化之后，先不要急着让 Agent 实现需求。第一步应该是补最核心的现有行为 spec。没有现有行为，后面的 delta spec 就缺参照物。

例如订单领域可以先有：

清单 19-7 安装和初始化要先看前置条件

```
# Order Cancellation Spec

## Requirements

### Requirement: Pending order cancellation
The system SHALL allow a pending order to be canceled.

#### Scenario: Pending order canceled by user
- GIVEN a pending order
- WHEN the user cancels the order
- THEN the order status becomes canceled

### Requirement: Shipped order cancellation
The system SHALL reject cancellation for shipped orders.

#### Scenario: Shipped order cannot be canceled
- GIVEN a shipped order
- WHEN the user cancels the order
- THEN the cancellation is rejected
- AND the order status remains shipped
```

这段文字不是给人看的“产品说明”，而是后续 Agent 读取的系统合同。它应该尽量贴近真实行为，不要写未来设想。

19.4 用 propose 开一个变更

OpenSpec 当前 getting started 文档给出的默认快速路径是：

```
/opsx:propose -> /opsx:apply -> /opsx:sync -> /opsx:archive
```

默认 global profile 是 `core`，包含 `propose`、`explore`、`apply`、`sync`、`archive`。扩展流程里的 `new`、`ff`、`continue`、`verify` 需要通过 profile 配置启用后更新。写到团队或个人规则里时，最好把这一点写清楚，避免不同机器上的命令不一致。

针对“已支付订单取消进入退款流程”，不要只说：

```
/opsx:propose 支持付费订单取消
```

这个描述太短，Agent 会自己补很多业务假设。

把业务边界写进输入：

示例 19-8 用 propose 开一个变更

```
/opsx:propose paid order cancellation should enter refund flow

Current behavior:
- pending orders can be canceled directly.
- shipped orders cannot be canceled.
- paid orders currently become canceled directly.

New behavior:
- paid orders should become refund_requested.
- a refund request should be created once.
- duplicate cancellation must not create duplicate refund requests.
- if refund request creation fails, order status should remain paid.

Out of scope:
- payment provider callback behavior.
- public API response shape.
- database schema migration.

Verification:
- focused order cancellation tests.
- idempotency test for duplicate paid cancellation.
- broader order module tests after focused tests pass.
```

这段输入有三层价值：

第一，它把旧行为和新行为分开。

第二，它明确不改什么。

第三，它把验证命令和风险放在变更开头，避免 Agent 只根据 happy path 生成设计。

OpenSpec 会为这个 change 创建独立目录。你要检查这些文件，而不是直接进入实现：

示例 19-9 用 propose 开一个变更

```
openspec/changes/paid-order-cancel-refund/
├── proposal.md
├── design.md
├── tasks.md
└── specs/
    └── order/
        └── spec.md
```

如果目录名、领域名、文件组织明显不符合你的 repo，就先改这些规格文件，不要让 Agent 带着错误结构实现。

19.5 proposal 要写清为什么改，不能替代 spec

`proposal.md` 适合写这次变更的动机、范围和方案概览。

比较好的内容是：

清单 19-10 proposal 要写清为什么改，不能替代 spec

```
# Proposal: Paid order cancellation enters refund flow

## Why
Paid orders cannot be treated like pending orders because payment capture has already happened.
Directly marking them canceled loses refund state and makes duplicate cancel requests unsafe.

## What Changes
- Paid order cancellation creates a refund request.
- Paid order status becomes refund_requested.
- Duplicate paid cancellation does not create another refund request.
- Refund request creation failure leaves the order paid.

## Out of Scope
- Payment provider callback flow.
- Public API response shape change.
- Database schema migration.
```

不要把 `proposal.md` 写成完整技术方案，也不要把所有验收条件都堆在这里。proposal 负责“为什么”和“改什么”，真正的行为变化应该进入 delta spec。

很多人用 AI 写 spec 时会犯一个错：把所有内容都写进一个大文档。读起来完整，执行时很难定位。OpenSpec 的价值恰恰在于拆开这些职责。

19.6 delta spec 是 OpenSpec 最关键的部分

OpenSpec 的 `changes/<change>/specs/<domain>/spec.md` 不是普通需求文档，而是相对当前 `openspec/specs/` 的变化。

官方文档里 delta spec 使用这些区域表达变化：

清单 19-11 delta spec 是 OpenSpec 最关键的部分

```
## ADDED Requirements
## MODIFIED Requirements
## REMOVED Requirements
```

已支付订单取消可以写成：

清单 19-12 delta spec 是 OpenSpec 最关键的部分

```
# Delta for Order

## MODIFIED Requirements

### Requirement: Paid order cancellation
The system SHALL move paid orders to refund_requested when a user cancels them.

#### Scenario: Paid order enters refund flow
- GIVEN a paid order without an existing refund request
- WHEN the user cancels the order
- THEN the order status becomes refund_requested
- AND exactly one refund request is created

#### Scenario: Duplicate paid cancellation is idempotent
- GIVEN a paid order that already has a refund request
- WHEN the user cancels the order again
- THEN no additional refund request is created
- AND the order remains refund_requested

#### Scenario: Refund request creation fails
- GIVEN a paid order
- WHEN refund request creation fails during cancellation
- THEN the order remains paid
- AND the failure is reported to the caller
```

这段 spec 有几个细节值得注意。

它没有写代码结构，也没有指定函数名。行为规格先描述系统必须满足什么。

它写了异常路径。AI 很容易只实现“paid -> refund_requested”，但真实生产问题往往出在重复请求和部分失败。

它写了可测试场景。后面的 `tasks.md` 可以直接从这些 scenario 拆出测试任务。

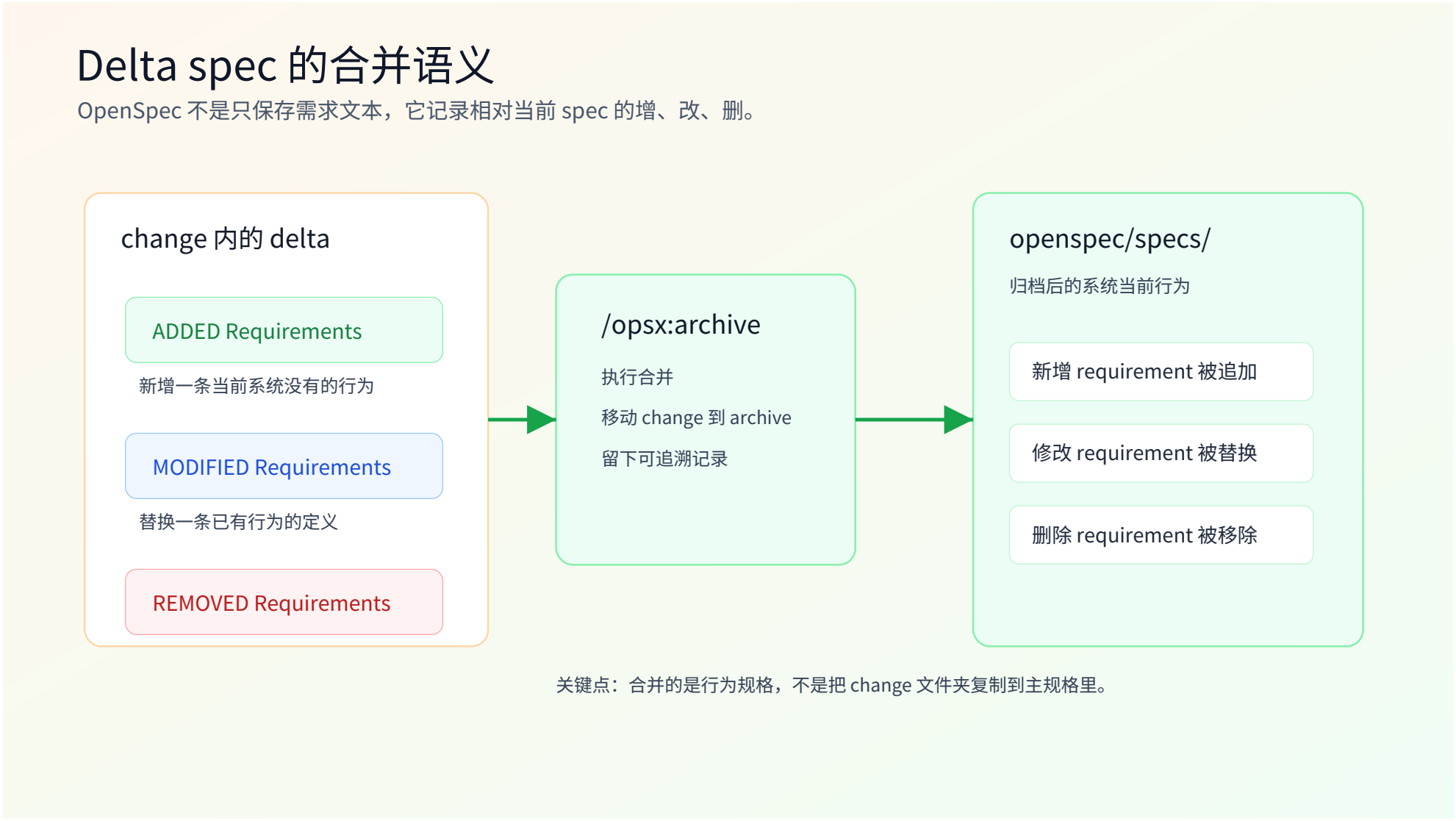


图 19-2　Delta spec 合并语义

19.7 design 负责技术约束，不负责重新发明需求

`design.md` 应该把实现边界写清楚。它的重点不是“我们将实现一个优雅的退款流程”，而是下面这些具体问题：

- 哪个模块拥有订单状态转换。
- 哪个模块负责创建退款请求。
- 状态更新和退款请求创建是否需要事务。
- 幂等键怎么生成。
- 失败时是否回滚状态。
- 哪些接口响应不能改变。
- 这次是否需要数据迁移。

一份可执行的 `design.md` 可以这样写：

清单 19-13 design 负责技术约束，不负责重新发明需求

```
# Design: Paid order cancellation enters refund flow

## Current Code Ownership
- `OrderCancelService` owns user cancellation orchestration.
- `OrderStatusTransition` owns allowed status transitions.
- `RefundRequestService` owns refund request creation.

## Proposed Flow
1. `OrderCancelService` loads the order.
2. For pending orders, keep existing pending -> canceled behavior.
3. For shipped orders, keep existing rejection behavior.
4. For paid orders:
   - validate paid -> refund_requested transition.
   - create refund request with idempotency key `order:<orderId>:cancel`.
   - persist status only when refund request creation succeeds.

## Compatibility
- Do not change public API response shape.
- Do not change payment provider callback handling.
- Do not add a database migration in this change.

## Failure Handling
- If refund request creation fails, keep order status paid.
- Return existing error envelope.

## Verification
- Focused cancellation tests.
- Duplicate cancellation test.
- Broader order module tests.
```

这里有一个关键点：design 不是越长越好。

如果 design 写了很多“采用分层架构”“提高可维护性”，但没有说清事务边界、幂等策略和兼容性，它对 AI 没什么约束力。

AI Coding 里真正有用的 design，是能防止模型越界的 design。

19.8 tasks 要让 Agent 小步执行

`tasks.md` 不能只是：

清单 19-14 tasks 要让 Agent 小步执行

```
- 实现退款流程
- 补测试
- 调整文档
```

这会让 Agent 一口气改完，最后给你一个很大的 diff。

拆成可验证任务：

清单 19-15 tasks 要让 Agent 小步执行

```
# Tasks

## 1. Add focused cancellation tests
- [ ] Add paid cancellation enters refund_requested test.
- [ ] Add duplicate paid cancellation idempotency test.
- [ ] Add refund request failure keeps order paid test.

Verify:
- `npm test -- order-cancel`

Stop if:
- existing pending or shipped cancellation tests fail before implementation.

## 2. Add paid status transition
- [ ] Add paid -> refund_requested transition.
- [ ] Keep pending and shipped behavior unchanged.

Verify:
- `npm test -- order-cancel`

Stop if:
- public API response shape must change.

## 3. Wire refund request creation
- [ ] Create refund request with idempotency key.
- [ ] Persist status only after refund request creation succeeds.

Verify:
- `npm test -- order-cancel`
```

```
Stop if:
- payment callback code needs to change.

## 4. Run broader verification
- [ ] Run order module tests.
- [ ] Record changed files and residual risks.
```

你可以让 Agent 先执行 Task 1，完成后停止：

示例 19-16 tasks 要让 Agent 小步执行

```
只执行 openspec/changes/paid-order-cancel-refund/tasks.md 的 Task 1。

要求：
- 不改实现代码。
- 只新增或修改 focused cancellation tests。
- 跑 tasks.md 里指定的验证命令。
- 输出失败用例和原因。
- 完成后停止。
```

这和“按 OpenSpec 实现”差别很大。前者保留人对 diff 和节奏的控制，后者容易变成一次性大改。

19.9 sync 和 archive 不是收尾动作那么简单

OpenSpec 默认快速路径里有 `/opsx:sync` 和 `/opsx:archive`。

`sync` 的工程意义是：实现过程中发现规格、设计、任务和代码不一致时，把 artifact 拉回一致状态。比如 Task 2 做到一半才发现当前 repo 里 `RefundRequestService` 已经有幂等逻辑，不应该重复实现。此时要更新 design 和 tasks，而不是让聊天记录承担这部分事实。

`archive` 的工程意义更大。根据当前 getting started 文档，归档时：

- `ADDED` requirements 会追加到主 spec。
- `MODIFIED` requirements 会替换已有版本。
- `REMOVED` requirements 会从主 spec 删除。
- change 文件夹会移动到 `openspec/changes/archive/`，作为历史记录。

这一步完成后，`openspec/specs/order/spec.md` 才描述新的系统当前行为。

如果代码已经合并，但没有 archive，后续 AI 再读 `openspec/specs/` 时还会看到旧行为。它会以为 paid order cancellation 仍然直接 canceled。这样的 spec 比没有 spec 更危险，因为它给了 Agent 一个过期但看起来权威的上下文。

19.10 归档前要做的检查

不要把 `/opsx:archive` 当作“文件整理命令”。归档前至少检查这些点：

清单 19-17 归档前要做的检查

```
git diff -- openspec/specs openspec/changes
git diff --stat
```

重点看：

- delta spec 是否只写本次变更。
- 主 spec 是否已有对应的旧 requirement 可替换。
- design 是否和最终代码一致。
- tasks 是否还有未完成项。
- 验证命令是否真实跑过。
- change 里有没有聊天废话或临时结论。

如果你的项目有测试命令，先跑聚焦测试，再跑受影响模块测试：

清单 19-18 归档前要做的检查

```
npm test -- order-cancel
npm test -- order
```

如果测试不能本地跑，要在 change 里记录原因和替代验证方式。不要让 Agent 写“all tests passed”，除非你能看到命令和输出。

19.11 OpenSpec 和轻量 SDD 的边界

轻量 SDD 的好处是快。一个 `.ai/tasks/<feature>/requirements.md` 加上 `design.md`、`tasks.md`，就能约束一次复杂任务。

OpenSpec 的好处是变更可归档。它多了三个关键能力：

- `specs/` 保存当前系统行为。
- `changes/` 隔离拟议变更。
- delta spec 在 archive 后合并进主规格。

如果你的项目长期由同一个人维护，且 AI 只做小任务，轻量 SDD 足够。

如果一个领域会反复改，或者你希望 AI 以后能读取历史行为，不要每次都重新解释，OpenSpec 更合适。

这不是工具信仰问题，而是维护成本问题。规格资产越长期，越需要明确“当前事实”和“拟议变更”的边界。

19.12 常见坑

坑一：没有先补当前 spec。

只写 change，不写 `openspec/specs/`，delta spec 就没有可靠参照。Agent 会把“当前行为”也当成自己可以猜的东西。

坑二：把 proposal 写成大杂烩。

proposal 里混入需求、设计、任务、验证和讨论结论，后续很难维护。它只需要写清为什么改、改什么、不改什么。

坑三：delta spec 写实现细节。

spec 应该写行为，不应该写“调用某个函数”“新增某个类”。这些放进 design。

坑四：archive 前没有同步 artifacts。

代码实现和 design 已经不一致，但 archive 仍然合并旧 design。后续 AI 会被这个过期设计带偏。

坑五：让 Agent 一次执行完整 change。

OpenSpec 管理的是变更，不代表每次都要全自动做完。复杂任务仍然应该按 tasks 小步授权。

坑六：把官方命令能力写死。

OpenSpec 仍在演进，不同 profile、不同 agent 集成可能有差异。项目规则里要写“以本 repo 当前安装结果为准”，并保留 `openspec --version`、`openspec list`、`openspec show` 这类检查动作。

19.13 可以先从一个高价值领域开始

不要一上来给整个项目补完整 specs。

先选一个 AI 经常改错、业务规则又比较稳定的领域：

- 订单状态。
- 权限校验。
- 支付退款。
- 库存扣减。
- 消息投递。
- 任务调度。

先补 3 到 5 条当前行为 requirement。然后选一个真实变更走完整流程：

示例 19-19 可以先从一个高价值领域开始

```
补当前 specs
开 changes/<change-name>
写 proposal
写 delta spec
写 design
写 tasks
按 task 小步实现
sync artifacts
archive 合并
```

做完后，把下一次 AI Coding 的上下文入口改成：

```
先读取 openspec/specs/order/spec.md。
再读取 openspec/changes/<change-name>/proposal.md、specs/order/spec.md、design.md、tasks.md。
只执行 tasks.md 中当前授权的 task。
```

到这一步，OpenSpec 才算进入个人工作流。它带来的变化不在目录数量，而在 AI 理解项目的入口。

19.14 改完后重点看什么

落地 OpenSpec 后，先看四个结果：

- AI 是否先读 `openspec/specs/`，再读当前 change。
- 需求讨论是否沉淀到 proposal 和 delta spec，而不是留在聊天记录。
- 实现过程中发现的新事实是否同步回 design/tasks。
- archive 后主 spec 是否能独立描述新系统行为。

如果这四点做不到，OpenSpec 只是换了一个地方写文档。

如果这四点能做到，它会把 AI Coding 从“这次怎么问”推进到“这个 repo 如何长期被 AI 正确理解”。

19.15 常见问题

OpenSpec 会不会太重？

对小改动会。对状态机、权限、支付、库存这类反复变化的领域，它反而能减少重复解释和返工。

可以只用 OpenSpec，不写测试吗？

不建议。spec 是行为合同，测试是可执行证据。AI 最容易在自然语言和实现之间出现偏差，测试能把偏差暴露出来。

delta spec 和 requirements.md 有什么区别？

普通 `requirements.md` 往往描述这次要什么。delta spec 明确描述相对当前 spec 的增、改、删，并能在 archive 后合并回主 spec。

OpenSpec 能代替设计评审吗?

不能。它能让设计和变更有固定位置，但事务边界、兼容性、风险取舍仍然需要程序员判断。

已经有产品 PRD 了，还需要 proposal 吗?

需要时可以把 PRD 摘要到 proposal。PRD 通常面向产品意图，proposal 要面向 repo 里的工程变更范围。

19.16 可复查资料

- OpenSpec 官方站点：https://openspec.dev/〔19-1〕
- OpenSpec GitHub 仓库：https://github.com/Fission-AI/OpenSpec/〔19-2〕
- OpenSpec 安装文档：https://github.com/Fission-AI/OpenSpec/blob/main/docs/installation.md〔19-3〕
- OpenSpec getting started：https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md〔19-4〕
- OpenSpec concepts：https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md〔19-5〕

本章资源

〔19-1〕 https://openspec.dev/
https://openspec.dev/

〔19-2〕 https://github.com/Fission-AI/OpenSpec/
https://github.com/Fission-AI/OpenSpec/

〔19-3〕 https://github.com/Fission-AI/OpenSpec/blob/main/docs/installation.md
https://github.com/Fission-AI/OpenSpec/blob/main/docs/installation.md

〔19-4〕 https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md
https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md

〔19-5〕 https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md
https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md

第20章 OpenSpec、Spec Kit 与轻量 SDD 怎么选

程序员做 AI Coding，很容易在 SDD 这里走偏。

一种偏法是完全不用规格。所有需求都写进聊天窗口，要求 AI “理解代码后实现”。这在小改动里能跑通，在复杂需求里很快失控：旧行为说不清，技术边界说不清，测试意图说不清，最后只剩一个很大的 diff。

另一种偏法是看到框架就上框架。一个字段展示改动，也要搞完整 spec、plan、tasks、workflow。流程看起来正规，实际成本比收益高。

不要先问“哪个工具更强”，先看任务形态：

- 这次变更是不是一次性的。
- 它会不会改变长期行为合同。
- 是否已有系统行为需要作为 source of truth。
- 是否需要工具帮你生成、校验、串联完整 SDD artifact。
- 后续 AI 是否会反复接手同一领域。

结论先说：

小而明确、一次性的复杂任务，用轻量 SDD。

存量系统里反复变化的业务领域，用 OpenSpec。

偏 0 到 1、希望用工具链驱动完整 Spec -> Plan -> Tasks -> Implement 流程，或者需要多 Agent 集成，用 Spec Kit。

不要把三者理解成低级、中级、高级。它们解决的是不同问题。



图 20-1　SDD 工具选择地图

20.1 先把三者的边界说清楚

轻量 SDD 是一种 repo-local 写法，不是一个官方框架。

它通常就是在项目里放一个临时规格目录：

```
.ai/tasks/paid-order-cancel-refund/
├── requirements.md
├── design.md
├── tasks.md
└── verification.md
```

它的优势是启动成本低。你不需要安装工具，不需要改项目模板，不需要学习新命令。只要把需求、设计、任务和验证写到固定位置，就能让 Agent 读到稳定上下文。

它的弱点也明显：完成后这些文件是否归档、是否成为长期系统合同、下一次 AI 是否会优先读取，全靠你自己的工作习惯。

OpenSpec 是变更管理层。

官方文档当前把 `openspec/specs/` 定义为系统当前行为，把 `openspec/changes/` 定义为拟议变更。每个 change 里有 `proposal.md`、`design.md`、`tasks.md` 和 delta specs。归档时，delta spec 里的 ADDED、MODIFIED、REMOVED 会合并回主 `specs/`。

它的优势是把“当前事实”和“拟议变更”分开。对存量系统很重要。

它的弱点是需要你维护主 specs。如果主 specs 过期，Agent 会被一个看起来权威但实际错误的上下文带偏。

Spec Kit 是完整 SDD 工具链。

GitHub Spec Kit 官方文档当前描述的核心流程是 Spec -> Plan -> Tasks -> Implement。quick start 里还推荐在有歧义的生产功能中加入 `/speckit.clarify`、`/speckit.checklist`、`/speckit.analyze`，并提供多种 AI coding agent integration、extensions、presets、workflows 等能力。

它的优势是流程完整、集成面广、模板和命令体系清楚，适合从需求到实现的结构化推进。

它的弱点是对小任务会显得重，而且常被误用成“把完整流程跑一遍就等于质量更高”。真正的质量仍然来自规格质量、代码审查和验证证据。

20.2 用一张决策图先分流

选型流程：先问工程问题，再选工具

不要因为工具更完整就默认上框架，也不要因为怕麻烦而丢掉规格资产。

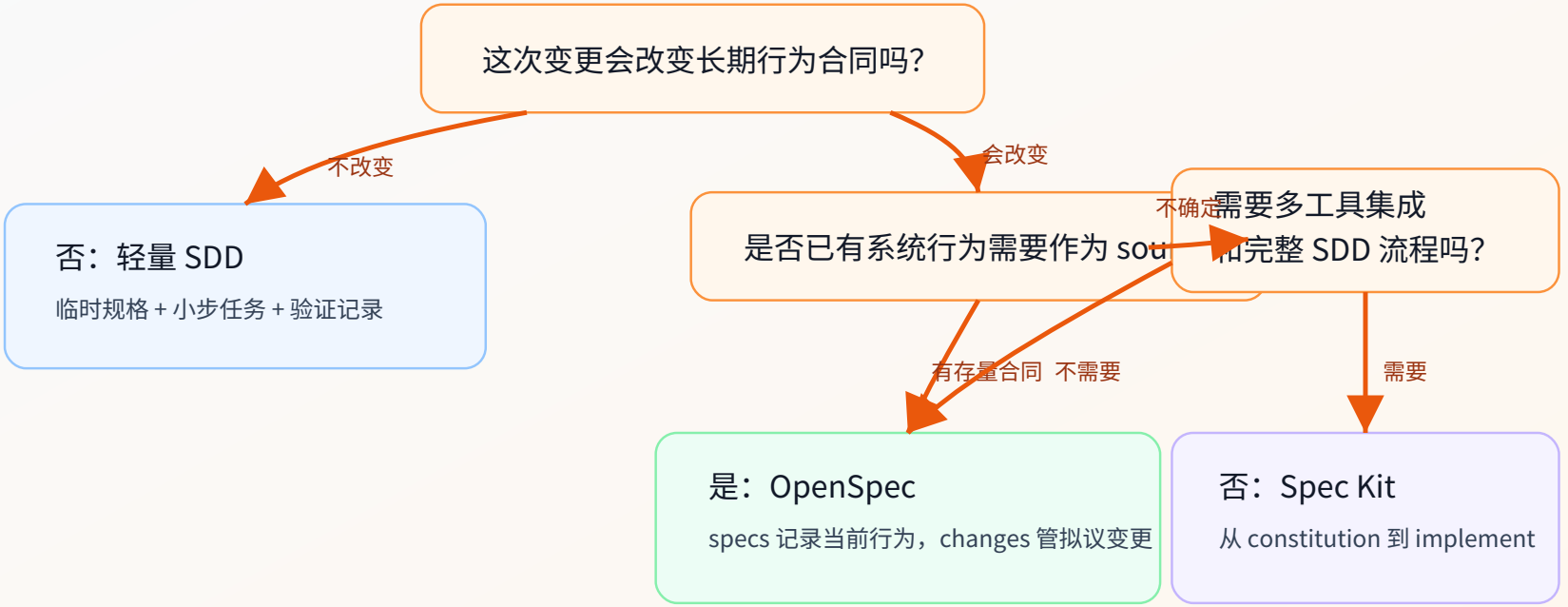


图 20-2　SDD 选型流程

实际工作里，我会先问第一个问题：

```
这次变更会改变长期行为合同吗？
```

不会，就用轻量 SDD。

例如：

- 给已有 API 增加一个日志字段。
- 修一个明确的空指针。
- 修改页面加载状态文案。
- 给已存在行为补一组回归测试。

这些任务需要上下文和验证，但不一定需要长期规格资产。你只要写清：

```
需求是什么
不改什么
涉及哪些文件
怎么验证
失败后怎么停
```

就够了。

第二个问题：

```
是否已有系统行为需要作为 source of truth？
```

有，就优先考虑 OpenSpec。

例如订单、支付、权限、库存、风控、结算、消息投递。这些领域里，AI 最容易犯的错不是语法，而是重新解释业务规则。OpenSpec 的 `specs/` 可以让它先读取当前行为，再处理 `changes/` 里的拟议变更。

第三个问题：

```
是否需要工具链帮你管理完整 SDD 流程和 Agent 集成？
```

需要，就考虑 Spec Kit。

例如你在做一个新功能包、新服务、新工具，想让工具生成规格、计划、任务，并且希望不同 AI coding agent 都能进入同一套流程。Spec Kit 的 integration、command、template、preset 更适合这类场景。

20.3 轻量 SDD：最适合个人高频使用

轻量 SDD 的最大价值，是让你不再把复杂需求交给一次聊天。

它适合这些任务：

- 需求不大，但有业务边界。
- 改动不需要长期归档成系统 spec。
- 你希望快速把 Agent 限制在小 diff 里。
- 项目不方便引入额外工具。
- 你还在摸索一个团队或个人的写法，不想过早绑定框架。

一个比较稳的目录是：

```
.ai/tasks/<task-name>/
├── requirements.md
├── design.md
├── tasks.md
└── verification.md
```

`requirements.md` 写行为：

清单 20-1 轻量 SDD：最适合个人高频使用

```
# Requirements: paid order cancellation

## Current Behavior
- Pending orders can be canceled directly.
- Shipped orders reject cancellation.
- Paid orders currently become canceled directly.

## Target Behavior
- Paid orders become refund_requested.
- Exactly one refund request is created.
- Duplicate cancellation does not create duplicate refund requests.
- Refund creation failure leaves the order paid.

## Out of Scope
- Payment provider callback behavior.
- Public API response shape.
- Database schema migration.
```

`design.md` 写技术边界：

清单 20-2 轻量 SDD：最适合个人高频使用

```
# Design

## Ownership
- `OrderCancelService` orchestrates cancellation.
- `OrderStatusTransition` owns allowed transitions.
- `RefundRequestService` owns refund request creation.

## Compatibility
- Keep API response shape unchanged.
- Do not change payment provider callback.
- No database migration in this change.

## Failure Handling
- Persist refund_requested only after refund request creation succeeds.
```

`tasks.md` 写小步 diff：

清单 20-3 轻量 SDD：最适合个人高频使用

```
# Tasks

## Task 1: Add focused tests
Files:
- `test/order/cancel.test.ts`

Verify:
- `npm test -- order-cancel`

Stop if:
- existing pending or shipped behavior fails before implementation.

## Task 2: Add status transition
Files:
- `src/order/status-transition.ts`

Verify:
- `npm test -- order-cancel`

Stop if:
- public API response shape must change.
```

`verification.md` 记录证据：

清单 20-4 轻量 SDD：最适合个人高频使用

```
# Verification

## Commands

```bash
npm test -- order-cancel
```
```

```
npm test -- order
```

Results
- `npm test -- order-cancel`: pass
- `npm test -- order`: pass

Residual Risk
- Payment provider callback was out of scope.
```

注意，这里没有为了“显得工程化”新建一个样板项目。它直接服务真实变更。

轻量 SDD 的风险也要看清：

- 没有工具强制你归档。
- 没有 delta spec 合并语义。
- 没有跨 Agent 的命令集成。
- 文档结构全靠项目规则维护。

所以它适合个人高频工作流，不适合承担长期行为合同。

## 20.4 OpenSpec：适合存量系统的行为变更

OpenSpec 的强项，是处理已有系统的变更。

官方 concepts 文档强调它是 brownfield-first，delta-based 的方式让你能指定对现有行为的修改，而不是只描述新系统。这一点对 AI Coding 很关键。

很多真实需求不是“做一个新能力”，而是：

- 旧逻辑 A 保持不变。
- 旧逻辑 B 改成 B2。
- 旧逻辑 C 删除。
- 新逻辑 D 加进来。

如果这些都写在一个 prompt 里，Agent 很容易把 A 也顺手改了。

OpenSpec 的目录给了明确边界：

示例 20-5 OpenSpec：适合存量系统的行为变更

```
openspec/
├── specs/
│   └── order/spec.md
└── changes/
    └── paid-order-cancel-refund/
```
```

```
        ├── proposal.md
        ├── design.md
        ├── tasks.md
        └── specs/order/spec.md
```

主 `specs/order/spec.md` 写当前行为：

清单 20-6 OpenSpec：适合存量系统的行为变更

```
### Requirement: Pending order cancellation
The system SHALL allow pending orders to be canceled.

### Requirement: Shipped order cancellation
The system SHALL reject cancellation for shipped orders.
```

change 里的 delta spec 写变化：

清单 20-7 OpenSpec：适合存量系统的行为变更

```
## MODIFIED Requirements

### Requirement: Paid order cancellation
The system SHALL move paid orders to refund_requested when a user cancels them.
```

这个结构比轻量 SDD 多了一层语义：当前系统事实和当前变更不是同一个东西。

OpenSpec 适合这些情况：

- 同一领域会连续多次变更。
- 你希望 AI 每次先读取当前行为。
- 需求有增、改、删语义。
- 变更完成后需要归档成新的系统事实。
- 需要保留 change 历史，方便后续追溯。

OpenSpec 不适合这些情况：

- 项目还没有形成稳定业务领域。
- 只有一次性小改动。
- 没人维护 `openspec/specs/`。
- 团队会把 spec 当成形式文档，代码变了 spec 不更新。

最后这一点最重要。OpenSpec 不是自动真理机。它只是给规格资产一个结构。主 specs 是否可靠，仍然取决于你是否在变更后同步和归档。

20.5 Spec Kit：适合完整 SDD 流程和工具集成

Spec Kit 更像一个完整的 SDD 工作台。

从官方 quick start 看，它不是只提供一个模板目录，而是通过 Specify CLI 初始化项目，并给 AI coding agent 提供命令。核心命令包括：

```
/speckit.constitution
/speckit.specify
/speckit.plan
/speckit.tasks
/speckit.implement
```

生产功能里，官方 quick start 还建议把这些质量动作纳入常规流程：

```
/speckit.clarify
/speckit.checklist
/speckit.analyze
```

这几个命令背后的工程意图很好理解：

`constitution` 写项目原则和长期约束。

`specify` 写要做什么和为什么做，尽量不在这个阶段陷入技术栈。

`clarify` 处理规格里的歧义。

`checklist` 检查需求质量。

`plan` 进入技术方案。

`tasks` 拆任务。

`analyze` 检查 spec、plan、tasks 之间的一致性。

`implement` 执行。

Spec Kit 适合这些情况：

- 你要从 0 到 1 做一个较完整功能或工具。
- 你希望工具直接生成 SDD 目录、脚本和命令。
- 你希望同一个 SDD 流程能被不同 Agent 使用。
- 你需要模板、preset、extension 去适配团队或个人规范。
- 你愿意接受更完整的流程成本，换取更强的结构化。

Spec Kit 不适合这些情况：

- 只是修一个局部 bug。

- 当前项目不想引入新的 CLI 和模板。
- 你还没有能力审它生成的 spec、plan、tasks。
- 你把 `/speckit.implement` 当成“自动写完并保证正确”的按钮。

最后一条尤其要警惕。Spec Kit 能让上下文更结构化，但不能替代工程判断。`analyze` 能检查 artifact 之间的一致性，不等于业务一定正确；`implement` 能驱动实现，不等于不用做代码审查和测试。

20.6 三者怎么映射到真实工作

同一个需求，在三种路线下会长得很不一样。

需求：

```
已支付订单取消后，不再直接 canceled，而是进入 refund_requested，并创建一次退款请求。
```

如果用轻量 SDD，你会创建：

```
.ai/tasks/paid-order-cancel-refund/
├── requirements.md
├── design.md
├── tasks.md
└── verification.md
```

然后让 Agent 逐个 task 做。

如果用 OpenSpec，你会先确认当前 specs：

```
openspec/specs/order/spec.md
```

然后开 change：

```
openspec/changes/paid-order-cancel-refund/
├── proposal.md
├── design.md
├── tasks.md
└── specs/order/spec.md
```

实现完成后 archive，把 delta spec 合并回主 specs。

如果用 Spec Kit，你会围绕命令流程推进：

示例 20-8 三者怎么映射到真实工作

```
/speckit.constitution
/speckit.specify
/speckit.clarify
/speckit.checklist
/speckit.plan
/speckit.tasks
/speckit.analyze
/speckit.implement
```

然后在每个阶段审 artifact。

这三种方式都可以做同一个需求，但它们解决的主问题不同：

```
轻量 SDD：约束一次任务
OpenSpec：维护存量行为变更
Spec Kit：驱动完整 SDD 生命周期
```

如果你只记住这三句话，选型基本不会偏得太远。

20.7 一个更实用的选择表

表 20-1 一个更实用的选择表

场景	推荐	原因
修复明确 bug，影响范围小	轻量 SDD	写清复现、边界和验证即可
中等复杂需求，但不会长期维护规格	轻量 SDD	成本低，能控制小 diff
订单、权限、支付、库存等存量行为变化	OpenSpec	需要当前行为和拟议变更分离
同一领域会反复被 AI 接手	OpenSpec	主 specs 能成为长期上下文入口
新项目、新工具、新功能包	Spec Kit	完整流程更有价值
需要多 AI coding agent 使用同一套流程	Spec Kit	integration 和命令体系更强
需要自定义模板、preset、extension	Spec Kit	工具内建扩展机制
团队暂时没人维护规格资产	轻量 SDD	先不要引入长期 spec 负债
只改文案、样式或小配置	不必 SDD	直接改，但保留验证证据

这个表不是死规则。真实项目里可以组合使用。

例如，一个 repo 可以用 OpenSpec 维护订单、支付这些高风险领域，同时普通 UI 改动仍然用轻量 SDD。另一个新工具项目可以用 Spec Kit 初始化完整流程，再把其中的 project principles 和 task 拆分习惯迁移到自己的轻量 SDD 模板里。

工具之间不是互斥关系，真正要避免的是把它们混成一锅。

20.8 不要把 Spec Kit 和 OpenSpec 写进同一个任务上下文

很多人会想：“既然都好，那我把 Spec Kit、OpenSpec、自己手写的 requirements 都给 AI，看它综合一下。”

这通常会污染上下文。

因为三者对 artifact 的语义不同：

- 轻量 SDD 的 `requirements.md` 是一次任务合同。
- OpenSpec 的 `specs/` 是当前系统行为，`changes/` 是拟议变更。
- Spec Kit 的 spec/plan/tasks 是一个工具链阶段产物。

如果你同时给 Agent 三套结构，它可能分不清哪个是 source of truth。

一次任务只选一条主线。

用轻量 SDD，就明确：

```
本次任务以 `.ai/tasks/<task>/` 为准。
不要读取其他草稿目录。
```

用 OpenSpec，就明确：

```
先读取 `openspec/specs/<domain>/spec.md`。
再读取 `openspec/changes/<change>/` 下的 proposal、delta spec、design、tasks。
当前授权只执行 tasks.md 中指定 task。
```

用 Spec Kit，就明确：

```
按当前 feature 的 Spec Kit artifact 推进。
先运行 analyze 或人工检查 spec/plan/tasks 一致性，再实现。
```

主线清楚，Agent 才不容易把旧草稿、旧设计和当前任务混在一起。

20.9 选型时最容易犯的五个错

错误一：把轻量 SDD 当成低配方案。

轻量不等于粗糙。对个人高频任务，它反而是最实用的起点。只要 requirements、design、tasks、verification 写得扎实，它能解决大多数复杂但一次性的变更。

错误二：OpenSpec 只写 changes，不维护 specs。

这会失去 OpenSpec 最核心的价值。没有当前系统行为，delta spec 就没有参照。

错误三：Spec Kit 跑完命令就直接相信结果。

Spec Kit 生成 artifact 后仍然要审。尤其是 clarify 是否真的消除了歧义，plan 是否符合现有代码，tasks 是否能小步验证。

错误四：把框架能力当成项目能力。

工具支持 integration、extension、preset，不代表你的 repo 已经适合它。项目规则、测试命令、模块边界、CI 反馈这些基础能力如果缺失，框架只能让问题更结构化地暴露出来。

错误五：为了显得专业，把小任务流程复杂化。

如果一个改动 30 分钟能完成，验证也清楚，不需要把它扩成完整 SDD 流程。AI Coding 要服务交付，不是让每个任务都变成仪式。

20.10 给个人程序员的落地顺序

如果你现在没有任何 SDD 习惯，不建议一上来装一堆框架。

先从轻量 SDD 开始。

给自己的 repo 固定一个目录：

```
.ai/tasks/
```

每次复杂任务都写四个文件：

```
requirements.md
design.md
tasks.md
verification.md
```

连续做 5 个真实任务后，再回看：

- 哪些需求反复出现在同一领域。
- 哪些旧行为总需要重复解释。
- 哪些变更完成后应该沉淀成长期合同。

如果答案集中在某几个业务领域，再引入 OpenSpec。

先只覆盖一个领域，例如：

```
openspec/specs/order/spec.md
```

不要一次补全整个系统。一次补 3 到 5 条高风险行为，然后用一个真实 change 走完整流程。

如果你开始做新工具、新产品模块，或者需要让不同 Agent 共享一套 SDD 命令，再评估 Spec Kit。

安装前先做三件事：

清单 20-9 给个人程序员的落地顺序

```
uv --version
git status --short
specify --help
```

如果使用 one-shot：

清单 20-10 给个人程序员的落地顺序

```
uvx --from git+https://github.com/github/spec-kit.git specify init .
```

如果使用持久安装，先看官方安装文档和当前 release。不要把网上旧教程里的命令直接复制到生产 repo。

20.11 真正的选择标准：谁来维护 source of truth

所有 SDD 工具最后都会回到一个问题：

```
这次任务完成后，谁负责维护事实？
```

轻量 SDD 的事实，通常停留在任务目录和 verification 里。它适合短生命周期。

OpenSpec 的事实，应该沉淀到 `openspec/specs/`。它适合长期行为合同。

Spec Kit 的事实，分布在它生成的 spec、plan、tasks、constitution 和扩展模板里。它适合完整流程，但也更需要你理解每个 artifact 的职责。

如果没人维护事实，工具越完整，过期信息越多。

AI Coding 最怕的不是没有文档，而是有一堆过期但看起来很正式的文档。

所以选型时不要只问“哪个框架能力最多”，要问：

- 这个 repo 是否有稳定的当前行为规格。
- 我是否愿意在代码变更后更新规格。
- Agent 下次会不会优先读取这些规格。

· 测试和验证能否证明规格没有漂移。

这些问题答不上来，先用轻量 SDD。

这些问题答得上来，再引入 OpenSpec 或 Spec Kit。

20.12 常见问题

轻量 SDD 能不能和 OpenSpec 共存?

可以，但同一次任务最好只选一条主线。OpenSpec 覆盖高风险长期领域，轻量 SDD 处理普通一次性任务，是比较实际的组合。

Spec Kit 和 OpenSpec 谁更适合程序员?

看任务。存量系统行为变更更偏 OpenSpec；完整 SDD 流程、新项目、多 Agent 集成更偏 Spec Kit。专业不等于选更重的工具，而是能控制成本和风险。

已经用了 Spec Kit，还需要 AGENTS.md 吗?

需要。Spec Kit 能生成和组织流程，但项目长期规则、验证命令、禁止事项、代码风格仍然需要在 Agent 能稳定读取的位置维护。具体位置以你使用的 agent integration 为准。

OpenSpec 能不能替代测试?

不能。OpenSpec 维护行为规格，测试提供可执行证据。没有测试，Agent 仍然可能实现出“看起来符合 spec”的错误代码。

什么时候不要上任何 SDD?

很小、风险低、验证明确的改动可以直接做。例如修 typo、改配置默认值、调整非核心样式。但也要保留最基本的 diff 审查和验证记录。

20.13 可复查资料

· GitHub Spec Kit 官方文档：https://github.github.io/spec-kit/〔20-1〕

· GitHub Spec Kit quick start：https://github.github.io/spec-kit/quickstart.html〔20-2〕

· GitHub Spec Kit integrations：https://github.github.io/spec-kit/reference/integrations.html〔20-3〕

· GitHub Spec Kit CLI reference：https://github.github.io/spec-kit/reference/overview.html〔20-4〕

· OpenSpec getting started：https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md〔20-5〕

- OpenSpec concepts：https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md〔20-6〕

本章资源

〔20-1〕 https://github.github.io/spec-kit/
https://github.github.io/spec-kit/

〔20-2〕 https://github.github.io/spec-kit/quickstart.html
https://github.github.io/spec-kit/quickstart.html

〔20-3〕 https://github.github.io/spec-kit/reference/integrations.html
https://github.github.io/spec-kit/reference/integrations.html

〔20-4〕 https://github.github.io/spec-kit/reference/overview.html
https://github.github.io/spec-kit/reference/overview.html

〔20-5〕 https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md
https://github.com/Fission-AI/OpenSpec/blob/main/docs/getting-started.md

〔20-6〕 https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md
https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md

第21章　模块实战：用 OpenSpec CLI 推进一次真实产品需求

这次从一个真实产品需求开始：个人效率助手每天生成“今日计划”时，要自动纳入逾期未完成任务，但不能把一天排爆。

OpenSpec 在这里的作用，是把需求、设计、任务和验证证据放进同一个变更包里，让 AI 的实现有边界，也让人能复查每一步为什么这么改。

配套代码仓库 DavidDingXu/learn-ai-coding〔21-1〕 按一个大仓库组织，仓库根目录下有业务 module 和工具 module。这里使用业务 module：`personal-productivity-app`，后面的 OpenSpec、Superpowers、验证和规则沉淀都围绕同一条业务主线展开。

对应过程记录在：

```
learn-ai-coding/personal-productivity-app/dev-process/01-openspec-today-plan-rollover.md
```

需求很常见：

```
用户每天早上打开个人效率助手，希望看到今天该做什么。
昨天没做完的任务不能消失，但系统也不能把所有历史任务都塞进今天。
```

这类需求在真实项目里很容易扩散：

```
顺手做 UI。
顺手接数据库。
顺手加提醒系统。
顺手做复杂排程算法。
顺手把 blocked 任务也塞进今日计划。
```

OpenSpec 的价值不是让文档更漂亮，而是把“为什么做、做什么、不做什么、怎么验证”放进仓库，让 AI 在动手前读合同。

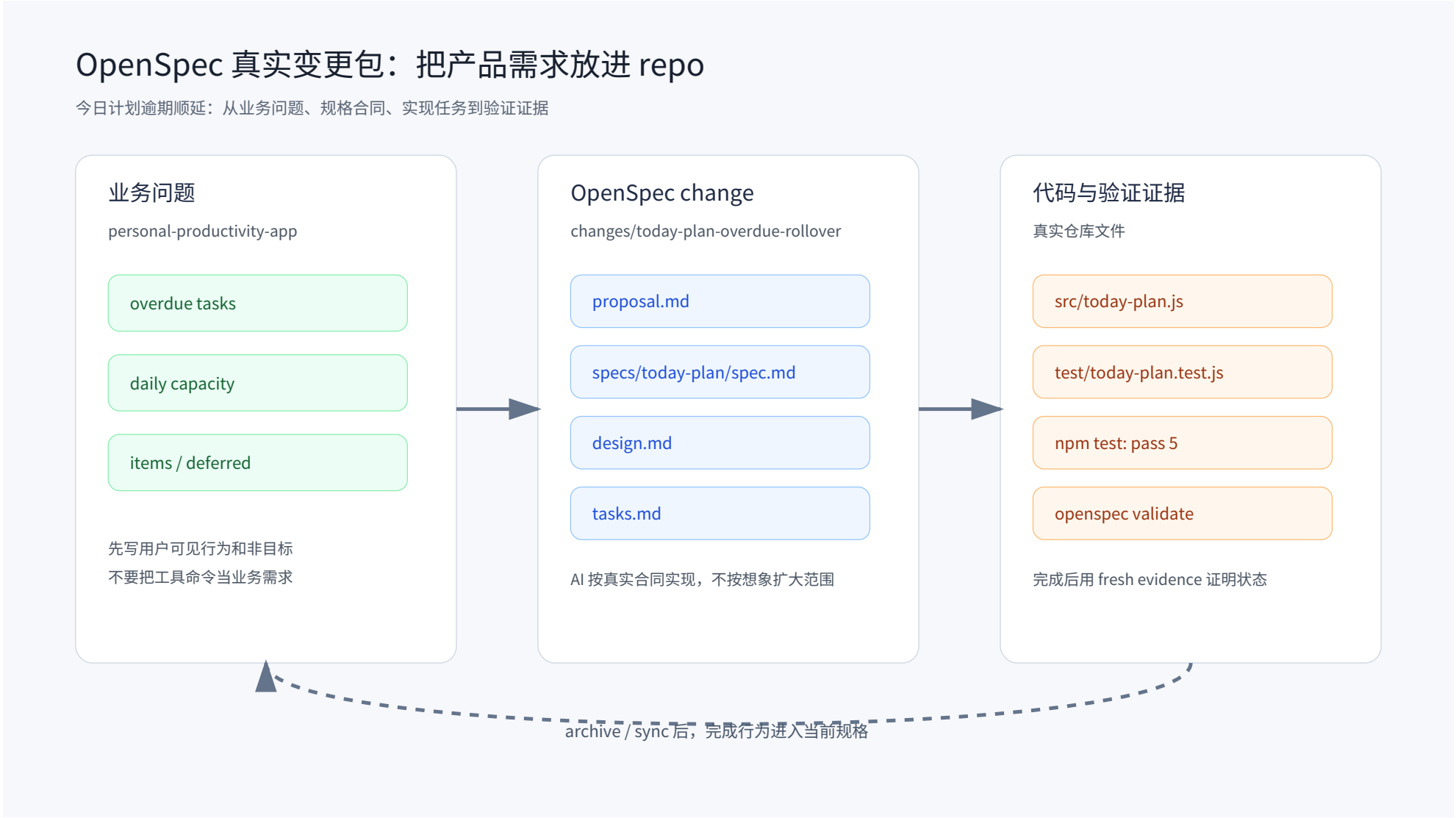


图 21-1　OpenSpec 真实变更包

21.1　先核对工具是真的

不能只说“按 OpenSpec 式材料做一下”。这里必须跑真实 OpenSpec CLI，并把 proposal、design、tasks、spec 和 validate 结果留在仓库里。

本机先核对当前包：

清单 21-1　先核对工具是真的

```
npm view @fission-ai/openspec name version bin engines --json
```

关键输出：

清单 21-2　先核对工具是真的

```
{
  "name": "@fission-ai/openspec",
  "version": "1.4.1",
  "bin": {
    "openspec": "bin/openspec.js"
  },
  "engines": {
    "node": ">=20.19.0"
  }
}
```

本机环境：

清单 21-3　先核对工具是真的

```
node --version
# v24.14.1

npm --version
# 11.11.0
```

安装：

清单 21-4　先核对工具是真的

```
npm install -g @fission-ai/openspec
```

验证：

清单 21-5　先核对工具是真的

```
openspec --version
# 1.4.1
```

这一步很重要。OpenSpec、Spec Kit、Superpowers 这类工具变化很快，不能靠历史印象写命令。

21.2 初始化项目

示例项目不是工具项目，而是一个产品领域逻辑项目：

```
learn-ai-coding/personal-productivity-app
```

初始化 OpenSpec：

清单 21-6 初始化项目

```
cd learn-ai-coding/personal-productivity-app
openspec init --tools codex .
```

关键输出：

示例 21-7 初始化项目

```
OpenSpec Setup Complete

Created: Codex
5 skills and 5 commands in .codex/
Config: skipped (non-interactive mode)

Getting started:
  Start your first change: /opsx:propose "your idea"
```

这说明 OpenSpec 真实生成了：

```
.codex/skills/
openspec/changes/
openspec/specs/
```

注意这里的 `/opsx:propose` 是 OpenSpec 给 Codex 配的 slash command 入口。为了让过程能在普通终端里复查，下面主要展示 CLI 命令。

21.3 创建真实 change

创建需求：

清单 21-8 创建真实 change

```
openspec new change today-plan-overdue-rollover \
  --description "今日计划自动纳入逾期未完成任务，并在每日容量限制下给出顺延列表。" \
  --goal "让个人效率助手每天生成可执行的今日计划，避免逾期任务丢失或一天被排爆。" \
  --json
```

输出里会看到 change 的真实路径：

清单 21-9　创建真实 change

```
{
  "change": {
    "id": "today-plan-overdue-rollover",
    "path": "learn-ai-coding/personal-productivity-app/openspec/changes/today-plan-overdue-
rollover",
    "schema": "spec-driven"
  }
}
```

此时还没有实现代码，只有一个 change 壳子。

下一步不能靠猜，要问 OpenSpec 当前 change 需要什么产物：

清单 21-10　创建真实 change

```
openspec status --change today-plan-overdue-rollover --json
```

OpenSpec 返回的工作流是：

```
proposal -> specs/design -> tasks
```

apply 前置是：

```
tasks
```

这意味着实现前至少要有 `proposal.md`、`specs/**/*.md`、`design.md` 和 `tasks.md`。

21.4　proposal：把产品问题写清楚

先读取 OpenSpec 的 proposal 指令：

清单 21-11　proposal：把产品问题写清楚

```
openspec instructions proposal --change today-plan-overdue-rollover --json
```

它要求 proposal 回答四件事：

```
Why
What Changes
Capabilities
Impact
```

本次写入：

```
openspec/changes/today-plan-overdue-rollover/proposal.md
```

核心内容不是“新增一个函数”，而是产品问题：

```
今日计划如果只看当天任务，会漏掉昨天或更早没完成的任务；
如果把所有逾期任务一次性塞进今天，又会把一天排爆。
```

做什么：

```
生成今日可执行任务。
自动纳入逾期未完成任务。
排除已完成、阻塞和未来任务。
按容量输出 items 和 deferred。
输出 usedMinutes 和 remainingMinutes。
```

能力名：

```
today-plan
```

这会决定后续 spec 文件路径：

```
openspec/changes/today-plan-overdue-rollover/specs/today-plan/spec.md
```

21.5 specs：写成可测试需求

读取 specs 指令：

清单 21-12 specs：写成可测试需求

```
openspec instructions specs --change today-plan-overdue-rollover --json
```

本机 OpenSpec 明确要求：

```
每个 requirement 使用 ### Requirement
每个 scenario 使用 #### Scenario
规范性描述使用 SHALL / MUST
每个 requirement 至少一个 scenario
```

这不是格式洁癖。

如果 scenario 写不出来，说明需求还不能被测试约束。

本次 spec 拆成三个 requirement：

```
Generate today's executable plan
Respect daily capacity
Prioritize overdue work predictably
```

其中一个场景是：

```
WHEN 输入包含逾期 open 任务、今天到期 open 任务和未来 open 任务
THEN 今日计划只包含逾期和今天到期的 open 任务
AND 逾期任务排在今天到期任务之前
```

这直接映射到测试：

```
carries overdue unfinished tasks into today plan before today tasks
```

到这里，OpenSpec 已经把“用户想要什么”变成了可验证合同。

21.6　design：限定实现边界

读取 design 指令：

清单 21-13　design：限定实现边界

```
openspec instructions design --change today-plan-overdue-rollover --json
```

本次 design 重点写四个决策：

```
使用纯领域函数。
只让 open 任务进入候选。
逾期任务优先于当天任务，优先级优先于日期年龄。
容量选择采用顺序扫描，放不下进入 deferred，继续尝试后续小任务。
```

也明确不做：

```
页面展示不放进这个 OpenSpec 变更。
不做数据库。
不做账号系统。
不接日历。
不做复杂排程优化。
```

真实项目里，设计文件不一定要长，但必须挡住需求扩散。

对 AI 来说，这比一句“不要做太多”有效得多。

这里要区分领域规则和页面展示：OpenSpec 变更先把领域规则做实，页面只能复用 `buildTodayPlan` 的输出。如果页面要引入独立状态、接口或排程逻辑，那就是另一个需求。

21.7 tasks：让实现可以逐项审查

读取 tasks 指令：

清单 21-14 tasks：让实现可以逐项审查

```
openspec instructions tasks --change today-plan-overdue-rollover --json
```

本机 OpenSpec 要求 checkbox 格式：

```
- [ ] 1.1 ...
```

因为 apply 阶段会解析 checkbox。

本次 tasks 拆成 8 项：

示例 21-15 tasks：让实现可以逐项审查

```
1.1 跑 npm test 并记录缺失实现的失败。
2.1 新增 src/today-plan.js。
2.2 过滤 open overdue / due-today。
2.3 排序。
2.4 容量选择和 deferred 输出。
2.5 输入校验。
3.1 跑 npm test。
3.2 写过程记录。
```

用 OpenSpec 检查 apply 状态：

清单 21-16 tasks：让实现可以逐项审查

```
openspec instructions apply --change today-plan-overdue-rollover --json
```

初始输出：

清单 21-17 tasks：让实现可以逐项审查

```
{
  "progress": {
    "total": 8,
    "complete": 0,
    "remaining": 8
  },
  "state": "ready"
}
```

这比“请按计划实现”更可靠。

AI 和人都能看到还有几项没完成。

21.8 严格校验 spec

实现前先验证 OpenSpec change：

清单 21-18 严格校验 spec

```
openspec validate today-plan-overdue-rollover --strict --json --no-interactive
```

结果：

清单 21-19 严格校验 spec

```
{
  "summary": {
    "totals": {
      "items": 1,
      "passed": 1,
      "failed": 0
    }
  }
}
```

这一步不能省。

否则你可能写了一堆看似 OpenSpec 的文件，但工具根本识别不了。

21.9 TDD 红灯：先证明测试能抓住缺失行为

测试已经写在：

```
test/today-plan.test.js
```

第一次运行：

清单 21-20 TDD 红灯：先证明测试能抓住缺失行为

```
npm test
```

失败：

```
Error [ERR_MODULE_NOT_FOUND]: Cannot find module
'/learn-ai-coding/personal-productivity-app/src/today-plan.js'
```

这个失败是有效红灯：测试已经指向 `buildTodayPlan`，但生产实现还不存在。

如果你没见过红灯，后面绿灯没有太大意义。

21.10　实现：只补领域函数

新增：

```
src/today-plan.js
```

核心 API：

清单 21-21　实现：只补领域函数

```
export function buildTodayPlan({ date, capacityMinutes = 0, tasks = [] } = {}) {
  // ...
}
```

输入任务字段：

示例 21-22　实现：只补领域函数

```
id
title
dueDate
status
priority
estimateMinutes
```

输出：

示例 21-23　实现：只补领域函数

```
date
capacityMinutes
usedMinutes
remainingMinutes
items
deferred
```

这里没有接 UI，也没有写文件。

因为 OpenSpec design 已经限定：当前变更只做领域逻辑。结果页只能调用这个函数展示输出，不能改写需求合同。

21.11　绿灯：业务场景通过

再次运行：

清单 21-24　绿灯：业务场景通过

```
npm test
```

输出：

示例 21-25 绿灯：业务场景通过

```
✔ carries overdue unfinished tasks into today plan before today tasks
✔ does not include completed or blocked tasks
✔ respects daily capacity and returns overflow as deferred tasks
✔ sorts overdue tasks by priority before due date age
✔ rejects invalid plan inputs before generating a plan
i pass 5
i fail 0
```

这组输出只覆盖领域函数测试。完整样例还会通过结果页演示测试确认页面没有另写一套业务规则。

这 5 个测试不是为了凑覆盖率。

它们分别对应：

```
逾期任务是否顺延。
不可执行任务是否排除。
容量是否生效。
排序是否符合产品优先级。
输入边界是否被拒绝。
```

21.12 完成状态：让 OpenSpec 自己确认

实现和记录完成后，tasks 被勾选。

再次运行：

清单 21-26 完成状态：让 OpenSpec 自己确认

```
openspec instructions apply --change today-plan-overdue-rollover --json
```

输出：

清单 21-27 完成状态：让 OpenSpec 自己确认

```
{
  "progress": {
    "total": 8,
    "complete": 8,
    "remaining": 0
  },
  "state": "all_done"
}
```

这时再说“这个 change 完成”，才有证据。

21.13 这类需求为什么适合 OpenSpec

不是所有需求都需要 OpenSpec。

如果只是改一个错别字、补一行日志、换一个按钮文案，用 OpenSpec 会显得重。

但这次需求值得用，因为它有：

```
状态边界：open / done / blocked。
日期边界：overdue / due-today / future。
容量边界：items / deferred。
排序边界：priority / dueDate / id。
交付边界：当前变更只做领域函数，不做 API/存储；页面展示只能复用这个函数，不能引入另一套业务规则。
```

这些边界只靠聊天提醒，很容易在实现中丢掉。

OpenSpec 把边界放进仓库，让后续 AI、reviewer 和你自己都能复查。

21.14 迁移到自己的项目

你可以把这个模式迁移到很多业务需求：

示例 21-28 迁移到自己的项目

```
订单超时自动关闭。
工单 SLA 逾期升级。
会员权益到期提醒。
账单分期到期滚动。
审批任务逾期催办。
内容审核任务优先级队列。
```

只要需求里有状态、时间、优先级、容量、过滤或排序，就适合先写 spec。

给 AI 的指令可以这样写：

```
请先读取 openspec/changes/<change>/proposal.md、design.md、specs/**/*.md、tasks.md。
只实现 tasks.md 中未完成事项。
如果实现中发现需求边界不清，先更新 OpenSpec 产物，不要直接扩大代码范围。
实现前跑现有测试，完成后跑 openspec validate 和项目测试。
```

21.15 配套资料

- OpenSpec 过程记录：`learn-ai-coding/personal-productivity-app/dev-process/01-openspec-today-plan-rollover.md`
- OpenSpec 变更包：`learn-ai-coding/personal-productivity-app/openspec/changes/today-plan-overdue-rollover/`
- 领域实现：`learn-ai-coding/personal-productivity-app/src/today-plan.js`

- 结果页：`learn-ai-coding/personal-productivity-app/index.html`
- 页面演示数据：`learn-ai-coding/personal-productivity-app/src/today-plan-demo.js`
- 业务测试：`learn-ai-coding/personal-productivity-app/test/today-plan.test.js`
- 页面演示测试：`learn-ai-coding/personal-productivity-app/test/today-plan-demo.test.js`
- OpenSpec 上游仓库：https://github.com/Fission-AI/OpenSpec〔21-2〕

本章资源

〔21-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔21-2〕 https://github.com/Fission-AI/OpenSpec
https://github.com/Fission-AI/OpenSpec

PART 06

第6篇　小步实现、测试与调试

进入执行阶段后，用计划、小 diff、失败测试和排障证据限制不确定性。把页面、接口、数据和副作用一起纳入验证。

阅读重点

能根据证据判断一次改动是否完成，以及失败是否来自本次变更。

第22章　计划先行：让 AI 先解释它准备怎么改

很多 AI Coding 事故，不是模型不会写代码，而是它太快开始写代码。

你给它一句需求：

> 把已支付订单取消改成走退款流程，补一下测试。

它马上改 `OrderCancelService`、状态机、退款服务、接口返回、测试、甚至顺手整理一些命名。最后测试可能跑过，但你打开 diff 会发现一个问题：这个改动已经很难审了。

你不知道它为什么改了这些文件，不知道它有没有先看旧行为，不知道它有没有保留 shipped 订单拒绝取消，不知道它有没有把支付回调也带进来。测试一旦失败，它还可能继续自己修，越修越远。

计划先行要解决的就是这个问题。

不是让 AI 写一段漂亮计划，也不是多一个形式步骤。计划要在动代码之前暴露五件事：上下文选择、改动范围、实现顺序、验证路径和停止条件。

计划能被人审，diff 才能被人控。

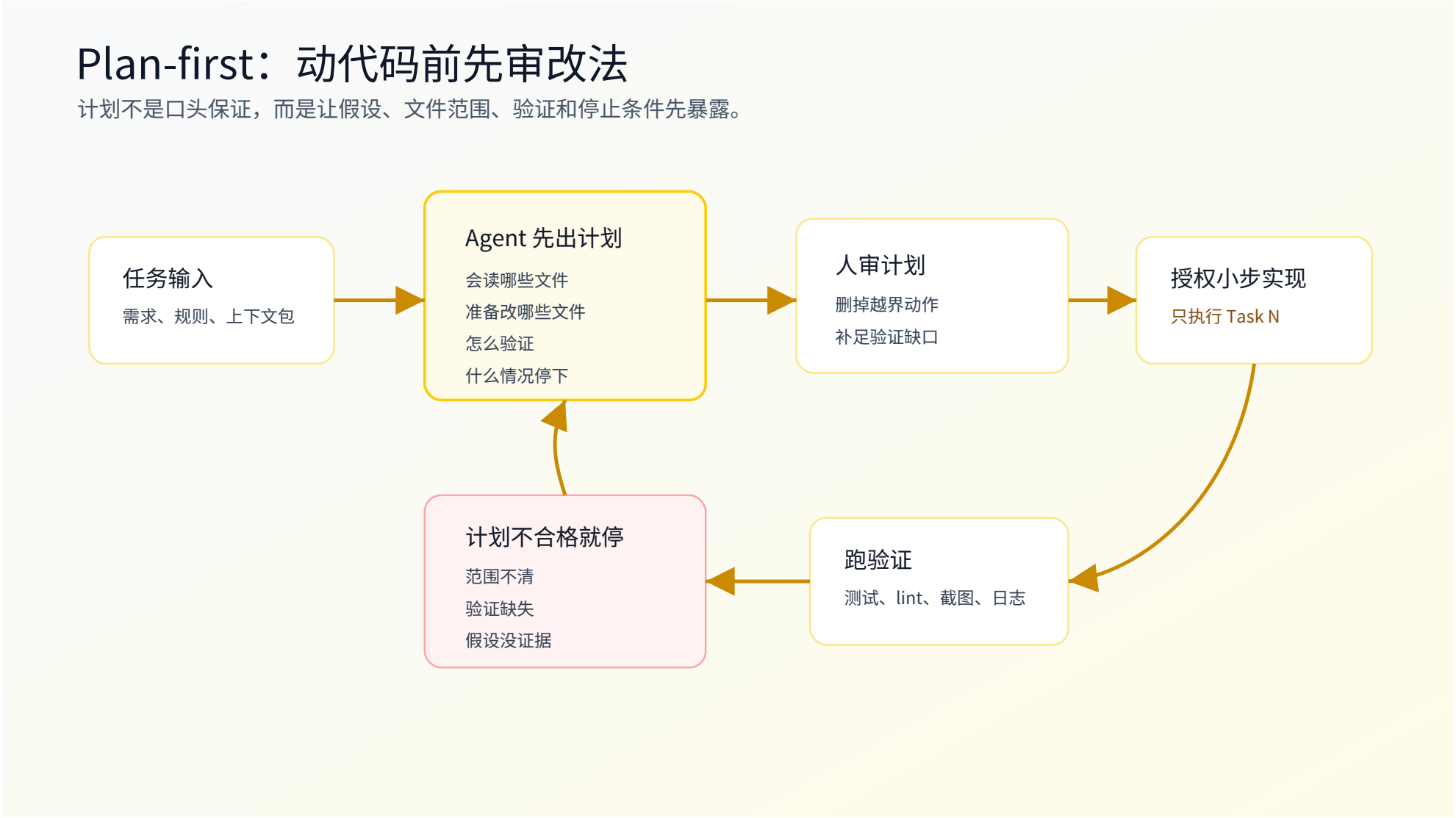


图 22-1　Plan-first 审查循环

22.1 为什么直接实现会产生大 diff

AI 接到任务后，会自然补全缺失信息。

人类程序员听到“已支付订单取消走退款流程”，会先问：

- 当前订单状态有哪些。
- paid 取消现在怎么处理。
- 退款请求是否已有服务。
- 状态更新和退款创建是否需要事务。
- duplicate cancel 怎么办。
- API 返回是否兼容。
- 哪些测试能证明行为没坏。

Agent 如果没有被要求先计划，它可能也会搜索代码，但搜索结果和推理过程不会稳定地变成你能审查的东西。它会在写代码过程中边看边改。

这会带来三个问题。

第一，文件范围提前失控。

模型看到一个调用点，就可能顺手改调用点；看到一个旧测试，就可能顺手重写测试；看到一段风格不统一的代码，就可能顺手整理。每个动作单看都合理，合起来就变成大 diff。

第二，需求和实现混在一起。

如果它先写实现，再补测试，测试往往会贴着实现走。你看到的是“测试通过”，但不一定是需求被锁住。

第三，失败后没有停机点。

测试失败时，模型会继续猜原因。它可能把失败归因于测试写错，也可能改动更底层的逻辑。没有计划里的 Stop 条件，它不知道什么时候该停下来把问题交回给人。

计划先行不是降低 AI 的能力，而是把它的能力放进可审查路径。

22.2 好计划应该长什么样

一个合格计划不需要长，但必须具体。

危险计划通常长这样：

```
我会先分析代码结构，然后实现退款流程，补充测试并运行验证。
```

这句话没有任何工程价值。

它没有说读哪些文件，没有说改哪些文件，没有说不改哪些地方，没有说先加什么测试，没有说验证命令，也没有说遇到什么情况停下。

可执行计划应该长这样：

清单 22-1　好计划应该长什么样

```
Plan

Context to inspect:
- `src/order/order-cancel-service.ts`
- `src/order/order-status-transition.ts`
- `src/refund/refund-request-service.ts`
- `test/order/order-cancel.test.ts`

Intended change:
1. Add focused tests for paid cancellation, duplicate paid cancellation and refund creation
failure.
2. Add paid -> refund_requested transition in `order-status-transition.ts`.
3. Wire refund request creation in `order-cancel-service.ts`.
4. Keep pending and shipped cancellation behavior unchanged.

Out of scope:
- public API response shape
- payment provider callback flow
- database schema migration

Verification:
- `npm test -- order-cancel`
- `npm test -- order`

Stop if:
- existing pending or shipped tests fail before implementation.
- refund request requires callback changes.
- public API response shape must change.
```

这个计划不是“口头承诺”，它已经能被审查。

你可以马上指出：

- 少看了 repository 层。
- duplicate cancel 需要先查是否已有幂等键。
- `npm test -- order` 太慢，先跑 focused test。
- API response shape 如果必须改，需要先回到需求。

plan-first 的价值在这里：错误会先暴露在计划里，而不是混进 diff 里。

22.3　计划不是让 AI 自由发挥

计划阶段也要给边界。

不要只说：

```
先给我一个计划。
```

这种输入太宽，AI 可能写出一份项目管理式计划。

把计划要求写具体：

示例 22-2　计划不是让 AI 自由发挥

```
先不要改代码。

请基于当前 repo，给出一个可执行修改计划。

要求：
- 先列出你准备读取的文件和原因。
- 再列出你预计修改的文件。
- 明确不修改的范围。
- 按小 diff 顺序拆步骤。
- 每一步写验证命令。
- 写 Stop if 条件。
- 如果信息不足，先列出需要核对的问题，不要假设。
```

这段话的关键是“先不要改代码”和“如果信息不足，不要假设”。

很多 Agent 的默认行为是边读边改。你要明确把它拉回只读状态，让它先建立计划。

如果工具支持只读模式、计划模式或审批机制，可以配合使用。不同工具的名称不一样，但工程意图相同：计划阶段只读，执行阶段再授权写入。

22.4　计划质量看 6 个维度

计划质量看 6 个维度

只要有一项说不清，就不要让 Agent 开始大范围改代码。

维度	危险计划	可执行计划
上下文	“我会阅读相关代码”	列出入口、调用点、测试文件
改动范围	“实现完整功能”	明确改哪些文件，不改哪些文件
实现顺序	测试、实现、重构混在一起	先证据，再最小实现，再扩大验证
验证路径	“运行测试”	命令、覆盖范围、预期失败/通过
风险假设	默认依赖和接口都存在	列出需先核对的事实和接口
停止条件	失败后继续尝试修	baseline 异常、接口变更、范围扩大即停

图 22-2　计划质量矩阵

审 AI 的计划，不要只看它是否“看起来完整”。

重点看六个维度。

上下文是否具体。

坏计划写“阅读相关代码”。好计划写具体文件、入口、调用点、测试文件。

如果它列不出文件，说明它还没真正理解任务。可以让它先做代码搜索：

清单 22-3 计划质量看 6 个维度

```
rg -n "cancel|refund_requested|RefundRequest|order status" src test
```

然后基于搜索结果重写计划。

改动范围是否收敛。

坏计划写“实现完整退款流程”。好计划写“只改状态转换、取消编排和 focused test，不改支付回调、不改 API response shape”。

AI Coding 最怕范围不清。范围不清时，模型会把所有看起来相关的东西都纳入改动。

实现顺序是否能小步验证。

坏计划把测试、实现、重构混在一起。好计划先写 focused test，再改最小实现，再扩大验证。

对复杂需求，顺序比速度重要。你要让每一步都能解释 diff。

验证路径是否有命令。

坏计划写“运行测试”。好计划写具体命令、覆盖范围和预期结果。

例如：

示例 22-4 计划质量看 6 个维度

```
Task 1:
- `npm test -- order-cancel`
- before implementation: paid cancellation test fails, existing pending/shipped tests pass

Task 2:
- `npm test -- order-cancel`
- after implementation: focused tests pass

Final:
- `npm test -- order`
- covers broader module impact
```

没有命令的验证，是不可审查的验证。

风险假设是否被列出。

AI 经常默认某个 service、字段、状态、接口存在。计划里应该写哪些事实需要先核对。

例如：

```
Need to verify:
- whether refund request creation already has idempotency support.
- whether order status is persisted before or after side effects.
- whether cancel API response shape is covered by snapshot or contract tests.
```

这些不是废话。它们决定实现是不是会越界。

停止条件是否明确。

Stop 条件不是给 AI 泼冷水，而是防止错误扩散。

例如：

示例 22-5　计划质量看 6 个维度

```
Stop if:
- baseline tests fail before implementation.
- existing code has no refund request service.
- implementation requires database schema changes.
- public API response shape must change.
- payment callback needs to be modified.
```

一旦触发 Stop，就不要让它继续“顺手解决”。停下来重新审需求、设计或任务范围。

22.5　计划要和任务粒度绑定

计划不是一个大纲。它应该直接决定执行粒度。

错误方式：

```
计划看起来可以，开始实现。
```

授权时只放开一步：

示例 22-6　计划要和任务粒度绑定

```
计划通过。现在只执行 Step 1：新增 focused tests。

要求：
- 不改实现代码。
- 只修改 `test/order/order-cancel.test.ts`。
```

```
- 运行 `npm test -- order-cancel`。
- 预期 paid cancellation 用例失败，pending/shipped 用例通过。
- 完成后停止，输出 diff 摘要和命令结果。
```

Step 1 完成后，你看 diff 和测试结果，再授权 Step 2。

这能避免一个常见问题：AI 计划写得很好，执行时又一次性改完整个任务。

计划必须和授权机制绑定。没有分步授权，计划只是开场白。

22.6 让 AI 在计划里写“不会做什么”

很多计划的问题，不在于它少写“会做什么”，而在于没写“不会做什么”。

对真实项目，out of scope 非常重要。

例如订单取消需求里，要明确：

示例 22-7 让 AI 在计划里写“不会做什么”

```
Out of scope:
- payment provider callback flow
- public API response shape
- database migration
- UI display copy
- refund settlement job
```

为什么要写这些?

因为 AI 会把语义相近的工作自动串起来。你说“退款流程”，它可能想到支付回调；你说“订单取消”，它可能想到前端订单详情页；你说“状态改成 refund_requested”，它可能想到数据库枚举迁移。

这些可能最终确实要做，但不一定在这个 diff 里做。

一个程序员要控制的不是 AI 是否“想得全面”，而是每个 diff 是否有边界。

22.7 计划里要写 baseline

很多测试失败不是这次改动导致的。

如果没有 baseline，AI 看到失败会直接修。修着修着，就把旧问题也混进当前 diff。

计划里可以要求：

```
Before implementation:
- run `npm test -- order-cancel` if existing focused tests exist.
- record existing failures.
- if unrelated baseline failure appears, stop before implementation.
```

如果 focused test 还不存在，也要写清预期：

```
After adding tests but before implementation:
- pending and shipped cancellation tests should pass.
- paid cancellation and duplicate cancellation tests should fail.
```

这个信息非常关键。它告诉 AI：某些失败是预期失败，不要在 Task 1 里偷偷改实现让它通过。

22.8 计划可以暴露上下文不足

计划阶段最有价值的输出之一，是发现信息不够。

一个合格 Agent 应该敢写：

```
I cannot produce an implementation plan yet because I need to inspect:
- current order status enum
- cancel service orchestration
- refund request creation API
- existing order cancellation tests
```

这比直接给出一个自信计划更好。

你可以让它先做只读探索：

示例 22-8 计划可以暴露上下文不足

```
先只做代码探索，不改文件。

请使用 `rg` 找到：
- 订单取消入口
- 订单状态定义
- 退款请求创建逻辑
- 现有取消测试

输出文件路径、关键函数、你对当前行为的判断，以及仍然不确定的地方。
```

探索完成后再生成计划。

真实项目里，计划通常不是一次生成就能用。它应该经历：

```
只读探索 -> 初版计划 -> 人审 -> 修订计划 -> 分步执行
```

如果跳过只读探索，计划很容易变成模型凭项目常识写出来的模板。

22.9 计划不应该替代设计

计划和设计容易混。

`design.md` 负责解释技术方案、架构边界、数据流、兼容性和风险取舍。

Plan 负责解释这次执行准备怎么走。

两者可以这样分工：

示例 22-9 计划不应该替代设计

```
design.md:
- paid -> refund_requested 的状态规则
- 退款请求创建和状态更新的事务边界
- 幂等策略
- 不改 API response shape 的兼容性要求

Plan:
- 先读哪些文件
- 先补哪些测试
- 第一步改哪个文件
- 每步跑什么命令
- 遇到什么情况停止
```

设计回答“方案是否合理”。

计划回答“执行是否可控”。

一个常见坏味道是：AI 在 plan 里重新设计一套更复杂的架构。看到这种计划，要让它回到已有 design，或者先产出 design 供人审。

22.10 计划通过后，要变成授权记录

很多人做了 plan-first，但执行时仍然失控，原因是计划没有变成授权记录。

AI 给出计划后，你不应该只回复：

```
可以，开始吧。
```

这会让它默认整个计划都被授权。

更好的回复是：

示例 22-10 计划通过后，要变成授权记录

```
计划通过。现在只授权 Step 1。

Step 1 scope:
```

```
- 只读或只改测试，按计划执行。
- 不改实现文件。
- 不执行 Step 2。

完成后输出：
- changed files
- verification command and result
- whether any Stop condition was hit
```

授权记录的价值在于：执行出问题时，你能判断 Agent 是计划错了、执行越界，还是任务本身出现新事实。

如果 Step 1 执行中发现必须改实现，Agent 应该停下来报告，而不是自己扩大范围。计划先行的真正控制点就在这里。

22.11　计划要写出证据来源

一个计划只列“准备改哪些文件”还不够，它还要说明这些文件是怎么来的。

可以要求 Agent 在计划里加一段：

清单 22-11　计划要写出证据来源

```
Evidence used:
- `rg -n "cancelOrder|RefundRequest|refund_requested" src test`
- `src/order/order-cancel-service.ts`: current cancel orchestration
- `test/order/order-cancel.test.ts`: existing pending/shipped behavior tests
- `src/refund/refund-request-service.ts`: refund request creation API
```

这能防止它凭常识写计划。

如果 evidence 里只有笼统描述，没有路径、命令和搜索结果，就说明计划还没站稳。让它先回到只读探索。

生产项目里尤其要警惕“看起来合理”的计划。AI 可能会写：

```
我会修改订单服务、退款服务和前端状态展示。
```

但如果当前需求没有前端展示范围，或者前端在另一个 repo，这个计划就是扩大范围。证据来源能帮你发现它是不是读了真实代码，还是在按通用电商系统想象。

22.12　计划变更也要重新审

执行过程中发现新事实很正常。比如计划里以为已有 `RefundRequestService`，读代码后发现只有支付网关回调，没有独立退款请求表。

这时不要让 Agent 直接改实现。应该要求它更新计划：

示例 22-12 计划变更也要重新审

```
当前计划假设不成立。
请先更新计划，不要改代码。

更新内容：
- 哪个假设不成立。
- 新证据是什么。
- 是否影响 requirements 或 design。
- 原 Step 2 是否还能执行。
- 需要新增哪些 Stop 条件。
```

计划更新后再授权下一步。

这不是拖流程。它是在防止任务从“订单取消状态改造”滑向“退款系统建模”。真实项目里的大事故，很多都是从一个未审的计划变更开始的。

22.13 plan-first 的适用边界

计划先行不是所有任务都要用。

可以跳过 plan-first 的任务：

- typo、文案、注释小修。
- 单个测试期望值修正，且原因明确。
- 明确的一行配置改动。
- 只读查询、生成报告、整理已有输出。

必须使用 plan-first 的任务：

- 行为变化。
- 跨模块改动。
- 涉及状态机、权限、支付、持久化、任务调度。
- 需要新增或修改测试。
- 可能影响 public API、数据库 schema、事件、队列、缓存。
- 用户让 Agent “顺便”处理多个问题。

这条边界很重要。所有事情都要求计划，会让团队讨厌计划；高风险事情不计划，会让 AI diff 失控。

22.14 把坏计划改成可执行计划

看一个常见坏计划：

清单 22-13 把坏计划改成可执行计划

```
Plan:
1. Analyze the code.
2. Implement refund flow.
3. Add tests.
4. Run tests.
```

这份计划没有办法审。它没有证据、没有文件、没有边界、没有验证半径。

可以改成：

清单 22-14 把坏计划改成可执行计划

```
Plan:

Read first:
- `src/order/order-cancel-service.ts`: cancel orchestration
- `src/order/order-status-transition.ts`: allowed transitions
- `src/refund/refund-request-service.ts`: refund request creation
- `test/order/order-cancel.test.ts`: existing behavior tests

Expected changes:
1. Add focused tests for paid, duplicate paid and refund failure paths.
2. Add paid -> refund_requested transition.
3. Wire refund request creation after transition.
4. Run focused and module-level verification.

Out of scope:
- payment provider callback
- public API response shape
- database migration
- UI display

Verification:
- after tests are added: `npm test -- order-cancel`, new paid tests fail and existing tests pass
- after implementation: `npm test -- order-cancel`, focused tests pass
- before handoff: `npm test -- order`

Stop if:
- existing pending or shipped behavior fails before implementation
- implementation needs schema migration
- refund service does not exist
- API response shape must change
```

这份计划仍然不长，但已经能审。

你可以指出：UI display 不在范围内是否符合需求？refund failure 是否需要先确认错误 envelope？`npm test -- order` 是否太慢，需要换成更聚焦命令？这些问题都能在写代码前讨论。

计划的好坏，不看它像不像“计划”，看它能不能阻止错误 diff。

22.15　计划要先看工作区状态

计划阶段还要看一个经常被忽略的东西：当前工作区是不是干净。

在共享工作区里，可能已经有用户改动、上一轮 Agent 改动、生成文件、未提交测试结果。计划如果不考虑这些，就可能把别人的改动当成自己可以重写的上下文。

计划前至少看：

清单 22-15　计划要先看工作区状态

```
git status --short
```

如果有未提交改动，计划里要说明：

清单 22-16　计划要先看工作区状态

```
Workspace state:
- `src/order/order-cancel-service.ts` already modified before this task.
- I will read the current file and avoid reverting unrelated changes.
- Any new diff must be attributable to the current authorized step.
```

这条规则对 AI Coding 很重要。Agent 很容易用“恢复文件”“重写实现”来解决冲突，但那可能会覆盖用户已有工作。

如果工作区很乱，计划应该先建议隔离：

```
Current workspace has unrelated changes.
Recommended:
- continue only on files required for this task, or
- create a branch/worktree before implementation, or
- ask which existing changes belong to this task.
```

不要在未知工作区状态下让 AI 大范围改代码。

22.16　计划要写 CI 之后还要看什么

计划里的验证命令不只包括本地命令。生产交付还要考虑 CI。

可以写：

清单 22-17　计划要写 CI 之后还要看什么

```
Verification plan:

Local:
- `npm test -- order-cancel`
- `npm test -- order`
```

```
- `npm run typecheck`

CI:
- wait for required checks.
- inspect failed job logs before changing code.
- do not mark broad verification complete if CI fails in affected module.

Human review:
- check response shape compatibility.
- check no payment callback changes.
- check residual risk for old clients.
```

CI 失败时，不要让 Agent 直接“修 CI”。先读日志、分类失败，再决定是否属于当前 diff。计划阶段写清这一点，后面才不会被一句“CI 失败无关”带偏。

22.17 一个可直接复用的 plan-first 模板

可以把这段放进你的项目规则或个人片段里：

示例 22-18 一个可直接复用的 plan-first 模板

```
Before editing files, produce a plan and wait for approval.

The plan must include:
1. Files to inspect and why.
2. Files expected to change.
3. Files and behaviors explicitly out of scope.
4. Step-by-step implementation order, each step producing a reviewable diff.
5. Verification command for each step.
6. Expected failing tests before implementation, if tests are added first.
7. Stop conditions.
8. Open questions and assumptions.

Do not start implementation until the plan is approved.
If the required context is missing, perform read-only exploration first and update the plan.
```

如果你不想每次都手敲，可以把它写进 `AGENTS.md`：

清单 22-19 一个可直接复用的 plan-first 模板

```
## Plan-first Changes

For non-trivial changes:
- inspect the code before planning.
- produce a plan before editing files.
- include expected changed files, verification commands, and stop conditions.
- do not implement until the current task step is authorized.
- stop if the change requires broader scope than requested.
```

注意，规则不要写得过大。所有小改动都强制计划，会拖慢工作。可以写“non-trivial changes”，再定义触发条件：

清单 22-20 一个可直接复用的 plan-first 模板

```
Plan-first is required when:
- behavior changes.
- more than one module may be touched.
- tests need to be added or changed.
- public API, persistence, auth, payment, job, or state machine code is involved.
```

这样规则才不会变成负担。

22.18 常见错误

错误一：计划太抽象。

“分析代码、实现功能、运行测试”不是计划。它没有任何可审查信息。

错误二：计划和实现一起做。

AI 一边写计划一边改代码，等于没计划。计划阶段必须只读。

错误三：计划里没有 out of scope。

没有不改什么，就很难阻止相关代码被顺手修改。

错误四：计划没有验证命令。

没有命令，就无法判断每一步是不是真的完成。

错误五：计划通过后一次性执行完。

计划必须绑定分步授权。复杂任务一次性执行，还是会变成大 diff。

错误六：人只看计划是否顺眼。

审计划要像审设计一样具体。看文件、顺序、验证、风险、停机点，不看文字是否完整。

22.19 改完后重点看几个结果

一次 plan-first 执行结束后，不要只看测试通过。

重点看：

- 实际改动文件是否和计划一致。
- 是否有计划外文件被修改。
- 每一步验证命令是否真实运行。
- Stop 条件是否被触发过，触发后是否停住。
- 计划里的假设是否在代码中得到验证。

· 最终 diff 是否能按计划步骤解释。

如果实际 diff 和计划对不上，要追问原因。原因合理，就更新计划；原因不合理，就回滚越界改动。

计划先行不是为了让流程看起来更严谨，而是为了让 AI 写代码这件事可控。

22.20 常见问题

小改动也要 plan-first 吗?

不需要。改 typo、改一行配置、补一个明确测试，可以直接做。plan-first 用在行为变化、跨模块改动、状态机、权限、支付、持久化、任务调度这类风险更高的任务上。

计划会不会浪费 token?

会消耗一些上下文，但通常比大 diff 返工便宜。关键是计划要短而具体，不要让 AI 写项目管理文档。

AI 计划写错了怎么办?

这正是 plan-first 的价值。计划写错，比代码写错便宜。让它补充只读探索，然后重写计划。

计划通过后 AI 仍然越界怎么办?

把“只执行当前 step，完成后停止”写进执行指令。必要时在工具层使用审批、只读、分支或 worktree 隔离。完成后用 `git diff --stat` 和文件列表核对。

计划和 `tasks.md` 是不是重复?

不完全重复。`tasks.md` 是需求拆分后的任务合同，plan 是当前执行会怎么落某个任务。简单任务可以合并；复杂任务建议分开。

22.21 可复查资料

· OpenAI Codex best practices：https://developers.openai.com/codex/learn/best-practices〔22-1〕

· OpenAI AGENTS.md guide：https://developers.openai.com/codex/guides/agents-md〔22-2〕

· Claude Code common workflows：https://code.claude.com/docs/en/common-workflows〔22-3〕

· GitHub Copilot coding agent best practices：https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results〔22-4〕

本章资源

〔22-1〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

〔22-2〕 https://developers.openai.com/codex/guides/agents-md
https://developers.openai.com/codex/guides/agents-md

〔22-3〕 https://code.claude.com/docs/en/common-workflows
https://code.claude.com/docs/en/common-workflows

〔22-4〕 https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results
https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results

第23章　小步开发：为什么要限制 AI 一次改动范围

让 AI 写代码，最容易低估的一件事是 diff 管理。

很多人会把注意力放在“它有没有实现需求”。但在真实工程里，另一个问题同样重要：这个 diff 能不能被审。

一个 800 行 diff，里面同时有行为修改、测试补充、格式化、重命名、顺手重构和文档调整，就算测试通过，也很难判断它到底安全不安全。你不知道哪一行是需求必须，哪一行是模型自作主张。更麻烦的是，一旦出问题，你也不知道该回滚哪一部分。

所以 AI Coding 不能只讲“让模型完成任务”，还要讲“让模型一次只产生一个可审查变化”。

小步开发的核心不是少写代码，而是控制改动半径。

一次改动最好能满足三个条件：

```
一句话能解释
一个命令能验证
失败后能单独回滚
```

做不到这三点，就应该继续拆。

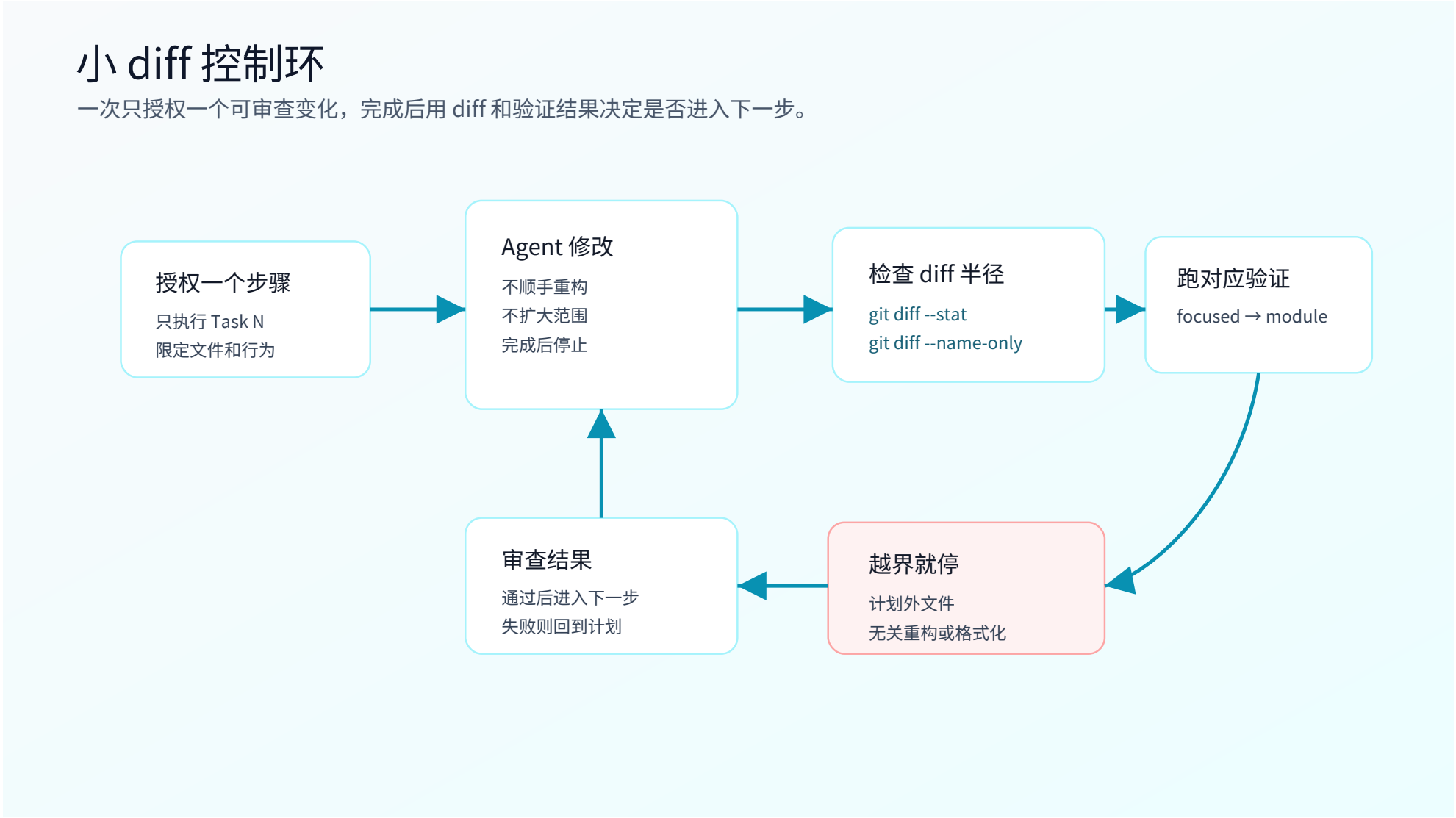


图 23-1　小 diff 控制环

23.1 AI 为什么特别容易产生大 diff

人类程序员也会写大 diff，但 AI 写大 diff 的方式更隐蔽。

它有几个天然倾向。

它会补全任务。

你说“订单取消走退款流程”，它可能联想到状态机、退款服务、支付回调、前端展示、接口返回、日志、文档。它的联想不一定错，但一次 diff 不该吞下所有东西。

它会顺手整理。

模型看到命名不一致，会想统一；看到旧测试写得丑，会想重写；看到重复逻辑，会想抽函数。很多顺手改动单看有道理，但会污染当前需求。

它会为了让测试通过扩大范围。

测试失败时，它可能不先判断失败是否属于当前任务，而是继续改更多代码。没有范围限制时，修复路径会越走越远。

它不天然感知 review 成本。

对模型来说，改 3 个文件和改 15 个文件只是输出长度不同。对人来说，审查成本完全不同。

所以要主动限制 AI 的一次改动范围。

23.2 小 diff 不是按行数判断

很多人会用行数判断 diff 大小，这不够。

一个 20 行公共接口变更，风险可能比 200 行测试快照更新更高。

更实用的判断方式是看改动半径：


改动半径越大，验证半径也要扩大
小 diff 不是按行数判断，而是按行为、模块、接口和回滚边界判断。
Level 1
单文件行为
一个规则、一个函数、一个测试点
git diff -- file
验证：focused test
适合：首选执行粒度
Level 2
同模块联动
service、状态机、测试一起变
git diff --stat
验证：module test
要求：任务边界清楚
风险：容易夹带重构
Level 3
跨模块行为
接口、持久化、异步任务联动
git diff --name-only
验证：module + integration
要求：设计和回滚边界
建议：拆成多个 diff
Level 4
系统级变更
协议、schema、公共 API
git diff --stat
验证：完整回归 + 人审
要求：明确迁移和兼容
AI 不应一次完成
判断标准：一个 diff 能否一句话解释、能否单独验证、能否单独回滚。

图 23-2　改动半径分级

Level 1 是单文件行为变化。比如给状态转换函数增加一个 paid -> refund_requested 分支，并补一个 focused test。

Level 2 是同模块联动。比如状态机、取消服务和订单取消测试一起改。

Level 3 是跨模块行为变化。比如订单取消、退款服务、事务处理和异步任务一起改。

Level 4 是系统级变化。比如公共 API response、数据库 schema、协议、权限模型、结算规则。

AI 一次最适合做 Level 1 或边界清楚的 Level 2。

Level 3 要拆成多个小 diff。

Level 4 不应该让 AI 一次完成。它可以协助写设计、补测试、做迁移草稿，但需要更严格的人审和阶段验证。

23.3 开始前先限制文件范围

小步开发不是等 diff 出来后再后悔，而是在执行前就限制范围。

错误指令：

```
按计划实现订单取消退款流程。
```

直接限制任务范围：

示例 23-1 开始前先限制文件范围

```
只执行 Task 2：添加 paid -> refund_requested 状态转换。

允许修改：
- `src/order/order-status-transition.ts`
- `test/order/order-cancel.test.ts`

禁止修改：
- `src/order/order-cancel-service.ts`
- `src/refund/**`
- API response 类型
- 数据库 schema
- 支付回调逻辑

验证：
- `npm test -- order-cancel`

完成后停止，输出：
- 改动文件列表
- diff 摘要
- 验证命令和结果
```

这段指令有四个控制点：

- 只做一个 Task。
- 明确允许文件。
- 明确禁止范围。
- 完成后停止。

很多 AI 越界不是因为它不听话，而是你没有把边界写到可执行粒度。

23.4　每一步都用 Git 看半径

不要只读 AI 的总结。

每一步完成后，先看：

清单 23-2　每一步都用 Git 看半径

```
git diff --name-only
```

再看：

清单 23-3　每一步都用 Git 看半径

```
git diff --stat
```

如果文件数量超过计划，先不要继续。

例如计划只允许两个文件：

```
src/order/order-status-transition.ts
test/order/order-cancel.test.ts
```

实际出现：

```
src/order/order-status-transition.ts
src/order/order-cancel-service.ts
src/refund/refund-request-service.ts
test/order/order-cancel.test.ts
```

这已经越界。不要让 AI 解释几句就继续。先要求它说明：

```
你修改了计划外文件。

请先不要继续实现。
说明每个计划外文件为什么被修改。
如果不是当前 Task 必需，请恢复这些文件的改动，只保留 Task 2 范围内的 diff。
```

这里要注意：恢复计划外文件时，不要粗暴 `git checkout -- .`。当前工作区可能有你自己的改动，也可能有前一步已经确认过的改动。让 Agent 根据 diff 删除自己刚产生的计划外改动，或者你手动按文件审。

小步开发的关键是“发现越界就停”，不是“越界后继续补救”。

23.5　不要让重构混进需求 diff

AI 很喜欢顺手重构。

它可能会说：

```
我顺手整理了重复逻辑，提升可读性。
```

这句话在 review 里要提高警惕。

重构不是不能做，但不要和行为变更混在同一个 diff。

如果确实需要重构，拆成两个 diff：

示例 23-4　不要让重构混进需求 diff

```
Diff 1: 纯重构
- 不改变行为。
- 只移动/抽取代码。
- 所有现有测试必须保持通过。

Diff 2: 行为变更
- 基于重构后的结构修改需求。
- 新增或修改测试。
```

如果重构不是需求必需，直接拒绝：

```
当前 Task 不做重构。
请恢复命名整理、抽函数和格式化改动。
只保留 paid -> refund_requested 状态转换和对应测试。
```

一个好习惯是要求 AI 在计划里声明：

```
No opportunistic refactoring.
No unrelated formatting.
No dependency updates.
```

这能减少大量噪音。

23.6 验证半径要匹配改动半径

小 diff 不是只改小，也要验证得准。

如果是 Level 1 单文件行为改动，先跑 focused test：

清单 23-5 验证半径要匹配改动半径

```
npm test -- order-cancel
```

如果是 Level 2 同模块联动，再跑模块测试：

清单 23-6 验证半径要匹配改动半径

```
npm test -- order
```

如果是 Level 3 跨模块联动，需要补集成路径：

清单 23-7 验证半径要匹配改动半径

```
npm test -- order
npm test -- refund
npm test -- api-contract
```

如果是 Level 4 系统级变化，除了自动化测试，还要有人工审查、迁移检查、兼容性检查和回滚方案。

很多人把“小步开发”理解成只跑小测试，这是错的。

正确做法是：

```
先用最小测试约束当前 diff。
通过后再扩大验证半径。
验证半径随着改动半径扩大。
```

否则你会得到一个局部通过、全局出事的改动。

23.7 用 Task 边界控制上下文

小步开发还能降低上下文污染。

一次只执行一个 Task，Agent 当前需要关注的上下文就更少。

例如 Task 1 只加测试：

```
只需要 requirements、现有测试、状态枚举。
不需要实现服务内部细节。
```

Task 2 只改状态转换：

```
只需要状态机文件、相关测试、状态定义。
不需要前端、不需要支付回调、不需要数据库 migration。
```

Task 3 再接退款请求创建：

```
才需要 refund request service、事务边界、幂等策略。
```

如果一开始就把所有文件都给 AI，它会同时考虑所有问题，输出会混在一起。

所以小步开发不只是 review 策略，也是上下文策略。

23.8　执行前写清“当前步骤完成定义”

一个 Task 必须有完成定义。

坏写法：

```
完成状态转换。
```

好写法：

示例 23-8　执行前写清“当前步骤完成定义”

```
Task 完成定义：
- `paid -> refund_requested` 状态转换存在。
- pending cancellation 行为不变。
- shipped cancellation 行为不变。
- duplicate cancellation 测试仍然失败，因为幂等还没实现。
- `npm test -- order-cancel` 中只有预期失败。
```

注意这里的“预期失败”。

小步开发不要求每一步都让所有测试通过。TDD 或分步实现时，中间步骤出现预期失败是正常的。关键是失败要被解释清楚，不能变成未知失败。

如果 Agent 看到失败就擅自实现后续 Task，就破坏了小步边界。

执行指令里可以写：

```
如果出现预期失败，请报告，不要为了让它通过而执行下一步。
```

23.9 用 diff 摘要训练审查节奏

每一步完成后，让 AI 输出固定格式：

清单 23-9 用 diff 摘要训练审查节奏

```
Changed files:
- `src/order/order-status-transition.ts`
- `test/order/order-cancel.test.ts`

What changed:
- Added paid -> refund_requested transition.
- Added focused test for paid cancellation.
- Kept pending and shipped tests unchanged.

Verification:
- `npm test -- order-cancel`
- Result: paid cancellation passes, duplicate cancellation still fails as expected.

Out of scope not touched:
- payment callback
- API response shape
- database schema
```

然后你用 Git 验证它的摘要：

清单 23-10 用 diff 摘要训练审查节奏

```
git diff --name-only
git diff --stat
```

AI 摘要不是证据，Git diff 和命令输出才是证据。

但固定摘要有用。它能让 Agent 养成每一步都对齐范围和验证的输出习惯。

23.10 计划外文件怎么处理

发现计划外文件，不要马上让 AI 继续。

先分类：

```
必要越界：当前 Task 的设计漏了文件。
非必要越界：顺手重构、格式化、猜测性实现。
误伤越界：格式化工具、自动生成文件、锁文件变化。
```

必要越界要回到计划：

```
当前 Task 需要扩大到 `src/refund/refund-request-service.ts`。
请先更新计划，说明为什么 Task 2 无法在原范围完成。
不要直接继续实现。
```

非必要越界要移除。

误伤越界要看项目规则。有些格式化或锁文件变化是工具自动产生的，但仍然要解释和验证。

小步开发不是禁止所有计划外变化，而是所有计划外变化都要重新授权。

23.11 生成文件、锁文件和依赖要单独处理

真实项目里，小步 diff 经常会碰到生成文件和锁文件。

比如：

```
package-lock.json
pnpm-lock.yaml
openapi-generated-client.ts
prisma/schema generated files
protobuf generated files
```

这些文件不一定不能改，但不能被悄悄带进 diff。

可以在 task 里写：

```
Generated and lock files:
- do not update dependencies in this task.
- do not regenerate API clients unless explicitly authorized.
- if a generated file changes, stop and explain which command produced it.
- if lock file changes, explain whether a dependency was added or package manager rewrote
metadata.
```

为什么要这么严？

因为 AI 可能为了让类型通过，顺手升级依赖；为了接口字段对上，顺手改生成 client；为了测试通过，顺手重跑生成命令。每一步都可能合理，但合在一个行为 diff 里，review 成本会暴涨。

依赖更新最好单独成 diff：

```
Diff A: dependency or generated artifact update
Diff B: behavior change based on updated artifact
```

这样出问题时才能判断是依赖升级导致，还是业务实现导致。

23.12 前端小步也要按行为切，不按组件切

前端任务容易按组件拆：

```
改按钮
改弹窗
改列表
改详情页
```

这不一定是好拆法。

如果需求是“已支付订单取消后显示退款处理中”，更合适的切法是按用户行为：

```
Step 1: 接口 mock 或测试数据里加入 refund_requested 状态
Step 2: 订单详情页能显示 refund_requested
Step 3: 取消成功后刷新为 refund_requested
Step 4: 重复点击被禁用或后端幂等结果被正确展示
```

每一步都能用页面测试、组件测试或截图验证。

如果按组件切，Agent 很容易同时改按钮、状态映射、API client、样式和错误处理，最后页面看起来能用，但你很难判断哪一块承担了业务行为。

前端小步同样要写 allowed files 和 forbidden changes。比如只做展示时，不应该改 API client；只做 API client 时，不应该顺手改页面样式。

23.13 小步执行可以用 checkpoint，但不要掩盖审查

有些 Agent 支持 checkpoint、审批、分支或 worktree。它们对小步开发很有帮助，但不能替代审查。

一个实际流程可以是：

示例 23-11 小步执行可以用 checkpoint，但不要掩盖审查

```
Step 1 完成
-> 查看 git diff --name-only
-> 运行 focused test
-> 记录结果
-> 接受 checkpoint
-> 授权 Step 2
```

不要把 checkpoint 当成“可以放心让它一直跑”的理由。checkpoint 只是降低回退成本，不会告诉你 diff 是否合理。

如果工具没有 checkpoint，也可以用普通 Git 做同样的事：

清单 23-12 小步执行可以用 checkpoint，但不要掩盖审查

```
git diff --name-only
git diff --stat
```

如果项目允许提交，可以在每个可审小步后提交；如果当前仓库还不适合提交，就至少保留清晰的验证记录。

23.14 小步和 PR 的关系

小步开发不等于每一步都发一个 PR。

真实项目里的关系是：

```
Task step 控制 Agent 一次改动半径
commit 记录相对完整的本地历史
PR 承载 reviewer 能一次理解的交付范围
```

一个需求可能有五个小步，但最后一个 PR：

示例 23-13 小步和 PR 的关系

```
Step 1: 加失败测试
Step 2: 改状态转换
Step 3: 接退款请求
Step 4: 补失败处理
Step 5: 扩大验证

PR: paid order cancellation enters refund flow
```

也可能拆成两个 PR：

```
PR 1: backend cancellation behavior
PR 2: frontend display for refund_requested
```

拆 PR 的判断标准不是“文章里说要小步”，而是 reviewer 能否在一次审查里理解和验证。

如果一个 PR 同时改后端状态、前端页面、移动端兼容、数据库迁移和运营后台，就算每一步执行时很小，最终审查仍然过重。

小步开发要服务交付。它控制 Agent 的执行半径，PR 控制人的审查半径。

23.15 什么时候应该停下来拆新任务

小步执行中出现这些信号，就不要继续当前任务：

- 需要新增依赖。

- 需要修改数据库 schema。
- 需要改 public API response shape。
- 需要跨 repo 改动。
- 发现当前需求和已有 spec 冲突。
- 需要把测试从 unit 升级到 E2E 才能证明行为。
- 改动文件数量超过计划两倍以上。

这些不是失败，而是任务边界变了。

正确处理是：

```
停止当前 step。
记录新发现。
更新 requirements/design/tasks。
重新授权新的小步。
```

不要让 Agent 带着“我快做完了”的状态继续扩张。AI 不会天然感知生产交付边界，边界变化必须由人重新确认。

23.16 每个小步都要能说明回滚方式

小步开发还有一个很现实的指标：这一步能不能单独回滚。

如果一个 Step 改完后，必须连着后面三个 Step 才能让项目恢复可用，这一步可能切得不对。

可以在任务里写：

清单 23-14 每个小步都要能说明回滚方式

```
Rollback:
- Step 1 tests can be reverted independently.
- Step 2 status transition depends on Step 1 tests.
- Step 3 refund request wiring should not be merged without Step 2.
- No migration or dependency update in this step.
```

回滚方式不一定是执行 `git revert`。它也可以是：

- 删除当前 Step 的新增测试。
- 恢复一个状态分支。
- 关闭一个 feature flag。
- 回退一个配置项。
- 不合并当前 PR。

关键是你要知道出问题时退哪一块，而不是回滚整天工作。

涉及数据库迁移、外部事件、队列消息、缓存 key 的任务，更要提前写回滚。AI 可以帮你生成候选方案，但是否能接受，要由程序员判断。

如果回滚说不清，就不要把它当成小步。

23.17　合并前再看一次整体 diff

小步执行过程中每一步都可控，不代表最终合并一定可控。

合并前要把所有小步放在一起看：

清单 23-15　合并前再看一次整体 diff

```
git diff --stat main...HEAD
git diff --name-only main...HEAD
```

重点看三类问题。

第一，是否有和需求无关的文件。比如锁文件、生成 client、样式文件、文档文件突然出现。

第二，是否有同一个行为被多个小步重复修改。重复修改可能说明前面的拆分不合理，也可能说明 Agent 在后续步骤推翻了前面的假设。

第三，是否有测试覆盖和改动半径不匹配。跨模块改动只跑了一个 focused test，就不能说验证充分。

小步开发的最后一步，是把局部证据合成整体交付证据。否则每一步都小，最终仍然可能是一个难审的大包。

23.18　不要让 AI 一次“修所有测试”

测试失败后，很多人会说：

```
把失败的测试都修了。
```

这类指令风险很高。

先分类失败：

示例 23-16　不要让 AI 一次“修所有测试”

```
先不要改代码。

请读取测试输出并分类：
- 当前 Task 引入的失败。
- 预期失败。
- baseline 已存在失败。
```

```
- 与当前 Task 无关的失败。

只给出分类和下一步建议，不要修。
```

分类完成后，再授权具体修复。

如果失败属于当前 Task，就在当前 diff 里修。

如果失败是后续 Task 的预期失败，不修。

如果失败是 baseline，就停止并记录。

如果失败无关，不要混进当前 diff。

这能避免 AI 把测试套件里的旧债一起拖进来。

23.19 一个完整的小步执行模板

可以把下面这段作为执行指令：

示例 23-17 一个完整的小步执行模板

```
Execute only Task <N>.

Scope:
- Allowed files:
  - <file 1>
  - <file 2>
- Forbidden changes:
  - no unrelated refactor
  - no formatting-only changes outside allowed files
  - no dependency update
  - no public API shape change unless explicitly requested

Expected result:
- <behavior change>
- <expected test state>

Verification:
- <command>

Stop if:
- you need to modify files outside the allowed list.
- baseline tests fail.
- the implementation requires a broader design change.
- the requested behavior conflicts with existing tests or specs.

After finishing:
- stop.
```

```
- report changed files.
- report verification command and result.
- do not start the next task.
```

这不是为了限制 AI 变笨，而是为了让每一步都可审查。

23.20　可以用配套工具辅助，但不要依赖工具替你判断

配套工程的工具 module `ai-coding-workflow-kit` 里有一个 `verify-ledger`，可以检查验证记录是否包含必要段落和命令证据：

清单 23-18　可以用配套工具辅助，但不要依赖工具替你判断

```
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
```

它不会判断业务正确性，也不会替你审 diff。它只能提醒你：有没有留下命令、结果和风险记录。

小步开发最核心的证据仍然是：

清单 23-19　可以用配套工具辅助，但不要依赖工具替你判断

```
git diff --name-only
git diff --stat
git diff
```

再加上对应测试命令。

工具可以帮你不漏记录，但不能替你判断“这个 diff 是否应该存在”。

23.21　常见错误

错误一：把小步理解成少写几行。

小步看的是行为边界、文件范围、验证半径和回滚能力，不是行数。

错误二：一个 Task 混测试、实现、重构、文档。

这些可以都要做，但不应该都在一个 diff 里。

错误三：计划外文件出现后继续推进。

计划外文件必须先解释、分类、重新授权。

错误四：让 AI 修所有测试。

测试失败先分类，再决定是否修。不要让旧失败混进当前 diff。

错误五：只看 AI 总结，不看 Git diff。

AI 总结可能漏掉计划外改动。`git diff --name-only` 和 `git diff --stat` 是必查动作。

错误六：每一步都跑全量测试。

小步阶段先跑 focused test，关键节点再扩大验证。每步都跑全量可能太慢，反而降低反馈频率。

23.22 改完后重点看什么

一次小步执行结束后，看这几个结果：

- 改动文件是否在允许列表里。
- 是否有无关格式化、重命名、抽函数。
- diff 能否一句话解释。
- 验证命令是否覆盖当前行为。
- 测试失败是否已经分类。
- 这一步能否单独回滚。

如果答案不清楚，先不要进入下一步。

AI Coding 的速度优势来自高频反馈，不是一次性吞下更多工作。更可控的节奏，是让 AI 快速完成一个小 diff，然后由人用证据决定是否继续。

23.23 常见问题

小步会不会变慢？

单步看会慢一点，整体通常更快。大 diff 出问题后的排查和回滚成本更高。

一次只改一个文件是不是太死？

不是按文件数绝对限制。关键是一个 diff 能否单独解释、验证和回滚。同模块两个文件可以是合理小步，跨模块五个文件通常要拆。

AI 说必须重构才能实现，怎么办？

让它先解释为什么不重构无法实现。如果确实必要，先做纯重构 diff，验证行为不变，再做行为变更 diff。

计划外文件已经改了，能不能直接保留？

先分类。必要越界要更新计划并重新授权；非必要越界要移除；误伤越界要解释来源和影响。

小步开发和 TDD 什么关系？

TDD 是很适合小步开发的一种方式。先用失败测试定义行为，再用最小实现让测试通过。后面会单独讲 TDD + AI Coding。

23.24 可复查资料

- Git diff 官方文档：https://git-scm.com/docs/git-diff〔23-1〕
- Git status 官方文档：https://git-scm.com/docs/git-status〔23-2〕
- GitHub Copilot coding agent best practices：https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results〔23-3〕
- VS Code Source Control：https://code.visualstudio.com/docs/sourcecontrol/overview〔23-4〕

本章资源

〔23-1〕 https://git-scm.com/docs/git-diff
https://git-scm.com/docs/git-diff

〔23-2〕 https://git-scm.com/docs/git-status
https://git-scm.com/docs/git-status

〔23-3〕 https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results
https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results

〔23-4〕 https://code.visualstudio.com/docs/sourcecontrol/overview
https://code.visualstudio.com/docs/sourcecontrol/overview

第24章　TDD + AI Coding：先让失败测试约束模型

AI Coding 里，测试不只是验证工具。

测试是约束模型的工具。

你用自然语言告诉 AI：

```
已支付订单取消后进入退款流程。
```

它可能理解成很多种实现：

- 订单状态直接变成 canceled，然后异步发起退款。
- 订单状态先变成 refund_requested，再创建退款请求。
- 先创建退款请求，再更新订单状态。
- 重复取消时再创建一个退款请求。
- 退款创建失败时仍然把订单状态改掉。

这些实现都可能被模型写得很像样。单靠 prompt，很难保证它选中你真正要的行为。

失败测试的价值，就是把“你想要什么”变成可执行约束。

TDD + AI Coding 的关键不是为了遵守某种开发仪式，而是先让模型面对一个明确的、当前会失败的行为检查，再让它用最小实现把这个检查变绿。

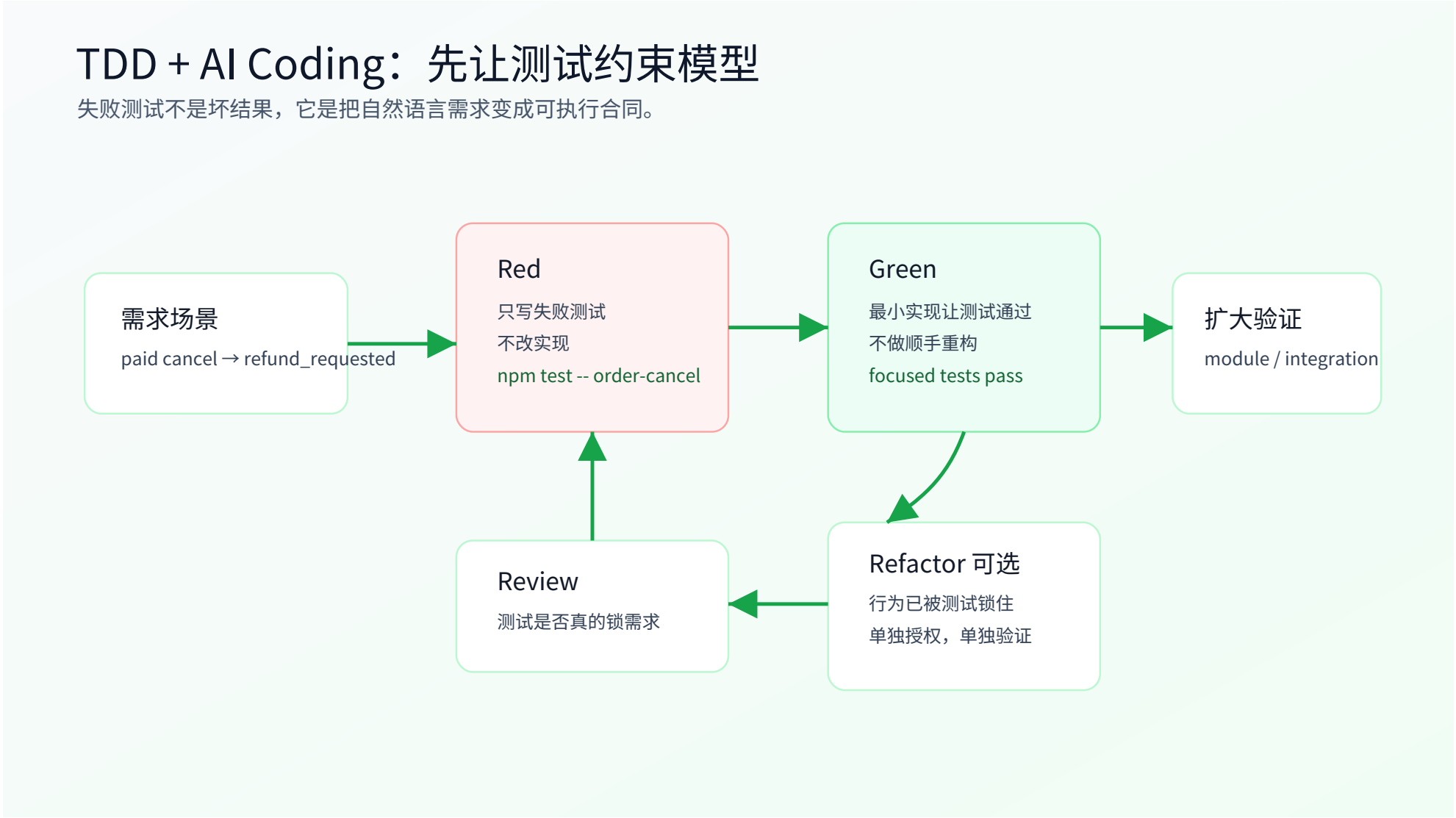


图 24-1　TDD + AI Coding 循环

24.1 为什么测试比自然语言更能约束 AI

自然语言有两个问题。

第一，它容易被补全。

“走退款流程”对不同项目含义不同。模型会根据训练中的常见模式和当前代码片段补全缺失细节。

第二，它很难证明执行结果。

AI 说“我已经处理了幂等”，你还要去看代码判断它是不是真的处理。测试能把这件事变成一个命令结果。

例如需求是：

```
paid order cancellation should enter refund_requested and create exactly one refund request.
```

一个行为测试可以直接锁住三个事实：

清单 24-1 为什么测试比自然语言更能约束 AI

```
test('paid order cancellation enters refund flow once', async () => {
  const order = await fixtures.createOrder({ status: 'paid' });

  const result = await cancelOrder(order.id);

  assert.equal(result.status, 'refund_requested');
  assert.equal(await refundRequests.countForOrder(order.id), 1);
});
```

这段测试不是为了展示语法。它在告诉 AI：

- 最终状态必须是 `refund_requested`。
- 退款请求必须创建。
- 数量必须是 1。

模型不能用“我觉得这样也合理”绕过去。只要行为不对，测试就失败。

24.2 AI 写测试也要先审

不要直接让 AI “补测试并实现”。

这会让测试配合实现，而不是约束实现。

把测试和实现拆开：

示例 24-2 AI 写测试也要先审

```
先只写测试。
不改实现。
运行 focused test。
确认测试因为目标行为缺失而失败。
人审测试是否真的表达需求。
再授权最小实现。
```

错误指令：

```
补测试并实现 paid order cancellation refund flow。
```

指令要先限制实现：

示例 24-3 AI 写测试也要先审

```
只执行测试步骤，不改实现代码。

请在 `test/order/order-cancel.test.ts` 中新增失败测试，覆盖：
- paid order cancellation enters refund_requested.
- duplicate paid cancellation does not create duplicate refund requests.
- refund request creation failure keeps order paid.

要求：
- 保留 pending 和 shipped cancellation 旧测试。
- 运行 `npm test -- order-cancel`。
- 预期新增 paid/duplicate/failure 用例失败。
- 如果旧测试失败，停止并报告。
```

这个指令的重点是“不改实现代码”。

如果 AI 在测试步骤里顺手改实现，让测试直接通过，你就失去了 TDD 的红灯阶段。看起来效率更高，实际少了最关键的证据：测试是否能在旧实现上抓到问题。

24.3 失败测试也要审质量

不是所有失败测试都有价值。

有些测试会失败，但失败原因和需求无关；有些测试只断言实现细节；有些测试太弱，随便写个假实现也能通过。

失败测试也要审质量

TDD 的关键不是先写测试，而是先写能约束行为的测试。

不合格测试

- ✗ 只断言函数被调用，不断言业务结果
- ✗ 为了通过实现而改弱断言
- ✗ 只覆盖 happy path，不覆盖幂等和失败
- ✗ 失败原因和目标需求无关

合格测试

- ✓ 断言状态、幂等、外部副作用
- ✓ 实现前稳定失败，旧行为测试通过
- ✓ 覆盖 happy path、重复请求、失败回滚
- ✓ 测试名能对应 requirement 或 scenario

先审测试，再授权实现。

图 24-2　失败测试质量过滤

一个合格的失败测试要满足四个条件：

- 失败原因就是目标行为缺失。
- 旧行为的回归测试仍然通过。
- 断言业务结果，而不是只断言内部调用。
- 测试名能对应 requirement 或 scenario。

例如这个测试就偏弱：

清单 24-4 失败测试也要审质量

```
test('calls refund service when canceling paid order', async () => {
  const refundService = mockRefundService();
  await cancelOrder(paidOrder.id, { refundService });
  assert.equal(refundService.create.called, true);
});
```

它只证明调用发生过，不证明订单状态正确，不证明幂等，不证明失败时是否回滚。

更好的测试是：

清单 24-5 失败测试也要审质量

```
test('paid cancellation creates one refund request and moves order to refund_requested', async ()
=> {
  const order = await fixtures.createOrder({ status: 'paid' });

  const result = await cancelOrder(order.id);

  assert.equal(result.status, 'refund_requested');
  assert.equal(await refundRequests.countForOrder(order.id), 1);
});
```

再补一个幂等测试：

清单 24-6 失败测试也要审质量

```
test('duplicate paid cancellation does not create another refund request', async () => {
  const order = await fixtures.createOrder({ status: 'paid' });

  await cancelOrder(order.id);
  await cancelOrder(order.id);

  assert.equal(await refundRequests.countForOrder(order.id), 1);
  assert.equal((await orders.find(order.id)).status, 'refund_requested');
});
```

再补一个失败路径：

清单 24-7 失败测试也要审质量

```
test('paid order remains paid when refund request creation fails', async () => {
  const order = await fixtures.createOrder({ status: 'paid' });
  refundRequests.failNextCreate(new Error('provider unavailable'));

  await assert.rejects(() => cancelOrder(order.id));

  assert.equal((await orders.find(order.id)).status, 'paid');
});
```

这三类测试组合起来，需求边界才被测试锁住。

24.4 不要让 AI 改弱测试

AI 在让测试变绿时，有时会选择修改测试。

比如测试原本断言：

清单 24-8 不要让 AI 改弱测试

```
assert.equal(result.status, 'refund_requested');
```

实现做不到时，它可能改成：

清单 24-9 不要让 AI 改弱测试

```
assert.ok(['canceled', 'refund_requested'].includes(result.status));
```

测试开始配合实现，约束就失效了。

执行实现步骤时，要明确：

```
不要修改新增测试的业务断言。
如果测试断言与当前代码冲突，优先修改实现。
如果你认为测试写错，先停止并说明原因，不要直接改测试。
```

测试不是不能改。测试写错时必须改。但修改测试需要重新授权，不能在实现阶段偷偷把约束放松。

一个实用规则：

```
Red 阶段可以改测试。
Green 阶段默认不改测试断言。
Refactor 阶段只在行为不变时整理测试结构。
```

这能减少 AI 为了通过测试而篡改目标的情况。

24.5 最小实现比“完整实现”更重要

TDD 的 Green 阶段，不要求一次做完整架构。

它要求用最小实现让当前测试通过。

错误指令：

```
实现完整退款流程，并顺手优化订单取消代码。
```

指令要限制改动范围：

示例 24-10 最小实现比“完整实现”更重要

```
只让当前失败测试变绿。

允许修改：
- `src/order/order-status-transition.ts`
- `src/order/order-cancel-service.ts`
- 必要时 `src/refund/refund-request-service.ts`

禁止：
- 修改新增测试的业务断言。
- 改 API response shape。
- 改 payment callback。
- 做无关重构。

验证：
- `npm test -- order-cancel`

完成后停止。
```

最小实现不是写烂代码。它的意思是：不要在没有测试约束的地方扩大设计。

如果测试只锁住 paid cancellation，就不要顺手实现退款结算 job。

如果测试只锁住重复取消幂等，就不要顺手改支付回调。

每个新增行为都应该先有测试约束。

24.6 TDD 可以降低 AI 的“自信错”

AI 很容易写出自信但错误的代码。

比如它看到一个 `RefundRequestService.create(orderId)`，就假设它有幂等能力。实际项目里可能没有。

如果先写 duplicate cancellation 测试，模型就会被迫面对：

```
调用两次 cancelOrder 后，refund request count 仍然必须是 1。
```

它不能只靠猜。

如果实现失败，测试会把问题暴露出来。此时不要让 AI 继续乱改，而是让它解释：

示例 24-11 TDD 可以降低 AI 的“自信错”

```
测试失败了。

先不要改代码。
请基于失败输出说明：
- 失败的是哪个 requirement。
- 实际行为是什么。
- 当前实现缺哪一层约束。
- 你准备改哪个文件。
```

这会把调试从“继续尝试”拉回证据循环。

24.7 什么时候不适合 TDD

不是所有任务都适合严格 TDD。

这些任务可以不用完整 TDD：

- 纯文案、样式、配置调整。
- 很薄的 glue code，已有上层测试覆盖。
- 探索性 spike，目标是理解可行性。
- 大规模机械迁移，重点是脚本和 diff 审查。
- 很难本地构造依赖的外部系统交互。

但即使不用完整 TDD，也应该有验证证据。

例如前端交互可以先用浏览器截图或 Playwright 路径锁行为；外部系统交互可以用契约测试、mock server 或人工验证记录；机械迁移可以用脚本输出和抽样 diff。

TDD 是强约束方式，不是唯一方式。

24.8 先选测试层级，不要把所有东西塞进一个测试

AI 很容易把一个需求写成一个巨大的端到端测试。这个测试覆盖了页面、接口、数据库、副作用，读起来完整，失败时却很难定位。

TDD 阶段先选测试层级。

表 24-1 先选测试层级，不要把所有东西塞进一个测试

要约束的行为	优先测试层级	原因
状态转换规则	unit / domain test	反馈快，定位准
API response shape	contract / API test	锁住调用方合同
数据持久化和事务	integration test	能证明数据库状态
用户点击路径	component / E2E	能证明真实交互
外部副作用	service test / fake adapter	能证明次数、幂等和失败路径

订单取消改退款，不应该只靠一个 Playwright 测试证明全部行为。

可以拆成：

```
domain test: paid -> refund_requested
service test: exactly one refund request is created
API test: response shape unchanged
E2E test: 页面显示退款处理中
```

这样每个失败都有定位依据。

对 AI 来说，测试层级越清楚，它越不容易为了让一个大测试通过而乱改多层代码。

24.9 测试数据要贴近真实状态

AI 写测试时，经常会造一个过度简化的 fixture：

清单 24-12 测试数据要贴近真实状态

```
const order = { status: 'paid' };
```

如果真实取消逻辑需要 `id`、`userId`、`paymentId`、`version`、`shippingStatus`，这个 fixture 可能让测试绕开关键路径。

可以要求：

```
测试数据必须使用项目已有 fixture factory。
不要手写和生产 schema 不一致的对象。
如果 fixture 缺少能力，先报告需要扩展的字段，不要在测试里造假的最小对象。
```

好的测试数据应该表达业务状态，而不是迎合函数签名：

清单 24-13 测试数据要贴近真实状态

```
const order = await fixtures.createOrder({
  status: 'paid',
  paymentStatus: 'captured',
  shippingStatus: 'not_shipped'
});
```

这比 `{ status: 'paid' }` 更能防止 AI 写出只在测试里成立的实现。

24.10 mock 只能隔离边界，不能伪造业务事实

Mock 用得不对，TDD 会变成“测试模型想象中的系统”。

退款请求创建可以 mock 外部支付 provider，但不应该 mock 掉订单状态持久化。如果你要证明订单失败时保持 paid，就必须检查真实 repository 或足够接近真实的测试数据库。

可以写规则：

示例 24-14 mock 只能隔离边界，不能伪造业务事实

```
Allowed mocks:
- external payment provider
- email / SMS / push gateway
- time and id generator

Avoid mocking:
- order status transition logic
- repository behavior under test
- API response serialization being verified
```

如果 mock 掉了正在验证的东西，测试就失去约束力。

比如这个测试不够好：

清单 24-15 mock 只能隔离边界，不能伪造业务事实

```
mockOrderRepository.findById.returns({ status: 'paid' });
mockOrderRepository.save.returns({ status: 'refund_requested' });
```

它证明不了持久化真的发生，只证明 mock 被按预期调用。

更有价值的是用测试数据库或内存 repository 检查最终状态：

清单 24-16 mock 只能隔离边界，不能伪造业务事实

```
const saved = await orders.find(order.id);
assert.equal(saved.status, 'refund_requested');
```

AI 可以写 mock，但你要审 mock 是否隔离的是外部边界，而不是掩盖核心行为。

24.11 红灯、绿灯、整理阶段要分开授权

TDD 的三个阶段在 AI Coding 里要拆成三次授权。

Red：

```
只写失败测试。
不改实现。
运行 focused test。
报告失败是否符合预期。
```

Green：

```
只做最小实现。
不改弱测试断言。
不做无关重构。
运行 focused test。
```

Refactor：

```
只做行为不变的整理。
不新增功能。
所有相关测试必须保持通过。
输出重构前后 diff 摘要。
```

不要把 Green 和 Refactor 合并。AI 很容易在“让测试通过”的同时整理结构，最后你不知道行为变化来自哪里。

如果 Green 阶段代码确实很丑，可以先接受一个小的可工作实现，再单独开 Refactor 步骤。重构要靠已有测试保护，而不是靠 AI 自信。

24.12 红灯证据要写进记录

TDD 最容易被做成形式：文章里说先写测试，实际执行时没人保留红灯证据。

至少记录三件事：

清单 24-17 红灯证据要写进记录

```
Red evidence:
- command: `npm test -- order-cancel`
- expected failing tests:
  - paid cancellation enters refund flow
  - duplicate paid cancellation creates one refund request
  - refund request failure keeps order paid
```

```
- existing tests:
  - pending cancellation passed
  - shipped cancellation passed
```

这段记录有两个作用。

第一，它证明新测试不是写完就被实现配合通过。 第二，它证明旧行为没有在 Red 阶段先坏掉。

如果没有红灯证据，reviewer 很难判断测试是不是事后补的。事后补测试不一定没价值，但它和 TDD 不是一回事。

24.13 审 AI 写的测试，先看四个问题

AI 生成测试草稿后，不要只看能不能跑。

先问四个问题：

```
它是否对应某条 requirement?
它是否会在旧实现上失败?
它是否断言用户可见行为、持久化状态或副作用结果?
它是否依赖过度 mock 或伪造 fixture?
```

如果答案不清楚，让 Agent 先解释：

```
请逐条说明新增测试对应哪个 requirement。
请说明为什么这些测试会在当前实现上失败。
请说明每个 mock 隔离的是外部边界还是核心行为。
```

这一步很实用。AI 有时会写很多测试，但只有一两个真正约束需求。测试数量不是质量，约束力才是质量。

24.14 什么时候可以接受事后补测试

生产项目里并不是所有改动都能严格 TDD。有时你是在排障，有时需要先做 spike，有时现有代码很难构造红灯。

事后补测试可以接受，但要诚实记录：

清单 24-18 什么时候可以接受事后补测试

```
Testing note:
- This was not strict TDD.
- Implementation came first because the failure required exploratory debugging.
- Regression test was added after root cause was confirmed.
- Test fails if the fix is reverted.
```

这比假装做了 TDD 更专业。

事后补测试的最低标准是：回退修复后测试会失败。否则它只是覆盖现状，不一定能防回归。

24.15 契约测试适合防 AI 改坏接口

很多 AI 实现会在不知不觉中改接口合同。

比如原 response 是：

清单 24-19 契约测试适合防 AI 改坏接口

```
{
  "id": "123",
  "status": "canceled",
  "message": "ok"
}
```

实现退款流程后，AI 可能改成：

清单 24-20 契约测试适合防 AI 改坏接口

```
{
  "orderId": "123",
  "state": "refund_requested",
  "refundRequestId": "r_1"
}
```

新结构可能更“合理”，但调用方会坏。

这类问题适合用契约测试先锁住：

清单 24-21 契约测试适合防 AI 改坏接口

```
test('cancel order response shape remains compatible', async () => {
  const response = await request.post('/api/orders/paid-order-1/cancel');
  const body = await response.json();

  assert.deepEqual(Object.keys(body).sort(), ['id', 'message', 'status'].sort());
  assert.equal(body.status, 'refund_requested');
});
```

这不是为了迷信字段列表，而是告诉 Agent：这次只允许状态值变化，不允许 response shape 变化。

只要需求涉及 API、事件、队列消息、配置文件、CLI 输出，都可以考虑契约测试。AI 最容易在这些边界上“顺手优化”。

24.16 外部依赖要用 fake 或 contract，不要真的打生产服务

TDD 里遇到支付、短信、邮件、AI 模型、第三方 API，不应该真的调用生产服务。

可以选择：

- fake adapter：本地实现同样接口，记录调用次数和参数。
- mock server：模拟第三方响应和失败。
- contract test：验证请求 shape 和错误处理。
- staging 手工验证：作为上线前补充，不替代本地测试。

订单退款场景里，测试应该证明：

```
refund request creation is called once.
idempotency key is stable.
failure keeps order paid.
payment callback code is untouched.
```

不应该在单元测试里真的请求支付渠道。

写给 Agent 的规则可以是：

```
Do not call real external services in tests.
Use the existing fake provider or test adapter.
If no fake exists, stop and propose a minimal fake boundary before implementation.
```

没有 fake 时，不要让 AI 在业务测试里临时写一个和真实接口完全不相干的 mock。先定义边界，再写测试。

24.17 测试维护也要小步

AI 改实现时，测试也可能需要整理。但测试维护要和行为断言分开。

可以拆成：

```
Step A：添加新行为测试，保留旧测试结构。
Step B：最小实现让测试通过。
Step C：如果测试重复严重，再整理测试 helper。
```

不要在 Step A 里大规模重写旧测试。旧测试是你的 baseline，重写 baseline 会让你失去判断新需求影响的依据。

如果确实要改测试 helper，要求 Agent 证明行为不变：

```
只重构测试 helper。
不改变任何业务断言。
重构前后同一测试命令都必须通过。
输出断言变化清单，如果断言变化为 0，请明确说明。
```

测试代码也是生产资产。AI 可以帮助维护，但不能让它在实现阶段随意削弱约束。

24.18 怎么把 TDD 写进 tasks

在 `tasks.md` 里，不要只写“补测试”。

要写成：

清单 24-22 怎么把 TDD 写进 tasks

```
## Task 1: Add failing behavior tests

Goal:
- Convert paid cancellation requirements into executable checks.

Files:
- `test/order/order-cancel.test.ts`

Add tests:
- paid cancellation moves order to refund_requested and creates one refund request.
- duplicate paid cancellation does not create another refund request.
- refund request creation failure keeps order paid.

Verify:
- `npm test -- order-cancel`

Expected before implementation:
- existing pending and shipped tests pass.
- new paid, duplicate and failure tests fail.

Stop if:
- existing behavior tests fail before implementation.
- test setup cannot create paid orders.
```

再写实现任务：

清单 24-23 怎么把 TDD 写进 tasks

```
## Task 2: Make focused tests pass with minimal implementation

Files:
- `src/order/order-status-transition.ts`
- `src/order/order-cancel-service.ts`
- `src/refund/refund-request-service.ts`

Verify:
- `npm test -- order-cancel`

Rules:
- Do not weaken Task 1 assertions.
- Do not change public API response shape.
- Do not modify payment callback flow.
```

```
Stop if:
- database schema changes are required.
- refund idempotency requires a broader design decision.
```

这样的 tasks 能直接驱动 AI 小步执行。

24.19 用本项目的测试看一个真实边界

配套工程的工具 module `ai-coding-workflow-kit` 里有一个很小的验证记录校验器。它不是业务系统，但测试展示了一个真实边界：AI 声称“验证完成”时，至少要有必要段落、命令块和通过信号。

对应测试长这样：

清单 24-24 用本项目的测试看一个真实边界

```
test('reports missing sections and missing command evidence', () => {
  const result = validateLedger('## Baseline\nNo command yet.');
  assert.equal(result.ok, false);
  assert.deepEqual(result.missing, [
    'Change Summary',
    'Verification Commands',
    'Evidence',
    'Residual Risk'
  ]);
  assert.equal(result.hasCommand, false);
  assert.equal(result.hasPassedSignal, false);
});
```

它锁住的不是语法，而是行为：缺段落、缺命令、缺通过信号时不能被认为可用。

如果让 AI 改这个校验器，正确的 TDD 步骤不是“优化校验逻辑”，而是先补一个失败测试。例如要求 ledger 必须包含 baseline 命令：

清单 24-25 用本项目的测试看一个真实边界

```
test('requires baseline command evidence', () => {
  const result = validateLedger(`
## Baseline
Existing failure was mentioned but no command was recorded.

## Change Summary
Restricted status transition.

## Verification Commands
\`\`\`bash
npm test
\`\`\`

## Evidence
```

```
PASS 12 tests passed.

## Residual Risk
Manual payment provider callback not covered.
`);

  assert.equal(result.ok, false);
});
```

这个测试失败后，再让 AI 修改 `verification-ledger.js`。这样它会被测试约束在“baseline 也要有命令证据”这个行为上，而不是随手重写整个校验器。

这里不是为了造代码，而是为了说明一个能迁移到多数项目的原则：先把你担心的失败模式写成测试。

24.20 常见错误

错误一：让 AI 同时写测试和实现。

这样看不到测试是否能在旧实现上失败，也容易让测试配合实现。

错误二：只测 happy path。

真实问题常出在重复请求、失败回滚、兼容性和边界状态。

错误三：断言内部调用，不断言业务结果。

内部调用可以作为补充，但核心测试要断言状态、数据、副作用和错误行为。

错误四：实现阶段修改测试断言。

测试写错可以改，但必须停下来说明原因并重新授权。

错误五：测试失败后让 AI 自由修。

先分类失败，再决定改测试、改实现还是停住。

错误六：把 TDD 用在所有任务上。

小文案、小配置、探索 spike 不一定需要 TDD，但仍然需要对应验证证据。

24.21 改完后重点看什么

TDD + AI Coding 做完后，看这些结果：

- 新测试是否先在旧实现上失败过。
- 失败原因是否对应目标需求。
- 旧行为测试是否仍然通过。
- 实现阶段是否改弱了测试断言。

- 最小实现是否只覆盖当前测试约束的行为。
- 后续是否扩大了模块测试或集成验证。

如果没有红灯证据，TDD 就只剩“先写了测试”的形式。

红灯记录还会影响后续排障。后面如果 CI 出现同一失败，reviewer 能判断它是预期红灯、旧失败，还是当前实现引入的新失败。

AI Coding 里的 TDD，是把模型从“根据文字猜实现”拉到“根据可执行行为收敛实现”。测试红灯比多写几句 prompt 更能约束结果。

24.22 常见问题

让 AI 写测试靠谱吗？

可以，但要先审测试。AI 很适合根据 requirement 生成测试草稿，不适合无人审查地决定测试是否充分。

失败测试是不是必须真的跑一遍？

复杂行为建议跑。没有红灯结果，你不知道测试是否能抓住旧实现的问题。跑不了时要记录原因和替代证据。

测试写错了怎么办？

停下来改测试。不要在实现阶段悄悄改断言。把“为什么测试错了”写清楚，再重新进入 Red 阶段。

已有测试很多，还需要新测试吗？

看现有测试是否覆盖目标 requirement。如果没有覆盖新增行为、幂等和失败路径，就需要补。

TDD 会不会让 AI 速度变慢？

单次输出会变慢，但减少返工。对状态机、权限、支付、持久化这类风险高的任务，红灯证据很值。

24.23 可复查资料

- Node.js Test Runner 官方文档：https://nodejs.org/api/test.html〔24-1〕
- Jest getting started：https://jestjs.io/docs/getting-started〔24-2〕
- Martin Fowler: TestDrivenDevelopment：https://martinfowler.com/bliki/TestDrivenDevelopment.html〔24-3〕
- OpenAI Codex best practices：https://developers.openai.com/codex/learn/best-practices〔24-4〕

本章资源

〔24-1〕 https://nodejs.org/api/test.html
https://nodejs.org/api/test.html

〔24-2〕 https://jestjs.io/docs/getting-started
https://jestjs.io/docs/getting-started

〔24-3〕 https://martinfowler.com/bliki/TestDrivenDevelopment.html
https://martinfowler.com/bliki/TestDrivenDevelopment.html

〔24-4〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

第25章　系统化调试：让 AI 基于证据排障

AI 写代码快，猜原因也快。

这在调试时很危险。

你贴一段报错：

```
TypeError: Cannot read properties of undefined (reading 'status')
```

AI 很可能马上给出修复建议：

```
这里加一个空值判断即可。
```

有时它是对的。更多时候，它只是把症状压下去了。真正的问题可能是上游查询没有返回订单、mock 数据漏字段、异步初始化顺序错了、接口响应 shape 变了，或者测试环境没有加载 fixture。

调试不是“看到错误就修”。调试是先证明问题从哪里开始，再修根因。

AI Coding 里的系统化调试，就是强制 Agent 先走证据循环：

```
复现 -> 收集证据 -> 缩小边界 -> 提出单一假设 -> 最小验证 -> 修根因 -> 回归
```

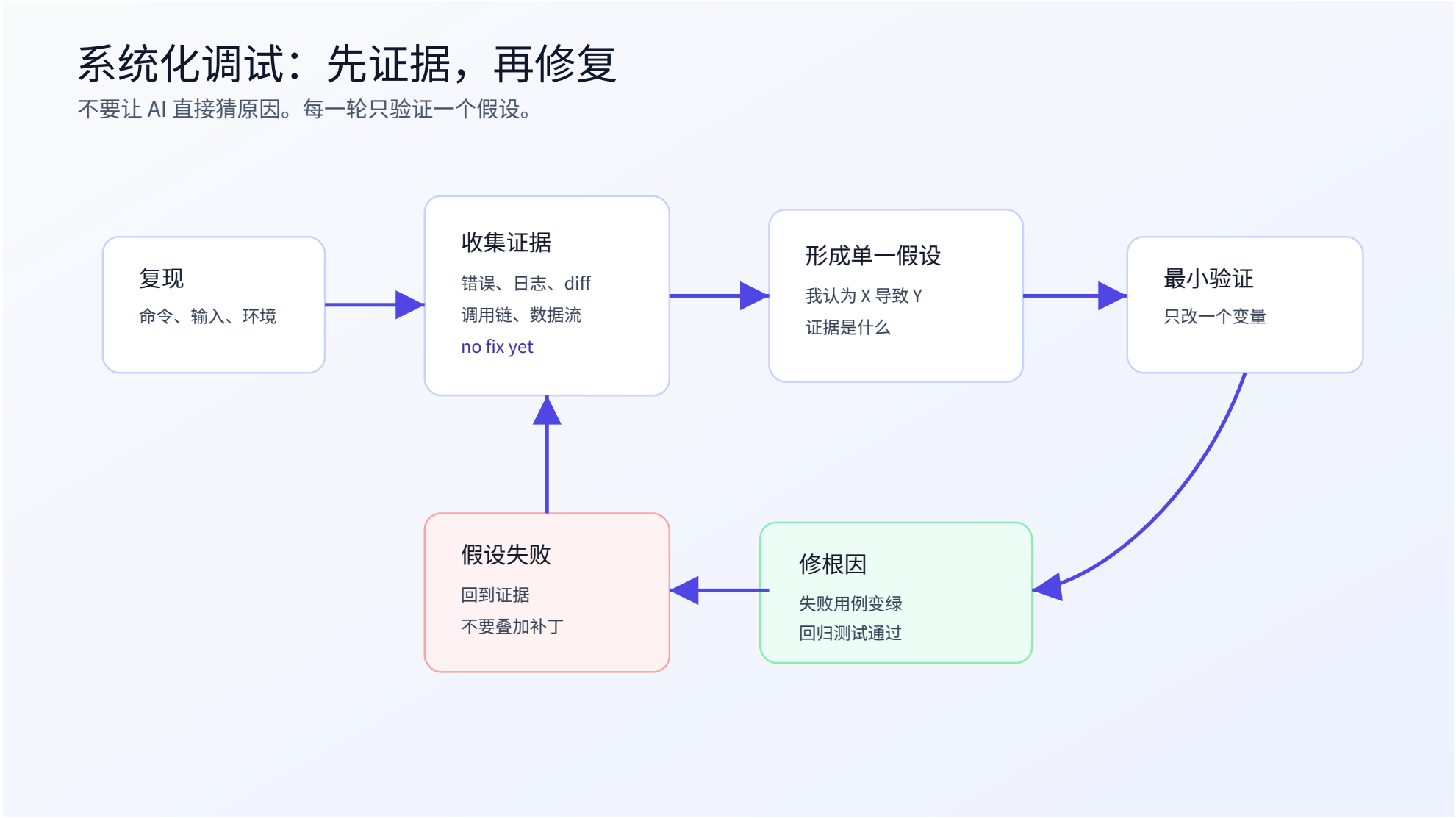


图 25-1　系统化调试证据循环

25.1　先禁止直接修

调试任务的第一条指令，应该是：

```
先不要修改代码。
```

这句话要放在最前面。

很多 Agent 默认会进入“修复模式”。它看到错误，就尝试补空值、改条件、重跑测试。如果第一次没过，再补一层判断。几轮之后，原始错误可能没了，但代码里多了一堆防御性分支，根因仍然存在。

调试输入可以这样写：

示例 25-1　先禁止直接修

```
先不要修改代码。

请基于下面的错误输出做系统化排障：
- 读完整错误和堆栈。
- 找到触发命令。
- 说明是否能稳定复现。
- 找出最近相关 diff。
- 沿调用链追踪 `status` 从哪里来。
- 只提出一个最可能假设。
- 给出最小验证动作。

在我确认前，不要修复。
```

这能把 AI 从“猜修复”切到“找证据”。

25.2　复现要写成命令，不要写成感觉

很多调试失败，是因为复现不清。

坏记录：

```
订单取消测试偶尔失败。
```

好记录：

清单 25-2　复现要写成命令，不要写成感觉

```
npm test -- order-cancel
```

输出：

```
FAIL test/order/order-cancel.test.ts
TypeError: Cannot read properties of undefined (reading 'status')
  at cancelOrder (src/order/order-cancel-service.ts:42:18)
  at test/order/order-cancel.test.ts:87:24
```

还要记录：

```
Reproduction:
- command: `npm test -- order-cancel`
- frequency: 5/5
- branch: `feature/paid-cancel-refund`
- last known passing state: before Task 3
```

如果不能稳定复现，不要让 AI 直接修。

应该先收集更多数据：

示例 25-3 复现要写成命令，不要写成感觉

```
这个失败不是稳定复现。
请先提出可以增加的诊断信息：
- 日志位置
- 关键变量
- 组件边界
- 需要重复运行的命令
不要修改业务逻辑。
```

不稳定问题最怕随机补丁。先让它变得可观察。

25.3 读完整错误，不要只读最后一行

AI 很容易被错误最后一行带偏。

例如：

```
Cannot read properties of undefined (reading 'status')
```

最后一行说 `status` 是 undefined，但根因不一定在读取这一行。要看：

- 哪个对象是 undefined。
- 它从哪个函数返回。
- 这个函数的输入是什么。
- 输入为什么变了。
- 最近 diff 是否改过 fixture、mock、查询条件或状态转换。

可以让 AI 这样做：

示例 25-4 读完整错误，不要只读最后一行

```
请完整读取堆栈。

输出：
- first failing line
- caller chain
- value that became undefined
- where this value is created
- which recent change could affect it
```

如果它直接给修复方案，要求它回到证据：

```
先不要修。
你还没有证明 `order` 为什么是 undefined。
请沿调用链往上追踪来源。
```

25.4 沿数据流往上追

深层错误要逆着调用链追。

比如错误发生在：

清单 25-5 沿数据流往上追

```
if (order.status === 'paid') {
  // ...
}
```

不要先加：

清单 25-6 沿数据流往上追

```
if (!order) throw new Error('order not found');
```

这可能只是把错误换了个形式。

先追：

```
order 从哪里来?
```

可能是：

清单 25-7 沿数据流往上追

```
const order = await orderRepository.findById(orderId);
```

继续追：

```
orderId 从哪里来?
```

可能是测试里：

清单 25-8　沿数据流往上追

```
const order = await fixtures.createOrder({ state: 'paid' });
await cancelOrder(order.id);
```

再追：

```
fixtures.createOrder 返回的 id 是否真实落库?
```

这时可能发现根因：Task 3 改了 fixture 默认字段，把 `status` 写成 `state`，repository 查询条件仍然按 `status` 查。

要修的不是 `cancelOrder` 的空值判断，而是 fixture 或 repository 的字段映射。

根因和症状的差别就在这里。

25.5　一次只提出一个假设

AI 调试时常见坏味道是一次给多个可能原因：

```
可能是数据未初始化、异步时序问题、mock 配置错误或状态字段不一致。
```

这没有错，但不能直接进入修复。

下一步要选一个最可能假设，并说明证据：

示例 25-9　一次只提出一个假设

```
Hypothesis:
- `fixtures.createOrder` now writes `state`, but `orderRepository.findById` expects `status`.

Evidence:
- failing test creates order through fixture.
- stack shows `order` is undefined inside cancel service.
- recent diff changed fixture field names.
- repository query still filters by `status`.

Minimal verification:
- inspect the created fixture record before cancel.
- run only `npm test -- order-cancel -- --test-name-pattern paid`.
```

这才是可验证假设。

如果假设失败，不要叠加第二个补丁。回到证据：

```
这个假设不成立。
请记录验证结果，并基于新证据提出一个新的单一假设。
```

调试最怕“改一点再改一点”。每一轮必须能说明上一轮假设为什么成立或失败。

25.6 最小验证动作不是完整修复

假设阶段的验证动作应该尽量小。

例如怀疑 fixture 字段写错，可以先加临时诊断或直接查看测试数据，而不是马上改业务代码。

可用动作包括：

清单 25-10 最小验证动作不是完整修复

```
npm test -- order-cancel -- --test-name-pattern paid
git diff -- test/order/order-cancel.test.ts
rg -n "createOrder|status|state" test src
```

前端问题可以用 Playwright 调试：

清单 25-11 最小验证动作不是完整修复

```
npx playwright test order-cancel.spec.ts --debug
```

CI 问题可以先看 workflow logs。GitHub Actions 官方文档也把 workflow run logs 作为排障入口。不要在本地猜一个“可能的环境变量问题”，先看日志里哪一步失败、输入是什么、环境差异是什么。

最小验证的原则：

```
一次只验证一个变量。
不要把验证动作和修复动作混在一起。
```

25.7 修复前最好有失败用例

如果问题能被测试表达，先写失败用例。

例如你确认根因是 duplicate cancel 会创建两条退款请求，那么修复前先补：

清单 25-12 修复前最好有失败用例

```
test('duplicate paid cancellation creates only one refund request', async () => {
  const order = await fixtures.createOrder({ status: 'paid' });

  await cancelOrder(order.id);
  await cancelOrder(order.id);
```

```
  assert.equal(await refundRequests.countForOrder(order.id), 1);
});
```

然后再修。

如果问题很难写自动化测试，也要保留复现脚本、日志、截图或命令输出。不要只让 AI 写“已修复”。

修复阶段的指令可以写：

示例 25-13　修复前最好有失败用例

```
现在只修已确认根因：duplicate paid cancellation lacks idempotency.

要求：
- 保留刚才新增的失败测试。
- 只修改 refund request 幂等逻辑。
- 不改 API response shape。
- 不改 payment callback。
- 跑 `npm test -- order-cancel`。
- 完成后停止。
```

这和“修一下测试失败”完全不同。

25.8　多层系统要定位坏在哪一层

真实项目常见的调试不是单函数问题，而是多层链路：

```
前端页面 -> API -> service -> repository -> database
```

或者：

```
CI workflow -> build script -> test command -> environment variables
```

这时不要在最深处猜。

要按边界收集证据：

```
API 收到的 payload 是什么？
service 传给 repository 的参数是什么？
repository 返回了什么？
数据库里实际记录是什么？
测试 fixture 写入了什么？
```

可以让 AI 先画出链路：

示例 25-14　多层系统要定位坏在哪一层

```
先不要修。
请画出这条失败路径的数据流：
- test input
- service call
- repository query
- database record
- returned object

标出每一层当前证据和缺失证据。
```

然后只补缺失证据。

不要让 AI 直接“检查一下相关代码”。先让它补齐缺失证据。

25.9　证据要分三层

调试时，证据可以分成三层。

第一层是症状证据：

```
哪个命令失败。
哪条断言失败。
实际值和期望值是什么。
第一条错误在哪里。
```

第二层是路径证据：

```
失败路径经过哪些函数、组件、接口、配置。
本次 diff 改了哪些路径上的文件。
哪些路径没有被改。
```

第三层是根因证据：

```
哪个输入、状态、数据或环境变量导致错误。
用什么最小验证证明它就是根因。
修复后同一证据如何变化。
```

很多 AI 调试只停在第一层：看到错误，补一个判断。系统化调试要求至少走到第二层，复杂问题要走到第三层。

可以让 Agent 按这个格式输出：

清单 25-15　证据要分三层

```
Evidence layers:

Symptom:
- `npm test -- order-cancel` fails on duplicate paid cancellation.
- Expected refund request count 1, received 2.

Path:
- test -> `cancelOrder` -> `RefundRequestService.create`
- changed files include cancel service and refund request service.

Root cause candidate:
- refund request creation lacks idempotency by order id.

Minimal verification:
- call cancel twice and inspect refund request count.
```

这比“可能是幂等问题”更可审。

25.10　Debug worksheet 可以固定下来

调试过程不要只留在聊天里。

可以在任务目录里放一个 `debug.md`：



图 25-2　Debug worksheet

模板如下：

清单 25-16　Debug worksheet 可以固定下来

```
# Debug Worksheet

## Reproduction
- Command:
- Frequency:
- Environment:
- Last known passing state:

## Error
- First failing line:
- Full stack:
- Actual:
- Expected:

## Evidence
- Recent diff:
- Relevant files:
- Logs / screenshots:
- Data flow:

## Boundary
- Works:
- Fails:
- Unknown:

## Hypothesis
- I think:
- Because:

## Minimal Verification
- Command or check:
- Expected result:

## Result
- Confirmed / rejected:
- New evidence:

## Fix
- Root cause:
- Changed files:
- Why this fixes root cause:

## Verification
- Focused:
- Broader:
- Residual risk:
```

这个文件不需要每次都完整填写。遇到复杂 bug、CI 失败、线上问题、间歇性测试失败时，它能减少排障时的遗漏。

25.11　调试时怎么和 AI 对话

可以按阶段给指令。

复现阶段：

```
先不要改代码。
请读取失败输出，整理 reproduction、first failing line、caller chain。
如果信息不足，只列出需要补充的命令。
```

证据阶段：

```
请使用 `rg` 查找相关调用点和最近修改文件。
输出 working path 和 failing path 的差异。
不要提出修复。
```

假设阶段：

```
基于当前证据，只提出一个最可能根因。
写出支持证据、反证可能、最小验证动作。
```

修复阶段：

```
只修已确认根因。
不做无关重构。
保留失败测试。
跑指定验证命令。
完成后停止。
```

回归阶段：

示例 25-17　调试时怎么和 AI 对话

```
请汇总：
- root cause
- changed files
- focused verification
- broader verification
- residual risk
```

这样做会让 AI 少猜很多。

25.12　什么时候要停下来回到设计

调试中经常会发现：问题不是一个 bug，而是设计假设错了。

比如订单取消失败，看起来是空指针，追下去发现根因是：

```
paid -> refund_requested 需要事务保证，
但当前实现先更新订单状态，再创建退款请求。
退款请求失败后，订单已经变成 refund_requested。
```

这不是加空值判断能解决的问题。它涉及状态更新和副作用顺序，应该回到 design。

出现下面情况，要停止调试并升级为设计问题：

- 修复需要改变事务边界。
- 修复需要改变 public API response shape。
- 修复需要新增状态、事件或数据库字段。
- 根因涉及多个入口共用规则。
- 当前 tests 无法表达正确行为，需要重写 requirement。
- 需要引入新依赖或新基础设施。

这时可以写：

清单 25-18 什么时候要停下来回到设计

```
Debug result:
- Symptom: refund request failure leaves order in wrong state.
- Root cause: state is persisted before side effect succeeds.
- This is a design boundary issue, not a local null-check bug.

Action:
- stop implementation.
- update design.md failure handling.
- update tasks.md with transaction / rollback behavior.
- add failing test before changing implementation.
```

AI 调试时最容易一路修下去。程序员要判断这是不是还在 bug 修复范围内，还是已经进入设计变更。

25.13 CI 失败不要直接本地乱改

CI 失败经常和本地不同。

常见差异包括：

- Node、Java、Python 版本不同。
- 环境变量缺失。
- 数据库、浏览器、系统依赖不同。
- 测试顺序不同。
- 时区、locale、文件路径不同。
- 缓存或依赖锁文件问题。

看到 CI 红，不要直接说：

```
修复 CI。
```

更好的输入：

示例 25-19 CI 失败不要直接本地乱改

```
先不要改代码。
请根据 CI 日志做排障：
- 哪个 job、step、command 失败。
- 本地是否有同样命令。
- CI 环境和本地环境差异。
- 最近 diff 是否影响该 step。
- 最小复现命令是什么。
- 一个最可能假设是什么。
```

如果本地无法复现，就不要假装已经验证。要写清：

```
Not locally reproduced.
Evidence from CI log:
- ...
Next verification:
- rerun failed job after config change
```

这比“本地测试通过，应该没问题”可靠得多。

25.14 间歇性失败先提高可观察性

间歇性失败最容易被 AI 随机修坏。

比如：

```
order-cancel E2E occasionally fails waiting for 退款处理中.
```

不要马上加超时。

先收集：

- 失败频率。
- 失败时网络请求是否成功。
- 前端状态是否更新。
- 后端日志里订单状态是什么。
- 测试数据是否唯一。
- 是否有并发测试污染。

可以让 Agent 先做诊断计划：

示例 25-20 间歇性失败先提高可观察性

```
先不要修改业务逻辑。
请为这个 flaky test 设计诊断步骤：
- 需要增加哪些临时日志或 trace。
- 需要重复运行哪个命令多少次。
- 如何区分前端等待问题、接口失败、数据污染和异步竞态。
- 哪些诊断改动最终必须删除。
```

Playwright 的 trace、截图、视频、网络日志都能帮助定位。后端测试可以加唯一 id、隔离数据库、固定时间和随机数。

只有确认是等待条件不准确，才考虑调整等待逻辑。没有证据时直接把 timeout 从 5 秒改 30 秒，通常只是把问题藏起来。

25.15 修复后要删除诊断噪音

调试过程中可能会加日志、临时断言、临时脚本。这些东西不一定都该进最终 diff。

修复完成后，让 Agent 自查：

示例 25-21 修复后要删除诊断噪音

```
请检查本次 diff 中是否还有临时诊断内容：
- console.log / print / debug output
- temporary comments
- skipped tests
- widened timeouts
- broad catch blocks
- diagnostic-only scripts

如果保留，请说明为什么它是生产需要，而不是调试残留。
```

调试残留很常见，尤其是 AI 修复多轮失败后。它可能留下过宽的 catch、过长的 timeout、无意义日志，表面上测试通过，代码质量下降。

真正的修复应该比调试过程干净。

25.16 修复说明要能被 reviewer 复查

调试完成后，不要只写“修复了空指针”。

可复查的修复说明应该包含：

清单 25-22 修复说明要能被 reviewer 复查

```
Root cause:
- `fixtures.createOrder` wrote `state`, but repository lookup expected `status`.

Fix:
- align fixture field with production schema.
- keep repository query unchanged.

Why this is root cause:
- before fix, created test record lacked `status`.
- after fix, `orders.find(order.id)` returns the paid order.

Verification:
- `npm test -- order-cancel -- --test-name-pattern paid`: PASS
- `npm test -- order-cancel`: PASS

Not changed:
- cancel service null handling
- API response shape
- payment callback flow
```

这段说明能让 reviewer 复查：修的是根因，不是把症状掩盖掉。

如果修复说明写不出“为什么这就是根因”，说明调试还没完成。

25.17 回归验证不能只跑失败用例

失败用例通过，只能证明当前症状消失。

修复后至少要跑两层：

```
focused reproduction:
- 证明原失败用例通过。

neighbor regression:
- 证明相邻旧行为没坏。
```

订单取消例子里，修 duplicate paid cancellation 后，不能只跑 duplicate 测试。还要跑 pending、paid、shipped、refund failure 这些相邻路径。

可以要求 Agent：

示例 25-23 回归验证不能只跑失败用例

```
修复后请运行：
1．原失败用例。
2．同文件全部测试。
```

```
3．受影响模块测试。

分别说明每个命令证明什么。
```

如果只跑原失败用例，很可能把一个 bug 修成另一个 bug。

25.18　一个 CI 失败案例：本地绿，CI 红

假设本地验证通过：

```
npm test -- order-cancel
PASS
```

但 CI 失败：

```
FAIL test/order/order-cancel.test.ts
duplicate paid cancellation creates one refund request
Expected: 1
Received: 2
```

不要让 AI 直接“修 CI”。先让它整理差异：

清单 25-24　一个 CI 失败案例：本地绿，CI 红

```
CI evidence:
- job: test-linux
- command: `npm test -- order-cancel`
- failing test: duplicate paid cancellation creates one refund request
- actual: 2 refund requests
- expected: 1 refund request

Local evidence:
- same command passed locally.

Diff surface:
- `src/order/order-cancel-service.ts`
- `src/refund/refund-request-service.ts`
- no lockfile change
- no CI config change

Environment difference:
- CI runs tests in parallel.
- local run uses default serial mode.
```

这时一个合理假设是：幂等逻辑在并发情况下不成立。

最小验证不是改代码，而是先复现并发：

```
Minimal verification:
- run duplicate cancellation test with parallel workers or concurrent requests.
- inspect whether idempotency key has unique constraint.
- inspect refund request creation transaction.
```

修复可能需要数据库唯一约束、事务或 upsert。这个结论已经超出“本地绿 CI 红”的表面现象。

如果直接加 sleep、延长 timeout、重试测试，可能让 CI 暂时绿，但生产里的重复退款问题仍然存在。

这个案例说明：CI 失败不是麻烦，它经常暴露本地没覆盖的执行条件。让 AI 基于 CI 证据排障，才能避免把真实并发问题当成环境问题。

25.19 给 Agent 的排障指令可以固定成四段

复杂调试时，直接贴错误让 AI 发挥，通常会得到一堆猜测。可以把指令固定成四段。

第一段，先锁住只读：

```
先不要修改代码。
你现在只做排障，不做修复。
```

第二段，要求复现和证据：

示例 25-25 给 Agent 的排障指令可以固定成四段

```
请整理：
- failing command
- first failing line
- actual vs expected
- stack / caller chain
- changed files that may affect this failure
```

第三段，要求单一假设：

```
只提出一个最可能根因。
必须写支持证据和反证可能。
不要列 5 个可能原因后直接修。
```

第四段，要求最小验证：

```
给出一个最小验证动作。
验证动作不能同时修改业务逻辑。
验证完成前不要进入修复。
```

完整指令可以这样写：

示例 25-26 给 Agent 的排障指令可以固定成四段

```
先不要修改代码。

请对下面失败做系统化排障：
1. 记录 failing command、first failing line、actual vs expected。
2. 沿调用链找出值从哪里来。
3. 对照 git diff，判断失败路径是否经过本次修改。
4. 只提出一个最可能根因。
5. 给出最小验证动作。

输出排障报告，不要修复。
```

这段指令的重点不是格式，而是阻止 AI 从错误直接跳到补丁。

25.20 排障报告的最低标准

一次排障报告至少要包含：

清单 25-27 排障报告的最低标准

```
## Reproduction
- command:
- frequency:
- environment:

## Failure
- failing test / scenario:
- actual:
- expected:
- first failing line:

## Path
- caller chain:
- data source:
- changed files on path:

## Hypothesis
- root cause candidate:
- supporting evidence:
- possible counter-evidence:

## Minimal verification
- command / check:
- expected result:

## Stop condition
- what would make this a design issue:
```

如果 Agent 输出里没有 command、actual/expected、caller chain，只给修复建议，直接退回。

调试工作不是靠“看起来懂了”推进，而是靠每一步证据能被复查。

25.21 不要把防御性代码当成修复

AI 调试时最常见的“假修复”是加防御性代码。

比如：

清单 25-28 不要把防御性代码当成修复

```
if (!order) {
  return null;
}
```

或者：

清单 25-29 不要把防御性代码当成修复

```
if (!order?.status) {
  return { status: 'unknown' };
}
```

这类代码可能让报错消失，但它没有解释为什么订单不存在，也没有证明业务应该返回 `null` 或 `unknown`。

防御性代码只有在满足下面条件时才合理：

- 这个输入在生产中确实可能为空。
- requirements 或接口合同定义了空值行为。
- 有测试覆盖空值路径。
- 不会掩盖上游数据错误。

否则它只是把异常藏起来。

可以要求 Agent：

```
如果你建议加 guard clause，请先证明：
- 空值是合法业务输入，还是上游 bug。
- 合法时应该返回什么。
- 哪个测试覆盖这个分支。
- 为什么不是修上游数据来源。
```

这条规则能挡住大量“看起来稳妥”的错误修复。

25.22 线上问题要保留可复现材料

线上问题不一定能在本地完全复现，但也不能只靠文字描述。

至少保留：

示例 25-30 线上问题要保留可复现材料

```
request id / trace id
时间范围
用户角色或租户
关键日志
输入 payload
实际响应
相关数据快照
当前版本或 commit
```

给 Agent 时，先脱敏，再让它基于这些材料找路径。

```
先不要修改代码。
基于以下脱敏 trace，找出请求经过的代码路径和可能的状态来源。
不要根据业务常识猜测缺失字段。
```

线上排障尤其不能让 AI 直接改。它缺少生产上下文，必须先把 trace、日志、数据和代码路径对齐。

25.23 调试后的 PR 描述要写根因，不写过程流水

PR 里不用贴完整 debug worksheet，但要留下能审的摘要。

清单 25-31 调试后的 PR 描述要写根因，不写过程流水

```
Root cause:
- Duplicate paid cancellation created two refund requests because refund creation was not guarded
by an idempotency key.

Fix:
- Use `order:<orderId>:cancel` as idempotency key.
- Reuse existing refund request when the key already exists.

Verification:
- `npm test -- order-cancel`: PASS
- duplicate paid cancellation test added
- refund failure path still keeps order paid

Not changed:
- payment provider callback
- public API response shape
```

这样的摘要比“修复重复取消问题”更有价值。它让 reviewer 能判断修复是否对着根因，测试是否覆盖风险，边界是否保持。

25.24 调试完成后的上线前检查

调试修复进入交付前，再做一遍短检查：

示例 25-32 调试完成后的上线前检查

```
Root cause:
- 是否一句话说得清。

Patch:
- 是否只改根因相关文件。
- 是否没有临时日志、skip、only、扩大 timeout。

Regression:
- 原失败用例是否通过。
- 相邻路径是否通过。
- broader command 是否跑过或记录了原因。

Risk:
- 是否改变接口、数据、权限、副作用。
- 是否有未验证外部依赖。
```

这段检查适合放在 PR 自查里。

AI 可以协助生成自查报告，但你要用 diff 和命令输出核对。尤其要看它有没有为了让测试通过而保留不该保留的诊断代码。

如果调试修复很小，也至少保留：

```
root cause + changed files + verification command
```

没有这三项，后续再出问题时很难判断当时修了什么。

还有一个简单判断：如果你删掉修复代码，原失败用例不会重新失败，那这个用例就没有锁住根因。此时要补回归测试，或者至少保留可复现脚本。否则这次调试只是一次临时处理，不能算真正闭环。

如果修复依赖外部条件，例如特定时区、特定浏览器、特定 CI runner，也要把条件写进记录。否则后续换环境后，同一个问题会被重新误判。

还有一种检查很有用：反证检查。

修复后可以让 Agent 回答：

```
如果这个根因判断是错的，哪些现象应该不会出现？
当前证据里有没有反例？
有没有其它假设也能解释同一组现象？
我们是否验证过这些假设？
```

这不是让流程变学术，而是防止“第一个看起来合理的解释”变成根因。线上问题、CI 偶发失败、异步竞态、缓存问题，尤其需要这一步。

反证检查还可以帮助你判断是否应该继续让 Agent 修改代码。如果仍有多个假设能解释同一现象，下一步应该是补证据，而不是补补丁。

25.25 常见错误

错误一：看到报错就补空值判断。

空值判断可能需要，但先证明为什么为空。

错误二：一次改多个可能原因。

同时改 fixture、service、repository，最后不知道哪个修好了问题。

错误三：没有复现命令。

没有复现，就没有可靠验证。

错误四：只读最后一行错误。

堆栈、上游调用、最近 diff 才能定位根因。

错误五：调试过程只留在聊天记录。

复杂问题要沉淀 debug worksheet，否则后面很难复盘。

错误六：CI 失败按本地经验猜。

先看 CI job、step、command、环境差异和日志。

25.26 改完后重点看什么

一次调试完成后，看这些结果：

- 是否有稳定复现或明确说明无法稳定复现。
- 是否定位到 first failing line 和上游来源。
- 是否提出并验证过单一假设。
- 修复是否针对根因，而不是只压症状。
- 是否有失败用例或等价复现证据。
- focused 和 broader verification 是否都跑过。
- 是否记录了 residual risk。

如果这些都没有，只是 AI 改完后测试绿了，你还不能确信它修对了。

调试的目标不是让错误消失，而是让错误为什么出现、为什么被修复、为什么不会回归，都有证据。

25.27 常见问题

简单 bug 也要写 debug worksheet 吗?

不需要。简单、稳定复现、根因明显的问题，可以直接记录命令和修复。但只要开始猜，或者第二次修还没好，就应该写下来。

AI 已经知道怎么修，还要先排查吗？

让它先说证据。能用代码、日志、堆栈证明的“知道”，才值得相信。

没有测试怎么验证修复？

先争取写最小复现测试。写不了就保留脚本、日志、截图、手工步骤和预期结果。

调试阶段可以加日志吗？

可以，但要区分诊断日志和最终代码。临时日志用于定位，修复后要删除或转成有意义的结构化日志。

多个假设都很可能怎么办？

按最小验证成本排序，一次验证一个。不要把多个修复混在一个 diff。

25.28 可复查资料

- Playwright debugging：https://playwright.dev/docs/debug〔25-1〕
- Node.js debugger：https://nodejs.org/api/debugger.html〔25-2〕
- GitHub Actions workflow logs：https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/using-workflow-run-logs〔25-3〕
- OpenAI Codex best practices：https://developers.openai.com/codex/learn/best-practices〔25-4〕

本章资源

〔25-1〕 https://playwright.dev/docs/debug
https://playwright.dev/docs/debug

〔25-2〕 https://nodejs.org/api/debugger.html
https://nodejs.org/api/debugger.html

〔25-3〕 https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/using-workflow-run-logs
https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/using-workflow-run-logs

〔25-4〕 https://developers.openai.com/codex/learn/best-practices
https://developers.openai.com/codex/learn/best-practices

第26章　前后端验证闭环：页面、接口、数据库和副作用怎么一起验

AI 做完一个前后端改动后，最常见的危险说法是：

```
我已经测试过了，页面正常。
```

或者：

```
接口测试通过。
```

这两句话都可能是真的，但都不够。

页面正常，只能证明用户眼前看到的东西在某个路径下符合预期。它不能证明接口响应结构没变、数据库状态正确、副作用只发生一次。

接口测试通过，也不能证明页面交互真的可用、前端状态会刷新、错误提示正确、按钮不会重复提交。

真实产品里的一个功能，通常跨四层：

```
UI 行为 -> HTTP 合同 -> 业务状态 -> 外部副作用
```

AI Coding 的验证闭环，就是让这四类证据能互相对上。

前后端验证闭环

页面可见、接口正确、状态落地、副作用发生，四类证据要能互相对上。

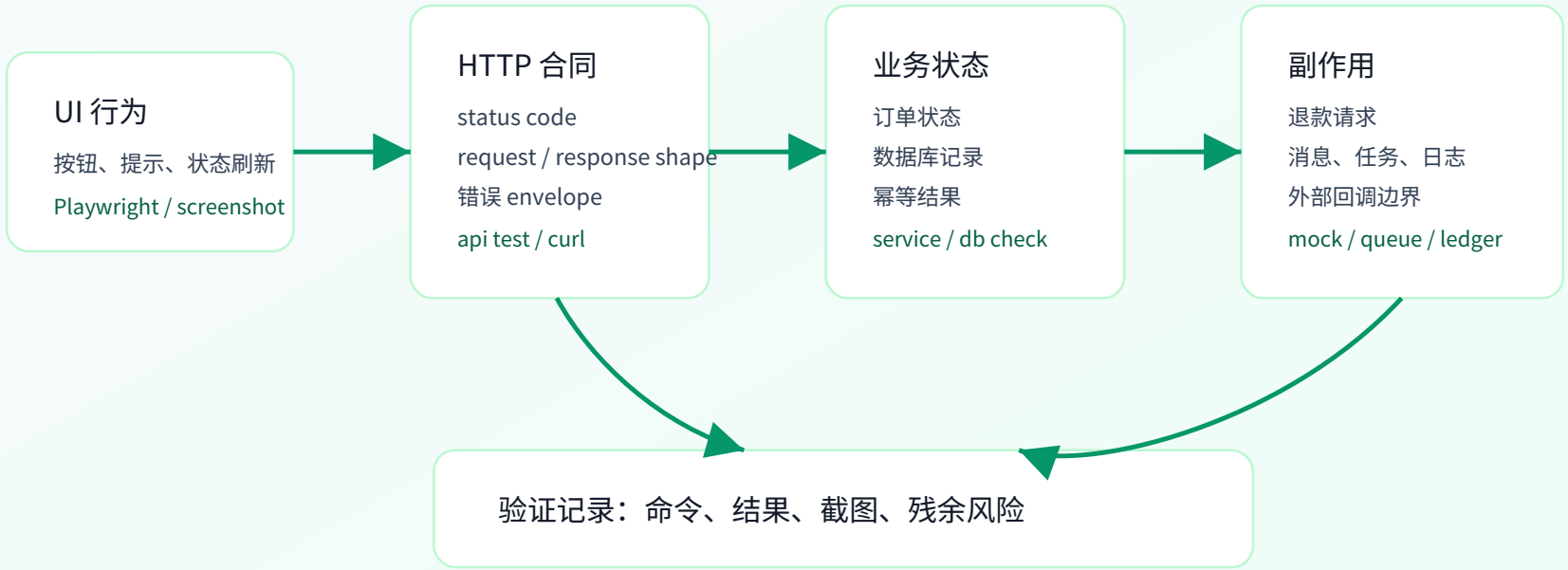


图 26-1　前后端验证链路

26.1　为什么单点验证不够

继续用订单取消进入退款流程这个例子。

需求是：

> 用户取消已支付订单后，页面显示“退款处理中”，后端订单状态变成 `refund_requested`，并且只创建一个退款请求。

如果只做页面验证：

> 点击取消后，页面显示“退款处理中”。

这只能证明前端最终显示了这个文案。

它不能证明：

- 后端状态是不是 `refund_requested`。
- 是否创建了退款请求。
- 重复点击是否创建多条退款请求。
- API response shape 是否兼容。
- 失败时页面是否提示正确。

如果只做接口验证：

清单 26-1　为什么单点验证不够

```
curl -X POST /api/orders/123/cancel
```

看到 200，也不能证明：

- 页面是否调用了正确接口。
- UI 是否禁用了重复点击。
- 错误状态是否显示。
- 用户刷新页面后状态是否一致。

所以前后端功能不能用单点证据收尾。

单点验证会留下盲区

截图、接口、数据库、队列各自只能证明一部分事实。

只做这个	能证明	不能证明
页面截图	用户看到了正确状态	接口合同、持久化、副作用是否正确
接口测试	HTTP 状态和响应结构	页面交互、前端状态、真实用户路径
数据库检查	状态是否落地	响应是否兼容，UI 是否刷新
队列/日志	副作用是否发出	业务状态是否一致，用户是否可见

图 26-2　单点验证盲区

26.2　先定义这次要证明什么

验证闭环不是测试越多越好。

先把需求拆成可证明事实：

示例 26-2　先定义这次要证明什么

```
UI:
- paid order detail page shows cancel action.
- after cancel succeeds, page shows refund_requested / 退款处理中.
- duplicate click is prevented or handled.

HTTP:
- POST /api/orders/:id/cancel returns expected status code.
- response shape remains compatible.
- failure uses existing error envelope.

Business state:
- order status becomes refund_requested.
- pending and shipped behavior remains unchanged.
- failed refund request leaves order paid.

Side effects:
- exactly one refund request is created.
- duplicate cancellation does not create another request.
- payment provider callback is not changed in this task.
```

这一步比直接写 Playwright 更重要。

因为你要先知道“验证什么”，再决定用什么工具。

26.3　页面验证：证明用户路径真的可走

页面验证适合证明：

- 用户能看到入口。
- 用户能完成交互。
- 页面状态会刷新。
- 加载、成功、失败提示符合预期。
- 重复点击、禁用状态、错误提示正确。

如果使用 Playwright，可以让 AI 生成测试草稿，但要给清楚路径：

示例 26-3　页面验证：证明用户路径真的可走

```
请写前端 E2E 测试草稿，覆盖 paid order cancellation 用户路径。

要求：
- 从订单详情页进入。
- 点击取消。
- 等待 API 成功响应后页面显示“退款处理中”。
- 验证取消按钮不再可重复触发。
- 不 mock 掉页面本身的状态更新逻辑。
- 不验证数据库，数据库由后端测试覆盖。
```

Playwright 测试可以类似：

清单 26-4　页面验证：证明用户路径真的可走

```
test('paid order cancellation shows refund requested state', async ({ page }) => {
  await page.goto('/orders/paid-order-1');

  await page.getByRole('button', { name: '取消订单' }).click();

  await expect(page.getByText('退款处理中')).toBeVisible();
  await expect(page.getByRole('button', { name: '取消订单' })).toBeHidden();
});
```

这段测试证明用户路径，不证明后端副作用。

不要让 AI 把所有验证都塞进 E2E。E2E 成本高、失败原因复杂，适合覆盖关键路径，不适合承担所有业务状态断言。

26.4　接口验证：证明合同没有被破坏

接口验证适合证明：

- status code。
- response body shape。
- 错误 envelope。
- 鉴权和权限边界。
- 幂等请求的响应。

例如：

清单 26-5　接口验证：证明合同没有被破坏

```
curl -i -X POST http://localhost:3000/api/orders/123/cancel \
  -H 'Content-Type: application/json'
```

要看：

```
HTTP/1.1 200 OK
```

也要看 response body：

清单 26-6　接口验证：证明合同没有被破坏

```
{
  "id": "123",
  "status": "refund_requested",
  "message": "Refund request created"
}
```

如果项目有 API test，可以写：

清单 26-7　接口验证：证明合同没有被破坏

```
test('cancel paid order returns refund_requested without response shape change', async
({ request }) => {
  const response = await request.post('/api/orders/paid-order-1/cancel');
  expect(response.status()).toBe(200);

  const body = await response.json();
  expect(body).toMatchObject({
    id: 'paid-order-1',
    status: 'refund_requested'
  });
  expect(Object.keys(body).sort()).toEqual(['id', 'message', 'status'].sort());
});
```

这里的重点是 response shape。AI 很容易为了实现需求顺手改返回结构，前端当前页面可能没坏，但其他调用方会坏。

接口验证不能被页面截图替代。

26.5　业务状态：证明数据真的落地

业务状态验证适合证明：

- 数据库记录是否正确。
- 状态转换是否符合规则。
- 事务失败是否回滚。
- 旧状态行为是否保持。

后端 focused test 可以写：

清单 26-8　业务状态：证明数据真的落地

```
test('paid order cancellation persists refund_requested status', async () => {
  const order = await fixtures.createOrder({ status: 'paid' });

  await cancelOrder(order.id);

  const saved = await orders.find(order.id);
  expect(saved.status).toBe('refund_requested');
});
```

失败路径也要有：

清单 26-9　业务状态：证明数据真的落地

```
test('order remains paid when refund request creation fails', async () => {
  const order = await fixtures.createOrder({ status: 'paid' });
  refundRequests.failNextCreate();

  await expect(cancelOrder(order.id)).rejects.toThrow();

  const saved = await orders.find(order.id);
  expect(saved.status).toBe('paid');
});
```

如果没有数据库测试条件，也要保留人工或脚本化检查：

清单 26-10　业务状态：证明数据真的落地

```
select id, status from orders where id = '123';
```

不要只看接口返回。接口返回可能是内存对象，数据库写入可能失败。

26.6　副作用：证明外部动作没有多发或漏发

退款、消息、邮件、队列、外部 API 调用，都属于副作用。

副作用验证最容易被忽略，因为页面和接口可能都看起来正常。

订单取消场景里，至少要证明：

```
paid cancel creates exactly one refund request.
duplicate cancel does not create another refund request.
refund request failure does not leave order in wrong state.
payment provider callback remains out of scope.
```

测试可以是：

清单 26-11　副作用：证明外部动作没有多发或漏发

```
test('duplicate paid cancellation creates one refund request', async () => {
  const order = await fixtures.createOrder({ status: 'paid' });

  await cancelOrder(order.id);
  await cancelOrder(order.id);

  expect(await refundRequests.countForOrder(order.id)).toBe(1);
});
```

如果副作用通过队列发出，要检查队列消息：

```
Queue evidence:
- topic: refund.requested
- key: order:123:cancel
- count: 1
```

如果外部服务不能在本地调用，要明确边界：

示例 26-12　副作用：证明外部动作没有多发或漏发

```
Not locally verified:
- real payment provider callback.

Verified instead:
- refund request record is created.
- provider callback code path was not changed.
- staging callback verification remains required before release.
```

这比假装“已完整验证”更专业。

26.7　测试数据也要进入验证闭环

前后端验证经常漏掉测试数据。

页面显示对了、接口返回对了，但如果测试数据不是目标状态，验证仍然不可信。

订单取消场景里，要明确：

示例 26-13　测试数据也要进入验证闭环

```
Test data:
- order id: paid-order-1
- order status before cancel: paid
- payment status: captured
- shipping status: not_shipped
- existing refund request: none
```

如果验证 duplicate cancel，还要写：

```
Before second cancel:
- order status: refund_requested
- refund request count: 1
```

这能防止 AI 用一个名字叫 `paid-order-1`、实际状态不是 paid 的 fixture 去跑测试。

对前端 E2E，测试数据准备方式也要写清：

- 通过 seed 脚本创建。
- 通过 API 创建。
- 使用固定 fixture。
- 使用 mock server。

不同方式证明的东西不同。mock server 证明前端响应某个合同；seed + real API 更接近真实链路。

26.8　验证 ledger 要能让别人复查

前后端验证完成后，可以把结果写成 ledger：

清单 26-14　验证 ledger 要能让别人复查

````
# Verification Ledger

## Test Data

- paid order: `paid-order-1`
- initial status: `paid`
- existing refund request count: 0

## Commands

```bash
npm test -- order-cancel
npm test -- order-api
npx playwright test order-cancel.spec.ts
```

## Evidence

- backend focused test: PASS, covers status and idempotency
- API contract test: PASS, response keys remain `id`, `status`, `message`
- Playwright path: PASS, page displays 退款处理中 after cancel

## Not Verified

- real payment provider callback
- old mobile client display
````

```
## Residual Risk

- staging check required for provider callback signature
```

ledger 不需要替代测试报告。它的价值是把多层证据串起来，让 reviewer 知道每个命令证明了什么。

如果没有 ledger，AI 很容易把“跑了几个命令”写成“验证完成”，但别人无法复查边界。

26.9　验证顺序：先内后外，再串起来

前后端验证建议按这个顺序：

```
1. 后端 focused test
2. API contract test
3. 前端组件或页面路径
4. E2E 关键路径
5. 副作用和残余风险记录
```

为什么不先跑 E2E？

E2E 失败时原因太多：选择器、网络、鉴权、后端状态、异步加载、数据准备都可能导致失败。先用后端和接口测试确认核心行为，再用 E2E 验用户路径，定位成本更低。

但最后不能只停在后端测试。用户路径也要验证。

比较稳的执行指令：

示例 26-15　验证顺序：先内后外，再串起来

```
请按下面顺序验证，不要跳步：

1. 运行后端 focused test：`npm test -- order-cancel`
2. 运行 API contract test：`npm test -- order-api`
3. 运行前端关键路径：`npx playwright test order-cancel.spec.ts`
4. 汇总每个命令证明了什么，以及仍然没有证明什么。
```

AI 如果只跑最容易通过的命令，就要求它补齐缺口。

26.10　让 AI 输出验证矩阵

完成后不要只让 AI 写：

```
测试已通过。
```

要求它输出验证矩阵：

清单 26-16　让 AI 输出验证矩阵

```
| Evidence | Command / Check | Proves | Does not prove |
|---|---|---|---|
| Backend focused test | `npm test -- order-cancel` | status transition, idempotency, failure rollback | UI path |
| API contract | `npm test -- order-api` | status code, response shape | browser rendering |
| E2E path | `npx playwright test order-cancel.spec.ts` | user can cancel and see refund state | real payment callback |
| Queue check | `refund.requested count = 1` | side effect emitted once | external provider success |
```

这个矩阵很有用。

它迫使 AI 说清每个证据的边界，避免把“某个测试通过”夸成“功能完全没问题”。

26.11　错误路径也要验证

前后端功能最容易漏错误路径。

至少要看：

- 后端返回 400/409/500 时页面怎么显示。
- 重复点击是否被禁止或幂等处理。
- 权限不足是否进入正确提示。
- 网络失败是否保留可重试状态。
- 后端部分成功时前端是否会显示错误状态。

例如：

清单 26-17　错误路径也要验证

```
test('cancel failure keeps user on paid state with error message', async ({ page }) => {
  await page.route('**/api/orders/*/cancel', route => {
    route.fulfill({
      status: 500,
      contentType: 'application/json',
      body: JSON.stringify({ error: { message: 'Refund service unavailable' } })
    });
  });

  await page.goto('/orders/paid-order-1');
  await page.getByRole('button', { name: '取消订单' }).click();

  await expect(page.getByText('Refund service unavailable')).toBeVisible();
  await expect(page.getByText('已支付')).toBeVisible();
});
```

这段测试证明的是前端错误状态，不证明数据库状态。数据库状态仍然要后端测试覆盖。

26.12　并发和重复提交要单独验证

前后端链路里，重复提交是高发问题。

用户可能双击按钮，浏览器可能重试请求，移动端可能在弱网下重复发送。页面禁用按钮有价值，但不能替代后端幂等。

订单取消场景至少要验证两层：

示例 26-18　并发和重复提交要单独验证

```
UI:
- first click disables or hides cancel action.
- repeated click does not send a second request, or handles idempotent response.

Backend:
- repeated cancel request creates at most one refund request.
- repeated request returns compatible response.
```

前端测试可以证明按钮行为：

清单 26-19　并发和重复提交要单独验证

```
await page.getByRole('button', { name: '取消订单' }).dblclick();
await expect(page.getByText('退款处理中')).toBeVisible();
```

后端测试证明副作用次数：

清单 26-20　并发和重复提交要单独验证

```
await cancelOrder(order.id);
await cancelOrder(order.id);
expect(await refundRequests.countForOrder(order.id)).toBe(1);
```

如果只验证 UI，不验证后端，脚本或移动端仍然可能绕过。 如果只验证后端，不验证 UI，用户可能看到重复加载、错误提示或状态抖动。

AI 很容易把“按钮禁用”当成完整幂等方案。验证闭环要把这两层分开。

26.13　权限和角色不要漏

前后端功能还要验证谁能做这件事。

同一个“取消订单”动作，可能有：

- 买家入口。
- 客服后台入口。
- 商家后台入口。

- 定时任务入口。
- 开放 API 入口。

如果需求只覆盖买家入口，就要写清：

示例 26-21　权限和角色不要漏

```
Verified:
- buyer paid order cancellation.

Not changed:
- support admin cancellation.
- merchant cancellation.
- scheduled cleanup job.
```

如果权限会变，至少要有 API 层测试：

清单 26-22　权限和角色不要漏

```
test('buyer cannot cancel another buyer order', async () => {
  const response = await requestAs(otherBuyer).post('/api/orders/paid-order-1/cancel');
  expect(response.status()).toBe(403);
});
```

页面看不到按钮，不代表接口不能被调用。权限类需求必须在后端验证。

26.14　不要让 mock 掩盖真实问题

前端测试经常 mock API。

mock 有价值，能稳定验证前端状态。但 mock 也可能掩盖真实合同问题。

如果前端测试 mock 了 response：

清单 26-23　不要让 mock 掩盖真实问题

```
{ "status": "refund_requested" }
```

而真实后端返回：

清单 26-24　不要让 mock 掩盖真实问题

```
{ "orderStatus": "refund_requested" }
```

前端 E2E mock 测试会过，真实联调会挂。

所以关键路径至少要有一个不 mock 合同的验证：

```
API contract test verifies real response shape.
Front-end route test verifies UI state.
One integration or E2E path verifies the two meet.
```

不要让 AI 为了测试稳定，把真实合同全部 mock 掉。

26.15　验证记录要包含残余风险

真实项目里，不是每件事都能本地验证。

比如：

- 真正的支付 provider。
- 第三方短信或邮件。
- 生产级队列重试。
- 大数据量下的性能。
- 某些权限组合。

不能验证就要写出来：

清单 26-25　验证记录要包含残余风险

```
## Residual Risk

- Real payment provider callback was not executed locally.
- Staging verification is required for callback signature and provider state transition.
- Queue retry policy was not changed in this task.
```

这不是示弱。它是在防止 AI 把未验证内容说成已验证。

26.16　验证环境也要写清

验证结果离不开环境。

同一个命令，在本地、CI、staging 的意义不同。

ledger 里可以写：

清单 26-26　验证环境也要写清

```
## Environment

- local node: 20.x
- database: local test container
```

```
- browser: Chromium via Playwright
- API base: local dev server
- payment provider: fake adapter
```

这样 reviewer 才知道结果的边界。

如果用 mock server，要写清 mock 覆盖了什么；如果用真实 dev server，要写清测试数据怎么创建和清理。

AI 常会省略环境，直接写“测试通过”。生产验证不能这么写。

26.17　一个完整验证计划长什么样

把订单取消改退款作为例子，一份可执行验证计划可以这样写：

清单 26-27　一个完整验证计划长什么样

```
Verification Plan: paid order cancellation

## Test Data

- paid order: `paid-order-1`
- pending order: `pending-order-1`
- shipped order: `shipped-order-1`
- paid order has captured payment and no refund request before test

## Backend

- `npm test -- order-cancel`
  - paid -> refund_requested
  - pending -> canceled unchanged
  - shipped -> existing rejection unchanged
  - duplicate paid cancel creates one refund request
  - refund request failure keeps order paid

## API

- `npm test -- order-api`
  - status code remains 200 for accepted cancel
  - response shape remains `id`, `status`, `message`
  - error envelope remains compatible

## Frontend

- `npx playwright test order-cancel.spec.ts`
  - user can cancel paid order from detail page
  - page shows 退款处理中
  - cancel action no longer repeats
  - failure response keeps paid state and shows error

## Side Effects
```

```
- refund request count for order is 1
- idempotency key is stable
- payment provider callback path not touched

## Residual Risk

- real provider callback requires staging verification
- old mobile client display not covered in this web test
```

这份计划不追求“所有测试都写在一个地方”。它把证据分层，每层证明自己的事情。

如果 Agent 完成后只报告 `npm test -- order-cancel` 通过，就能看出缺口：API、页面、副作用和残余风险都还没闭合。

26.18　给 Agent 的验证指令不要只写“跑测试”

更好的指令是让它按证据层输出。

示例 26-28　给 Agent 的验证指令不要只写“跑测试”

```
请按验证计划执行，不要自行缩减命令。

输出必须包含：
1. Test data：使用了哪些订单、初始状态是什么。
2. Commands：实际执行的命令。
3. Evidence：每个命令证明什么。
4. Not verified：哪些路径没有验证。
5. Residual risk：未验证路径是否阻塞发布。
```

如果某个命令跑不了，也不能跳过：

```
如果命令无法执行：
- 写出失败原因。
- 说明它原本要证明什么。
- 给出替代验证。
- 标记残余风险。
```

比如 Playwright 跑不了，不能直接写“前端未测”。要说明是浏览器依赖缺失、dev server 未启动、登录态不可用，还是测试本身不存在。不同原因对应的处理完全不同。

26.19　验证闭环要防“只验证改动路径”

AI 经常只验证新增路径：paid order cancel。旧路径 pending、shipped、权限、失败回滚容易被漏掉。

可以让它输出覆盖表：

清单 26-29　验证闭环要防“只验证改动路径”

```
| Scenario | Verified | Evidence |
| --- | --- | --- |
| pending cancel unchanged | yes | `npm test -- order-cancel` |
| paid cancel refund_requested | yes | `npm test -- order-cancel` |
| shipped cancel rejected | yes | `npm test -- order-cancel` |
| duplicate paid cancel idempotent | yes | `npm test -- order-cancel` |
| refund failure keeps paid | yes | `npm test -- order-cancel` |
| page shows refund state | yes | Playwright |
| old mobile client display | no | residual risk |
```

这张表能一眼看出是否只测了 happy path。

验证不是为了证明 AI 做对了，而是为了找出它可能没做对的地方。

26.20　验证命令要和风险一一对应

验证计划里每个风险都应该有对应证据。

清单 26-30　验证命令要和风险一一对应

```
| Risk | Evidence |
| --- | --- |
| paid order status not persisted | backend integration test |
| response shape changed | API contract test |
| duplicate refund request | idempotency test |
| page does not refresh after cancel | Playwright path |
| old pending cancel broken | regression test |
| payment callback accidentally changed | diff review + callback tests unchanged |
```

如果某个风险没有证据，就不要把它从风险列表里删掉。要么补验证，要么保留 residual risk。

AI 很容易只跑测试，不解释测试覆盖了哪个风险。风险到证据的映射，是前后端验证闭环的核心。

26.21　手工验证也要可复查

有些路径短期没有自动化测试，只能手工验证。手工验证不是随便点一点。

记录格式可以是：

清单 26-31　手工验证也要可复查

```
Manual check:
- environment: staging
- account: buyer test account
- order: paid order created by seed script
- steps:
  1. open order detail
  2. click cancel
```

```
  3. confirm modal
  4. refresh page
- expected:
  - page shows 退款处理中
  - cancel button hidden
  - order API returns refund_requested
- evidence:
  - screenshot path or attached image
  - request id / log id
```

手工验证不能替代所有自动化测试，但它可以作为外部服务、真实浏览器、权限组合的补充证据。

不要让 AI 把“我手工看了”写成验证结论。手工验证也要有步骤和结果。

26.22　上线前把证据压成一句话

验证材料很多，最后要能压成一句 reviewer 能理解的话。

例如：

```
Paid order cancellation was verified at backend, API, web UI and side-effect layers:
focused order tests passed, API response shape stayed compatible, Playwright covered the cancel path,
refund request count stayed 1 under duplicate cancel, real provider callback remains staging-only residual risk.
```

中文 PR 里可以写：

```
已验证后端状态、接口合同、Web 页面路径和退款请求副作用：
订单 focused tests 通过，API response shape 未变，Playwright 覆盖取消路径，
重复取消只创建一条退款请求；真实支付回调未在本地验证，需 staging 复核。
```

这句话不是替代详细 ledger，而是给 reviewer 一个入口。

如果你压不出这句话，说明验证证据还没有串起来。

26.23　数据清理和可重复执行也要检查

前后端验证如果依赖固定数据，要确认测试能重复跑。

常见问题：

- 第一次测试创建 refund request，第二次测试因为数据残留失败。
- E2E 用固定账号，状态被上一次测试改掉。
- 本地数据库没有清理，CI 每次干净环境，结果不同。
- 并行测试使用同一个 order id。

验证计划里可以写：

```
Data isolation:
- each test creates its own order id.
- refund request table is cleaned between tests.
- E2E seed runs before each test.
- no test depends on previous test order.
```

这类细节很工程化，但生产里非常常见。AI 如果只写一个能跑一次的测试，后面会变成 flaky test。

26.24　验证失败时不要继续补页面

前后端任务里，常见错误是后端验证失败时继续改页面。

比如 API contract 已经发现 response shape 变化，Agent 仍然去改前端适配新 shape。这样页面可能恢复正常，但合同已经被破坏。

规则应该是：

```
如果后端 focused 或 API contract 验证失败，先停止。
不要通过修改前端来掩盖后端合同变化。
```

页面适配只有在合同变化被 requirements/design 明确接受后才做。否则就是让调用方迁就一次未审的后端改动。

同理，如果页面验证失败，也不要先改后端去迎合页面。先确认 API 合同和业务状态是否正确，再判断页面是状态映射问题、缓存问题、异步刷新问题，还是后端确实返回错了。

验证闭环不是“哪层红就改哪层”，而是先定位红灯属于哪一层。

定位清楚后，再按层修复：合同错修后端，展示错修前端，数据错修 seed 或业务状态，副作用错修服务边界。不要让一个层的补丁掩盖另一个层的问题。

如果时间紧，也要保留验证顺序。

可以少跑一些命令，但不要颠倒层次：

```
先确认合同。
再确认状态。
再确认页面。
最后确认副作用和残余风险。
```

否则页面补丁可能掩盖接口破坏，mock 测试可能掩盖真实合同变化，副作用遗漏可能在上线后才暴露。

26.25　常见错误

错误一：只看页面截图。

截图不能证明接口合同、数据库状态和副作用。

错误二：只跑接口测试。

接口测试不能证明用户路径、前端状态刷新和错误提示。

错误三：mock 掉所有后端。

mock 能稳定前端测试，但不能替代真实合同验证。

错误四：只测成功路径。

失败、重复点击、权限不足、网络异常是前后端问题高发区。

错误五：不写残余风险。

没验证外部 provider，就不要写“端到端验证完成”。

错误六：让 AI 自己挑验证命令。

它可能选最容易过的命令。验证命令应该来自 tasks、项目规则或人工授权。

26.26　改完后重点看什么

一次前后端改动结束后，看这些结果：

- 页面路径是否验证。
- HTTP status 和 response shape 是否验证。
- 数据库或业务状态是否验证。
- 副作用是否验证。
- 错误路径是否验证。
- mock 和真实合同是否区分。
- 残余风险是否写清。

如果只有“页面看起来正常”或“接口测试通过”，验证闭环还没完成。

AI Coding 的交付质量，最终不是看它写了多少代码，而是看它留下了多少能互相印证的证据。

26.27　常见问题

没有 E2E 测试基础怎么办？

先从关键路径的一条 Playwright 测试开始，不要试图一次覆盖所有页面。也可以先用手工步骤加截图记录，但要明确它不是自动化回归。

前端测试 mock API 还算验证吗？

算前端状态验证，不算真实合同验证。要配接口测试或至少一次真实 API 检查。

后端已经有测试，还要页面验证吗？

只要需求包含用户交互，就要验证页面路径。后端测试不能证明按钮、提示、状态刷新和重复点击行为。

外部服务不能本地跑怎么办？

用 mock 或 contract 测本地边界，记录真实外部路径未验证，并安排 staging 或人工验证。

验证命令太慢怎么办？

先 focused，再 module，再关键 E2E。不要每一步都跑全量，但合并前要覆盖受影响面。

26.28　可复查资料

- Playwright intro：https://playwright.dev/docs/intro〔26-1〕
- Playwright assertions：https://playwright.dev/docs/test-assertions〔26-2〕
- Playwright API testing：https://playwright.dev/docs/api-testing〔26-3〕
- MDN HTTP response status codes：https://developer.mozilla.org/en-US/docs/Web/HTTP/Status〔26-4〕
- MDN Fetch API：https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API〔26-5〕

本章资源

〔26-1〕 https://playwright.dev/docs/intro
https://playwright.dev/docs/intro

〔26-2〕 https://playwright.dev/docs/test-assertions
https://playwright.dev/docs/test-assertions

〔26-3〕 https://playwright.dev/docs/api-testing
https://playwright.dev/docs/api-testing

〔26-4〕 https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

〔26-5〕 https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

第27章　AI 说“测试失败无关”时，程序员该怎么判断

AI Coding 里有一句话特别危险：

```
这个测试失败和本次修改无关。
```

它可能是真的。

真实项目里确实会有 baseline failure：改代码前就已经失败的测试、环境不稳定导致的失败、外部依赖不可用、CI 上才出现的问题。

但它也可能是 AI 在逃避当前 diff 的影响。

如果你直接相信这句话，就可能把本次改坏的行为带进下一步。如果你完全不信，又会被旧失败拖住，无法推进。

程序员需要的不是态度，而是分类方法。

判断一个测试失败是否“无关”，至少要回答四个问题：

```
改前是否已经失败？
同一个命令能否复现？
失败路径是否经过本次修改？
失败行为是否被当前需求覆盖？
```

回答不了，就不能说无关。

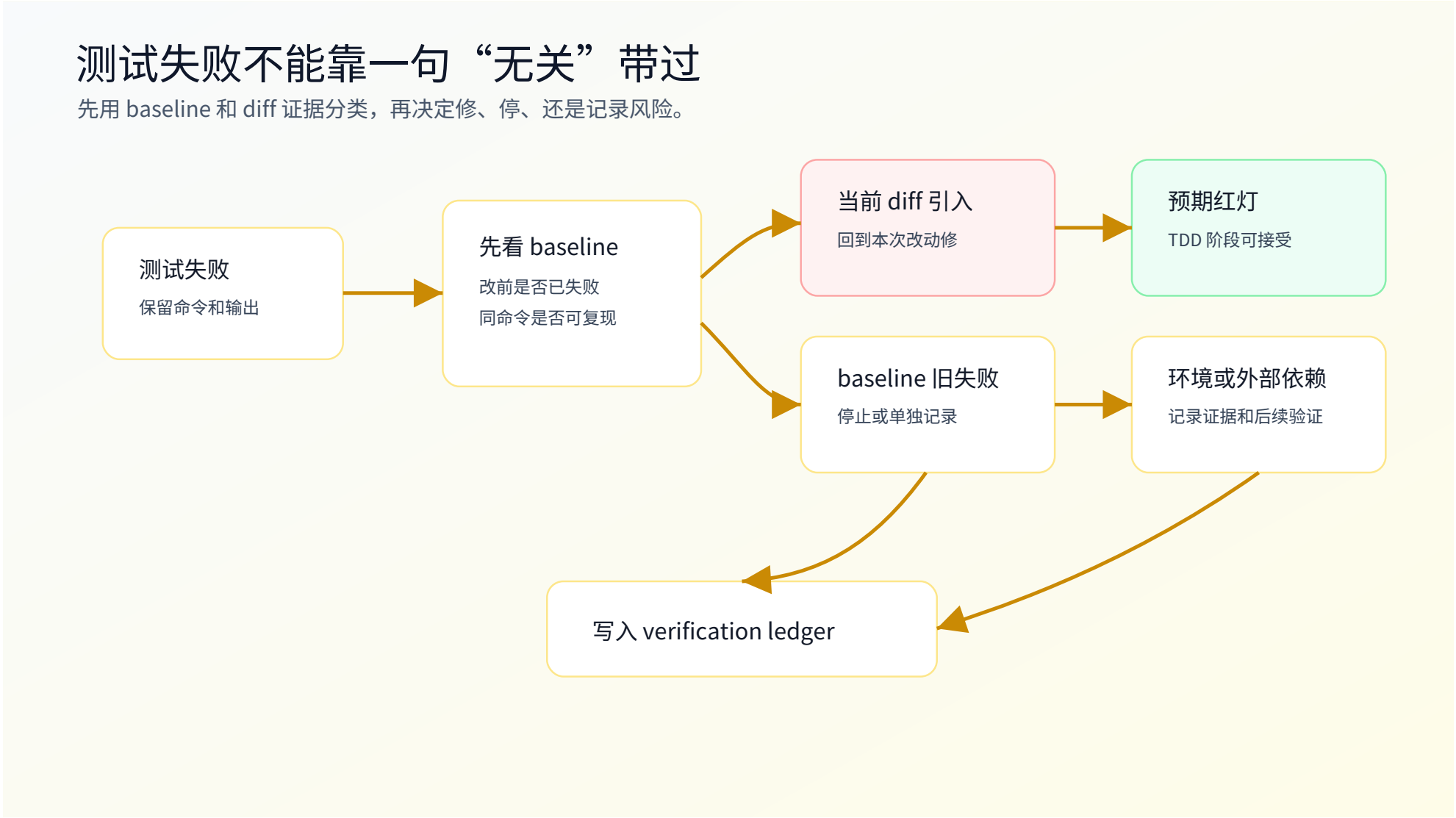


图 27-1　测试失败分类流程

27.1 先保留原始失败证据

第一步不是问 AI 怎么看。

第一步是保留命令和输出。

例如：

清单 27-1 先保留原始失败证据

```
npm test -- order-cancel
```

输出：

```
FAIL test/order/order-cancel.test.ts
  duplicate paid cancellation creates one refund request

Expected: 1
Received: 2
```

这类失败看起来就很可能和本次修改有关，因为它直接对应订单取消和退款请求。

另一个例子：

```
FAIL test/user/avatar-upload.test.ts
Timeout waiting for image resize worker
```

它可能无关，但还不能直接下结论。要看本次 diff 有没有改 worker 配置、测试环境、全局 setup、依赖版本。

所以不要让 AI 用一句话盖过去。要求它输出：

示例 27-2 先保留原始失败证据

```
Failure:
- command
- failing test
- first failing assertion
- actual / expected
- stack or log
- changed files that could affect it
```

没有这些证据，所谓“无关”只是判断，不是结论。

27.2 baseline 是判断的关键

baseline 的意思是：改动前，同一个命令是什么状态。

最理想的情况，是你在开工前就跑过：

清单 27-3 baseline 是判断的关键

```
npm test -- order-cancel
```

并记录：

```
Baseline:
- `npm test -- order-cancel`: pass
```

如果改后失败，那就不能轻易说无关。

如果改前已经失败，要记录：

```
Baseline:
- `npm test -- avatar-upload`: failed before this change.
- failure: Timeout waiting for image resize worker.
- evidence: CI run #123 / local command output.
```

问题是，很多时候你没有改前 baseline。

这时可以用 Git 辅助判断：

清单 27-4 baseline 是判断的关键

```
git diff --name-only
git diff -- test/order/order-cancel.test.ts
```

还可以回到最近的干净提交或主分支复现。但要注意，不要在当前工作区粗暴切分支，避免丢掉未提交改动。可以使用单独 worktree，或者在 CI 里看历史结果。

没有 baseline 时，最多只能写：

```
Not proven unrelated.
Current evidence suggests it may be unrelated because ...
Need baseline confirmation.
```

不能写“确认无关”。

27.3 baseline 可以从哪里来

baseline 不一定只能来自你开工前本地跑过的命令。

常见来源有几种：

· 开工前本地命令输出。

- 主分支或目标分支的 CI 结果。
- 最近一次通过的 workflow run。
- 单独 worktree 上复现结果。
- 团队已知 flaky test 记录。
- issue 或测试稳定性看板。

每种证据强度不同。

最强的是同一分支、同一命令、同一环境、改动前已经失败或通过。

CI 历史有用，但要看是否同环境。同一个测试在 macOS 本地通过、Linux CI 失败，不能简单归类。

如果要在本地验证主分支，不要在当前脏工作区乱切分支。可以用单独 worktree：

清单 27-5 baseline 可以从哪里来

```
git worktree add ../baseline-check main
cd ../baseline-check
npm test -- order-cancel
```

如果仓库不适合新建 worktree，就查 CI 历史，或者记录 baseline 未确认。

关键是不要让 AI 用“我觉得不像相关”代替 baseline。

27.4 失败可以分成四类

测试失败通常可以分成四类。

第一类：当前 diff 引入。

特征：

- 改前通过，改后失败。
- 失败路径经过本次修改文件。
- failure assertion 对应当前需求或相邻行为。
- 回滚当前 diff 后失败消失。

这类必须在当前任务里处理，不能记录成残余风险。

第二类：baseline 旧失败。

特征：

- 改前已经失败。
- 与本次 diff 没有调用路径或配置关系。
- 失败日志和历史失败一致。

这类可以记录，但不要混进当前 diff。需要单独任务修。

第三类：预期红灯。

TDD 阶段新增失败测试，本来就应该失败。

例如 Task 1 只写测试：

```
Expected before implementation:
- paid cancellation test fails.
- duplicate cancellation test fails.
- pending and shipped tests pass.
```

这种失败不是无关，也不是 bug。它是当前阶段的预期证据。

第四类：环境或外部依赖。

例如浏览器下载失败、测试数据库不可用、CI secrets 缺失、第三方服务不可达。

这类也不能直接忽略。要记录：

- 哪个环境失败。
- 本地是否复现。
- 失败是否阻断当前结论。
- 后续在哪里验证。

只有分类清楚，才能决定下一步。

27.5 让 AI 先分类，不要先修

看到失败后，不要说：

```
修一下测试。
```

要说：

示例 27-6 让 AI 先分类，不要先修

```
先不要改代码。

请对测试失败做分类：
- current diff introduced
- baseline existing failure
- expected red test
- environment / external dependency

要求：
```

```
- 引用失败命令和关键输出。
- 检查本次 changed files。
- 说明失败路径是否经过本次修改。
- 如果不能证明无关，就写 not proven unrelated。
- 不要修复。
```

这能阻止 AI 直接把失败压掉。

如果它说“无关”，继续追问：

```
你需要给出证据：
- 改前是否失败？
- 哪个日志或历史运行证明？
- 本次 diff 为什么不可能影响它？
- 是否存在共享配置、全局 setup、依赖版本、测试数据影响？
```

没有证据，就不能接受。

27.6 失败路径是否经过本次 diff

这是很实用的判断。

看改动文件：

清单 27-7 失败路径是否经过本次 diff

```
git diff --name-only
```

看失败堆栈：

```
at cancelOrder (src/order/order-cancel-service.ts:42:18)
at test/order/order-cancel.test.ts:87:24
```

如果堆栈经过本次修改文件，就高度相关。

如果失败是前端 E2E：

```
Timeout waiting for getByText('退款处理中')
```

本次 diff 改了：

```
src/order/order-cancel-service.ts
src/api/order-controller.ts
src/ui/order-detail.tsx
```

也不能说无关。页面等不到文案，可能是后端没返回新状态、接口 shape 变了、前端状态没刷新。

反过来，如果失败是：

```
test/user/avatar-upload.test.ts
```

本次 diff 只改：

```
src/order/order-status-transition.ts
test/order/order-cancel.test.ts
```

且没有共享测试 setup、依赖、全局配置变化，才有初步无关的可能。

注意，是“初步可能”，不是直接确认。

27.7 验证 ledger：把判断写下来

测试失败分类不能只留在聊天里。

可以写到 verification ledger。



图 27-2　Verification ledger 结构

一个可用结构：

清单 27-8 验证 ledger：把判断写下来

````
# Verification Ledger

## Baseline

- `npm test -- order-cancel`: pass before Task 2.
- No known failure in order cancellation tests.

## Change Summary

- Added paid -> refund_requested transition.
- Wired refund request creation.
- Did not change payment provider callback.

## Verification Commands

```bash
npm test -- order-cancel
npm test -- order
````

27.8 Evidence

- `npm test -- order-cancel`: PASS, 12 tests.
- `npm test -- order`: FAIL, `avatar-upload.test.ts` timeout.

27.9 Failure Classification

- `avatar-upload.test.ts` failure is not proven related to this diff.
- It does not pass through changed files.
- No dependency, global setup, or worker config changed.
- Baseline still needs confirmation from main branch or CI history.

27.10 Residual Risk

- Broader `npm test -- order` did not complete cleanly.
- Do not claim full module verification until baseline is confirmed.

示例 27-9 Residual Risk

````
注意这段没有写“测试全过”。它写的是更准确的结论：

```text
````

```
focused tests pass.
broader command has an unclassified or not-proven-related failure.
full module verification not complete.
```

这比 AI 的一句“无关失败”更可信。

27.11 配套工具能检查记录是否完整

配套工程的工具 module 里有一个很小的 `verify-ledger` 命令，用来检查验证记录是否至少包含必要段落、命令块和通过信号。

通过样例：

清单 27-10 配套工具能检查记录是否完整

```
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
```

输出：

```
Verification ledger looks usable: required sections, command block, and pass signal found.
```

失败样例：

清单 27-11 配套工具能检查记录是否完整

```
node src/cli.js verify-ledger --file examples/verification-ledger.fail.md
```

输出：

```
Verification ledger is incomplete.
Missing sections: Change Summary, Verification Commands, Evidence, Residual Risk
Missing command block with the verification command.
```

这个工具不判断业务正确性，也不判断失败是否真的无关。

它只做一件事：防止验证记录连基本证据都没有。

真正的判断仍然要靠你看命令、输出、diff、堆栈和影响范围。

27.12 AI 说“无关”时的追问模板

可以直接用这段：

示例 27-12 AI 说“无关”时的追问模板

```
不要只说无关。

请按下面格式证明：

1. Failing command:
2. Failing test / assertion:
3. Actual vs expected:
4. Changed files in this diff:
5. Does the failing stack/path pass through changed files?
6. Was this failing before the change? Evidence?
7. Could shared setup/config/dependency/test data connect it to this diff?
8. Classification:
9. Recommended action:

如果缺少 baseline evidence，请写 "not proven unrelated"。
```

这个模板能逼 AI 从结论退回证据。

27.13 哪些失败不能归为无关

下面这些情况，不要轻易接受“无关”：

- 失败测试和本次需求同领域。
- 失败堆栈经过本次修改文件。
- 失败 assertion 对应本次需求的边界。
- 本次 diff 改了共享工具、测试 setup、配置、依赖、全局类型。
- 本次 diff 改了 API response shape。
- 本次 diff 改了异步时序、缓存、队列、事务边界。
- 没有改前 baseline。

特别是共享配置和依赖。

很多看似无关的失败，实际来自：

示例 27-13 哪些失败不能归为无关

```
test setup
mock server
global fixture
date/time helper
serializer
dependency lockfile
environment variable
```

AI 很容易只看业务文件名，忽略这些共享层。

27.14 什么时候可以接受“先记录，不阻塞”

不是所有失败都要在当前任务里修。

可以先记录、不阻塞的条件是：

- focused tests 覆盖当前需求且通过。
- broader failure 有证据显示不是当前 diff 引入。
- 失败已经有 baseline 或历史记录。
- 当前 diff 没有触碰共享 setup、配置、依赖。
- ledger 里明确写了残余风险。
- 后续有单独任务或 CI 跟踪。

例如：

示例 27-14 什么时候可以接受“先记录，不阻塞”

```
Accept with residual risk:
- `npm test -- order-cancel`: pass.
- `npm test -- order`: fails on existing `avatar-upload.test.ts` timeout.
- Same failure exists on main branch CI run #123.
- Current diff touches only order status transition and order cancel tests.
- Follow-up issue: stabilize avatar upload worker test.
```

这时可以继续，但不能把验证口径写成“所有测试通过”。

只能写：

```
Focused verification passed. Broader suite has known baseline failure.
```

27.15 什么时候必须阻塞

下面这些情况不能放行：

- focused tests 失败。
- 失败路径经过本次修改文件，且没有反证。
- 本次需求的核心 scenario 失败。
- API contract、数据库状态或副作用验证失败。
- baseline 未确认，且失败在同一业务领域。
- 测试被 AI 改弱后才通过。
- CI 必需检查失败，且失败与当前模块相关。

这时应该写：

```
Blocked:
- failure may be related to current diff.
- baseline is not confirmed.
- do not proceed to next task.
```

“记录残余风险继续推进”只适合已证明不影响当前交付核心结论的情况。否则就是把未知风险往后推。

27.16 检查 shared setup，别只看业务文件

很多“看起来无关”的测试失败，最后都来自共享层。

需要重点检查：

示例 27-15 检查 shared setup，别只看业务文件

```
test setup
global fixture
mock server
serializer
date/time helper
environment variables
dependency lockfile
build config
browser config
database migration
```

比如本次只改订单取消，但顺手改了 `test/setup.ts` 里的全局 mock，头像上传测试失败就不能直接说无关。

可以让 AI 输出：

示例 27-16 检查 shared setup，别只看业务文件

```
Shared surface check:
- Did this diff touch test setup? no
- Did this diff touch dependency lockfile? no
- Did this diff touch serializers or API client? yes, `src/api/client.ts`
- Could failing test use that API client? yes
- Classification: not proven unrelated
```

这比只看失败测试文件名可靠得多。

27.17 AI 改测试后，要重新分类

如果 AI 在修复过程中改了测试，原来的失败分类可能失效。

需要看：

· 是否删除了失败断言。

- 是否放宽了 expected。
- 是否把测试改成 skip 或 only。
- 是否把真实路径换成 mock 路径。
- 是否改了 fixture，让测试不再覆盖原场景。

可以要求：

示例 27-17 AI 改测试后，要重新分类

```
请列出本次测试文件改动：
- 新增了哪些断言。
- 删除或放宽了哪些断言。
- 是否有 skip/only。
- 是否改变 fixture 状态。
- 每个测试改动对应哪个 requirement。
```

如果测试被改弱，不能接受“测试通过”作为证据。要回到 requirement 和失败输出重新判断。

27.18 把无关失败变成可追踪事项

确认是 baseline 旧失败后，也不要只在聊天里说一句“无关”。

至少写进 ledger 或 issue：

清单 27-18 把无关失败变成可追踪事项

```
Known baseline failure:
- `test/user/avatar-upload.test.ts` timeout.
- Exists on main branch CI run #123.
- Current diff does not touch upload, worker setup, or shared test config.
- Follow-up: stabilize avatar upload worker test.
```

这样后续不会反复争论同一个失败。

如果是团队项目，最好有 issue 或 CI 注释承接。否则“无关失败”会不断消耗 review 信任。

27.19 一个判断案例：avatar 测试失败是否无关

场景：

示例 27-19 一个判断案例：avatar 测试失败是否无关

```
本次改动：
- `src/order/order-status-transition.ts`
- `test/order/order-cancel.test.ts`
```

```
验证:
- `npm test -- order-cancel`: PASS
- `npm test -- order`: FAIL `test/user/avatar-upload.test.ts`
```

AI 说：

```
avatar-upload 和订单取消无关，可以忽略。
```

这句话不能直接接受。先查四件事。

第一，改动文件是否真的只在订单模块：

清单 27-20 一个判断案例：avatar 测试失败是否无关

```
git diff --name-only
```

如果没有共享文件、锁文件、test setup、API client 变化，初步支持无关。

第二，失败堆栈是否经过本次改动：

```
avatar-upload -> image resize worker -> upload service
```

如果路径不经过订单模块，也支持无关。

第三，baseline 是否存在。

如果主分支 CI 上同一个 avatar 测试已经失败，证据更强。没有 baseline，就只能写 not proven unrelated。

第四，当前验证口径怎么写。

可以写：

清单 27-21 一个判断案例：avatar 测试失败是否无关

```
Focused verification:
- `npm test -- order-cancel`: PASS

Broader verification:
- `npm test -- order`: FAIL on `avatar-upload.test.ts`

Classification:
- not proven related to current diff.
- changed files limited to order status and order cancel tests.
- failing path does not pass through changed files.
- baseline still needs confirmation.
```

```
Conclusion:
- do not claim full module verification.
- current order focused verification passed.
```

如果后来确认主分支也失败，再更新：

清单 27-22 一个判断案例：avatar 测试失败是否无关

```
Baseline confirmed:
- same avatar-upload timeout exists on main CI run #123.
- follow-up issue linked.
```

这样既不被旧失败拖住，也不会把未知失败包装成已验证。

27.20 给 Agent 的失败分类指令

可以固定使用这段：

示例 27-23 给 Agent 的失败分类指令

```
先不要修改代码。

请对当前测试失败做分类：
1. current diff introduced
2. baseline existing failure
3. expected red test
4. environment / external dependency

必须输出：
- failing command
- failing test / assertion
- actual vs expected
- changed files
- whether failing path passes through changed files
- shared setup / config / dependency check
- baseline evidence, if any
- classification
- recommended action

如果没有 baseline evidence，不要写 confirmed unrelated，只能写 not proven unrelated。
```

这段指令会迫使 AI 把“无关”拆成证据。

27.21 放行结论要分级

测试失败后的结论不要只有“能不能继续”两档。

可以分四级：

示例 27-24 放行结论要分级

```
Pass:
- required focused and broader checks passed.

Pass with known baseline:
- focused checks passed.
- broader failure confirmed as baseline.
- residual risk recorded.

Blocked:
- failure may be related to current diff.
- baseline missing or path overlaps.

Expected red:
- failure is expected in current TDD step.
- implementation not started yet.
```

对应写法：

清单 27-25 放行结论要分级

```
Verification conclusion:
- Pass with known baseline.
- Current order cancel focused tests passed.
- Broader suite still has known avatar-upload baseline failure.
- Full suite is not clean; follow-up issue linked.
```

这比“测试基本通过”准确得多。

27.22 不要把环境失败当成业务成功

有些失败看起来和业务无关，比如浏览器没装、数据库没启动、CI secret 缺失。它们确实可能不是当前 diff 引入的，但它们也意味着某部分验证没有完成。

例如：

```
Playwright browser executable not found.
```

不能写：

```
前端验证无关，后端通过即可。
```

应该写：

清单 27-26 不要把环境失败当成业务成功

```
Frontend E2E not executed:
- reason: Playwright browser missing in local environment.
- planned evidence: user path shows refund state after cancel.
- replacement: none yet.
- residual risk: page path not verified.
```

环境失败不一定阻塞所有任务，但必须降低验证结论的等级。

27.23 PR 里怎么写失败分类

PR 描述里可以这样写：

清单 27-27 PR 里怎么写失败分类

```
Verification:
- `npm test -- order-cancel`: PASS
- `npm test -- order`: FAIL on `test/user/avatar-upload.test.ts`

Failure classification:
- avatar failure is a known baseline failure on main CI run #123.
- current diff does not touch upload, worker setup, shared test config, or lockfile.
- order focused tests cover the changed behavior and passed.

Residual risk:
- full `npm test -- order` is not clean in this workspace.
- follow-up issue: stabilize avatar upload worker test.
```

这段话不会把失败藏起来，也不会让旧失败阻塞当前订单改动。

如果没有 baseline，就要改成：

清单 27-28 PR 里怎么写失败分类

```
Failure classification:
- avatar failure is not proven unrelated.
- current diff does not appear to touch the failing path.
- baseline confirmation is still missing.

Conclusion:
- do not claim full module verification.
- decide whether this blocks merge based on release risk.
```

这种表达比“无关测试失败”更准确，reviewer 也能直接复查。

27.24 失败分类要防止范围漂移

如果为了证明失败无关，Agent 开始修改失败测试所在模块，那任务已经漂移。

比如当前任务是订单取消，AI 为了处理 avatar baseline failure，开始改图片 worker timeout。即使改对了，也不该混进当前 PR。

可以要求：

```
如果失败分类指向 baseline 旧失败：
- 不要修这个失败。
- 只记录证据和 follow-up。
- 当前 diff 不要新增与本任务无关的文件。
```

旧失败要修，应该开独立任务。这样当前需求的 diff 才保持可审。

27.25 最终判断要写得克制

测试失败分类后的结论不要夸大。

可以写：

```
Confirmed unrelated
```

只有在 baseline、路径、共享层都查清楚时才用。

多数情况下更适合写：

```
Not proven related to current diff.
```

或者：

```
Known baseline failure, tracked separately.
```

这几个表达差别很大。

“确认无关”是强结论；“未证明相关”是当前证据下的保守判断；“已知 baseline”说明有外部证据支持。

程序员最需要学会的是这种克制。AI Coding 不是把所有不方便的失败都解释掉，而是把证据强度和结论强度对齐。

如果结论强度不够，就不要用它支撑合并决策。可以继续补 baseline，也可以把失败作为阻塞项。

27.26 常见错误

错误一：没有 baseline 就说无关。

没有改前证据，只能说尚未证明相关或无关。

错误二：只看文件名。

测试文件名不相关，不代表共享配置、fixture、依赖不相关。

错误三：让 AI 直接修所有失败。

先分类，再决定修不修。否则旧失败会污染当前 diff。

错误四：把预期红灯当成 bug。

TDD 阶段新增失败测试可以是预期结果，要写清 expected before implementation。

错误五：把残余风险写成验证通过。

没跑完 broader suite，就不要写 full verification passed。

错误六：验证记录只有总结，没有命令。

没有命令和输出，就无法复查。

27.27 改完后重点看什么

判断测试失败是否无关，最后看这些证据：

- 是否有 failing command。
- 是否有 actual vs expected。
- 是否有改前 baseline。
- 是否检查了 changed files。
- 是否检查失败路径和共享 setup。
- 是否分类为当前 diff、baseline、预期红灯或环境问题。
- 是否写入 verification ledger。
- 是否准确表达 residual risk。

如果这些都没有，AI 的“无关”不值得采信。

AI Coding 的验证纪律，不是要求所有测试永远全绿，而是不能把未知失败包装成无关失败，也不能把部分验证包装成完整验证。

27.28 常见问题

如果没有时间确认 baseline 怎么办？

不要写“确认无关”。写“baseline 未确认，focused tests passed，broader failure unresolved”，并把它作为残余风险或阻塞项。

CI 上有旧失败，本地没有，算 baseline 吗？

要看同分支、同命令、同环境。CI 历史可以作为证据，但要说明环境差异。

AI 已经修好了失败，还需要分类吗？

需要。否则你不知道它修的是当前问题、旧问题，还是把测试改弱了。

所有测试都通过，还需要 ledger 吗？

复杂任务建议要。ledger 不只是记录失败，也记录你跑了什么、证明了什么、没证明什么。

verify-ledger 能判断测试失败是否无关吗？

不能。它只检查记录是否有基本结构和命令证据。业务判断必须由人看 diff、日志和测试覆盖。

27.29 可复查资料

- GitHub Actions workflow logs：https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/using-workflow-run-logs〔27-1〕
- GitHub Actions troubleshooting workflows：https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/troubleshooting-workflows〔27-2〕
- Git diff 官方文档：https://git-scm.com/docs/git-diff〔27-3〕
- Git status 官方文档：https://git-scm.com/docs/git-status〔27-4〕

本章资源

〔27-1〕 https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/using-workflow-run-logs
https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/using-workflow-run-logs

〔27-2〕 https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/troubleshooting-workflows
https://docs.github.com/en/actions/monitoring-and-troubleshooting-workflows/troubleshooting-workflows

〔27-3〕 https://git-scm.com/docs/git-diff
https://git-scm.com/docs/git-diff

〔27-4〕 https://git-scm.com/docs/git-status
https://git-scm.com/docs/git-status

第28章　模块实战：用 AI 完成一次 CLI 交互改造

CLI 交互改造看起来像改输出文案，实际是在改用户可见合同。

真实项目里的 CLI 经常被人、脚本、CI、Git hooks 或 README 复制命令调用。这个合同至少包括：

```
stdout
stderr
exit code
README 示例命令
```

这几个面任何一个漂移，都会让“测试通过”的改动变成真实用户路径上的问题。

配套代码仓库 DavidDingXu/learn-ai-coding〔28-1〕 按一个大仓库组织，仓库根目录下有业务 module 和工具 module。这里用工具 module：`ai-coding-workflow-kit`。它适合承载 CLI 合同、验证记录和只读工具边界这类工作流样例，不替代综合业务实战。

这一篇复盘 `ai-coding-workflow-kit` 的一次 CLI 合同改造。过程记录在：

```
learn-ai-coding/ai-coding-workflow-kit/dev-process/02-cli-contract-change.md
```

对应真实需求是：

```
未知命令必须失败并输出可读命令列表。
verify-ledger 缺少 --file 必须失败。
verify-ledger 校验失败时 exit code 必须为 1。
README 中的命令必须引用仓库里真实存在的文件。
```

重点不是证明 `node src/cli.js unknown` 能跑，而是看 AI 如何围绕 CLI 合同做一个可审的小 diff。



图 28-1　CLI 合同表面

28.1　先把任务写成合同

过程记录里的任务卡先限定目标：

```
补齐 CLI 用户可见交互合同，保证 stdout、stderr、exit code 和 README 示例可验证。
```

范围也写得很窄：

```
src/cli.js
test/cli.test.js
README.md
```

非目标更关键：

```
不引入 commander/yargs。
不重做 CLI 架构。
不改变成功命令的 Markdown 输出格式。
不修改 lib 层业务函数。
```

如果没有这些非目标，AI 很可能把“补错误路径测试”扩大成“升级 CLI 框架”。那不是这次需求。

CLI 改造最容易出问题的地方，就是把用户可见行为当成内部实现细节。

28.2　先测进程边界，不先改实现

库函数测试不能证明 CLI 合同。

例如 `validateLedgerFile()` 返回错误，不等于命令行进程会：

```
把错误写到 stderr。
返回非 0 exit code。
保持 stdout 为空。
```

所以这次先补进程级测试：

```
test/cli.test.js
```

测试方式是用 Node 内置能力启动真实 CLI：

清单 28-1　先测进程边界，不先改实现

```
import { spawnSync } from 'node:child_process';

function runCli(args) {
```

```
  return spawnSync('node', ['src/cli.js', ...args], {
    cwd: new URL('..', import.meta.url),
    encoding: 'utf8'
  });
}
```

这段测试比直接调用函数更慢一点，但它测到的是用户真正会遇到的边界。

本次覆盖三条路径：

```
unknown command -> stderr + non-zero exit code
verify-ledger without --file -> stderr + non-zero exit code
failing ledger -> non-zero exit code
```

对 CLI 来说，这些就是行为合同。

28.3 实现只允许小 diff

当前 CLI 入口在：

```
learn-ai-coding/ai-coding-workflow-kit/src/cli.js
```

实现保持手写分发，不引入新框架：

示例 28-2 实现只允许小 diff

```
codemap
context-audit
task-context
context-pack
verify-ledger
unknown command
```

未知命令走 stderr：

```
Unknown command: <name>
Commands: codemap, context-audit, task-context, context-pack, verify-ledger
```

缺少 `--file` 也走 stderr：

```
--file is required
```

这次不改变成功命令的 Markdown 输出。原因很简单：成功输出可能已经被人复制、被文档引用、被其他脚本消费。目标是补齐错误路径证据，不是重新设计 CLI。

28.4 README 也是用户路径

这次还有一个真实问题：README 里的 `verify-ledger` 示例必须指向仓库里存在的文件。

过程记录要求 README 使用：

```
examples/verification-ledger.pass.md
examples/verification-ledger.fail.md
```

如果 README 写的是不存在的 `verification-ledger.md`，测试可能仍然全绿，但读者复制命令就会失败。

所以 CLI 合同不只在 `src/cli.js`。它还包括：

```
README 里的命令是否能运行。
examples 文件是否真实存在。
失败示例是否返回非 0。
```

这就是为什么实战不能停在“跑一下命令”。命令只是验证点之一，真正的需求是让用户路径稳定。



图 28-2　CLI 行为改造循环

28.5　给 AI 的执行指令

让 AI 做这类改动时，不要写：

```
优化一下 CLI 错误提示。
```

应该写成：

示例 28-3　给 AI 的执行指令

```
请先读取：
- learn-ai-coding/ai-coding-workflow-kit/dev-process/02-cli-contract-change.md
- learn-ai-coding/ai-coding-workflow-kit/src/cli.js
- learn-ai-coding/ai-coding-workflow-kit/test/cli.test.js
- learn-ai-coding/ai-coding-workflow-kit/README.md
- learn-ai-coding/ai-coding-workflow-kit/examples/

只处理 CLI 用户可见合同：
1. 未知命令必须 stderr 输出可读命令列表，并返回非 0。
2. verify-ledger 缺少 --file 必须 stderr 输出错误，并返回非 0。
3. 失败 ledger 必须返回非 0。
4. README 命令必须引用真实 examples 文件。

不要引入 CLI 框架。
不要改变成功命令的 Markdown 输出格式。
不要修改 lib 层函数。
先补进程级测试，再做最小实现。
完成后运行 npm test，并手动执行三条 CLI 命令。
```

这段指令把任务压回了真实需求：用户可见合同。

28.6　Review 要看四个面

审查这次 diff 时，不要只看测试文件有没有新增。

要逐项检查：

```
stdout：成功输出是否保持不变？
stderr：错误信息是否写到 stderr？
exit code：失败路径是否非 0？
README：示例命令是否真实可运行？
```

还要看 scope：

```
是否只改 src/cli.js、test/cli.test.js、README.md?
是否引入了新依赖?
是否改了 lib 层业务函数?
是否把帮助信息复制到多个地方导致未来漂移?
```

如果 AI 为了“体验更好”改了成功 Markdown 输出，即使测试通过，也要退回。因为这已经超过本次合同改造。

28.7 验证：自动测试加手动命令

自动测试：

清单 28-4 验证：自动测试加手动命令

```
cd learn-ai-coding/ai-coding-workflow-kit
npm test
```

当前结果应为：

```
tests 12
pass 12
fail 0
```

手动命令：

清单 28-5 验证：自动测试加手动命令

```
node src/cli.js unknown
node src/cli.js verify-ledger
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
node src/cli.js verify-ledger --file examples/verification-ledger.fail.md
```

检查点不是“有没有输出”，而是：

示例 28-6 验证：自动测试加手动命令

```
unknown 是否非 0。
unknown 是否只写 stderr。
缺 --file 是否非 0。
pass ledger 是否返回 0。
fail ledger 是否返回 1。
README 示例是否能复制执行。
```

命令输出是证据，不是实战主体。实战主体是：任务卡、TDD、最小实现、review checklist、验证和残余风险。

28.8　残余风险要写出来

过程记录没有把任务说成“完美解决”。

它保留了两个残余风险：

```
README 命令没有单独文档测试，只靠人工命令复核。
CLI 当前仍是手写 parser，后续命令增多时要重新评估是否需要框架。
```

这类记录很重要。

真实项目里，完成不是把所有风险消灭，而是清楚哪些风险已经被测试覆盖，哪些风险只是人工复核，哪些风险留到后续任务。

28.9　迁移到你的项目

只要你的 CLI、脚本或命令行工具被人复制使用，就可以套这个流程。

先列合同面：

示例 28-7　迁移到你的项目

```
命令名。
参数。
stdout。
stderr。
exit code。
README 或文档示例。
脚本调用方。
```

再写测试策略：

```
库函数测试覆盖业务逻辑。
进程级测试覆盖真实 CLI 行为。
README 命令至少人工抽查，关键项目可以加文档命令测试。
```

最后限制 AI：

```
只修当前合同。
不重构 CLI 框架。
不改变成功输出格式。
不删除失败路径测试。
完成前必须报告 stdout/stderr/exit code。
```

CLI 是文本界面。文本就是协议。

28.10　常见错误

错误一：只测函数，不测进程。

函数测试通过，不代表 stderr、exit code 和参数解析都对。

错误二：把 README 当文档，不当用户路径。

README 命令不可运行，就是用户路径失败。

错误三：顺手换 CLI 框架。

除非需求就是框架迁移，否则这会把一个小合同修复变成大风险改造。

错误四：只断言包含某个词。

CLI 测试要覆盖 stdout/stderr/exit code，不只是字符串包含。

错误五：没有记录残余风险。

README 命令是否有自动化测试、是否有外部脚本调用方，这些都应该写清楚。

28.11　可复查资料

- CLI 合同改造过程记录：`learn-ai-coding/ai-coding-workflow-kit/dev-process/02-cli-contract-change.md`
- CLI 入口：`learn-ai-coding/ai-coding-workflow-kit/src/cli.js`
- CLI 进程级测试：`learn-ai-coding/ai-coding-workflow-kit/test/cli.test.js`
- README 示例命令：`learn-ai-coding/ai-coding-workflow-kit/README.md`
- 验证记录样例：`learn-ai-coding/ai-coding-workflow-kit/examples/`
- OpenSpec 变更记录：`learn-ai-coding/ai-coding-workflow-kit/openspec/changes/cli-contract-tests/`

本章资源

〔28-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

PART 07

第7篇　把方法变成可复用能力

先理解 Skill 如何承载流程，再讨论外部工具、多 Agent 和自动化。扩展能力之前，明确触发条件、权限和验证方式。

阅读重点

能识别值得固化的流程，并给工具接入设置清晰边界。

第29章　Skill 是什么：从提示词到可复用工作流

很多程序员第一次看到 Skill，会把它理解成“高级 Prompt”。

这个理解太窄。

Prompt 解决的是当前这一次怎么说。Skill 解决的是以后每一次类似任务都按什么流程、读取什么资料、使用什么脚本、产出什么证据。

比如你已经形成了一套 AI Coding 调试方法：

示例 29-1　本章示例

```
先复现
读完整错误
追调用链
提出单一假设
最小验证
修根因
跑回归
记录残余风险
```

如果每次调试都把这段复制进聊天窗口，它只是 prompt。

如果你把它做成一个 Skill，让 Agent 在遇到 bug、测试失败、CI 失败时自动读取这套流程，并配合 references、scripts、模板和输出要求，那它就是可复用工作流。

Skill 的价值，不是让 AI 多记几句话，而是把个人经验从“临时提醒”变成“可重复执行的工程程序”。



图 29-1　Skill 结构拆解

29.1 Skill 适合固化什么

不是所有经验都适合做成 Skill。

一次性的任务要求，放 prompt。

某个 repo 的长期规则，放 `AGENTS.md`、`CLAUDE.md` 或对应工具的项目规则。

跨项目反复使用的工作流，才适合做 Skill。

适合 Skill 的场景通常有几个特征：

- 每次任务都有固定步骤。
- 步骤中需要查固定资料或模板。
- 需要运行固定脚本或校验器。
- 需要防止 AI 走常见错误路径。
- 输出有固定结构。
- 这个流程会在多个项目或多类任务里复用。

例如：

- 代码审查。
- 系统化调试。
- TDD 实现。
- 文章发布检查。
- API 文档核对。
- 生成项目代码地图。
- 前后端验证记录。
- 技术方案 red-team。

这些都不是一句 prompt 能长期解决的。

29.2 一个 Skill 通常包含什么

不同工具对 Skill 的具体格式不完全一样，但从当前 Codex Skills、Claude Code Skills 和 Superpowers 的结构看，核心思路很接近：用一个入口说明文件告诉 Agent 什么时候使用、怎么使用、有哪些资料和脚本。

一个常见结构是：

示例 29-2 一个 Skill 通常包含什么

```
my-skill/
├── SKILL.md
├── references/
│   ├── checklist.md
│   └── examples.md
├── scripts/
│   └── validate-output.js
└── assets/
    └── template.md
```

`SKILL.md` 是入口。它至少要说明：

- skill 名称。
- 什么时候触发。
- 执行前要读什么。
- 执行步骤。
- 哪些行为禁止。
- 输出格式。
- 验证方式。

`references/` 放相对稳定的资料，例如检查清单、案例、模板说明、领域规则。

`scripts/` 放可执行工具，例如校验 Markdown 结构、生成索引、检查链接、转换文件。

`assets/` 放固定资产，例如模板、样例、图片、脚手架片段。

Skill 不是把所有内容都塞进一个超长 prompt。它应该让 Agent 按需读取相关资料，必要时运行脚本，最后产出可验证结果。

29.3 Skill 和 Prompt 的区别

Prompt 是一次性的。

例如：

```
请帮我 review 这个 PR，重点看 bug、风险和测试缺口。
```

Skill 是持久的。

它会把 review 方法固定下来：

示例 29-3 Skill 和 Prompt 的区别

```
先读需求和 diff。
按严重程度输出 findings。
每个 finding 必须有文件和行号。
先列问题，再写摘要。
没有问题就明确说没有发现。
必须说明未运行的测试。
```

Prompt 的问题是容易漏。今天你记得说“先列问题”，明天可能忘了。Agent 也可能在长上下文里被别的任务带偏。

Skill 的作用是把这些长期有效的行为放到稳定入口里。

但不要把所有 prompt 都变成 Skill。

如果一个要求只对当前任务有效，例如“这次只改订单模块，不改支付回调”，它就应该留在 prompt 或当前 `tasks.md`，不要写进 Skill。

29.4 Skill 和项目规则的区别

很多人会把 Skill 和 `AGENTS.md` 混用。

这会导致两个问题：

- 项目规则变成工具教程。
- Skill 变成某个 repo 的一次性约定。

边界可以这样判断：

```
这个约束是否只属于当前 repo?
```

如果是，放项目规则。

例如：

清单 29-4 Skill 和项目规则的区别

```
## Test Commands

- Backend focused tests: `npm test -- order`
- Frontend tests: `npm test -- web`
- E2E tests: `npx playwright test`
```

这属于项目规则。

再问：

```
这个流程是否可以跨 repo 复用?
```

如果是，放 Skill。

例如：

```
调试时必须先复现、收集证据、提出单一假设，再修复。
```

这属于工作流。

Skill 可以引用项目规则，但不要替代项目规则。一个 TDD Skill 可以要求“运行项目指定的 focused test”，具体命令应该来自 repo。



图 29-2　Skill 边界图

29.5 Skill 和 Plugin 的区别

Plugin 更像分发单位。

一个 plugin 里可以包含多个 skills，也可以包含 MCP 配置、hooks、资源、应用或其他安装信息。Superpowers 就是一个典型例子：它不是单个 skill，而是一组围绕软件开发流程的技能和安装方式。

对个人程序员来说，可以先这样理解：

```
Skill = 一个可复用工作流
Plugin = 一组能力的安装包
```

如果你只是想固化“怎么做代码审查”，写 Skill。

如果你想把一组工作流、脚本、资源和工具配置打包给另一个工具或另一台机器安装，再考虑 Plugin。

不要一开始就做 plugin。先让单个 Skill 在自己的项目里跑通。

29.6 Skill 和 MCP 的区别

Skill 不是外部工具接入。

MCP 更适合让 Agent 访问外部资源或执行外部能力，例如文档、Issue、浏览器、只读数据库、代码索引服务。

Skill 负责“怎么做”。

MCP 负责“能访问什么”。

例如调试一个线上问题：

- Skill 规定先复现、收集日志、提出假设。
- MCP 可以提供只读日志查询工具、Issue 读取工具、监控查询工具。

如果只接 MCP，没有 Skill，Agent 可能拿到很多数据但不知道怎么排障。

如果只有 Skill，没有 MCP，Agent 有方法但拿不到外部证据。

两者可以组合，但不要混淆。

29.7 Skill 在 AI Coding 工作流里的位置

把前面讲过的内容放到一条链路里，Skill 通常不负责定义需求，也不负责保存系统长期行为。它更像执行阶段的工作流入口。

```
requirements / issue / spec
-> task context / design / tasks
-> Skill 触发对应执行纪律
-> Agent 读文件、改代码、跑命令
-> verification ledger / PR / review
```

例如：

- OpenSpec 保存“当前系统行为”和“拟议变更”。
- tasks.md 定义“这次要做哪几个小 diff”。
- TDD Skill 规定“执行行为变更前先写失败测试”。
- Debug Skill 规定“失败时先复现、找根因，再修复”。
- Verification Skill 规定“声明完成前必须报告命令和残余风险”。

这样看，Skill 不是又一层文档，而是把任务执行过程里的高频纪律变成可复用流程。

如果你的任务上下文本身已经足够具体，但 Agent 仍然经常跳步骤，那就适合把跳过的步骤做成 Skill。

29.8 从项目规则提取到 Skill 的时机

不要一开始就把规则写成 Skill。

很多规则先出现在项目里。比如你在一个支付项目中反复提醒 Agent：

```
不要改支付回调签名。
不要把幂等失败当成普通异常。
改状态机必须补状态转换测试。
完成前必须跑 payment focused tests。
```

这些话先应该进入当前 repo 的项目规则，因为它们高度依赖业务边界。

等你在多个项目里看到同一种问题，才考虑提取为 Skill。

例如下面这部分可以跨项目复用：

```
涉及状态机、持久化、认证、支付或 job 的行为变更：
- 先识别状态来源和副作用。
- 先写能失败的 focused test。
- 不把 refactor 和行为变化混在一个 diff。
- 完成前报告命令、结果和未验证范围。
```

这已经不是某个支付项目的规则，而是一类高风险改动的执行纪律。

提取时要保留边界。

Skill 里不要写：

```
Run `npm test -- payment`.
```

应该写：

```
Run the focused test command documented by the repository. If no focused command exists, inspect
project scripts and report the chosen command before using it as evidence.
```

这样 Skill 负责流程，项目规则负责具体命令。

29.9 Skill 的生命周期

Skill 不是写完就永远正确。

一个个人 Skill 通常会经历四个阶段。

```
试用：从一次真实失误里提炼流程。
收敛：只保留会改变 Agent 行为的触发、步骤、停止点和输出。
脚本化：把格式、链接、字段、命令记录这类确定性检查交给脚本。
淘汰：工具能力变化、流程不再适用或被项目规则替代时删除。
```

很多 Skill 失效，不是因为模型变差，而是因为它已经过期。

例如一个文档核对 Skill 里写死了某个工具的旧命令。半年后工具安装方式变了，Agent 仍然会按旧命令执行。正确做法是把易变事实换成核对动作：

```
Before using tool-specific commands, verify the current official installation and command
reference.
```

再把稳定流程留下：

```
先核对当前版本。
再列出破坏性变化。
只改当前任务需要的调用点。
最后用测试证明改动。
```

维护 Skill 时，不要只追加规则。

每次修改都问一句：

```
这条规则是否还在真实任务里发挥作用？
```

如果答案不清楚，先删掉或移到 reference，而不是继续增加主流程噪音。

29.10 一个最小 Skill 应该怎么写

假设你想把“验证记录检查”固化成 Skill。

先不要写复杂工具。一个最小 `SKILL.md` 可以这样：

清单 29-5 一个最小 Skill 应该怎么写

```
---
name: verification-ledger-review
description: Use when reviewing whether an AI Coding task has enough verification evidence before
declaring completion.
---

# Verification Ledger Review

Use this skill before claiming an implementation is complete.

## Required Inputs

- changed file list
- verification commands and output
- known baseline failures
- residual risks

## Workflow

1. Read the task goal and changed files.
2. Check whether focused verification was run.
3. Check whether broader verification was needed and run.
4. Classify any failures as current diff, baseline, expected red test, or environment.
5. Reject completion if command evidence is missing.
6. Report residual risk explicitly.

## Output

- Completion status: complete / incomplete
- Evidence used
- Missing verification
- Failure classification
- Residual risk
```

这已经比临时 prompt 稳定。

但它还不够强。下一步可以加 references：

```
references/
└── failure-classification.md
```

再加脚本：

```
scripts/
└── check-ledger.js
```

Skill 的演进路线应该是：

```
先固化流程
再补参考资料
再加脚本校验
最后才考虑分发
```

不要反过来，一上来就做很重的目录和安装流程。

29.11 好 Skill 的关键是触发条件

Skill 写得再好，触发条件模糊也会出问题。

坏 description：

```
Use this skill to improve code quality.
```

这个范围太大，Agent 不知道什么时候该用。

好 description：

```
Use when encountering a bug, test failure, CI failure, or unexpected runtime behavior before
proposing fixes.
```

它明确：

- 触发对象是 bug、测试失败、CI 失败、异常行为。
- 触发时间是在提出修复之前。

再比如 TDD Skill：

```
Use when implementing a behavior change where a focused automated test can be written before
implementation.
```

这比“Use for development”清楚得多。

触发条件要具体到工程场景，而不是写抽象价值。

29.12 Skill 里不要写工具事实以外的幻想能力

写 Skill 时，一个常见错误是把愿望写成能力。

例如：

```
The agent must guarantee the implementation is production ready.
```

这没有用。Agent 不能保证生产就绪。

更好的写法是：

```
Before declaring completion, the agent must report:
- changed files
- verification commands and outputs
- tests not run and why
- residual risk
```

Skill 不能替代真实验证。它只能规定 Agent 必须怎样收集、使用和报告证据。

另一个错误是写太多“永远”“必须”但没有可执行动作。

例如：

```
Always write high-quality maintainable code.
```

这不如：

```
Do not combine refactoring with behavior changes unless explicitly authorized.
If a refactor is required, stop and request a separate refactor step.
```

Skill 里的规则必须能落到行为。

29.13 Skill 什么时候不该写

下面这些不适合做 Skill：

- 只服务一次任务的约束。
- 某个 repo 的具体命令和路径。
- 还没被你实际验证过的流程。
- 过长的工具百科。
- 没有稳定触发条件的经验总结。
- 只是为了显得“工作流化”的 prompt 集合。

尤其是“还没被验证过的流程”。

一个 Skill 会放大你的经验。如果经验是错的，它会稳定地产生错误。

先在真实任务里跑几次，再固化。

29.14 Skill 也要能被验证

一个 Skill 是否有用，不能靠感觉。

你可以拿同一类任务做对照：

示例 29-6 Skill 也要能被验证

```
没有 Skill：
- Agent 是否直接写代码？
- 是否遗漏验证命令？
- 是否没有报告 residual risk？

使用 Skill：
- 是否先读要求的上下文？
- 是否按阶段停住？
- 是否输出了证据？
- diff 是否更小、更可审？
```

如果使用 Skill 后行为没有明显变化，原因通常有三种：

- 触发条件不够具体。
- 停止条件太软。
- 输出要求不可检查。

这时不要继续堆更多说明。先改这三处。

例如 verification Skill 如果没有拦住“测试已通过”这种空结论，就应该把输出要求改成：

```
Do not accept completion unless the response includes:
- command
- result
- failures or explicit no failures
- what was not verified
```

Skill 的效果要看它是否改变 Agent 行为，而不是看它写得多像规范。

29.15 Skill 要接到交付证据上

Skill 最后要落到交付证据。

否则它只是对话里的流程提示。

以 verification Skill 为例，任务完成时至少应该能回答：

示例 29-7 Skill 要接到交付证据上

```
这次 Skill 触发了吗?
它要求的上下文读了吗?
它要求的停止点有没有触发?
它要求的验证命令跑了吗?
结果是什么?
哪些范围没有验证?
```

可以把这段写进 PR 描述、任务记录或本地 verification ledger：

清单 29-8 Skill 要接到交付证据上

```
Workflow evidence:
- skill: verification-ledger-review
- trigger: before completion claim
- changed files checked: yes
- focused command: `npm test -- order-cancel`
- result: pass, 18 tests
- not verified: payment provider sandbox callback
```

这样 reviewer 才能判断 Agent 是按流程做了，还是只说自己做了。

Skill 和交付记录之间缺一层证据，就会回到老问题：AI 说完成，人不知道该不该信。

29.16 一个坏 prompt 到 Skill 的改造过程

坏 prompt：

```
以后调试要认真点，先分析原因，不要乱改。
```

这句话对人有提醒作用，对 Agent 的约束很弱。

改成 Skill 时，要拆成行为：

清单 29-9 一个坏 prompt 到 Skill 的改造过程

```
Use when:
- bug, test failure, CI failure, unexpected runtime behavior.

Before editing:
- reproduce or record why reproduction is unavailable.
- read full error output.
- identify first failing line.
- inspect recent diff.
- trace the value or state source.

Stop if:
- reproduction is missing.
- fix requires changing public API or schema.
```

```
- failure may be baseline and baseline is not confirmed.

Output:
- root cause.
- changed files.
- verification commands and results.
- residual risk.
```

这样 Agent 才知道具体该做什么、什么时候不能继续、最后要交什么证据。

Skill 改造的关键，是把抽象要求翻译成可检查行为。

29.17 Skill 的投入产出怎么判断

写 Skill 需要维护成本，所以要算值不值。

适合写 Skill 的任务，通常满足：

```
高频：一个月会遇到多次。
高风险：漏一步会导致错误 diff、错误文章或错误验证。
跨项目：不是某个 repo 的独有规则。
可检查：结果能用命令、diff、清单或输出结构判断。
```

例如“完成前验证”值得写。因为 AI 经常提前宣布完成，漏掉命令、失败分类和残余风险。

“某个项目的订单取消目录结构”不适合写。它属于项目规则或代码地图。

“某个库的旧 API 用法”也不适合写。它可能很快过期，应该通过官方文档或项目依赖版本核对。

一个实用判断：

```
如果这个规则明天换一个 repo 仍然成立，可以考虑 Skill。
如果换一个 repo 就失效，先放项目规则。
```

29.18 个人程序员可以先做三类 Skill

不要一开始做很多。

我建议从三类开始。

第一类：调试 Skill。

触发测试失败、CI 失败、异常行为。核心流程是先证据、再假设、后修复。

第二类：验证 Skill。

在 AI 声称完成前触发。检查命令、输出、失败分类、残余风险。

第三类：写作或文档 Skill。

如果你经常写技术文档、变更说明、PR 描述，可以把结构、禁用口径、证据要求固化。

这三类最容易产生复用价值，而且不会强行改变项目架构。

29.19 常见错误

错误一：把 Skill 写成 prompt 合集。

Skill 应该有触发、流程、资料、脚本和输出约束，不只是几段提示词。

错误二：把 repo 命令写进跨项目 Skill。

跨项目 Skill 应该说“运行项目指定测试命令”，具体命令由项目规则提供。

错误三：触发条件太泛。

“提高质量”“写更好代码”没有触发价值。

错误四：没有验证方式。

Skill 执行完也要能检查结果。没有输出约束和检查点，工作流会漂。

错误五：把 MCP 和 Skill 混为一谈。

Skill 是方法，MCP 是外部能力接入。两者可以组合，但职责不同。

错误六：过早做 Plugin。

先让单个 Skill 在真实任务中稳定，再考虑打包分发。

29.20 改完后重点看什么

一个 Skill 写完后，不要只看文件结构。

重点看：

- 触发条件是否具体。
- 是否明确什么时候不该用。
- 流程是否能按步骤执行。
- 是否引用了必要资料。
- 是否有脚本或检查方式。
- 输出是否可审查。
- 是否避免写入项目私有命令。
- 是否在真实任务里跑过。

如果一个 Skill 不能让 Agent 按固定步骤留下证据，它就只是换了位置的 prompt。

Skill 的目标，是把你已经证明有效的 AI Coding 方法，沉淀成下一次还能被正确调用的工作流资产。

29.21 常见问题

Skill 是不是越多越好?

不是。Skill 多了会增加触发冲突和维护成本。先做高频、高风险、可复用的流程。

Skill 可以替代 AGENTS.md 吗?

不能。AGENTS.md 适合 repo 规则；Skill 适合跨项目工作流。两者职责不同。

Skill 需要写代码吗?

不一定。最小 Skill 可以只有 `SKILL.md`。当流程需要稳定校验或生成时，再加脚本。

Skill 能保证 AI 不犯错吗?

不能。它只能提高流程一致性，要求 AI 使用证据、模板和校验。最终还要看 diff、测试和人工判断。

什么时候该把 Skill 做成 Plugin?

当你需要分发一组 skills、脚本、MCP 配置或其他资源，并希望跨工具安装时，再考虑 Plugin。

29.22 可复查资料

- OpenAI Codex Agent Skills：https://developers.openai.com/codex/skills〔29-1〕
- Anthropic Claude Code Skills：https://code.claude.com/docs/en/skills〔29-2〕
- Superpowers GitHub 仓库：https://github.com/obra/superpowers〔29-3〕
- Model Context Protocol 文档：https://modelcontextprotocol.io/docs/getting-started/intro〔29-4〕

本章资源

〔29-1〕 https://developers.openai.com/codex/skills
https://developers.openai.com/codex/skills

〔29-2〕 https://code.claude.com/docs/en/skills
https://code.claude.com/docs/en/skills

〔29-3〕 https://github.com/obra/superpowers
https://github.com/obra/superpowers

〔29-4〕 https://modelcontextprotocol.io/docs/getting-started/intro
https://modelcontextprotocol.io/docs/getting-started/intro

第30章　Skill 原理解析：它如何约束 Agent 行为

Skill 不能让 AI 变成不会犯错的程序员。

它能做的，是减少 Agent 的自由发挥空间。

一个没有 Skill 的 Agent，拿到任务后通常靠当前 prompt、上下文、模型习惯和工具反馈行动。它可能先写代码，也可能先读文件；可能先跑测试，也可能直接总结；可能遇到失败就排查，也可能补一个看似合理的判断。

Skill 的作用，是在某类任务出现时，把 Agent 拉进一条固定轨道：

示例 30-1　本章示例

```
什么时候触发
先读什么
先做什么
哪些事不能做
什么时候必须停
要运行什么检查
最后如何报告证据
```

所以 Skill 的原理不是“提示词更长”，而是给 Agent 增加控制点。

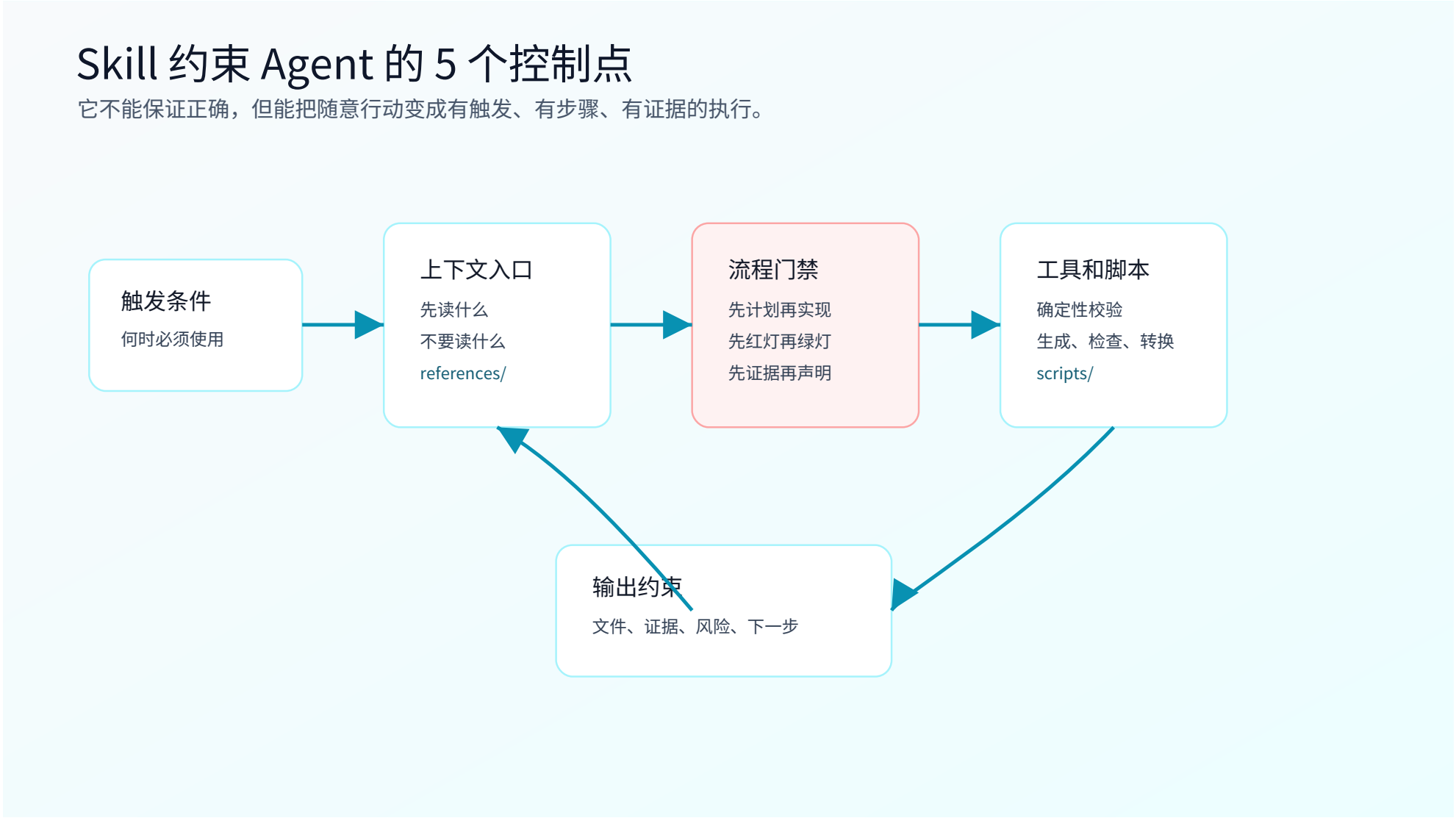


图 30-1　Skill 控制点

30.1 第一个控制点：触发条件

Skill 的第一层约束，是触发。

如果触发条件写得好，Agent 会在正确场景下进入正确流程。

比如调试类 Skill 的触发条件可以是：

```
Use when encountering a bug, test failure, CI failure, or unexpected runtime behavior before
proposing fixes.
```

这里有两个关键信息：

- 场景：bug、测试失败、CI 失败、异常行为。
- 时机：提出修复之前。

这会阻止 Agent 看到报错就直接修。

触发条件写坏，Skill 就会失效。

坏写法：

```
Use this skill for better engineering quality.
```

这个范围太大，几乎等于没写。Agent 不知道何时必须用，也不知道什么时候不用。

Skill 的 description 应该像函数签名一样精确：

```
输入场景是什么
触发时间是什么
任务边界是什么
```

对 AI Coding 来说，触发条件越具体，越能减少误用。

30.2 第二个控制点：上下文入口

Skill 还能规定 Agent 先读什么。

同一个任务，如果上下文入口错了，后面的推理都会偏。

例如 review Skill 可以要求：

```
Before reviewing:
- read the task goal.
- inspect the diff.
- inspect relevant tests.
- do not summarize before listing findings.
```

调试 Skill 可以要求：

```
Before proposing fixes:
- read the full error output.
- identify the first failing line.
- trace the value source.
- inspect recent diff.
```

这不是格式要求，而是上下文控制。

没有这个控制，Agent 可能只读最后一行错误，只看当前打开文件，或者根据常见模式猜原因。

Skill 可以把“先读什么”写成入口，让 Agent 每次先看指定材料。

30.3 第三个控制点：流程门禁

好的 Skill 会设置门禁。

门禁不是口号，而是阶段顺序。

例如 TDD：

```
先写失败测试
确认测试按预期失败
再写最小实现
确认测试通过
再考虑重构
```

例如验证完成：

```
先确定要证明什么
运行命令
读取输出
确认退出码和失败数量
再声明结果
```

例如复杂需求：

```
先写计划
人审计划
只授权当前步骤
完成后停止
```

这些门禁会改变 Agent 的执行节奏。

没有门禁时，Agent 会倾向于一次性完成任务。门禁把任务切成可审查阶段。

Superpowers 这类框架之所以值得研究，不是因为它的文字更强，而是它把 brainstorming、TDD、systematic debugging、verification-before-completion、code review 等环节做成了一组可触发的流程门禁。

这背后的工程价值是：把“希望 Agent 自觉”变成“Agent 每次都要走流程”。

30.4 第四个控制点：参考资料

Skill 可以把稳定知识放进 references。

例如：

```
references/
├── failure-classification.md
├── review-severity.md
└── verification-ledger-template.md
```

这比把所有规则塞进 `SKILL.md` 更好。

入口文件只写流程和路由，细节资料按需读取。

这样有三个好处：

- 减少主指令噪音。
- 让长资料可维护。
- 让 Agent 在需要时读取具体参考，而不是每次吞下全部内容。

但 reference 也有边界。

它适合放稳定模板、检查清单、案例、术语说明。

不适合放当前项目路径、当前任务结论、某次任务里的临时取舍、过时工具能力。

如果 reference 过期，Skill 会稳定地产生过期行为。

30.5 第五个控制点：脚本和确定性校验

Skill 最容易被低估的部分，是 scripts。

能用脚本检查的东西，不要只靠模型自觉。

比如验证记录是否包含必要段落、命令块、通过信号，可以用脚本检查。

比如 Markdown 链接是否能访问，可以用脚本检查。

比如生成的配置是否符合 schema，可以用脚本检查。

模型擅长理解和取舍，但不适合做机械一致性检查。

一个成熟 Skill 应该尽量把确定性检查下沉到脚本：

示例 30-2 第五个控制点：脚本和确定性校验

```
模型负责判断：
- 哪些证据相关。
- 哪些风险需要说明。
- 当前任务是否越界。

脚本负责检查：
- 文件结构是否存在。
- 必要字段是否齐全。
- 命令输出是否包含 pass/fail。
- 生成格式是否有效。
```



图 30-2　Skill 执行栈

所以 Skill 不该只停在一个 `.md` 文件。最小 Skill 可以只有说明，但高价值 Skill 往往会逐步长出脚本。

30.6 第六个控制点：输出约束

Skill 还要规定输出。

没有输出约束，Agent 可能执行了流程，但最后只给你一句：

```
已完成。
```

这对工程没有价值。

验证类 Skill 的输出应该包含：

示例 30-3 第六个控制点：输出约束

```
Completion status
Changed files
Commands run
Command results
Failures
Residual risk
What was not verified
```

代码审查类 Skill 的输出应该包含：

示例 30-4 第六个控制点：输出约束

```
Findings first
Severity
File and line
Why it matters
Suggested fix only when useful
Tests not run
```

调试类 Skill 的输出应该包含：

示例 30-5 第六个控制点：输出约束

```
Reproduction
Evidence
Hypothesis
Minimal verification
Root cause
Fix
Verification
Residual risk
```

输出约束的作用，是让人能审。

AI Coding 不是让 Agent 自己满意，而是让人能判断它做得对不对。

30.7 第七个控制点：停止条件

很多 Skill 只写“应该怎么做”，没有写“什么时候不能继续”。

这会留下一个大洞。

AI Coding 里最危险的行为，往往发生在信息不够时：

```
需求边界不清，继续实现。
复现失败，继续猜原因。
测试命令找不到，继续说已验证。
工具不可用，继续编一个结论。
改动会影响 public API，但没有停下来确认。
```

停止条件要写得硬。

例如调试 Skill 可以写：

```
Stop if:
- the failure cannot be reproduced after two focused attempts.
- the proposed fix changes public API, schema, or persistence behavior not mentioned in the task.
- the failure may be baseline but no baseline evidence has been collected.
```

代码审查 Skill 可以写：

```
Stop and report missing context if:
- no diff is available.
- no task goal is provided.
- line references cannot be produced.
```

停止不是让 Agent 放弃，而是避免它在证据不足时继续扩大错误。

一个 Skill 如果没有 stop conditions，只能算流程建议，还不能算工程约束。

30.8 Skill 不能替代工具权限

Skill 只是指令和资源。

它不能凭空获得权限，也不能绕过工具限制。

如果一个 Skill 说“查询生产数据库”，但当前环境没有只读数据库工具，Agent 不能真的查询。

如果一个 Skill 说“打开浏览器验证”，但当前会话没有浏览器能力，它只能说明无法执行，不能伪造结果。

这点要写清楚。

例如：

```
If the required tool is unavailable, report the missing tool and provide the next best local
verification. Do not claim the unavailable verification was performed.
```

好的 Skill 应该约束证据口径：

```
没有运行，就说没有运行。
不能访问，就说不能访问。
只有推断，就标明推断。
```

否则 Skill 会让错误结论看起来像按流程得出的结果。

30.9 Skill 的执行证据怎么审

审一个 Skill 是否被正确执行，不要看 Agent 有没有提到 Skill 名称。

要看证据。

例如 review Skill 执行得好，输出里应该有：

示例 30-6 Skill 的执行证据怎么审

```
任务目标。
diff 范围。
检查过的文件。
按严重程度排序的 finding。
文件和行号。
没有运行的测试。
open questions。
```

如果输出只是：

```
我已经根据 review skill 检查过，整体没问题。
```

这等于没审。

调试 Skill 也是一样。

有效证据应该包括：

示例 30-7 Skill 的执行证据怎么审

```
复现命令。
完整错误的关键行。
最小假设。
```

```
验证这个假设的动作。
根因和修复之间的对应关系。
修复后的回归命令。
```

没有这些证据，Skill 就没有提供可审查价值。

程序员审 Skill 执行结果时，可以用一个短问题：

```
如果我不相信 Agent 的结论，只看它给出的证据，能不能复查到同样判断？
```

能，就说明 Skill 输出可用。

不能，就让 Agent 补证据，而不是继续下一步。

30.10 Skill 也不能替代人工判断

Skill 可以要求 Agent 先做计划，但计划是否合理仍然需要人审。

Skill 可以要求 Agent 写失败测试，但测试是否覆盖业务风险仍然需要人审。

Skill 可以要求 Agent 输出验证记录，但残余风险是否可接受仍然需要人判断。

所以 Skill 的定位应该是：

```
提高流程一致性
减少遗漏
暴露证据
降低重复沟通成本
```

而不是：

```
自动保证质量
自动替代评审
自动做完交付
```

一旦把 Skill 当作质量保证本身，就会过度信任工具。

30.11 一个反例：坏 Skill 怎么误导 Agent

坏 Skill：

清单 30-8 一个反例：坏 Skill 怎么误导 Agent

```
---
name: complete-task
description: Use for all coding tasks.
---

# Complete Task

Always implement the requested feature cleanly.
Make sure code quality is high.
Run tests if needed.
Summarize what you did.
```

这个 Skill 几乎没有价值。

问题在于：

- 触发条件太宽。
- 没有上下文入口。
- 没有阶段门禁。
- 没有禁止事项。
- 没有具体验证命令。
- 没有输出证据结构。
- “if needed” 给了模型逃避空间。

改成有约束的写法：

清单 30-9 一个反例：坏 Skill 怎么误导 Agent

```
---
name: focused-change-execution
description: Use when executing an approved implementation task that should produce a small
reviewable diff.
---

# Focused Change Execution

## Before Editing

- Read the approved task.
- List allowed files and forbidden changes.
- Identify the focused verification command.
- Stop if required files are outside the approved scope.

## During Editing

- Execute only the current task.
- Do not perform unrelated refactoring.
```

```
- Do not change public API shape unless explicitly authorized.
- Do not start the next task.

## After Editing

- Report changed files.
- Run the focused verification command.
- Report command output.
- State residual risk.
```

这才像一个可执行工作流。

30.12 约束不是越多越好

Skill 里的约束要服务任务风险。

如果一个 Skill 对所有任务都要求计划、TDD、review、全量测试，它会让小任务变慢，也会让 Agent 在不需要的地方加载大量上下文。

更合理的写法是分层触发：

示例 30-10 约束不是越多越好

```
Low risk:
- typo、文案、小配置，不触发完整流程。

Medium risk:
- 单模块行为改动，触发 plan-first 和 focused verification。

High risk:
- 状态机、权限、支付、持久化、跨模块改动，触发 SDD/TDD/debug/review/verification 链路。
```

Skill 的职责是让风险和流程匹配，而不是让所有事情都重流程。

如果一个 Skill 触发太频繁，程序员会开始忽略它。忽略几次后，它就失去约束力。

30.13 Skill 之间会冲突

一个任务可能同时命中多个 Skill。

例如修一个测试失败：

- Debug Skill 想先复现、找根因。
- TDD Skill 想先写失败测试。
- Small diff Skill 想限制文件范围。
- Verification Skill 想完成前检查证据。

这些不一定冲突，但必须有主次。

可以写成：

示例 30-11 Skill 之间会冲突

```
Primary workflow:
- systematic-debugging for root cause.

Supporting constraints:
- focused-change-execution for small diff.
- verification-before-completion before completion claim.
```

不要让多个 Skill 同时定义 source of truth。

任务主线只能有一条：当前是调试、实现、review，还是验证？主线确定后，其他 Skill 只能提供约束，不能抢流程。

这也是为什么项目规则里要写清楚：当用户给出更具体的 spec、plan 或 task 时，以当前 task 为准，Skill 服务于当前 task。

一个实际排序例子：

示例 30-12 Skill 之间会冲突

```
场景：实现过程中测试失败。

Primary:
- systematic-debugging，先确定失败是不是当前 diff 引入。

Supporting:
- focused-change-execution，限制修复文件范围。
- verification-before-completion，修复后要求新鲜验证。

Not primary:
- test-driven-development，此时已经进入调试，不重新启动 TDD 流程。
```

如果主线不清，Agent 可能一边调试、一边补测试、一边重构，最后又回到大 diff。

Skill 越多，越需要主线排序。

30.14 工具不可用时，Skill 也要给出口径

生产任务里经常会遇到工具不可用。

浏览器起不来，日志工具没有权限，MCP server 超时，测试环境缺变量。

这时 Skill 不能只写“使用某某工具验证”。它还要规定不可用时怎么说。

正确输出应该像这样：

清单 30-13 工具不可用时，Skill 也要给出口径

```
Verification not performed:
- required tool: browser preview
- reason: dev server failed to start, missing DATABASE_URL
- fallback used: component tests only
- risk: real browser interaction and network request not verified
```

错误输出是：

```
页面应该可以正常工作。
```

前者能让人判断风险，后者只是猜。

所以验证类 Skill、MCP 类 Skill、前端检查 Skill 都要有 unavailable 分支。

只写成功路径的 Skill，在真实工程里不够用。

30.15 reference 和 scripts 也会过期

Skill 的入口文件会过期，reference 和 scripts 也会。

reference 过期的表现通常是：

```
引用旧工具命令。
保留已经废弃的 API。
把旧项目里的目录当成通用规则。
示例输出和当前工具不一致。
```

scripts 过期的表现更隐蔽：

```
检查字段和新模板不匹配。
把合法的新格式判成失败。
只检查结构，不检查关键证据。
依赖旧 Node/Python 版本。
```

所以成熟 Skill 应该有自检。

最简单的办法是给 Skill 准备两个样例：

```
examples/pass.md
examples/fail.md
```

再让脚本跑：

清单 30-14 reference 和 scripts 也会过期

```
node scripts/check-ledger.js examples/pass.md
node scripts/check-ledger.js examples/fail.md
```

pass 样例应该通过，fail 样例应该失败，并且失败原因要准确。

这不是为了把 Skill 做成软件项目，而是为了防止一个已经过期的检查器继续给 Agent 提供错误约束。

30.16 Skill 的失败模式

Skill 也会失败，而且失败方式很隐蔽。

第一种，触发失败。

任务明明是 CI 失败，调试 Skill 没触发。通常是 description 没写到 CI failure，或者触发时机没写“before proposing fixes”。

第二种，执行失败。

Skill 触发了，但 Agent 仍然直接改代码。通常是停止条件太软，或者输出没有要求复现证据。

第三种，过度触发。

改 typo 也触发完整调试流程。说明 description 太宽，或者没有写 do not use。

第四种，资料过期。

reference 里写了旧工具命令，Agent 稳定执行过期步骤。

第五种，冲突失败。

多个 Skill 同时定义流程，Agent 不知道以哪个为准。

所以维护 Skill 时，要看失败属于哪一类。

不要一失败就加更多文字。先判断是触发、流程、资料、输出还是冲突问题。

30.17 怎么测试一个 Skill 是否有效

Skill 写完后，要用真实任务试。

可以看四个结果：

它有没有在正确场景触发。

如果调试任务没有触发调试流程，description 可能太弱。

它有没有改变 Agent 行为。

如果 Agent 仍然直接修、不复现、不跑验证，流程门禁写得不够硬。

它有没有减少遗漏。

比如每次都能报告 changed files、commands、residual risk。

它有没有制造新负担。

如果小任务也被迫走长流程，Skill 触发范围太大。

不要只测试 happy path。

故意给一个测试失败，让调试 Skill 触发；给一个没有验证记录的完成声明，让验证 Skill 拦住；给一个需要 TDD 的行为改动，看它是否先要求失败测试。

30.18 Skill 的维护成本

Skill 是资产，也是负债。

它需要维护：

- 工具命令变化。
- 官方文档变化。
- 项目实践变化。
- 模板和脚本 bug。
- 触发条件过宽或过窄。

所以不要无限增加 Skill。

建议定期做三件事：

```
删除低频 Skill
合并重复 Skill
把有效规则沉淀到脚本或模板
```

如果一个 Skill 三个月没有触发，或者每次触发都要人工纠正，它可能不值得保留。

30.19 常见错误

错误一：把 Skill 当成绝对约束。

Skill 是指令和工作流，不是形式化证明。最终还要看工具执行和证据。

错误二：只写原则，不写步骤。

“保持高质量”没有用。要写先做什么、禁止什么、怎么验证。

错误三：没有停止条件。

Agent 遇到范围扩大、工具缺失、验证失败时必须知道何时停。

错误四：不使用脚本。

所有检查都靠模型，会增加遗漏。能确定性校验的地方应下沉到脚本。

错误五：输出不可审查。

没有 changed files、commands、results、risk，就无法判断执行质量。

错误六：触发范围太大。

所有任务都触发同一个 Skill，会让流程变重，也会产生上下文噪音。

30.20 改完后重点看什么

评估一个 Skill，不看它写得多完整。

看这些结果：

- 触发是否准确。
- 是否减少 Agent 直接行动。
- 是否把关键阶段拆开。
- 是否要求读取正确上下文。
- 是否使用确定性脚本或命令。
- 是否输出可审查证据。
- 是否明确无法验证时的口径。

好的 Skill 不是让 AI “更聪明”，而是让它少走你已经知道不可靠的路。

30.21 常见问题

Skill 能不能强制 Agent 一定照做？

不能从工程上绝对保证。它能提高触发和执行一致性，但仍要用 diff、测试、脚本和人工审查验证。

Skill 里要不要写很多硬性用语？

只在关键门禁处写硬规则。泛泛的强硬措辞没有意义。硬规则必须能对应行为和检查。

脚本是不是必须有？

不是。最小 Skill 可以没有脚本。但当检查能被程序化时，脚本比模型自查可靠。

Skill 会不会污染上下文？

会。触发条件过宽、references 太大、内容过期，都会污染上下文。所以要精简、分层、按需读取。

Skill 和多 Agent 有什么关系？

Skill 规定怎么做，多 Agent 规定谁去做、在哪个上下文做。多 Agent 任务也需要 Skill 约束，否则只是并行放大错误。

30.22 可复查资料

- OpenAI Codex Agent Skills：https://developers.openai.com/codex/skills〔30-1〕
- Anthropic Claude Code Skills：https://code.claude.com/docs/en/skills〔30-2〕
- Superpowers GitHub 仓库：https://github.com/obra/superpowers〔30-3〕
- Superpowers `test-driven-development`：https://github.com/obra/superpowers/blob/main/skills/test-driven-development/SKILL.md〔30-4〕
- Superpowers `verification-before-completion`：https://github.com/obra/superpowers/blob/main/skills/verification-before-completion/SKILL.md〔30-5〕

本章资源

〔30-1〕 https://developers.openai.com/codex/skills
https://developers.openai.com/codex/skills

〔30-2〕 https://code.claude.com/docs/en/skills
https://code.claude.com/docs/en/skills

〔30-3〕 https://github.com/obra/superpowers
https://github.com/obra/superpowers

〔30-4〕 https://github.com/obra/superpowers/blob/main/skills/test-driven-development/SKILL.md
https://github.com/obra/superpowers/blob/main/skills/test-driven-development/SKILL.md

〔30-5〕 https://github.com/obra/superpowers/blob/main/skills/verification-before-completion/SKILL.md
https://github.com/obra/superpowers/blob/main/skills/verification-before-completion/SKILL.md

第31章　Superpowers 工作流拆解：把工程纪律变成 Skill 链

Superpowers 值得拆开看。

它的价值不在于替你选定唯一框架，而在于把软件开发纪律拆成多个可触发的 Skill：

示例 31-1　本章示例

```
brainstorming
writing-plans
test-driven-development
systematic-debugging
requesting-code-review
verification-before-completion
subagent-driven-development / executing-plans
finishing-a-development-branch
```

这些 Skill 单独看都不神秘。设计前澄清需求、写计划、TDD、系统化调试、代码审查、完成前验证，程序员本来就知道。

难点在于：AI 做任务时，经常跳过这些步骤。

先拆清楚一个基础问题：Superpowers 到底把哪些工程动作拆成了 Skill 链，为什么它和前面的 SDD / OpenSpec 不重复。

SDD / OpenSpec 解决的是“这次需求到底是什么、边界在哪里、任务怎么拆”。Superpowers 解决的是“开发过程中哪些动作不能跳过、失败时怎么收敛、完成前怎么留证据”。一个偏需求合同，一个偏执行纪律。

Superpowers 的思路是把这些步骤变成一条 Skill 链，让 Agent 在不同阶段进入不同工作流，而不是靠一条万能 prompt 从头管到尾。



图 31-1　Superpowers 工作路线

31.1 它解决的是流程漂移

AI Coding 常见的流程漂移是这样的：

示例 31-2 它解决的是流程漂移

```
需求还没澄清 -> 直接实现
设计还没确认 -> 开始改代码
测试还没失败 -> 写实现
调试还没找根因 -> 补判断
review 还没做 -> 继续下一步
验证没运行 -> 声称完成
```

每一步看起来都只是“快一点”。合起来就是不可控。

Superpowers 把这些容易被跳过的动作前置成门禁：

- 做东西前先 brainstorming。
- 有 spec 后再 writing-plans。
- 实现行为前触发 TDD。
- 遇到失败先 systematic-debugging。
- 完成任务后 requesting-code-review。
- 声称完成前 verification-before-completion。

它约束的不是语言能力，而是 Agent 进入下一步前必须经过哪些证据点。

31.2 一个业务需求大概怎么走

还是用“已支付订单取消进入退款流程”的需求。

如果随手让 AI 做：

```
把 paid order cancel 改成 refund_requested，并创建退款请求。
```

它可能一次性改出一个大 diff。

用 Superpowers 思路，流程应该拆开。

第一步，brainstorming 负责把需求变成设计。

要澄清：

示例 31-3 一个业务需求大概怎么走

```
paid 订单当前行为是什么?
pending 和 shipped 是否保持不变?
退款请求失败时订单状态怎么处理?
重复取消是否幂等?
payment provider callback 是否在范围内?
API response shape 是否允许变化?
```

输出不是代码，而是设计。

如果项目已经有 OpenSpec change、issue 或设计文档，brainstorming 不应该另起一套需求。它更适合用来检查缺口：哪些业务状态没说清楚，哪些异常路径没确认，哪些兼容性风险没有进入当前需求合同。

第二步，writing-plans 把设计拆成任务。

计划里要写：

```
Task 1: Add failing tests
Task 2: Add paid -> refund_requested transition
Task 3: Wire refund request creation
Task 4: Add failure handling
Task 5: Run broader verification
```

每个任务要有文件、步骤、命令和预期结果。

如果前面已经有 `tasks.md`，writing-plans 也不是重新发明任务列表，而是把任务落到更细的执行步骤、文件路径、测试命令和预期输出。

第三步，执行任务。

如果平台支持 subagent，可以每个任务独立执行并审查。没有 subagent，也可以按 executing-plans 的方式在当前会话里分批执行。

关键不是“必须用 subagent”，而是每个任务都要小、可审、可验证。

第四步，TDD 和 debugging 在执行中触发。

Task 1 必须先写失败测试并确认红灯。

如果测试失败不是预期红灯，就进入 systematic debugging，先找根因。

第五步，review 和 verification 收尾。

每个任务之后看 diff。完成前跑新鲜验证命令，不能只靠上一次输出或 Agent 总结。

这条路线比“一个 prompt 做完”慢一点，但它把风险拆开了。

到这里为止，仍然只是工作流拆解。真正落到项目里，还必须看到真实文件、测试、diff、验证输出和 review 风险。否则“实战”两个字就会变成概念演示。

31.3 Superpowers 的几个关键 Skill 怎么分工

brainstorming：把想法变成设计。

它不是头脑风暴聊天，而是防止 AI 在需求不清时直接实现。对个人程序员来说，可以把它理解成“需求和设计闸门”。

用在：

- 新功能。
- 行为变化。
- 不确定方案。
- 需要权衡边界的改动。

不适合：

- 明确 typo。
- 单行配置。
- 已有 tasks 里的小步骤。

writing-plans：把设计拆成执行计划。

它强调 exact file paths、具体步骤、验证命令、bite-sized tasks。这和我们前面讲的 `tasks.md` 一致。

它能防止：

- 计划只有大标题。
- 任务里没有验证。
- AI 一次性吞掉整个需求。

test-driven-development：用红灯约束实现。

它要求先写失败测试，确认失败，再写最小实现。

这对 AI 很关键，因为 AI 很容易先写实现，再让测试配合实现。

systematic-debugging：遇到失败先找根因。

它要求复现、读错误、查最近改动、追数据流、提出单一假设，再修。

这能防止 AI 看到错误就补空值判断。

requesting-code-review：引入独立审查。

它强调 reviewer 只看工作产物和需求，不继承当前会话历史。这个限制能减少当前会话带来的偏见。

当前会话里的 Agent 已经带着自己的推理路径和偏见，独立 review 可以减少“自己审自己”的盲区。

verification-before-completion：完成前必须有证据。

它的核心是：没有新鲜验证命令，就不能声称通过。

这是防止 AI 过早宣布完成的关键 Skill。

31.4 它的边界要说清楚

Superpowers 不会替你判断需求是否正确。

它也不会保证生成的计划一定合理。

它更不会保证测试覆盖了真实业务风险。

它能做的是把过程变得更难跳过：



图 31-2　Superpowers 风险边界

适合交给 Skill 链约束的是：

- 阶段顺序。
- 任务粒度。
- TDD 红绿步骤。
- 调试证据循环。
- review 触发。
- 完成前验证。

仍然需要程序员判断的是：

- 需求本身是否正确。
- 设计取舍是否值得。
- 测试是否真的覆盖风险。
- review 反馈是否成立。
- 残余风险是否可接受。

如果你把 Superpowers 当成自动驾驶，就会过度信任。

如果把它当成“工程纪律执行器”，它适合个人工作流。

31.5 使用时不要全盘照搬

Superpowers 的默认工作流很强，但不一定适合每个任务。

对个人程序员，建议先借鉴三点。

第一，借鉴阶段门禁。

```
设计通过前不实现
计划通过前不改代码
测试红灯前不写实现
验证前不声称完成
```

第二，借鉴小任务。

每个任务要能在几分钟内完成、单独验证、单独 review。

第三，借鉴证据输出。

不要让 AI 只说“完成”。让它报告 changed files、commands、results、residual risk。

你不必一开始就完整引入所有 Skill，也不必所有任务都走完整链路。

小任务用轻流程。

复杂行为改动用完整链路。

31.6 一个可执行的使用路线

如果你已经安装了 Superpowers，可以这样用在复杂需求上。

先给任务，但不让它实现：

示例 31-4 一个可执行的使用路线

```
我们要修改 paid order cancellation 行为：
- paid cancel enters refund_requested.
- exactly one refund request is created.
- duplicate cancel is idempotent.
- refund request failure keeps order paid.
- payment provider callback is out of scope.

先不要改代码。
请按设计和计划流程推进。
```

然后看它是否先澄清和设计。

设计通过后，再让它写 plan。

计划应该包含：

```
Task 1: failing tests
Task 2: status transition
Task 3: refund request creation
Task 4: failure handling
Task 5: verification
```

每个 task 要有：

```
Files
Steps
Commands
Expected result
Stop conditions
```

执行时不要说“全部做完”。

指令只放开一个任务：

```
只执行 Task 1。
完成后停止。
不要修改实现代码。
运行指定测试并报告红灯结果。
```

Task 1 通过 review 后，再进入 Task 2。

这样就能把 Superpowers 思路落到个人工作流里。

31.7 失败时不要跳过调试链路

Superpowers 这类 Skill 链最容易在失败时被绕开。

常见情况是：

```
Task 1 写了失败测试。
Task 2 开始实现。
测试失败了，但失败信息和预期不一样。
Agent 说“这可能是测试环境问题”，然后继续改实现。
```

这时应该切到 systematic-debugging。

处理顺序是：

示例 31-5 失败时不要跳过调试链路

```
读取完整错误。
确认是预期红灯、实现缺口、基线失败还是环境问题。
如果无法分类，停止实现。
提出一个假设。
用最小命令或文件读取验证假设。
分类清楚后再继续原任务。
```

比如订单取消测试失败：

```
Expected: refund_requested
Received: cancelled
```

这可能是预期红灯，可以继续实现。

如果失败变成：

```
Cannot connect to test database
```

这就不是业务红灯。继续写实现只会污染证据。

Skill 链不是线性流水线。遇到非预期失败时，要允许流程回到调试，而不是为了完成计划继续往前走。

31.8 review 反馈要进入决策

requesting-code-review 的价值不是让另一个 Agent 说几句意见。

它的价值是让当前会话的惯性停下来。

但 review 反馈不能照单全收。

处理 review 的方式应该是：

```
finding 有文件、行号和可复现输入：优先处理。
finding 是需求理解差异：回到 spec 或 plan 核对。
finding 只是风格偏好：看项目规则是否有约束。
finding 会扩大范围：拆成后续任务，不混进当前 diff。
```

例如 reviewer 提出：

```
OrderService.cancelPaidOrder 里创建 refund request 后没有处理 duplicate cancel。
```

如果需求里明确要求幂等，这就是当前任务问题。

如果 reviewer 又提出：

```
可以顺手重构整个 OrderService 状态机。
```

这可能是好建议，但不应该混进当前交付。

Superpowers 的工程纪律不是“谁提反馈就改谁”，而是让反馈有证据、有归属、有边界。

31.9 不要把 Superpowers 讲成必选框架

Superpowers 值得研究，但不是程序员 AI Coding 的唯一入口。

你可以不用它，仍然采用同样的纪律：

示例 31-6 不要把 Superpowers 讲成必选框架

```
需求不清先澄清。
设计不清先计划。
行为变化先测试。
失败先排障。
完成前跑验证。
复杂任务要独立 review。
```

Superpowers 的优势是把这些纪律打包成已经写好的 Skill 链，少了自己从零整理的成本。

它的成本是：你要理解每个 Skill 的触发条件、适用边界和输出要求。如果只是安装后盲目执行，也可能让小任务变重，或者让当前任务被不合适的流程包住。

实际使用时先选最有价值的几个：

- systematic-debugging
- test-driven-development
- writing-plans
- verification-before-completion
- requesting-code-review

这几个覆盖了 AI Coding 最常见的失控点。

等你熟悉后，再考虑 subagent-driven-development、executing-plans、finishing-a-development-branch 这类更完整的协作流程。

31.10 什么时候不适合用完整链路

下面这些任务不适合重流程：

- 改错别字。
- 调整一行配置。
- 改一个明确的测试断言文案。
- 更新 README 中过期命令。
- 一次性探索命令输出。

如果所有任务都强制 brainstorming、plan、TDD、review，会让工作流变成负担。

可以直接定义触发条件：

示例 31-7 什么时候不适合用完整链路

```
Use full Superpowers-style workflow when:
- behavior changes.
- multiple modules may be touched.
- tests need to be added.
- bug root cause is not obvious.
- public API, payment, auth, persistence, jobs, or state machine code is involved.
```

这样既保留纪律，也不拖慢小改动。

31.11 怎么检查 Superpowers 是否真的发挥作用

不要只看它有没有被安装。

看结果：

- 它是否在实现前澄清了需求。

- 是否写了可执行 plan。
- 是否把任务拆小。
- 是否先写失败测试。
- 遇到失败是否先排障。
- 是否在任务后做 review。
- 声称完成前是否运行了验证命令。
- 是否报告没有验证的范围。

如果这些都没有发生，只是工具名出现在环境里，那它没有真正进入工作流。

31.12 和 OpenSpec / Spec Kit 怎么搭配

OpenSpec 和 Spec Kit 更偏规格和开发流程组织。

Superpowers 更偏 Agent 执行纪律。

可以这样组合：

示例 31-8 和 OpenSpec / Spec Kit 怎么搭配

```
OpenSpec：
- 管 specs、changes、delta spec、archive。

Spec Kit：
- 管 constitution、spec、plan、tasks、implement 这条 SDD 流程。

Superpowers：
- 管 Agent 在设计、计划、TDD、调试、review、验证时是否按纪律执行。
```

不要在同一次任务里给 AI 三套互相竞争的 artifact。

如果当前任务以 OpenSpec 为主，就让 Superpowers 的执行纪律服务于 OpenSpec change：

```
以 openspec/changes/<change>/tasks.md 为任务来源。
执行时遵守 TDD、debug、review、verification 流程。
```

如果当前任务以 Spec Kit 为主，就让 Superpowers 服务于 Spec Kit 生成的 tasks。

主线只能有一条。

31.13 一个任务里的 Skill 链报告

使用 Superpowers 思路后，最终应该能留下类似报告：

清单 31-9 一个任务里的 Skill 链报告

```
Workflow evidence:

- Brainstorming / design:
  - clarified paid, pending, shipped and duplicate cancel behavior.

- Plan:
  - 5 tasks created, each with files and commands.

- TDD:
  - paid cancel and duplicate cancel tests failed before implementation.

- Debug:
  - no unexpected failure; expected red tests classified.

- Review:
  - reviewed diff for API response shape, callback changes and migration.

- Verification:
  - `npm test -- order-cancel`: PASS
  - `npm test -- order`: PASS
  - residual risk: real payment provider callback not verified locally.
```

这份报告不需要很长。它说明 Skill 链确实改变了执行路径，而不是只在对话里喊了几个工具名。

31.14 怎样把它迁移到自己的项目规则

如果你不想直接安装整套 Superpowers，也可以把关键纪律写进项目规则。

例如：

清单 31-10 怎样把它迁移到自己的项目规则

```
## Complex Changes

For behavior changes involving state, persistence, auth, payment, jobs, or public APIs:
- produce a plan before editing.
- split into reviewable tasks.
- write failing tests before implementation when practical.
- use systematic debugging for unexpected failures.
- run focused verification before completion.
- report commands, results, and residual risks.
```

这不是复制 Superpowers，而是把它的工程精神迁移到自己的 repo。

等你发现这些规则在多个项目里反复出现，再把它们沉淀成个人 Skill。

迁移时不要带入工具专属话术。项目规则要写你真正执行的命令、目录和验证方式。

31.15 对个人程序员最有价值的三条纪律

如果只能从 Superpowers 学三件事，我会选这三条。

第一，完成前验证必须是新鲜的。

不能用“之前跑过”或“应该通过”。完成前要跑当前命令，读取输出，并报告未验证内容。

第二，调试前先找根因。

遇到失败时，先复现、读堆栈、看 diff、提单一假设。不要先补判断。

第三，计划要能交给别人执行。

一个 plan 如果只有你当前会话知道上下文，就不够。它应该写清文件、步骤、命令、预期结果和停机点。

这三条足以覆盖大多数 AI Coding 事故：

```
过早宣布完成。
猜修复。
大 diff。
```

工具可以换，但这三条纪律不该换。

31.16 常见错误

错误一：以为装了就会自动变稳。

安装只是开始。要看 Agent 是否真的按 Skill 链执行。

错误二：把所有任务都走完整链路。

小任务重流程会降低效率。要定义触发条件。

错误三：跳过计划审查。

计划是后续执行的合同。计划错了，执行越认真越危险。

错误四：把 TDD 当形式。

没有红灯证据，就不能证明测试约束了旧实现。

错误五：忽略 review 反馈。

独立 review 的意义在于打断当前会话的惯性。不能只挑喜欢的反馈看。

错误六：没有新鲜验证就说完成。

完成前必须跑当前命令、读输出、报告结果。

31.17 改完后重点看什么

一次 Superpowers 风格任务完成后，看这些结果：

- 设计是否明确。
- plan 是否具体到文件、步骤、命令。
- 每个 task 是否小。
- 测试是否经历红绿过程。
- 调试是否有根因证据。
- review 是否有独立视角。
- completion claim 是否有新鲜验证。
- 残余风险是否写清。

如果这些证据完整，说明 Skill 链确实在工作。

如果只有“工具已安装”和“AI 说完成”，那还不够。

31.18 常见问题

Superpowers 是否必须使用？

不是。你可以不用它，但应该借鉴它背后的工程纪律：设计、计划、小步、TDD、调试、review、验证。

没有 subagent 支持还能用吗？

可以用 inline execution，但独立 review 和任务隔离会弱一些。复杂任务上，subagent 或独立会话确实更有价值。

它和自己写的 Skill 冲突怎么办？

一次任务只保留一条主流程。项目规则里写清当前任务以哪个 plan、spec 或 change 为 source of truth。

它会不会太重？

对小任务会。对复杂行为改动、bug 修复和跨模块需求，它能减少返工。

能不能只借鉴 Superpowers 的工作流？

可以。重点看它如何拆 Skill、如何设置门禁、如何要求证据，而不是照抄所有话术。

31.19 可复查资料

- Superpowers GitHub 仓库：https://github.com/obra/superpowers〔31-1〕

- Superpowers `writing-plans`：https://github.com/obra/superpowers/blob/main/skills/writing-plans/SKILL.md〔31-2〕
- Superpowers `test-driven-development`：https://github.com/obra/superpowers/blob/main/skills/test-driven-development/SKILL.md〔31-3〕
- Superpowers `systematic-debugging`：https://github.com/obra/superpowers/blob/main/skills/systematic-debugging/SKILL.md〔31-4〕
- Superpowers `requesting-code-review`：https://github.com/obra/superpowers/blob/main/skills/requesting-code-review/SKILL.md〔31-5〕
- Superpowers `verification-before-completion`：https://github.com/obra/superpowers/blob/main/skills/verification-before-completion/SKILL.md〔31-6〕

本章资源

〔31-1〕 https://github.com/obra/superpowers
https://github.com/obra/superpowers

〔31-2〕 https://github.com/obra/superpowers/blob/main/skills/writing-plans/SKILL.md
https://github.com/obra/superpowers/blob/main/skills/writing-plans/SKILL.md

〔31-3〕 https://github.com/obra/superpowers/blob/main/skills/test-driven-development/SKILL.md
https://github.com/obra/superpowers/blob/main/skills/test-driven-development/SKILL.md

〔31-4〕 https://github.com/obra/superpowers/blob/main/skills/systematic-debugging/SKILL.md
https://github.com/obra/superpowers/blob/main/skills/systematic-debugging/SKILL.md

〔31-5〕 https://github.com/obra/superpowers/blob/main/skills/requesting-code-review/SKILL.md
https://github.com/obra/superpowers/blob/main/skills/requesting-code-review/SKILL.md

〔31-6〕 https://github.com/obra/superpowers/blob/main/skills/verification-before-completion/SKILL.md
https://github.com/obra/superpowers/blob/main/skills/verification-before-completion/SKILL.md

第32章　自定义 Skill：把你的经验固化成个人能力

程序员用 AI Coding 一段时间后，通常会有一类很烦的问题。

你已经知道怎么让 AI 稳一点，但每次都要重新提醒。

比如：

```
先读 diff，不要直接总结。
先复现失败，不要猜原因。
改动不要跨模块扩大。
跑完命令再说结果。
引用官方文档，不要靠记忆写工具能力。
```

这些话说一两次还行。一个月后，同一批提醒会在不同项目、不同工具、不同会话里反复出现。

自定义 Skill 适合处理这种重复。

自定义 Skill 的目标不是写一份更长的提示词，而是把你已经验证过的个人工程经验，固化成 Agent 能稳定触发、能按步骤执行、能留下证据的工作流。

Skill 写得好，它会变成你的个人能力放大器。

Skill 写得差，它只是换了位置的 prompt。

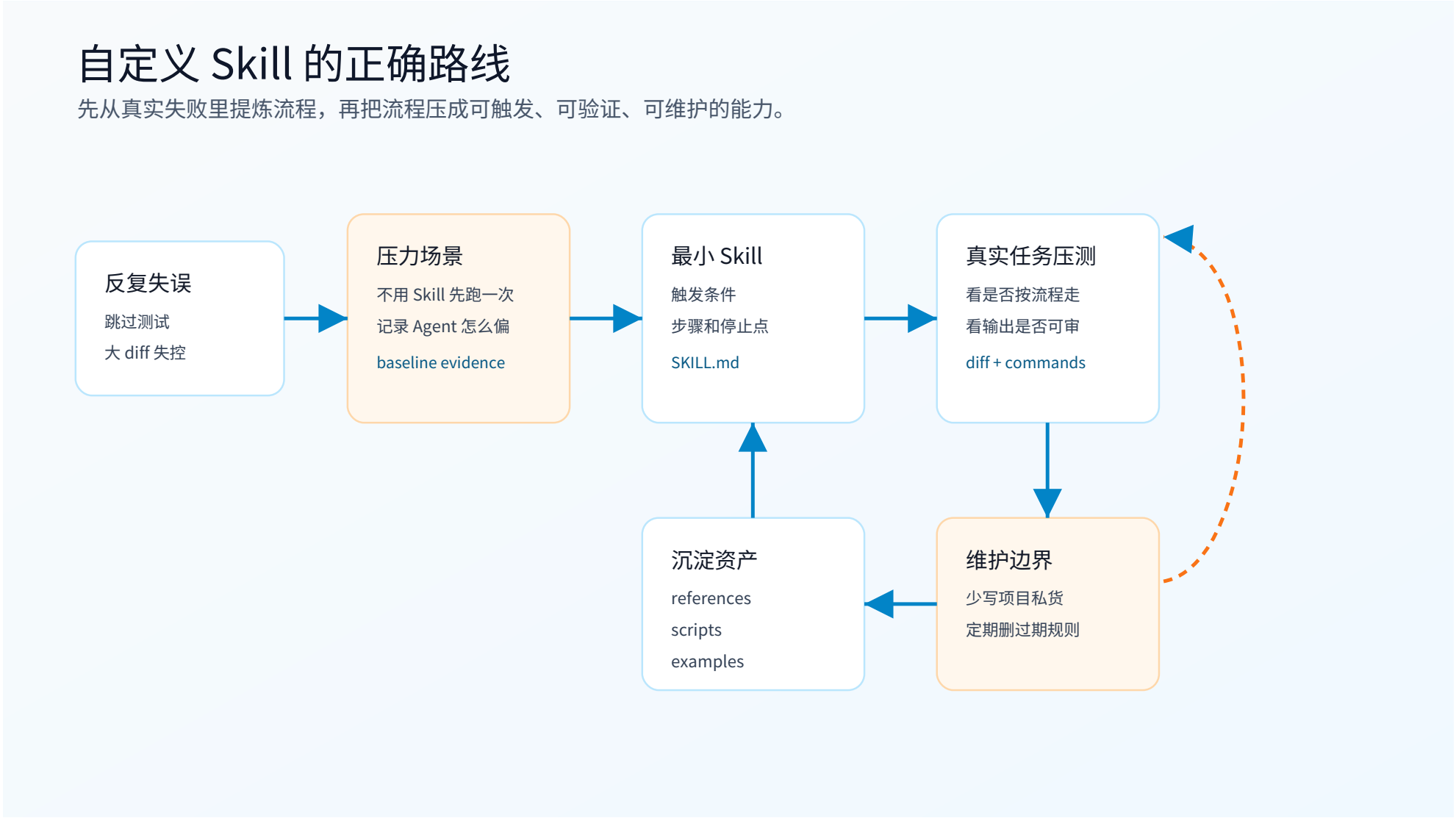


图 32-1　自定义 Skill 路线

32.1 先从真实失误开始，不要从模板开始

很多人写 Skill 的第一步是找模板。

这会把路带偏。

模板只能告诉你文件长什么样，不能告诉你这个 Skill 到底要解决什么工程问题。

一个值得写成 Skill 的流程，应该来自真实反复出现的失误。

比如你经常遇到这样的场景：

示例 32-1 先从真实失误开始，不要从模板开始

```
需求：修复订单取消时库存回滚异常。

AI 的常见行为：
- 直接改 cancelOrder。
- 没看库存扣减入口。
- 没看幂等表。
- 没跑失败用例。
- 看到测试失败后说“可能是已有问题”。
```

这不是一个“订单 Skill”。

订单逻辑属于当前项目，不该写进跨项目 Skill。

可复用的是调试流程：

示例 32-2 先从真实失误开始，不要从模板开始

```
遇到 bug 时，必须先复现。
必须定位第一个失败信号。
必须追数据来源。
必须检查最近 diff。
必须提出单一假设。
必须用最小命令验证。
修复后必须跑回归。
最后必须报告残余风险。
```

这才适合沉淀。

写 Skill 前，先把失误写成一段事实记录：

清单 32-3 先从真实失误开始，不要从模板开始

```
## Repeated Failure

- Task type: bug fix
```

```
- Symptom: agent proposes a fix before reproducing the failure
- Consequence: patch hides the symptom but leaves inconsistent state
- Desired behavior: reproduce, isolate, fix root cause, verify regression
```

这段记录不是正文给读者看的东西，而是你写 Skill 时的设计输入。

没有真实失误，Skill 会写成空泛流程。

32.2 Skill 的本质是行为合同

自定义 Skill 要回答四个问题：

```
什么时候用？
先读什么？
按什么顺序做？
怎么证明做完？
```

这四个问题比“怎么写得全面”重要。

Skill 不是知识库。它更像一个小型行为合同。

例如代码审查 Skill 的合同可以是：

```
触发：用户要求 review、检查 PR、看 diff。
输入：任务目标、diff、相关测试、项目规则。
流程：先列问题，按严重程度排序，每个问题必须有文件和行号。
输出：findings、open questions、test gaps，不把摘要放在问题前。
```

调试 Skill 的合同可以是：

```
触发：bug、测试失败、CI 失败、异常行为。
输入：完整错误、复现步骤、最近 diff、相关日志。
流程：复现 -> 定位信号 -> 单一假设 -> 验证 -> 修根因 -> 回归。
输出：root cause、fix、verification、residual risk。
```

写作检查 Skill 的合同可以是：

```
触发：准备发布技术文章。
输入：正文、来源链接、配图、禁用口径、发布平台要求。
流程：事实核对 -> 结构检查 -> 语气清理 -> 风险审查 -> 文件落盘。
输出：修改摘要、仍未验证的事实、最终文件路径。
```

这些合同不需要很长，但必须可执行。

如果一个 Skill 只能写出“提高质量”“认真检查”“保持专业”，它没有约束力。

32.3 一个可迁移的 Skill 骨架

不同工具对 Skill 的目录和元数据支持不完全一样。OpenAI Codex Skills、Claude Code Skills 和 Superpowers 的实现细节不同，但核心结构接近：入口文件负责触发和流程，辅助文件负责按需提供资料，脚本负责确定性检查。

可以从这个结构开始：

示例 32-4 一个可迁移的 Skill 骨架

```
my-debug-workflow/
├── SKILL.md
├── references/
│   ├── failure-classification.md
│   └── verification-ledger.md
└── scripts/
    └── check-ledger.js
```

最小版本只需要 `SKILL.md`。

不要一开始就把目录做满。

先写能改变 Agent 行为的入口文件：

清单 32-5 一个可迁移的 Skill 骨架

```
---
name: systematic-bug-fix
description: Use when fixing a bug, failed test, CI failure, or unexpected runtime behavior before
proposing code changes.
---

# Systematic Bug Fix

## Purpose

Use this workflow to prevent speculative fixes. The goal is to reproduce the failure, isolate the
cause, make the smallest correct change, and verify the result.

## Before editing

1. Read the full error or bug report.
2. Identify the first failing signal.
3. Find the code path that produces the signal.
4. Inspect the current diff.
5. State one hypothesis and the command or file read that will test it.

## Hard stops

- Do not edit code before a reproduction path is known.
- Do not add broad fallback logic without showing the failing data path.
- Do not claim the failure is unrelated without baseline evidence.
```

```
## Verification

Report:

- changed files.
- commands run.
- exact pass or fail result.
- risk that remains.
```

这个 Skill 已经能用。

它还不完美，但它有触发、步骤、停止点和输出。

很多团队和个人写不好 Skill，是因为一开始就想写成一本手册。Agent 真正需要的不是长篇解释，而是清楚的执行路线。

32.4 description 要像触发器，不要像广告语

Skill 最容易写坏的是 `description`。

它决定 Agent 什么时候加载 Skill。

坏写法：

清单 32-6 description 要像触发器，不要像广告语

```
description: Improve coding quality with a professional engineering workflow.
```

这个描述没有实际触发价值。

好写法：

清单 32-7 description 要像触发器，不要像广告语

```
description: Use when fixing a bug, failed test, CI failure, or unexpected runtime behavior before
proposing code changes.
```

差别在三个地方：

```
任务类型明确：bug、failed test、CI failure。
触发时机明确：before proposing code changes。
行为意图明确：不是泛泛提高质量，而是阻止先改代码。
```

另一个例子，代码审查 Skill。

坏写法：

清单 32-8 description 要像触发器，不要像广告语

```
description: Use this to perform high-quality code review.
```

好写法：

清单 32-9 description 要像触发器，不要像广告语

```
description: Use when reviewing a code diff, pull request, or implementation against a stated
requirement, before summarizing the work.
```

这里要看“before summarizing the work”。

它能阻止 Agent 先写一段礼貌摘要，再把问题埋到后面。

触发条件要写成可识别场景，不要写成价值主张。

32.5 流程要短，但停止点要硬

Skill 不是越详细越好。

长流程有两个问题：

```
Agent 不一定能稳定执行。
你自己后面也不愿意维护。
```

一个好的 Skill 应该把篇幅花在停止点上。

停止点就是不满足条件时不能继续。

例如：

清单 32-10 流程要短，但停止点要硬

```
## Stop conditions

Stop and ask for clarification when:

- the requested behavior conflicts with repository rules.
- the failing command cannot be reproduced after two attempts.
- the task requires credentials or production access that are not available.
- the implementation would require changing public API behavior not mentioned in the task.
```

这比写十段“注意安全”“保持谨慎”有效。

AI Coding 里最危险的不是 Agent 不会写代码，而是它在不该继续时继续。

Skill 要把这些点写死。

32.6 references 放稳定资料，不放临时结论

入口文件应该短。

细节资料可以放到 `references/`。

例如代码审查 Skill 可以有：

```
references/
├── severity-rubric.md
├── test-gap-patterns.md
└── review-output-examples.md
```

调试 Skill 可以有：

```
references/
├── failure-classification.md
├── log-reading-guide.md
└── rollback-risk.md
```

但 reference 不是垃圾桶。

不要把这些内容放进去：

```
某次需求讨论。
某个项目的专属路径。
还没验证的工具能力。
一次失败后的情绪判断。
某次任务里的临时取舍。
```

reference 一旦被 Skill 反复读取，就会把旧结论放大成长期行为。

如果资料只对当前 repo 有效，优先放项目规则。

如果资料只对当前任务有效，放任务上下文。

只有跨项目稳定复用的资料，才值得进入 Skill。

哪些经验该写进 Skill

Skill 不是所有规则的仓库。边界越清楚，触发越稳定，维护成本越低。

适合

跨项目流程

代码审查顺序

系统化调试步骤

验证记录检查

文章发布前复核

触发清楚 + 输出可审

谨慎

项目规则

测试命令

目录约定

业务边界

发布分支策略

优先放 AGENTS.md / CLAUDE.md

不要

临时结论

某次需求的取舍

过时工具能力

聊天里的口头约定

无法验证的偏好

留在任务上下文或删掉

图 32-2　Skill 边界

32.7　和项目 AGENTS.md 怎么配合

个人 Skill 最常见的落地方式，是和项目规则一起工作。

项目规则回答：

```
这个 repo 怎么跑？
哪些目录不能动？
测试命令是什么？
代码风格和架构边界是什么？
发布或迁移有什么约束？
```

个人 Skill 回答：

```
遇到 bug 怎么排障？
做行为变更怎么走 TDD？
完成前怎么核验证据？
review 时怎么输出问题？
```

两者不要互相替代。

例如一个验证 Skill 可以写：

清单 32-11　和项目 AGENTS.md 怎么配合

```
Before completion:
- read repository instructions for test commands.
- choose the focused command that matches the changed surface.
- if no command is documented, inspect project scripts and report the basis.
- do not invent a command and treat it as official.
```

项目 `AGENTS.md` 则写：

清单 32-12　和项目 AGENTS.md 怎么配合

```
## Test Commands

- Unit tests: `npm test`
- CLI focused tests: `npm test -- cli`
- Lint: `npm run lint`
```

这样换项目时，Skill 不需要改。项目只需要维护自己的命令。

如果把 `npm test -- cli` 写进个人 Skill，到了下一个 Java、Go 或 Python 项目就会污染上下文。

一个简单规则：

```
命令、路径、模块边界，留在 repo。
行为顺序、停止条件、证据格式，放进 Skill。
```

这个边界清楚，个人 Skill 才能长期复用。

32.8 scripts 只做确定性工作

Skill 里可以带脚本，但不要为了显得高级而写脚本。

脚本适合做确定性检查：

示例 32-13 scripts 只做确定性工作

```
Markdown 是否包含必要段落。
JSON 是否符合 schema。
链接是否能访问。
验证记录是否包含命令和结果。
文件名是否符合约定。
生成资产是否存在。
```

脚本不适合替代工程判断：

```
这个需求该不该做。
这个设计是否合理。
这个风险是否可接受。
这个测试是否覆盖真实业务。
```

在 AI Coding 里，一个实用的原则是：

```
模型判断相关性和取舍。
脚本检查结构和一致性。
```

例如验证记录检查可以用脚本做：

清单 32-14 scripts 只做确定性工作

```
node src/cli.js verify-ledger --file examples/verification-ledger.pass.md
```

输出应该像这样：

```
Verification ledger looks usable: required sections, command block, and pass signal found.
```

失败时要明确缺什么：

清单 32-15 scripts 只做确定性工作

```
node src/cli.js verify-ledger --file examples/verification-ledger.fail.md
```

```
Verification ledger is incomplete.
Missing sections: Change Summary, Verification Commands, Evidence, Residual Risk
Missing command block with the verification command.
```

这类脚本适合放进 Skill，因为它把“请认真检查”变成了可执行动作。

但不要把业务实现塞进 Skill 的脚本。Skill 是工作流资产，不是项目逻辑的藏身处。

32.9 先压测，再扩展

写完 `SKILL.md` 后，不要立刻觉得它可以长期使用。

先拿真实任务压测。

压测方式很简单：

```
选一个过去容易失败的任务。
先用普通 prompt 让 Agent 做一遍，记录它怎么偏。
再显式调用 Skill 做一遍，看行为是否变化。
比较两次输出中的证据、步骤、停止点和验证。
```

如果没有变化，说明 Skill 没有抓住关键失误。

比如你写了调试 Skill，但 Agent 还是直接改代码，通常是因为：

```
触发条件没写到“修复前”。
停止点太软。
流程里没有要求复现证据。
输出没有要求命令和结果。
```

这时不要继续加长解释。

先改触发和停止点。

Skill 的迭代顺序应该是：

示例 32-16 先压测，再扩展

```
触发条件
停止点
执行步骤
```

```
输出证据
reference
script
```

很多人反过来做，先写一堆 reference 和脚本，结果 Agent 还是不按流程走。

32.10　不要把 Skill 写成项目规则

项目规则要留在项目里。

例如：

清单 32-17　不要把 Skill 写成项目规则

```
## Test Commands

- Unit tests: `npm test`
- Type check: `npm run typecheck`
- E2E: `npx playwright test`
```

这不应该写进跨项目 Skill。

跨项目 Skill 应该写：

清单 32-18　不要把 Skill 写成项目规则

```
Run the focused test command defined by the repository instructions. If no command is documented,
inspect package scripts or ask before inventing one.
```

这样 Skill 才能迁移。

项目规则会变，Skill 的流程不一定要变。

如果把具体项目命令写进个人 Skill，你下次在另一个 repo 里使用它，就会把旧项目的命令带过去。

这类污染不容易立刻暴露，但会把旧项目的约束带进新项目。

32.11　不要把 Skill 写成工具百科

自定义 Skill 里也不要写太多工具介绍。

比如你写一个 OpenSpec 工作流 Skill，不需要解释 OpenSpec 的所有概念。

它应该写：

```
什么时候进入 proposal。
什么时候写 design。
tasks 什么时候可以执行。
archive 前必须检查什么。
哪些命令要以当前官方文档为准。
```

工具细节如果变化快，Skill 里只保留核对路径：

清单 32-19 不要把 Skill 写成工具百科

```
Before writing tool-specific commands, verify the current official docs:

- OpenSpec docs: https://openspec.dev/
- OpenSpec repo: https://github.com/Fission-AI/OpenSpec/
```

不要把时效性强的命令永久写死，除非你愿意维护。

Skill 稳定，不代表里面的信息永远正确。

32.12 一个个人 Skill 的实战模板

下面这个模板适合迁移到自己的工作流里。

它不是工具专属模板，而是写 AI Coding Skill 时可以复用的骨架：

清单 32-20 一个个人 Skill 的实战模板

```
---
name: <short-action-name>
description: Use when <specific task type>, <specific trigger timing>, <before/after a risky
action>.
---

# <Skill Name>

## Goal

State the workflow outcome in one or two sentences.

## Use when

- Concrete task type.
- Concrete signal.
- Concrete timing.

## Do not use when

- One-off task.
- Repo-specific rule.
- User already supplied a more specific plan.
```

```
## Required context

Before acting, read:

1. Current user request.
2. Relevant repo instructions.
3. Relevant diff or files.
4. Relevant test or command output.

## Workflow

1. Restate the task boundary.
2. Gather evidence.
3. Propose the smallest valid next step.
4. Execute only that step.
5. Run the focused verification.
6. Report evidence and remaining risk.

## Stop conditions

Stop when:

- task boundary is unclear.
- required source cannot be found.
- verification cannot be run.
- change would exceed requested scope.

## Output

Return:

- changed files.
- commands run.
- result.
- risks.
- next suggested step.
```

这个模板有几个特点。

它不追求华丽。

它把输入、流程、停止点和输出写清楚。

它没有写死项目命令。

它允许 Agent 根据 repo 规则找具体命令。

这才是可复用 Skill 的基础。

32.13 什么时候需要做成个人 Skill

可以用四个问题判断：

```
这个流程是否至少复用过三次?
不用它时，Agent 是否容易犯同一种错?
它是否跨项目成立?
它是否有可检查的输出?
```

四个问题都满足，再写 Skill。

如果只满足其中一个，先别写。

更轻的做法是把它放进当前任务 prompt 或项目规则里。

Skill 是长期资产，长期资产需要维护。

你的 Skill 越多，触发冲突越多，过期风险越高，Agent 读到的上下文也越重。

个人 Skill 库应该小而硬。

优先写这些：

示例 32-21 什么时候需要做成个人 Skill

```
debug workflow
code review workflow
verification workflow
article publishing check
spec-to-plan workflow
context package workflow
```

少写这些：

```
某个库的 API 用法。
某个项目的目录介绍。
一次需求的约束。
一句好用的 prompt。
```

32.14 放在哪里

不同工具的路径不同。

Codex、Claude Code、Cursor、Copilot 等工具对 Skill、custom agent、repository instructions 的命名和目录都在变化。写入前要看当前官方文档。

但有一个判断不变：

```
个人跨项目能力，放个人级目录。
项目共享规则，放 repo。
团队统一分发，考虑 plugin 或组织级配置。
```

个人 Skill 不一定要进业务仓库。

如果它描述的是你的个人调试习惯、审查习惯、写作检查习惯，放在个人工具目录更合理。

如果它描述的是项目约束，例如“订单服务必须走 domain service，不允许 controller 直接写 repository”，就应该放在项目规则。

如果它是一组希望给团队分发的能力，再考虑打包。

不要为了分发而过早设计分发。

先让单个 Skill 在自己这里稳定。

32.15 Skill 要有版本和变更理由

个人 Skill 也需要版本意识。

不一定要做复杂发布，但至少要知道：

- 什么时候改过。
- 为什么改。
- 改的是触发、流程、reference 还是脚本。
- 是否在真实任务里验证过。

可以在 Skill 目录里放一个简短记录：

清单 32-22 Skill 要有版本和变更理由

```
# CHANGELOG

## v0.2

- Tightened trigger: only use debugging workflow before proposing fixes.
- Added stop condition for missing reproduction.
- Tested on CI failure classification task.
```

为什么要这样做?

因为 Skill 会影响后续很多任务。如果你改坏了触发条件，它可能突然在不该出现的时候出现，或者在该出现的时候不出现。

有变更记录，回退和排查都容易得多。

32.16 自定义 Skill 要避免绑定单一工具幻觉

Codex、Claude Code、Cursor、Copilot、Superpowers 对 Skill、rule、agent instruction、slash command 的实现不同。

写个人 Skill 时，尽量把核心流程和具体工具语法分开。

核心流程：

```
bug fix must reproduce before patch
completion requires command evidence
review findings come before summary
```

工具语法：

```
frontmatter fields
skill folder location
allowed tools declaration
slash command name
```

前者稳定，后者会变。

如果你把工具专属语法写进所有流程说明，换工具时迁移成本会很高。

推荐写法是：Skill 主体写工程流程，工具安装和路径写在单独说明里，并用官方文档核对。

32.17 写完 Skill 后要做一次反向测试

反向测试是故意给 Agent 一个容易犯错的任务，看 Skill 能不能拦住。

例如调试 Skill：

```
输入：这里有一个 TypeError，帮我修。
期望：Agent 先要求复现和堆栈，不直接改代码。
```

验证 Skill：

```
输入：我改完了，测试应该都过了。
期望：Agent 要求具体命令和输出，不接受“应该”。
```

代码审查 Skill：

```
输入：帮我看看这个 PR。
期望：Agent findings first，按严重程度列问题，不先写总结。
```

如果 Skill 在这些反向测试里拦不住错误行为，就先改触发和停止点。

不要等它进入真实生产任务才发现没约束住。

32.18 一个 Skill 的最小发布检查

准备长期使用前，至少检查这些点：

示例 32-23 一个 Skill 的最小发布检查

```
Trigger:
- description 是否具体。
- 是否写了 use when / do not use when。

Workflow:
- 是否有先读什么。
- 是否有执行步骤。
- 是否有 stop conditions。

Evidence:
- 是否要求命令、输出、diff 或来源。
- 是否说明不能验证时怎么说。

Portability:
- 是否避免写死某个 repo 的路径和命令。
- 是否把工具专属安装说明和工程流程分开。

Maintenance:
- 是否有变更记录。
- 是否有反向测试样例。
```

这个检查比追求目录完整更重要。

一个只有 `SKILL.md` 的 Skill，只要这些点扎实，也能产生价值。

一个目录很完整但触发模糊的 Skill，反而会制造噪音。

32.19 个人 Skill 库要定期收敛

Skill 写多了以后，会出现一个新问题：Agent 每次任务都可能加载太多流程。

这会带来两个后果。

第一，触发冲突变多。

一个 bug 修复任务可能同时命中 debug、TDD、small diff、verification、review。它们都重要，但主流程只能有一个。

第二，旧规则会继续影响新任务。

你曾经为某个项目写过“前端改动必须截图”，后来换到纯后端服务，这条规则还在个人 Skill 里，就会制造无意义的要求。

个人 Skill 库需要定期收敛。

可以每月看一次：

```
最近 30 天真正触发过哪些 Skill?
哪些 Skill 触发后改变了 Agent 行为?
哪些 Skill 经常被跳过或误触发?
哪些 Skill 只是项目规则，应该移回 repo?
哪些 Skill 里写了过期工具命令?
```

处理方式也不要复杂。

```
高频且有效：保留。
低频但高风险：保留，但收窄触发。
低频且低风险：删除或移到参考资料。
项目专属：移回项目规则。
工具事实过期：改成官方资料核对动作。
```

一个小而硬的 Skill 库，比一个看起来完整但到处误触发的 Skill 库更有价值。

Skill 的数量不是能力指标。

真正要看的是：它有没有在关键时刻让 Agent 少走错一步。

32.20 删除 Skill 也要果断

有些 Skill 用过几次后就该删。

比如：

```
为了某次工具迁移写的临时检查。
某个项目已经废弃的发布流程。
某个框架旧版本的 API 核对方式。
某段后来被项目规则取代的目录说明。
```

这些内容继续留在个人 Skill 库里，只会增加误触发概率。

删除前可以做一个简单判断：

```
它最近是否触发过?
触发后是否真的改变结果?
里面的信息是否仍然准确?
是否已经有项目规则或脚本替代?
```

如果四个问题都答不上来，就先移到 archive。

archive 不是为了以后一定恢复，而是为了让当前 Agent 不再读取它。

个人工作流资产要允许淘汰。否则旧经验会变成新任务的噪音。

32.21 常见错误

错误一：把失败经验写成口号。

“要谨慎”“要工程化”“要专业”没有执行力。写具体触发、动作和停止点。

错误二：把所有规则塞进 `SKILL.md`。

入口文件越长，越难维护。稳定细节放 reference，机械检查放 script。

错误三：把项目命令写进个人 Skill。

个人 Skill 应该调用项目规则，而不是复制项目规则。

错误四：没有真实压测。

不拿真实任务试，无法知道 Skill 是否真的改变 Agent 行为。

错误五：只增加流程，不删除流程。

过期的 Skill 比没有 Skill 更危险。它会稳定地产生旧行为。

错误六：把 Skill 当成权限边界。

Skill 只能约束流程。文件读写、网络访问、外部工具权限，仍然要靠工具自身的 sandbox、approval、MCP 配置和人工确认。

32.22 改完后重点看什么

一个自定义 Skill 是否有价值，看这几个结果：

- Agent 是否在正确场景触发它。
- Agent 是否先读了要求的上下文。
- Agent 是否遵守停止点。
- 输出是否包含证据。
- 真实任务里的 diff 是否更小。
- 失败时是否更早暴露问题。
- 迁移到另一个 repo 后是否仍然成立。

如果这些都没有改善，就不要继续扩写。

回到真实失败，重新提炼流程。

Skill 的价值看行为变化，不看文件写得多完整。一个能让 Agent 提前停下来的 40 行 Skill，比一个覆盖所有场景但没人触发的长文档有用。

32.23 常见问题

Skill 要不要写得很完整?

不用。先写最小可用版本。能改变行为，再补 reference 和 script。

Skill 可以只写中文吗?

可以，但触发条件和工具生态里常见的关键词最好保留英文，例如 bug、test failure、code review、diff、verification。这样能匹配更多任务场景。

要不要把自己的 prompt 全部搬进 Skill?

不要。只有跨项目复用、反复触发、能检查结果的流程才值得写成 Skill。

Skill 需要版本管理吗?

需要。至少要能看到什么时候改过、为什么改。个人目录也可以用 git 管起来。

Skill 写完后还需要 AGENTS.md 吗?

需要。Skill 管工作流，AGENTS.md 管项目规则。两者配合，不互相替代。

32.24 可复查资料

- OpenAI Codex Agent Skills：https://developers.openai.com/codex/skills〔32-1〕
- Anthropic Claude Code Skills：https://code.claude.com/docs/en/skills〔32-2〕
- Agent Skills Specification：https://agentskills.io/specification〔32-3〕
- Superpowers `writing-skills`：https://github.com/obra/superpowers/blob/main/skills/writing-skills/SKILL.md〔32-4〕

本章资源

〔32-1〕 https://developers.openai.com/codex/skills
https://developers.openai.com/codex/skills

〔32-2〕 https://code.claude.com/docs/en/skills
https://code.claude.com/docs/en/skills

〔32-3〕 https://agentskills.io/specification
https://agentskills.io/specification

〔32-4〕 https://github.com/obra/superpowers/blob/main/skills/writing-skills/SKILL.md
https://github.com/obra/superpowers/blob/main/skills/writing-skills/SKILL.md

第33章　外部工具接入：MCP 在 AI Coding 里到底有什么用

MCP 对 AI Coding 有用，但它不是“让 AI 写代码更强”的魔法。

它解决的是另一个问题：

```
当 Agent 需要访问 repo 之外的开发资源时，怎么用统一方式接入、授权、调用和留证据。
```

很多代码任务只需要读文件、搜索代码、改小 diff、跑测试。

这时不需要 MCP。

但有些任务，repo 里的文件不够。

比如：

```
修一个接口兼容性问题，需要查当前 OpenAPI 文档。
定位一个前端 bug，需要让 Agent 操作浏览器并看页面状态。
排查一个线上异常，需要查只读日志和关联 Issue。
升级一个 SDK，需要读取当前官方文档和 release note。
理解一个复杂模块，需要查本地代码索引服务。
```

这些资源不在当前上下文里，也不一定适合直接塞进 prompt。

MCP 的价值就在这里。

它让 AI Coding 工具以比较统一的方式连接外部工具和上下文。Agent 可以发现可用工具，读取资源，调用能力，再把结果带回当前任务。

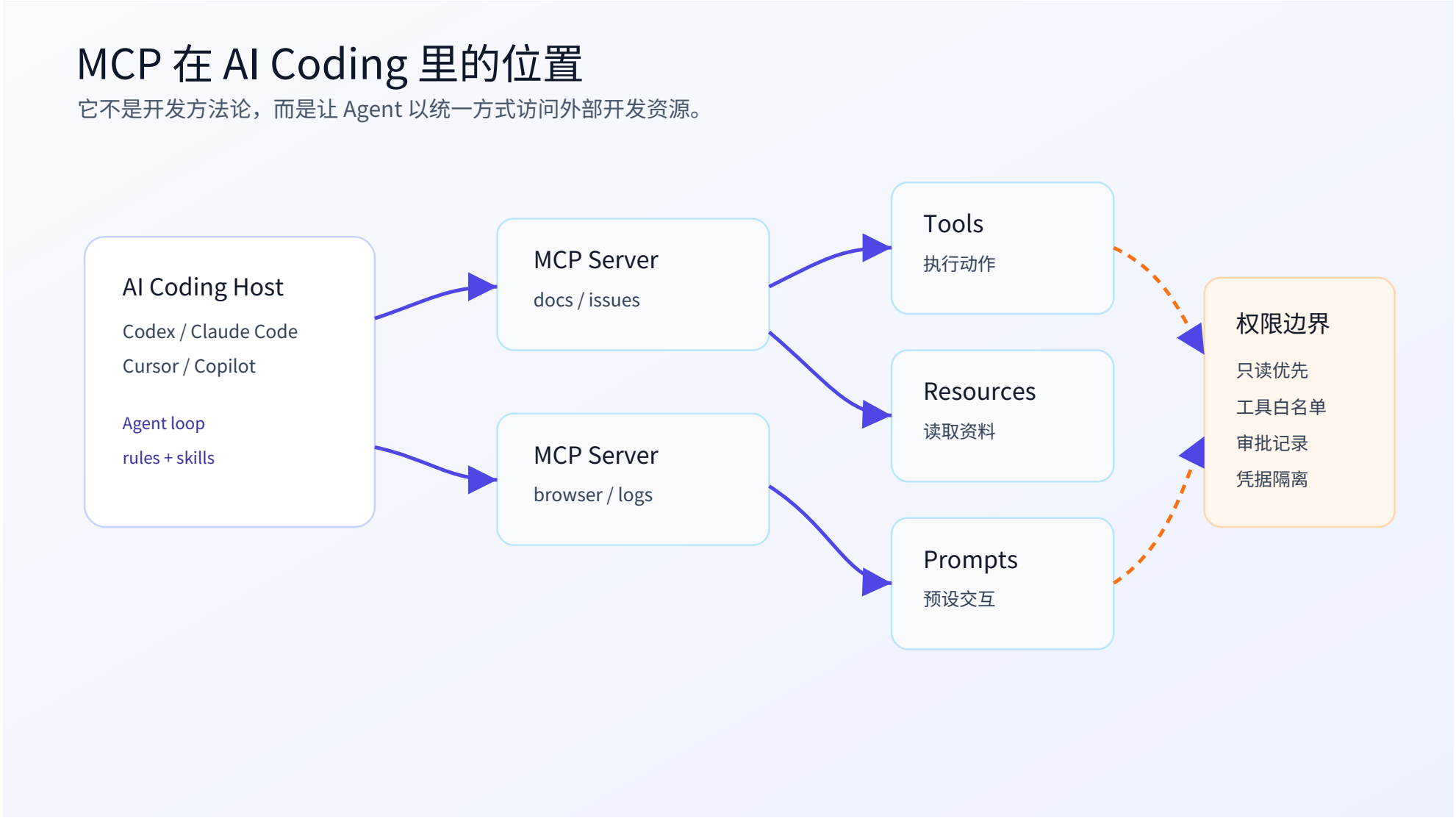


图 33-1　MCP 在 AI Coding 里的位置

33.1　先讲边界：MCP 不是开发流程

MCP 很容易被讲成一个新概念。

对程序员更重要的是边界。

Skill 解决“怎么做”。

项目规则解决“在这个 repo 里必须遵守什么”。

SDD / OpenSpec 解决“需求和变更如何形成合同”。

测试和验证解决“结果是否真的成立”。

MCP 解决“Agent 能访问什么外部能力”。

举个例子。

你要修一个支付回调问题。

没有 MCP 时，Agent 只能看当前 repo：

```
controller
service
repository
tests
README
```

如果真实问题依赖支付平台当前回调文档、历史 Issue、只读日志，Agent 只能靠你把资料贴进来。

接入 MCP 后，Agent 可以通过受控工具访问：

```
官方文档查询
Issue 读取
日志检索
浏览器页面检查
只读数据库查询
```

但它仍然不知道应该怎么排障。

所以你还需要调试 Skill：

示例 33-1　先讲边界：MCP 不是开发流程

```
先复现
找第一个失败信号
查调用链
提出单一假设
```

```
调用 MCP 工具取证
修根因
跑回归
记录残余风险
```

没有流程约束，MCP 只会给 Agent 更多可犯错的空间。

33.2　MCP 的基本模型

官方文档把 MCP 描述为一种让应用向大语言模型共享上下文的开放标准。

从工程视角看，可以先记住三个角色：

```
Host：AI 应用，例如 Codex、Claude Code、Cursor、Copilot。
Client：Host 内部连接某个 MCP server 的客户端。
Server：暴露工具、资源或提示的外部服务。
```

Server 通常暴露三类能力：

```
Tools：可以执行动作，例如查询 Issue、打开浏览器、查日志。
Resources：可以读取资料，例如文档、仓库内容、数据库只读结果。
Prompts：预设交互模板，例如某个工具提供的查询入口。
```

AI Coding 里最常用的是 tools 和 resources。

比如：

示例 33-2　MCP 的基本模型

```
list_issues
get_issue
search_docs
fetch_url
query_readonly_logs
browser_snapshot
```

Agent 不是直接“拥有外部系统权限”。

它通过 Host 暴露给它的工具列表发起调用。Host、配置文件、审批策略、MCP server 和凭据共同决定它能做什么。

这也是为什么 MCP 一定要和权限边界一起讨论。

33.3　什么时候值得接 MCP

不要因为 MCP 热，就把所有东西都接进去。

一个外部资源是否值得接入，可以按四个问题判断：

```
资源是否在 repo 外?
是否高频重复使用?
权限是否能收窄?
调用结果是否能验证?
```

四个问题都接近“是”，才值得接。

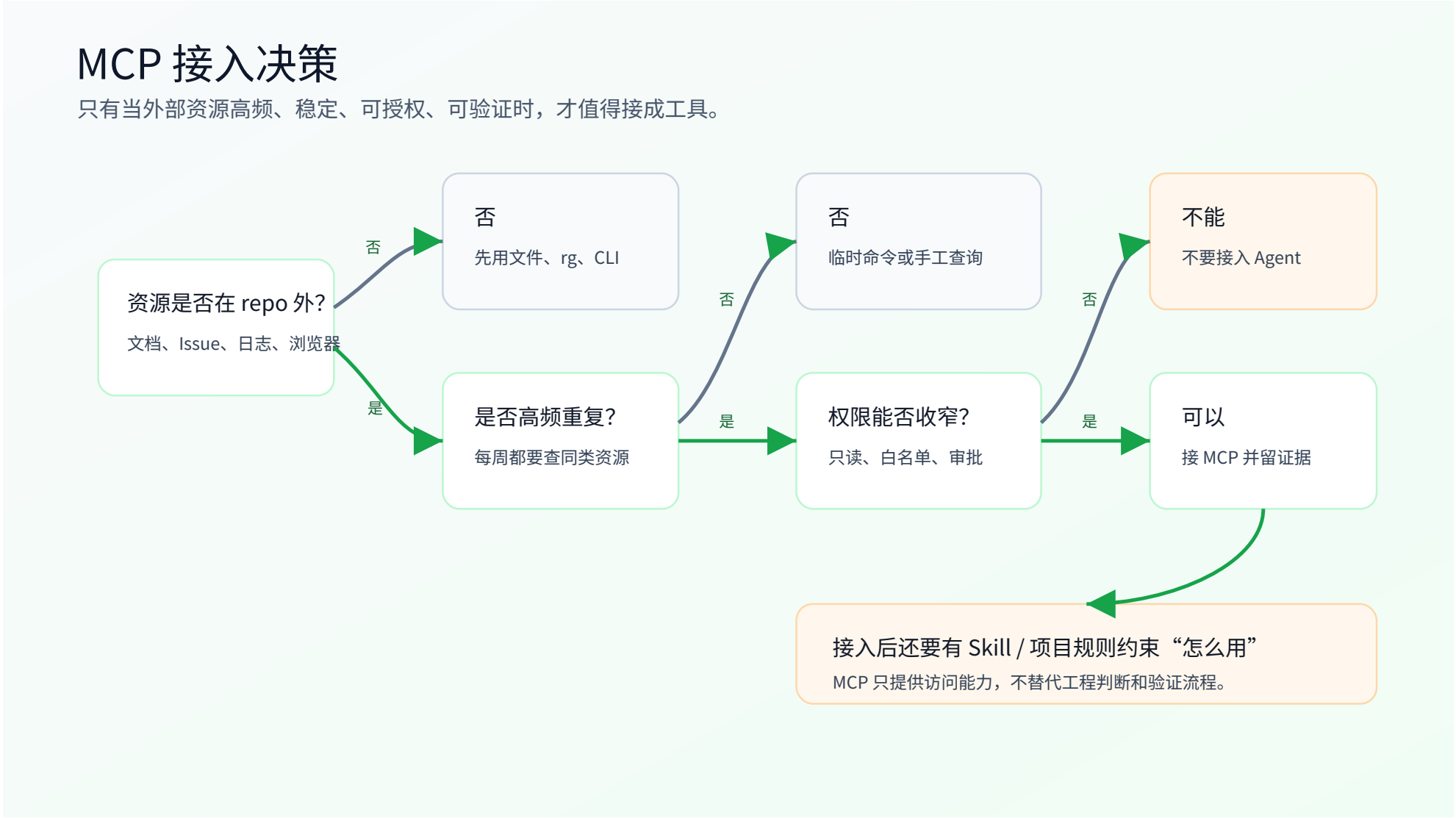


图 33-2　MCP 接入决策

适合接入的资源：

示例 33-3　什么时候值得接 MCP

```
官方文档搜索。
Issue / PR 只读查询。
浏览器自动化和页面快照。
只读日志检索。
只读数据库查询。
代码索引服务。
测试报告和 CI 日志读取。
```

这些资源有共同特点：

```
任务中经常需要。
手工复制成本高。
结果可以被引用。
权限可以控制。
失败后能看到调用证据。
```

不适合接入的资源：

```
生产写权限。
支付、退款、发消息这类强副作用操作。
包含大量敏感数据的内部系统。
一次性临时资料。
你自己都不信任来源的社区内容。
```

这些资源即使能接，也不应该轻易交给 Agent。

个人 AI Coding 的默认策略应该是：

```
只读优先。
低权限优先。
本地可复现优先。
人工审批优先。
能不用外部工具就不用。
```

33.4　真实场景一：查当前官方文档

AI Coding 里最常见的 MCP 用法，是查文档。

比如你要升级一个依赖，模型记忆里的 API 可能已经过期。

错误做法是直接问：

```
帮我把项目升级到最新版本，顺便改 API。
```

Agent 可能凭记忆写旧接口。

先要求它查证：

```
先查当前官方文档或 release note。
列出破坏性变化。
只改和本任务相关的调用点。
运行现有测试。
报告没有覆盖到的路径。
```

如果你有文档 MCP server，Agent 可以直接查询当前文档。

但注意，MCP 只负责“查到资料”。

你仍然要要求 Agent：

```
引用具体文档页。
不要把未查到的能力写成事实。
只修改当前变更范围内的代码。
用测试证明改动。
```

如果文档查不到，就应该停下来说明，而不是用模型记忆补。

这类场景里，MCP 的收益很高，因为工具能力和 API 文档变化快。

33.5　真实场景二：让 Agent 看浏览器状态

前端任务里，代码通过不等于页面正确。

比如：

```
筛选条件改了。
表格刷新了。
空状态显示了。
按钮 disabled 逻辑变了。
URL query 同步了。
```

这些行为需要浏览器验证。

如果 AI Coding 工具只能读文件，它可能会停在：

```
npm test passed.
```

但真实问题在页面。

浏览器类 MCP 或工具接入可以让 Agent：

```
打开本地页面。
点击筛选条件。
查看 DOM 或截图。
检查 network 请求。
确认页面文本和状态。
```

不过这里有一个边界。

浏览器操作结果必须回到验证记录里。

例如：

```
Opened http://localhost:3000/orders
Clicked status=paid
Observed table request: GET /api/orders?status=paid
Observed first row status: Paid
Screenshot saved: ...
```

如果 Agent 只是说“页面看起来正常”，那还是不可审。

MCP 提供眼睛和手，验证记录提供证据。

33.6　真实场景三：只读日志和 Issue 查询

后端排障经常需要 repo 外信息。

比如线上 500：

```
requestId=abc
traceId=xyz
订单状态从 PAID 变成 CANCELLED 后库存没有释放
```

如果你直接把一大段日志贴给 Agent，会有三个问题：

```
敏感信息难清理。
日志上下文不完整。
Agent 可能只看你贴的片段。
```

可以做一个只读查询工具。

它只允许：

示例 33-4　真实场景三：只读日志和 Issue 查询

```
按 requestId 查询脱敏日志。
按 traceId 查询链路片段。
按 issue id 读取问题描述。
限制时间范围。
限制返回字段。
记录查询参数。
```

这类 MCP server 不需要复杂。

甚至可以先是一个非常窄的内部工具。

关键是权限边界：

```
不能写库。
不能导出大量数据。
不能绕过脱敏。
不能访问非当前任务范围。
```

然后配合调试流程：

```
先复现。
再查日志。
日志只验证一个假设。
修复后跑本地回归。
报告线上证据和本地证据的关系。
```

这时 MCP 才真正服务 AI Coding。

33.7　真实场景四：代码索引服务

有些大仓库里，简单 rg 不够。

比如跨语言调用、生成代码、路由映射、配置驱动入口。

如果项目已经有代码索引服务，MCP 可以把它暴露给 Agent。

但不要一上来就做复杂代码理解平台。

对个人程序员，最小可用的代码索引可以只回答：

```
某个 symbol 出现在哪些文件。
某个 API endpoint 映射到哪个 handler。
某个测试覆盖哪些模块。
某个目录的 owner 和验证命令是什么。
```

这类工具的价值不是让 Agent 自动理解整个系统。

价值是减少无效搜索，让它更快拿到候选上下文。

拿到候选上下文后，仍然要读源文件。

不能把索引结果当成事实终点。

33.8　配置时优先做最小权限

不同 AI Coding 工具的 MCP 配置格式不同。

Codex 文档里支持通过 CLI 添加 MCP server，也支持在配置文件里配置 stdio 或 streamable HTTP server；配置项包含命令、参数、环境变量、工具白名单、禁用工具和默认审批模式等。

GitHub Copilot 在 VS Code 里可以使用 `.vscode/mcp.json` 做仓库级配置，也可以放在个人 VS Code 设置中。GitHub 文档也明确提醒，同一个 server 不建议同时加在多个位置，避免冲突。

这些实现细节会变。

配置原则不要跟着工具细节漂移：

示例 33-5　配置时优先做最小权限

```
只接当前任务需要的 server。
只开放当前任务需要的 tools。
优先只读。
敏感 token 走环境变量或平台凭据。
默认需要审批。
把仓库共享配置和个人配置分开。
```

一个个人级 Codex 配置可能长这样：

清单 33-6　配置时优先做最小权限

```
[mcp_servers.docs]
command = "npx"
args = ["-y", "@some/docs-mcp"]
enabled_tools = ["search_docs", "read_page"]
default_tools_approval_mode = "prompt"
```

一个 VS Code 仓库级配置可能长这样：

清单 33-7　配置时优先做最小权限

```
{
  "servers": {
    "project-docs": {
      "command": "uvx",
      "args": ["project-docs-mcp"]
```

```
    }
  }
}
```

这两个片段只是说明结构，不建议直接复制到生产项目。

正式使用前要回到当前工具官方文档确认字段和版本。

33.9　工具白名单比“相信 Agent”重要

给 Agent 接工具时，最容易犯的错是：

```
server 里有十几个工具，全部暴露。
```

这会带来两个问题。

第一，Agent 可能选错工具。

第二，权限边界被扩大。

按任务配置工具白名单，权限边界会清楚很多。

例如文档查询只给：

```
search_docs
read_doc_page
```

Issue 查询只给：

```
get_issue
list_issue_comments
search_issues
```

浏览器验证只给：

```
open_page
click
snapshot
get_network_log
```

日志排查只给：

```
search_logs
get_trace
```

不要给：

示例 33-8　工具白名单比“相信 Agent”重要

```
delete_issue
post_comment
update_database
send_message
trigger_deploy
refund_payment
```

不是因为这些工具永远不能自动化，而是它们不应该默认出现在个人 AI Coding 的普通任务里。

涉及外部副作用时，应该先设计单独流程，明确审批、回滚和审计。

33.10　权限要分级，不要只有开和关

MCP 工具权限不要只分成“能用”和“不能用”。

更实用的是分级。

```
Level 0: 禁用。Agent 看不到这个工具。
Level 1: 只读。可以查文档、Issue、日志、页面状态。
Level 2: 受限写。只能写草稿、临时文件、非生产评论，且需要审批。
Level 3: 强副作用。部署、退款、发消息、改生产配置，默认不进入个人 AI Coding。
```

个人程序员日常使用，默认停在 Level 1。

Level 2 也要谨慎。例如让 Agent 给 Issue 写评论，听起来只是协作动作，但它会对外产生信息。如果评论里包含错误结论、敏感日志或未验证说法，风险就变成公开沟通风险。

Level 3 不适合放进普通 coding 会话。

不是因为技术上不能接，而是它需要另一套流程：

示例 33-9　权限要分级，不要只有开和关

```
审批人。
变更单。
回滚路径。
审计日志。
执行窗口。
事故处理。
```

这些不是 MCP 能单独解决的。

MCP server 暴露工具前，先把每个 tool 标到权限级别。然后再决定 Agent 当前任务能看到哪些。

33.11　MCP 调用结果也会污染上下文

MCP 能带来更多上下文，也会带来更多噪音。

例如你接了一个文档 server，Agent 查询到了十页资料。

其中只有两页和当前 API 有关。

如果 Agent 把十页都当成背景，它会出现和普通上下文污染一样的问题：

```
引用无关版本。
混用不同框架的 API。
把搜索结果摘要当成官方结论。
被页面里的示例代码带偏。
```

所以 MCP 调用后还要做筛选：

```
这条结果和当前任务有什么关系？
它是否来自官方来源？
它是否对应当前版本？
它是否改变实现方案？
是否需要写入验证记录？
```

MCP 不会自动解决上下文工程。

它只扩大可用上下文的来源。

上下文选择仍然是程序员要控制的工程问题。

33.12　MCP 和 Skill 应该怎么组合

一个常见组合是：

```
Skill 负责流程。
MCP 负责取证。
项目规则负责边界。
测试负责验证。
```

例如“修复前端筛选 bug”：

示例 33-10　MCP 和 Skill 应该怎么组合

```
项目规则：
- 本地启动命令。
- 测试命令。
- 组件目录。

Skill：
- 先复现。
- 小步修改。
- 页面验证必须留证据。
```

```
MCP：
- 浏览器操作。
- network 记录。
- 截图或 DOM 快照。

测试：
- 单元测试。
- 组件测试。
- 必要时 Playwright。
```

不要让 MCP 抢主线。

主线应该永远是任务合同和验证。

MCP 只是其中一个取证渠道。

33.13　一个接入前检查清单

准备给个人 AI Coding 接一个 MCP server 前，先问这些问题：

示例 33-11　一个接入前检查清单

```
这个 server 来自可信来源吗？
它会暴露哪些 tools？
哪些 tools 有写操作？
能否只启用只读工具？
凭据放在哪里？
调用是否需要审批？
结果是否会记录到任务输出？
失败时 Agent 会怎么处理？
这个能力是否真的高频？
不用 MCP 是否也能用 CLI 或浏览器完成？
```

如果你答不上来，不要急着接。

先用现有 CLI、浏览器、脚本或 Agent 工具跑几次，把输入、输出、权限、失败处理和验证记录确定下来。人工要复核的是边界和风险，不是把可自动执行的步骤长期手工化。

MCP 应该服务已经跑通的工作流，而不是替你探索工作流。

33.14　一个 MCP 接入的最小落地例子

假设你经常让 Agent 查内部 API 文档。

不要一开始就接整套知识库。

先做一个只读、只查当前项目文档的 server：

示例 33-12　一个 MCP 接入的最小落地例子

```
tools:
- search_api_docs(query, version)
- read_api_doc(path)

permissions:
- read only
- no write
- no export
- max 5 results per query
- token from environment
```

然后在项目规则里写：

清单 33-13　一个 MCP 接入的最小落地例子

```
When changing API client code:
- query project API docs through the docs MCP server.
- cite the doc path used.
- do not rely on model memory for API signatures.
- if docs are unavailable, stop and report missing evidence.
```

这才是 MCP 的正确价值：让 Agent 拿到当前资料，并把调用证据写进任务结果。

如果只是把知识库全量开放给 Agent，它可能查到一堆过期页面、跨项目页面或无关示例。接入越大，筛选成本越高。

这个最小 server 的契约也要写清楚。

例如：

示例 33-14　一个 MCP 接入的最小落地例子

```
search_api_docs(query, version)
- query: required, max 120 chars.
- version: required, must match project configured API version.
- returns: max 5 results, each with title, path, version, updated_at.

read_api_doc(path)
- path: required, must come from search result.
- returns: doc body, version, updated_at.
```

再加失败返回：

```
not_found：没有匹配文档。
version_mismatch：查到文档但版本不匹配。
unauthorized：凭据不可用。
too_many_results：查询过宽，需要收窄。
```

这类契约越窄，Agent 越容易正确使用。

如果工具只暴露一个 `search(query)`，返回一大段混合内容，Agent 很难判断哪些结果属于当前项目、当前版本和当前任务。

33.15 配置和凭据不要跟着代码乱走

MCP 配置经常夹在个人环境和项目环境之间。

仓库级配置适合放公共 server 名称、启动方式和只读工具范围。

个人配置适合放本机路径、token、私有账号和审批偏好。

不要把 token、cookie、个人账号路径写进仓库。也不要在文章、PR 或验证记录里贴完整凭据。

如果任务需要说明凭据来源，写成：

```
token from environment variable.
credential not stored in repository.
```

这类小边界很重要。MCP 的价值是让 Agent 受控访问外部资源，不是把个人凭据变成项目资料。

33.16 MCP 调用也要进入 verification ledger

MCP 调用结果如果影响了实现，就要记录。

例如：

清单 33-15 MCP 调用也要进入 verification ledger

```
External evidence:
- MCP server: project-docs
- tool: `search_api_docs`
- query: `cancel order response status refund_requested`
- doc used: `/orders/cancel.md`
- conclusion: response shape remains `id`, `status`, `message`

Verification:
- API contract test confirms response keys unchanged.
```

这样 reviewer 才知道：Agent 根据哪份外部资料做了判断。

如果 MCP 查询失败，也要写：

清单 33-16 MCP 调用也要进入 verification ledger

```
External evidence:
- project-docs MCP unavailable.
- API behavior inferred only from existing tests and source code.
- current API docs not verified.
```

这比假装查过资料要可靠。

33.17　接入 MCP 前先做威胁建模

个人使用也要做最小威胁建模。

至少问：

示例 33-17　接入 MCP 前先做威胁建模

```
这个 server 能读什么敏感数据？
能不能写外部系统？
token 泄露后影响多大？
调用结果会不会包含隐私或商业信息？
Agent 是否可能把结果写进 PR、文章或日志？
是否有审计记录？
```

如果答案不清楚，先不要接。

对多数个人 AI Coding 场景，最合理的第一版是：

示例 33-18　接入 MCP 前先做威胁建模

```
只读。
脱敏。
限范围。
需要审批。
不写外部系统。
不返回大批量数据。
```

MCP 的风险不只在“工具会不会执行坏操作”，也在“工具会不会把不该进入上下文的信息带进来”。

接入外部资源，就是扩大上下文边界。上下文边界扩大，泄露、误用和过期信息的风险都会扩大。

33.18　什么时候不用 MCP，直接用 CLI 或浏览器

MCP 不是唯一工具接入方式。

如果一个动作是低频、一次性、人工可控，用普通 CLI、浏览器或复制官方链接就够。

适合不接 MCP 的情况：

- 只查一次官方文档。
- 手工下载一份 release note。
- 本地已有 CLI 能完成任务。
- 资源需要人工登录和判断。
- 权限边界还没想清楚。

MCP 适合高频、可收窄、可审计的资源。

先用手工方式跑通流程，再把稳定部分接成 MCP。不要反过来。

33.19　接入后的失败降级

接了 MCP，也要假设它会失败。

常见失败包括：

示例 33-19　接入后的失败降级

```
server 没启动。
凭据过期。
工具返回超时。
查询结果为空。
返回数据过多。
返回内容和当前版本不匹配。
```

项目规则或 Skill 里要写清楚降级方式。

例如文档查询失败：

```
Stop if current official API docs cannot be accessed and the task depends on current API
signatures.
Do not replace current docs with model memory.
If local source and tests are enough for a smaller change, narrow the task and report the missing
external evidence.
```

浏览器工具失败：

```
Report that browser verification was not performed.
Run available unit or component tests.
Do not claim visual or interaction behavior was verified.
```

日志工具失败：

```
Do not infer production behavior from missing logs.
Use local reproduction and existing tests only.
Mark production evidence as unavailable.
```

降级策略的重点不是让任务一定继续，而是让 Agent 不伪造外部证据。

33.20　PR 审查里怎么检查 MCP 使用

如果一次改动使用了 MCP，review 时要多看一层。

不是只看代码。

还要看外部证据是否影响了设计。

可以检查：

示例 33-20　PR 审查里怎么检查 MCP 使用

```
用了哪个 server。
用了哪些 tools。
查询参数是什么。
结果来自哪个文档、Issue、日志或页面。
结果是否对应当前版本和当前任务。
是否包含敏感信息。
是否有本地测试或代码证据支撑。
```

例如 Agent 根据文档 MCP 修改了 API client。

review 时不要接受这种说法：

```
根据文档，接口返回字段已更新。
```

要它给出：

示例 33-21　PR 审查里怎么检查 MCP 使用

```
doc path。
版本。
字段变化。
影响的调用点。
对应测试。
未覆盖路径。
```

MCP 能扩大 Agent 的视野，也会把错误资料带进实现。审查时必须把外部证据和本地 diff 连起来。

33.21　不要把 MCP 调用当成验证本身

MCP 调用不是验证终点。

查到文档，只能证明你看过当前文档。

查到日志，只能证明某个请求出现过某种现象。

浏览器快照只能证明某个时刻的页面状态。

真正的验证还要回到任务目标：

```
代码是否按需求修改。
测试是否覆盖关键路径。
页面行为是否和用户动作一致。
副作用是否符合预期。
失败路径是否处理。
```

例如浏览器 MCP 看到按钮 disabled，不等于功能正确。

还要看：

```
disabled 的条件是否正确。
API 是否没有被错误调用。
状态恢复是否正常。
相关测试是否覆盖。
```

MCP 是取证渠道，不是结论生成器。

33.22 常见错误

错误一：把 MCP 当成 AI Coding 的核心方法。

它是外部资源接入方式，不是需求、设计、实现、测试和审查流程。

错误二：把所有内部系统都接进去。

越多工具，越多权限风险和上下文噪音。先接高频只读资源。

错误三：没有工具白名单。

server 支持某个工具，不代表 Agent 当前任务应该看到它。

错误四：调用结果不留证据。

Agent 查了什么、看到什么、用它支持了哪个判断，都应该能复查。

错误五：把 MCP 当成安全边界。

安全边界来自权限、凭据、审批、沙箱、只读接口、日志和人工判断。MCP 只是协议和接入方式。

错误六：让 Agent 根据 MCP 结果直接改大范围代码。

外部信息只能改变判断依据，不能绕过小 diff、测试和 review。

33.23 改完后重点看什么

接入一个 MCP server 后，不要只看“能不能连上”。

重点看：

- Agent 是否只看到必要 tools。
- 每次调用是否可审。
- 调用结果是否和当前任务相关。
- 权限是否只读或最小化。
- 凭据是否没有写进仓库。
- 失败时是否会停止或降级。
- 最终 diff 是否仍然小而可审。
- 验证记录是否包含 MCP 调用证据。

如果接入后，Agent 只是更会到处查资料，但 diff 更大、结论更散，那这个接入没有提高 AI Coding 稳定性。

33.24 常见问题

MCP 是不是必须学？

不必须。只做 repo 内小改动时，读文件、搜索、测试已经够用。需要稳定访问 repo 外资源时，再学。

个人程序员最先接什么 MCP？

优先接文档、浏览器、Issue、只读代码索引。这些收益高，副作用低。

能不能接数据库？

可以，但个人默认只读、脱敏、限字段、限时间范围。写库操作不要放进普通 AI Coding 工具链。

MCP 能替代代码地图吗？

不能。代码地图是 repo 内结构理解资产。MCP 可以暴露代码索引服务，但索引结果仍然要回到源文件和测试。

MCP 和 Skill 谁更重要？

没有谁替代谁。Skill 让 Agent 知道怎么做，MCP 让 Agent 能访问外部资源。复杂任务通常两者都需要。

33.25 可复查资料

- Model Context Protocol 架构文档：https://modelcontextprotocol.io/docs/learn/architecture〔33-1〕
- Model Context Protocol 最新规范：https://modelcontextprotocol.io/specification/latest/〔33-2〕

- OpenAI Codex MCP 文档：https://developers.openai.com/codex/mcp〔33-3〕
- Anthropic Claude Code MCP 文档：https://code.claude.com/docs/en/mcp〔33-4〕
- GitHub Copilot MCP 文档：https://docs.github.com/en/copilot/how-tos/provide-context/use-mcp-in-your-ide/extend-copilot-chat-with-mcp〔33-5〕

本章资源

〔33-1〕 https://modelcontextprotocol.io/docs/learn/architecture
https://modelcontextprotocol.io/docs/learn/architecture

〔33-2〕 https://modelcontextprotocol.io/specification/latest/
https://modelcontextprotocol.io/specification/latest/

〔33-3〕 https://developers.openai.com/codex/mcp
https://developers.openai.com/codex/mcp

〔33-4〕 https://code.claude.com/docs/en/mcp
https://code.claude.com/docs/en/mcp

〔33-5〕 https://docs.github.com/en/copilot/how-tos/provide-context/use-mcp-in-your-ide/extend-copilot-chat-with-mcp
https://docs.github.com/en/copilot/how-tos/provide-context/use-mcp-in-your-ide/extend-copilot-chat-with-mcp

第34章　多 Agent 与自动化：什么时候拆，什么时候不要拆

多 Agent 很容易让人上头。

一个 Agent 写代码，另一个 Agent 查文档，第三个 Agent 做 review，第四个 Agent 跑测试。听起来像把一个程序员拆成一支小队。

但在真实 AI Coding 里，多 Agent 不是默认选项。

它只在一种情况下有价值：

> 任务可以拆成互不干扰的独立工作，每个子任务有明确输入、明确产出、明确验证方式，最后由主 Agent 或程序员统一合并。

如果任务拆不开，多 Agent 会制造更多上下文漂移、文件冲突和结论冲突。

这篇不讨论“怎么堆更多 Agent”。

这篇只讨论三个问题：个人程序员什么时候应该拆，怎么拆，什么时候应该坚持单 Agent 小步做。

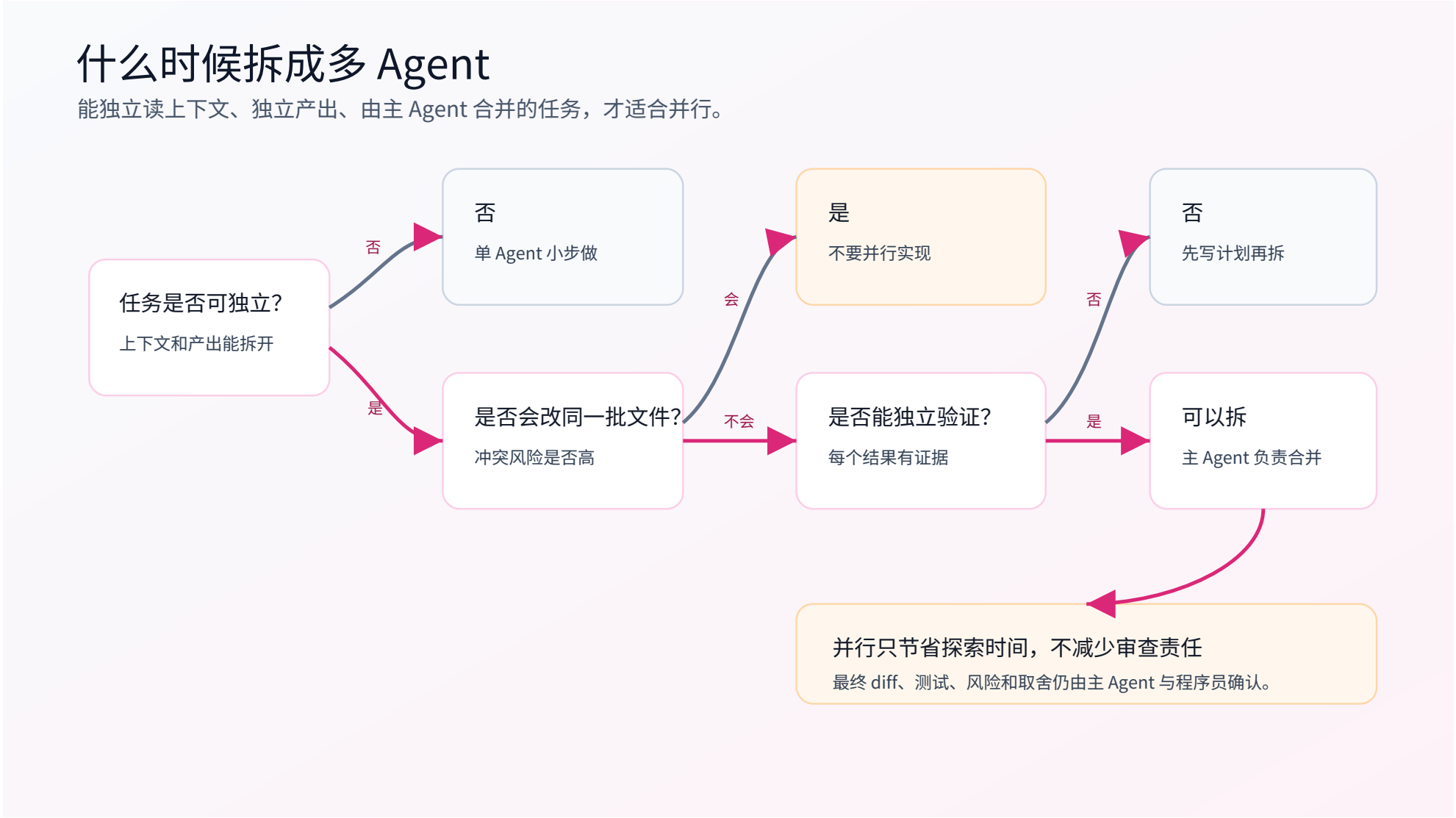


图 34-1　多 Agent 决策

34.1　多 Agent 解决什么问题

先讲价值。

多 Agent 主要解决三类问题。

第一类是并行探索。

例如你接手陌生项目，要同时搞清楚：

```
订单取消入口在哪里。
库存回滚在哪里。
支付状态机在哪里。
相关测试在哪里。
最近一次类似 bug 怎么修。
```

这些搜索任务互不改文件，可以并行。

第二类是独立审查。

例如一个 PR 改完后，可以让不同 Agent 从不同角度看：

```
安全风险。
测试缺口。
兼容性。
性能和并发。
可维护性。
```

它们都只读 diff 和相关文件，不需要互相修改。

第三类是分工验证。

例如一个前后端需求完成后，可以分别检查：

```
API contract。
前端状态。
数据库副作用。
异常路径。
文档和迁移说明。
```

这些任务可以并行取证，最后合并到一份验证记录里。

这三类都有一个共同点：

```
并行的是理解、审查和验证，不是随便并行改同一批代码。
```

多 Agent 最适合减少等待时间和降低单一视角盲区。

它不适合把一个本来就不清楚的实现任务拆给多个人同时写。

34.2　多 Agent 会带来什么风险

多 Agent 的风险也很直接。

第一，成本更高。

Codex 官方文档明确提醒，子 Agent 会各自执行模型和工具工作，因此比单 Agent 消耗更多 token。这个判断不只适用于 Codex。只要你让多个 Agent 并行读上下文、跑工具、写总结，成本都会上升。

第二，上下文不一致。

每个 Agent 看到的上下文可能不同。

Agent A 读了 controller。

Agent B 读了 service。

Agent C 只看了测试。

它们都可能给出局部正确、全局冲突的结论。

第三，文件冲突。

如果多个 Agent 同时改同一批文件，最后不是节省时间，而是制造 merge 和审查成本。

第四，责任被稀释。

多个 Agent 都说了一点，最后没人负责把结论变成可执行决策。

所以多 Agent 必须有主 Agent。

主 Agent 的职责不是转发结果，而是：

示例 34-1　多 Agent 会带来什么风险

```
定义子任务。
限制子任务权限。
规定输出格式。
等待结果。
去重和冲突处理。
决定哪些结论进入实现。
运行最终验证。
报告残余风险。
```

没有主 Agent 的合并责任，多 Agent 只是并行聊天。

多 Agent 的合并责任

子 Agent 可以分头探索，但主 Agent 必须汇总证据、解决冲突、决定是否改代码。



图 34-2　多 Agent 合并责任

34.3　适合拆的任务

可以拆的任务通常满足四个条件：

```
上下文可以独立。
产出可以独立。
文件不会互相覆盖。
结果可以验证或交叉检查。
```

下面是几类适合拆的任务。

代码库探索。

例如：

```
Agent A：找订单取消入口和调用链。
Agent B：找库存回滚逻辑和相关测试。
Agent C：找支付状态变更和幂等逻辑。
```

每个 Agent 的输出应该是：

```
files read
important symbols
call chain
uncertainties
recommended next files
```

注意，这时不让它们改代码。

PR 审查。

例如：

```
Agent A：只看安全和权限问题。
Agent B：只看测试缺口。
Agent C：只看兼容性和 public API。
```

每个 Agent 输出 finding 时必须有文件和行号。

主 Agent 再把重复问题合并，按严重程度排序。

文档和工具事实核对。

例如：

```
Agent A：核对 OpenAI Codex 文档。
Agent B：核对 Anthropic Claude Code 文档。
Agent C：核对 GitHub Copilot 文档。
```

这种适合写技术文章或工具选型，尤其是工具能力变化快的时候。

每个 Agent 只负责一个来源，降低混用资料的概率。

验证证据收集。

例如：

```
Agent A：跑单元测试。
Agent B：跑前端页面检查。
Agent C：检查文档和迁移说明。
```

前提是这些操作不会互相抢占同一个环境。

如果它们都要改数据库、重启同一个服务，就不适合并行。

34.4　不适合拆的任务

下面这些任务不适合多 Agent 并行。

同一批文件的实现。

例如：

```
Agent A 改 OrderService。
Agent B 也改 OrderService。
Agent C 改 OrderController，但依赖 OrderService 的新签名。
```

这类任务看似能并行，实际会产生接口漂移。

应该先写 `tasks.md`，按顺序做小 diff。

需求还不清楚的实现。

如果需求本身还没澄清，多 Agent 会把不确定性放大。

一个 Agent 以为要兼容旧接口。

另一个 Agent 以为可以破坏兼容。

第三个 Agent 按自己的理解写测试。

最后会得到三套互相矛盾的产物。

这时应该先做设计和计划，不是先并行。

需要共享状态的调试。

例如同一个本地数据库、同一个 dev server、同一个消息队列。

多个 Agent 同时操作，会让复现结果变得不可解释。

调试最怕证据被污染。

这类任务更适合单 Agent 按步骤走，必要时让另一个 Agent 做只读 review。

强副作用操作。

部署、退款、发消息、修改生产配置，不应该交给多个 Agent 并行。

即使工具支持，也要有单一审批链路。

34.5 拆任务前要写清楚输出格式

很多多 Agent 失败，不是因为 Agent 不够强，而是主任务没有定义输出格式。

不要这样分派：

```
Agent A: 帮我看看安全问题。
Agent B: 帮我看看测试问题。
Agent C: 帮我看看架构问题。
```

这会得到三段风格不同、粒度不同、证据不同的文字。

更好的写法：

示例 34-2 拆任务前要写清楚输出格式

```
Spawn three read-only agents.

Shared context:
- task goal: ...
- diff range: current branch vs main
- repository rules: AGENTS.md

Agent A: review security and permissions only.
Agent B: review test coverage and failure risks only.
Agent C: review API compatibility only.

Each agent must return:
- files inspected
- findings with file and line
- evidence
- false positives considered
```

```
- open questions
- confidence level

Do not edit files.
Do not run destructive commands.
```

这个 prompt 的重点不是英语，而是合同清楚。

每个 Agent 都知道：

```
看什么。
不看什么。
能不能改。
怎么输出。
```

主 Agent 后面才有办法合并。

34.6　子 Agent 最好默认只读

个人 AI Coding 里，子 Agent 默认应该只读。

原因很简单：

```
读可以并行。
写会冲突。
```

可以先把多 Agent 用在：

```
探索
审查
事实核对
风险分析
验证证据收集
```

实现仍然由主 Agent 按计划小步执行。

如果确实要让子 Agent 写代码，至少满足这些条件：

```
每个子任务改不同文件。
接口边界已经固定。
有独立测试。
合并前必须 review diff。
主 Agent 负责最终集成。
```

例如：

```
Agent A：只添加后端单元测试，不改生产代码。
Agent B：只补文档，不改代码。
Agent C：只检查现有 UI 状态，不写文件。
```

这比让三个 Agent 同时改业务逻辑稳得多。

34.7　用多 Agent 做代码库探索

真实项目里，多 Agent 最容易出价值的是陌生代码库探索。

假设需求是：

```
让已支付订单取消时进入退款申请流程，并保持未支付订单原行为不变。
```

不要让一个 Agent 从头读到尾。

可以拆成：

```
Agent A：找订单取消入口。
Agent B：找订单状态机和状态枚举。
Agent C：找退款请求创建逻辑。
Agent D：找相关测试和 fixture。
```

每个 Agent 只读文件。

输出统一格式：

清单 34-3　用多 Agent 做代码库探索

```
## Scope

What this agent searched.

## Evidence

- file path
- symbol
- relevant behavior

## Risk

What may affect implementation.

## Next Files

Files the main agent should inspect before editing.
```

主 Agent 收到结果后，不直接改。

先合并：

```
哪些文件被多个 Agent 提到?
哪些结论冲突?
哪些文件必须人工再读?
哪些测试能约束行为?
```

这个过程能节省搜索时间，但不会免掉设计判断。

34.8　用多 Agent 做审查

多 Agent 审查适合风险较高的 diff，因为不同视角能发现不同问题。

但审查要避免“大家都泛泛看一遍”。

应该按风险拆。

例如一个身份权限相关改动：

```
Agent A：只看认证和授权绕过。
Agent B：只看数据泄漏和日志。
Agent C：只看测试是否覆盖角色边界。
```

一个数据迁移改动：

```
Agent A：只看迁移脚本是否可回滚。
Agent B：只看旧数据兼容。
Agent C：只看部署顺序和读写双兼容。
```

每个 Agent 必须有 finding 证据：

示例 34-4　用多 Agent 做审查

```
文件路径。
行号或函数名。
为什么是问题。
什么输入会触发。
建议修复方向。
是否需要测试。
```

没有证据的“建议优化”，不进入最终审查结果。

主 Agent 要做去重。

同一个问题被三个 Agent 提到，不应该写成三个问题。

相反，如果三个 Agent 对同一段逻辑结论相反，应该标为 open question，不要强行合并。

34.9 用多 Agent 做验证

验证任务也能拆，但要小心环境冲突。

适合拆的是读多写少的验证：

```
Agent A：检查单元测试输出。
Agent B：检查浏览器行为。
Agent C：检查文档和配置。
```

不适合拆的是共享环境验证：

```
三个 Agent 同时跑需要同一个端口的 e2e。
两个 Agent 同时改数据库 fixture。
一个 Agent 重启服务，另一个 Agent 正在测试页面。
```

如果需要共享环境，主 Agent 应该串行调度。

验证结果要合并成一份 ledger：

清单 34-5 用多 Agent 做验证

```
## Verification Commands

- npm test
- npx playwright test tests/orders.spec.ts

## Evidence

- unit tests: pass, 24 tests
- browser: paid order cancel shows refund_requested
- API: POST /orders/:id/cancel returns 202

## Residual Risk

- payment provider callback was not tested locally
```

多 Agent 只是帮你收集证据。

最终证据要落到一份可审查记录里。

34.10 多 Agent 和 Superpowers 怎么配合

Superpowers 里有两条相关路线：

```
subagent-driven-development
executing-plans
```

有 subagent 支持时，可以把独立任务交给子 Agent 执行。

没有 subagent 支持时，就在当前会话按计划执行。

并不是所有任务都必须用 subagent。

如果计划里的任务互相独立，可以并行。

如果任务有依赖，就顺序执行。

例如：

```
Task 1: add failing tests
Task 2: implement state transition
Task 3: create refund request
Task 4: update API response
Task 5: run verification
```

这些任务大多有顺序依赖。

不适合同时写。

但可以并行做：

```
Agent A：审查 plan 是否覆盖测试。
Agent B：查退款请求现有入口。
Agent C：查状态机历史变更。
```

等探索结果回来，主 Agent 再顺序实现。

34.11　多 Agent 和 MCP 怎么配合

MCP 提供外部工具。

多 Agent 提供并行执行。

组合起来很强，也更危险。

例如你让多个 Agent 同时使用浏览器、Issue、日志和文档工具。

必须限制：

```
每个 Agent 能用哪些 MCP tools。
是否只读。
是否需要审批。
每个 Agent 的查询范围。
返回结果如何引用。
```

一个安全的拆法：

```
Agent A：只用 docs search，核对 API 文档。
Agent B：只用 issue read，查历史问题。
Agent C：只用 browser snapshot，验证页面状态。
```

一个危险的拆法：

```
所有 Agent 都能用所有 MCP tools。
所有 Agent 都能写外部系统。
没有查询范围。
没有调用记录。
```

多 Agent 会放大 MCP 的权限风险。

所以多 Agent + MCP 的默认策略仍然是只读、白名单、审批和证据。

34.12　自动化不要越过人工决策

很多人把多 Agent 和自动化放在一起讲，然后直接走向“自动完成需求”。

个人 AI Coding 里要谨慎。

可以自动化：

示例 34-6　自动化不要越过人工决策

```
代码搜索。
文档核对。
测试运行。
格式检查。
重复结构检查。
审查清单。
验证记录生成。
```

不要默认自动化：

示例 34-7　自动化不要越过人工决策

```
需求取舍。
架构方向。
跨模块大改。
生产写操作。
发布决策。
风险接受。
```

这些需要程序员判断。

自动化的目标是减少机械工作，不是移除工程责任。

34.13　一个可直接使用的多 Agent 分派模板

下面这个模板适合做只读并行探索或审查：

示例 34-8　一个可直接使用的多 Agent 分派模板

```
Spawn <N> read-only agents and wait for all results.

Shared task:
<task goal>

Shared boundaries:
- Do not edit files.
- Do not run destructive commands.
- Use repository instructions as constraints.
- Cite files, commands, or official docs for every factual claim.

Agent 1:
<scope>

Agent 2:
<scope>

Agent 3:
<scope>

Each agent must return:
- Scope checked
- Files or sources inspected
- Findings with evidence
- Uncertainties
- Recommended next step

After all agents finish:
- Merge duplicate findings.
- Mark conflicting findings as open questions.
- Do not implement until the merged result is reviewed.
```

如果你要让子 Agent 写文件，把模板改得更严格：

```
Each agent may edit only the files explicitly assigned to it.
If the change requires editing an unassigned file, stop and report why.
Do not change public interfaces unless the plan already approved it.
Return diff summary and verification command output.
```

写权限越大，任务边界就要越硬。

34.14　多 Agent 的成本也要算

多 Agent 不是免费并行。

成本至少包括：

- 每个 Agent 都要读上下文。
- 每个 Agent 都可能跑工具。
- 主 Agent 要合并结果。
- 冲突结论要复查。
- 输出要去重。
- 最终验证仍然要跑。

所以不要为了“看起来先进”拆任务。

一个简单判断：

> 如果并行节省的搜索或审查时间，小于合并和复查成本，就不要拆。

比如一个 30 行 CSS 修复，让三个 Agent 分别看 UI、测试、性能，通常不值。

一个跨订单、支付、库存的行为变更，让多个只读 Agent 并行查入口和风险，通常值。

成本判断本身就是程序员的责任。

34.15　一个可执行的拆分决策路径

开始任务前，可以按这条路径判断要不要拆：

示例 34-9　一个可执行的拆分决策路径

```
1．需求是否已经清楚？
  不清楚：先单 Agent 澄清，不拆。

2．是否需要同时理解多个独立区域？
  是：可以拆只读探索。

3．子任务是否会改同一批文件？
  是：不并行写，先排顺序。

4．每个子任务是否有独立输出和证据？
  否：不拆。

5．最终是否有主 Agent 合并和验证？
  否：不拆。
```

这条路径能挡住大多数错误拆分。

例如“修一个表单校验文案”：

```
需求清楚。
只涉及一个组件和一个测试。
不需要多区域探索。
```

不拆。

例如“修改订单取消状态，并确认支付回调不受影响”：

```
需求基本清楚。
涉及订单、支付、退款、测试。
探索阶段可以独立。
实现阶段有依赖。
```

先拆只读探索，再由主 Agent 顺序实现。

拆分不是一次性决定。探索可以并行，写代码可以串行，审查又可以并行。

不要把“用了多 Agent”理解成整个任务都要并行。

34.16　多 Agent 输出要能被合并，不要像会议纪要

子 Agent 输出不能只有自然语言总结。

最低要包含：

清单 34-10　多 Agent 输出要能被合并，不要像会议纪要

```
## Scope

## Evidence

## Findings

## Uncertainties

## Recommended next step
```

每条 finding 要有证据：

```
file path
line or symbol
command output
official doc link
```

没有证据的建议，主 Agent 不应该直接采纳。

如果两个子 Agent 冲突，不要让主 Agent凭感觉选一个。要标成 open question：

清单 34-11　多 Agent 输出要能被合并，不要像会议纪要

```
Open question:
- Agent A says API response shape unchanged based on controller.
- Agent B says generated client expects `orderStatus`.
- Need to inspect API contract test before implementation.
```

多 Agent 的价值在于暴露视角差异，不是把差异自动抹平。

34.17　多 Agent 的合并报告模板

主 Agent 最后应该输出一份合并报告，而不是把子 Agent 原文全贴出来。

可以这样写：

清单 34-12　多 Agent 的合并报告模板

```
## Merged Findings

### Confirmed

- Finding A ...
  - evidence: Agent 1, file ...
  - action: ...

### Duplicates merged

- Agent 1 and Agent 2 both reported ...

### Conflicts

- Agent 2 says response shape unchanged.
- Agent 3 found generated client expects `orderStatus`.
- action: inspect API contract test before implementation.

### Discarded

- Agent 1 suggested refactor, but it is outside current task.

### Next step

- update plan before editing files.
```

这份报告体现主 Agent 的责任：去重、判断、标冲突、决定下一步。

如果没有合并报告，多 Agent 的结果只是几段并排材料，无法进入工程执行。

34.18　多 Agent 也要有 stop 条件

子 Agent 发现下面情况时应该停：

· 需要写文件，但任务是只读。

- 需要访问未授权 MCP tool。
- 发现任务边界不清。
- 发现和其他子任务可能冲突。
- 发现当前需求和 spec 冲突。

输出应该是：

```
Stopped:
- reason
- evidence
- required decision
```

不要让子 Agent 自己扩大范围。并行任务一旦各自扩大，主 Agent 很难收回来。

34.19　最终验证只能对集成结果负责

多 Agent 任务里，子 Agent 可能分别说自己的部分通过了。

这不等于整体通过。

例如：

```
Agent A：后端测试通过。
Agent B：前端组件测试通过。
Agent C：文档检查通过。
```

这些只能说明各自范围内没有发现问题。

主 Agent 还要在合并后的工作树上跑最终验证：

```
当前 diff 是否包含所有子任务。
子任务之间是否改了同一接口。
测试是否基于合并后的代码运行。
浏览器或 API 验证是否使用最新构建。
最终报告是否只引用最新命令输出。
```

不要把子 Agent 的局部结果拼成整体结论。

正确说法是：

清单 34-13　最终验证只能对集成结果负责

```
Sub-agent evidence collected.
Integrated diff reviewed by main agent.
Final verification run on merged workspace:
```

```
- command: ...
- result: ...
Residual risk: ...
```

多 Agent 可以并行收集证据，但完成声明只能基于最终集成结果。

子 Agent 的原始输出也不要直接放进 PR 或发布文档。

它可能包含未验证推断、重复结论、临时猜测或已经被主 Agent 否掉的建议。

进入交付材料的，应该是主 Agent 合并后的结论：

```
采纳了什么。
为什么采纳。
丢弃了什么。
为什么丢弃。
还剩哪些风险。
```

这样 review 才看的是真正的工程决策，而不是几段并行对话记录。

34.20 一个写权限拆分的反例

下面这种拆分很常见，也很危险：

```
Agent A：改后端接口。
Agent B：改前端调用。
Agent C：改测试。
```

看起来分工清楚，实际风险很高。

后端接口还没定，前端就可能按自己的理解改。

测试 Agent 可能根据前端实现写断言，而不是根据需求写断言。

最后三个 diff 能合并，但不一定表达同一个设计。

更合适的顺序是：

```
先由主 Agent 固定 API contract。
再补 contract test 或 failing test。
后端实现通过 contract。
前端按 contract 调用。
最后统一跑前后端验证。
```

如果要并行，只并行只读部分：

```
Agent A：查现有后端接口。
Agent B：查前端调用点。
Agent C：查现有测试覆盖。
```

写代码阶段保持主线顺序。

多 Agent 不是把一个强依赖链拆开同时写。强依赖链要先定合同，再顺序推进。

34.21 常见错误

错误一：把并行当成默认提效。

并行会增加 token、合并和审查成本。只有独立任务才值得拆。

错误二：多个 Agent 同时改同一批代码。

这通常会制造接口漂移和冲突。实现任务优先小步串行。

错误三：没有统一输出格式。

没有格式，主 Agent 无法合并，只能转述。

错误四：没有主 Agent 决策。

子 Agent 给的是局部结论。最终取舍必须统一。

错误五：让多 Agent 共享可变环境。

调试、数据库、dev server、端口和消息队列都容易互相污染。

错误六：把子 Agent 的结论当成事实。

子 Agent 也会错。结论必须有文件、命令、官方资料或运行结果支撑。

34.22 改完后重点看什么

一次多 Agent 任务结束后，看这些结果：

- 子任务是否真的独立。
- 每个 Agent 是否只做了自己的范围。
- 是否有统一输出格式。
- 是否有证据。
- 是否存在互相冲突的结论。
- 主 Agent 是否做了去重和取舍。
- 最终 diff 是否仍然小。

· 验证是否由最终集成结果触发，而不是子 Agent 分别声明完成。

如果这些做不到，多 Agent 就没有提高稳定性。

回到单 Agent、小任务、顺序执行。

34.23 常见问题

多 Agent 适合个人程序员吗?

适合，但不要从并行实现开始。先用于探索、审查和验证。

什么时候一定不要拆?

需求不清、共享状态、同文件修改、强副作用操作，都不要拆。

子 Agent 能不能自己提交代码?

个人工作流里不建议。让它返回 diff 和证据，最终由主 Agent 或程序员决定。

多 Agent 能减少 review 吗?

不能。它可以增加审查视角，但最终审查责任还在主 Agent 和程序员。

多 Agent 和自定义 Skill 谁优先?

先有清楚流程，再拆 Agent。没有 Skill 或明确规则，多 Agent 只会把不稳定流程并行化。

34.24 可复查资料

- OpenAI Codex Subagents：https://developers.openai.com/codex/subagents〔34-1〕
- OpenAI Codex Subagent concepts：https://developers.openai.com/codex/concepts/subagents〔34-2〕
- Anthropic Claude Code Subagents：https://code.claude.com/docs/en/sub-agents〔34-3〕
- GitHub Copilot cloud agent best practices：https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results〔34-4〕
- Superpowers `subagent-driven-development`：https://github.com/obra/superpowers/blob/main/skills/subagent-driven-development/SKILL.md〔34-5〕

本章资源

〔34-1〕 https://developers.openai.com/codex/subagents
https://developers.openai.com/codex/subagents

〔34-2〕 https://developers.openai.com/codex/concepts/subagents
https://developers.openai.com/codex/concepts/subagents

〔34-3〕 https://code.claude.com/docs/en/sub-agents
https://code.claude.com/docs/en/sub-agents

〔34-4〕 https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results
https://docs.github.com/en/copilot/tutorials/cloud-agent/get-the-best-results

〔34-5〕 https://github.com/obra/superpowers/blob/main/skills/subagent-driven-development/SKILL.md
https://github.com/obra/superpowers/blob/main/skills/subagent-driven-development/SKILL.md

第35章　模块实战：用 Superpowers 固化一次业务需求的调试和评审

Superpowers 不是“装几个 Skill 就变专业”。

它真正有用的地方，是在真实需求里把几个高风险动作固定下来：先看需求合同，先写失败测试，失败时先找根因，完成前跑新鲜验证，review 时先找行为风险。

Superpowers 的 Skill 链负责约束 Agent 的阶段动作，不负责重新定义业务需求。

所以 Superpowers 实战不能停在“订单退款”这类口头例子上，要回到已经建好需求合同的产品需求里，看它怎么在真实开发过程中约束 TDD、调试、review 和完成前验证。

配套代码仓库 DavidDingXu/learn-ai-coding〔35-1〕 按一个大仓库组织，仓库根目录下有业务 module 和工具 module。这里继续使用业务 module：`personal-productivity-app`，不重新换案例。

这里使用个人效率助手的需求：

```
今日计划自动纳入逾期未完成任务，并在每日容量限制下返回顺延列表。
```

对应过程记录在：

```
learn-ai-coding/personal-productivity-app/dev-process/02-superpowers-today-plan-workflow.md
```

前一篇已经用 OpenSpec 建了 change：

```
learn-ai-coding/personal-productivity-app/openspec/changes/today-plan-overdue-rollover/
```

这次不重新发明需求，只讲 Superpowers 如何约束开发过程。



图 35-1　Superpowers 调试评审循环

35.1　先把入口说准

Superpowers 的入口会随版本和宿主环境变化：可能是 Codex Skill、Claude Skill、插件命令，也可能沉淀进项目规则。

真实使用时，不应该背某个固定命令，而应该确认当前环境里真正生效的约束。

在当前 Superpowers 安装形态里，早期的 `/brainstorm`、`/write-plan`、`/execute-plan` 已提示改用对应 Skill。因此这里不把旧 slash command 当推荐入口。

这次需求实际使用的是这些 Skill 约束：

```
superpowers:test-driven-development
superpowers:systematic-debugging
superpowers:requesting-code-review
superpowers:receiving-code-review
superpowers:verification-before-completion
```

迁移到自己的项目时，重点不是入口长得像不像这个示例，而是两个检查：

```
1. 当前工具环境里推荐入口是什么，不沿用旧资料里的过期命令。
2. TDD、调试、review、完成前验证、review 反馈处理这些高风险节点有没有被规则约束。
```

35.2　分工：OpenSpec 管需求，Superpowers 管纪律

这次需求已经有 OpenSpec 产物：

```
proposal.md
design.md
specs/today-plan/spec.md
tasks.md
```

Superpowers 不应该再定义一个新需求。

正确分工是：

示例 35-1　分工：OpenSpec 管需求，Superpowers 管纪律

```
OpenSpec：需求边界、行为合同、任务列表。
TDD：先写测试并看到红灯。
systematic-debugging：失败时先定位根因。
requesting-code-review：完成后按风险审查 diff。
verification-before-completion：完成前跑新鲜验证。
receiving-code-review：处理反馈时不扩大 scope。
```

如果 Skill 建议和 OpenSpec 边界冲突，以当前 change 为准，并把冲突报告出来。

35.3　TDD：红灯必须是真的

TDD Skill 的核心规则很硬：

```
NO PRODUCTION CODE WITHOUT A FAILING TEST FIRST
```

本次先有测试：

```
test/today-plan.test.js
```

第一次运行：

清单 35-2　TDD：红灯必须是真的

```
npm test
```

红灯：

```
Error [ERR_MODULE_NOT_FOUND]: Cannot find module
'/learn-ai-coding/personal-productivity-app/src/today-plan.js'
```

这不是“测试挂了所以随便修”。

它说明：测试已经表达了 `buildTodayPlan` 的业务行为，但生产文件还不存在。

如果测试第一次就绿，说明你可能只是在验证旧行为。

如果测试因为拼写或 import 错误失败，也不能算有效红灯。

35.4　systematic-debugging：先找根因再修

系统化调试不是只在复杂 bug 里用。

这次红灯看起来简单，但也要按顺序确认：

```
1. 读完整错误：ERR_MODULE_NOT_FOUND。
2. 复现命令：npm test。
3. 查测试 import：../src/today-plan.js。
4. 查项目文件：src/today-plan.js 不存在。
5. 得出根因：缺少领域函数实现。
```

因此修复范围被限制为：

```
新增 src/today-plan.js。
不改测试期望。
页面展示不放进这次调试修复。
不接数据库。
不引入第三方依赖。
```

很多 AI Coding 的大 diff，就是因为跳过这一步。

失败一出现，AI 直接开始“顺手重构”，最后测试可能绿了，但需求边界也被改没了。

35.5　实现：只实现当前任务

OpenSpec tasks 里写的是：

```
2.1 Add src/today-plan.js and export buildTodayPlan.
2.2 Filter eligible tasks to open overdue and due-today tasks only.
2.3 Sort eligible tasks by overdue status, priority, due date, and stable id fallback.
2.4 Apply daily capacity and return items, deferred, usedMinutes, remainingMinutes.
2.5 Validate date and estimate inputs at the domain boundary.
```

实现文件：

```
src/today-plan.js
```

核心规则：

示例 35-3　实现：只实现当前任务

```
status 必须是 open。
dueDate 小于 date -> overdue。
dueDate 等于 date -> due-today。
future 不进入候选。
done / blocked 不进入候选。
逾期优先于当天。
优先级 high -> medium -> low。
放不进容量的进入 deferred。
```

这已经足够满足当前产品需求。

如果此时 AI 提议新增页面、接日历或做复杂优化，要直接挡住：那是新需求。

页面当然可以存在，但它只能复用领域函数展示输出。如果为了页面重写排序、过滤或容量逻辑，就已经越过了当前调试任务的边界。

35.6　Review：看行为风险，不写空泛总结

`requesting-code-review` 的价值不是生成一句“整体实现清晰”。

本次 review 应该围绕需求合同问：

示例 35-4　Review：看行为风险，不写空泛总结

```
1. 是否误把 done / blocked 任务放进今日计划?
2. 是否误把 future 任务放进今日计划?
3. overdue 是否一定排在 due-today 前?
4. priority 是否优先于 dueDate age?
5. selected items 是否可能超过 capacityMinutes?
6. deferred 是否保留放不下的候选任务?
7. usedMinutes / remainingMinutes 是否和 items 一致?
8. 输入校验是否拒绝非法日期、负容量、负估时?
9. 实现是否偷偷依赖系统时间、I/O 或外部服务?
```

这类 review 才能迁移到真实项目。

比如在订单、工单、审批、账单这些业务里，review 也应该看状态过滤、时间边界、排序规则、容量限制和副作用边界。

35.7　完成前验证：不能引用旧输出

`verification-before-completion` 的规则也很硬：

```
NO COMPLETION CLAIMS WITHOUT FRESH VERIFICATION EVIDENCE
```

本次完成前跑三条：

清单 35-5　完成前验证：不能引用旧输出

```
openspec validate today-plan-overdue-rollover --strict --json --no-interactive
npm test
openspec instructions apply --change today-plan-overdue-rollover --json
```

结果：

```
OpenSpec strict validate: passed 1, failed 0
npm test: pass 7, fail 0
OpenSpec apply: total 8, complete 8, remaining 0
```

完成报告应该写这些证据，而不是写：

```
已完成，测试通过。
```

后者对 reviewer 没有价值。

35.8　receiving-code-review：别把反馈变成新需求

收到 review 后也要有边界。

对这次需求，反馈可以分三类：

必须修：

```
容量可能超限。
排序和 spec 不一致。
done / blocked 被误收。
deferred 丢失任务。
测试缺少关键场景。
```

可选修：

```
错误文案更清楚。
局部函数命名更准确。
测试 helper 可读性提升。
```

不在当前需求修：

```
新增 UI。
接数据库。
接日历。
做提醒系统。
做全局最优排程。
```

这一步能防止 review 变成新的大需求。

35.9　Skill 不替代工程判断

Superpowers 解决的是流程纪律，不替代你判断业务边界。

它不能替你决定：

```
blocked 任务是否应该单独提醒。
低优先级旧任务是否永远排在高优先级新任务之后。
容量应该按分钟、番茄钟还是工作块计算。
是否要区分工作、生活、学习不同桶。
```

这些仍然要写进产品需求或 OpenSpec change。

Skill 负责的是：当你开始做时，不要跳过红灯、根因、review 和验证。

35.10　迁移到自己的项目

你可以不用 Superpowers 插件，但建议保留这条工作流：

示例 35-6　迁移到自己的项目

```
需求合同：OpenSpec / spec / issue
红灯测试：先让行为失败
根因调试：先复现、读错误、定位
小步实现：只做当前任务
风险评审：按行为合同找问题
新鲜验证：完成前重新跑命令
反馈处理：分类，不扩大 scope
```

给 AI 的指令可以这样写：

示例 35-7　迁移到自己的项目

```
以 openspec/changes/today-plan-overdue-rollover/tasks.md 为任务来源。
实现前使用 superpowers:test-driven-development，先确认失败测试。
如果测试失败，使用 superpowers:systematic-debugging，先记录第一个失败信号和根因假设。
实现完成后按 requesting-code-review 的方式审查状态过滤、容量、排序和 deferred 行为。
完成前使用 verification-before-completion，重新运行 openspec validate、npm test 和 openspec
instructions apply。
收到 review 后使用 receiving-code-review 分类处理，超出当前 change 的建议不混进本次 diff。
```

35.11　配套资料

- Superpowers 过程记录：`learn-ai-coding/personal-productivity-app/dev-process/02-superpowers-today-plan-workflow.md`
- OpenSpec 过程记录：`learn-ai-coding/personal-productivity-app/dev-process/01-openspec-today-plan-rollover.md`
- OpenSpec 变更包：`learn-ai-coding/personal-productivity-app/openspec/changes/today-plan-overdue-rollover/`
- 领域实现：`learn-ai-coding/personal-productivity-app/src/today-plan.js`
- 业务测试：`learn-ai-coding/personal-productivity-app/test/today-plan.test.js`
- 结果页：`learn-ai-coding/personal-productivity-app/index.html`
- 页面演示测试：`learn-ai-coding/personal-productivity-app/test/today-plan-demo.test.js`
- Superpowers 上游仓库：https://github.com/obra/superpowers〔35-2〕

本章资源

〔35-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔35-2〕 https://github.com/obra/superpowers
https://github.com/obra/superpowers

第36章　模块实战：给 AI 接一个只读项目工具

给 AI 接工具，最容易走偏的地方不是 MCP 配置。

真正危险的是：工具还没定义清楚读写权限、失败行为和验证方式，就开始让 Agent 调用。

配套代码仓库 DavidDingXu/learn-ai-coding〔36-1〕 按一个大仓库组织，仓库根目录下有业务 module 和工具 module。这里用工具 module：`ai-coding-workflow-kit`。它适合验证工具权限边界，不替代业务 module 的综合实战。

这一篇用 `ai-coding-workflow-kit` 的 `context-pack` 需求做案例。过程记录在：

```
learn-ai-coding/ai-coding-workflow-kit/dev-process/05-readonly-tool-boundary.md
```

对应真实需求是：

> 在接外部工具前，先做一个只读项目工具 `context-pack`，再判断是否值得包装成 MCP。

重点不是执行 `context-pack` 看输出，而是记录一个工具从需求、权限边界、失败行为、验证方式，到 MCP 判断的完整过程。

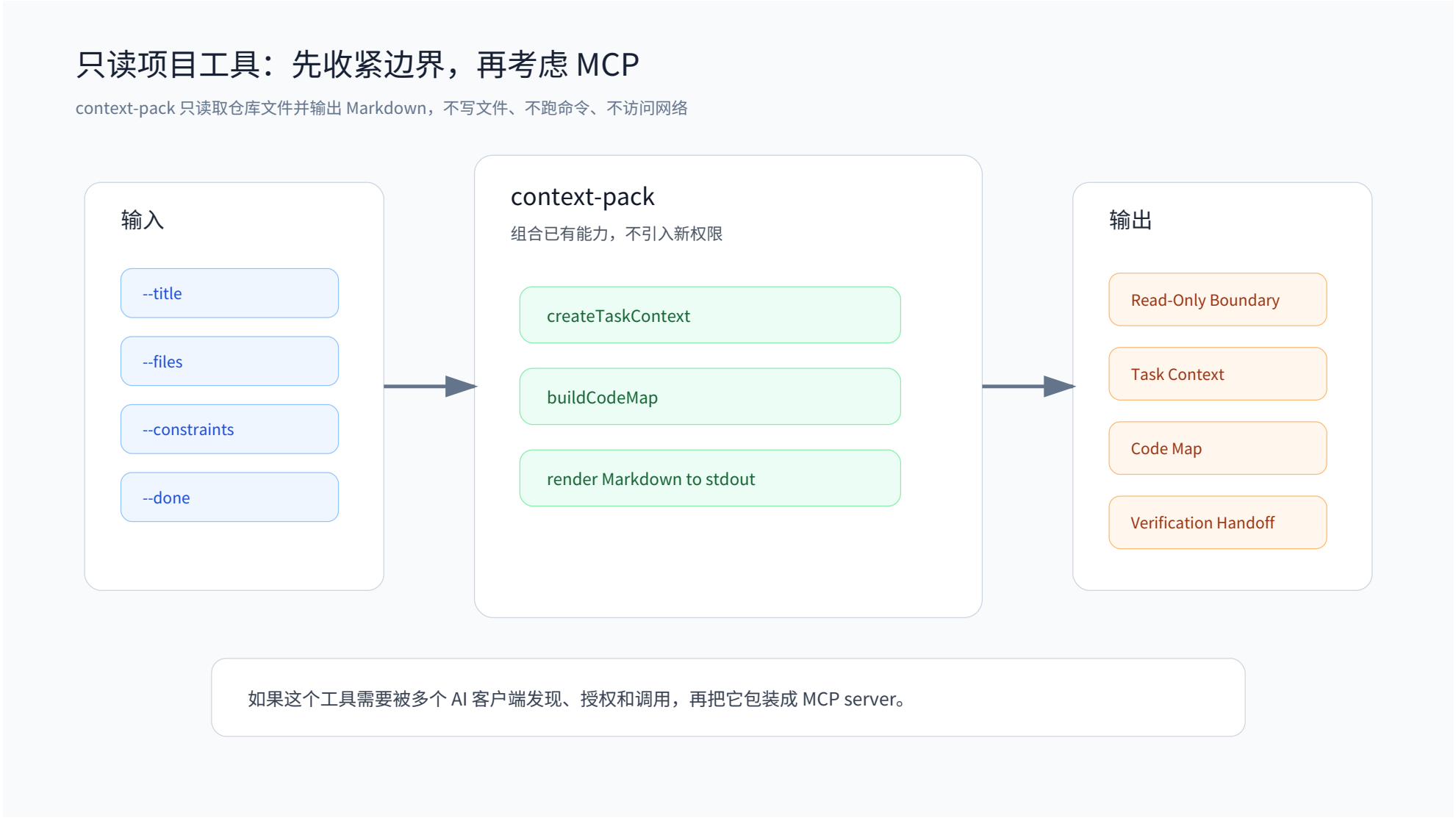

只读项目工具：先收紧边界，再考虑 MCP
context-pack 只读取仓库文件并输出 Markdown，不写文件、不跑命令、不访问网络
输入
--title
--files
--constraints
--done
context-pack
组合已有能力，不引入新权限
createTaskContext
buildCodeMap
render Markdown to stdout
输出
Read-Only Boundary
Task Context
Code Map
Verification Handoff
如果这个工具需要被多个 AI 客户端发现、授权和调用，再把它包装成 MCP server。

图 36-1　只读工具边界

36.1 先问工具要解决什么问题

context-pack 要解决的问题很具体：

```
AI 接手任务时，需要一份紧凑交接材料。
这份材料应该包含任务目标、相关文件、代码地图和验证提示。
已有 codemap 和 task-context，但需要人工拼接。
```

所以这次需求不是：

```
做一个万能项目助手。
做一个 MCP server。
自动读取 Issue。
自动执行 npm test。
自动生成并写入 context-pack.md。
```

它只做一件事：

```
把任务上下文、代码地图和验证提醒组合成 Markdown，输出到 stdout。
```

真实项目里，工具需求越小，越要把边界写清楚。因为工具一旦接给 Agent，Agent 会倾向于把它当作“可以继续做更多事”的能力。

36.2 权限边界先于实现

过程记录里的 Boundary Checklist 很直接：

示例 36-1 权限边界先于实现

```
能读什么?
- root 下的文件树和指定文件路径。

能写什么?
- 什么都不写，只输出 stdout。

能执行什么?
- 不执行项目命令，只把验证命令写成提醒。

能联网吗?
- 不能。

失败时怎么表现?
- 参数缺失走 CLI 错误路径。

输出怎么验证?
- 测试检查只读边界文本、task context、code map 和 done 提示。
```

这段清单就是工具需求的一部分。

如果它缺失，AI 很可能顺手做这些事：

```
把输出写成 context-pack.md。
自动运行 npm test 并收集结果。
读取浏览器、Issue 或网络文档。
把项目扫描扩大到不相关目录。
```

这些能力不一定永远不能做，但都不属于这次需求。

36.3　实现先做本地安全内核

本次工具先做 CLI 安全内核，而不是直接接 MCP。

涉及文件是：

```
src/lib/context-pack.js
src/cli.js
test/context-pack.test.js
README.md
openspec/changes/context-pack-readonly/*
```

核心实现复用已有能力：

```
createTaskContext
buildCodeMap
renderCodeMapMarkdown
```

`src/lib/context-pack.js` 输出包括：

```
Read-Only Boundary
Task Context
Repository Code Map
Verification Handoff
```

这几个 section 都是有意设计的。

`Read-Only Boundary` 告诉人和 Agent：这个工具不能写文件、不能执行命令、不能联网。

`Verification Handoff` 则说明：验证命令只是提醒，真正执行和记录结果要在任务验证阶段完成。

36.4　给 AI 的实现指令

这类需求不要写成：

```
给项目加一个 context-pack 工具。
```

应该写成：

示例 36-2 给 AI 的实现指令

```
请读取：
- learn-ai-coding/ai-coding-workflow-kit/dev-process/05-readonly-tool-boundary.md
- learn-ai-coding/ai-coding-workflow-kit/openspec/changes/context-pack-readonly/design.md
- learn-ai-coding/ai-coding-workflow-kit/src/lib/codemap.js
- learn-ai-coding/ai-coding-workflow-kit/src/lib/task-context.js
- learn-ai-coding/ai-coding-workflow-kit/src/cli.js
- learn-ai-coding/ai-coding-workflow-kit/test/context-pack.test.js

只实现只读 context-pack：
1. 组合 task context 和 code map。
2. 输出 read-only boundary。
3. 输出 verification handoff。
4. 不写文件。
5. 不执行项目命令。
6. 不访问网络。

如果你认为需要 MCP、写文件或执行命令，先报告理由，不要直接实现。
```

这段指令先保护权限边界，再谈实现。

36.5 验证工具边界

自动测试：

清单 36-3 验证工具边界

```
cd learn-ai-coding/ai-coding-workflow-kit
npm test
```

目标命令：

清单 36-4 验证工具边界

```
node src/cli.js context-pack \
  --title "Inspect CLI contract" \
  --files "src/cli.js,test/cli.test.js" \
  --done "npm test"
```

要检查的不是“输出很多 Markdown”，而是：

示例 36-5　验证工具边界

```
是否包含 Read-Only Boundary。
是否说明 Does not edit files。
是否说明 Does not execute project commands。
是否说明 Does not access network resources。
是否包含 Task Context。
是否包含 Code Map。
是否把 npm test 放在 Verification Handoff，而不是自动执行。
```

如果工具自动写文件或执行命令，即使看起来更方便，也违反了这次需求。

36.6　什么时候才需要 MCP

过程记录里的 MCP 判断很克制：

```
暂时不包装成 MCP。
先保留 CLI。
后续如果多个 Agent 或编辑器都需要稳定调用，再考虑 MCP。
```

理由是：

```
还没有多个客户端复用这个能力的需求。
还没有外部资源接入需求。
只读本地仓库和 stdout 已经满足当前任务。
```

很多 MCP 接入失败，不是协议难，而是安全内核还没定义清楚。

如果以后要包装 MCP，也应该保留同一份契约：

示例 36-6　什么时候才需要 MCP

```
输入是什么。
读取范围是什么。
是否写文件。
是否执行命令。
是否联网。
输出结构是什么。
失败怎么返回。
本地 CLI 如何验证。
```

MCP 是接入层，不应该改变工具内核的权限边界。

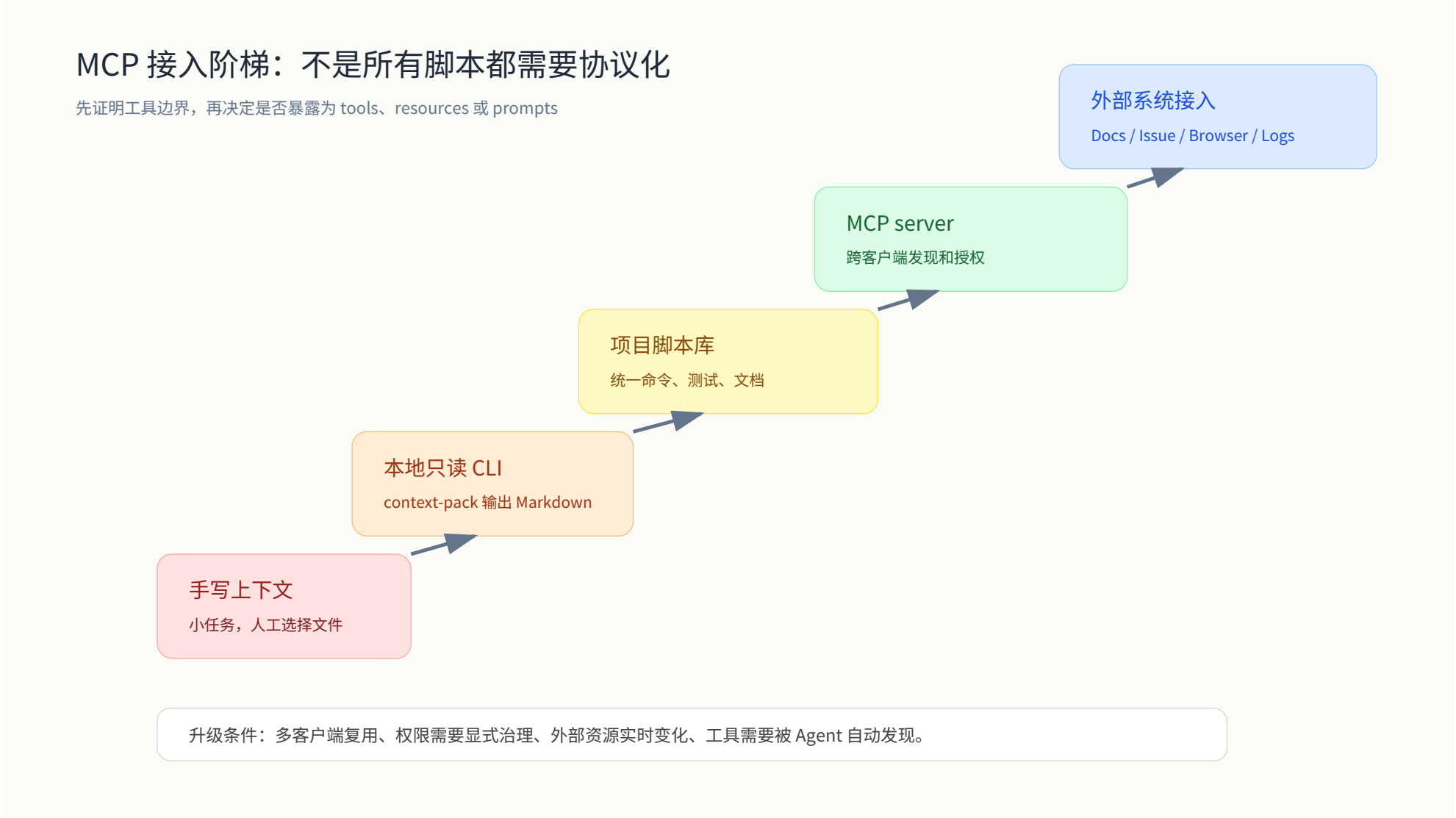


图 36-2　MCP 采用阶梯

36.7 Review 要盯权限扩张

这类工具的代码审查重点不是语法，而是权限有没有扩大。

重点看：

示例 36-7 Review 要盯权限扩张

```
有没有 fs.writeFile。
有没有 child_process。
有没有 fetch/http/network。
有没有读取 root 之外的路径。
有没有把验证命令执行掉。
有没有新增依赖。
有没有把错误吞掉。
```

在当前实现里，`context-pack` 只组合 `codemap` 和 `task-context`，输出 Markdown。

如果 diff 里出现写文件、执行 shell、请求网络，就要退回到需求层重新讨论。

36.8 迁移到你的项目

给 AI 接任何项目工具前，先写一张边界卡。

示例 36-8 迁移到你的项目

```
工具名：
解决的问题：
输入：
读取范围：
写入范围：
是否执行命令：
是否联网：
输出格式：
失败行为：
验证方式：
不做什么：
```

然后再让 AI 实现。

对于多数个人项目，推荐顺序是：

```
先做只读 CLI。
用测试保护输入输出。
用 README 写清命令和边界。
真实使用几次后，再决定是否需要 MCP。
```

不要一上来就把工具做成 Agent 可以写文件、执行命令、访问外部系统的全能助手。

36.9 常见错误

错误一：把 MCP 当第一步。

如果工具本身的权限、输出和失败行为没定义清楚，包装成 MCP 只会把风险扩散。

错误二：为了方便让工具自动执行命令。

验证命令应该由任务验证阶段执行并记录。上下文工具只负责提醒。

错误三：只测输出，不测边界文本。

只读工具的输出必须明确告诉使用者它不会写文件、执行命令或联网。

错误四：把工具输出当正确性证明。

`context-pack` 是导航材料，不证明代码正确。

错误五：没有保留本地验证路径。

即使以后包装 MCP，也应该保留 CLI 或单元测试，方便在没有 Agent 的情况下验证内核。

36.10 可复查资料

- 只读工具边界过程记录：`learn-ai-coding/ai-coding-workflow-kit/dev-process/05-readonly-tool-boundary.md`
- `context-pack` 需求记录：`learn-ai-coding/ai-coding-workflow-kit/dev-process/03-openspec-context-pack.md`
- `context-pack` 实现：`learn-ai-coding/ai-coding-workflow-kit/src/lib/context-pack.js`
- CLI 入口：`learn-ai-coding/ai-coding-workflow-kit/src/cli.js`
- 测试文件：`learn-ai-coding/ai-coding-workflow-kit/test/context-pack.test.js`
- OpenSpec 变更包：`learn-ai-coding/ai-coding-workflow-kit/openspec/changes/context-pack-readonly/`

本章资源

〔36-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

PART 08

第8篇　串起一次完整交付

通过真实需求连接上下文、规格、开发、验证与复盘。检查各阶段产物是否围绕同一个用户问题。

阅读重点

能复盘一条完整需求，并提取有用的工程资产。

第37章　综合实战：一个真实需求的 AI Coding 最佳实践

真实需求不会按工具章节出现。它通常同时牵涉上下文、spec、TDD、debug、Skill、OpenSpec 和验证。

如果这些能力只停留在单点命令，真实需求进来时仍然很容易断链。

下面用同一个产品需求把整条链路走完：

```
个人效率助手每天生成今日计划时，自动纳入逾期未完成任务，并在每日容量限制下返回顺延列表。
```

配套代码仓库 DavidDingXu/learn-ai-coding〔37-1〕 按一个大仓库组织，仓库根目录下有业务 module 和工具 module。这里使用业务 module：`personal-productivity-app`。工具 module 里的 CLI、context-pack、verification ledger 可以提供方法样例，但不能替代这一条业务链路。

端到端过程记录在：

```
learn-ai-coding/personal-productivity-app/dev-process/03-end-to-end-today-plan-delivery.md
```

这不是工具项目里的命令演示，而是一个可迁移的业务规则需求：有状态、有日期、有优先级、有容量、有测试、有 review 边界。



图 37-1　端到端交付证据链

37.1　第一步：把需求翻译成业务边界

原始需求可以很口语：

```
今日计划别漏掉昨天没做完的任务，但也别把今天排爆。
```

AI 如果直接改代码，容易把需求扩大成：

```
做一个任务系统。
做一个日历同步。
做一个提醒服务。
做一个复杂排程算法。
```

真实开发里，第一步不是写代码，而是限定边界。

本次做：

示例 37-1　第一步：把需求翻译成业务边界

```
buildTodayPlan 领域函数。
逾期任务和当天任务候选选择。
done / blocked / future 过滤。
priority / dueDate 排序。
capacityMinutes 容量约束。
items / deferred 输出。
今日计划结果页。
容量调节后即时重算。
```

本次不做：

示例 37-2　第一步：把需求翻译成业务边界

```
完整任务系统页面。
API。
数据库。
账号系统。
提醒系统。
日历同步。
复杂排程优化。
```

这一步是后续所有工具能发挥作用的前提。

37.2　第二步：把上下文交给 AI

给 AI 的上下文不是“你帮我做一下”。

最少应该包含：

示例 37-3 第二步：把上下文交给 AI

```
README.md
package.json
test/today-plan.test.js
openspec/changes/today-plan-overdue-rollover/proposal.md
openspec/changes/today-plan-overdue-rollover/specs/today-plan/spec.md
openspec/changes/today-plan-overdue-rollover/design.md
openspec/changes/today-plan-overdue-rollover/tasks.md
```

这些文件分别回答：

示例 37-4 第二步：把上下文交给 AI

```
项目是什么。
怎么跑测试。
需求为什么做。
行为合同是什么。
实现边界是什么。
任务怎么拆。
```

如果 AI 只读测试文件，可能知道要实现什么。

但它不知道为什么 UI 只能展示已有领域规则、为什么 blocked 不进入计划、为什么 priority 要优先于 dueDate age。

上下文不是越多越好，而是要覆盖决策所需事实。

37.3 第三步：用 OpenSpec 固定需求合同

OpenSpec 本次真实命令链是：

清单 37-5 第三步：用 OpenSpec 固定需求合同

```
openspec init --tools codex .

openspec new change today-plan-overdue-rollover \
  --description "今日计划自动纳入逾期未完成任务，并在每日容量限制下给出顺延列表。" \
  --goal "让个人效率助手每天生成可执行的今日计划，避免逾期任务丢失或一天被排爆。" \
  --json

openspec instructions proposal --change today-plan-overdue-rollover --json
openspec instructions specs --change today-plan-overdue-rollover --json
openspec instructions design --change today-plan-overdue-rollover --json
openspec instructions tasks --change today-plan-overdue-rollover --json
openspec validate today-plan-overdue-rollover --strict --json --no-interactive
```

生成的核心文件：

```
proposal.md
design.md
specs/today-plan/spec.md
tasks.md
```

最关键的不是文件名，而是它们让需求变成可审合同。

proposal 写为什么做。

spec 写 SHALL / MUST 和 scenario。

design 写技术取舍和不做什么。

tasks 写每一步怎么完成。

37.4　第四步：先红灯再实现

测试先表达业务行为：

```
逾期任务进入今日计划，且排在当天任务前。
done / blocked 不进入今日计划。
容量不足时放不下的任务进入 deferred。
逾期任务按 priority 优先于 dueDate age。
非法日期、负容量、负估时被拒绝。
```

第一次运行：

清单 37-6　第四步：先红灯再实现

```
npm test
```

红灯：

```
ERR_MODULE_NOT_FOUND: Cannot find module 'src/today-plan.js'
```

这个红灯把实现范围钉住了：

```
要新增 src/today-plan.js。
测试期望不用改。
没有理由新增数据库或外部服务。页面展示必须复用 `buildTodayPlan`，不能另写一套规则。
```

37.5　第五步：小 diff 实现领域规则

实现文件：

```
src/today-plan.js
```

核心函数：

清单 37-7 第五步：小 diff 实现领域规则

```
export function buildTodayPlan({ date, capacityMinutes = 0, tasks = [] } = {}) {
  // 过滤、排序、容量选择、输出 items/deferred
}
```

核心实现边界：

示例 37-8 第五步：小 diff 实现领域规则

```
不读取系统时间。
不访问网络。
不写文件。
不引入依赖。
不做持久化。
页面只展示 `buildTodayPlan` 的结果，不接后端。
```

这就是小 diff 的意义：只改当前需求需要的地方。

37.6 第六步：用 Superpowers 约束过程

本次 Superpowers 没有替代 OpenSpec。

它约束的是执行纪律：

```
test-driven-development：实现前先红灯。
systematic-debugging：测试失败时先找根因。
requesting-code-review：完成后按行为风险审查。
verification-before-completion：完成前跑新鲜验证。
receiving-code-review：处理反馈时不扩大 scope。
```

这里也核对了当前入口：本机 `brainstorm`、`write-plan`、`execute-plan` 命令文件都标了 deprecated，当前写法以 Skill 触发为准。

所以不能把旧命令当成当前推荐路径。

37.7 第七步：验证不是一句话

本次验证有四层。

OpenSpec strict validate：

清单 37-9 第七步：验证不是一句话

```
openspec validate today-plan-overdue-rollover --strict --json --no-interactive
```

结果：

```
passed: 1
failed: 0
```

业务测试：

清单 37-10 第七步：验证不是一句话

```
npm test
```

结果：

```
tests 7
pass 7
fail 0
```

OpenSpec apply 进度：

清单 37-11 第七步：验证不是一句话

```
openspec instructions apply --change today-plan-overdue-rollover --json
```

结果：

```
total: 8
complete: 8
remaining: 0
state: all_done
```

这才叫完成证据。

只写“已完成，测试通过”，reviewer 不知道通过了什么，也不知道需求合同是否仍然有效。

最终页面也要验证。启动本地页面：

清单 37-12 第七步：验证不是一句话

```
npm run dev
```

打开：

```
http://127.0.0.1:4173
```

默认 90 分钟容量下，页面应该展示：

```
今日执行：补齐客户发票资料、回复客户验收问题、写本周项目进展
顺延列表：整理版本发布说明
未进入计划：准备下周路线评审、提交差旅报销、等待法务反馈合同条款
```

把容量调到 60 分钟后，页面应该重新计算：

```
今日执行：补齐客户发票资料、写本周项目进展
顺延列表：回复客户验收问题、整理版本发布说明
已安排：55 分钟
剩余：5 分钟
```

这一步不是把样例扩成完整产品，而是确认业务规则确实落到了用户可见结果里。



图 37-2 今日计划最终结果页

37.8 第八步：Review 看业务风险

这次 review 应该盯这些问题：

示例 37-13 第八步：Review 看业务风险

```
done / blocked 是否被误收?
future 是否被误收?
overdue 是否排在 due-today 前?
priority 是否先于 dueDate age?
capacity 是否可能超限?
deferred 是否保留放不下的任务?
remainingMinutes 是否计算正确?
实现是否依赖系统时间或外部副作用?
```

这些问题比“代码是否优雅”更重要。

因为真实用户只关心：今天看到的计划是否可信。

37.9 第九步：复盘沉淀可迁移规则

这次需求之后，值得沉淀的不是 today-plan 这个函数名。

真正可迁移的是：

```
有状态、日期、优先级、容量边界的需求，先写 spec 再实现。
业务实战必须先写用户可见行为，不要用工具命令冒充业务需求。
OpenSpec 实战必须有真实 CLI 命令和 validate 输出。
Superpowers 实战必须写当前 Skill 入口和触发条件。
完成前必须重新跑验证命令，不能引用旧输出。
```

这些规则可以迁移到：

```
订单超时关闭。
工单 SLA 升级。
审批任务催办。
会员权益到期提醒。
账单分期滚动。
```

37.10 给 AI 的完整执行指令

真实项目里可以这样交给 AI：

示例 37-14 给 AI 的完整执行指令

```
需求：今日计划自动纳入逾期未完成任务，并在每日容量限制下返回顺延列表。

请先读取：
- README.md
- package.json
- openspec/changes/today-plan-overdue-rollover/proposal.md
- openspec/changes/today-plan-overdue-rollover/specs/today-plan/spec.md
```

```
- openspec/changes/today-plan-overdue-rollover/design.md
- openspec/changes/today-plan-overdue-rollover/tasks.md
- test/today-plan.test.js

执行要求：
1. 以 tasks.md 为 source of truth。
2. 实现前运行 npm test，记录红灯。
3. 先实现 src/today-plan.js；页面展示必须复用该函数，不接数据库、API 或外部依赖。
4. 如果测试失败，按 systematic-debugging 先定位第一个失败信号。
5. 完成后运行 npm test。
6. 运行 openspec validate today-plan-overdue-rollover --strict --json --no-interactive。
7. 运行 openspec instructions apply --change today-plan-overdue-rollover --json。
8. 如果改动影响页面展示，运行 npm run dev 并检查 http://127.0.0.1:4173。
9. 汇报修改文件、验证结果和残余风险。
```

这比“帮我实现一下”稳定得多。

37.11 配套资料

- 端到端过程记录：`learn-ai-coding/personal-productivity-app/dev-process/03-end-to-end-today-plan-delivery.md`
- OpenSpec 过程记录：`learn-ai-coding/personal-productivity-app/dev-process/01-openspec-today-plan-rollover.md`
- Superpowers 过程记录：`learn-ai-coding/personal-productivity-app/dev-process/02-superpowers-today-plan-workflow.md`
- OpenSpec 变更包：`learn-ai-coding/personal-productivity-app/openspec/changes/today-plan-overdue-rollover/`
- 领域实现：`learn-ai-coding/personal-productivity-app/src/today-plan.js`
- 结果页：`learn-ai-coding/personal-productivity-app/index.html`
- 页面渲染：`learn-ai-coding/personal-productivity-app/src/ui.js`
- 页面演示数据：`learn-ai-coding/personal-productivity-app/src/today-plan-demo.js`
- 业务测试：`learn-ai-coding/personal-productivity-app/test/today-plan.test.js`
- 页面演示测试：`learn-ai-coding/personal-productivity-app/test/today-plan-demo.test.js`

本章资源

〔37-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

第38章　综合实战：从一次交付沉淀个人规则库

一次需求交付完，不应该只剩一段总结。

真正能提高下一次 AI Coding 稳定性的，是把这次暴露出来的失败模式、判断边界和验证方式沉淀成可复用规则。

这里继续用个人效率助手的需求：

```
今日计划自动纳入逾期未完成任务，并在每日容量限制下返回顺延列表。
```

配套代码仓库 DavidDingXu/learn-ai-coding〔38-1〕 按一个大仓库组织。规则沉淀不能只写工具 module 的命令经验，还要回到业务 module 的需求边界、验证证据和 review 风险。这里使用 `personal-productivity-app`。

规则沉淀记录在：

```
learn-ai-coding/personal-productivity-app/dev-process/04-rule-asset-sedimentation.md
```



图 38-1　规则沉淀反馈循环

38.1　先看这次交付暴露了什么

这次需求并不大，但它暴露出几个常见问题：

示例 38-1　先看这次交付暴露了什么

```
业务实战容易被写成工具实战。
OpenSpec 容易被写成“风格”，而不是实际 CLI 和仓库产物。
Superpowers 容易只写 Skill 名字，不写当前入口和触发条件。
测试容易只看最终绿灯，不记录红灯。
完成汇报容易只写“测试通过”，不写验证证据。
review 容易看代码风格，漏掉业务边界。
```

这些问题在别的项目里也会反复出现。

所以它们值得沉淀。

38.2　规则 1：实战需求必须先写用户可见行为

不能把“执行一个命令”“新增一个工具”“整理一份上下文”直接当综合实战主线。

这些动作可以作为工具能力验证，但很难撑起一条完整交付链路；完整交付更适合从业务或产品需求开始。

判断方式很简单：

```
这个需求解决了哪个用户问题?
这个需求有没有状态、边界、异常、验证?
读者能不能迁移到自己的业务项目?
```

这次选择的是：

```
今日计划别漏掉逾期任务，也别把当天排爆。
```

读者可以迁移到：

```
订单超时关闭。
工单 SLA 升级。
审批任务催办。
会员权益到期提醒。
账单分期滚动。
```

这比“给 AI 工具加一个命令”更容易复用。

38.3　规则 2：OpenSpec 实战必须有真实命令

如果 OpenSpec 只剩这些文件名：

```
proposal.md
design.md
tasks.md
```

还不够。

一次可复查的 OpenSpec 变更至少要能追到这些证据：

清单 38-2　规则 2：OpenSpec 实战必须有真实命令

```
openspec --version
openspec init --tools codex .
openspec new change today-plan-overdue-rollover ...
openspec instructions proposal --change today-plan-overdue-rollover --json
openspec instructions specs --change today-plan-overdue-rollover --json
openspec instructions design --change today-plan-overdue-rollover --json
openspec instructions tasks --change today-plan-overdue-rollover --json
openspec validate today-plan-overdue-rollover --strict --json --no-interactive
openspec instructions apply --change today-plan-overdue-rollover --json
```

以及真实目录：

```
openspec/changes/today-plan-overdue-rollover/
```

以后遇到 OpenSpec 相关交付，这条规则要优先生效：

```
没有真实 CLI 和 validate 输出，就不要写成 OpenSpec 实战。
```

38.4　规则 3：Superpowers 要写当前入口

Superpowers 的入口不是固定在某个 slash command 上。不同宿主环境可能通过 Skill、插件命令或项目规则触发同一套流程。

所以不能把旧 slash command 当作默认最佳实践，而要先核对当前环境推荐的触发方式。

沉淀规则：

```
使用 Superpowers 时，先核对当前平台推荐入口和 Skill/command 状态。
如果旧命令已经 deprecated，就不要把它写成当前推荐入口。
每个 Skill 都必须说明触发条件、输入、输出和验证证据。
```

本次真实使用的 Skill 是：

```
test-driven-development
systematic-debugging
requesting-code-review
receiving-code-review
verification-before-completion
```

38.5　规则 4：完成前必须重新跑验证

完成汇报不能引用旧输出。

本次完成前验证是：

清单 38-3　规则 4：完成前必须重新跑验证

```
openspec validate today-plan-overdue-rollover --strict --json --no-interactive
npm test
npm run dev
openspec instructions apply --change today-plan-overdue-rollover --json
```

证据是：

```
OpenSpec strict validate: passed 1, failed 0
npm test: pass 7, fail 0
页面验证：90 分钟下 3 个今日执行、1 个顺延；60 分钟下 2 个今日执行、2 个顺延
OpenSpec apply: total 8, complete 8, remaining 0
```

沉淀规则：

```
任何“已完成”“已修复”“测试通过”之前，都必须有当前轮新鲜命令输出。
```

这条规则应该放进个人工作流，成为每次完成前的固定动作。

38.6　规则 5：规则只沉淀可迁移部分

不是所有东西都值得进规则库。

不沉淀：

```
today-plan 这个函数名。
某次临时标题。
某个临时目录路径。
某个截图摆放方式。
某次临时判断口径。
```

要沉淀：

示例 38-4 规则 5：规则只沉淀可迁移部分

```
什么需求需要 spec。
什么失败需要 systematic-debugging。
什么完成声明需要 verification-before-completion。
什么 review 意见属于当前 scope。
什么交付练习必须有真实业务需求。
什么交付结果需要用户可见展示。
```

规则库要服务下一次开发，不是记录一次临时复盘怎么组织。

38.7 放到哪里

项目事实放仓库：

```
README.md
AGENTS.md
openspec/specs/
```

需求过程放仓库：

```
openspec/changes/
dev-process/
```

跨项目流程放个人资产：

```
Skill
Superpowers 使用约定
个人 AI Coding 检查清单
```

可执行证据放测试或脚本：

```
test/
scripts/
verification ledger
```

规则不能只留在聊天记录里。

聊天记录太容易丢，也太难被下一次任务自动读取。

38.8 一次交付后的复盘模板

你可以用这 6 个问题复盘：

示例 38-5 一次交付后的复盘模板

```
1. 这次真实需求是什么，用户可见行为是什么？
2. 哪些边界一开始不清楚？
3. 哪个测试先红灯，最后哪个命令绿灯？
4. 哪些 review 风险和业务行为直接相关？
5. 哪条规则下次还能复用？
6. 这条规则应该放进项目事实、跨项目流程，还是可执行验证？
```

不要写成空泛总结。

比如这次可沉淀为：

```
有状态、日期、优先级和容量边界的需求，先落 OpenSpec spec，再写测试。
```

这句话能指导下一次任务。

38.9 迁移到自己的项目

假设你做的是工单系统，需求是：

```
SLA 快超时的工单要自动升级，但不能把所有低优工单都推给同一个值班人。
```

你可以直接复用这次规则：

示例 38-6 迁移到自己的项目

```
先写用户可见行为。
用 OpenSpec 写状态、时间、优先级和容量边界。
用 TDD 先写失败测试。
用 systematic-debugging 处理测试失败。
review 时检查状态过滤、时间边界、容量分配和 deferred/overflow。
完成前跑新鲜验证。
交付后沉淀“有 SLA/容量边界的需求必须先 spec”。
```

这就是规则沉淀的价值。

38.10 配套资料

- 规则沉淀记录：`learn-ai-coding/personal-productivity-app/dev-process/04-rule-asset-sedimentation.md`
- 端到端过程记录：`learn-ai-coding/personal-productivity-app/dev-process/03-end-to-end-today-plan-delivery.md`
- OpenSpec 过程记录：`learn-ai-coding/personal-productivity-app/dev-process/01-openspec-today-plan-rollover.md`

- Superpowers 过程记录：`learn-ai-coding/personal-productivity-app/dev-process/02-superpowers-today-plan-workflow.md`
- OpenSpec 变更包：`learn-ai-coding/personal-productivity-app/openspec/changes/today-plan-overdue-rollover/`
- 结果页：`learn-ai-coding/personal-productivity-app/index.html`

本章资源

〔38-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

PART 09

第9篇　建立个人工程系统

把规则、案例和验证习惯留下来，让下一次任务有可以复用的起点。工具升级时仍能保留自己的判断框架。

阅读重点

能区分临时记录与长期规则，并安排持续维护。

第39章　从个人提效到个人工程系统

AI Coding 用到最后，拼的不是谁会更多工具。

也不是谁收藏了更多 prompt。

拉开差距的是：你能不能把 AI 放进自己的工程系统里。

这个系统不复杂。

它由几件很朴素的东西组成：

示例 39-1　本章示例

```
知道什么任务适合 AI。
知道什么时候必须自己主导。
知道怎么给上下文。
知道怎么写规格和边界。
知道怎么限制 diff。
知道怎么用测试和手动验证定义完成。
知道怎么把成功经验沉淀下来。
```

如果这些能力没有建立，再强的模型也会让你在真实项目里反复返工。

如果这些能力建立起来，工具可以替换，工作流仍然保留。

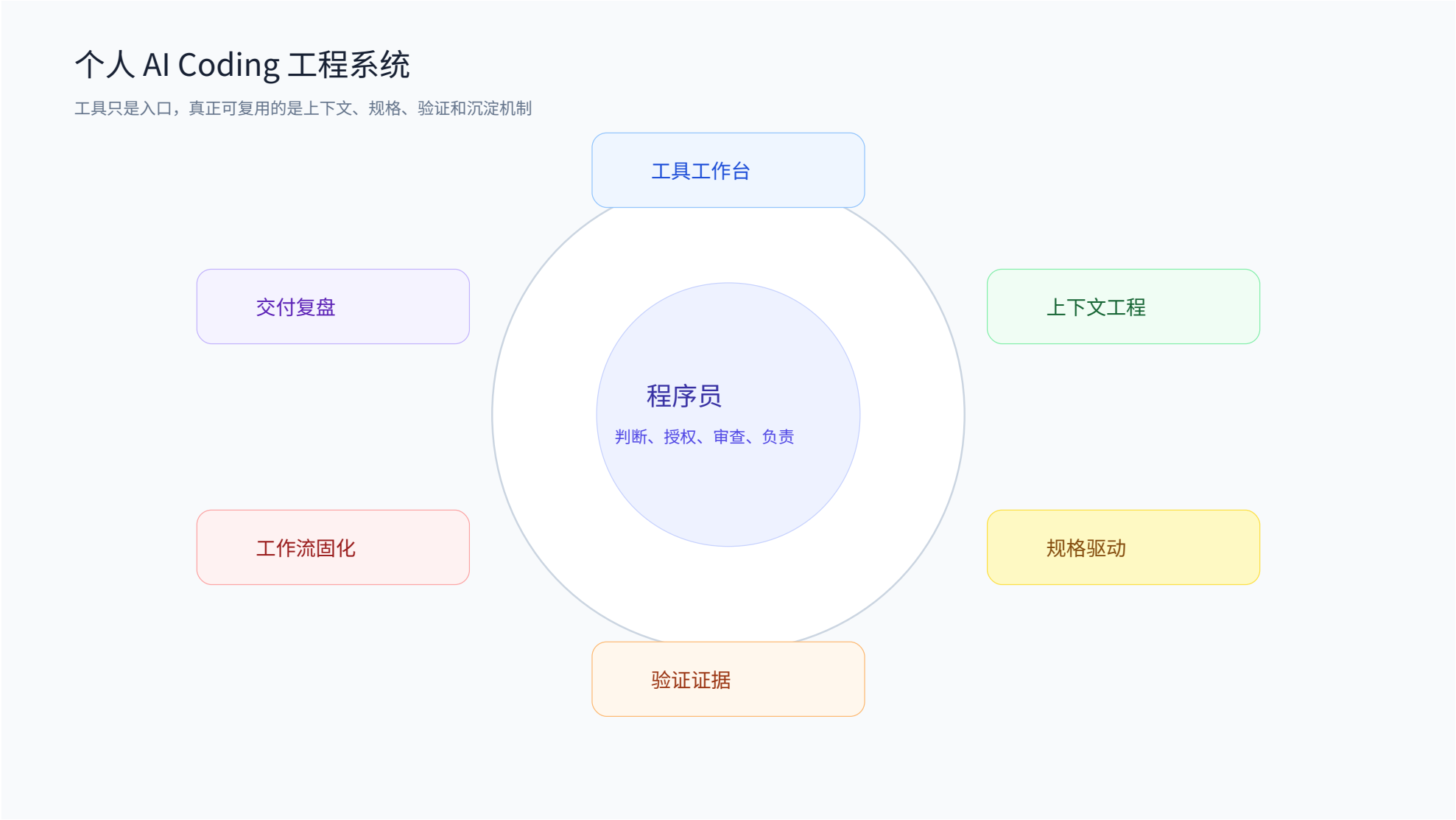


图 39-1　个人 AI Coding 工程系统

39.1 工具只是入口

Codex、Claude Code、Cursor、Copilot、MCP、Skill、OpenSpec、Superpowers，这些都重要。

但它们不是终点。

程序员最容易掉进一个坑：以为换一个工具，就能解决 AI Coding 不稳定的问题。

工具确实会影响效率，但不要把 App、CLI、IDE 当成互不重叠的能力盒子。

同一个 Agent，如果拿到了仓库、终端和编辑权限，在 CLI 里也能计划、解释、搜索、修改、测试和总结。App 的价值更多在结果预览、长任务管理、异步审查、文件查看、材料整理和协作反馈。

CLI 适合贴近终端和 repo 操作，也适合把一次改动从搜索、计划、实现到验证串起来。

IDE 适合局部补全和代码导航。

浏览器适合验证 UI 和查资料。

MCP 适合把必要外部资源以受控方式接进来。

Skill 适合固化重复工作流。

但如果任务边界不清、上下文混乱、验证缺失、diff 过大，换工具只能让错误发生得更快。

所以工具工作台只是第一层。

要建立的是任务到交付的整套路径。

这套路径有三条底线。

第一，AI 不能替你定义业务正确性。它可以读代码、找相似实现、补测试、提出候选方案，但订单能不能回退、权限能不能放宽、金额怎么取整、历史数据怎么兼容，必须由人确认。代码里已有的行为也不一定都是正确需求，可能只是历史遗留。

第二，AI 不能替你承担变更责任。它生成的 diff 进了仓库，责任在提交者，不在模型。你可以让 Agent 写完成报告，但不能把完成报告当成审查结论。真正的结论来自 diff、测试、日志、截图、接口响应和 reviewer 判断。

第三，AI 不能替你决定权限边界。让它读仓库、跑测试、打开浏览器、访问 Issue、查数据库、调用 MCP server，都是不同授权级别。能力越强，越要明确只读、可写、可执行、可联网和需要确认的动作。

个人工程系统存在的意义，就是把这三条底线落到日常动作里。

39.2 一次任务开始前

不要急着问 AI 怎么写。

先问四个问题：

```
这个任务适合交给 AI 到什么程度？
AI 需要读哪些文件才不会误判？
哪些行为不能被改？
完成后用什么证明？
```

小任务可以很轻：

```
目标：修 README 命令。
文件：README.md。
约束：不改 CLI 实现。
完成：命令能跑。
```

复杂任务要更正式：

```
requirements.md
design.md
tasks.md
openspec/changes/<change>/
verification ledger
```

这不是流程表演。

它是在给 AI 行动边界。

边界越清楚，diff 越容易审。

39.3 执行中要看 diff，不要看情绪

AI 很会总结。

它可以把一次不可靠的改动说得很完整。

所以执行中不要被总结带走。

你要看：

示例 39-2 执行中要看 diff，不要看情绪

```
改了哪些文件。
有没有越过任务边界。
是否引入新依赖。
```

```
是否改变外部行为。
测试是否先表达新行为。
失败是否基于证据排查。
README、规格、规则是否同步。
```

如果 diff 失控，不要继续让 AI “优化”。

先收住范围。

必要时回到任务上下文，重新授权一个更小的步骤。

程序员在 AI Coding 里的核心角色，不是给 AI 当审核员那么简单。

你要负责判断：

```
哪一步可以让 AI 做。
哪一步必须先停。
哪一步要补证据。
哪一步要人工决策。
```

39.4 完成后必须有证据

“完成”至少要有一类证据。

代码任务通常不止一类。

示例 39-3 完成后必须有证据

```
测试命令。
目标命令。
浏览器验证。
日志或数据库检查。
diff 审查。
README 命令复跑。
verification ledger。
```

比如 `context-pack` 任务，完成证据是：

```
npm test：12 pass，0 fail。
context-pack 输出 Read-Only Boundary、Task Context、Code Map、Verification Handoff。
unknown command stderr 包含 context-pack，exit code 非 0。
README 写了可运行命令。
OpenSpec 式变更包记录了只读边界。
```

这比“已实现 context-pack”可靠得多。

没有证据的完成，只是模型输出。

证据能被复查，交付才可持续。

证据也要和任务风险匹配。

文档改动，看 diff 和链接即可。CLI 行为改动，要跑目标命令、错误命令和测试。前端交互改动，要有测试或浏览器操作记录。接口合同改动，要看 status、response shape、兼容路径和调用方影响。状态机、权限、金额、数据迁移这类任务，要有反向用例和残余风险说明。

证据不足时，不要把话说满。可以写“已通过本地 focused test，未覆盖真实第三方回调”；可以写“已验证页面主路径，未跑浏览器兼容性”；可以写“当前失败是 baseline，但需要独立任务处理”。这种表达比“全部完成”更专业，也更适合进入 review。

AI Coding 的完成感经常来得太早。程序员要做的，是把完成感推迟到证据之后。


个人 AI Coding 运行清单
每次任务前、中、后只问少数关键问题
开始前
任务是否适合 AI
上下文是否足够小
规格和边界是否写清
验证命令是否明确
执行中
diff 是否小
AI 是否越界改文件
测试是否先约束行为
失败是否基于证据排查
完成后
命令是否跑过
证据是否可复查
风险是否写清
经验是否需要沉淀
这套清单越短越好。能长期执行，比一次性覆盖所有风险更重要。

图 39-2　运行清单

39.5 哪些东西要沉淀

每次交付后，不要把所有东西都写进规则库。

只沉淀会复用的。

项目事实放项目：

```
AGENTS.md
CLAUDE.md
README.md
openspec/specs/
```

任务边界放任务：

```
task-context
openspec/changes/
verification ledger
```

跨项目流程放 Skill：

```
systematic debugging
focused code review
CLI contract review
verification before completion
```

确定性检查放脚本或测试：

```
npm test
node --test
lint
check-ledger
browser verification
```

不要把它们混在一个超长 prompt 里。

超长 prompt 会过期，会互相冲突，也会让 AI 忽略真正重要的上下文。

39.6 这套系统的最小版本

如果你只想从明天开始改一个习惯，不需要一次建完所有资产。

最小版本是：

示例 39-4 这套系统的最小版本

```
一个项目规则文件。
一个任务上下文模板。
一个 verification ledger 模板。
一个调试 Skill。
一个 review Skill。
一个固定的测试命令习惯。
```

在每次任务里执行：

示例 39-5 这套系统的最小版本

```
先写任务上下文。
让 AI 先解释计划。
限制一次 diff。
先看 diff 再看总结。
跑验证命令。
写剩余风险。
把重复问题沉淀。
```

做到这些，已经能过滤掉大量不可靠的 AI Coding 输出。

后续再引入 OpenSpec、MCP、多 Agent、自动化脚本。

顺序不能反。

如果没有任务边界和验证习惯，多 Agent 只会并行制造更多不确定性。

如果没有只读工具边界，MCP 只会扩大权限风险。

如果没有可迁移流程，Skill 只是长 prompt。

39.7 常见误区

误区一：把 AI Coding 当更快的编码。

它不只是写代码。它会影响需求、设计、测试、文档和交付证据。

误区二：把 prompt 当核心资产。

prompt 容易丢、容易过期。更耐用的资产是规则、规格、测试、Skill 和复盘记录。

误区三：复杂任务不写规格。

复杂需求只靠聊天窗口，很容易把当前事实和目标行为混在一起。

误区四：测试绿了就交付。

测试是必要证据，不是唯一证据。CLI、UI、文档、数据副作用也要看。

误区五：为了高级工具增加复杂度。

OpenSpec、MCP、多 Agent 都有使用成本。任务不需要时，不要硬上。

39.8 从一个真实任务开始

可以从一个真实项目开始，不要从空项目开始。

选一个中等难度任务：

```
改一个 CLI 行为。
修一个接口边界 bug。
补一个表单校验。
改一个状态机分支。
给老模块补测试。
```

按这个顺序做：

示例 39-6 从一个真实任务开始

```
写任务上下文。
生成代码地图。
让 AI 先读入口、测试、规则。
必要时写 spec/design/tasks。
让 AI 小步改。
看 diff。
跑测试。
手动验证目标行为。
写 verification ledger。
复盘沉淀规则。
```

你不需要每次都把流程跑满。

但你要知道哪些步骤可以省，哪些不能省。

程序员和随手聊天式开发的区别，就在这些可复查的步骤里。

AI 可以参与越来越多的开发工作。

但最终负责的人仍然是程序员。

你负责判断、授权、审查和交付。

工具会变。

这套工程系统应该留下。

39.9 自测：你的系统到了哪一层

可以用下面这组问题检查自己的状态。

第一层，只会用工具：

```
我能让 AI 写代码。
我会在 IDE 或 CLI 里调用 AI。
我会让 AI 解释报错。
```

这一层能提高速度，但结果不稳定。

第二层，能控制上下文：

```
我知道让 AI 先读哪些入口文件。
我会写任务上下文。
我会把项目规则放进 AGENTS.md 或对应规则文件。
我知道什么时候要清理旧结论。
```

到这一层，AI 开始不那么乱改。

第三层，能控制需求和 diff：

```
我会把复杂需求拆成 requirements、design、tasks。
我知道什么时候用 OpenSpec 或轻量 SDD。
我会限制一次改动范围。
我会先看 diff，而不是先看总结。
```

到这一层，复杂任务才开始可审。

第四层，能定义完成：

```
我会让测试约束行为。
我会手动验证 CLI、页面或接口。
我会记录 baseline、命令和残余风险。
我不会接受没有证据的完成声明。
```

到这一层，AI Coding 才能进入真实交付。

第五层，能沉淀系统：

```
我会把反复出现的问题写成规则、Skill、脚本或测试。
我知道哪些经验只属于当前项目。
我会删除过期规则。
我能判断是否需要 MCP 或多 Agent。
```

到这一层，个人经验开始复利。

不要急着跳到第五层。

如果第二层还没建立，直接玩 MCP 和多 Agent，大概率只会增加不确定性。

39.10 30 天落地路线

可以用一个月建立最小系统。

第一周，只做上下文。

```
给一个真实项目写 AGENTS.md。
整理 README、测试命令、入口文件。
为一个任务写 task context。
让 AI 每次改前先复述目标、文件和约束。
```

不要急着写复杂工具。

先让 AI 读对项目。

第二周，做规格和小 diff。

```
挑一个中等复杂需求。
写 requirements.md、design.md、tasks.md。
每次只授权一个小任务。
每步看 diff。
越界就停。
```

如果需求会长期迭代，再考虑 OpenSpec。

第三周，做验证。

```
给一个行为变化补测试。
补一次失败路径验证。
写一份 verification ledger。
手动跑一次 README 命令或页面流程。
```

这一周的目标是改变“AI 说完成就算完成”的习惯。

第四周，做沉淀。

```
整理三条重复提醒。
把一条写进项目规则。
把一条写成 Skill 草稿。
把一条写成测试或脚本。
删掉一条不再适用的规则。
```

一个月后，你不一定有很多工具。

但你会有一套能继续长的工作流。

39.11 工具升级顺序

不要反过来。

更合适的顺序是：

```
任务上下文 -> 项目规则 -> 测试和 ledger -> SDD/OpenSpec -> Skill -> 只读工具 -> MCP -> 多 Agent
```

前面的东西没有做好，后面的东西会变重。

没有任务上下文，项目规则会被误用。

没有测试和 ledger，SDD 只会变成文档。

没有稳定流程，Skill 只是换了位置的 prompt。

没有只读工具边界，MCP 会扩大权限风险。

没有独立审查面，多 Agent 会制造冲突。

这不是说后面的工具不重要。

而是它们需要前面的工程纪律支撑。

39.12 常见问题

是不是每个需求都要写 spec?

不是。小改动可以只写任务上下文。影响外部行为、跨模块、状态机、权限、支付、库存、CLI/API 合同这类需求，才值得写完整规格。

是不是所有项目都要接 MCP?

不是。单 repo、本地命令、手动上下文足够时，CLI 或脚本更简单。MCP 适合多客户端复用、外部资源实时变化、权限需要集中治理的场景。

Skill 会不会让流程变重?

写坏会。Skill 应该只固化高频、高风险、跨项目的流程。改 typo、整理一段 Markdown，不需要完整 Skill 链。

多 Agent 能不能提高速度?

能，但只适合边界清楚的独立任务或独立审查视角。强依赖实现不要硬拆，否则合并成本会吃掉并行收益。

个人规则库维护成本高吗?

如果规则很多，会高。正确做法是少写、常删、尽量用测试和脚本替代自然语言规则。

39.13 回到工程事实

每次让 AI 参与开发，都把它放回工程事实里。

事实包括：

示例 39-7 回到工程事实

```
源码。
测试。
日志。
命令。
截图。
规格。
diff。
review。
ledger。
```

不要让 AI 只在自然语言里自洽。

自然语言很容易说通。

工程事实不容易骗人。

整套工作流都建立在这个判断上：AI 可以扩大你的执行能力，但不能替你承担工程责任。

责任还在你这里。

39.14 每个月做一次维护

个人工程系统不是写完就不动。

建议每个月做一次小维护，时间不用长，半小时够用。

先看规则文件：

```
AGENTS.md 里有没有过期路径?
测试命令是否变了?
项目结构是否变了?
有没有已经不再适用的限制?
有没有最近重复提醒 AI 的规则还没写进去?
```

再看 Skill：

```
哪个 Skill 最近真实触发过?
哪个 Skill 经常误触发?
哪个 Skill 太长，导致 Agent 读到一堆无关内容?
哪个 Skill 缺少明确输出证据?
```

然后看脚本和模板：

```
verification ledger 是否还适合当前项目?
task context 模板是否太重?
code map 是否漏掉新目录?
README 命令是否还能跑?
```

最后看一次交付记录：

```
最近一次返工是为什么?
是上下文错了，规格缺了，测试漏了，还是验证没跑?
有没有一个小规则或测试能防止它再发生?
```

维护的目标不是让资产越来越多。

恰好相反。

每次维护最好删掉一点东西。

删掉过期规则，删掉从未触发的流程，删掉只服务某次任务的上下文。

规则越少，AI 越容易遵守。

39.15 系统有没有变好，看这些信号

个人工程系统不是靠文件数量判断。

看这些信号更直接：

示例 39-8 系统有没有变好，看这些信号

```
同类任务是否少解释了。
AI 是否少改计划外文件了。
大 diff 是否减少了。
测试失败是否能先分类了。
完成报告里是否有命令和残余风险了。
reviewer 是否能从 PR 描述里看懂验证范围了。
返工时是否能找到当时的规格、diff 和 ledger。
```

如果这些信号没有变化，说明你只是多写了一些文档。

反过来，如果文档很少，但这些信号变好了，系统已经在工作。

可以给自己设一个很朴素的月度指标：

示例 39-9 系统有没有变好，看这些信号

```
本月有几次 AI 改动越界？
有几次测试失败被误判？
有几次因为上下文不清返工？
有几次完成报告缺验证证据？
有几条规则被证明有用？
有几条规则应该删除？
```

这些指标不需要做成复杂看板。每月复盘 10 分钟就够。

AI Coding 的个人系统，最终要减少两类成本：重复解释成本和返工成本。

如果系统增加了大量维护成本，却没有减少这两类成本，就该删减。

39.16 什么时候应该降级

不是所有任务都要动用完整系统。

下面这些情况应该降级：

```
任务只是文案、注释、README 小改。
改动只在一个明确文件内。
验证命令非常直接。
没有业务状态、权限、金额、数据和兼容性风险。
不需要长期沉淀。
```

这时可以只做：

```
说明目标。
限制文件。
改完看 diff。
跑一个命令。
```

不需要写 spec，不需要 OpenSpec，不需要 MCP，不需要多 Agent。

降级不是偷懒。

降级是让流程重量和任务风险匹配。

程序员用 AI Coding，不能只会加流程，也要会删流程。

39.17 换一个项目时怎么迁移

个人工程系统最怕写死在一个项目里。

换到新项目时，不要直接复制旧项目的 `AGENTS.md`、OpenSpec 目录和所有 Skill。先迁移方法，再迁移资产。

第一步，只迁移问题。

你需要重新问：

```
这个项目怎么安装和测试？
主要入口在哪里？
哪些目录不能直接改？
哪些命令能证明改动结果？
哪些业务路径风险最高？
```

这些问题通用，但答案必须来自新项目。

第二步，迁移最小规则。

先放五条：

```
不要覆盖用户改动。
不要改 generated 和 build output。
不要新增依赖，除非任务明确要求。
行为改动要跑 focused verification。
完成报告要写命令、结果和残余风险。
```

等真实任务跑过以后，再补项目特有规则。

第三步，迁移流程，不迁移结论。

调试 Skill、review Skill、verification ledger 可以迁移，因为它们是流程。旧项目的状态机规则、目录结构、测试命令不能直接迁移，因为它们是结论。

第四步，用一个中等风险任务试跑。

不要一上来处理最大需求。选一个能本地验证、涉及两三个文件、有明确测试或手动验证路径的任务。它能暴露规则是否加载、上下文是否准确、验证命令是否可用。

迁移成功的标志不是“所有文件都复制过去”，而是新项目里第一次 AI 改动能收住范围、跑出证据、留下可复查记录。

39.18 系统失效时怎么处理

个人工程系统也会失效。

常见失效原因有四类。

第一，规则过期。

测试命令、目录结构、生成路径、依赖管理方式变了，但规则文件没变。Agent 读到旧规则，就会稳定地做错事。

第二，流程过重。

简单任务也要求完整 spec、完整 ledger、多个 Skill，最后人和 Agent 都在维护流程，而不是解决问题。

第三，权限扩大。

一开始只是读文档，后来接了浏览器、Issue、数据库、内部服务，却没有重新定义只读边界、确认动作和日志记录。

第四，沉淀失控。

每次任务都往规则库里加东西，从不删除。规则越来越长，真正关键的限制被埋掉。

处理方式很直接：

示例 39-10 系统失效时怎么处理

```
先冻结新增资产。
找最近三次返工。
判断返工原因属于上下文、规格、测试、权限还是规则。
只修最常出现的一类。
删除明显过期和从未触发的规则。
用一次真实任务验证修复是否有效。
```

不要在系统失效时再加一套系统。

先删，再修，再验证。

39.19 个人系统和团队系统的边界

个人工程系统不是团队研发平台。

个人系统解决的是：你如何在自己的机器、自己的仓库、自己的任务里稳定使用 AI Coding。

它关注：

示例 39-11 个人系统和团队系统的边界

```
任务怎么判断。
上下文怎么给。
Agent 能改到什么程度。
diff 怎么收住。
验证证据怎么留下。
哪些经验值得沉淀。
```

团队系统要解决的是另一类问题：

示例 39-12 个人系统和团队系统的边界

```
统一权限。
统一审计。
统一 CI 和发布。
成本统计。
代码资产治理。
跨团队协同。
安全和合规。
```

这两者有关，但不能混在一起。

个人程序员应该先把自己的流程跑顺。否则一上来讨论团队平台、指标看板、权限中心，很容易把真正的开发问题藏起来。

反过来，个人系统也不能假装解决团队问题。你自己的 `AGENTS.md`、Skill、ledger，不能替代团队的安全审计、生产权限、CI 策略和发布流程。

一个健康的边界是：个人系统负责把每次 AI 参与开发的证据留清楚；团队系统负责让这些证据进入统一的审查、测试、合并和发布链路。

39.20 最后一次复盘应该看什么

如果你跟着这套方法做完一个真实需求，最后不要只看“有没有省时间”。

省时间当然重要，但它不是唯一结果。

更应该看：

示例 39-13 最后一次复盘应该看什么

```
AI 是否读到了正确入口?
计划是否提前暴露了风险?
diff 是否比以前小?
测试是否先锁住行为?
失败是否被分类，而不是被解释掉?
完成报告是否能支撑 review?
有没有一条经验值得沉淀?
有没有一条旧规则应该删除?
```

这些问题能告诉你系统有没有变强。

如果一次任务没有省时间，但让你发现了测试缺口、上下文缺口、规则冲突，也值得。个人工程系统不是靠一次任务回本，而是靠持续减少返工。

AI Coding 的长期收益，来自每次任务结束后多留下了一点可复用的工程资产。

39.21 最终应该留下什么

把前面的内容收成一个个人工作台，最后应该留下这些资产：

示例 39-14 最终应该留下什么

```
1. 项目规则文件：告诉 AI 当前 repo 的事实和约束。
2. 任务上下文模板：每次需求开始前写目标、文件、边界和完成定义。
3. 规格模板：复杂需求用 requirements、design、tasks 或 OpenSpec 管理。
4. 代码地图工具：帮助 AI 接手陌生模块。
5. 验证记录模板：记录命令、结果、证据和残余风险。
6. 调试 Skill：防止没复现就改。
7. Review Skill：防止评审变成总结。
8. 只读上下文工具：在需要时给 AI 稳定导航材料。
9. 复盘记录：把失败模式变成下一次任务输入。
```

这些东西不需要一次完成。

你可以从前三个开始：

```
项目规则。
任务上下文。
验证记录。
```

再补两个：

```
调试 Skill。
Review Skill。
```

最后再考虑：

```
OpenSpec。
MCP。
多 Agent。
自动化脚本。
```

工具链可以慢慢长。

底层习惯要先建立：

示例 39-15 最终应该留下什么

```
先写边界。
先选上下文。
先让测试约束行为。
先看 diff。
先跑验证。
最后再沉淀。
```

这一套动作比任何单个工具都更抗变化。

最后留下来的东西，一定要能被下一次任务直接使用。不能执行、不能验证、不能减少返工的材料，不要留给 Agent 读取。个人工程系统的价值不在收藏，而在下一次真实改动开始时，少一点猜测，多一点证据。

39.22 可复查资料

- 项目规则：`AGENTS.md`
- 配套工具：DavidDingXu/learn-ai-coding〔39-1〕 中的 `learn-ai-coding/ai-coding-workflow-kit/`
- OpenAI Codex Skills：https://developers.openai.com/codex/skills〔39-2〕
- OpenAI Codex MCP：https://developers.openai.com/codex/mcp〔39-3〕
- OpenAI Codex Subagents：https://developers.openai.com/codex/subagents〔39-4〕
- MCP 架构：https://modelcontextprotocol.io/docs/learn/architecture〔39-5〕

本章资源

〔39-1〕 DavidDingXu/learn-ai-coding
https://github.com/DavidDingXu/learn-ai-coding

〔39-2〕 https://developers.openai.com/codex/skills
https://developers.openai.com/codex/skills

〔39-3〕 https://developers.openai.com/codex/mcp
https://developers.openai.com/codex/mcp

〔39-4〕 https://developers.openai.com/codex/subagents
https://developers.openai.com/codex/subagents

〔39-5〕 https://modelcontextprotocol.io/docs/learn/architecture
https://modelcontextprotocol.io/docs/learn/architecture

结语　把 AI Coding 留在日常工程里

如果一路读到这里，最重要的结论其实很简单：

AI Coding 不应该停在“我会用某个工具写代码”。

它应该变成你日常开发里的一套工程习惯。

这套习惯不依赖某个固定工具，也不靠一段神奇 prompt。它依赖的是更基础的东西：

示例 结-1　本章示例

```
任务开始前，先定义边界。
动手之前，先让 Agent 读对上下文。
复杂需求，先把规格写清楚。
执行过程中，用小 diff 控制风险。
完成之前，用测试、页面、接口、日志和 diff 留证据。
任务结束后，把可复用经验沉淀下来。
```

工具会变，模型会变，入口也会变。

但只要你还在真实代码库里交付需求，这几件事不会过时。

这也是整套内容真正想留下来的东西。

不是某个工具菜单。

不是一串命令。

不是一份可以复制粘贴的 prompt。

而是一套程序员能长期复用的 AI Coding 操作方式。



图 结-1　从个人流程到团队化实战

结.1　不要把结束理解成学完

AI Coding 这件事，很容易让人产生一种错觉：

```
我把工具都试过了。
我看懂了 Agent、Skill、MCP、OpenSpec。
我知道怎么让 AI 改代码。
所以我学完了。
```

这还不够。

真正的变化发生在下一次真实需求里。

比如你接到一个很普通的任务：

```
列表页新增一个筛选条件。
接口多返回一个状态字段。
CLI 多支持一个参数。
审批流加一个异常分支。
老模块补一条失败路径测试。
```

以前你可能会直接问 AI：

```
帮我实现这个需求。
```

看起来很快。

但在真实项目里，这句话通常缺了太多东西：

```
它不知道现有入口在哪里。
它不知道哪些行为不能改。
它不知道测试命令是什么。
它不知道这次改动的风险边界。
它不知道完成后要拿什么证明。
```

所以读完整套内容之后，不是要把提示词写得更长，而是要把这句话换成一套更稳的动作：

示例 结-2　不要把结束理解成学完

```
先找入口。
先看相似实现。
先确认任务边界。
先列验证动作。
先让 AI 解释准备怎么改。
再授权一个小 diff。
```

这才是从“会用 AI”切到“能用 AI 交付”的分界线。

结.2　你要留下四类资产

读完以后，不建议马上去搭一个很重的平台。

也不建议把每篇文章都整理成复杂模板。

先留下四类最小资产。

第一类，是项目入口卡。

它不需要长，能帮 Agent 和你自己快速进入项目就够了：

示例 结-3　你要留下四类资产

```
项目做什么。
主要入口在哪。
核心测试命令是什么。
本地怎么启动。
哪些目录不能随便改。
哪些接口、状态、权限、金额、数据迁移属于高风险。
```

这些内容适合放在项目规则文件、README 或团队已有开发文档里。

不要每次都在聊天窗口重新解释。

第二类，是任务上下文包。

它服务单次需求，不必长期保存所有细节。

最小结构可以是：

示例 结-4　你要留下四类资产

```
目标：这次要改变什么用户可见行为。
非目标：这次明确不改什么。
相关文件：入口、调用方、测试、文档。
约束：兼容性、权限、数据、副作用。
验证：完成后要跑什么、看什么。
未知：哪些信息需要先确认，不能让 AI 猜。
```

这份上下文包的价值，不是写得漂亮。

它的价值是防止 AI 一边改一边补脑。

第三类，是验证记录。

这类记录可以很短，但必须具体：

示例 结-5　你要留下四类资产

```
命令是什么。
结果是什么。
失败是否是改前已有。
页面或接口看到了什么。
哪些路径还没覆盖。
剩余风险是什么。
```

真实交付里，最危险的不是 AI 写错代码。

最危险的是你不知道它错在哪里，还被一段完成总结说服了。

验证记录就是反制这个问题的。

第四类，是个人规则库。

不要把所有经验都写进去。

只写会反复发生、反复提醒、反复影响结果的东西。

例如：

```
接口合同改动必须看调用方。
CLI 行为改动必须覆盖 stdout、stderr、exit code。
前端交互改动必须有浏览器验证。
测试失败不能直接听 AI 说无关，要先看 baseline。
多 Agent 只拆独立任务，不拆强依赖实现。
```

这些规则以后可以继续升级成 Skill、脚本、测试或检查项。

但一开始，不要追求完整。

能在下一次任务里生效，比写成一套宏大的文档更重要。

结.3　把一套文章压成一条日常流程

这套内容看起来覆盖了很多主题：

示例 结-6　把一套文章压成一条日常流程

```
工具工作台。
上下文工程。
代码地图。
项目规则。
任务上下文包。
SDD 和 OpenSpec。
计划先行。
小步开发。
```

```
TDD。
系统化调试。
验证闭环。
Skill。
MCP。
多 Agent。
个人规则库。
```

但落到每天的开发，不需要每次都跑满。

你可以把它压缩成一条普通流程：

示例 结-7 把一套文章压成一条日常流程

```
判断任务风险。
准备上下文。
让 AI 侦察代码。
要求 AI 给计划。
按小步授权修改。
看 diff。
跑验证。
记录证据和剩余风险。
沉淀一条可复用经验。
```

低风险任务可以轻一点。

例如改一段 README、修一个 typo、补一条简单文案，只需要明确目标、看 diff、检查链接或命令。

中风险任务要多做几步。

例如改 CLI 行为、补接口字段、调整表单校验，就要看调用方、补测试、跑目标命令、同步文档。

高风险任务必须拉满。

例如权限、金额、库存、状态机、数据迁移、兼容接口、批量任务、生产副作用，就不能只靠聊天窗口推进。至少要有规格、设计边界、任务拆分、失败用例、回滚考虑和完成证据。

很多人用 AI Coding 不稳定，不是因为少了某个工具。

而是低风险任务和高风险任务用了同一套粗糙流程。

轻任务被搞复杂，重任务被处理得太轻。

程序员要做的判断，就在这里。

结.4 最容易带走的不是 prompt，而是停止条件

很多人会问：

有没有一段最终版 prompt？

没有。

至少不应该把它当成核心资产。

不同项目、不同任务、不同风险级别，prompt 一定要变。

真正更值得带走的是停止条件。

比如任务开始前的停止条件：

```
AI 没说清相关文件，不实现。
AI 没说清验证方式，不实现。
AI 把非目标也纳入计划，不实现。
任务里有业务歧义，先确认，不让 AI 猜。
```

执行中的停止条件：

```
diff 变大，先停。
新增依赖，先问为什么。
越过授权目录，先停。
测试失败原因不清楚，先复现和分类。
实现开始改公共抽象，先回到设计。
```

完成前的停止条件：

```
没有新鲜验证输出，不说完成。
只跑了单测但改了 UI，不说完成。
只看页面但改了接口合同，不说完成。
没有说明剩余风险，不说全部完成。
AI 只给总结，没有证据，不接受。
```

这些停止条件比 prompt 更硬。

prompt 是给 AI 的。

停止条件是给人的。

AI Coding 能不能稳定，很大程度取决于程序员能不能在这些节点停下来。

结.5　以后工具升级，按这个顺序

工具会继续变多。

你还会看到新的 Agent、新的 IDE 插件、新的模型、新的 MCP server、新的 Skill 形态、新的自动化平台。

不要一看到新东西就搬进工作流。

更稳的顺序是：

示例 结-8　以后工具升级，按这个顺序

```
先把任务上下文写清楚。
再把项目规则放到仓库里。
再把测试和验证记录做起来。
再用 SDD / OpenSpec 管复杂需求。
再用 Skill 固化高频流程。
再用只读工具降低上下文成本。
再考虑 MCP 接入外部资源。
最后才考虑多 Agent 和自动化。
```

这个顺序不是为了保守。

它是为了让每一步升级都能解决真实问题。

如果任务上下文还不清楚，MCP 接更多资料只会把噪音变多。

如果验证习惯还没有建立，多 Agent 只会并行产生更多难审的输出。

如果项目规则还在聊天窗口里来回复制，Skill 也只是换个地方存 prompt。

如果测试和手动验证不稳定，OpenSpec 也只能停留在文档层面。

工具升级应该服务已有流程，而不是替代工程判断。

结.6　适合从哪里开始

不要从空项目开始。

空项目太干净，反而看不出 AI Coding 的真实问题。

选一个你熟悉但不完美的项目。

最好有这些特征：

```
有真实业务逻辑。
有一些测试，但覆盖不完整。
有 README，但命令可能过期。
有旧模块，也有近期改动。
有一两个你一直想修的小问题。
```

然后选一个中等大小的任务。

不要太小，小到看不出流程价值。

也不要太大，大到一开始就要跨很多模块。

合适的任务像这样：

示例 结-9　适合从哪里开始

```
给一个 CLI 命令补错误提示。
给列表页新增一个状态筛选。
给接口补一个向后兼容字段。
把一段重复校验抽成内部函数。
给老模块补一条失败路径测试。
修一个能稳定复现的小 bug。
```

用下面这份最小任务卡开始：

示例 结-10　适合从哪里开始

```
目标：
这次要改变的用户可见行为是什么？

非目标：
哪些文件、行为、抽象这次不碰？

上下文：
让 AI 先读哪些入口、测试、文档和相似实现？

计划：
AI 准备改哪些文件，为什么？

授权：
第一步只允许改什么？

验证：
改完必须跑什么命令、看什么页面或接口？

沉淀：
这次如果踩坑，哪一条经验值得写回规则？
```

这张卡足够短。

短到你愿意在真实任务里用。

也足够硬。

硬到 AI 很难完全脱离工程事实乱改。

结.7　团队里也可以这么用

这套方法不是只能个人使用。

团队里更适合从轻量约定开始，而不是直接搭平台。

比如先约定三件事：

```
涉及外部行为变化的 AI 代码，PR 里必须写验证证据。
AI 参与的复杂需求，必须先有任务上下文或规格。
项目规则只能写长期有效内容，临时结论不能塞进规则文件。
```

再约定三类文件：

```
项目规则文件：长期有效的工程约束。
任务上下文：单次需求的目标、范围和验证。
验证记录：本次改动跑过什么、没覆盖什么。
```

这样做不会强迫所有人使用同一个 AI 工具。

有人用 Codex，有人用 Claude Code，有人用 Cursor，有人用 Copilot，都可以。

团队真正需要统一的，不是工具界面。

而是证据和边界：

```
为什么这么改。
改了哪些范围。
怎么证明没破坏关键行为。
还有哪些风险没覆盖。
```

这几件事统一了，工具可以各用各的。

结.8 如果你要把这套方法带进团队

个人 AI Coding 稳定以后，下一步自然会遇到团队问题。

个人可以靠经验记住很多边界。

团队不行。

一旦多人、多仓库、多应用、多工具一起使用 AI Coding，问题会变成另一组：

```
不同项目是不是在重复建设 Agent、Prompt、Skill 和规则?
需求、上下文、验证记录有没有统一承接方式?
AI 生成的产物怎么被团队 review、接管和恢复?
测试失败、baseline、剩余风险有没有团队共识?
个人沉淀的规则怎样变成团队可复用资产?
```

这些问题已经超出了个人工作流。

它们需要团队层面的共享运行时、SPEC 系统、需求拆分、有界上下文、交付协作、验证治理和资产沉淀。

如果你正在把 AI Coding 从个人实践带到团队，可以继续看《AI Coding 团队化实战》。

那套内容讨论的不是“团队成员怎么写更好的 prompt”，而是：

```
怎样把个人 AI Coding 经验，
变成团队可复制、可审查、可恢复、可沉淀的工程能力。
```

个人专栏解决的是：

```
一个程序员怎样稳定用 AI 完成真实代码库里的需求。
```

团队化实战继续解决的是：

```
一支团队怎样让这些经验不只停留在少数高手身上。
```

这两个问题前后衔接，但不要跳步。

如果个人任务边界、上下文、验证和规则沉淀还没跑顺，直接做团队平台，很容易把个人阶段的问题放大。

更合适的顺序是：

```
先让个人流程稳定。
再把高频规则抽出来。
再统一团队产物和证据。
再建设共享运行时和工具适配。
最后再考虑多 Agent、自动化和平台化治理。
```

这样团队化不是新起一套口号，而是把已经有效的工程习惯放大。

结.9 几个不要做的事

第一，不要把所有规则塞进一个超长 prompt。

规则一长，冲突、过期和被忽略都会变多。

长期规则放项目。

单次规则放任务。

流程规则放 Skill。

可执行规则放测试和脚本。

第二，不要为了显得专业而每次都上 OpenSpec。

小改动不需要完整规格。

但一旦涉及状态、权限、金额、兼容性、跨模块调用，就不要只靠一句话让 AI 猜。

第三，不要把 AI 的完成总结当成交付证据。

总结可以读，但不能替代命令输出、diff、截图、接口响应、测试结果和 review。

第四，不要把个人规则库写成垃圾桶。

规则越多，越没人看。

只留下能改变下一次行为的规则。

第五，不要把多 Agent 当成默认答案。

多 Agent 适合并行侦察、独立 review、独立实现小任务。

它不适合边界不清、强依赖、需要大量共享状态的任务。

结.10 常见问题

我是不是要把前面的文章都重新整理成模板？

不需要。

先抽一套能跑起来的最小流程。真实用三次以后，再决定哪些东西值得模板化。没被真实任务验证过的模板，很容易变成摆设。

我所在团队暂时不能使用外部 AI 工具怎么办？

仍然可以迁移工程方法。任务上下文、规格、测试、验证记录、规则沉淀，本来就是程序员应该做的事。AI 只是放大这些方法的收益，不是这些方法存在的前提。

每次都写验证记录会不会太重？

取决于任务风险。文案改动可以只记录 diff 和链接；接口、CLI、状态机、权限、数据类改动就应该记录命令和结果。验证记录不要求长，只要求可复查。

规则库越来越多怎么办？

删。

删掉过期规则、临时规则、重复规则、无法改变行为的规则。能用测试表达的，就不要长期写自然语言提醒。能用脚本检查的，就不要靠人记。

工具一直变，前面学的会不会失效？

具体入口会变，工程控制点不会变。读文件、选上下文、限制 diff、写测试、跑验证、审查风险、沉淀规则，这些动作仍然存在。工具升级以后，先看它能不能更好地服务这些控制点。

结.11　最后回到一句话

AI Coding 的长期价值，不是让你少写几行代码。

而是让你把开发过程里那些原本靠经验、靠记忆、靠临场判断的东西，逐步变成可复查、可复用、可迁移的工程资产。

一次任务开始前，多一份上下文。

一次改动执行中，多一次 diff 边界。

一次完成之前，多一条验证证据。

一次交付之后，多一条可复用规则。

这些东西看起来都不大。

但它们会累积。

累积到某一天，你会发现自己使用 AI 的方式已经变了：

示例 结-11　最后回到一句话

```
不再急着让 AI 写代码。
先让它理解项目。
不再接受漂亮总结。
先看证据。
不再把错误归因给模型不稳定。
先检查边界、上下文和验证。
不再追一个万能工具。
开始维护自己的工程系统。
```

这时，AI Coding 才真正进入日常工程。

它不再是一段演示。

也不再是一种新鲜感。

它变成你处理真实需求、真实代码、真实风险的一部分。

这比学会任何一个工具都重要。

术语索引

Agent

能够结合上下文和工具连续执行任务的系统。 参见第3章。

代码地图

可复查的代码入口、模块关系与验证位置导航。 参见第9章。

任务上下文包

为一个任务准备的范围、证据、相关代码与约束集合。 参见第12章。

SDD

以规格明确预期行为，再推进设计、实现和验证的开发方式。 参见第15章。

OpenSpec

把需求变更及其规格、设计、任务放入仓库管理的工具与流程。 参见第19章。

小 diff

把修改限制在可以有效审查和验证的范围内。 参见第23章。

TDD

先用失败测试表达预期，再实现并整理代码的反馈过程。 参见第24章。

验证基线

变更前的已知结果，用于判断失败是否由当前改动引入。 参见第27章。

Skill

具有触发条件、流程和产物要求的可复用能力说明。 参见第29章。

MCP

将外部能力以一致接口提供给 AI 应用的互操作协议。 参见第33章。

多 Agent

通过明确分工协作完成任务，需要考虑共享状态、依赖与整合成本。 参见第34章。

章节与配套资源

正文可独立阅读。需要动手验证时，按下表选择两个配套项目中的代码或开发案例。章节编号与目录一致，工程与案例入口需联网访问。

章节	配套入口
导读、00—07、22—27、结语	personal-productivity-app/README.md
08—14	ai-coding-workflow-kit/dev-process/01-module-onboarding.md
15—21	personal-productivity-app/dev-process/01-openspec-today-plan-rollover.md
28	ai-coding-workflow-kit/dev-process/02-cli-contract-change.md
29—35	personal-productivity-app/dev-process/02-superpowers-today-plan-workflow.md
36	ai-coding-workflow-kit/dev-process/05-readonly-tool-boundary.md
37	personal-productivity-app/dev-process/03-end-to-end-today-plan-delivery.md
38—39	personal-productivity-app/dev-process/04-rule-asset-sedimentation.md

勘误与交流

反馈时注明书名、电子版版本、章节和具体位置。代码问题请附运行环境与复现步骤。

https://github.com/DavidDingXu/learn-ai-coding/issues

微信公众号：好好用 AI